

常见问题（必看）

为什么要再单独弄一个面试突击版？

JavaGuide 已经有了在线阅读版本（地址：<https://javaguide.cn/>），阅读体验也很不错，为什么我还要再花这么多时间单独弄一个面试突击版呢？

1. 很多同学由于某些原因比较喜欢看 PDF 电子版或者有打印的需求，JavaGuide 原项目内容过多，不太适合整理成 PDF 版本；
2. 《JavaGuide 面试突击版》转为面试打造，内容相比于JavaGuide 原项目更精简。

如何获取最新版本？

你可以通过我的公众号获取到《JavaGuide 面试突击版》的最新版本。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

如何学习本项目？

不论是在线版本还是 PDF 版本都提供了非常详细的目录，建议可以从头到尾看一遍，如果基础不错的话也可以挑自己需要的章节查看。看的过程中自己要多思考，碰到不懂的地方，自己记得要勤搜索，需要记忆的地方也不要吝啬自己的脑子。

为什么会有少部分文章无法查看？

《JavaGuide 面试突击版》共有 40+篇文章，仅有 4 篇文章是我的[知识星球](#)专属，属于星球内部小册《[Java 面试指北](#)》中的文章，不影响整体阅读体验。

如何贡献？

大家阅读过程中如果遇到错误或者可以完善的地方，可以在 Github/Gitee 的 issue 区与我交流：

- Github：<https://github.com/Snailclimb/JavaGuide-Interview>
- Gitee：<https://gitee.com/SnailClimb/JavaGuide-Interview>

或者，你可以通过邮箱 koushuangbwcx@163.com 与我交流。

希望大家给我提反馈的时候可以按照如下格式：

问题：描述清楚哪一篇文章的描述存在问题。

改进：描述清楚如何去改进有问题的描述。

参考文档（可选）：相关的一些参考资料比如官方文档的描述、书籍中的描述。

为了提高准确性已经不必要的时间花费，希望大家尽量确保自己想法的准确性。

面试指北（配套教程）

《[Java 面试指北](#)》是我的[知识星球](#)的一个内部小册，和《JavaGuide 面试突击版》的内容互补。相比于开源版本来说，《Java 面试指北》添加了下面这些内容（不仅仅是这些内容）：

- 10+ 篇文章手把手教你如何准备面试。
- 更全面的八股文面试题（系统设计、常见框架、分布式、高并发）。
- 优质面经精选。
- 技术面试题自测。
- 练级攻略（有助于个人成长的经验）。

内容概览

介绍	02-24 18:03
▼ 面试准备篇	+ :
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java优质面试视频汇总	02-11 12:28
Java优质开源面试资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14

面试准备篇

在「面试准备篇」，我写了 10+ 篇文章手把手教你如何准备面试，涵盖项目经验、简历编写、源码学习、算法准备、面试资源等内容。

▼ 面试准备篇

[如何准备 Java 面试？](#)

[程序员简历到底该怎么写？有哪些注意的点？](#)

[为什么要学习源码？源码这块面试会怎么问呢？如何阅读源码？](#)

[为什么要准备算法面试？怎么高效刷 Leetcode？](#)

[没有项目经验怎么办？跟着视频做的项目会被面试官嫌弃不？](#)

[接触不到高并发场景咋办？如何获得高并发的经验？](#)

[什么项目算是有亮点的或者是面试官认为有价值的？](#)

[去外包对自己的简历有影响么？](#)

[如果面试官问“你有什么问题问我吗？”，该如何回答？](#)

[Java 优质实战项目汇总](#)

[Java 优质面试视频汇总](#)

[Java 优质开源面试资源汇总](#)

另外，考虑到很多小伙伴缺少项目经历，我还推荐了很多小众但优质的实战项目，有视频也有开源项目，有业务系统，也有各种含金量比较高的轮子类项目。



技术面试题篇

「技术面试题篇」的内容和 JavaGuide 开源版本互补，不仅仅包括最基本的 Java、常见框架等八股文，还包括系统设计、分布式、高并发等进阶内容。

技术面试题篇

系统设计

如何准备系统设计面试？

如何设计一个秒杀系统？

如何自己实现一个 RPC 框架？

如何设计一个排行榜？

如何设计微博 Feed 流/信息流系统？

如何设计一个短链系统？

如何设计一个站内消息系统？

如何解决大文件上传问题?

如何统计网站UV?

▼ Java

IO 模型

基本数据类型&包装类型

泛型&通配符

String

▼ 数据库

缓存基础

Redis 集群

Redis 高可用

▶ 常见框架

▼ 分布式

分布式事务

分布式配置中心

▼ 高并发

如何设计一个高可用系统?

从单体架构到微服务架构演进

池化技术

降级&熔断

集群架构有什么好处? 分布式与集群的区别是什么?

SQL优化

▶ 服务器

面经篇

古人云:“他山之石，可以攻玉”。善于学习借鉴别人的面试的成功经验或者失败的教训，可以让自己少走许多弯路。

「面经篇」 主要会分享一些高质量的 Java 后端面经，有校招的，也有社招的，有大厂的，也有中小厂的。

如果你是非科班的同学，也能在这些文章中找到对应的非科班的同学写的面经。

《Java面试指北》

保存并发布于: 2022-09-29 15:24:17

正文 16px B I S U A

☆ 分

介绍

面试准备篇

技术面试刷题篇

- 系统设计
 - Java
 - 数据库
 - 常见框架
 - 分布式
 - 高并发
 - 服务器
 - Devops
- 技术面试题自测
- 面经篇
 - 双非本科、0实习、0比赛项目经历。3个月上...
 - 从考研失败到收获到自己满意的Offer
 - 五面阿里，终获 offer!
 - 4年经验，面试Bigo挂在了第三轮
 - 2年经验，2021 阿里、头条、美团，滴滴...
 - 2021 华为(字节)腾讯(京东)网易(滴滴)面经...
 - 2021 虾皮、网易云、京东、阿里校招面...
 - 2022 滴滴、网易、Shopee、B站、携程...
 - 2022腾讯云Java工程师一面+被捞一面...
 - 2022 步步高 Java 后端 6 面面经
 - 2022 金蝶 Java 后三面面经 (已OC)
 - 2022美团、华为、字节 Offer 面经 (附参...
 - 2022 四年经验高级 Java 工程师面经

双非本科、0实习、0比赛项目经历。3个月上...

前段时间，小贾在星球向我询问 offer 选择的问题，我才知道小贾已经斩获两个还不错的 offer。

小贾和我一样都是双非本科，学历上面我们和大部分一样都没有任何优势。他的校招经历挺波折的，非常有参考价值。

于是，我就找到小贾让他写一篇文章分享一下自己校招的一些准备面试的经历以及经验。

就梦

Guide哥，您好~ 校招基本上结束了，备战了那么久，也看了很多面经，感谢Guide哥一直以来的优质输出，公众号和专栏的优质内容为找工作的应届生带来了很大的帮助，我有个offer选择问题，想听听过来人的建议和看法。

一百度，base北京，贴贴研发部，客户端开发岗位，年26W左右白菜。

一ThoughtWorks（成都），软开Java后端，非常荣幸能够有机会和Guide哥到同一家公司，尚未谈薪。

【纠结点】：客户端开发，网上看了好多类似的选择，都是建议选择Java后端。客观来讲觉得客户端的发展道路比较窄，天花板低，不如Java后台开发。我本人对这两个方向的兴趣差不多，感觉都是起跑线开始。目前学习的方向是后端开发

主要还是担心客户端开发的职业发展会不如Java后端，想向您这样深入一个领域学习。虽然都是Java，但是安卓好像前后端都在干，方向有很大差别。选了安卓日后转后端是否还得从头再来？希望听听您和大家的建议！非常感谢！

贾哥写的太用心了，整篇文章大概有1w+字，我将分为两次来发。觉得内容不错的話，大家记得点赞催更。

希望贾哥分享对小伙伴们有帮助！

大纲

01 关于我

02 确立目标

03 应聘的一段时间

04 复习基础知识

05 准备项目

06 完善简历

07 开始投递简历

08 我的第一场正式面试

09 被笔试虐打的日子

10 获得百度 Offer

11 下一站是未来

并且，知识星球还有专门分享面经和面试题的专题，里面会分享很多优质的面经和面试题。

全部

精华

只看星主

面经

面试题

工具

开源项目

好文分享

进阶攻略

文件

等你回答



Guide哥

3 分钟前

一位球友的 2022 京东、佳创视讯、源创科技跳槽Java面试经验

京东一面

1. 单例模式、模板方法

3. Redis 分布式锁

4. Redis 的缓存击穿

展开全部





r09er

2022/5/23

收进专栏

...

#面试# shipline一面（已挂）

5年Java，shipline主站高级Java岗，全程八股文，难顶。😓

1.多线程

1.1如何打破线程池先放队列再扩容的逻辑，有什么框架实现了

2.synchronized和ReentrantLock区别

2.1synchronized如何实现等待唤醒

2.2ReentrantLock用了Unsafe类的哪个方法

3.dubbo

展开全部

面试

面经



Indelibili*

2022/5/7

收进专栏

...

GO后端实习面经

1.先自我介绍一下吧

2.说一下进程和线程的定义，以及他们之间的关系

3.进程间的通讯方式有哪些

4.了解mysql的索引吗，索引有什么好处，有什么坏处

5.了解mysql的索引的底层结构是什么，为什么选这个，有什么好处？

6.如果mysql的访问量过大，该使用什么措施？

7.缓存有什么用？为什么要用缓存，有使用过缓存吗？

8.redis除了可以做缓存还能做什么东西？

9.说一下r...

GO后端实习面经

技术面试题自测篇

为了让小伙伴们自测以检查自己的掌握情况，我还推出了「技术面试题自测」系列。不过，目前只更新了 Java 和数据库的自测，正在持续更新中。

▼ 技术面试题自测	05-29 15:16
Java基础	05-26 20:58
Java 集合	02-11 16:50
Java并发	05-29 15:24
JVM	05-26 20:58
MySQL	05-29 15:18
Redis	06-08 21:13

练级攻略篇

「练级攻略篇」 这个系列主要内容一些有助于个人成长的经验分享。

▼ 练级攻略篇
如何快速上手一个新项目？
如何提高编程能力？
如何更高效地自学？
如何提高工作效率？
如何提升工作表现？
如何快速学习新技术？
如何提高个人硬实力？

每一篇内容都非常干货，不少球友看了之后表示收获满满。不过，最重要的还是知行合一。

星球其他资源

除了《Java 面试指北》之外，星球还有《Java 必读源码系列》（目前已经整理了 Dubbo 2.6.x 、Netty 4.x、SpringBoot2.1 的源码）、《从零开始写一个 RPC 框架》（已更新完）、《Kafka 常见面试题/知识点总结》等多个专属小册。



- 1、《Java面试指北》(配合 JavaGuide 食用即可, JavaGuide 地址: [🔗 主页 | JavaGuide](#)): [🔗 输入密码 · 语雀](#) (密码:)
- 2、《Java 必读源码系列》(目前已经整理了 Dubbo 2.6.x、Netty 4.x、SpringBoot2.1 的源码): [🔗 输入密码 · 语雀](#) (密码:)
- 3、《从零开始写一个RPC框架》: [🔗 输入密码 · 语雀](#) (密码:) RPC 框架 Github地址: [🔗 GitHub - Snailclimb/guide-rpc-framework: A custom ...](#)
- 4、《分布式、高并发、Devops知识扫盲》: 已经并入《Java面试指北》中。
- 5、《Kafka常见面试题/知识点总结》: [🔗 输入密码 · 语雀](#) (密码:)
- 6、《程序员副业赚钱之路》 [🔗 输入密码 · 语雀](#) (密码:)
- 7、《Guide的读书笔记与文章精选集》(经典书籍精读笔记分享) [🔗 输入密码 · 语雀](#) (密码:)

另外, 星球还会有读书活动、学习打卡、简历修改、免费提问、海量 Java 优质面试资源以及各种不定时的福利。



Guide哥
4 小时前

2022 年第 2 次读书活动来啦! 一个既能督促自己阅读好书以及记录笔记的好机会, 同时, 为了奖励用心读书的小伙伴, 读书活动结束之后还会有各种福利相送。

这次入选的书籍是:

- 1、技术书籍: 《程序员修炼之道》、《高性能MySQL第三版》(可选定章节阅读, 至少阅读 5 章)、《Redis开发与运维》、《鸟哥的Linux私房菜》(可选定章节阅读, 至少阅读 5 章)
- 2、非技术书籍: 《被讨厌的勇气》、《穷查理宝典》、《博弈论与生活》

历史读书活动概览:

- 1、第一次读书活动: [🔗 https://t.zsxq.com/aYBIqJE](https://t.zsxq.com/aYBIqJE), 入选书籍: 《Clean Code》和《The Clean Coder》。
- 2、第二次读书活动: [🔗 https://t.zsxq.com/RZZ7y7A](https://t.zsxq.com/RZZ7y7A), 入选书籍: 《Java编程的逻辑》
- 3、第三次读书活动: [🔗 https://t.zsxq.com/UVbyBiU](https://t.zsxq.com/UVbyBiU) 入选书籍: 《重学Java设计模式》、《凤凰架构》、《数据密集型应用系统设计》、《Java 并发实现原理: JDK 源码剖析》
- 4、第四次读书活动: [🔗 https://t.zsxq.com/ZvrNBQb](https://t.zsxq.com/ZvrNBQb), 入选书籍: 《深入理解分布式事务》、《MySQL 是怎样运行的》、《凤凰架构》。
- 5、第五次读书活动: [🔗 https://t.zsxq.com/iyb2zjQ](https://t.zsxq.com/iyb2zjQ), 入选书籍: 《Java 并发实现原理: JDK 源码剖析》、《深入理解Java虚拟机(第3版)》、《从 Paxos 到 Zookeeper》。



Redis开发与运维.pdf



程序员修炼之道.epub



高性能MySQL第三版_compressed.pdf



鸟哥的Linux私房菜.epub



穷查理宝典 by 查理·芒格 (z-lib.org).epub

被讨厌的勇气.epub

博弈论与生活 by [英]兰·费雪 [[英]兰·费雪] (z-lib.org).epub

写作业

作业榜

读书活动

查看详情 >

会飞的小🐱、小傅哥、JaneRoad、Kong、夜不滚烫、Zärtlichke*、lyc、沉默王二、国家级焦虑大王、Guide哥等22人觉得很赞

Guide哥：电子书都帮大家找好了😁这次不要下次一定了。

另外，建议阅读非技术书籍的小伙伴可以再找一本技术书籍看看（不需要都看完）然后记录一些笔记分享。

4 小时前

星球限时优惠

两年前，星球的定价是 **50/年**，这是星球的最低定价，我还附送了 33 元优惠券。扣除了星球手续费，发了各种福利之后，几乎就是纯粹做公益。

感兴趣的小伙伴可以看看我在 2020-01-03 发的头条：[做了一个很久没敢做的事情](#)，去考古一下。

JavaGuide&Java面试交流圈

星主：Guide哥



1.5万
成员数量

9800+
内容数量

1024
运营天数

最优质的 Java 面试交流星球，多次被知识星球官方推荐。门票即将上调到 199 元，IOS 用户请在公众号“JavaGuide”回复“知识星球”加入/续费。

...

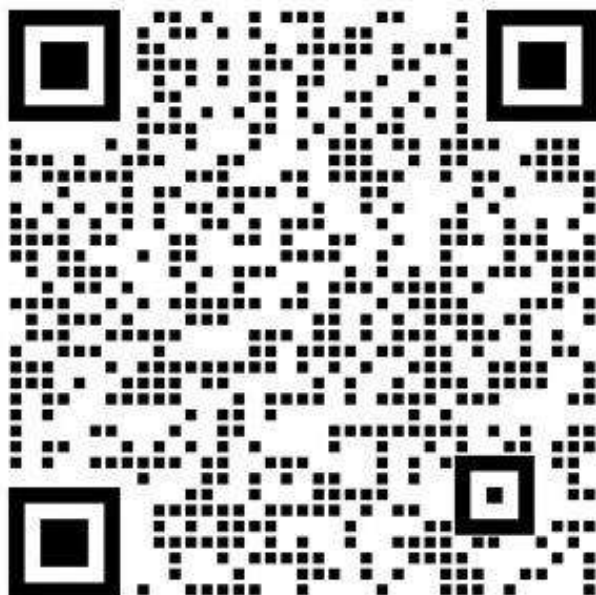
知识星球

微信扫码加入星球 ▶



随着时间推移，星球积累的干货资源越来越多，我花在星球上的时间也越来越多。于是，我将星球的定价慢慢调整为了 **159/年**！后续会将星球的价格调整为 **199/年**，想要加入的小伙伴一定要尽早。

你可以添加我的微信（没有手机号再申请微信，故使用企业微信。不过，请放心，这个号的消息也是我本人处理，平时最常看这个微信）领取星球专属优惠券(推荐👍)，限时 **130/年** 加入(续费半价)！



或者你也可以直接使用下面这张 **20** 元的优惠券，**139/年** 加入。



进入星球之后，你可以为自己制定一个目标，比如自己想要进入某某还不错的公司或者达成什么成就（一定要是还算有点挑战的目标）。待你完成目标在星球分享之后，我会将星球的门票费退还给你。

真诚欢迎准备面试的小伙伴加入星球一起交流！真心希望能够帮助到更多小伙伴！

更新记录

V1.0—2020-03-07

第一版《JavaGuide 面试突击版》正式完结发布！

V1.1—2020-03-13

修复问题：

- ✓ 每个章节都重复一遍目录，多滑了好多页
- ✓ 强烈要求加上版本号和发布日期，读者就知道自己的是什么版本了
- ✓ 2.1 Java 基础部分 p36+p37 文章链接失效
- ✓ 3.3 节 ThreadLocal 部分的一个笔误
- ✓ 水印过重，有一点影响阅读
- ✓ 文档名字开头加上版本表示示例：V1.1-JavaGuide 面试突击版

增加/修改内容：

- ✓ 一备战面试部分：完善了“自我介绍”部分的内容并且增加技术面可能会问哪些方向的问题、如何学习等内容。
- ✓ 第三节常见框架部分增加了 Kafka 常见面试题

V2.0—2020-04-02

修复问题：

- ✓ 修复了部分错别字,这部分对整体阅读影响不大所以不做过多阐述。
- ✓ 增加了页码

增加/修改内容：

- ✓ Java 基础知识部分自动拆装箱添加了一个参考文章。
- ✓ 提供了在线阅读版本：<https://snailclimb.gitee.io/javaguide-interview/#/>
- ✓ 计算机基础这一章节增加了：操作系统常见问题总结，这篇文章也更新在了公众号：[我和面试官之间关于操作系统的一场对弈！](#) 写了很久，希望对你有帮助！

V3.0—2020-06-16

- ✓ 修复多出部分读者提到了笔误
- ✓ 第九章- 真实大厂面试现场 增加了 我和阿里面试官的一次邂逅(下)（一篇花了 Guide 很多时间的文章，发在公众号上阅读不是蛮好，绝对干货~~~）
- ✓ 增加万众期待的 **Netty** 常见面试题总结
- ✓ 增加 Java 面试相关的开源项目

- ✓ 增加算法类面试相关的开源项目

V4.0—2020-10-16

修复问题：

- ✓ 修复部分文章参考阅读链接

增加/修改内容：

- ✓ 备战面试部分重构完善，细分成了 3 部分：

1. 校招/社招面试指南
2. 程序员简历之道
3. 大部分程序员在面试前很关心的一些问题

- ✓ Java 基础、集合、多线程、JVM 部分重构完善

- ✓ 数据结构部分重构完善

- ✓ 操作系统部分重构完善

- ✓ Redis 部分内容重构完善

- ✓ 增加了系统设计面试指北

- ✓ 增加了 18 道最常见的 Spring Boot 面试题。不过，这部分内容的答案更新在了[知识星球](#)。

- ✓ 优质面经部分增加了两篇读者面经：双非本科、0 实习、0 比赛/项目经历。3 个月上岸百度、华为|字节|腾讯|京东|网易|滴滴面经分享（6 个 offer）

V5.0—2022-8-25

全新版本，拒绝堆内容，持续完善精进！

不仅仅局限于下面这些工作：

- ✓ 重新绘制 100+ 图解

- ✓ 面试准备部分新增项目经验指南、面试常见词汇扫盲等内容。

- ✓ 根据当前 Java 面试实际情况，完善《JavaGuide 面试突击版》涉及到的所有知识点-----

1. 面试准备

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！



[Github](#) |

[Gitee](#)

面试准备部分属于补充内容，精选自《[Java 面试指北](#)》，少部分内容属于[我的知识星球](#)专属，还望理解。

1.1 程序员面试求职指南

本文节选自《[Java 面试指北](#)》的「面试准备篇」

前言

大家身边一定有很多编程比你厉害但是找的工作并没有你好的朋友！**技术面试不同于编程，编程厉害不代表技术面试就一定能过。**

现在你去面个试，不简单准备一下子，那简直就是往枪口上撞。我们大部分都只是普通人，没有发过顶级周刊或者获得过顶级大赛奖项。在这样一个技术面试氛围下，我们需要花费很多精力来准备面试，来提高自己的技术能力。“[面试造火箭，工作拧螺丝钉](#)”就是目前的一个常态，预计未来很久也还是会这样。

准备面试不等于耍小聪明或者死记硬背面试题。 **一定不要对面试抱有侥幸心理。打铁还需自身硬！**千万不要觉得自己看几篇面经，看几篇面试题解析就能通过面试了。一定要静下心来深入学习！

这篇我会从宏观面出发简单聊聊如何准备 Java 面试。

尽早以求职为导向来学习

我是比较建议还在学校的同学尽可能早一点以求职为导向来学习的。

这样更有针对性，并且可以大概率减少自己处在迷茫的时间，很大程度上还可以让自己少走很多弯路。

但是！不要把“以求职为导向学习”理解为“我就不用学课堂上那些计算机基础课程了”！

我在之前的很多次分享中都强调过：**一定要用心学习计算机基础知识！操作系统、计算机组成原理、计算机网络真的不是没有实际用处的学科！！**

你会发现大厂面试你会用到，以后工作之后你也会用到。我分别列举 2 个例子吧！

- **面试中：**像字节、腾讯这些大厂的技术面试以及几乎所有公司的笔试都会考操作系统相关的问题。
- **工作中：**在实际使用缓存的时候，你会发现在操作系统中可以找到很多缓存思想的影子。比如 CPU Cache 缓存的是内存数据用于解决 CPU 处理速度和内存不匹配的问题，内存缓存的是硬盘数据用于解决硬盘访问速度过慢的问题。再比如操作系统在页表方案基础之上引入了快表来加速虚拟地址到物理地址的转换。我们可以把快表理解为一种特殊的高速缓冲存储器（Cache）。

如何求职为导向学习呢？简答来说就是：根据招聘要求整理一份目标岗位的技能清单，然后按照技能清单去学习和提升。

1. 你首先搞清楚自己要找什么工作
2. 然后根据招聘岗位的要求梳理一份技能清单
3. 根据技能清单写好最终的简历
4. 最后再按照建立的要求去学习和提升。

这其实也是 **以终为始** 思想的运用。

何为以终为始？简单来说，以终为始就是我们可以站在结果来考虑问题，从结果出发，根据结果来确定自己要做的事情。

你会发现，其实几乎任何领域都可以用到 **以终为始** 的思想。

了解投递简历的黄金时间

面试之前，你肯定是先要搞清楚春招和秋招的具体时间的。

正所谓金三银四，金九银十，错过了这个时间，很多公司都没有 HC 了。

秋招一般 7 月份就开始了，大概一直持续到 9 月底。

春招一般 3 月份就开始了，大概一直持续到 4 月底。

很多公司（尤其大厂）到了 9 月中旬(秋招)/3 月中旬（春招），很可能就会没有 HC 了。面试的话一般都是至少是 3 轮起步，一些大厂比如阿里、字节可能会有 5 轮面试。面试失败话的不要紧，某一面表现差的话也不要紧，调整好心态。又不是单一选择对吧？你能投这么多企业呢！调整心态。今年面试的话，因为疫情原因，有些公司还是可能会还是集中在线上面试。然后，还是因为疫情的影响，可能会比往年更难找工作（对大厂影响较小）。

知道如何获取招聘信息

1.目标企业的官网/公众号：最及时最权威的获取秋招信息的途径。

2.牛客网：每年秋招/春招，都会有大批量的公司会到牛客网发布招聘信息，并且还会有大量的公司员工来到这里发内推的帖子。

3.超级简历

超级简历目前整合了各大企业的校园招聘入口，地址：<https://www.wondercv.com/jobs/>。

如果你是校招的话，点击“校招网申”就可以直接跳转到各大企业的校园招聘入口的整合页面了。

超级简历wondercv.com

简历模板简历修改热门职位求职攻略求职讨论

APP登录注册

全国Java开发工程师

行业领域:不限互联网金融房地产/建筑/物业IT/通信/硬件教育汽车广告/传媒咨询/财会/法律交通运输医疗/健康/制药更多

职位类型:不限全职实习兼职

学历要求:不限经验要求:不限公司规模:不限公司类型:不限薪资要求:不限

清空筛选

【21年春招】客户端开发工程师

19-25K北京 | 应届生 | 本科

客户端开发

陌陌

IT/互联网 | 民企 | 1000-9999人

【21年春招】推荐算法工程师

20-33K北京 | 应届生 | 本科

算法

陌陌

IT/互联网 | 民企 | 1000-9999人

【21年春招】自然语言处理工程师

20-33K北京 | 应届生 | 硕士

自然语言处理PythonJavaC++TensorFlowSparkHadoop

陌陌

IT/互联网 | 民企 | 1000-9999人

【21年春招】Java开发工程师

19-25K北京 | 应届生 | 本科

JavaSQLPythonShellMySQLRedis

陌陌

IT/互联网 | 民企 | 1000-9999人

【21年春招】Java开发工程师

19-25K北京 | 应届生 | 本科

JavaSQLShellSpringMybatis网络协议MySQLRedis

陌陌

IT/互联网 | 民企 | 1000-9999人

【21年春招】语音工程师

20-33K北京 | 应届生 | 本科

陌陌

IT/互联网 | 民企 | 1000-9999人

扫码下载App

随时随地和HR沟通

4.认识的朋友

如果你有认识的朋友在目标企业工作的话，你也可以找他们了解秋招信息，并且可以让他们帮你内推。

5.宣讲会现场

Guide 当时也参加了几场宣讲会。不过，我是在荆州上学，那边没什么比较好的学校，一般没有公司去开宣讲会。所以，我当时是直接跑到武汉来了，参加了武汉理工大学以及华中科技大学的几场宣讲会。总体感觉还是很不错的！

6.其他

校园就业信息网、学校论坛、班级 or 年级 QQ 群、各大招聘网站比如拉勾.....

多花点时间完善简历

一定一定一定要重视简历啊！朋友们！至少要花 2~3 时间来专门完善自己的简历。

最近看了很多份简历，满意的很少，我简单拿出一份来说分析一下（欢迎在评论区补充）。

1.个人介绍没太多实用的信息。



技术博客、Github 以及在校获奖经历的话，能写就尽量写在这里。你可以参考下面📌的模板进行修改：



2.项目经历过于简单，完全没有质量可言

项目经历

智能家居小程序 2020.03 – 2020.07

组员

- 协助智能家居小程序的系统建设、架构设计
- 协助小程序后台的模块开发和优化

基于 SSM 的众筹商城 2019.06 – 2019.09

组员

- 参与商城系统用户模块的功能开发和优化

每一个项目经历真的就一两句话可以描述了么？还是自己不想写？还是说不是自己做的，不敢多写。

如果有项目的话，技术面试第一步，面试官一般都是让你自己介绍一下你的项目。你可以从下面几个方向来考虑：

1. 对项目整体设计的一个感受（面试官可能会让你画系统的架构图）
2. 在这个项目中你负责了什么、做了什么、担任了什么角色
3. 从这个项目中你学会了那些东西，使用到了那些技术，学会了那些新技术的使用
4. 你是如何协调项目组成员协同开发的或者在遇到某一个棘手的问题的时候你是如何解决的又或者说你在这个项目用了什么技术实现了什么功能比如：优化了数据库的设计减少了冗余字段、用 redis 做缓存提高了访问速度、使用消息队列削峰和降流、进行了服务拆分并集成了 dubbo 和 nacos 等等。

3.计算机二级这个证书对于计算机专业完全不用写了，没有含金量的。

技能及证书

- **证书/执照：** 计算机二级（Office）、英语（CET-4）

4.技能介绍问题太大。

Java开发

- 具有扎实的Java基础，对面向对象编程有深刻的理解，熟练掌握java io流、集合、多线程、泛型、网络编程等基础开发技术；
- 了解并使用过Java常用框架，如spring boot、spring mvc、mybatis等；
- 了解TCP/IP协议，HTTP协议等；
- 熟悉GIT工具进行协同开发；

python

- 熟悉python数据处理，能对数据进行清理，导入，并进行分析处理，熟悉python pandas和numpy数据处理包；
- 了解并使用过flask框架，设计实现基本的RESTful API；
- 有责任心，学习能力强，积极进取，工作严谨。

前端

- 熟练掌握 HTML5、CSS3、JavaScript 等前端基本知识；
- 了解并使用过NodeJS，并开发过简单的uni-app应用；

- 技术名词最好规范大小写比较好，比如 java->Java，spring boot -> Spring Boot。这个虽然有些面试官不会介意，但是很多面试官都会在意这个细节的。
- 技能介绍太杂，没有亮点。不需要全才，某个领域做得好就行了！
- 对 Java 后台开发的部分技能比如 Spring Boot 的熟悉度仅仅为了解，无法满足企业的要求。

相关阅读：[程序员简历到底该怎么写？有哪些注意的点？](#)

提前准备技术面试和手撕算法

面试之前一定要提前准备一下常见的面试题：

- 自己面试中可能涉及哪些知识点、那些知识点是重点。
- 面试中哪些问题会被经常问到、面试中自己该如何回答。(强烈不推荐死记硬背，第一：通过背这种方式你能记住多少？能记住多久？第二：背题的方式的学习很难坚持下去！)

这块内容只会介绍面试大概会涉及到哪方面的知识点，具体这些知识点涵盖哪些问题，后面的文章有介绍到。

Java :

- Java 基础
- Java 集合
- Java 并发
- JVM

计算机基础 :

- 算法
- 数据结构
- 计算机网络
- 操作系统

数据库 :

- MySQL
- Redis

常用框架 :

- Spring
- SpringBoot
- MyBatis
- Netty
- Zookeeper
- Dubbo

分布式 :

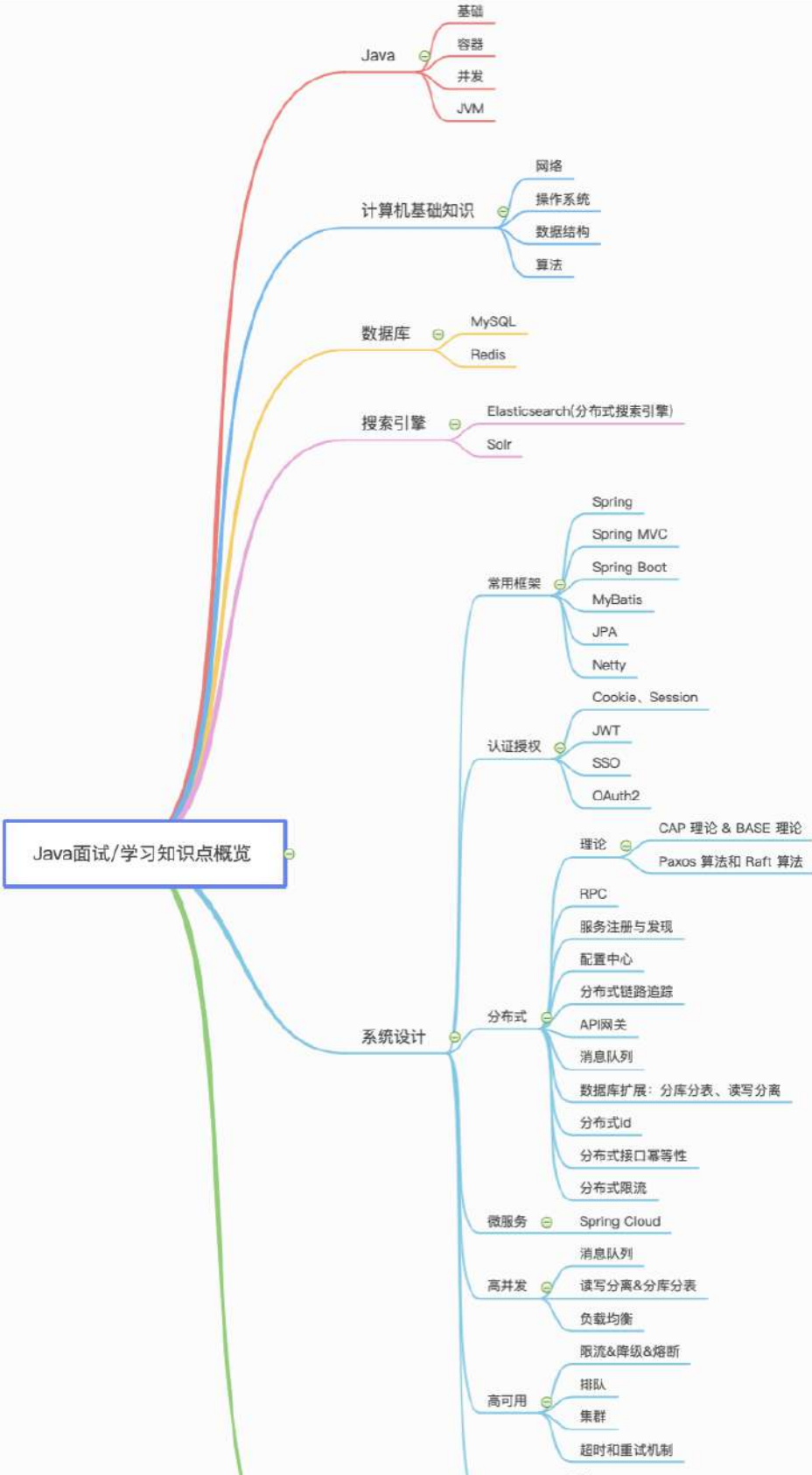
- CAP 理论 和 BASE 理论、Paxos 算法和 Raft 算法
- RPC
- 分布式事务
- 分布式 ID

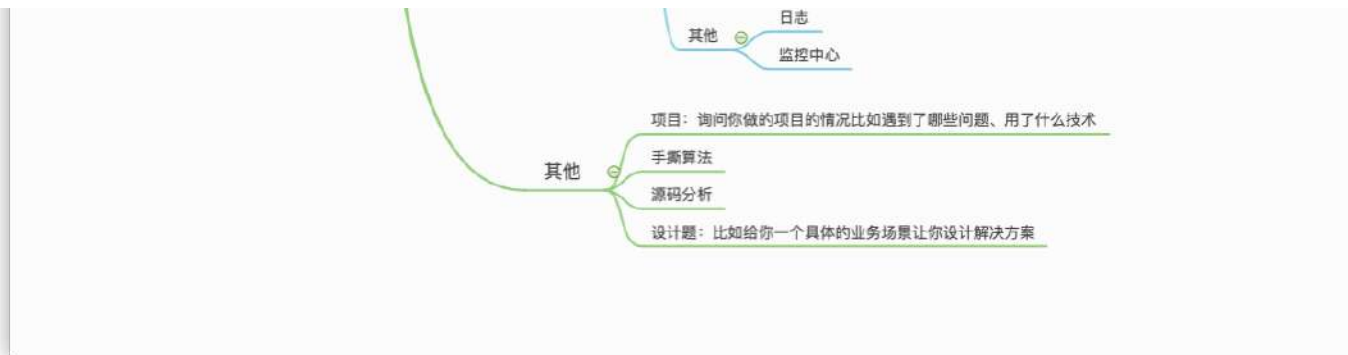
高并发 :

- 消息队列
- 读写分离&分库分表
- 负载均衡

高可用：

- 限流
- 降级
- 熔断





不同类型的公司对于技能的要求侧重点是不同的比如腾讯、字节可能更重视计算机基础比如网络、操作系统这方面的内容。阿里、美团这种可能更重视你的项目经历、实战能力。

关于如何准备算法面试请看《Java 面试指北》的「面试准备篇」中对应的文章。

提前准备自我介绍

自我介绍一般是你和面试官的第一次面对面正式交流，换位思考一下，假如你是面试官的话，你想听到被你面试的人如何介绍自己呢？一定不是客套地说说自己喜欢编程、平时花了很多时间来学习、自己的兴趣爱好是打球吧？

我觉得一个好的自我介绍应该包含这几要素：

1.



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“PDF”领取原创面试 PDF 手册
- 2、关注公众号回复“加群”进入面试交流群
- 3、关注公众号回复“八股文”获取大厂面试真题+面经

- 2. 把重点放在自己在行的地方以及自己的优势之处；
- 3. 重点突出自己的能力比如自己的定位的 bug 的能力特别厉害；

从社招和校招两个角度来举例子吧！我下面的两个例子仅供参考，自我介绍并不需要死记硬背，记住要说的要点，面试的时候根据公司的情况临场发挥也是没问题的。另外，网上一般建议的是准备好两份自我介绍：一份对 hr 说的，主要讲能突出自己的经历，会的编程技术一语带过；另一份对技术面试官说的，主要讲自己会的技术细节和项目经验。

社招：

面试官，您好！我叫独秀儿。我目前有 1 年半的工作经验，熟练使用 Spring、MyBatis 等框架、了解 Java 底层原理比如 JVM 调优并且有着丰富的分布式开发经验。离开上一家公司是因为我想在技术上得到更多的锻炼。在上一个公司我参与了一个分布式电子交易系统的开发，负责搭建了整个项目的基础架构并且通过分库分表解决了原始数据库以及一些相关表过于庞大的问题，目前这个网站最高支持 10 万人同时访问。工作之余，我利用自己的业余时间写了一个简单的 RPC 框架，这个框架用到了 Netty 进行网络通信，目前我已经将这个项目开源，在 Github 上收获了 2k 的 Star！说到业余爱好的话，我比较喜欢通过博客整理分享自己所学知识，现在已经是多个博客平台的认证作者。生活中我是一个比较积极乐观的人，一般会通过运动打球的方式来放松。我一直都非常想加入贵公司，我觉得贵公司的文化和技术氛围我都非常喜欢，期待能与你共事！

校招：

面试官，您好！我叫秀儿。大学时间我主要利用课外时间学习了 Java 以及 Spring、MyBatis 等框架。在校期间参与过一个考试系统的开发，这个系统的主要用了 Spring、MyBatis 和 shiro 这三种框架。我在其中主要担任后端开发，主要负责了权限管理功能模块的搭建。另外，我在大学的时候参加过一次软件编程大赛，我和我的团队做的在线订餐系统成功获得了第二名的成绩。我还利用自己的业余时间写了一个简单的 RPC 框架，这个框架用到了 Netty 进行网络通信，目前我已经将这个项目开源，在 Github 上收获了 2k 的 Star！说到业余爱好的话，我比较喜欢通过博客整理分享自己所学知识，现在已经是多个博客平台的认证作者。生活中我是一个比较积极乐观的人，一般会通过运动打球的方式来放松。我一直都非常想加入贵公司，我觉得贵公司的文化和技术氛围我都非常喜欢，期待能与你共事！

减少抱怨

就像现在的技术面试一样，大家都说内卷了，抱怨现在的面试真特么难。然而，单纯抱怨有用么？你对其他求职者说：“大家都不要刷 Leetcode 了啊！都不要再准备高并发、高可用的面试题了啊！现在都这么卷了！”

会有人听你的么？你不准备面试，但是其他人会准备面试啊！那你是不是傻啊？还是真的厉害到不需要准备面试呢？

因此，准备 Java 面试的第一步，我们一定要尽量减少抱怨。抱怨的声音多了之后，会十分影响自己，会让自己变得十分焦虑。

面试之后及时复盘

如果失败，不要灰心；如果通过，切勿狂喜。面试和工作实际上是两回事，可能很多面试未通过的人，工作能力比你强的多，反之亦然。

面试就像是一场全新的征程，失败和胜利都是平常之事。所以，劝各位不要因为面试失败而灰心、丧失斗志。也不要因为面试通过而沾沾自喜，等待你的将是更美好的未来，继续加油！

总结

这篇文章内容有点多，如果这篇文章只能让你记住 4 句话，那请记住下面这 4 句：

1. 一定要提前准备面试！技术面试不同于编程，编程厉害不代表技术面试就一定能过。
2. 一定不要对面试抱有侥幸心理。打铁还需自身硬！千万不要觉得自己看几篇面经，看几篇面试题解析就能通过面试了。一定要静下心来深入学习！
3. 建议大学生尽可能早一点以求职为导向来学习的。这样更有针对性，并且可以大概率减少自己处在迷茫的时间，很大程度上还可以让自己少走很多弯路。但是，不要把“以求职为导向学习”理解为“我就不用学课堂上那些计算机基础课程了”！
4. 手撕算法是当下技术面试的标配，尽早准备！

本文节选自 [《Java 面试指北》](#) 的「面试准备篇」。

介绍	02-24 18:03
▼ 面试准备篇	+ :
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java 优质面试视频汇总	02-11 12:28
Java 优质开源面试题资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“PDF”领取原创面试 PDF 手册
- 2、关注公众号回复“加群”进入面试交流群
- 3、关注公众号回复“八股文”获取大厂面试真题+面经

1.2 程序员简历就该这样写

本文节选自 《Java 面试指北》 的「面试准备篇」

前言

一份好的简历可以在整个申请面试以及面试过程中起到非常重要的作用。

为什么说简历很重要呢？我们可以从下面几点来说：

1.简历就像是我们的一个门面一样，它在很大程度上决定了是否能够获得面试机会。

- 假如你是网申，你的简历必然会经过 HR 的筛选，一张简历 HR 可能也就花费 10 秒钟看一下，然后 HR 就会决定你这一关是 Fail 还是 Pass。
- 假如你是内推，如果你的简历没有什么优势的话，就算是内推你的人再用心，也无能为力。

另外，就算你通过了第一轮的筛选获得面试机会，后面的面试中，面试官也会根据你的简历来判断你究竟是否值得他花费很多时间去面试。

2.简历上的内容很大程度上决定了面试官提问的侧重点。

- 一般情况下你的简历上注明你会的东西才会被问到（Java、数据结构、网络、算法这些基础是每个人必问的），比如写了你熟练使用 Redis,那面试官就很大概率会问你 redis 的一些问题。再比如你写了你在项目中使用了消息队列，那面试官大概率问很多消息队列相关的问题。
- 技能熟练度在很大程度上也决定了面试官提问的深度。

在不夸大自己能力的情况下，写出一份好的简历也是一项很棒的能力。

简历模板

简历的样式真的非常非常重要！！！如果你的简历样式丑到没朋友的话，面试官真的没有看下去的欲望。一天处理上百份的简历的痛苦，你不懂！

我这里的话，推荐大家使用 Markdown 语法写简历，然后再将 Markdown 格式转换为 PDF 格式后进行简历投递。

如果你对 Markdown 语法不太了解的话，可以花半个小时简单看一下 Markdown 语法说明：<http://www.markdown.cn/>。

下面是我收集的一些还不错的简历模板：

- 木及简历（推荐👍）：<https://resume.mdedit.online>。
- typora+markdown+css 自定义简历模板（推荐👍）：<https://github.com/Snailclimb/typora-markdown-resume>
- 极简简历：<https://www.polebrief.com/index>
- Markdown 简历排版工具：<https://resume.mdnice.com/>
- 超级简历：<https://www.wondercv.com/>

上面这些简历模板大多是只有 1 页内容，很难展现足够的信息量。如果你不是顶级大牛（比如 ACM 大赛获奖）的话，我建议还是尽可能多写一点可以突出你自己能力的内容（2~3 页皆可，记得精炼语言，不要过多废话）。

再总结几点简历排版的注意事项：

1. 尽量简洁，不要太花里胡哨。
2. 一些技术名词不要弄错了大小写比如 MySQL 不要写成 mysql，Java 不要写成 java。
3. 中文和数字英文之间加上空格的话看起来会舒服一点。

简历内容

个人信息

- 最基本的：姓名（身份证上的那个）、年龄、电话、籍贯、联系方式、邮箱地址
- 潜在加分项：Github地址、博客地址（如果技术博客和Github上没有什么内容的话，就不要写了）

示例：

个人信息

- 张秀儿/男/1996/湖北/英语6级
- 手机：11164201041，邮箱：guidege666@163.com
- 技术博客：<https://snailclimb.gitee.io/javaguide/#/>（没有东西的话就不要放上来）
- Github：<https://github.com/Snailclimb>（没有东西的话就不要放上来）

求职意向

你想要应聘什么岗位，希望在什么城市。另外，你也可以将求职意向放到个人信息这块写。

示例：

求职意向

- 期望职位：Java后端开发
- 期望城市：上海/苏州/杭州

教育经历

教育经历也不可或缺。通过教育经历的介绍，你要确保能让面试官就可以知道你的学历、专业、毕业学校以及毕业的日期。

示例：

北京理工大学	硕士，软件工程	2019.09 - 2022.01
湖南大学	学士，应用化学	2015.09 ~ 2019.06

专业技能

先问一下你自己会什么，然后看看你意向的公司需要什么。一般 HR 可能并不太懂技术，所以他在筛选简历的时候可能就盯着你专业技能的关键词来看。对于公司有要求而你不会的技能，你可以花几天时间学习一下，然后在简历上可以写上自己了解这个技能。

下面这个专业技能介绍，你可以根据自己的实际情况参考一下。

技能清单

- **计算机基础**：熟练掌握计算机网络、数据结构和算法、操作系统，了解计算机组成原理
- **Linux**：熟练使用 Linux，有 Linux 下开发的实际经验
- **Java**：熟练掌握 Java 基础知识、Java 并发、JVM，有过 JVM 排查问题和调优的经历
- **数据库**：熟练掌握 MySQL 数据库以及常见优化手段（比如索引、SQL 优化、读写分离&分库分表），Redis 使用经验丰富，熟悉 MongoDB
- **搜索引擎**：熟练掌握 Elasticsearch 的使用及原理，熟悉 Solr 的使用
- **框架**：熟练掌握 Spring、Spring MVC、SpringBoot、MyBatis、JPA、Spring Security 等主流开发框架
- **分布式**：
 - 熟练掌握分布式下的常见理论 CAP、BASE。熟悉 Paxos 算法和 Raft 算法
 - 熟练掌握 RPC（Dubbo）、分布式事务（Seata、2PC、3PC、TCC）、配置中心（Apollo）、分布式链路追踪（SkyWalking）、分布式 id（UUID、Snowflake）的使用及原理
 - 熟悉 Spring Cloud 全家桶常见组件的使用
- **高并发&高可用**：熟练掌握消息队列 Kafka 的使用及原理、有限流、降级、熔断的实战经验、
- **工具**：熟练掌握 Git、Maven、Docker
- **前端**：熟悉 TypeScript，有 React、Vue 的实际开发经验

我这里再单独放一个我看过的某位同学的技能介绍，我们来看看问题。

Java开发

- 具有扎实的Java基础，对面向对象编程有深刻的理解，熟练掌握java io流、集合、多线程、泛型、网络编程等基础开发技术；
- 了解并使用过Java常用框架，如spring boot、spring mvc、mybatis等；
- 了解TCP/IP协议，HTTP协议等；
- 熟悉GIT工具进行协同开发；

python

- 熟悉python数据处理，能对数据进行清理，导入，并进行分析处理，熟悉python pandas和numpy数据处理包；
- 了解并使用过flask框架，设计实现基本的RESTful API；
- 有责任心，学习能力强，积极进取，工作严谨。

前端

- 熟练掌握 HTML5、CSS3、JavaScript 等前端基本知识；
- 了解并使用过NodeJS，并开发过简单的uni-app应用；

上图中的技能介绍存在的问题：

- 技术名词最好规范大小写比较好，比如 java->Java，spring boot -> Spring Boot。这个虽然有些面试官不会介意，但是很多面试官都会在意这个细节的。
- 技能介绍太杂，没有亮点。不需要全才，某个领域做得好就行了！
- 对 Java 后台开发的部分技能比如 Spring Boot 的熟悉度仅仅为了解，无法满足企业的要求。

实习经历/工作经历

工作经历针对社招，实际经历针对校招。

工作经历建议采用时间倒序的方式来介绍，实习经历建议将最有价值的放在最前面。

示例：

XXX 公司（201X 年 X 月 ~ 201X 年 X 月）

- 职位：Java 后端开发工程师
- 工作内容：主要负责 XXX

项目经历

简历上有一两个项目经历很正常，但是真正能把项目经历很好的展示给面试官的非常少。

很多求职者的项目经历介绍都会面临过于啰嗦、过于简单、没突出亮点等问题。

项目经历应该突出自己做了什么，简单概括项目基本情况。项目经历取得的成果尽量要量化一下，多挖掘一些亮点比如自己是如何解决项目中存在的一个痛点的。除了解决痛点，还能如何挖掘亮点呢？从你项目涉及到的技术上来挖掘，想想这些技术能为项目带来哪些改进。

技术优化取得的成果尽量要量化一下：

- 我使用 xxx 技术解决了 xxx 问题，系统 qps 从 xxx 提高到了 xxx。
- 我使用 xxx 技术优化了 xxx 接口，系统 qps 从 xxx 提高到了 xxx。

另外，如果你觉得你的项目技术比较落后的话，可以自己私下进行改进。重要的是让项目比较有亮点，通过什么方式就无所谓了。

项目经历介绍模板：

- **项目描述**：简单描述项目是做什么的，用了什么技术
- **工作内容**：简单描述自己做了什么，解决了什么问题，带来了什么实质性的改善。突出自己的能力，不要过于平淡的叙述。
- **个人收获（可选）**：从这个项目中你学会了那些东西，使用到了那些技术，学会了那些新技术的使用
- **项目成果（可选）**：简单描述这个项目取得了什么成绩。

个人工作内容描述最好可以体现自己的综合素质，比如你是如何协调项目组成员协同开发的或者在遇到某一个棘手的问题的时候你是如何解决的又或者说你在这个项目优化了某个模块的性能。示例：项目的 MySQL 数据库中的某张表的数据量达到千万级别，查询速度非常缓慢，数据库压力非常大，我使用 `Sharding-JDBC` 进行了分库分表，单表的数据量都在 300w 以下。

荣誉奖项（可选）

如果你有含金量比较高的竞赛（比如ACM、阿里的天池大赛）的获奖经历的话，荣誉奖项这块内容一定要写一下！并且，你还可以将荣誉奖项这块内容适当往前放，放在一个更加显眼的位置。

校园经历

如果有比较亮眼的校园经历的话就简单写一下，没有就不写！

个人评价

个人评价就是对自己的解读，一定要突出自己的亮点（勤奋、吃苦这些比较虚的东西就不要扯了）。比如你可以说自己的学习能力强，示例：大三参加国家软件设计大赛的时候快速上手 Python 写了一个可配置化的爬虫系统。再比如你可以说自己有团队精神，示例：大三参加某软件设计比赛的时候协调项目组内 5 名开发同学，并对编码遇到困难的同学提供帮助，最终顺利在 1 个月的时间完成项目的核心功能。

个人评价一定要简洁清晰，避免废话！

STAR 法则和 FAB 法则

STAR 法则 (Situation Task Action Result)

相信大家一定听说过 STAR 法则。对于面试，你可以将这个法则用在自己的简历以及和面试官沟通交流的过程中。

STAR 法则由下面 4 个单词组成（STAR 法则的名字就是由它们的首字母组成）：

- **Situation**：情景。事情是在什么情况下发生的？
- **Task**：任务。你的任务是什么？
- **Action**：行动。你做了什么？
- **Result**：结果。最终的结果怎样？

FAB 法则 (Feature Advantage Benefit)

除了 STAR 法则，你还需要了解在销售行业经常用到的一个叫做 FAB 的法则。

FAB 法则由下面 3 个单词组成（FAB 法则的名字就是由它们的首字母组成）：

- **Feature**：你的特征/优势是什么？
- **Advantage**：比别人好在哪些地方；
- **Benefit**：如果雇佣你，招聘方会得到什么好处。

简单来说，**FAB 法则**主要是让你的面试官知道你的优势和你能为公司带来的价值。

注意事项

1. 一定要使用 PDF 格式投递，不要使用 word 或者其他格式投递。这是最基本的！
2. 大部分公司的 HR 都说我们不看重学历（骗你的！）。如果你的学历比较差，记得通过其他方式弥补比如某某大厂的实习经历、获得了某某大赛的奖等等。
3. 大部分应届生找工作的硬伤是没有工作经验或实习经历，所以如果你是应届生就不要错过秋招和春招。一旦错过，你后面就极大可能会面临社招，这个时候没有工作经验的你可能就会面临各种碰壁，导致找不到一个好的工作。
4. 你不会的东西就不要写在简历。
5. 将自己的项目经历完美的展示出来非常重要，突出亮点。
6. 面试和工作是两回事，聪明的人会把面试官往自己擅长的领域领，其他人则被面试官牵着鼻子走。虽说面试和工作是两回事，但是你要想要获得自己满意的 offer，你自身的实力必须要强。

技巧

1. 尽量避免主观表述，少一点语义模糊的形容词，尽量要简洁明了，逻辑结构清晰。
2. 技术博客、Github 以及获奖经历等可以直接证明自己能力的东西，能写就尽量写在这里。但是，如果技术博客和 Github 上没有什么内容的话，就不要写了。
3. 注意简历真实性，一定不要写自己不会的东西，或者带有欺骗性的内容。适当润色没有问题。
4. 项目经历建议以时间倒序排序，另外项目经历不在于多（精选 2~3 即可），而在于有亮点。
5. 如果内容过多的话，不需要非把内容压缩到一页，保持排版干净整洁就可以了。
6. 简历最后最好能加上：“感谢您花时间阅读我的简历，期待能有机会和您共事。”这句话，显的你会很有礼貌。

本文节选自 [《Java 面试指北》](#) 的「面试准备篇」。

介绍	02-24 18:03
▼ 面试准备篇	+ :
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java 优质面试视频汇总	02-11 12:28
Java 优质开源面试题资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

1.3 常见面试题自测（付费）

面试之前，强烈建议大家多拿常见的面试题来进行自测，检查一下自己的掌握情况，这是一种非常实用的备战技术面试的小技巧。

在《Java 面试指北》的「技术面试题自测篇」，我总结了 Java 面试中最重要的知识点的最常见的面试题并按照面试提问的方式展现出来。

▼ 技术面试题自测篇

Java基础

Java 集合

Java并发

JVM

MySQL

Redis

每一道用于自测的面试题我都会给出重要程度，方便大家在时间比较紧张的时候根据自身情况来选择性自测。并且，我还会给出提示，方便你回忆起对应的知识点。

在面试中如果你实在没有头绪的话，一个好的面试官也是会给你提示的。

MySQL

注意!!!：下面这些问题的参考答案你几乎都可以在 [JavaGuide 在线阅读网站](#) 和 《JavaGuide 面试指北》中找到。并且，这两个参考资料没有给出解答的问题，我也都给了对应的参考文章。

建议你先阅读学习了对应的内容之后再自行自测。

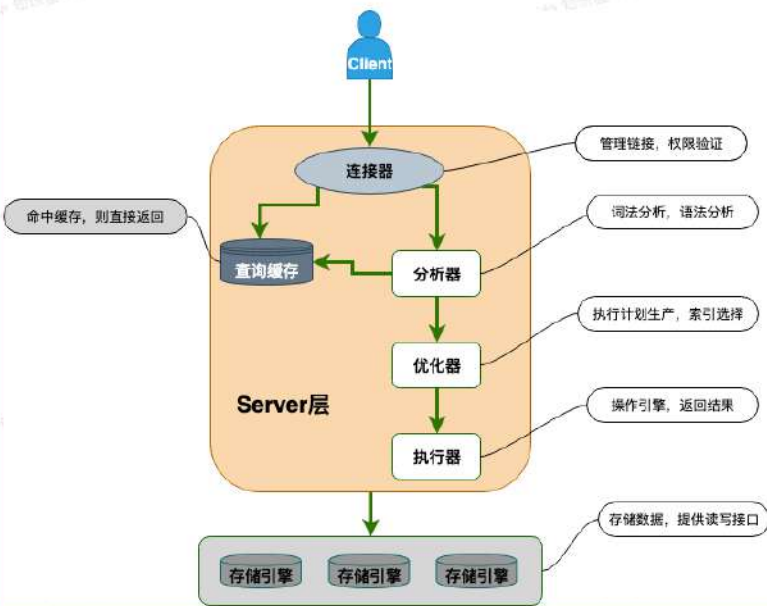
MySQL 基础架构

说说 MySQL 的架构?

🔥 重要程度: ★★★★★

💡 提示：面试官一般不会直接问你 MySQL 基础架构，通常会由“一个 SQL 语句在 MySQL 中的执行流程”类似的问题引出。

你需要搞懂下图每一个组件所提供的主要功能。



MySQL 基础架构

说说 MySQL 的架构?
一条 SQL 语句在 MySQL 中的...

MySQL 存储引擎

MySQL 提供了哪些存储引擎?
MySQL 存储引擎架构了解吗?
MyISAM 和 InnoDB 的区别

MySQL 事务

何谓事务?
何谓数据库事务?
ACID 特性指的是什么?
并发事务带来了哪些问题?
不可重复读和幻读区别
SQL 标准定义了哪些事务隔离...
MySQL 的默认隔离级别是什...
什么是 MVCC? 有什么用? 原...
InnoDB 事务隔离级别实现原理

MySQL 锁

表级锁和行级锁了解吗? 有什...
行级锁的使用有什么注意事项?
共享锁和排他锁呢?
意向锁有什么作用?
InnoDB 有哪几类行锁?

MySQL 索引

何为索引? 有什么作用?
索引的优缺点
索引的底层数据结构
MySQL 的索引结构为什么使...
主键索引和二级索引
聚集索引与非聚集索引
覆盖索引
联合索引
最左前缀匹配原则
创建索引的注意事项有哪些?

MySQL 日志

MySQL 中常见的日志有哪些?
慢查询日志有什么用?
binlog 主要记录了什么? 有...
redo log 如何保证事务的持久...
页修改之后为什么不直接刷...
binlog 和 redo log 有什么区别?

欢迎准备 Java 面试以及学习 Java 的同学加入我的 [知识星球](#)，干货非常多，学习氛围非常好！收费虽然是白菜价，但星球里的内容或许比你参加上万的培训班质量还要高。

JavaGuide&Java面试交流圈

星主: Guide哥



1.5万
成员数量

9800+
内容数量

1024
运营天数

最优质的 Java 面试交流星球，多次被知识星球官方推荐。门票即将上调到 199 元，IOS 用户请在公众号“JavaGuide”回复“知识星球”加入/续费。

...

 知识星球

微信扫码加入星球 ▶



我有自己的原则，不割韭菜，用心做内容，真心希望帮助到你！

如果你感兴趣的话，不妨花 3 分钟左右看看星球的详细介绍：[JavaGuide 知识星球详细介绍](#)（文末有优惠券）。



1.4 面试常见词汇扫盲

本文节选自 [《Java 面试指北》](#) 的「面试准备篇」

春招和秋招

春招的时候一般会同时进行 **准应届生暑期实习生招聘** 和 **应届生校园招聘**（准应届生指的是来年毕业的在校大学生）。

不过，这个时候应届生校园招聘的岗位相对已经比较少了，基本是对秋招的补招，秋招的时候才是应届生校园招聘的关键时期。

春招期间，集中进行的实习生招聘一般是暑期实习生招聘。

秋招一般 7 月份就开始了，大概一直持续到 9 月底。春招一般 3 月份就开始了，大概一直持续到 4 月底。很多公司（尤其大厂）到了 9 月中旬(秋招)/3 月中旬（春招），很可能就会没有 HC 了。

暑期实习和日常实习

暑期实习通常是在春招的时候开始大规模招聘，面试难度大于日常的实习招聘，性价比也比日常实习要高。



JavaGuide

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

扫码关注

一般来说，暑期实习会在 6-7 月也就是暑期那会入职。

日常实习通常全年都会进行，一般为部门的散招，一不会给转正名额。日常实习生的招聘对象通常是大一、大二、研一、研二的同学。

一般来说，拿到日常实习 offer 后，立刻就会入职。

提前批

为什么很多公司有提前批？

很明显啊！提前批就是各个公司提前抢夺一波优秀毕业生。

你没必要担心这个提前批的含金量如何，觉得自己能力足够的话，一定要把握这次机会！提前批还是会有很多 sp 甚至 ssp offer 的！

为什么推荐提前批呢？

因为，提前批的结果并不影响你的秋招，也就是说你可以多一次机会。这样的话，即使你失败了，也没关系，好好分析一下自己的短板，努力准备秋招就完事了！并且许多公司的提前批是直接面试，免笔试的。

但是！我这里建议，投提前批的时候，不要一次把你最想去的公司全投了。比如你最想去腾讯、百度、阿里。那么你提前批可以投百度，再投两个小一些的公司，然后根据几次的面试反馈继续提升自己，再陆续去投自己最想去的公司。虽然很多公司都说面试挂了不影响正式批再战，但是你面试的时候会有评价记录的，这个面试记录 hr 是可以看到的，以后的面试官面试也会看到。如果面试官给你的评价记录比较中性还好，但如果面试官给你一个很差的面试评价。那么正式批的时候 hr 筛简历就不会通过你了。我去年面试快手提前批没过，不知道那位面试官给我写的是什么评价，简历再投别的部门就通不过了。但是面字节虽然第一次面试没通过，我后续还是被很多部门捞。

如果提前批有那种部门组织的预面试，就是不会被录入公司系统的面试，这种机会你要果断投简历。这种面试机会很难得，公司不会有你的面试记录，面试没过也不会影响你后续投别的部门，还获得了一次难得的面试机会。一定不要因为觉得自己没准备好而放弃这种面试，大厂的每一次面试都是特别好的学习机会。其实许多人最初几次面试都是不能通过的，经历过几次失败，然后总结面试中的问题，你就离大厂 offer 越来越近了。

偷偷告诉你：这些大厂可能会组织那种不留面试记录的部门预面试，阿里、百度、京东、字节跳动～大家可以去找在这些公司工作的学长学姐了解，也可以去牛客上了解。

内推

每年的秋招开始以后大家可能会看到大量的内推宣传。但是不同形式的内推差别其实是很大的。如果只是从网上随便找一个内推码，内推人都不认识就把简历投了，这种内推是没用的。有用的内推是，内推者可以直接把你的简历交到筛选简历的部门 HR 手里，这样 HR 能快速看到你的简历，并且给你安排面试。

HC (Headcount)

俗称人头，稍微专业点讲就是这家公司打算招的人数。公司会录用很多实习生，也有“广撒 offer”的说法，把人留住，但实际最后只会录用其中的一部分，不会录取所有。最后真正录取的实习生，即可转正。而不被录取的一部分，可能是不在 HC 之内，由于工作能力、工作需要等等。以往都是先定了 HC 再发 offer，但最近新闻上也有很多企业是先发了 offer，但后来再以 HC 已招够为由来拒收实习生的。所以同学们在找实习，申请校招的时候要格外注意这一点。

面试记录

大家进行互联网公司组织的面试，都会留下自己的面试记录。面试记录上会有面试官的面试评语。这个面试记录，是以后面试你的面试官还有 HR 都能看到的。

预面试

部门收到你的简历后，先不录入公司系统，由 HR 筛选。如果通过简历筛选。部门直接发起预面试，面试通过后，录入系统直接走下面的流程。面试不过，不影响你投这个公司的其它部门，因为公司没有你的面试记录。找预面试的途径是找自己在这个公司的师兄师姐，或者在牛客网上找部门直招的帖子。预面试在部分公司是不合规的。

主管面

主管面指的是部门的技术主管对你进行面试，走到这一关可以证明大家的技术已经问题不大了。主管面基本上都会采用半问技术，半聊理想的形式对你进行面试。有时候也会问你在校的一些活动经历，甚至会问你毕业论文在做什么。主管面除了考察技术外，一个重要的考察点是考察你是否和团队契合。

HR 面

HR 面指的就是人力资源对你进行面试。HR 通常第一个问题就是你是哪人，这个问题其实是想看你是不是来公司面试解闷子的。如果你面的是一家北京的公司，而且你是河北人、河南人、山西人等北京周边的城市，你说了你是哪人以后你就不用多说了。但是如果你家是西北那边的，上学又是在东北那嘎达上的，又恰巧你面的是一个广州深圳的公司，你最好说清楚你为啥想去那边工作。另外，HR 会问一些在校经历，通过交流来判断你的性格是否符合团队。对了，还有一个 HR 常问问题，你拿到了哪些 offer？这个问题你就要甩出一些比较硬的 offer 了，因为优质人才谁都想抢。但是你甩出的 offer 要和现在面试的公司是在一个量级上的。不要你面试的是一个公司，你跟人家说你已经拿到了字节的工牌，你觉得人家相信不相信给了你 offer 你会来？

八股文

各种面试题题目，主要是一些概念性的知识，比如 `jvm` 的运行时数据区的构成、`mysql` 的索引之类的，这些问题的回答一般有固定套路。现在的面试主要就是八股文+算法。我在之后的文章也在总结面试八股文的重点，预计一周内能发出来。面试八股文背的熟是面试成功的必要不充分条件。现在背八股文也是一个潮流，但是我其实不太喜欢这个潮流。大家在平时学习时还是要打好基础，我把平时看到的比较好的计算机基础资料收集在我的公众号里，大家关注 CS 指南，回复计算机基础就能领取。

手撕算法

手撕算法简单来说就是完成面试官给你布置的算法题（有些公司提供思路即可）。国内现在的校招面试开始越来越重视算法了，尤其是像字节跳动、腾讯这类大公司。绝大部分公司的校招笔试是有算法题的，如果 AC 率比较低的话，基本就挂掉了。

常规面试

现在互联网大厂的常规面试大多都采用这种形式，前半小时自我介绍、问项目、背面试八股文，后半小时一道代码题。

本文节选自 [《Java 面试指北》](#) 的「面试准备篇」。

介绍	02-24 18:03
▼ 面试准备篇	+
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java 优质面试视频汇总	02-11 12:28
Java 优质开源面试题资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14



JavaGuide

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

扫码关注

1.5 优质面经汇总（付费）

古人云：“他山之石，可以攻玉”。善于学习借鉴别人的面试的成功经验或者失败的教训，可以让自己少走许多弯路。

在《Java 面试指北》的「面经篇」，我分享了 15+ 篇高质量的 Java 后端面经，有校招的，也有社招的，有大厂的，也有中小厂的。

如果你是非科班的同学，也能在这些文章中找到对应的非科班的同学写的面经。

全部

精华

只看星主

面经

面试题

工具

开源项目

好文分享

进阶攻略

文件

等你回答



Guide哥

3 分钟前

一位球友的 2022 京东、佳创视讯、源创科技跳槽Java面试经验

京东一面

1. 单例模式、模板方法

3. Redis 分布式锁

4. Redis 的缓存击穿

展开全部





r09er

2022/5/23

收进专栏

...

#面试# shipline一面（已挂）

5年Java，shipline主站高级Java岗，全程八股文，难顶。😓

1.多线程

1.1如何打破线程池先放队列再扩容的逻辑，有什么框架实现了

2.synchronized和ReentrantLock区别

2.1synchronized如何实现等待唤醒

2.2ReentrantLock用了Unsafe类的哪个方法

3.dubbo

展开全部

面试

面经



Indelibili*

2022/5/7

收进专栏

...

GO后端实习面经

1.先自我介绍一下吧

2.说一下进程和线程的定义，以及他们之间的关系

3.进程间的通讯方式有哪些

4.了解mysql的索引吗，索引有什么好处，有什么坏处

5.了解mysql的索引的底层结构是什么，为什么选这个，有什么好处？

6.如果mysql的访问量过大，该使用什么措施？

7.缓存有什么用？为什么要用缓存，有使用过缓存吗？

8.redis除了可以做缓存还能做什么东西？

9.说一下r...

GO后端实习面经

欢迎准备 Java 面试以及学习 Java 的同学加入我的 [知识星球](#)，干货非常多，学习氛围非常好！收费虽然是白菜价，但星球里的内容或许比你参加上万的培训班质量还要高。

JavaGuide&Java面试交流圈

星主: Guide哥



1.5万
成员数量

9800+
内容数量

1024
运营天数

最优质的 Java 面试交流星球，多次被知识星球官方推荐。门票即将上调到 199 元，IOS 用户请在公众号“JavaGuide”回复“知识星球”加入/续费。

...

知识星球

微信扫码加入星球 ▶



我有自己的原则，不割韭菜，用心做内容，真心希望帮助到你！

如果你感兴趣的话，不妨花 3 分钟左右看看星球的详细介绍：[JavaGuide 知识星球详细介绍](#)（文末有优惠券）。



Guide哥 送你一张星球优惠券

「JavaGuide&Java面试交流圈」

立减

¥ 20

新人立减券

2023/08/01 12:00 后失效

知识星球

长按扫码领取优惠



1.6 项目经验指南

本文节选自 《Java 面试指北》 的「面试准备篇」

没有项目经验怎么办？

没有项目经验是大部分应届生会碰到的一个问题。甚至说，有很多有工作经验的程序员，对自己在公司做的项目不满意，也想找一个比较有技术含量的项目来做。

说几种我觉得比较靠谱的获取项目经验的方式，希望能够对你有启发。

实战项目视频/专栏

在网上找一个符合自己能力与找工作需求的实战项目视频或者专栏，跟着老师一起做。

你可以通过慕课网、哔哩哔哩、拉勾、极客时间、培训机构（比如黑马、尚硅谷）等渠道获取到适合自己的实战项目视频/专栏。



实战课

真实项目实战演练

热搜:

Java工程师

Vue

Python

Go

方向:

全部

前端开发

后端开发

移动开发

计算机基础

前沿技术

云计算&大数据

运维&测试

数据库

UI设计&多媒体

游戏

求职面试

分类:

不限

Java

SpringBoot

Spring Cloud

SSM

PHP

.net

Python

爬虫

Django

Flask

Go

C

C++

C#

最新

销量

升级

学习路线

数据工程师
实战进阶指南

构建数据分析工程师能力模型，实战八大企业级项目
进阶，66人报名
暑期特惠 ¥359.00 ¥399.00 加入购物车

Go+ES8构建
高性能搜索微服务

海量数据高并发场景，构建Go+ES8企业级搜索微服务
高阶，99人报名
暑期特惠 ¥545.00 ¥599.00 加入购物车

算法与数据结构
高质量进阶

算法与数据结构高手养成-求职提升特训课
进阶，101人报名
暑期特惠 ¥1179.00 ¥1299.00 加入购物车

轻松入门
大数据之Hadoop

轻松入门大数据(1) 一站式完成核心能力构建
零基础，83人报名
暑期特惠 ¥899.00 ¥1199.00 加入购物车

Go底层原理+实战
重构Redis中间件

深入Go底层原理，重写Redis中间件实战
高阶，244人报名
暑期特惠 ¥459.00 ¥499.00 加入购物车

2022版
Java 分布式架构
设计与实战

2022全新版-Java分布式架构设计与开发实战
进阶，173人报名
暑期特惠 ¥359.00 ¥399.00 加入购物车

KubeEdge实战
云原生边缘计算

云原生+边缘计算项目实战-KubeEdge打造边缘管理平台
进阶，105人报名
¥468.00 加入购物车

商业级代驾项目
全流程落地

多端全栈项目实战，大型商业级代驾业务全流程落地
进阶，275人报名
暑期特惠 ¥1179.00 ¥1299.00 加入购物车

尽量选择一个适合自己的项目，没必要必须做分布式/微服务项目，对于绝大部分同学来说，能把一个单机项目做好就已经很不错的了。

我面试过很多求职者，简历上看着有微服务的项目经验，结果随便问两个问题就知道根本不是自己做的或者说做的时候压根没认真思考。这种情况会给我留下非常不好的印象。

我在《Java 面试指北》的「面试准备篇」中也说过：

个人认为也没必要非要去微服务或者分布式项目，不一定对你面试有利。微服务或者分布式项目涉及的知识点太多，一般人很难吃透。并且，这类项目其实对于校招生来说稍微有一点超标了。即使你做出来，很多面试官也会认为不是你独立完成的。

其实，你能把一个单体项目做到极致也很好，对于个人能力提升不比做微服务或者分布式项目差。如何做到极致？代码质量这里就不提了，更重要的是你要尽量让自己的项目有一些亮点（比如你是如何提升项目性能的、如何解决项目中存在的一个痛点的），项目经历取得的成果尽量要量化一下比如我使用 xxx 技术解决了 xxx 问题，系统 qps 从 xxx 提高到了 xxx。

跟着老师做的过程中，你一定要有自己的思考，不要浅尝辄止。对于很多知识点，别人的讲解可能只是满足项目就够了，你自己想多点知识的话，对于重要的知识点就要自己学会去深入学习。

实战类开源项目

Github 或者码云上面有很多实战类别项目，你可以选择一个来研究，为了让自己对这个项目更加理解，在理解原有代码的基础上，你可以对原有项目进行改进或者增加功能。

你可以参考 [Java 优质开源实战项目](#) 上面推荐的实战类开源项目，质量都很高，项目类型也比较全面，涵盖博客/论坛系统、考试/刷题系统、商城系统、权限管理系统、快速开发脚手架以及各种轮子。



技术教程

实战项目

[博客/论坛系统](#)[考试/刷题系统](#)[商城系统](#)[权限管理系统](#)[快速开发脚手架](#)[造轮子](#)

系统设计

工具类库

开发工具

机器学习

大数据

[Java 面试指南](#) / [Java 开源项目精选](#) / [Java 优质开源实战项目](#)

Java 优质开源实战项目

[Guide](#) [开源项目](#) [2022年3月13日](#) [约 2480 字](#)

博客/论坛系统

下面这几个项目都是非常适合 Spring Boot 初学者学习的，下面的大部分项目的总体代码不错，不会误导没有实际做过项目的朋友。

- [blog](#)：一款基于 SpringBoot + Vue 开发的前后端分离博客，非常精致，功能也比通用 SpringSecurity 进行权限管理，ElasticSearch 全文搜索，支持 QQ、微博第三方登录
相关阅读：[这个 SpringBoot+ Vue 开源博客系统太酷炫了！](#)
- [forest](#)：下一代的知识社区系统，可以自定义专题和作品集。后端基于 SpringBoot + Redis，前端基于 Vue + NuxtJS + Element-UI。
- [vhr](#)：微人事是一个前后端分离的人力资源管理系统，项目采用 SpringBoot+Vue 开发
- [favorites-web](#)：云收藏 Spring Boot 2.X 开源项目。云收藏是一个使用 Spring Boot 构随时随地收藏的一个网站，在网站上分类整理收藏的网站或者文章。
- [community](#)：开源论坛、问答系统，现有功能提问、回复、通知、最新、最热、消除技术栈 Spring、Spring Boot、MyBatis、MySQL/H2、Bootstrap。
- [VBlog](#)：V 部落，Vue+SpringBoot 实现的多用户博客管理平台！
- [My-Blog](#)：My Blog 是由 SpringBoot + Mybatis + Thymeleaf 等技术实现的 Java 博客，部署简单及完善的代码，一定会给使用者无与伦比的体验。

考试/刷题系统

- [uexam](#)：一个非常不错的考试系统！考试系统应用场景还挺多的，不论是对于在校大学生，类似的私活也有很多。相关阅读：[《好一个 Spring Boot 开源在线考试系统！解决面试刷题系统》](#)
- [PassJava-Platform](#)：一个基于微服务(SpringBoot、Spring Cloud)的面试刷题系统！[Cloud 的面试刷题系统。面试、毕设、项目经验一网打尽！](#)

相关文章：[想要搭建个人博客？我调研了 100 来个 Java 开源博客系统，发现这 5 个最好用](#)

一定要记住：不光要做，还要改进，改善。不论是实战项目视频或者专栏还是实战类开源项目，都一定会有很多可以完善改进的地方。

从头开始做

自己动手去做一个自己想完成的东西，遇到不会的东西就临时去学，现学现卖。

这个要求比较高，我建议你已经有了一个项目经验之后，再采用这个方法。如果你没有做过项目的话，还是老老实实采用上面两个方法比较好。

参加各种大公司组织的各种大赛

如果参加这种赛事能获奖的话，项目含金量非常高。即使没获奖也没啥，也可以写简历上。

阿里云 TIANCHI 天池

登录注册

首页天池大赛天池实验室AI学习数据集技术圈天池7号馆其他

En

天池大数据竞赛

打造国际高端算法竞赛，让选手用算法解决社会或业务问题

Active算法大赛创新应用大赛程序设计大赛新人赛可视化大赛诸神之战

【中间件极客榜】自适应负载均衡的设计实现

程序设计大赛

赛事简况：中间件性能挑战赛是由阿里云发起的工程视角品牌赛事，初衷是为热爱技术的年轻人提供一个挑战世界级技术问题的舞台，希望选手在追求性能极致时能深刻体会技术人的匠心精神。...

主办方：阿里云

奖金¥ 0

团队641

赛季 12020-04-10

进行中

Apache Flink极客挑战赛——Flink TPC-DS性能优化

程序设计大赛

赛事简况：Apache Flink 极客挑战赛由 Apache Flink Community China 发起，阿里云计算平台事业部、天池平台、Intel 联合举办。作为新一代大数据计算引擎，Apache Flink 强大的计算性能及...

奖金¥ 200000

团队1415

赛季 22019-11-12

已结束

参与实际项目

通常情况下，你有如下途径接触到企业实际项目的开发：

- 1. 老师接的项目；
- 2. 自己接的私活；
- 3. 实习/工作接触到的项目；

老师接的项目和自己接的私活通常都是一些偏业务的项目，很少会涉及到性能优化。这种情况下，你可以考虑对项目进行改进，别怕花时间，某个时间用心做好一件事情就好比如你对项目的数据模型进行改进、引入缓存提高访问速度等等。

实习/工作接触到的项目类似，如果遇到一些偏业务的项目，也是要自己私下对项目进行改进优化。

尽量是真的对项目进行了优化，这本身也是对个人能力的提升。如果你实在是没时间去实践的话，也没关系，吃透这个项目优化手段就好，把一些面试可能会遇到的问题提前准备一下。

跟着视频做的项目会被面试官嫌弃不？

很多应届生都是跟着视频做的项目，这个大部分面试官都心知肚明。

不排除确实有些面试官不吃这一套，这个也看人。不过我相信大多数面试官都是能理解的，毕竟你在学校的时候实际上是没有获得实际项目经验的途径的。

大部分应届生的项目经验都是自己在网上找的或者像你一样买的付费课程跟着做的，极少部分是比较真实的项目。从你能想着做一个实战项目来说，我觉得初衷是好的，确实也能真正学到东西。但是，究竟有多少是自己掌握了很重要。看视频最忌讳的是被动接受，自己多改进一下，多思考一下！就算是你跟着视频做的项目，也是可以优化的！

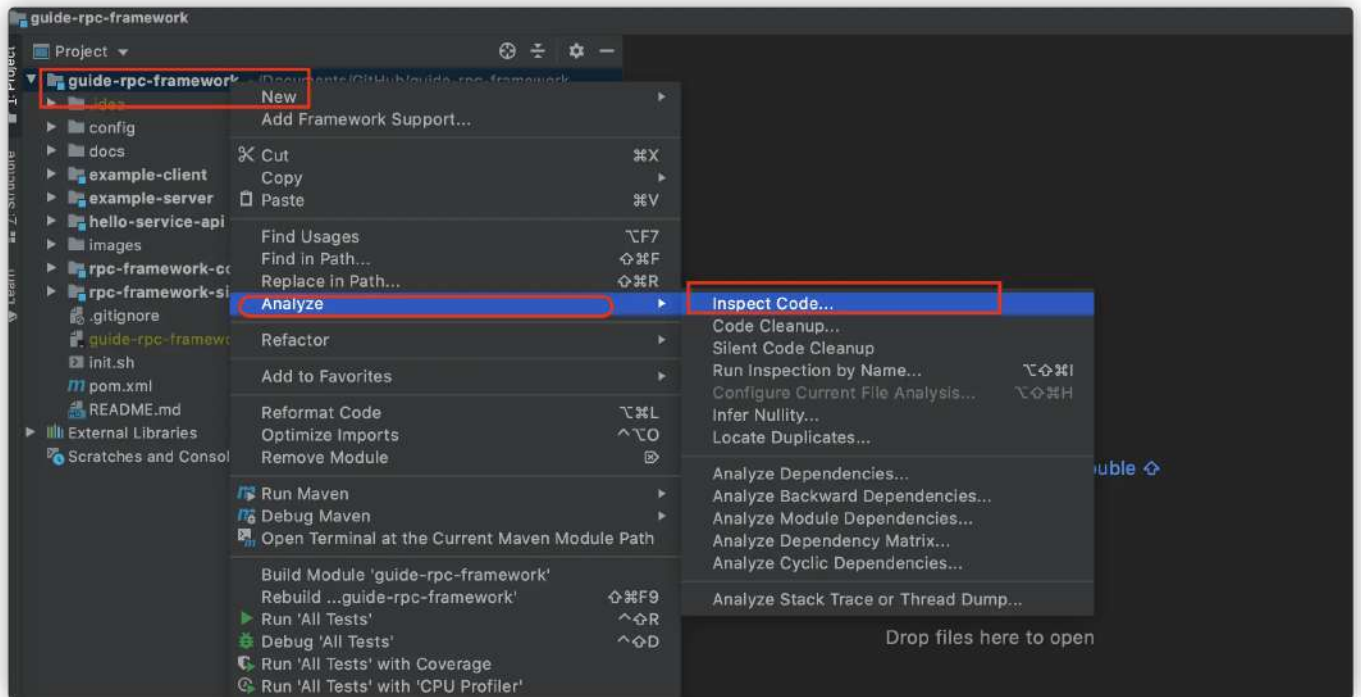
如果你想真正学到东西的话，建议不光要把项目单纯完成跑起来，还要去自己尝试着优化！

简单说几个比较容易的优化点：

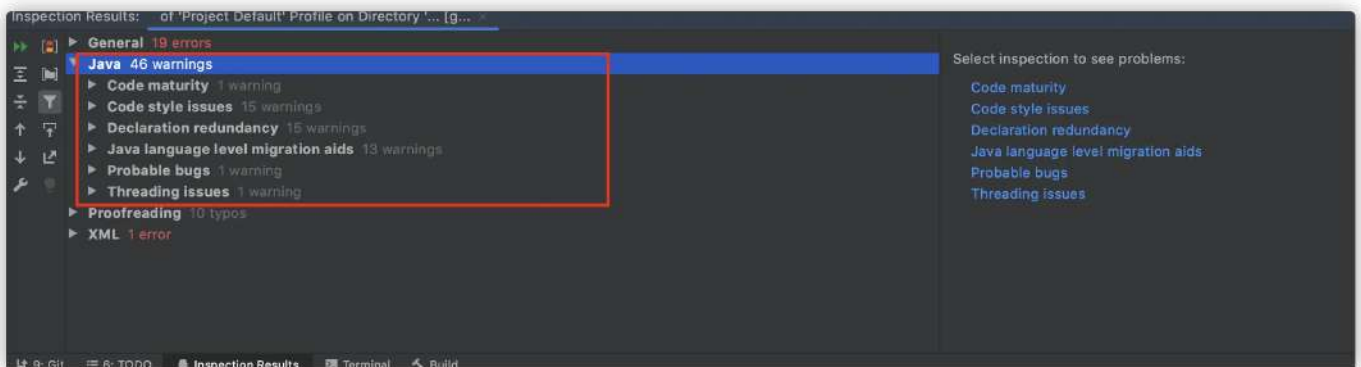
1. **全局异常处理**：很多项目这方面都做的不是很好，可以参考我的这篇文章：[《使用枚举简单封装一个优雅的 Spring Boot 全局异常处理！》](#) 来做优化。
2. **项目的技术选型优化**：比如使用 Guava 做本地缓存的地方可以换成 **Caffeine**。Caffeine 的各方面的表现要更加好！再比如 Controller 层是否放了太多的业务逻辑。
3. **数据库方面**：数据库设计可否优化？索引是否使用使用正确？SQL 语句是否可以优化？是否需要进行读写分离？
4. **缓存**：项目有没有哪些数据是经常被访问的？是否引入缓存来提高响应速度？
5. **安全**：项目是否存在安全问题？
6.

然后，再给大家推荐一个 **IDEA** 优化代码的小技巧，超级实用！

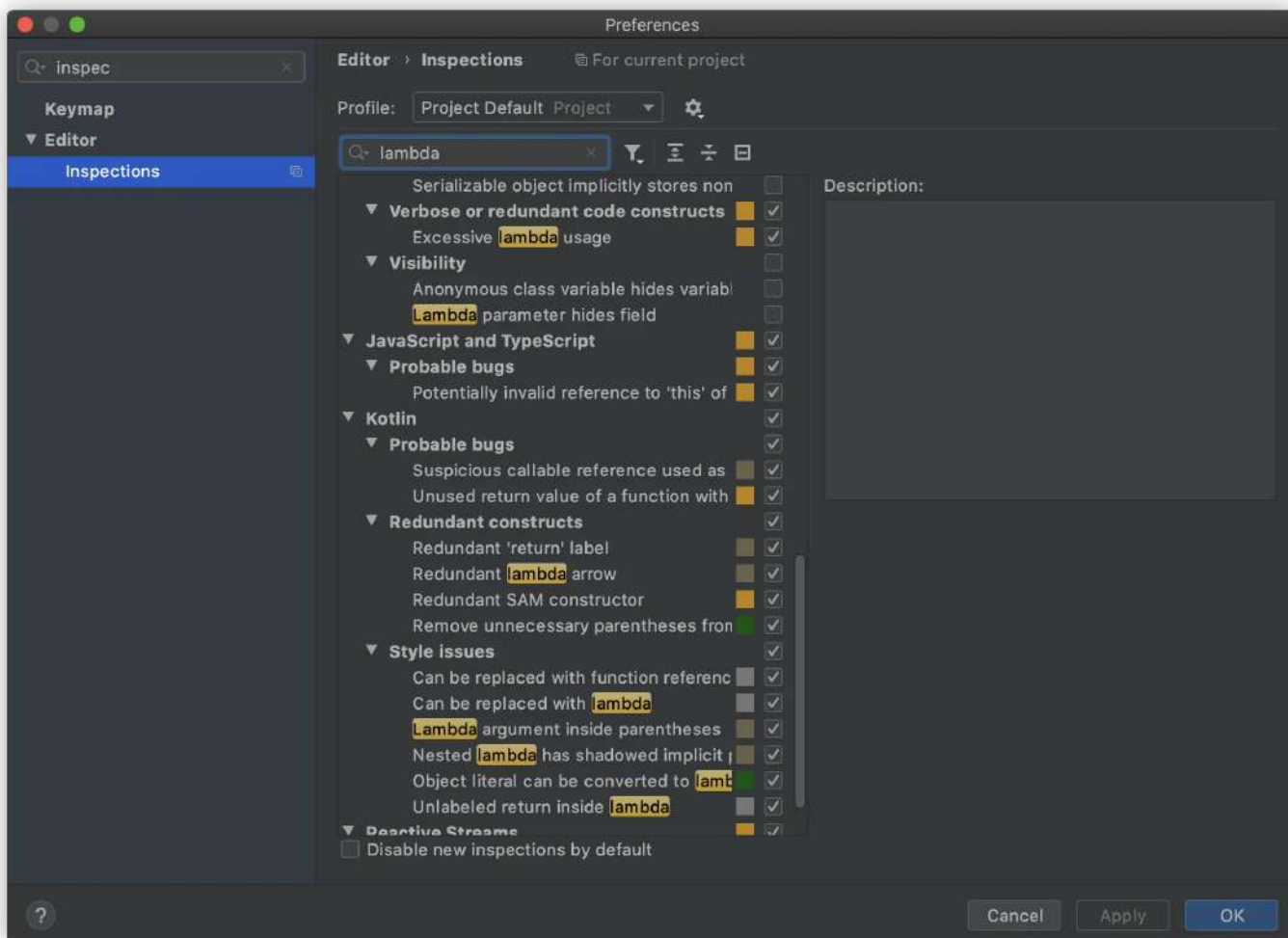
分析你的代码：右键项目->Analyze->Inspect Code



扫描完成之后，IDEA 会给出一些可能存在的代码坏味道比如命名问题。



并且，你还可以自定义检查规则。



如何优化项目性价比更高？

面试之前，你可以跟着网上的教程，从性能优化方向入手去改进一下自己的项目。为什么建议从性能优化方向入手呢？因为性能优化方向改进相比较于业务方向的改进性价比会更高，更容易体现在简历上。并且，更重要的是，性能优化方向更容易在面试之前提前准备，面试官也更喜欢提问这类问题。

你项目没有用到的性能优化手段，只要你搞懂吃透并且觉得合理，你就完全可以写在简历上。不过，建议你还是要实践一下，压测一波，取得的成果也要量化一下比如我使用 xxx 技术解决了 xxx 问题，系统 qps 从 xxx 提高到了 xxx。

必须是微服务项目才有亮点？

个人认为也没必要非要去做微服务或者分布式项目，不一定对你面试有利。微服务或者分布式项目涉及的知识点太多，一般人很难吃透。并且，这类项目其实对于校招生来说稍微有一点超标了。即使你做出来，很多面试官也会认为不是你独立完成的。

其实，你能把一个单体项目做到极致也很好，对于个人能力提升不比做微服务或者分布式项目差。如何做到极致？代码质量这里就不提了，更重要的是你要尽量让自己的项目有一些亮点（比如你是如何提升项目性能的、如何解决项目中存在的一个痛点的），项目经历取得的成果尽量要量化一下比如我使用 xxx 技术解决了 xxx 问题，系统 qps 从 xxx 提高到了 xxx。

本文节选自 [《Java 面试指北》](#) 的「面试准备篇」。

介绍	02-24 18:03
▼ 面试准备篇	+
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java 优质面试视频汇总	02-11 12:28
Java 优质开源面试题资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“PDF”领取原创面试 PDF 手册
- 2、关注公众号回复“加群”进入面试交流群
- 3、关注公众号回复“八股文”获取大厂面试真题+面经

2. Java

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide!



2.1. Java 基础

这部分内容摘自 [JavaGuide](#) 下面几篇文章：

- [Java基础常见面试题总结\(上\)](#)
- [Java基础常见面试题总结\(中\)](#)
- [Java基础常见面试题总结\(下\)](#)

JVM vs JDK vs JRE

JVM

Java 虚拟机（JVM）是运行 Java 字节码的虚拟机。JVM 有针对不同系统的特定实现（Windows, Linux, macOS），目的是使用相同的字节码，它们都会给出相同的结果。字节码和不同系统的 JVM 实现是 Java 语言“一次编译，随处可以运行”的关键所在。

JVM 并不是只有一种！只要满足 JVM 规范，每个公司、组织或者个人都可以开发自己的专属 JVM。 也就是说我们平时接触到的 HotSpot VM 仅仅是是 JVM 规范的一种实现而已。

除了我们平时最常用的 HotSpot VM 外，还有 J9 VM、Zing VM、JRockit VM 等 JVM。维基百科上就有常见 JVM 的对比：[Comparison of Java virtual machines](#)，感兴趣的可以去看看。并且，你可以在 [Java SE Specifications](#) 上找到各个版本的 JDK 对应的 JVM 规范。

Java Language and Virtual Machine Specifications

Java SE 18

Released March 2022 as [JSR 393](#)



The Java Language Specification, Java SE 18 Edition

- [HTML](#) | [PDF](#)
- Preview feature: [Pattern Matching for switch](#)



The Java Virtual Machine Specification, Java SE 18 Edition

- [HTML](#) | [PDF](#)

Java SE 17

Released September 2021 as [JSR 392](#)



The Java Language Specification, Java SE 17 Edition

- [HTML](#) | [PDF](#)
- Preview feature: [Pattern Matching for switch](#)



The Java Virtual Machine Specification, Java SE 17 Edition

- [HTML](#) | [PDF](#)

JDK 和 JRE

JDK 是 Java Development Kit 缩写，它是功能齐全的 Java SDK。它拥有 JRE 所拥有的一切，还有编译器（javac）和工具（如 javadoc 和 jdb）。它能够创建和编译程序。

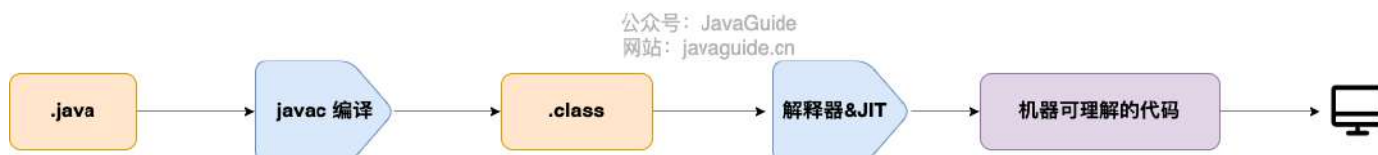
JRE 是 Java 运行时环境。它是运行已编译 Java 程序所需的所有内容的集合，包括 Java 虚拟机（JVM），Java 类库，java 命令和其他的一些基础构件。但是，它不能用于创建新程序。

如果你只是为了运行一下 Java 程序的话，那么你只需要安装 JRE 就可以了。如果你需要进行一些 Java 编程方面的工作，那么你就需要安装 JDK 了。但是，这不是绝对的。有时，即使您不打算在计算机上进行任何 Java 开发，仍然需要安装 JDK。例如，如果要使用 JSP 部署 Web 应用程序，那么从技术上讲，您只是在应用程序服务器中运行 Java 程序。那你为什么需要 JDK 呢？因为应用程序服务器会将 JSP 转换为 Java servlet，并且需要使用 JDK 来编译 servlet。

什么是字节码?采用字节码的好处是什么?

在 Java 中, JVM 可以理解的代码就叫做字节码(即扩展名为 `.class` 的文件), 它不面向任何特定的处理器, 只面向虚拟机。Java 语言通过字节码的方式, 在一定程度上解决了传统解释型语言执行效率低的问题, 同时又保留了解释型语言可移植的特点。所以, Java 程序运行时相对来说还是高效的(不过, 和 C++, Rust, Go 等语言还是有一定差距的), 而且, 由于字节码并不针对一种特定的机器, 因此, Java 程序无须重新编译便可在多种不同操作系统的计算机上运行。

Java 程序从源代码到运行的过程如下图所示:



我们需要格外注意的是 `.class->机器码` 这一步。在这一步 JVM 类加载器首先加载字节码文件, 然后通过解释器逐行解释执行, 这种方式的执行速度会相对比较慢。而且, 有些方法和代码块是经常需要被调用的(也就是所谓的热点代码), 所以后面引进了 JIT (just-in-time compilation) 编译器, 而 JIT 属于运行时编译。当 JIT 编译器完成第一次编译后, 其会将字节码对应的机器码保存下来, 下次可以直接使用。而我们知道, 机器码的运行效率肯定是高于 Java 解释器的。这也解释了我们为什么经常会说 **Java 是编译与解释共存的语言**。

HotSpot 采用了惰性评估(Lazy Evaluation)的做法, 根据二八定律, 消耗大部分系统资源的只有那一小部分的代码(热点代码), 而这也就是 JIT 所需要编译的部分。JVM 会根据代码每次被执行的情况收集信息并相应地做出一些优化, 因此执行的次数越多, 它的速度就越快。JDK 9 引入了一种新的编译模式 AOT(Ahead of Time Compilation), 它是直接将字节码编译成机器码, 这样就避免了 JIT 预热等各方面的开销。JDK 支持分层编译和 AOT 协作使用。

为什么不全部使用 AOT 呢?

AOT 可以提前编译节省启动时间, 那为什么不全部使用这种编译方式呢?

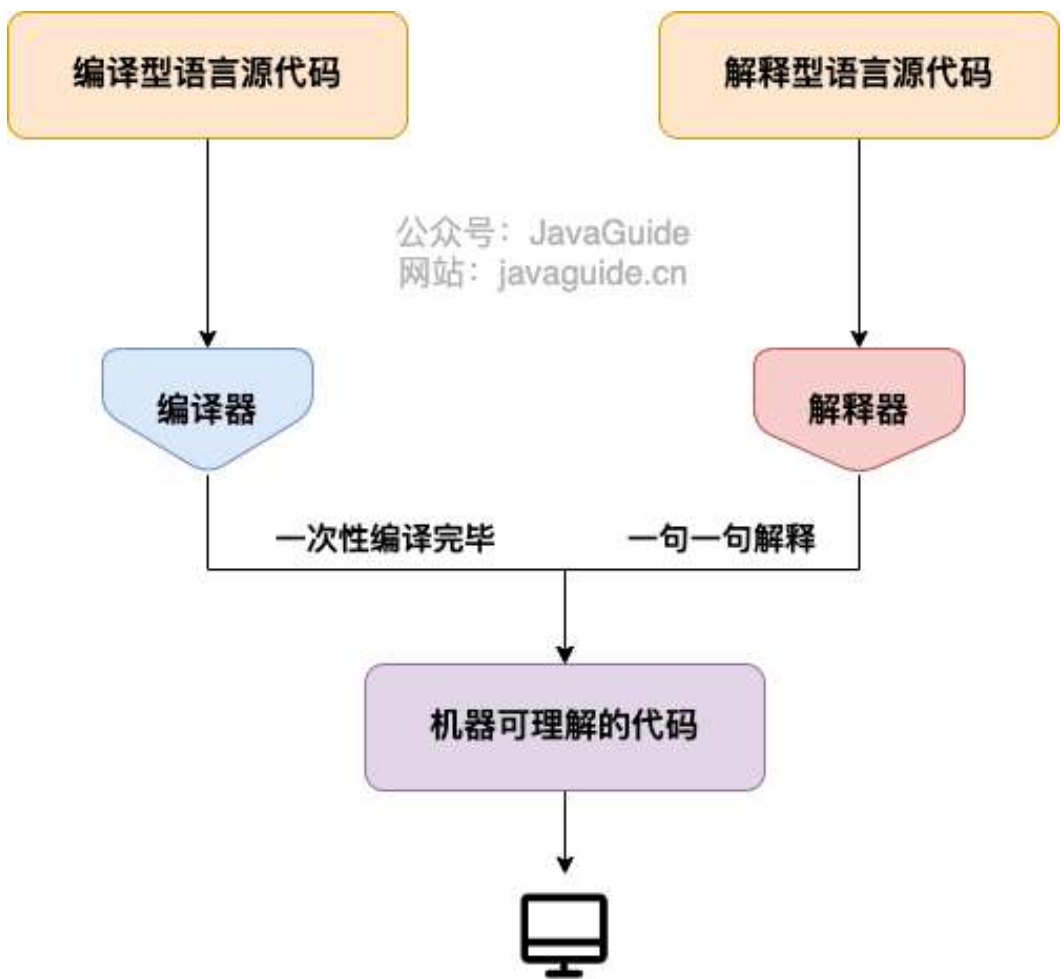
长话短说, 这和 Java 语言的动态特性有千丝万缕的联系了。举个例子, CGLIB 动态代理使用的是 ASM 技术, 而这种技术大致原理是运行时直接在内存中生成并加载修改后的字节码文件也就是 `.class` 文件, 如果全部使用 AOT 提前编译, 也就不能使用 ASM 技术了。为了支持类似的动态特性, 所以选择使用 JIT 即时编译器。

为什么说 Java 语言“编译与解释并存”？

其实这个问题我们讲字节码的时候已经提到过，因为比较重要，所以我们这里再提一下。

我们可以将高级编程语言按照程序的执行方式分为两种：

- **编译型**：编译型语言会通过编译器将源代码一次性翻译成可被该平台执行的机器码。一般情况下，编译语言的执行速度比较快，开发效率比较低。常见的编译性语言有 C、C++、Go、Rust 等等。
- **解释型**：解释型语言会通过解释器一句一句的将代码解释（interpret）为机器代码后再执行。解释型语言开发效率比较快，执行速度比较慢。常见的解释性语言有 Python、JavaScript、PHP 等等。



根据维基百科介绍：

为了改善编译语言的效率而发展出的**即时编译**技术，已经缩小了这两种语言间的差距。这种技术混合了编译语言与解释型语言的优点，它像编译语言一样，先把程序源代码编译成**字节码**。到执行期时，再将字节码直译，之后执行。**Java**与**LLVM**是这种技术的代表产物。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

这是因为 Java 语言既具有编译型语言的特征，也具有解释型语言的特征。因为 Java 程序要经过先编译，后解释两个步骤，由 Java 编写的程序需要先经过编译步骤，生成字节码（`.class` 文件），这种字节码必须由 Java 解释器来解释执行。

Oracle JDK vs OpenJDK

可能在看这个问题之前很多人和我一样并没有接触和使用过 OpenJDK。那么 Oracle JDK 和 OpenJDK 之间是否存在重大差异？下面我通过收集到的一些资料，为你解答这个被很多人忽视的问题。

对于 Java 7，没什么关键的地方。OpenJDK 项目主要基于 Sun 捐赠的 HotSpot 源代码。此外，OpenJDK 被选为 Java 7 的参考实现，由 Oracle 工程师维护。关于 JVM，JDK，JRE 和 OpenJDK 之间的区别，Oracle 博客帖子在 2012 年有一个更详细的答案：

问：OpenJDK 存储库中的源代码与用于构建 Oracle JDK 的代码之间有什么区别？

答：非常接近 - 我们的 Oracle JDK 版本构建过程基于 OpenJDK 7 构建，只添加了几个部分，例如部署代码，其中包括 Oracle 的 Java 插件和 Java WebStart 的实现，以及一些闭源的第三方组件，如图形光栅化器，一些开源的第三方组件，如 Rhino，以及一些零碎的东西，如附加文档或第三方字体。展望未来，我们的目的是开源 Oracle JDK 的所有部分，除了我们考虑商业功能的部分。

总结：（提示：下面括号内的内容是基于原文补充说明的，因为原文太过于晦涩难懂，用人话重新解释了下，如果你看得懂里面的术语，可以忽略括号解释的内容）

1. Oracle JDK 大概每 6 个月发一次主要版本（从 2014 年 3 月 JDK 8 LTS 发布到 2017 年 9 月 JDK 9 发布经历了长达 3 年多的时间，所以并不总是 6 个月），而 OpenJDK 版本大概每三个月发布一次。但这不是固定的，我觉得了解这个没啥用处。详情参见：<https://blogs.oracle.com/java-platform-group/update-and-faq-on-the-java-se-release-cadence>。
2. OpenJDK 是一个参考模型并且是完全开源的，而 Oracle JDK 是 OpenJDK 的一个实现，并不是完全开源的；（个人观点：众所周知，JDK 原来是 SUN 公司开发的，后来 SUN 公司又卖给了 Oracle 公司，Oracle 公司以 Oracle 数据库而著名，而 Oracle 数据库又是闭源的，这个时候 Oracle 公司就不想完全开源了，但是原来的 SUN 公司又把 JDK 给开源了，如果这个时候 Oracle 收购回来之后就把他给闭源，必然会引其很多 Java 开发者的不满，导致大家对 Java 失去信心，那 Oracle 公司收购回来不就把 Java 烂在手里了吗！然后，Oracle 公司就想了个骚操作，这样吧，我把一部分核心代码开源出来给你们玩，并且我要和你们自己搞的 JDK 区分下，你们叫 OpenJDK，我叫 Oracle JDK，我发布我的，你们继续玩你们的，要是你们搞出来什么好玩的东西，我后续发布 Oracle JDK 也会拿来用一下，一举两得！）OpenJDK 开源项目：<https://github.com/openjdk/jdk>
3. Oracle JDK 比 OpenJDK 更稳定（肯定啦，Oracle JDK 由 Oracle 内部团队进行单独研发的，而且发布时间比 OpenJDK 更长，质量更有保障）。OpenJDK 和 Oracle JDK 的代码几乎相同（OpenJDK 的代码是从 Oracle JDK 代码派生出来的，可以理解为在 Oracle JDK 分支上拉了一条新的分支叫 OpenJDK，所以大部分代码相同），但 Oracle JDK 有更多的类和一些错误修复。因此，如果您想开发企业/商业软件，我建议您选择 Oracle JDK，因为它经过了彻底的测试和稳定。某些情况下，有些人提到在使用 OpenJDK 可能会遇到了许多应用程序崩溃的问题，但是，只需切换到 Oracle JDK 就可以解决问题；
4. 在响应性和 JVM 性能方面，Oracle JDK 与 OpenJDK 相比提供了更好的性能；
5. Oracle JDK 不会为即将发布的版本提供长期支持（如果是 LTS 长期支持版本的话也会，比如 JDK 8，但并不是每个版本都是 LTS 版本），用户每次都必须通过更新到最新版本获得支持来获取最新版本；
6. Oracle JDK 使用 BCL/OTN 协议获得许可，而 OpenJDK 根据 GPL v2 许可获得许可。

既然 Oracle JDK 这么好，那为什么还要有 OpenJDK？

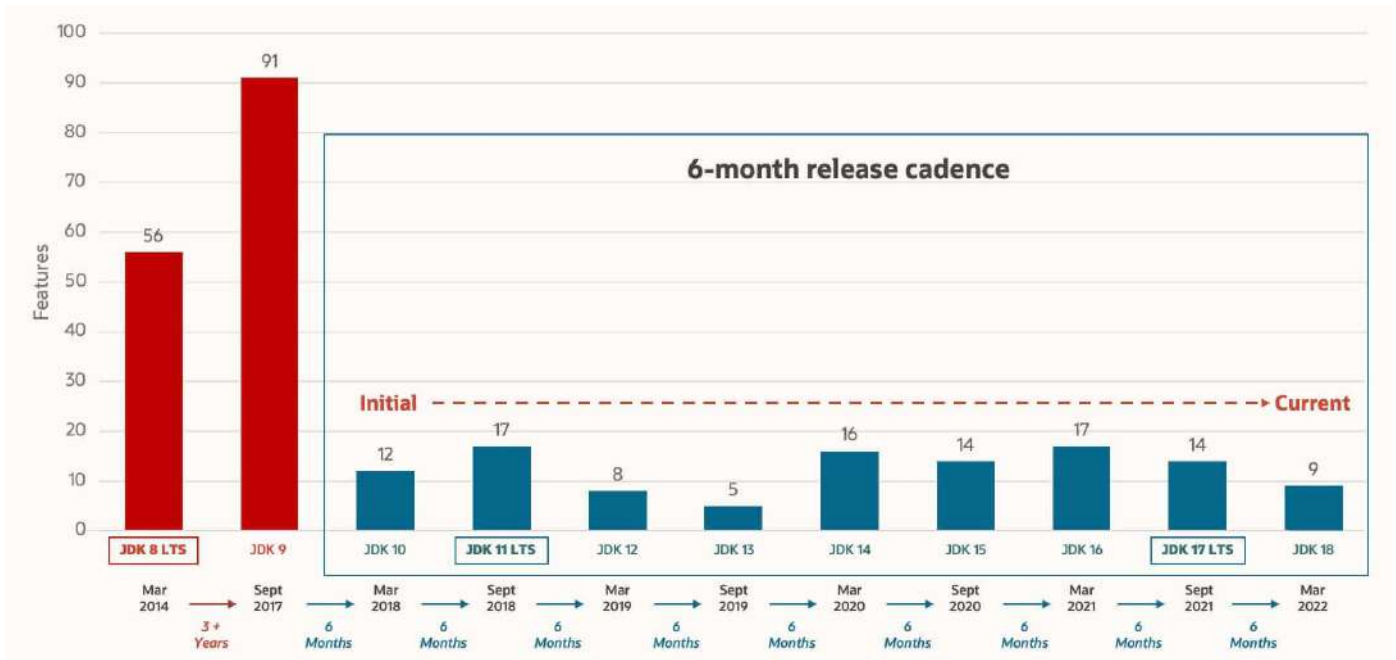
答：

1. OpenJDK 是开源的，开源意味着你可以对它根据你自己的需要进行修改、优化，比如 Alibaba 基于 OpenJDK 开发了 Dragonwell8：<https://github.com/alibaba/dragonwell8>
2. OpenJDK 是商业免费的（这也是为什么通过 yum 包管理器上默认安装的 JDK 是 OpenJDK 而不是 Oracle JDK）。虽然 Oracle JDK 也是商业免费（比如 JDK 8），但并不

是所有版本都是免费的。

3. OpenJDK 更新频率更快。Oracle JDK 一般是每 6 个月发布一个新版本，而 OpenJDK 一般是每 3 个月发布一个新版本。（现在你知道为啥 Oracle JDK 更稳定了吧，先在 OpenJDK 试试水，把大部分问题都解决掉了才在 Oracle JDK 上发布）

基于以上这些原因，OpenJDK 还是有存在的必要的！



拓展一下：

- BCL 协议（Oracle Binary Code License Agreement）：可以使用 JDK（支持商用），但是不能进行修改。
- OTN 协议（Oracle Technology Network License Agreement）：11 及之后新发布的 JDK 用的都是这个协议，可以自己私下用，但是商用需要付费。

Oracle JDK 各个版本所用的协议		
Oracle JDK 版本	BCL协议	OTN协议
6	最后一个公共更新6u45之前	
7	最后一个公共更新7u80之前	
8	8u201/8u202之前	8u211/8u212之后
9	✓	
10	✓	
11		✓
12		✓

相关阅读👍：[《Differences Between Oracle JDK and OpenJDK》](#)

Java 语言关键字有哪些？

分类	关键字						
访问控制	private	protected	public				
类，方法和变量修饰符	abstract	class	extends	final	implements	interface	native
	new	static	strictfp	synchronized	transient	volatile	enum
程序控制	break	continue	return	do	while	if	else
	for	instanceof	switch	case	default	assert	
错误处理	try	catch	throw	throws	finally		
包相关	import	package					
基本类型	boolean	byte	char	double	float	int	long
	short						
变量引用	super	this	void				
保留字	goto	const					

Tips: 所有的关键字都是小写的, 在 IDE 中会以特殊颜色显示。

`default` 这个关键字很特殊, 既属于程序控制, 也属于类, 方法和变量修饰符, 还属于访问控制。

- 在程序控制中, 当在 `switch` 中匹配不到任何情况时, 可以使用 `default` 来编写默认匹配的情况。
- 在类, 方法和变量修饰符中, 从 JDK8 开始引入了默认方法, 可以使用 `default` 关键字来定义一个方法的默认实现。
- 在访问控制中, 如果一个方法前没有任何修饰符, 则默认会有一个修饰符 `default`, 但是这个修饰符加上了就会报错。

⚠ 注意: 虽然 `true`, `false`, 和 `null` 看起来像关键字但实际上他们是字面值, 同时你也不可以作为标识符来使用。

官方文档: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/_keywords.html

自增自减运算符

在写代码的过程中, 常见的一种情况是需要某个整数类型变量增加 1 或减少 1, Java 提供了一种特殊的运算符, 用于这种表达式, 叫做自增运算符 (`++`) 和自减运算符 (`--`)。

`++` 和 `--` 运算符可以放在变量之前, 也可以放在变量之后, 当运算符放在变量之前时(前缀), 先自增/减, 再赋值; 当运算符放在变量之后时(后缀), 先赋值, 再自增/减。例如, 当 `b = ++a` 时, 先自增 (自己增加 1), 再赋值 (赋值给 b); 当 `b = a++` 时, 先赋值 (赋值给 b), 再自增 (自己增加 1)。也就是, `++a` 输出的是 `a+1` 的值, `a++` 输出的是 `a` 值。用一句口诀就是: “符号在前就先加/减, 符号在后就后加/减”。

成员变量与局部变量的区别?

- **语法形式**: 从语法形式上看, 成员变量是属于类的, 而局部变量是在代码块或方法中定义的变量或是方法的参数; 成员变量可以被 `public`, `private`, `static` 等修饰符所修饰, 而局部变量不能被访问控制修饰符及 `static` 所修饰; 但是, 成员变量和局部变量都能被 `final` 所修饰。
- **存储方式**: 从变量在内存中的存储方式来看, 如果成员变量是使用 `static` 修饰的, 那么这个成员变量是属于类的, 如果没有使用 `static` 修饰, 这个成员变量是属于实例的。而对象存在于堆内存, 局部变量则存在于栈内存。
- **生存时间**: 从变量在内存中的生存时间上看, 成员变量是对象的一部分, 它随着对象的创建而存在, 而局部变量随着方法的调用而自动生成, 随着方法的调用结束而消亡。

- **默认值**：从变量是否有默认值来看，成员变量如果没有被赋初始值，则会自动以类型的默认值而赋值（一种情况例外：被 `final` 修饰的成员变量也必须显式地赋值），而局部变量则不会自动赋值。

静态变量有什么作用？

静态变量可以被类的所有实例共享。无论一个类创建了多少个对象，它们都共享同一份静态变量。

通常情况下，静态变量会被 `final` 关键字修饰成为常量。

字符型常量和字符串常量的区别？

1. **形式**：字符常量是单引号引起的一个字符，字符串常量是双引号引起的 0 个或若干个字符。
2. **含义**：字符常量相当于一个整型值(ASCII 值),可以参加表达式运算;字符串常量代表一个地址值(该字符串在内存中存放位置)。
3. **占内存大小**：字符常量只占 2 个字节;字符串常量占若干个字节。

(注意：`char` 在 Java 中占两个字节)

静态方法和实例方法有何不同？

1、调用方式

在外部调用静态方法时，可以使用 `类名.方法名` 的方式，也可以使用 `对象.方法名` 的方式，而实例方法只有后面这种方式。也就是说，**调用静态方法可以无需创建对象**。

不过，需要注意的是一般不建议使用 `对象.方法名` 的方式来调用静态方法。这种方式非常容易造成混淆，静态方法不属于类的某个对象而是属于这个类。

因此，一般建议使用 `类名.方法名` 的方式来调用静态方法。

```
public class Person {  
    public void method() {  
        //.....  
    }  
  
    public static void staicMethod(){  
        //.....  
    }  
}
```

```
public static void main(String[] args) {  
    Person person = new Person();  
    // 调用实例方法  
    person.method();  
    // 调用静态方法  
    Person.staicMethod()  
}  
}
```

2、访问类成员是否存在限制

静态方法在访问本类的成员时，只允许访问静态成员（即静态成员变量和静态方法），不允许访问实例成员（即实例成员变量和实例方法），而实例方法不存在这个限制。

重载和重写有什么区别？

重载就是同样的一个方法能够根据输入数据的不同，做出不同的处理

重写就是当子类继承自父类的相同方法，输入数据一样，但要做出有别于父类的响应时，你就要覆盖父类方法

重载

发生在同一个类中（或者父类和子类之间），方法名必须相同，参数类型不同、个数不同、顺序不同，方法返回值和访问修饰符可以不同。

《Java 核心技术》这本书是这样介绍重载的：

如果多个方法(比如 `StringBuilder` 的构造方法)有相同的名字、不同的参数，便产生了重载。

```
StringBuilder sb = new StringBuilder();  
StringBuilder sb2 = new StringBuilder("HelloWorld");
```

编译器必须挑选出具体执行哪个方法，它通过用各个方法给出的参数类型与特定方法调用所使用的值类型进行匹配来挑选出相应的方法。如果编译器找不到匹配的参数，就会产生编译时错误，因为根本不存在匹配，或者没有一个比其他的更好(这个过程被称为重载解析(overloading resolution))。

Java 允许重载任何方法，而不只是构造器方法。

综上：重载就是同一个类中多个同名方法根据不同的传参来执行不同的逻辑处理。

重写

重写发生在运行期，是子类对父类的允许访问的方法的实现过程进行重新编写。

1. 方法名、参数列表必须相同，子类方法返回值类型应比父类方法返回值类型更小或相等，抛出的异常范围小于等于父类，访问修饰符范围大于等于父类。
2. 如果父类方法访问修饰符为 `private/final/static` 则子类就不能重写该方法，但是被 `static` 修饰的方法能够被再次声明。
3. 构造方法无法被重写

综上：重写就是子类对父类方法的重新改造，外部样子不能改变，内部逻辑可以改变。

区别点	重载方法	重写方法
发生范围	同一个类	子类
参数列表	必须修改	一定不能修改
返回类型	可修改	子类方法返回值类型应比父类方法返回值类型更小或相等
异常	可修改	子类方法声明抛出的异常类应比父类方法声明抛出的异常类更小或相等；
访问修饰符	可修改	一定不能做更严格的限制（可以降低限制）
发生阶段	编译期	运行期

方法的重写要遵循“两同两小一大”（以下内容摘录自《疯狂 Java 讲义》，[issue#892](#)）：

- “两同”即方法名相同、形参列表相同；
- “两小”指的是子类方法返回值类型应比父类方法返回值类型更小或相等，子类方法声明抛出的异

常类应比父类方法声明抛出的异常类更小或相等；

- “一大”指的是子类方法的访问权限应比父类方法的访问权限更大或相等。

★ 关于 **重写的返回值类型** 这里需要额外多说明一下，上面的表述不太清晰准确：如果方法的返回类型是 void 和基本数据类型，则返回值重写时不可修改。但是如果方法的返回值是引用类型，重写时是可以返回该引用类型的子类的。

```
public class Hero {
    public String name() {
        return "超级英雄";
    }
}

public class Superman extends Hero{
    @Override
    public String name() {
        return "超人";
    }

    public Hero hero() {
        return new Hero();
    }
}

public class SuperSuperMan extends Superman {
    public String name() {
        return "超级超级英雄";
    }

    @Override
    public Superman hero() {
        return new Superman();
    }
}
```

什么是可变长参数？

从 Java5 开始，Java 支持定义可变长参数，所谓可变长参数就是允许在调用方法时传入不定长度的参数。就比如下面的这个 `printVariable` 方法就可以接受 0 个或者多个参数。

```
public static void method1(String... args) {  
    //.....  
}
```

另外，可变参数只能作为函数的最后一个参数，但其前面可以有也可以没有任何其他参数。

```
public static void method2(String arg1, String... args) {  
    //.....  
}
```

遇到方法重载的情况怎么办呢？会优先匹配固定参数还是可变参数的方法呢？

答案是会优先匹配固定参数的方法，因为固定参数的方法匹配度更高。

我们通过下面这个例子来证明一下。

```
/**  
 * 微信搜 JavaGuide 回复"面试突击"即可免费领取个人原创的 Java 面试手册  
 *  
 * @author Guide哥  
 * @date 2021/12/13 16:52  
 **/  
public class VariableLengthArgument {  
  
    public static void printVariable(String... args) {  
        for (String s : args) {  
            System.out.println(s);  
        }  
    }  
}
```

```

public static void printVariable(String arg1, String arg2) {
    System.out.println(arg1 + arg2);
}

public static void main(String[] args) {
    printVariable("a", "b");
    printVariable("a", "b", "c", "d");
}
}

```

输出：

```

ab
a
b
c
d

```

另外，Java 的可变参数编译后实际会被转换成一个数组，我们看编译后生成的 `class` 文件就可以看出来了。

```

public class VariableLengthArgument {

    public static void printVariable(String... args) {
        String[] var1 = args;
        int var2 = args.length;

        for(int var3 = 0; var3 < var2; ++var3) {
            String s = var1[var3];
            System.out.println(s);
        }

    }

    // .....
}

```

Java 中的几种基本数据类型了解么？

Java 中有 8 种基本数据类型，分别为：

- 6 种数字类型：
 - 4 种整数型： byte 、 short 、 int 、 long
 - 2 种浮点型： float 、 double
- 1 种字符类型： char
- 1 种布尔型： boolean 。

这 8 种基本数据类型的默认值以及所占空间的大小如下：

基本类型	位数	字节	默认值	取值范围
byte	8	1	0	-128 ~ 127
short	16	2	0	-32768 ~ 32767
int	32	4	0	-2147483648 ~ 2147483647
long	64	8	0L	-9223372036854775808 ~ 9223372036854775807
char	16	2	'u0000'	0 ~ 65535
float	32	4	0f	1.4E-45 ~ 3.4028235E38
double	64	8	0d	4.9E-324 ~ 1.7976931348623157E308
boolean	1		false	true、false

对于 boolean ，官方文档未明确定义，它依赖于 JVM 厂商的具体实现。逻辑上理解是占用 1 位，但是实际中会考虑计算机高效存储因素。

另外，Java 的每种基本类型所占存储空间的大小不会像其他大多数语言那样随机器硬件架构的变化而变化。这种所占存储空间大小的不变性是 Java 程序比用其他大多数语言编写的程序更具可移植性的原因之一（《Java 编程思想》2.2 节有提到）。

注意：

1. Java 里使用 `long` 类型的数据一定要在数值后面加上 `L`，否则将作为整型解析。
2. `char a = 'h'` `char` :单引号， `String a = "hello"` :双引号。

这八种基本类型都有对应的包装类分别

为： `Byte` 、 `Short` 、 `Integer` 、 `Long` 、 `Float` 、 `Double` 、 `Character` 、 `Boolean` 。

基本类型和包装类型的区别？

- 成员变量包装类型不赋值就是 `null` ，而基本类型有默认值且不是 `null` 。
- 包装类型可用于泛型，而基本类型不可以。
- 基本数据类型的局部变量存放在 Java 虚拟机栈中的局部变量表中，基本数据类型的成员变量（未被 `static` 修饰）存放在 Java 虚拟机的堆中。包装类型属于对象类型，我们知道几乎所有对象实例都存在于堆中。
- 相比于对象类型，基本数据类型占用的空间非常小。

为什么说几乎是所有对象实例呢？这是因为 HotSpot 虚拟机引入了 JIT 优化之后，会对对象进行逃逸分析，如果发现某一个对象并没有逃逸到方法外部，那么就可能通过标量替换来实现栈上分配，而避免堆上分配内存

 注意：基本数据类型存放在栈中是一个常见的误区！基本数据类型的成员变量如果没有被 `static` 修饰的话（不建议这么使用，应该要使用基本数据类型对应的包装类型），就存放在堆中。

```
class BasicTypeVar{  
    private int x;  
}
```

包装类型的缓存机制了解么？

Java 基本数据类型的包装类型的大部分都用到了缓存机制来提升性能。

`Byte` ， `Short` ， `Integer` ， `Long` 这 4 种包装类默认创建了数值 `[-128, 127]` 的相应类型的缓存数据， `Character` 创建了数值在 `[0,127]` 范围的缓存数据， `Boolean` 直接返回 `True` or `False` 。

Integer 缓存源码：

```

public static Integer valueOf(int i) {
    if (i >= IntegerCache.low && i <= IntegerCache.high)
        return IntegerCache.cache[i + (-IntegerCache.low)];
    return new Integer(i);
}

private static class IntegerCache {
    static final int low = -128;
    static final int high;
    static {
        // high value may be configured by property
        int h = 127;
    }
}

```

Character 缓存源码:

```

public static Character valueOf(char c) {
    if (c <= 127) { // must cache
        return CharacterCache.cache[(int)c];
    }
    return new Character(c);
}

private static class CharacterCache {
    private CharacterCache(){}
    static final Character cache[] = new Character[127 + 1];
    static {
        for (int i = 0; i < cache.length; i++)
            cache[i] = new Character((char)i);
    }
}

```

Boolean 缓存源码:


```
public static Boolean valueOf(boolean b) {  
    return (b ? TRUE : FALSE);  
}
```

如果超出对应范围仍然会去创建新的对象，缓存的范围区间的大小只是在性能和资源之间的权衡。

两种浮点数类型的包装类 `Float`，`Double` 并没有实现缓存机制。

```
Integer i1 = 33;  
Integer i2 = 33;  
System.out.println(i1 == i2); // 输出 true  
  
Float i11 = 333f;  
Float i22 = 333f;  
System.out.println(i11 == i22); // 输出 false  
  
Double i3 = 1.2;  
Double i4 = 1.2;  
System.out.println(i3 == i4); // 输出 false
```

下面我们来看一下问题。下面的代码的输出结果是 `true` 还是 `false` 呢？

```
Integer i1 = 40;  
Integer i2 = new Integer(40);  
System.out.println(i1==i2);
```

`Integer i1=40` 这一行代码会发生装箱，也就是说这行代码等价于 `Integer i1=Integer.valueOf(40)`。因此，`i1` 直接使用缓存中的对象。而 `Integer i2 = new Integer(40)` 会直接创建新的对象。

因此，答案是 `false`。你答对了吗？

记住：所有整型包装类对象之间值的比较，全部使用 `equals` 方法比较。

7. **【强制】**所有整型包装类对象之间值的比较，全部使用 equals 方法比较。

说明：对于 Integer var = ? 在-128 至 127 之间的赋值，Integer 对象是在 IntegerCache.cache 产生，会复用已有对象，这个区间内的 Integer 值可以直接使用 == 进行判断，但是这个区间之外的所有数据，都会在堆上产生，并不会复用已有对象，这是一个大坑，推荐使用 equals 方法进行判断。

自动装箱与拆箱了解吗？原理是什么？

什么是自动拆装箱？

- 装箱：将基本类型用它们对应的引用类型包装起来；
- 拆箱：将包装类型转换为基本数据类型；

举例：

```
Integer i = 10;    //装箱
int n = i;         //拆箱
```

上面这两行代码对应的字节码为：

```
L1

LINENUMBER 8 L1

ALOAD 0

BIPUSH 10

INVOKESTATIC java/lang/Integer.valueOf (I)Ljava/lang/Integer;

PUTFIELD AutoBoxTest.i : Ljava/lang/Integer;

L2

LINENUMBER 9 L2
```

```

ALOAD 0

ALOAD 0

GETFIELD AutoBoxTest.i : Ljava/lang/Integer;

INVOKEVIRTUAL java/lang/Integer.intValue ()I

PUTFIELD AutoBoxTest.n : I

RETURN

```

从字节码中，我们发现装箱其实就是调用了包装类的 `valueOf()` 方法，拆箱其实就是调用了 `xxxValue()` 方法。

因此，

- `Integer i = 10` 等价于 `Integer i = Integer.valueOf(10)`
- `int n = i` 等价于 `int n = i.intValue()` ；

注意：如果频繁拆装箱的话，也会严重影响系统的性能。我们应该尽量避免不必要的拆装箱操作。

```

private static long sum() {
    // 应该使用 long 而不是 Long
    Long sum = 0L;
    for (long i = 0; i <= Integer.MAX_VALUE; i++)
        sum += i;
    return sum;
}

```

为什么浮点数运算的时候会有精度丢失的风险？

浮点数运算精度丢失代码演示：

```
float a = 2.0f - 1.9f;
float b = 1.8f - 1.7f;

System.out.println(a); // 0.100000024
System.out.println(b); // 0.099999905
System.out.println(a == b); // false
```

为什么会出现这个问题呢？

这个和计算机保存浮点数的机制有很大关系。我们知道计算机是二进制的，而且计算机在表示一个数字时，宽度是有限的，无限循环的小数存储在计算机时，只能被截断，所以就会导致小数精度发生损失的情况。这也就是解释了为什么浮点数没有办法用二进制精确表示。

就比如说十进制下的 0.2 就没办法精确转换成二进制小数：

```
// 0.2 转换为二进制数的过程为，不断乘以 2，直到不存在小数为止，
// 在这个计算过程中，得到的整数部分从上到下排列就是二进制的结果。

0.2 * 2 = 0.4 -> 0
0.4 * 2 = 0.8 -> 0
0.8 * 2 = 1.6 -> 1
0.6 * 2 = 1.2 -> 1
0.2 * 2 = 0.4 -> 0 (发生循环)
...
```

关于浮点数的更多内容，建议看一下[计算机系统基础（四）浮点数](#)这篇文章。

如何解决浮点数运算的精度丢失问题？

`BigDecimal` 可以实现对浮点数的运算，不会造成精度丢失。通常情况下，大部分需要浮点数精确运算结果的业务场景（比如涉及到钱的场景）都是通过 `BigDecimal` 来做的。

```
BigDecimal a = new BigDecimal("1.0");
BigDecimal b = new BigDecimal("0.9");
BigDecimal c = new BigDecimal("0.8");

BigDecimal x = a.subtract(b);
BigDecimal y = b.subtract(c);

System.out.println(x); /* 0.1 */
System.out.println(y); /* 0.1 */
System.out.println(Objects.equals(x, y)); /* true */
```

关于 `BigDecimal` 的详细介绍，可以看看我写的这篇文章：[BigDecimal 详解](#)。

超过 long 整型的数据应该如何表示？

基本数值类型都有一个表达范围，如果超过这个范围就会有数值溢出的风险。

在 Java 中，64 位 long 整型是最大的整数类型。

```
long l = Long.MAX_VALUE;
System.out.println(l + 1); // -9223372036854775808
System.out.println(l + 1 == Long.MIN_VALUE); // true
```

`BigInteger` 内部使用 `int[]` 数组来存储任意大小的整形数据。

相对于常规整数类型的运算来说，`BigInteger` 运算的效率会相对较低。

如果一个类没有声明构造方法，该程序能正确执行吗？

如果一个类没有声明构造方法，也可以执行！因为一个类即使没有声明构造方法也会有默认的不带参数的构造方法。如果我们自己添加了类的构造方法（无论是否有参），Java 就不会再添加默认的无参数的构造方法了，我们一直在不知不觉地使用构造方法，这也是为什么我们在创建对象的时候要加一个括号（因为要调用无参的构造方法）。如果我们重载了有参的构造方法，记得都要把无参的构造方法也写出来（无论是否用到），因为这可以帮助我们在创建对象的时候少踩坑。

构造方法有哪些特点？是否可被 override？

构造方法特点如下：

- 名字与类名相同。
- 没有返回值，但不能用 void 声明构造函数。
- 生成类的对象时自动执行，无需调用。

构造方法不能被 override（重写），但是可以 overload（重载），所以你可以看到一个类中有多个构造函数的情况。

面向对象三大特征

封装

封装是指把一个对象的状态信息（也就是属性）隐藏在对象内部，不允许外部对象直接访问对象的内部信息。但是可以提供一些可以被外界访问的方法来操作属性。就好像我们看不到挂在墙上的空调的内部的零件信息（也就是属性），但是可以通过遥控器（方法）来控制空调。如果属性不想被外界访问，我们大可不必提供方法给外界访问。但是如果一个类没有提供给外界访问的方法，那么这个类也没有什么意义了。就好像如果没有空调遥控器，那么我们就无法操控空调制冷，空调本身就没有意义了（当然现在还有很多其他方法，这里只是为了举例子）。

```
public class Student {  
    private int id;//id属性私有化  
    private String name;//name属性私有化  
  
    //获取id的方法  
    public int getId() {  
        return id;  
    }  
  
    //设置id的方法  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    //获取name的方法  
    public String getName() {  
        return name;  
    }  
}
```

```
    }

    //设置name的方法
    public void setName(String name) {
        this.name = name;
    }
}
```

继承

不同类型的对象，相互之间经常有一定数量的共同点。例如，小明同学、小红同学、小李同学，都共享学生的特性（班级、学号等）。同时，每一个对象还定义了额外的特性使得他们与众不同。例如小明的数学比较好，小红的性格惹人喜爱；小李的力气比较大。继承是使用已存在的类的定义作为基础建立新类的技术，新类的定义可以增加新的数据或新的功能，也可以用父类的功能，但不能选择性地继承父类。通过使用继承，可以快速地创建新的类，可以提高代码的重用，程序的可维护性，节省大量创建新类的时间，提高我们的开发效率。

关于继承如下 3 点请记住：

1. 子类拥有父类对象所有的属性和方法（包括私有属性和私有方法），但是父类中的私有属性和方法子类是无法访问，**只是拥有**。
2. 子类可以拥有自己属性和方法，即子类可以对父类进行扩展。
3. 子类可以用自己的方式实现父类的方法。（以后介绍）。

多态

多态，顾名思义，表示一个对象具有多种的状态，具体表现为父类的引用指向子类的实例。

多态的特点：

- 对象类型和引用类型之间具有继承（类）/实现（接口）的关系；
- 引用类型变量发出的方法调用的到底是哪个类中的方法，必须在程序运行期间才能确定；
- 多态不能调用“只在子类存在但在父类不存在”的方法；
- 如果子类重写了父类的方法，真正执行的是子类覆盖的方法，如果子类没有覆盖父类的方法，执行的是父类的方法。

接口和抽象类有什么共同点和区别？

共同点：

- 都不能被实例化。
- 都可以包含抽象方法。
- 都可以有默认实现的方法（Java 8 可以用 `default` 关键字在接口中定义默认方法）。

区别：

- 接口主要用于对类的行为进行约束，你实现了某个接口就具有了对应的行为。抽象类主要用于代码复用，强调的是所属关系。
- 一个类只能继承一个类，但是可以实现多个接口。
- 接口中的成员变量只能是 `public static final` 类型的，不能被修改且必须有初始值，而抽象类的成员变量默认 `default`，可在子类中被重新定义，也可被重新赋值。

深拷贝和浅拷贝区别了解吗？什么是引用拷贝？

关于深拷贝和浅拷贝区别，我这里先给结论：

- **浅拷贝**：浅拷贝会在堆上创建一个新的对象（区别于引用拷贝的一点），不过，如果原对象内部的属性是引用类型的话，浅拷贝会直接复制内部对象的引用地址，也就是说拷贝对象和原对象共用同一个内部对象。
- **深拷贝**：深拷贝会完全复制整个对象，包括这个对象所包含的内部对象。

上面的结论没有完全理解的话也没关系，我们来看一个具体的案例！

浅拷贝

浅拷贝的示例代码如下，我们这里实现了 `Cloneable` 接口，并重写了 `clone()` 方法。

`clone()` 方法的实现很简单，直接调用的是父类 `Object` 的 `clone()` 方法。

```
public class Address implements Cloneable{
    private String name;
    // 省略构造函数、Getter&Setter方法
    @Override
    public Address clone() {
        try {
```

```

        return (Address) super.clone();
    } catch (CloneNotSupportedException e) {
        throw new AssertionError();
    }
}

public class Person implements Cloneable {
    private Address address;
    // 省略构造函数、Getter&Setter方法
    @Override
    public Person clone() {
        try {
            Person person = (Person) super.clone();
            return person;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }
}

```

测试：

```

Person person1 = new Person(new Address("武汉"));
Person person1Copy = person1.clone();
// true
System.out.println(person1.getAddress() == person1Copy.getAddress());

```

从输出结构就可以看出，`person1` 的克隆对象和 `person1` 使用的仍然是同一个 `Address` 对象。

深拷贝

这里我们简单对 `Person` 类的 `clone()` 方法进行修改，连带着要把 `Person` 对象内部的 `Address` 对象一起复制。

```

@Override
public Person clone() {
    try {
        Person person = (Person) super.clone();
        person.setAddress(person.getAddress().clone());
        return person;
    } catch (CloneNotSupportedException e) {
        throw new AssertionError();
    }
}

```

测试：

```

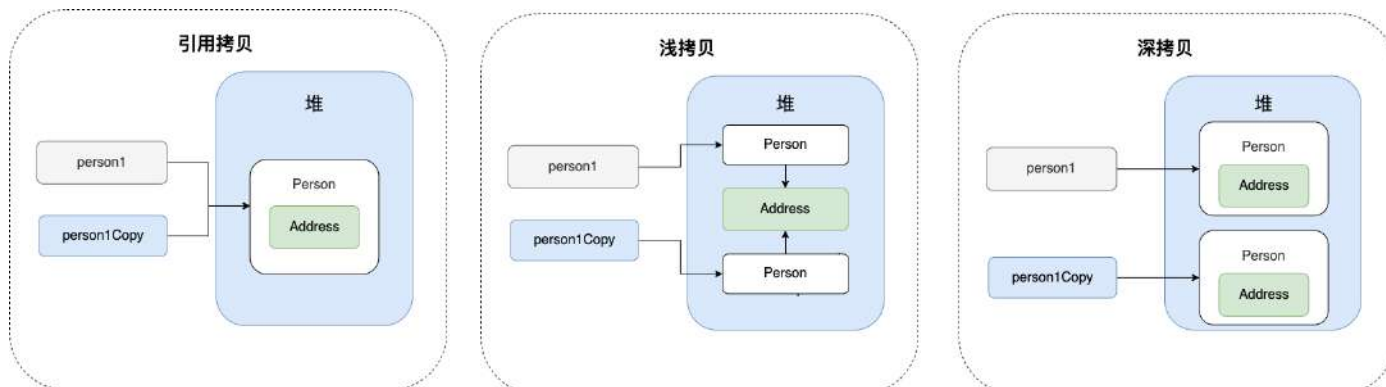
Person person1 = new Person(new Address("武汉"));
Person person1Copy = person1.clone();
// false
System.out.println(person1.getAddress() == person1Copy.getAddress());

```

从输出结构就可以看出，虽然 `person1` 的克隆对象和 `person1` 包含的 `Address` 对象已经是不同的了。

那什么是引用拷贝呢？简单来说，引用拷贝就是两个不同的引用指向同一个对象。

我专门画了一张图来描述浅拷贝、深拷贝、引用拷贝：



Object

Object 类的常见方法有哪些？

Object 类是一个特殊的类，是所有类的父类。它主要提供了以下 11 个方法：

```
/**
 * native 方法，用于返回当前运行时对象的 Class 对象，使用了 final 关键字修饰，故不允许子类
重写。
 */
public final native Class<?> getClass()

/**
 * native 方法，用于返回对象的哈希码，主要使用在哈希表中，比如 JDK 中的HashMap。
 */
public native int hashCode()

/**
 * 用于比较 2 个对象的内存地址是否相等，String 类对该方法进行了重写以用于比较字符串的值是否相
等。
 */
public boolean equals(Object obj)

/**
 * native 方法，用于创建并返回当前对象的一份拷贝。
 */
protected native Object clone() throws CloneNotSupportedException

/**
 * 返回类的名字实例的哈希码的 16 进制的字符串。建议 Object 所有的子类都重写这个方法。
 */
public String toString()

/**
 * native 方法，并且不能重写。唤醒一个在此对象监视器上等待的线程(监视器相当于就是锁的概念)。
如果有多个线程在等待只会任意唤醒一个。
 */
public final native void notify()

/**
 * native 方法，并且不能重写。跟 notify 一样，唯一的区别就是会唤醒在此对象监视器上等待的所有
线程，而不是一个线程。
 */
public final native void notifyAll()

/**
```

```

    * native方法，并且不能重写。暂停线程的执行。注意：sleep 方法没有释放锁，而 wait 方法释放了
    锁，timeout 是等待时间。
    */

    public final native void wait(long timeout) throws InterruptedException

    /**
     * 多了 nanos 参数，这个参数表示额外时间（以毫微秒为单位，范围是 0-999999）。所以超时的时间
     还需要加上 nanos 毫秒。。
     */

    public final void wait(long timeout, int nanos) throws InterruptedException

    /**
     * 跟之前的2个wait方法一样，只不过该方法一直等待，没有超时时间这个概念
     */

    public final void wait() throws InterruptedException

    /**
     * 实例被垃圾回收器回收的时候触发的操作
     */

    protected void finalize() throws Throwable { }

```

== 和 equals() 的区别

== 对于基本类型和引用类型的作用效果是不同的：

- 对于基本数据类型来说，== 比较的是值。
- 对于引用数据类型来说，== 比较的是对象的内存地址。

因为 Java 只有值传递，所以，对于 == 来说，不管是比较基本数据类型，还是引用数据类型的变量，其本质比较的都是值，只是引用类型变量存的值是对象的地址。

equals() 不能用于判断基本数据类型的变量，只能用来判断两个对象是否相等。equals() 方法存在于 Object 类中，而 Object 类是所有类的直接或间接父类，因此所有的类都有 equals() 方法。

Object 类 equals() 方法：

```

public boolean equals(Object obj) {
    return (this == obj);
}

```

`equals()` 方法存在两种使用情况：

- **类没有重写 `equals()` 方法**：通过 `equals()` 比较该类的两个对象时，等价于通过“`==`”比较这两个对象，使用的默认是 `Object` 类 `equals()` 方法。
- **类重写了 `equals()` 方法**：一般我们都重写 `equals()` 方法来比较两个对象中的属性是否相等；若它们的属性相等，则返回 `true`(即，认为这两个对象相等)。

举个例子（这里只是为了举例。实际上，你按照下面这种写法的话，像 IDEA 这种比较智能的 IDE 都会提示你将 `==` 换成 `equals()`）：

```
String a = new String("ab"); // a 为一个引用
String b = new String("ab"); // b为另一个引用,对象的内容一样
String aa = "ab"; // 放在常量池中
String bb = "ab"; // 从常量池中查找
System.out.println(aa == bb); // true
System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
System.out.println(42 == 42.0); // true
```

`String` 中的 `equals` 方法是被重写过的，因为 `Object` 的 `equals` 方法是比较的对象的内存地址，而 `String` 的 `equals` 方法比较的是对象的值。

当创建 `String` 类型的对象时，虚拟机会在常量池中查找有没有已经存在的值和要创建的值相同的对象，如果有就把它赋给当前引用。如果没有就在常量池中重新创建一个 `String` 对象。

`String` 类 `equals()` 方法：

```
public boolean equals(Object anObject) {
    if (this == anObject) {
        return true;
    }
    if (anObject instanceof String) {
        String anotherString = (String)anObject;
        int n = value.length;
        if (n == anotherString.value.length) {
            char v1[] = value;
```

```

        char v2[] = anotherString.value;

        int i = 0;

        while (n-- != 0) {
            if (v1[i] != v2[i])
                return false;

            i++;
        }

        return true;
    }

    }

    return false;
}

```

hashCode() 有什么用？

`hashCode()` 的作用是获取哈希码（`int` 整数），也称为散列码。这个哈希码的作用是确定该对象在哈希表中的索引位置。

`hashCode()` 定义在 JDK 的 `Object` 类中，这就意味着 Java 中的任何类都包含有 `hashCode()` 函数。另外需要注意的是：`Object` 的 `hashCode()` 方法是本地方法，也就是用 C 语言或 C++ 实现的，该方法通常用来将对象的内存地址转换为整数之后返回。

```
public native int hashCode();
```

散列表存储的是键值对(key-value)，它的特点是：能根据“键”快速的检索出对应的“值”。这其中就利用到了散列码！（可以快速找到所需要的对象）

为什么要有 hashCode？

我们以“`HashSet` 如何检查重复”为例子来说明为什么要有 `hashCode` ？

下面这段内容摘自我的 Java 启蒙书《Head First Java》：

当你把对象加入 `HashSet` 时，`HashSet` 会先计算对象的 `hashCode` 值来判断对象加入的位置，同时也会与其他已经加入的对象的 `hashCode` 值作比较，如果没有相符的 `hashCode`，`HashSet` 会假设对象没有重复出现。但是如果发现有相同 `hashCode` 值的对象，这时会调用 `equals()` 方法来检查 `hashCode` 相等的对象是否真的相同。如果两者相

同，`HashSet` 就不会让其加入操作成功。如果不同的话，就会重新散列到其他位置。这样我们就大大减少了 `equals` 的次数，相应就大大提高了执行速度。

其实，`hashCode()` 和 `equals()` 都是用于比较两个对象是否相等。

那为什么 JDK 还要同时提供这两个方法呢？

这是因为在一些容器（比如 `HashMap`、`HashSet`）中，有了 `hashCode()` 之后，判断元素是否在对应容器中的效率会更高（参考添加元素进 `HashSet` 的过程）！

我们在前面也提到了添加元素进 `HashSet` 的过程，如果 `HashSet` 在对比的时候，同样的 `hashCode` 有多个对象，它会继续使用 `equals()` 来判断是否真的相同。也就是说 `hashCode` 帮助我们大大缩小了查找成本。

那为什么不只提供 `hashCode()` 方法呢？

这是因为两个对象的 `hashCode` 值相等并不代表两个对象就相等。

那为什么两个对象有相同的 `hashCode` 值，它们也不一定是相等的？

因为 `hashCode()` 所使用的哈希算法也许刚好会让多个对象传回相同的哈希值。越糟糕的哈希算法越容易碰撞，但这也与数据值域分布的特性有关（所谓哈希碰撞也就是指的是不同的对象得到相同的 `hashCode`）。

总结下来就是：

- 如果两个对象的 `hashCode` 值相等，那这两个对象不一定相等（哈希碰撞）。
- 如果两个对象的 `hashCode` 值相等并且 `equals()` 方法也返回 `true`，我们才认为这两个对象相等。
- 如果两个对象的 `hashCode` 值不相等，我们就可以直接认为这两个对象不相等。

相信大家看了我前面对 `hashCode()` 和 `equals()` 的介绍之后，下面这个问题已经难不倒你们了。

为什么重写 `equals()` 时必须重写 `hashCode()` 方法？

因为两个相等的对象的 `hashCode` 值必须是相等。也就是说如果 `equals` 方法判断两个对象是相等的，那这两个对象的 `hashCode` 值也要相等。

如果重写 `equals()` 时没有重写 `hashCode()` 方法的话就可能会导致 `equals` 方法判断是相等的两个对象，`hashCode` 值却不相等。

思考：重写 `equals()` 时没有重写 `hashCode()` 方法的话，使用 `HashMap` 可能会出现什么问题。

总结：

- `equals` 方法判断两个对象是相等的，那这两个对象的 `hashCode` 值也要相等。
- 两个对象有相同的 `hashCode` 值，他们也不一定是相等的（哈希碰撞）。

更多关于 `hashCode()` 和 `equals()` 的内容可以查看：[Java hashCode\(\) 和 equals\(\)的若干问题解答](#)

String

String、StringBuffer、StringBuilder 的区别？

可变性

`String` 是不可变的（后面会详细分析原因）。

`StringBuilder` 与 `StringBuffer` 都继承自 `AbstractStringBuilder` 类，在 `AbstractStringBuilder` 中也是使用字符数组保存字符串，不过没有使用 `final` 和 `private` 关键字修饰，最关键的是这个 `AbstractStringBuilder` 类还提供了很多修改字符串的方法比如 `append` 方法。

```
abstract class AbstractStringBuilder implements Appendable, CharSequence {
    char[] value;

    public AbstractStringBuilder append(String str) {
        if (str == null)
            return appendNull();

        int len = str.length();
        ensureCapacityInternal(count + len);
        str.getChars(0, len, value, count);
        count += len;
        return this;
    }

    //...
}
```

线程安全性

`String` 中的对象是不可变的，也就可以理解为常量，线程安全。`AbstractStringBuilder` 是 `StringBuilder` 与 `StringBuffer` 的公共父类，定义了一些字符串的基本操作，如 `expandCapacity`、`append`、`insert`、`indexOf` 等公共方法。`StringBuffer` 对方法加了同步锁或者对调用的方法加了同步锁，所以是线程安全的。`StringBuilder` 并没有对方法进行加同步锁，所以是非线程安全的。

性能

每次对 `String` 类型进行改变的时候，都会生成一个新的 `String` 对象，然后将指针指向新的 `String` 对象。`StringBuffer` 每次都会对 `StringBuffer` 对象本身进行操作，而不是生成新的对象并改变对象引用。相同情况下使用 `StringBuilder` 相比使用 `StringBuffer` 仅能获得 10%~15% 左右的性能提升，但却要冒多线程不安全的风险。

对于三者使用的总结：

1. 操作少量的数据: 适用 `String`
2. 单线程操作字符串缓冲区下操作大量数据: 适用 `StringBuilder`
3. 多线程操作字符串缓冲区下操作大量数据: 适用 `StringBuffer`

String 为什么是不可变的？

`String` 类中使用 `final` 关键字修饰字符数组来保存字符串，所以 `String` 对象是不可变的。

```
public final class String implements java.io.Serializable, Comparable<String>,
CharSequence {
    private final char value[];
    //...
}
```



修正：我们知道被 `final` 关键字修饰的类不能被继承，修饰的方法不能被重写，修饰的变量是基本数据类型则值不能改变，修饰的变量是引用类型则不能再指向其他对象。因此，`final` 关键字修饰的数组保存字符串并不是 `String` 不可变的根本原因，因为这个数组保存的字符串是可变的（`final` 修饰引用类型变量的情况）。

`String` 真正不可变有下面几点原因：

1. 保存字符串的数组被 `final` 修饰且为私有的，并且 `String` 类没有提供/暴露修改这个字符串的方法。
2. `String` 类被 `final` 修饰导致其不能被继承，进而避免了子类破坏 `String` 不可变。

相关阅读：[如何理解 String 类型值的不可变？ - 知乎提问](#)

补充（来自[issue 675](#)）：在 Java 9 之后，`String`、`StringBuilder` 与 `StringBuffer` 的实现改用 `byte` 数组存储字符串。

```
public final class String implements
    java.io.Serializable, Comparable<String>, CharSequence {
    // @Stable 注解表示变量最多被修改一次，称为“稳定的”。
    @Stable
    private final byte[] value;
}

abstract class AbstractStringBuilder implements Appendable, CharSequence {
    byte[] value;
}
```

Java 9 为何要将 `String` 的底层实现由 `char[]` 改成了 `byte[]` ？

新版的 `String` 其实支持两个编码方案：`Latin-1` 和 `UTF-16`。如果字符串中包含的汉字没有超过 `Latin-1` 可表示范围内的字符，那就会使用 `Latin-1` 作为编码方案。`Latin-1` 编码方案下，`byte` 占一个字节(8 位)，`char` 占用 2 个字节（16），`byte` 相较 `char` 节省一半的内存空间。

JDK 官方就说了绝大部分字符串对象只包含 `Latin-1` 可表示的字符。

Motivation

The current implementation of the `String` class stores characters in a `char` array, using two bytes (sixteen bits) for each character. Data gathered from many different applications indicates that strings are a major component of heap usage and, moreover, that most `String` objects contain only `Latin-1` characters. Such characters require only one byte of storage, hence half of the space in the internal `char` arrays of such `String` objects is going unused.

如果字符串中包含的汉字超过 Latin-1 可表示范围内的字符，byte 和 char 所占用的空间是一样的。

这是官方的介绍：<https://openjdk.java.net/jeps/254>。

字符串拼接用“+” 还是 StringBuilder?

Java 语言本身并不支持运算符重载，“+”和“+=”是专门为 String 类重载过的运算符，也是 Java 中仅有的两个重载过的运算符。

```
String str1 = "he";
String str2 = "llo";
String str3 = "world";
String str4 = str1 + str2 + str3;
```

上面的代码对应的字节码如下：

```
1  0 ldc #2 <he>
2  2 astore_1
3  3 ldc #3 <llo>
4  5 astore_2
5  6 ldc #4 <world>
6  8 astore_3
7  9 new #5 <java/lang/StringBuilder>
8 12 dup
9 13 invokespecial #6 <java/lang/StringBuilder.<init> : ()V>
10 16 aload_1
11 17 invokevirtual #7 <java/lang/StringBuilder.append : (Ljava/lang/String;)Ljava/lang/StringBuilder;>
12 20 aload_2
13 21 invokevirtual #7 <java/lang/StringBuilder.append : (Ljava/lang/String;)Ljava/lang/StringBuilder;>
14 24 aload_3
15 25 invokevirtual #7 <java/lang/StringBuilder.append : (Ljava/lang/String;)Ljava/lang/StringBuilder;>
16 28 invokevirtual #8 <java/lang/StringBuilder.toString : ()Ljava/lang/String;>
17 31 astore 4
18 33 return
```

创建 StringBuilder

调用 StringBuilder 的 append 方法

调用 StringBuilder 的 toString 方法

可以看出，字符串对象通过“+”的字符串拼接方式，实际上是通过 `StringBuilder` 调用 `append()` 方法实现的，拼接完成之后调用 `toString()` 得到一个 `String` 对象。

不过，在循环内使用“+”进行字符串的拼接的话，存在比较明显的缺陷：编译器不会创建单个 `StringBuilder` 以复用，会导致创建过多的 `StringBuilder` 对象。

```
String[] arr = {"he", "llo", "world"};
String s = "";
for (int i = 0; i < arr.length; i++) {
    s += arr[i];
}
System.out.println(s);
```

StringBuilder 对象是在循环内部被创建的，这意味着每循环一次就会创建一个 StringBuilder 对象。

```
20 25 aload_0
21 26 arraylength
22 27 istore 4
23 29 iconst_0
24 30 istore 5
25 32 iload 5
26 34 iload 4
27 36 if_icmpge 71 (+35)
28 39 aload_3
29 40 iload 5
30 42 aaload
31 43 astore 6
32 45 new #7 <java/lang/StringBuilder>
33 48 dup
34 49 invokespecial #8 <java/lang/StringBuilder.<init> : ()V>
35 52 aload_2
36 53 invokevirtual #9 <java/lang/StringBuilder.append : (Ljava/lang/String;)Ljava/lang/StringBuilder;>
37 56 aload 6
38 58 invokevirtual #9 <java/lang/StringBuilder.append : (Ljava/lang/String;)Ljava/lang/StringBuilder;>
39 61 invokevirtual #10 <java/lang/StringBuilder.toString : ()Ljava/lang/String;>
40 64 astore_2
41 65 iinc 5 by 1
42 68 goto 32 (-36)
43 71 getstatic #11 <java/lang/System.out : Ljava/io/PrintStream;>
44 74 aload_2
45 75 invokevirtual #12 <java/io/PrintStream.println : (Ljava/lang/String;)V>
46 78 return
```

循环内部创建 StringBuilder 对象



如果直接使用 StringBuilder 对象进行字符串拼接的话，就不会存在这个问题了。

```
String[] arr = {"he", "llo", "world"};
StringBuilder s = new StringBuilder();
for (String value : arr) {
    s.append(value);
}
System.out.println(s);
```



```

16 20 new #6 <java/lang/StringBuilder>
17 23 dup
18 24 invokespecial #7 <java/lang/StringBuilder.<init> : ()V>
19 27 astore_2
20 28 aload_1
21 29 astore_3
22 30 aload_3
23 31 arraylength
24 32 istore 4
25 34 iconst_0
26 35 istore 5
27 37 iload 5
28 39 iload 4
29 41 if_icmpge 63 (+22)
30 44 aload_3
31 45 iload 5
32 47 aaload
33 48 astore 6
34 50 aload_2
35 51 aload 6
36 53 invokevirtual #8 <java/lang/StringBuilder.append : (Ljava/lang/String;)Ljava/lang/StringBuilder;>
37 56 pop
38 57 iinc 5 by 1
39 60 goto 37 (-23)

```

如果你使用 IDEA 的话，IDEA 自带的代码检查机制也会提示你修改代码。

String#equals() 和 Object#equals() 有何区别？

String 中的 equals 方法是被重写过的，比较的是 String 字符串的值是否相等。Object 的 equals 方法是比较的对象的内存地址。

字符串常量池的作用了解吗？

字符串常量池 是 JVM 为了提升性能和减少内存消耗针对字符串（String 类）专门开辟的一块区域，主要目的是为了减少字符串的重复创建。

```

// 在堆中创建字符串对象"ab"
// 将字符串对象"ab"的引用保存在字符串常量池中
String aa = "ab";
// 直接返回字符串常量池中字符串对象"ab"的引用
String bb = "ab";

System.out.println(aa==bb); // true

```

更多关于字符串常量池的介绍可以看一下 [Java 内存区域详解](#) 这篇文章。

String s1 = new String("abc");这句话创建了几个字符串对象？

会创建 1 或 2 个字符串对象。

1、如果字符串常量池中不存在字符串对象“abc”的引用，那么会在堆中创建 2 个字符串对象“abc”。

示例代码（JDK 1.8）：

```
String s1 = new String("abc");
```

对应的字节码：

```
1 0 new #2 <java/lang/String>
2 3 dup
3 4 ldc #3 <abc>
4 6 invokespecial #4 <java/lang/String.<init> : (Ljava/lang/String;)V
5 9 astore_1
6 10 return
```

→ 堆中创建一个 String 对象，此时未被初始化

→ 堆中创建字符串对象“abc”并在字符串常量池中保存对应的引用

↓
调用构造方法对第 0 步创建的 String 对象赋值

ldc 命令用于判断字符串常量池中是否保存了对应的字符串对象的引用，如果保存了的话直接返回，如果没有保存的话，会在堆中创建对应的字符串对象并将该字符串对象的引用保存到字符串常量池中。

2、如果字符串常量池中已存在字符串对象“abc”的引用，则只会在堆中创建 1 个字符串对象“abc”。

示例代码（JDK 1.8）：

```
// 字符串常量池中已存在字符串对象“abc”的引用
String s1 = "abc";
// 下面这段代码只会在堆中创建 1 个字符串对象“abc”
String s2 = new String("abc");
```

对应的字节码：

```

1  0 ldc #2 <abc>
2  2 astore_1
3  3 new #3 <java/lang/String>
4  6 dup
5  7 ldc #2 <abc>
6  9 invokespecial #4 <java/lang/String.<init> : (Ljava/lang/String;)V>
7 12 astore_2
8 13 return

```

这里就不对上面的字节码进行详细注释了，7 这个位置的 `ldc` 命令不会在堆中创建新的字符串对象“abc”，这是因为 0 这个位置已经执行了一次 `ldc` 命令，已经在堆中创建过一次字符串对象“abc”了。7 这个位置执行 `ldc` 命令会直接返回字符串常量池中字符串对象“abc”对应的引用。

intern 方法有什么作用？

`String.intern()` 是一个 native（本地）方法，其作用是将指定的字符串对象的引用保存在字符串常量池中，可以简单分为两种情况：

- 如果字符串常量池中保存了对应的字符串对象的引用，就直接返回该引用。
- 如果字符串常量池中没有保存了对应的字符串对象的引用，那就在常量池中创建一个指向该字符串对象的引用并返回。

示例代码（JDK 1.8）：

```

// 在堆中创建字符串对象"Java"
// 将字符串对象"Java"的引用保存在字符串常量池中
String s1 = "Java";
// 直接返回字符串常量池中字符串对象"Java"对应的引用
String s2 = s1.intern();
// 会在堆中在单独创建一个字符串对象
String s3 = new String("Java");
// 直接返回字符串常量池中字符串对象"Java"对应的引用
String s4 = s3.intern();
// s1 和 s2 指向的是堆中的同一个对象
System.out.println(s1 == s2); // true
// s3 和 s4 指向的是堆中不同的对象
System.out.println(s3 == s4); // false
// s1 和 s4 指向的是堆中的同一个对象
System.out.println(s1 == s4); //true

```

String 类型的变量和常量做“+”运算时发生了什么？

先来看字符串不加 `final` 关键字拼接的情况（JDK1.8）：

```
String str1 = "str";
String str2 = "ing";
String str3 = "str" + "ing";
String str4 = str1 + str2;
String str5 = "string";
System.out.println(str3 == str4); // false
System.out.println(str3 == str5); // true
System.out.println(str4 == str5); // false
```

注意： 比较 String 字符串的值是否相等，可以使用 `equals()` 方法。String 中的 `equals` 方法是被重写过的。Object 的 `equals` 方法是比较的对象的内存地址，而 String 的 `equals` 方法比较的是字符串的值是否相等。如果你使用 `==` 比较两个字符串是否相等的话，IDEA 还是提示你使用 `equals()` 方法替换。



```
String str1 = "str";
String str2 = "ing";

String str3 = "str" + "ing"; // 常量池中的对象
String str4 = str1 + str2; // 在堆上创建的新的对象
String str5 = "string"; // 常量池中的对象
System.out.println(str3 == str4); // false
System.out.println(str3 == str5); // true
System.out.println(str4 == str5); // false
}
```

The screenshot shows an IDE with a code snippet. A red box highlights the suggestion "Replace '==' with 'equals()'" in the context menu for the line `System.out.println(str3 == str4);`. Other suggestions include "Replace '==' with null-safe 'equals()'", "Flip '=='", and "Negate '==' to '!='".

对于编译期可以确定值的字符串，也就是常量字符串，jvm 会将其存入字符串常量池。并且，字符串常量拼接得到的字符串常量在编译阶段就已经被存放字符串常量池，这个得益于编译器的优化。

在编译过程中，Javac 编译器（下文中统称为编译器）会进行一个叫做 **常量折叠(Constant Folding)** 的代码优化。《深入理解 Java 虚拟机》中是也有介绍到：

第四部分 程序编译与代码优化

第10章 前端编译与优化

10.1 概述

10.2 Javac 编译器

10.2.1 Javac 的源码与调试

10.2.2 解析与填充符号表

10.2.3 注解处理器

10.2.4 语义分析与字节码生成

10.3 Java 语法的味道

10.4 实战：插入式注解处理器

10.5 本章小结

第11章 后端编译与优化

第五部分 高效并发

附录 A 在 Windows 系统下编译 OpenJDK 6

```
int d = a + c;  
int d = b + c;  
char d = a + c;
```

后续代码中如果出现了如上3种赋值运算的话，那它们都能构成结构正确的抽象语法树，但是只有第一种写法在语义上是没有错误的，能够通过检查和编译。其余两种在Java语言中是不合逻辑的，无法编译（是否合乎语义逻辑必须限定在具体的语言与具体的上下文环境之中才有意义。如在C语言中，a、b、c的上下文定义不变，第二、三种写法都是可以正确编译的）。我们编码时经常能在IDE中看到由红线标注的错误提示，其中绝大部分都是来源于语义分析阶段的检查结果。

1. 标注检查

Javac在编译过程中，语义分析过程可分为标注检查和数据及控制流分析两个步骤，分别由图10-5的attribute()和flow()方法（分别对应图10-5中的过程3.1和过程3.2）完成。

标注检查步骤要检查的内容包括诸如变量使用前是否已被声明、变量与赋值之间的数据类型是否能够匹配，等等，刚才3个变量定义的例子就属于标注检查的处理范畴。在标注检查中，还会顺便进行一个称为**常量折叠（Constant Folding）**的代码优化，这是Javac编译器会对源代码做的极少量优化措施之一（代码优化几乎都在即时编译器中进行）。如果我们在Java代码中写下如下所示的变量定义：

常量折叠会把常量表达式的值求出来作为常量嵌在最终生成的代码中，这是 Javac 编译器会对源代码做的极少量优化措施之一(代码优化几乎都在即时编译器中进行)。

对于 `String str3 = "str" + "ing";` 编译器会给你优化成 `String str3 = "string";` 。

并不是所有的常量都会进行折叠，只有编译器在程序编译期就可以确定值的常量才可以：

- 基本数据类型(`byte` 、 `boolean` 、 `short` 、 `char` 、 `int` 、 `float` 、 `long` 、 `double`)以及字符串常量。
- `final` 修饰的基本数据类型和字符串变量
- 字符串通过“+”拼接得到的字符串、基本数据类型之间算数运算（加减乘除）、基本数据类型的位运算（<<、>>、>>>）

引用的值在程序编译期是无法确定的，编译器无法对其进行优化。

对象引用和“+”的字符串拼接方式，实际上是通过 `StringBuilder` 调用 `append()` 方法实现的，拼接完成之后调用 `toString()` 得到一个 `String` 对象。

```
String str4 = new StringBuilder().append(str1).append(str2).toString();
```

我们在平时写代码的时候，尽量避免多个字符串对象拼接，因为这样会重新创建对象。如果需要改变字符串的话，可以使用 `StringBuilder` 或者 `StringBuffer` 。

不过，字符串使用 `final` 关键字声明之后，可以让编译器当做常量来处理。

示例代码：

```
final String str1 = "str";
final String str2 = "ing";
// 下面两个表达式其实是等价的
String c = "str" + "ing";// 常量池中的对象
String d = str1 + str2; // 常量池中的对象
System.out.println(c == d);// true
```

被 `final` 关键字修改之后的 `String` 会被编译器当做常量来处理，编译器在程序编译期就可以确定它的值，其效果就相当于访问常量。

如果，编译器在运行时才能知道其确切值的话，就无法对其优化。

示例代码（`str2` 在运行时才能确定其值）：

```
final String str1 = "str";
final String str2 = getStr();
String c = "str" + "ing";// 常量池中的对象
String d = str1 + str2; // 在堆上创建的新的对象
System.out.println(c == d);// false
public static String getStr() {
    return "ing";
}
```

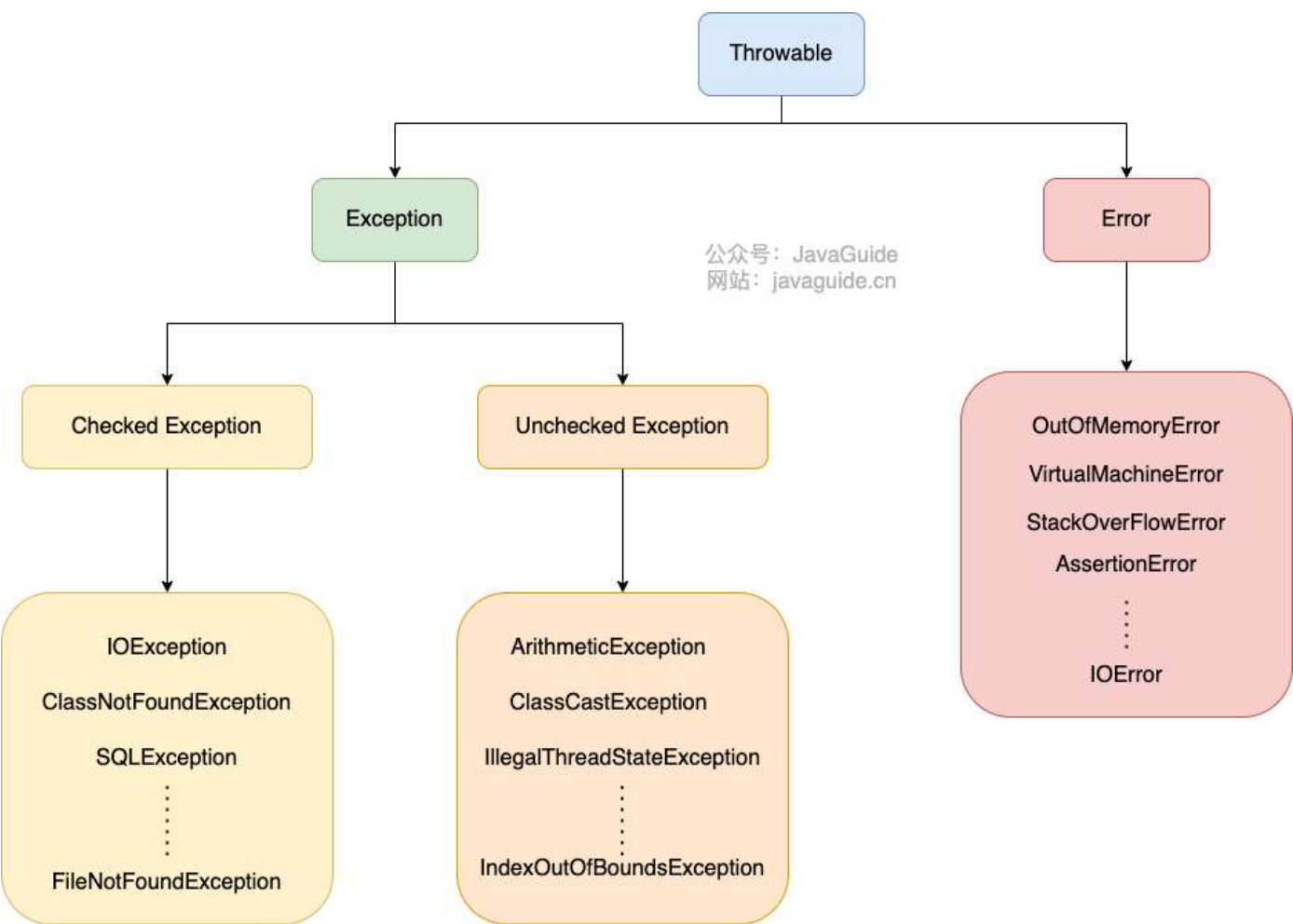
Exception 和 Error 有什么区别？

在 Java 中，所有的异常都有一个共同的祖先 `java.lang` 包中的 `Throwable` 类。`Throwable` 类有两个重要的子类：

- **Exception** :程序本身可以处理的异常，可以通过 `catch` 来进行捕获。`Exception` 又可以分为 Checked Exception (受检查异常，必须处理) 和 Unchecked Exception (不受检查异常，可以不处理)。
- **Error** : `Error` 属于程序无法处理的错误，我们没办法通过 `catch` 来进行捕获不建议通过 `catch` 捕获。例如 Java 虚拟机运行错误（`Virtual MachineError`）、虚拟机内存不够错误（`OutOfMemoryError`）、类定义错误（`NoClassDefFoundError`）等。这些异常发生时，Java 虚拟机（JVM）一般会选择线程终止。

Checked Exception 和 Unchecked Exception 有什么区别？

Java 异常类层次结构图概览：



Checked Exception 即 受检查异常，Java 代码在编译过程中，如果受检查异常没有被 `catch` 或者 `throws` 关键字处理的话，就没办法通过编译。

比如下面这段 IO 操作的代码：

```
String fileName = "file does not exist";
File file = new File(fileName);
FileInputStream stream = new FileInputStream(file);
}
```

Unhandled exception: java.io.FileNotFoundException
Add exception to method signature More actions...

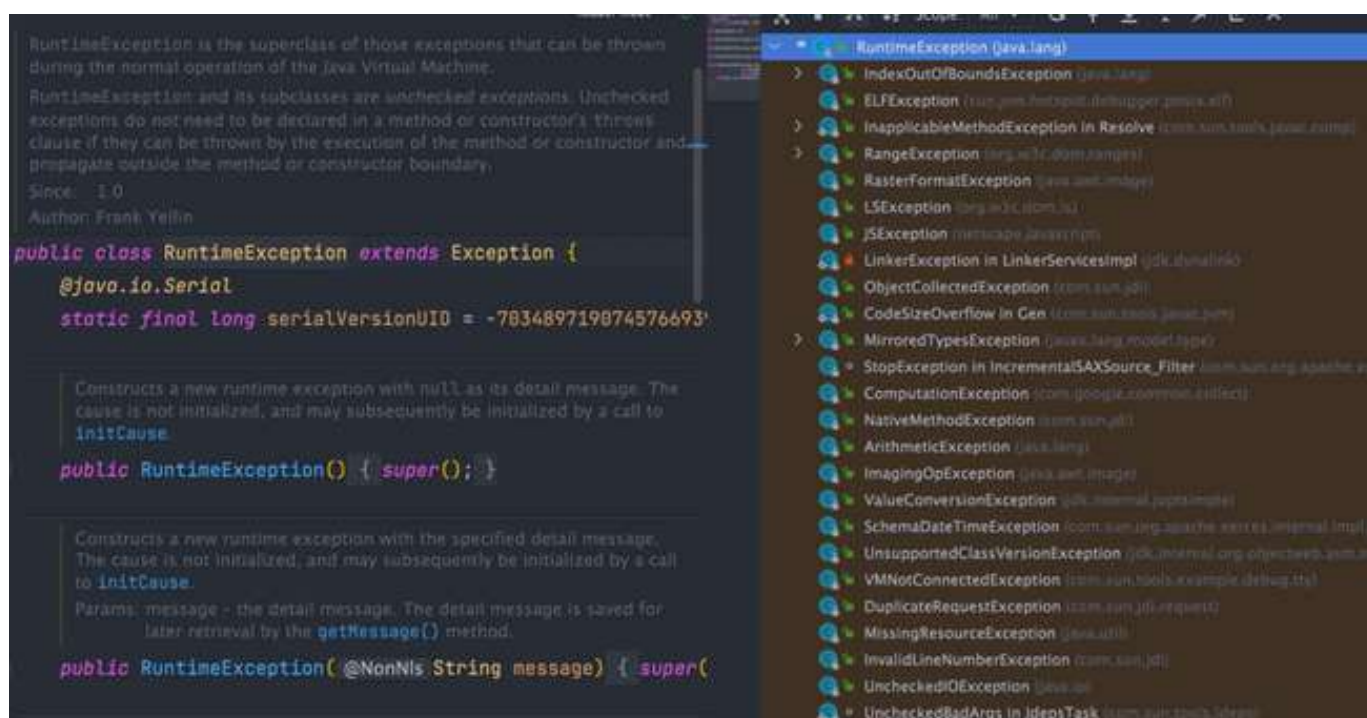
java.io.FileInputStream
public FileInputStream(@NotNull java.io.File file)
throws java.io.FileNotFoundException
Creates a FileInputStream by opening a connection to a

除了 `RuntimeException` 及其子类以外，其他的 `Exception` 类及其子类都属于受检查异常。常见的受检查异常有：IO 相关的异常、`ClassNotFoundException`、`SQLException` ...。

Unchecked Exception 即 不受检查异常，Java 代码在编译过程中，我们即使不处理不受检查异常也可以正常通过编译。

`RuntimeException` 及其子类都统称为非受检查异常，常见的有（建议记下来，日常开发中会经常用到）：

- `NullPointerException`（空指针错误）
- `IllegalArgumentException`（参数错误比如方法入参类型错误）
- `NumberFormatException`（字符串转换为数字格式错误，`IllegalArgumentException` 的子类）
- `ArrayIndexOutOfBoundsException`（数组越界错误）
- `ClassCastException`（类型转换错误）
- `ArithmeticException`（算术错误）
- `SecurityException`（安全错误比如权限不够）
- `UnsupportedOperationException`（不支持的操作错误比如重复创建同一用户）
-



Throwable 类常用方法有哪些？

- `String getMessage()`：返回异常发生时的简要描述
- `String toString()`：返回异常发生时的详细信息
- `String getLocalizedMessage()`：返回异常对象的本地化信息。使用 `Throwable` 的子类覆盖这个方法，可以生成本地化信息。如果子类没有覆盖该方法，则该方法返回的信息与 `getMessage()` 返回的结果相同
- `void printStackTrace()`：在控制台上打印 `Throwable` 对象封装的异常信息

try-catch-finally 如何使用？

- `try` 块： 用于捕获异常。其后可接零个或多个 `catch` 块，如果没有 `catch` 块，则必须跟一个 `finally` 块。
- `catch` 块： 用于处理 `try` 捕获到的异常。
- `finally` 块： 无论是否捕获或处理异常，`finally` 块里的语句都会被执行。当在 `try` 块或 `catch` 块中遇到 `return` 语句时，`finally` 语句块将在方法返回之前被执行。

代码示例：

```
try {
    System.out.println("Try to do something");
    throw new RuntimeException("RuntimeException");
} catch (Exception e) {
    System.out.println("Catch Exception -> " + e.getMessage());
} finally {
    System.out.println("Finally");
}
```

输出：

```
Try to do something
Catch Exception -> RuntimeException
Finally
```

注意：不要在 `finally` 语句块中使用 `return`！当 `try` 语句和 `finally` 语句中都有 `return` 语句时，`try` 语句块中的 `return` 语句会被忽略。这是因为 `try` 语句中的 `return` 返回值会先被暂存在一个本地变量中，当执行到 `finally` 语句中的 `return` 之后，这个本地变量的值就变为了 `finally` 语句中的 `return` 返回值。

[jvm 官方文档](#)中有明确提到：

If the `try` clause executes a *return*, the compiled code does the following:

1. Saves the return value (if any) in a local variable.
2. Executes a *jsr* to the code for the `finally` clause.
3. Upon return from the `finally` clause, returns the value saved in the local variable.

代码示例：

```
public static void main(String[] args) {  
    System.out.println(f(2));  
}  
  
public static int f(int value) {  
    try {  
        return value * value;  
    } finally {  
        if (value == 2) {  
            return 0;  
        }  
    }  
}
```

输出：

0

finally 中的代码一定会执行吗？

不一定的！在某些情况下，finally 中的代码不会被执行。

就比如说 finally 之前虚拟机被终止运行的话，finally 中的代码就不会被执行。

```
try {
    System.out.println("Try to do something");
    throw new RuntimeException("RuntimeException");
} catch (Exception e) {
    System.out.println("Catch Exception -> " + e.getMessage());
    // 终止当前正在运行的Java虚拟机
    System.exit(1);
} finally {
    System.out.println("Finally");
}
```

输出：

```
Try to do something
Catch Exception -> RuntimeException
```

另外，在以下 2 种特殊情况下，`finally` 块的代码也不会被执行：

1. 程序所在的线程死亡。
2. 关闭 CPU。

相关 issue：<https://github.com/Snailclimb/JavaGuide/issues/190>。

 进阶一下：从字节码角度分析 `try catch finally` 这个语法糖背后的实现原理。

如何使用 `try-with-resources` 代替 `try-catch-finally` ?

1. 适用范围（资源的定义）：任何实现 `java.lang.AutoCloseable` 或者 `java.io.Closeable` 的对象
2. 关闭资源和 `finally` 块的执行顺序：在 `try-with-resources` 语句中，任何 `catch` 或 `finally` 块在声明的资源关闭后运行

《Effective Java》中明确指出：

面对必须要关闭的资源，我们总是应该优先使用 `try-with-resources` 而不是 `try-finally`。随之产生的代码更简短，更清晰，产生的异常对我们也更有用。`try-with-resources` 语句让我们更容易编写必须要关闭的资源的代码，若采用 `try-finally` 则几乎做不到这点。

Java 中类似于 `InputStream`、`OutputStream`、`Scanner`、`PrintWriter` 等的资源都需要我们调用 `close()` 方法来手动关闭，一般情况下我们都是通过 `try-catch-finally` 语句来实现这个需求，如下：

```
//读取文本文件的内容
Scanner scanner = null;
try {
    scanner = new Scanner(new File("D://read.txt"));
    while (scanner.hasNext()) {
        System.out.println(scanner.nextLine());
    }
} catch (FileNotFoundException e) {
    e.printStackTrace();
} finally {
    if (scanner != null) {
        scanner.close();
    }
}
```

使用 Java 7 之后的 `try-with-resources` 语句改造上面的代码：

```
try (Scanner scanner = new Scanner(new File("test.txt"))) {
    while (scanner.hasNext()) {
        System.out.println(scanner.nextLine());
    }
} catch (FileNotFoundException fnfe) {
    fnfe.printStackTrace();
}
```

当然多个资源需要关闭的时候，使用 `try-with-resources` 实现起来也非常简单，如果你还是用 `try-catch-finally` 可能会带来很多问题。

通过使用分号分隔，可以在 `try-with-resources` 块中声明多个资源。

```
try (BufferedInputStream bin = new BufferedInputStream(new FileInputStream(new
File("test.txt")));
    BufferedOutputStream bout = new BufferedOutputStream(new
FileOutputStream(new File("out.txt")))) {
    int b;
    while ((b = bin.read()) != -1) {
        bout.write(b);
    }
}
catch (IOException e) {
    e.printStackTrace();
}
```

异常使用有哪些需要注意的地方？

- 不要把异常定义为静态变量，因为这样会导致异常栈信息错乱。每次手动抛出异常，我们都需要手动 `new` 一个异常对象抛出。
- 抛出的异常信息一定要有意义。
- 建议抛出更加具体的异常比如字符串转换为数字格式错误的时候应该抛出 `NumberFormatException` 而不是其父类 `IllegalArgumentException` 。
- 使用日志打印异常之后就不要再抛出异常了（两者不要同时存在一段代码逻辑中）。
-

何谓反射？

如果说大家研究过框架的底层原理或者咱们自己写过框架的话，一定对反射这个概念不陌生。反射之所以被称为框架的灵魂，主要是因为它赋予了我们在运行时分析类以及执行类中方法的能力。通过反射你可以获取任意一个类的所有属性和方法，你还可以调用这些方法和属性。

反射的优缺点？

反射可以让我们的代码更加灵活、为各种框架提供开箱即用的功能提供了便利。

不过，反射让我们在运行时有了分析操作类的能力的同时，也增加了安全问题，比如可以无视泛型参数的安全检查（泛型参数的安全检查发生在编译时）。另外，反射的性能也要稍差点，不过，对于框架来说实际是影响不大的。

相关阅读：[Java Reflection: Why is it so slow?](#)。

反射的应用场景？

像咱们平时大部分时候都是在写业务代码，很少会接触到直接使用反射机制的场景。但是！这并不代表反射没有用。相反，正是因为反射，你才能这么轻松地使用各种框架。像 Spring/Spring Boot、MyBatis 等等框架中都大量使用了反射机制。

这些框架中也大量使用了动态代理，而动态代理的实现也依赖反射。

比如下面是通过 JDK 实现动态代理的示例代码，其中就使用了反射类 `Method` 来调用指定的方法。

```
public class DebugInvocationHandler implements InvocationHandler {  
    /**  
     * 代理类中的真实对象  
     */  
    private final Object target;  
  
    public DebugInvocationHandler(Object target) {  
        this.target = target;  
    }  
  
    public Object invoke(Object proxy, Method method, Object[] args) throws  
    InvocationTargetException, IllegalAccessException {  
        System.out.println("before method " + method.getName());  
        Object result = method.invoke(target, args);  
        System.out.println("after method " + method.getName());  
        return result;  
    }  
}
```

另外，像 Java 中的一大利器 **注解** 的实现也用到了反射。

为什么你使用 Spring 的时候，一个 `@Component` 注解就声明了一个类为 Spring Bean 呢？为什么你通过一个 `@Value` 注解就读取到配置文件中的值呢？究竟是怎么起作用的呢？

这些都是因为你可以基于反射分析类，然后获取到类/属性/方法/方法的参数上的注解。你获取到注解之后，就可以做进一步的处理。

何谓 SPI?

SPI 即 Service Provider Interface，字面意思就是：“服务提供者的接口”，我的理解是：专门提供给服务提供者或者扩展框架功能的开发者去使用的一个接口。

SPI 将服务接口和具体的服务实现分离开来，将服务调用方和服务实现者解耦，能够提升程序的扩展性、可维护性。修改或者替换服务实现并不需要修改调用方。

很多框架都使用了 Java 的 SPI 机制，比如：Spring 框架、数据库加载驱动、日志接口、以及 Dubbo 的扩展实现等等。

SPI 扩展实现



协议扩展

调用拦截扩展

引用监听扩展

暴露监听扩展

集群扩展

路由扩展

负载均衡扩展

合并结果扩展

注册中心扩展

监控中心扩展

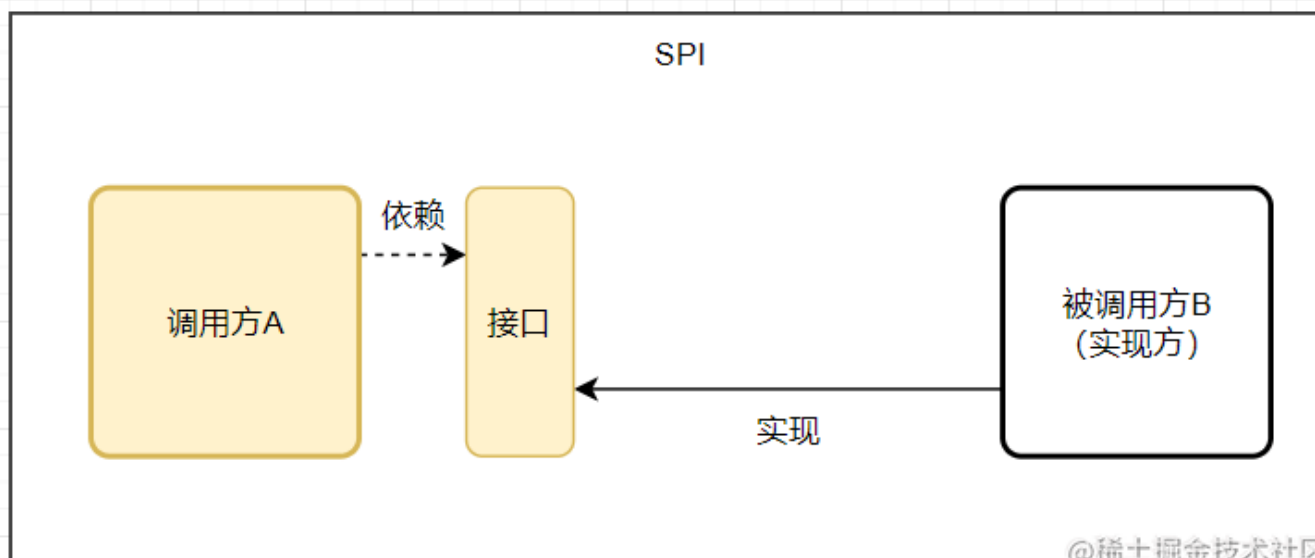
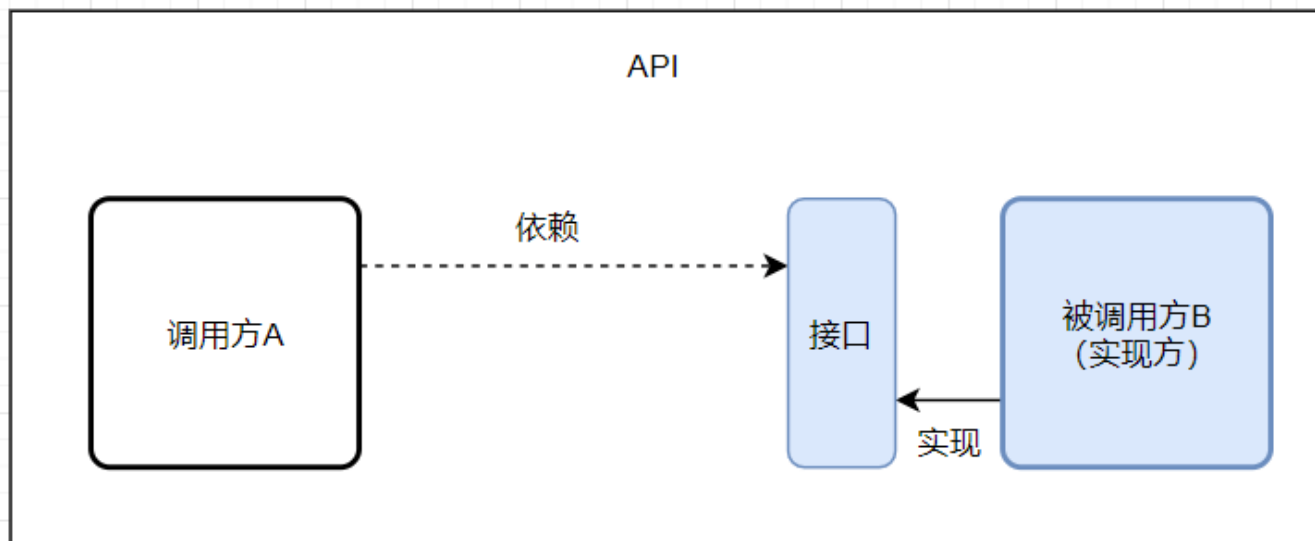
扩展点加载扩展

动态代理扩展

SPI 和 API 有什么区别？

那 SPI 和 API 有啥区别？

说到 SPI 就不得不说一下 API 了，从广义上来说它们都属于接口，而且很容易混淆。下面先用一张图说明一下：



一般模块之间都是通过通过接口进行通讯，那我们在服务调用方和服务实现方（也称服务提供者）之间引入一个“接口”。

当实现方提供了接口和实现，我们可以通过调用实现方的接口从而拥有实现方给我们提供的的能力，这就是 API，这种接口和实现都是放在实现方的。

当接口存在于调用方这边时，就是 SPI，由接口调用方确定接口规则，然后由不同的厂商去根绝这个规则对这个接口进行实现，从而提供服务。

举个通俗易懂的例子：公司 H 是一家科技公司，新设计了一款芯片，然后现在需要量产了，而市面上有好几家芯片制造业公司，这个时候，只要 H 公司指定好了这芯片生产的标准（定义好了接口标准），那么这些合作的芯片公司（服务提供者）就按照标准交付自家特色的芯片（提供不同方案的实现，但是给出来的结果是一样的）。

SPI 的优缺点？

通过 SPI 机制能够大大地提高接口设计的灵活性，但是 SPI 机制也存在一些缺点，比如：

- 需要遍历加载所有的实现类，不能做到按需加载，这样效率还是相对较低的。
- 当多个 `ServiceLoader` 同时 `load` 时，会有并发问题。

什么是序列化？什么是反序列化？

如果我们需要持久化 Java 对象比如将 Java 对象保存在文件中，或者在网络传输 Java 对象，这些场景都需要用到序列化。

简单来说：

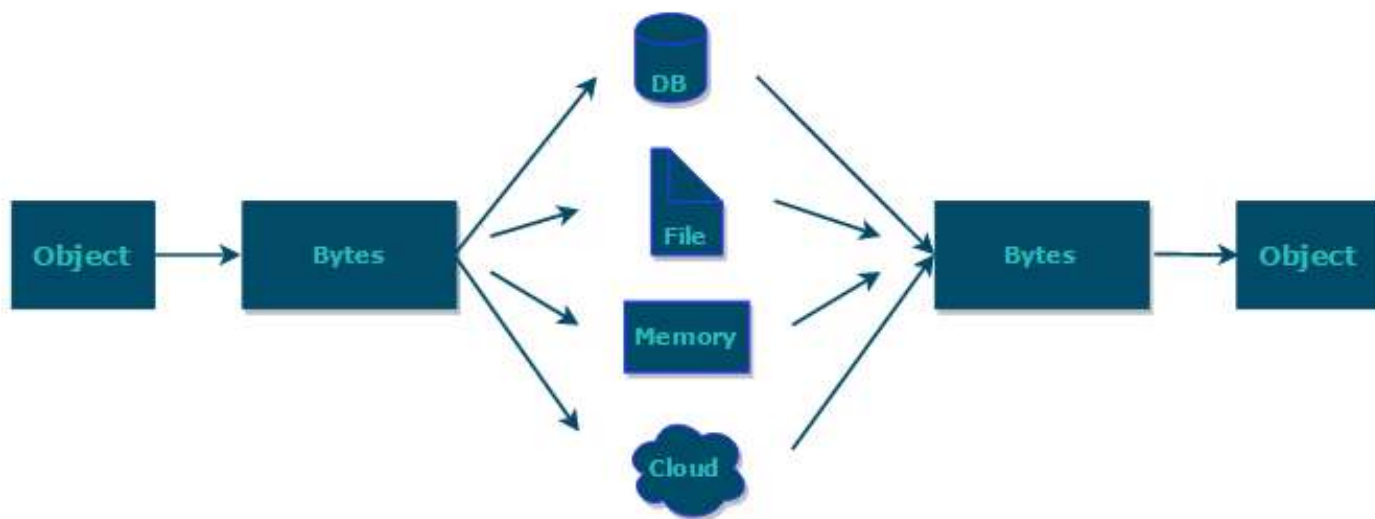
- **序列化**：将数据结构或对象转换成二进制字节流的过程
- **反序列化**：将在序列化过程中所生成的二进制字节流转换成数据结构或者对象的过程

对于 Java 这种面向对象编程语言来说，我们序列化的都是对象（Object）也就是实例化后的类（Class），但是在 C++ 这种半面向对象的语言中，`struct`（结构体）定义的是数据结构类型，而 `class` 对应的是对象类型。

维基百科是如是介绍序列化的：

序列化（serialization）在计算机科学的数据处理中，是指将数据结构或对象状态转换成可取用格式（例如存成文件，存于缓冲，或经由网络中发送），以留待后续在相同或另一台计算机环境中，能恢复原先状态的过程。依照序列化格式重新获取字节的结果时，可以利用它来产生与原始对象相同语义的副本。对于许多对象，像是使用大量引用的复杂对象，这种序列化重建的过程并不容易。面向对象中的对象序列化，并不概括之前原始对象所关系的函数。这种过程也称为对象编组（marshalling）。从一系列字节提取数据结构的反向操作，是反序列化（也称为解编组、deserialization、unmarshalling）。

综上：序列化的主要目的是通过网络传输对象或者说是将对象存储到文件系统、数据库、内存中。



<https://www.corejavaguru.com/java/serialization/interview-questions-1>

如果有些字段不想进行序列化怎么办？

对于不想进行序列化的变量，使用 `transient` 关键字修饰。

`transient` 关键字的作用是：阻止实例中那些用此关键字修饰的变量序列化；当对象被反序列化时，被 `transient` 修饰的变量值不会被持久化和恢复。

关于 `transient` 还有几点注意：

- `transient` 只能修饰变量，不能修饰类和方法。
- `transient` 修饰的变量，在反序列化后变量值将会被置成类型的默认值。例如，如果是修饰 `int` 类型，那么反序列化后结果就是 `0`。
- `static` 变量因为不属于任何对象(Object)，所以无论有没有 `transient` 关键字修饰，均不会被序列化。

Java IO 流了解吗？

IO 即 `Input/Output`，输入和输出。数据输入到计算机内存的过程即输入，反之输出到外部存储（比如数据库，文件，远程主机）的过程即输出。数据传输过程类似于水流，因此称为 IO 流。IO 流在 Java 中分为输入流和输出流，而根据数据的处理方式又分为字节流和字符流。

Java IO 流的 40 多个类都是从如下 4 个抽象类基类中派生出来的。

- `InputStream / Reader`：所有的输入流的基类，前者是字节输入流，后者是字符输入流。
- `OutputStream / Writer`：所有输出流的基类，前者是字节输出流，后者是字符输出流。

相关阅读：[Java IO 基础知识总结](#)。

I/O 流为什么要分为字节流和字符流呢？

问题本质想问：不管是文件读写还是网络发送接收，信息的最小存储单元都是字节，那为什么 I/O 流操作要分为字节流操作和字符流操作呢？

个人认为主要有两点原因：

- 字符流是由 Java 虚拟机将字节转换得到的，这个过程还算是比较耗时；
- 如果我们不知道编码类型的话，使用字节流的过程中很容易出现乱码问题。

Java IO 中的设计模式有哪些？

[Java IO 设计模式总结](#)。

BIO、NIO 和 AIO 的区别？

[Java IO 模型详解](#)。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

2.2. Java集合

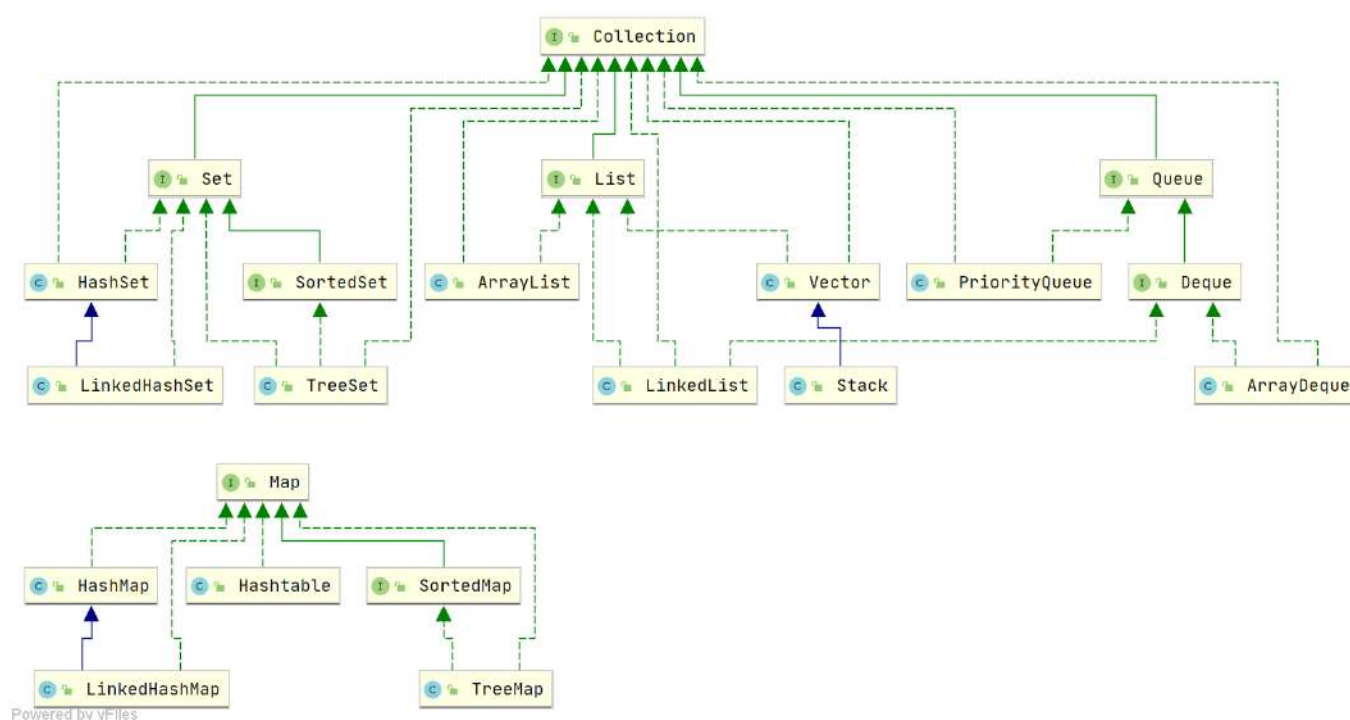
[JavaGuide](#)：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

这部分内容摘自 [JavaGuide](#) 下面几篇文章：

- [Java集合常见面试题总结\(上\)](#)
- [Java集合常见面试题总结\(下\)](#)

Java 集合，也叫作容器，主要是由两大接口派生而来：一个是 `Collection` 接口，主要用于存放单一元素；另一个是 `Map` 接口，主要用于存放键值对。对于 `Collection` 接口，下面又有三个主要的子接口：`List`、`Set` 和 `Queue`。

Java 集合框架如下图所示：



注：图中只列举了主要的继承派生关系，并没有列举所有关系。比方省略了 `AbstractList`，`NavigableSet` 等抽象类以及其他的一些辅助类，如想深入了解，可自行查看源码。

说说 List, Set, Queue, Map 四者的区别?

- List (对付顺序的好帮手): 存储的元素是有序的、可重复的。
- Set (注重独一无二的性质): 存储的元素是无序的、不可重复的。
- Queue (实现排队功能的叫号机): 按特定的排队规则来确定先后顺序, 存储的元素是有序的、可重复的。
- Map (用 key 来搜索的专家): 使用键值对 (key-value) 存储, 类似于数学上的函数 $y=f(x)$, "x" 代表 key, "y" 代表 value, key 是无序的、不可重复的, value 是无序的、可重复的, 每个键最多映射到一个值。

集合框架底层数据结构总结

先来看一下 Collection 接口下面的集合。

List

- ArrayList : Object[] 数组
- Vector : Object[] 数组
- LinkedList : 双向链表(JDK1.6 之前为循环链表, JDK1.7 取消了循环)

Set

- HashSet (无序, 唯一): 基于 HashMap 实现的, 底层采用 HashMap 来保存元素
- LinkedHashSet : LinkedHashSet 是 HashSet 的子类, 并且其内部是通过 LinkedHashMap 来实现的。有点类似于我们之前说的 LinkedHashMap 其内部是基于 HashMap 实现一样, 不过还是有一点点区别的
- TreeSet (有序, 唯一): 红黑树(自平衡的排序二叉树)

Queue

- PriorityQueue : Object[] 数组来实现二叉堆
- ArrayQueue : Object[] 数组 + 双指针

再看看 Map 接口下面的集合。

Map

- HashMap : JDK1.8 之前 HashMap 由数组+链表组成的, 数组是 HashMap 的主体, 链表则是主要为了解决哈希冲突而存在的 (“拉链法”解决冲突)。JDK1.8 以后在解决哈希冲突时有了较大的变化, 当链表长度大于阈值 (默认为 8) (将链表转换成红黑树前会判断, 如果当前数组的长度小于 64, 那么会选择先进行数组扩容, 而不是转换为红黑树) 时, 将链表转化为红黑

树，以减少搜索时间

- `LinkedHashMap`：`LinkedHashMap` 继承自 `HashMap`，所以它的底层仍然是基于拉链式散列结构即由数组和链表或红黑树组成。另外，`LinkedHashMap` 在上面结构的基础上，增加了一条双向链表，使得上面的结构可以保持键值对的插入顺序。同时通过对链表进行相应的操作，实现了访问顺序相关逻辑。详细可以查看：[《LinkedHashMap 源码详细分析（JDK1.8）》](#)
- `Hashtable`：数组+链表组成的，数组是 `Hashtable` 的主体，链表则是主要为了解决哈希冲突而存在的
- `TreeMap`：红黑树（自平衡的排序二叉树）

如何选用集合？

主要根据集合的特点来选用，比如我们需要根据键值获取到元素值时就选用 `Map` 接口下的集合，需要排序时选择 `TreeMap`，不需要排序时就选择 `HashMap`，需要保证线程安全就选用 `ConcurrentHashMap`。

当我们只需要存放元素值时，就选择实现 `Collection` 接口的集合，需要保证元素唯一时选择实现 `Set` 接口的集合比如 `TreeSet` 或 `HashSet`，不需要就选择实现 `List` 接口的比如 `ArrayList` 或 `LinkedList`，然后再根据实现这些接口的集合的特点来选用。

为什么要使用集合？

当我们需要保存一组类型相同的数据的时候，我们应该是用一个容器来保存，这个容器就是数组，但是，使用数组存储对象具有一定的弊端，因为我们在实际开发中，存储的数据的类型是多种多样的，于是，就出现了“集合”，集合同样也是用来存储多个数据的。

数组的缺点是一旦声明之后，长度就不可变了；同时，声明数组时的数据类型也决定了该数组存储的数据的类型；而且，数组存储的数据是有序的、可重复的，特点单一。但是集合提高了数据存储的灵活性，Java 集合不仅可以用来存储不同类型不同数量的对象，还可以保存具有映射关系的数据。

ArrayList 和 Vector 的区别？

- `ArrayList` 是 `List` 的主要实现类，底层使用 `Object[]` 存储，适用于频繁的查找工作，线程不安全；
- `Vector` 是 `List` 的古老实现类，底层使用 `Object[]` 存储，线程安全的。

ArrayList 与 LinkedList 区别？

- **是否保证线程安全：** `ArrayList` 和 `LinkedList` 都是不同步的，也就是不保证线程安全；
- **底层数据结构：** `ArrayList` 底层使用的是 `Object` 数组； `LinkedList` 底层使用的是 双向链表 数据结构（JDK1.6 之前为循环链表，JDK1.7 取消了循环。注意双向链表和双向循环链表的区别，下面有介绍到！）
- **插入和删除是否受元素位置的影响：**
 - `ArrayList` 采用数组存储，所以插入和删除元素的时间复杂度受元素位置的影响。比如：执行 `add(E e)` 方法的时候，`ArrayList` 会默认在将指定的元素追加到此列表的末尾，这种情况时间复杂度就是 $O(1)$ 。但是如果要在指定位置 `i` 插入和删除元素的话（`add(int index, E element)`）时间复杂度就为 $O(n-i)$ 。因为在进行上述操作的时候集合中第 `i` 和第 `i` 个元素之后的 $(n-i)$ 个元素都要执行向后位/向前移一位的操作。
 - `LinkedList` 采用链表存储，所以，如果是在头尾插入或者删除元素不受元素位置的影响（`add(E e)`、`addFirst(E e)`、`addLast(E e)`、`removeFirst()`、`removeLast()`），时间复杂度为 $O(1)$ ，如果是要在指定位置 `i` 插入和删除元素的话（`add(int index, E element)`，`remove(Object o)`），时间复杂度为 $O(n)$ ，因为需要先移动到指定位置再插入。
- **是否支持快速随机访问：** `LinkedList` 不支持高效的随机元素访问，而 `ArrayList` 支持。快速随机访问就是通过元素的序号快速获取元素对象(对应于 `get(int index)` 方法)。
- **内存空间占用：** `ArrayList` 的空间浪费主要体现在在 `list` 列表的结尾会预留一定的容量空间，而 `LinkedList` 的空间花费则体现在它的每一个元素都需要消耗比 `ArrayList` 更多的空间（因为要存放直接后继和直接前驱以及数据）。

我们在项目中一般是不会使用到 `LinkedList` 的，需要用到 `LinkedList` 的场景几乎都可以使用 `ArrayList` 来代替，并且，性能通常会更好！就连 `LinkedList` 的作者约书亚·布洛克（Josh Bloch）自己都说从来不会使用 `LinkedList`。



Joshua Bloch

@joshbloch

关注

回复 @jerrykuch

@jerrykuch @shipilev @AmbientLion Does anyone actually use LinkedList? I wrote it, and I never use it.

下午7:10 - 2015年4月2日

272 转推 313 喜欢



20

272

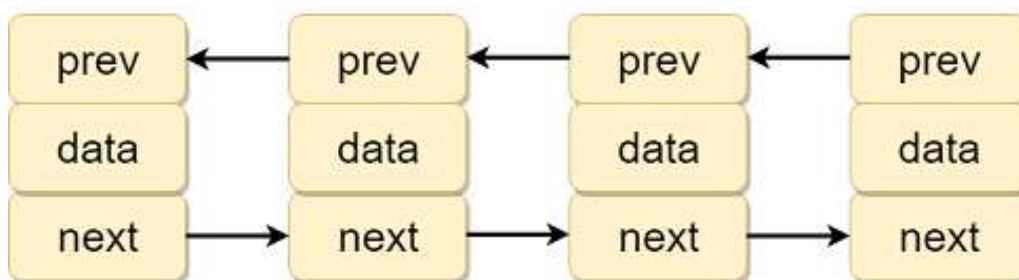
313

另外，不要下意识地认为 `LinkedList` 作为链表就最适合元素增删的场景。我在上面也说了，`LinkedList` 仅仅在头尾插入或者删除元素的时候时间复杂度近似 $O(1)$ ，其他情况增删元素的时间复杂度都是 $O(n)$ 。

补充内容:双向链表和双向循环链表

双向链表：包含两个指针，一个 `prev` 指向前一个节点，一个 `next` 指向后一个节点。

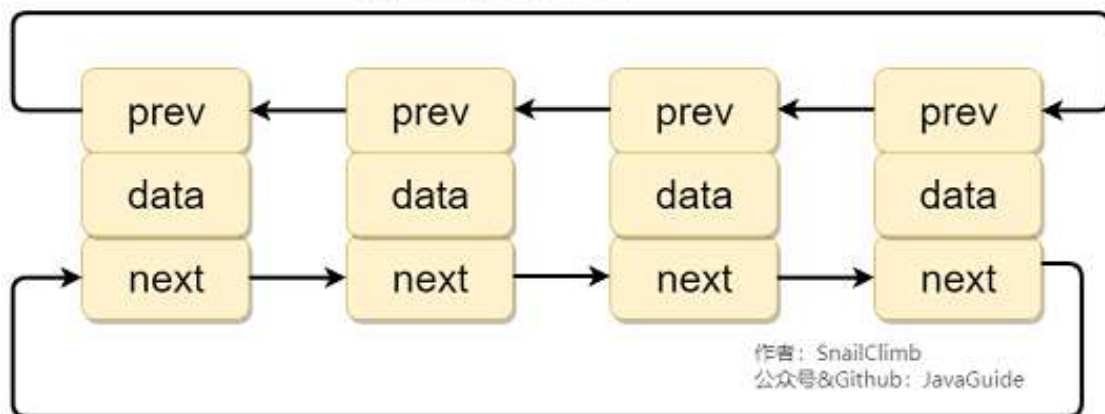
双向链表



作者: SnailClimb
公众号&Github: JavaGuide

双向循环链表：最后一个节点的 `next` 指向 `head`，而 `head` 的 `prev` 指向最后一个节点，构成一个环。

双向循环链表



补充内容:RandomAccess 接口

```
public interface RandomAccess {  
}
```

查看源码我们发现实际上 `RandomAccess` 接口中什么都没有定义。所以，在我看来 `RandomAccess` 接口不过是一个标识罢了。标识什么？标识实现这个接口的类具有随机访问功能。

在 `binarySearch()` 方法中，它要判断传入的 `list` 是否 `RandomAccess` 的实例，如果是，调用 `indexedBinarySearch()` 方法，如果不是，那么调用 `iteratorBinarySearch()` 方法

```
public static <T>  
int binarySearch(List<? extends Comparable<? super T>> list, T key) {  
    if (list instanceof RandomAccess || list.size() < BINARYSEARCH_THRESHOLD)  
        return Collections.indexedBinarySearch(list, key);  
    else  
        return Collections.iteratorBinarySearch(list, key);  
}
```

`ArrayList` 实现了 `RandomAccess` 接口，而 `LinkedList` 没有实现。为什么呢？我觉得还是和底层数据结构有关！`ArrayList` 底层是数组，而 `LinkedList` 底层是链表。数组天然支持随机访问，时间复杂度为 $O(1)$ ，所以称为快速随机访问。链表需要遍历到特定位置才能访问特定位置的元素，时间复杂度为 $O(n)$ ，所以不支持快速随机访问。，`ArrayList` 实现了 `RandomAccess` 接口，就表明了他具有快速随机访问功能。`RandomAccess` 接口只是标识，并不是说 `ArrayList` 实现 `RandomAccess`

接口才具有快速随机访问功能的！

说一说 ArrayList 的扩容机制吧

详见笔主的这篇文章: [ArrayList 扩容机制分析](#)

comparable 和 Comparator 的区别

- comparable 接口实际上是出自 java.lang 包 它有一个 compareTo(Object obj) 方法用来排序
- comparator 接口实际上是出自 java.util 包 它有一个 compare(Object obj1, Object obj2) 方法用来排序

一般我们需要对一个集合使用自定义排序时，我们就要重写 compareTo() 方法或 compare() 方法，当我们需要对某一个集合实现两种排序方式，比如一个 song 对象中的歌名和歌手名分别采用一种排序方法的话，我们可以重写 compareTo() 方法和使用自制的 Comparator 方法或者以两个 Comparator 来实现歌名排序和歌星名排序，第二种代表我们只能使用两个参数版的 Collections.sort() 。

Comparator 定制排序

```
ArrayList<Integer> arrayList = new ArrayList<Integer>();
arrayList.add(-1);
arrayList.add(3);
arrayList.add(3);
arrayList.add(-5);
arrayList.add(7);
arrayList.add(4);
arrayList.add(-9);
arrayList.add(-7);

System.out.println("原始数组:");
System.out.println(arrayList);
// void reverse(List list): 反转
Collections.reverse(arrayList);
System.out.println("Collections.reverse(arrayList):");
System.out.println(arrayList);

// void sort(List list),按自然排序的升序排序
Collections.sort(arrayList);
System.out.println("Collections.sort(arrayList):");
```

```

        System.out.println(arrayList);

        // 定制排序的用法

        Collections.sort(arrayList, new Comparator<Integer>() {

            @Override

            public int compare(Integer o1, Integer o2) {

                return o2.compareTo(o1);

            }

        });

        System.out.println("定制排序后: ");

        System.out.println(arrayList);

```

Output:

```

原始数组:
[-1, 3, 3, -5, 7, 4, -9, -7]
Collections.reverse(arrayList):
[-7, -9, 4, 7, -5, 3, 3, -1]
Collections.sort(arrayList):
[-9, -7, -5, -1, 3, 3, 4, 7]
定制排序后:
[7, 4, 3, 3, -1, -5, -7, -9]

```

重写 compareTo 方法实现按年龄来排序

```

// person对象没有实现Comparable接口，所以必须实现，这样才不会出错，才可以使treemap中的数据
按顺序排列

// 前面一个例子的String类已经默认实现了Comparable接口，详细可以查看String类的API文档，另外
其他

// 像Integer类等都已经实现了Comparable接口，所以不需要另外实现了

public class Person implements Comparable<Person> {

    private String name;

    private int age;

    public Person(String name, int age) {

        super();

```

```
        this.name = name;

        this.age = age;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    /**
     * T重写compareTo方法实现按年龄来排序
     */
    @Override
    public int compareTo(Person o) {
        if (this.age > o.getAge()) {
            return 1;
        }

        if (this.age < o.getAge()) {
            return -1;
        }

        return 0;
    }
}
```



```

public static void main(String[] args) {
    TreeMap<Person, String> pdata = new TreeMap<Person, String>();
    pdata.put(new Person("张三", 30), "zhangsan");
    pdata.put(new Person("李四", 20), "lisi");
    pdata.put(new Person("王五", 10), "wangwu");
    pdata.put(new Person("小红", 5), "xiaohong");
    // 得到key的值的同时得到key所对应的值
    Set<Person> keys = pdata.keySet();
    for (Person key : keys) {
        System.out.println(key.getAge() + "-" + key.getName());
    }
}

```

Output:

```

5-小红
10-王五
20-李四
30-张三

```

无序性和不可重复性的含义是什么

- 无序性不等于随机性，无序性是指存储的数据在底层数组中并非按照数组索引的顺序添加，而是根据数据的哈希值决定的。
- 不可重复性是指添加的元素按照 `equals()` 判断时，返回 `false`，需要同时重写 `equals()` 方法和 `hashCode()` 方法。

比较 HashSet、LinkedHashSet 和 TreeSet 三者的异同

- `HashSet`、`LinkedHashSet` 和 `TreeSet` 都是 `Set` 接口的实现类，都能保证元素唯一，并且都不是线程安全的。
- `HashSet`、`LinkedHashSet` 和 `TreeSet` 的主要区别在于底层数据结构不同。`HashSet` 的底层数据结构是哈希表（基于 `HashMap` 实现）。`LinkedHashSet` 的底层数据结构是链表和哈希表，元素的插入和取出顺序满足 FIFO。`TreeSet` 底层数据结构是红黑树，元素是有序的，排序的方式有自然排序和定制排序。

- 底层数据结构不同又导致这三者的应用场景不同。 `HashSet` 用于不需要保证元素插入和取出顺序的场景， `LinkedHashSet` 用于保证元素的插入和取出顺序满足 FIFO 的场景， `TreeSet` 用于支持对元素自定义排序规则的场景。

Queue 与 Deque 的区别

`Queue` 是单端队列，只能从一端插入元素，另一端删除元素，实现上一般遵循 **先进先出（FIFO）** 规则。

`Queue` 扩展了 `Collection` 的接口，根据 **因为容量问题而导致操作失败后处理方式的不同** 可以分为两类方法：一种在操作失败后会抛出异常，另一种则会返回特殊值。

<code>Queue</code> 接口	抛出异常	返回特殊值
插入队尾	<code>add(E e)</code>	<code>offer(E e)</code>
删除队首	<code>remove()</code>	<code>poll()</code>
查询队首元素	<code>element()</code>	<code>peek()</code>

`Deque` 是双端队列，在队列的两端均可以插入或删除元素。

`Deque` 扩展了 `Queue` 的接口，增加了在队首和队尾进行插入和删除的方法，同样根据失败后处理方式的不同分为两类：

<code>Deque</code> 接口	抛出异常	返回特殊值
插入队首	<code>addFirst(E e)</code>	<code>offerFirst(E e)</code>
插入队尾	<code>addLast(E e)</code>	<code>offerLast(E e)</code>
删除队首	<code>removeFirst()</code>	<code>pollFirst()</code>
删除队尾	<code>removeLast()</code>	<code>pollLast()</code>
查询队首元素	<code>getFirst()</code>	<code>peekFirst()</code>
查询队尾元素	<code>getLast()</code>	<code>peekLast()</code>

事实上，`Deque` 还提供有 `push()` 和 `pop()` 等其他方法，可用于模拟栈。

ArrayDeque 与 LinkedList 的区别

`ArrayDeque` 和 `LinkedList` 都实现了 `Deque` 接口，两者都具有队列的功能，但两者有什么区别呢？

- `ArrayDeque` 是基于可变长的数组和双指针来实现，而 `LinkedList` 则通过链表来实现。
- `ArrayDeque` 不支持存储 `NULL` 数据，但 `LinkedList` 支持。
- `ArrayDeque` 是在 JDK1.6 才被引入的，而 `LinkedList` 早在 JDK1.2 时就已经存在。
- `ArrayDeque` 插入时可能存在扩容过程，不过均摊后的插入操作依然为 $O(1)$ 。虽然 `LinkedList` 不需要扩容，但是每次插入数据时均需要申请新的堆空间，均摊性能相比更慢。

从性能的角度上，选用 `ArrayDeque` 来实现队列要比 `LinkedList` 更好。此外，`ArrayDeque` 也可以用于实现栈。

说一说 PriorityQueue

`PriorityQueue` 是在 JDK1.5 中被引入的，其与 `Queue` 的区别在于元素出队顺序是与优先级相关的，即总是优先级最高的元素先出队。

这里列举其相关的一些要点：

- `PriorityQueue` 利用了二叉堆的数据结构来实现的，底层使用可变长的数组来存储数据
- `PriorityQueue` 通过堆元素的上浮和下沉，实现了在 $O(\log n)$ 的时间复杂度内插入元素和删除堆顶元素。
- `PriorityQueue` 是非线程安全的，且不支持存储 `NULL` 和 `non-comparable` 的对象。
- `PriorityQueue` 默认是小顶堆，但可以接收一个 `Comparator` 作为构造参数，从而来自定义元素优先级的先后。

`PriorityQueue` 在面试中可能更多的会出现在手撕算法的时候，典型例题包括堆排序、求第K大的数、带权图的遍历等，所以需要会熟练使用才行。

HashMap 和 Hashtable 的区别

- **线程是否安全：** `HashMap` 是非线程安全的，`Hashtable` 是线程安全的，因为 `Hashtable` 内部的方法基本都经过 `synchronized` 修饰。（如果你要保证线程安全的话就使用 `ConcurrentHashMap` 吧！）；
- **效率：** 因为线程安全的问题，`HashMap` 要比 `Hashtable` 效率高一点。另外，`Hashtable` 基本

被淘汰，不要在代码中使用它；

- **对 Null key 和 Null value 的支持：** `HashMap` 可以存储 null 的 key 和 value，但 null 作为键只能有一个，null 作为值可以有多个；`Hashtable` 不允许有 null 键和 null 值，否则会抛出 `NullPointerException`。
- **初始容量大小和每次扩充容量大小的不同：** ① 创建时如果不指定容量初始值，`Hashtable` 默认的初始大小为 11，之后每次扩充，容量变为原来的 $2n+1$ 。`HashMap` 默认的初始化大小为 16。之后每次扩充，容量变为原来的 2 倍。② 创建时如果给定了容量初始值，那么 `Hashtable` 会直接使用你给定的大小，而 `HashMap` 会将其扩充为 2 的幂次方大小（`HashMap` 中的 `tableSizeFor()` 方法保证，下面给出了源代码）。也就是说 `HashMap` 总是使用 2 的幂作为哈希表的大小，后面会介绍到为什么是 2 的幂次方。
- **底层数据结构：** JDK1.8 以后的 `HashMap` 在解决哈希冲突时有了较大的变化，当链表长度大于阈值（默认为 8）时，将链表转化为红黑树（将链表转换成红黑树前会判断，如果当前数组的长度小于 64，那么会选择先进行数组扩容，而不是转换为红黑树），以减少搜索时间（后文中我会结合源码对这一过程进行分析）。`Hashtable` 没有这样的机制。

`HashMap` 中带有初始容量的构造函数：

```
public HashMap(int initialCapacity, float loadFactor) {
    if (initialCapacity < 0)
        throw new IllegalArgumentException("Illegal initial capacity: " +
                                           initialCapacity);

    if (initialCapacity > MAXIMUM_CAPACITY)
        initialCapacity = MAXIMUM_CAPACITY;
    if (loadFactor <= 0 || Float.isNaN(loadFactor))
        throw new IllegalArgumentException("Illegal load factor: " +
                                           loadFactor);

    this.loadFactor = loadFactor;
    this.threshold = tableSizeFor(initialCapacity);
}

public HashMap(int initialCapacity) {
    this(initialCapacity, DEFAULT_LOAD_FACTOR);
}
```

下面这个方法保证了 `HashMap` 总是使用 2 的幂作为哈希表的大小。

```

/**
 * Returns a power of two size for the given target capacity.
 */
static final int tableSizeFor(int cap) {
    int n = cap - 1;
    n |= n >>> 1;
    n |= n >>> 2;
    n |= n >>> 4;
    n |= n >>> 8;
    n |= n >>> 16;
    return (n < 0) ? 1 : (n >= MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n + 1;
}

```

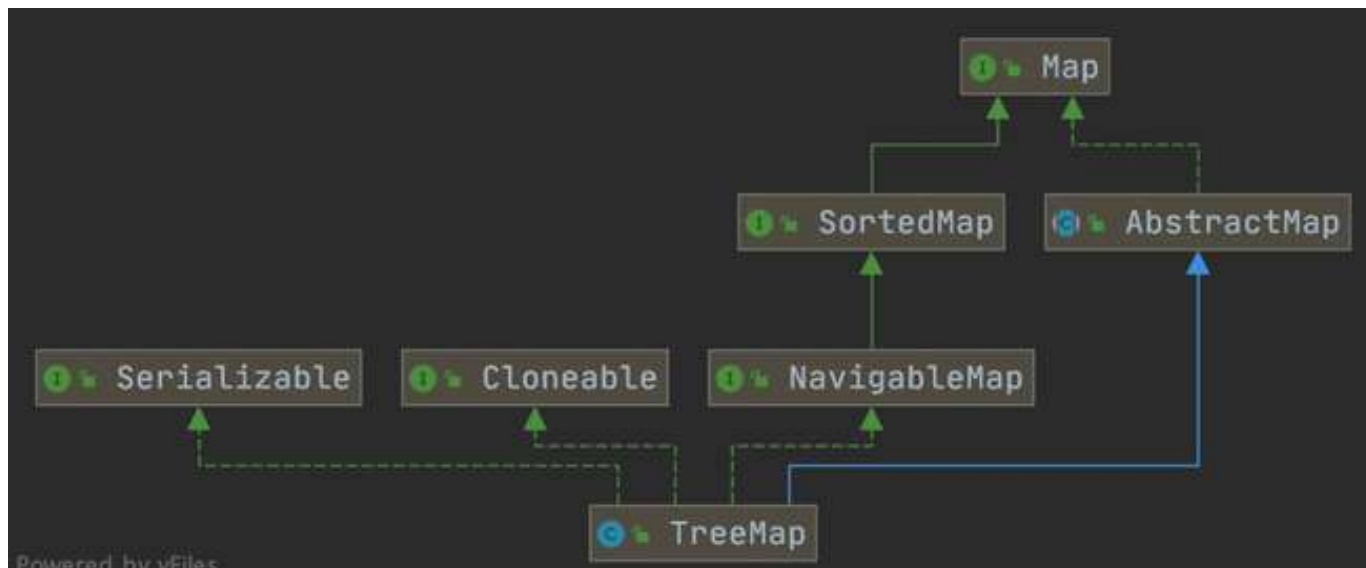
HashMap 和 HashSet 区别

如果你看过 HashSet 源码的话就应该知道：HashSet 底层就是基于 HashMap 实现的。
 (HashSet 的源码非常非常少，因为除了 clone() 、 writeObject() 、 readObject() 是 HashSet 自己不得不实现之外，其他方法都是直接调用 HashMap 中的方法。

HashMap	HashSet
实现了 Map 接口	实现 Set 接口
存储键值对	仅存储对象
调用 put() 向 map 中添加元素	调用 add() 方法向 Set 中添加元素
HashMap 使用键 (Key) 计算 hashCode	HashSet 使用成员对象来计算 hashCode 值，对于两个对象来说 hashCode 可能相同，所以 equals() 方法用来判断对象的相等性

HashMap 和 TreeMap 区别

TreeMap 和 HashMap 都继承自 AbstractMap ，但是需要注意的是 TreeMap 它还实现了 NavigableMap 接口和 SortedMap 接口。



实现 `NavigableMap` 接口让 `TreeMap` 有了对集合内元素的搜索的能力。

实现 `SortedMap` 接口让 `TreeMap` 有了对集合中的元素根据键排序的能力。默认是按 key 的升序排序，不过我们也可以指定排序的比较器。示例代码如下：

```
/**
 * @author shuang.kou
 * @createTime 2020年06月15日 17:02:00
 */
public class Person {
    private Integer age;

    public Person(Integer age) {
        this.age = age;
    }

    public Integer getAge() {
        return age;
    }

    public static void main(String[] args) {
        TreeMap<Person, String> treeMap = new TreeMap<>(new Comparator<Person>())
        {
            @Override
            public int compare(Person person1, Person person2) {
```

```

        int num = person1.getAge() - person2.getAge();
        return Integer.compare(num, 0);
    }
});
treeMap.put(new Person(3), "person1");
treeMap.put(new Person(18), "person2");
treeMap.put(new Person(35), "person3");
treeMap.put(new Person(16), "person4");
treeMap.entrySet().stream().forEach(personStringEntry -> {
    System.out.println(personStringEntry.getValue());
});
}
}

```

输出:

```

person1
person4
person2
person3

```

可以看出，`TreeMap` 中的元素已经是按照 `Person` 的 `age` 字段的升序来排列了。

上面，我们是通过传入匿名内部类的方式实现的，你可以将代码替换成 Lambda 表达式实现的方式：

```

TreeMap<Person, String> treeMap = new TreeMap<>((person1, person2) -> {
    int num = person1.getAge() - person2.getAge();
    return Integer.compare(num, 0);
});

```

综上，相比于 `HashMap` 来说 `TreeMap` 主要多了对集合中的元素根据键排序的能力以及对集合内元素的搜索的能力。

HashSet 如何检查重复？

以下内容摘自我的 Java 启蒙书《Head first java》第二版：

当你把对象加入 HashSet 时，HashSet 会先计算对象的 hashCode 值来判断对象加入的位置，同时也会与其他加入的对象的 hashCode 值作比较，如果没有相符的 hashCode，HashSet 会假设对象没有重复出现。但是如果发现有相同 hashCode 值的对象，这时会调用 equals() 方法来检查 hashCode 相等的对象是否真的相同。如果两者相同，HashSet 就不会让加入操作成功。

在 JDK1.8 中，HashSet 的 add() 方法只是简单的调用了 HashMap 的 put() 方法，并且判断了一下返回值以确保是否有重复元素。直接看一下 HashSet 中的源码：

```
// Returns: true if this set did not already contain the specified element
// 返回值：当 set 中没有包含 add 的元素时返回真
public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}
```

而在 HashMap 的 putVal() 方法中也能看到如下说明：

```
// Returns : previous value, or null if none
// 返回值：如果插入位置没有元素返回null，否则返回上一个元素
final V putVal(int hash, K key, V value, boolean onlyIfAbsent,
               boolean evict) {
    ...
}
```

也就是说，在 JDK1.8 中，实际上无论 HashSet 中是否已经存在了某元素，HashSet 都会直接插入，只是会在 add() 方法的返回值处告诉我们插入前是否存在相同元素。

HashMap 的底层实现

JDK1.8 之前

JDK1.8 之前 `HashMap` 底层是 **数组和链表** 结合在一起使用也就是 **链表散列**。`HashMap` 通过 `key` 的 `hashCode` 经过扰动函数处理过后得到 `hash` 值，然后通过 `(n - 1) & hash` 判断当前元素存放的位置（这里的 `n` 指的是数组的长度），如果当前位置存在元素的话，就判断该元素与要存入的元素的 `hash` 值以及 `key` 是否相同，如果相同的话，直接覆盖，不相同就通过拉链法解决冲突。

所谓扰动函数指的就是 `HashMap` 的 `hash` 方法。使用 `hash` 方法也就是扰动函数是为了防止一些实现比较差的 `hashCode()` 方法 换句话说使用扰动函数之后可以减少碰撞。

JDK 1.8 `HashMap` 的 `hash` 方法源码:

JDK 1.8 的 `hash` 方法 相比于 JDK 1.7 `hash` 方法更加简化，但是原理不变。

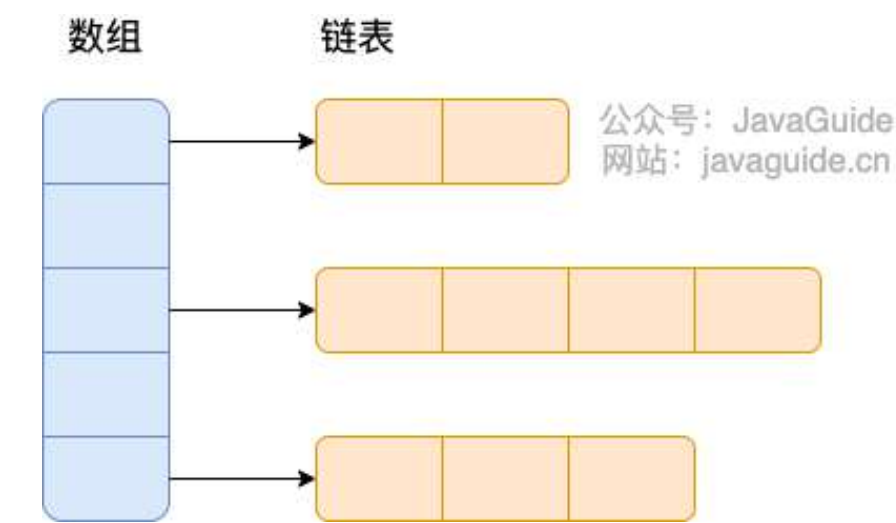
```
static final int hash(Object key) {  
    int h;  
    // key.hashCode(): 返回散列值也就是hashCode  
    // ^ : 按位异或  
    // >>>: 无符号右移，忽略符号位，空位都以0补齐  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

对比一下 JDK1.7 的 `HashMap` 的 `hash` 方法源码.

```
static int hash(int h) {  
    // This function ensures that hashCodes that differ only by  
    // constant multiples at each bit position have a bounded  
    // number of collisions (approximately 8 at default load factor).  
  
    h ^= (h >>> 20) ^ (h >>> 12);  
    return h ^ (h >>> 7) ^ (h >>> 4);  
}
```

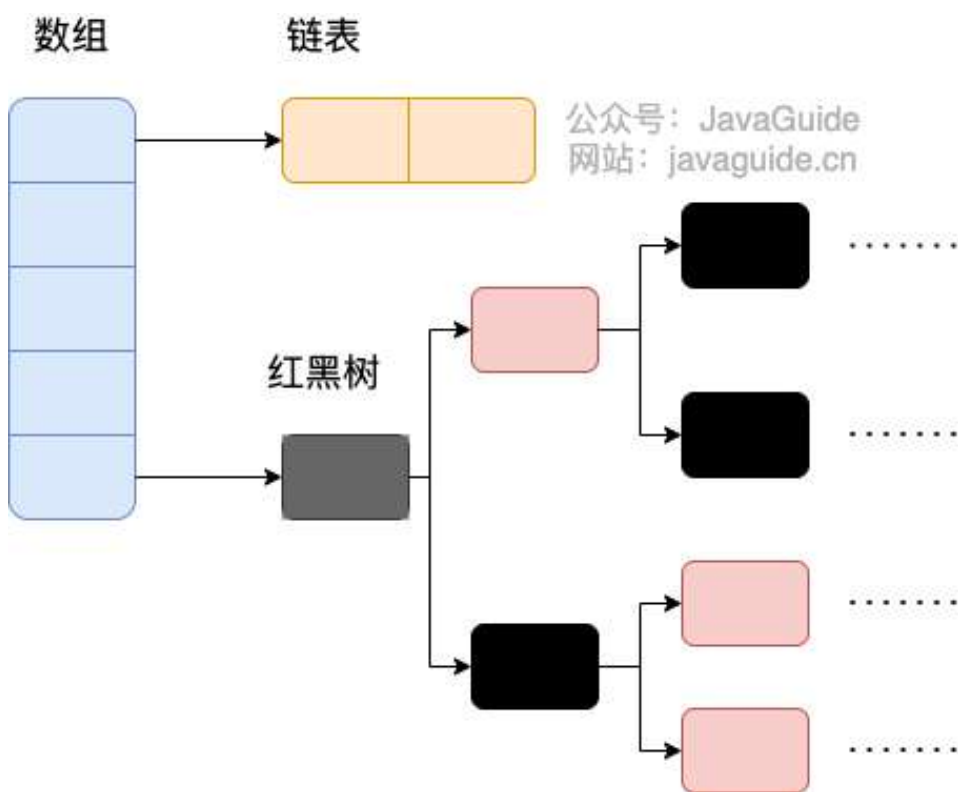
相比于 JDK1.8 的 `hash` 方法，JDK 1.7 的 `hash` 方法的性能会稍差一点点，因为毕竟扰动了 4 次。

所谓“拉链法”就是：将链表和数组相结合。也就是说创建一个链表数组，数组中每一格就是一个链表。若遇到哈希冲突，则将冲突的值加到链表中即可。



JDK1.8 之后

相比于之前的版本，JDK1.8 之后在解决哈希冲突时有了较大的变化，当链表长度大于阈值（默认为 8）（将链表转换成红黑树前会判断，如果当前数组的长度小于 64，那么会选择先进行数组扩容，而不是转换为红黑树）时，将链表转化为红黑树，以减少搜索时间。



TreeMap、TreeSet 以及 JDK1.8 之后的 HashMap 底层都用到了红黑树。红黑树就是为了解决二叉查找树的缺陷，因为二叉查找树在某些情况下会退化成一个线性结构。

我们来结合源码分析一下 `HashMap` 链表到红黑树的转换。

1、`putVal` 方法中执行链表转红黑树的判断逻辑。

链表的长度大于 8 的时候，就执行 `treeifyBin`（转换红黑树）的逻辑。

```
// 遍历链表
for (int binCount = 0; ; ++binCount) {
    // 遍历到链表最后一个节点
    if ((e = p.next) == null) {
        p.next = newNode(hash, key, value, null);
        // 如果链表元素个数大于等于TREEIFY_THRESHOLD (8)
        if (binCount >= TREEIFY_THRESHOLD - 1) // -1 for 1st
            // 红黑树转换（并不会直接转换成红黑树）
            treeifyBin(tab, hash);
        break;
    }
    if (e.hash == hash &&
        ((k = e.key) == key || (key != null && key.equals(k))))
        break;
    p = e;
}
```

2、`treeifyBin` 方法中判断是否真的转换为红黑树。

```
final void treeifyBin(Node<K,V>[] tab, int hash) {
    int n, index; Node<K,V> e;
    // 判断当前数组的长度是否小于 64
    if (tab == null || (n = tab.length) < MIN_TREEIFY_CAPACITY)
        // 如果当前数组的长度小于 64，那么会选择先进行数组扩容
        resize();
    else if ((e = tab[index = (n - 1) & hash]) != null) {
        // 否则才将列表转换为红黑树

        TreeNode<K,V> hd = null, tl = null;
        do {
            TreeNode<K,V> p = replacementTreeNode(e, null);
```

```

        if (tl == null)
            hd = p;
        else {
            p.prev = tl;
            tl.next = p;
        }
        tl = p;
    } while ((e = e.next) != null);
    if ((tab[index] = hd) != null)
        hd.treeify(tab);
}
}

```

将链表转换成红黑树前会判断，如果当前数组的长度小于 64，那么会选择先进行数组扩容，而不是转换为红黑树。

HashMap 的长度为什么是 2 的幂次方

为了能让 HashMap 存取高效，尽量较少碰撞，也就是要尽量把数据分配均匀。我们上面也讲到了过了，Hash 值的范围值-2147483648 到 2147483647，前后加起来大概 40 亿的映射空间，只要哈希函数映射得比较均匀松散，一般应用是很难出现碰撞的。但问题是一个 40 亿长度的数组，内存是放不下的。所以这个散列值是不能直接拿来用的。用之前还要先做对数组的长度取模运算，得到的余数才能用来要存放的位置也就是对应的数组下标。这个数组下标的计算方法是“ $(n - 1) \& \text{hash}$ ”。（n 代表数组长度）。这也就解释了 HashMap 的长度为什么是 2 的幂次方。

这个算法应该如何设计呢？

我们首先可能会想到采用%取余的操作来实现。但是，重点来了：“取余(%)操作中如果除数是 2 的幂次则等价于与其除数减一的与(&)操作（也就是说 $\text{hash} \% \text{length} == \text{hash} \& (\text{length} - 1)$ 的前提是 length 是 2 的 n 次方；）。”并且采用二进制位操作 &，相对于%能够提高运算效率，这就解释了 HashMap 的长度为什么是 2 的幂次方。

HashMap 多线程操作导致死循环问题

主要原因在于并发下的 Rehash 会造成元素之间会形成一个循环链表。不过，jdk 1.8 后解决了这个问题，但是还是不建议在多线程下使用 HashMap，因为多线程下使用 HashMap 还是会存在其他问题比如数据丢失。并发环境下推荐使用 ConcurrentHashMap。

详情请查看：<https://coolshell.cn/articles/9606.html>

HashMap 有哪几种常见的遍历方式？

HashMap 的 7 种遍历方式与性能分析！

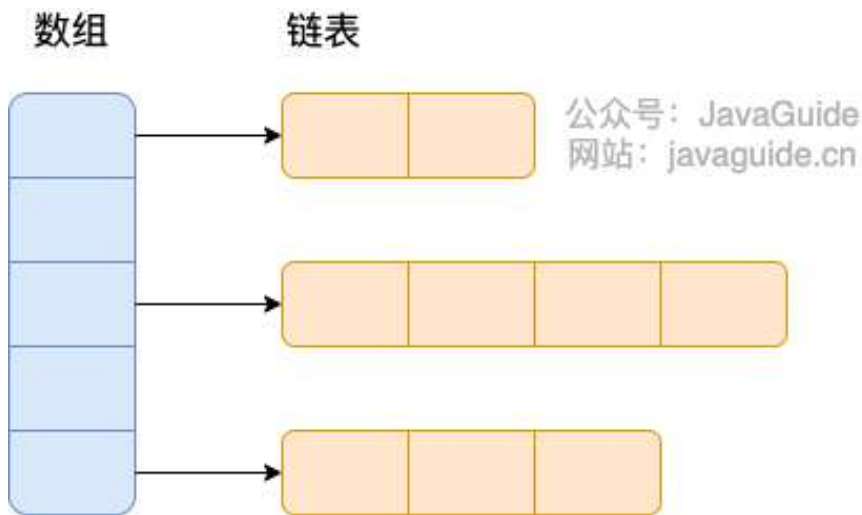
ConcurrentHashMap 和 Hashtable 的区别

ConcurrentHashMap 和 Hashtable 的区别主要体现在实现线程安全的方式上不同。

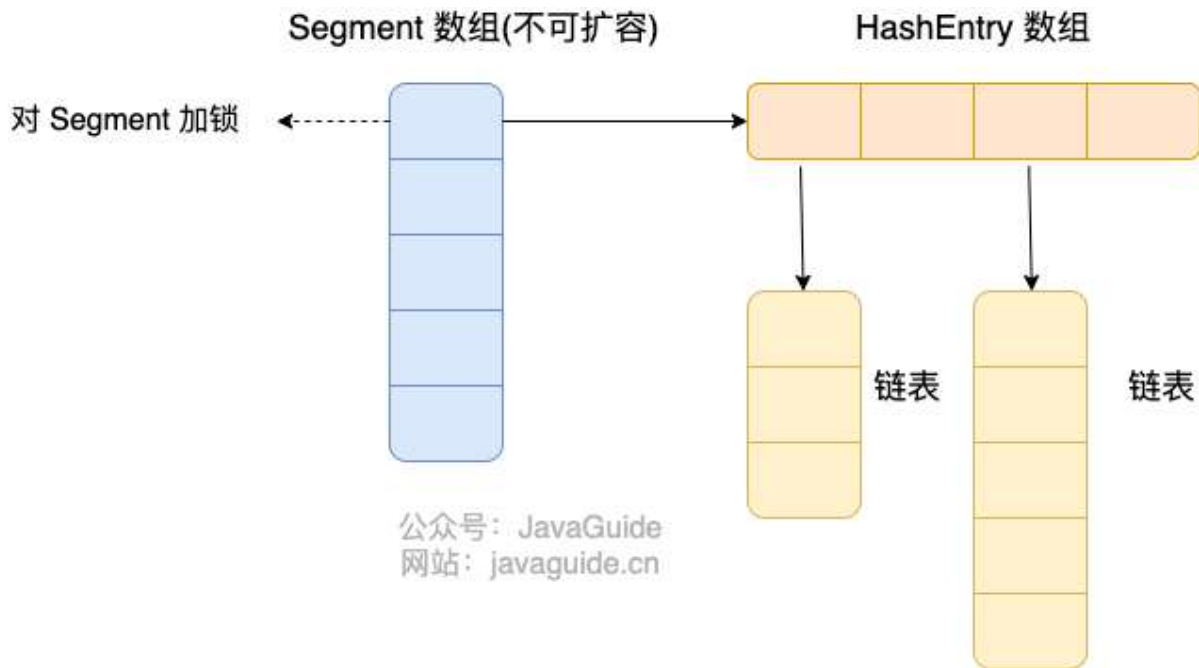
- **底层数据结构：** JDK1.7 的 ConcurrentHashMap 底层采用 **分段的数组+链表** 实现，JDK1.8 采用的数据结构跟 HashMap1.8 的结构一样，数组+链表/红黑二叉树。 Hashtable 和 JDK1.8 之前的 HashMap 的底层数据结构类似都是采用 **数组+链表** 的形式，数组是 HashMap 的主体，链表则是主要为了解决哈希冲突而存在的；
- **实现线程安全的方式（重要）：**
 - 在 JDK1.7 的时候， ConcurrentHashMap 对整个桶数组进行了分割分段(Segment ，分段锁)，每一把锁只锁容器其中一部分数据（下面有示意图），多线程访问容器里不同数据段的数据，就不会存在锁竞争，提高并发访问率。
 - 到了 JDK1.8 的时候， ConcurrentHashMap 已经摒弃了 Segment 的概念，而是直接用 Node 数组+链表+红黑树的数据结构来实现，并发控制使用 synchronized 和 CAS 来操作。（JDK1.6 以后 synchronized 锁做了很多优化）整个看起来就像是优化过且线程安全的 HashMap ，虽然在 JDK1.8 中还能看到 Segment 的数据结构，但是已经简化了属性，只是为了兼容旧版本；
 - **Hashtable (同一把锁)** :使用 synchronized 来保证线程安全，效率非常低下。当一个线程访问同步方法时，其他线程也访问同步方法，可能会进入阻塞或轮询状态，如使用 put 添加元素，另一个线程不能使用 put 添加元素，也不能使用 get，竞争会越来越激烈效率越低。

下面，我们再来看看两者底层数据结构的对比图。

Hashtable :



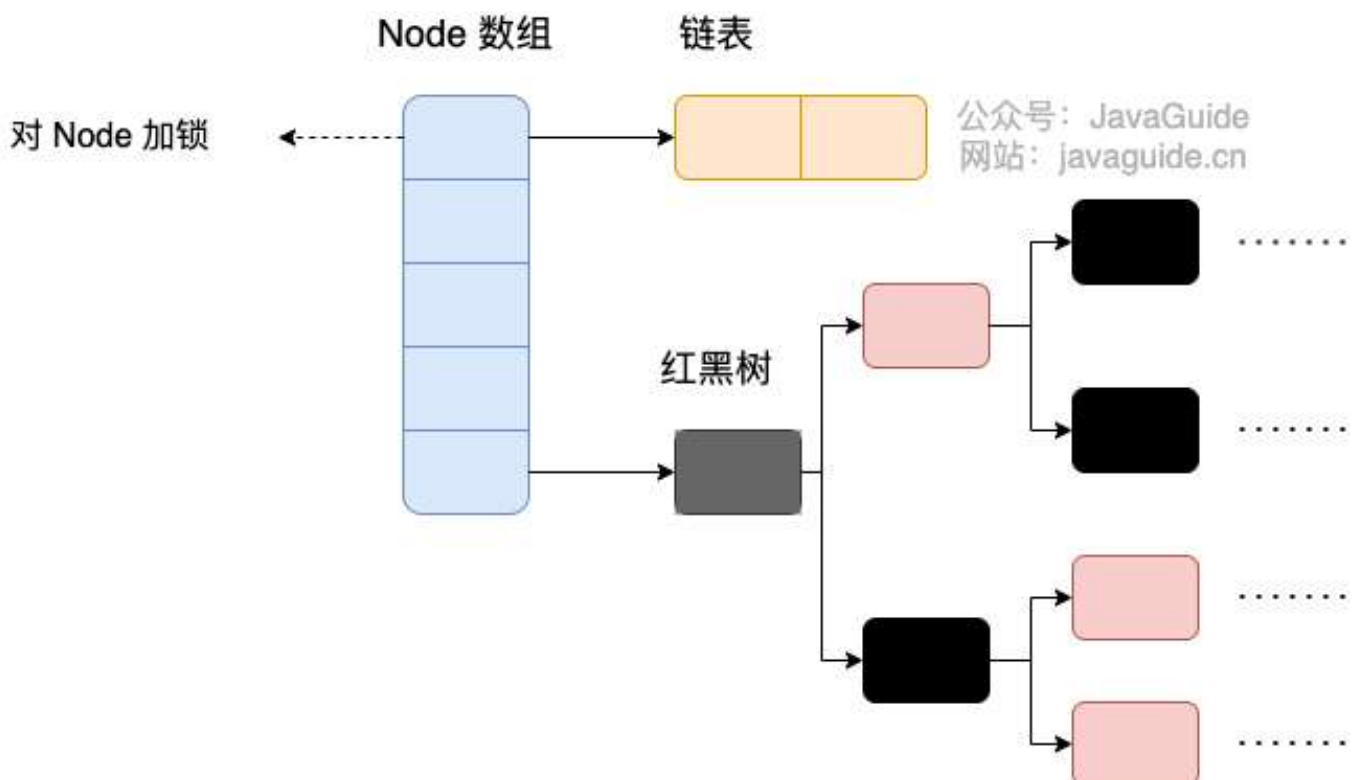
JDK1.7 的 ConcurrentHashMap :



ConcurrentHashMap 是由 Segment 数组结构和 HashEntry 数组结构组成。

Segment 数组中的每个元素包含一个 HashEntry 数组，每个 HashEntry 数组属于链表结构。

JDK1.8 的 ConcurrentHashMap :



JDK1.8 的 `ConcurrentHashMap` 不再是 **Segment 数组 + HashEntry 数组 + 链表**，而是 **Node 数组 + 链表 / 红黑树**。不过，Node 只能用于链表的情况，红黑树的情况需要使用 `TreeNode`。当冲突链表达到一定长度时，链表会转换成红黑树。

`TreeNode` 是存储红黑树节点，被 `TreeBin` 包装。`TreeBin` 通过 `root` 属性维护红黑树的根结点，因为红黑树在旋转的时候，根结点可能会被它原来的子节点替换掉，在这个时间点，如果有其他线程要写这棵红黑树就会发生线程不安全问题，所以在 `ConcurrentHashMap` 中 `TreeBin` 通过 `waiter` 属性维护当前使用这棵红黑树的线程，来防止其他线程的进入。

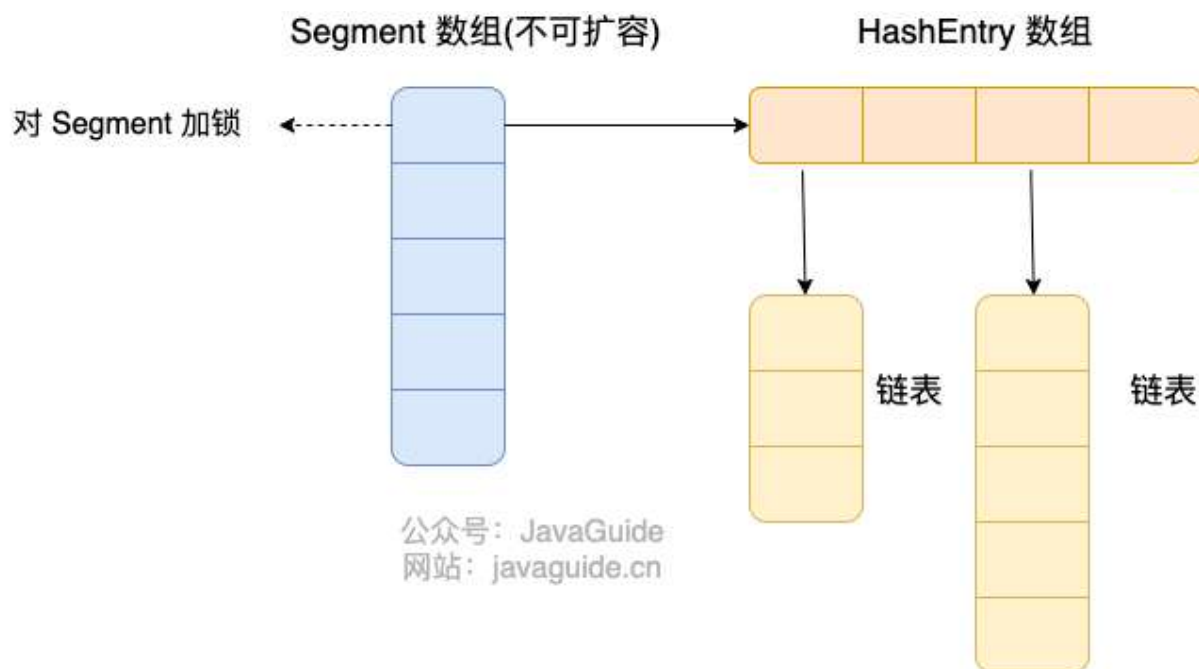
```
static final class TreeBin<K,V> extends Node<K,V> {
    TreeNode<K,V> root;

    volatile TreeNode<K,V> first;
    volatile Thread waiter;
    volatile int lockState;
    // values for lockState
    static final int WRITER = 1; // set while holding write lock
    static final int WAITER = 2; // set when waiting for write lock
    static final int READER = 4; // increment value for setting read lock

    ...
}
```

ConcurrentHashMap 线程安全的具体实现方式/底层具体实现

JDK1.8 之前



首先将数据分为一段一段（这个“段”就是 Segment）的存储，然后给每一段数据配一把锁，当一个线程占用锁访问其中一个段数据时，其他段的数据也能被其他线程访问。

ConcurrentHashMap 是由 Segment 数组结构和 HashEntry 数组结构组成。

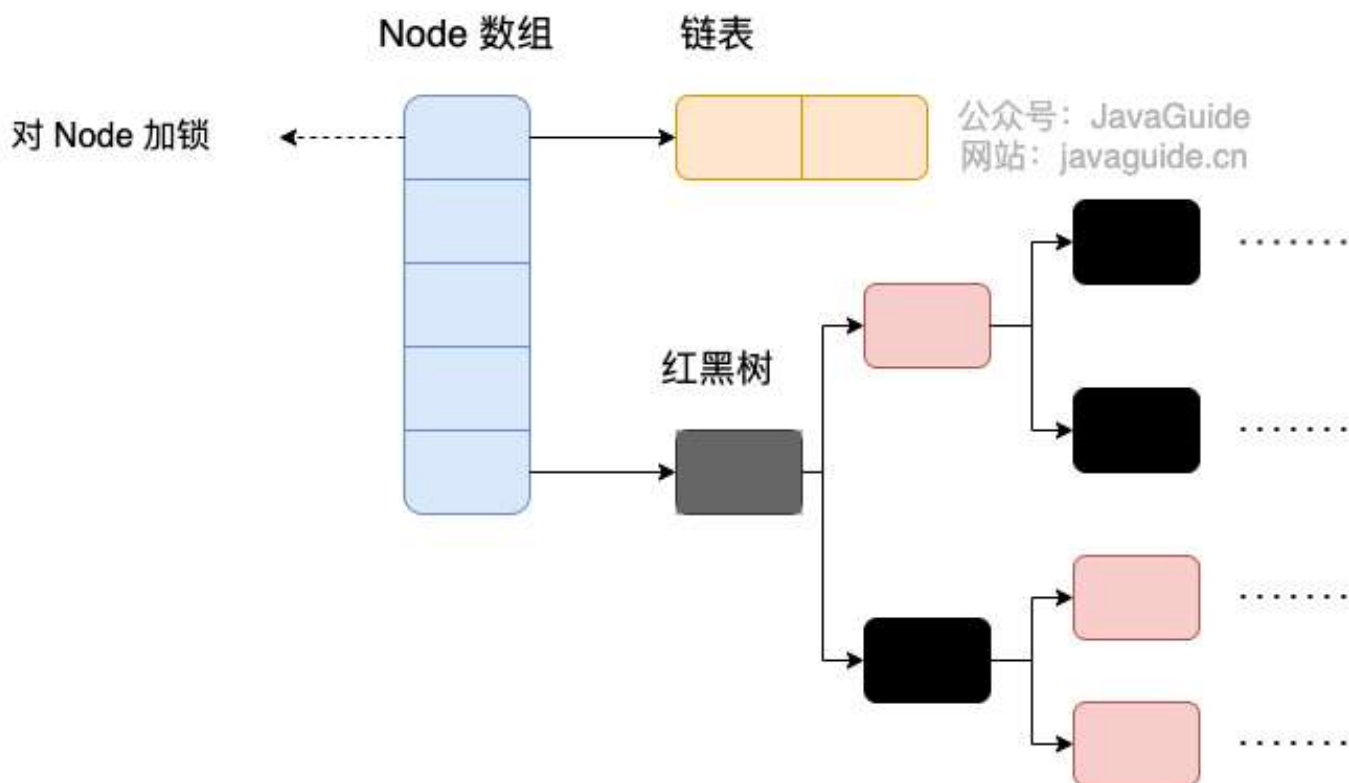
Segment 继承了 ReentrantLock，所以 Segment 是一种可重入锁，扮演锁的角色。HashEntry 用于存储键值对数据。

```
static class Segment<K,V> extends ReentrantLock implements Serializable {  
    }  
}
```

一个 ConcurrentHashMap 里包含一个 Segment 数组，Segment 的个数一旦初始化就不能改变。Segment 数组的大小默认是 16，也就是说默认可以同时支持 16 个线程并发写。

Segment 的结构和 HashMap 类似，是一种数组和链表结构，一个 Segment 包含一个 HashEntry 数组，每个 HashEntry 是一个链表结构的元素，每个 Segment 守护着一个 HashEntry 数组里的元素，当对 HashEntry 数组的数据进行修改时，必须首先获得对应的 Segment 的锁。也就是说，对同一 Segment 的并发写入会被阻塞，不同 Segment 的写入是可以并发执行的。

JDK1.8 之后



Java 8 几乎完全重写了 `ConcurrentHashMap`，代码量从原来 Java 7 中的 1000 多行，变成了现在的 6000 多行。

`ConcurrentHashMap` 取消了 `Segment` 分段锁，采用 `Node + CAS + synchronized` 来保证并发安全。数据结构跟 `HashMap` 1.8 的结构类似，数组+链表/红黑二叉树。Java 8 在链表长度超过一定阈值（8）时将链表（寻址时间复杂度为 $O(N)$ ）转换为红黑树（寻址时间复杂度为 $O(\log(N))$ ）。

Java 8 中，锁粒度更细，`synchronized` 只锁定当前链表或红黑二叉树的首节点，这样只要 hash 不冲突，就不会产生并发，就不会影响其他 Node 的读写，效率大幅提升。

JDK 1.7 和 JDK 1.8 的 `ConcurrentHashMap` 实现有什么不同？

- **线程安全实现方式**：JDK 1.7 采用 `Segment` 分段锁来保证安全，`Segment` 是继承自 `ReentrantLock`。JDK1.8 放弃了 `Segment` 分段锁的设计，采用 `Node + CAS + synchronized` 保证线程安全，锁粒度更细，`synchronized` 只锁定当前链表或红黑二叉树的首节点。
- **Hash 碰撞解决方法**：JDK 1.7 采用拉链法，JDK1.8 采用拉链法结合红黑树（链表长度超过一定阈值时，将链表转换为红黑树）。
- **并发度**：JDK 1.7 最大并发度是 `Segment` 的个数，默认是 16。JDK 1.8 最大并发度是 Node 数组的大小，并发度更大。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

2.3. 多线程

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

这部分内容摘自 **JavaGuide** 下面几篇文章：

- [Java 并发常见面试题总结（上）](#)
- [Java 并发常见面试题总结（中）](#)
- [Java 并发常见面试题总结（下）](#)

什么是线程和进程？

何为进程？

进程是程序的一次执行过程，是系统运行程序的基本单位，因此进程是动态的。系统运行一个程序即是一个进程从创建，运行到消亡的过程。

在 Java 中，当我们启动 main 函数时其实就是启动了一个 JVM 的进程，而 main 函数所在的线程就是这个进程中的一个线程，也称主线程。

如下图所示，在 Windows 中通过查看任务管理器的方式，我们就可以清楚看到 Windows 当前运行的进程（.exe 文件的运行）。

任务管理器

文件(F) 选项(O) 查看(V)

进程 性能 应用历史记录 启动 用户 详细信息 服务

名称	状态	11% CPU	57% 内存	1% 磁盘	0% 网络
应用 (6)					
> Google Chrome (20)		2.4%	1,177.3	0.1 MB/秒	0 Mbps
> Microsoft OneNote		0%	13.5 MB	0 MB/秒	0 Mbps
> TIM (32 位)		0.2%	85.5 MB	0 MB/秒	0 Mbps
> WeChat (32 位) (3)		4.7%	109.4 MB	0.1 MB/秒	0 Mbps
> 金山PDF (32 位)		0%	192.0 MB	0 MB/秒	0 Mbps
> 任务管理器		0.4%	24.1 MB	0 MB/秒	0 Mbps
后台进程 (71)					
> 64-bit Synaptics Pointing Enh...		0%	0.1 MB	0 MB/秒	0 Mbps
Application Frame Host		0%	0.1 MB	0 MB/秒	0 Mbps
BaiduNetdisk (32 位)		0%	17.3 MB	0 MB/秒	0 Mbps
BaiduNetdiskHost (32 位)		0%	0.1 MB	0 MB/秒	0 Mbps
COM Surrogate		0%	1.0 MB	0 MB/秒	0 Mbps

<

>

简略信息(D)

结束任务(E)

何为线程？

线程与进程相似，但线程是一个比进程更小的执行单位。一个进程在其执行的过程中可以产生多个线程。与进程不同的是同类的多个线程共享进程的堆和方法区资源，但每个线程有自己的程序计数器、虚拟机栈和本地方法栈，所以系统在产生一个线程，或是在各个线程之间作切换工作时，负担要比进程小得多，也正因为如此，线程也被称为轻量级进程。

Java 程序天生就是多线程程序，我们可以通过 JMX 来看看一个普通的 Java 程序有哪些线程，代码如下。

```

public class MultiThread {
    public static void main(String[] args) {
        // 获取 Java 线程管理 MBean
        ThreadMXBean threadMXBean = ManagementFactory.getThreadMXBean();
        // 不需要获取同步的 monitor 和 synchronizer 信息，仅获取线程和线程堆栈信息
        ThreadInfo[] threadInfos = threadMXBean.dumpAllThreads(false, false);
        // 遍历线程信息，仅打印线程 ID 和线程名称信息
        for (ThreadInfo threadInfo : threadInfos) {
            System.out.println "[" + threadInfo.getThreadId() + " ] " +
threadInfo.getThreadName());
        }
    }
}

```

上述程序输出如下（输出内容可能不同，不用太纠结下面每个线程的作用，只用知道 main 线程执行 main 方法即可）：

```

[5] Attach Listener //添加事件
[4] Signal Dispatcher // 分发处理给 JVM 信号的线程
[3] Finalizer //调用对象 finalize 方法的线程
[2] Reference Handler //清除 reference 线程
[1] main //main 线程,程序入口

```

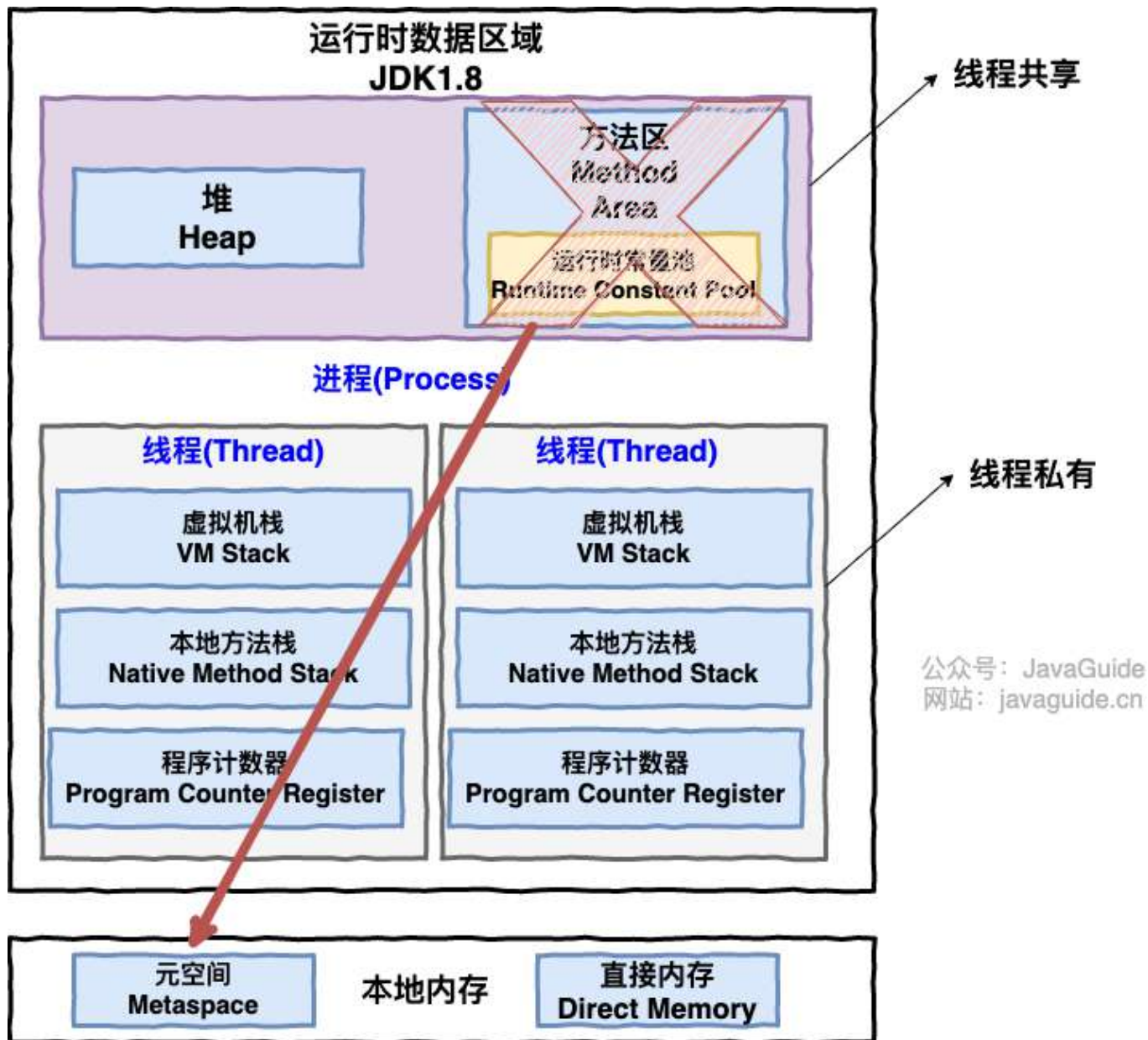
从上面的输出内容可以看出：一个 Java 程序的运行是 main 线程和多个其他线程同时运行。

请简要描述线程与进程的关系,区别及优缺点?

从 JVM 角度说进程和线程之间的关系。

图解进程和线程的关系

下图是 Java 内存区域，通过下图我们从 JVM 的角度来说一下线程和进程之间的关系。



从上图可以看出：一个进程中可以有多个线程，多个线程共享进程的堆和方法区 (JDK1.8 之后的元空间)资源，但是每个线程有自己的程序计数器、虚拟机栈 和 本地方法栈。

总结：线程是进程划分成的更小的运行单位。线程和进程最大的不同在于基本上各进程是独立的，而各线程则不一定，因为同一进程中的线程极有可能会相互影响。线程执行开销小，但不利于资源的管理和保护；而进程正相反。

下面是该知识点的扩展内容！

下面来思考这样一个问题：为什么程序计数器、虚拟机栈和本地方法栈是线程私有的呢？为什么堆和方法区是线程共享的呢？

程序计数器为什么是私有的？

程序计数器主要有下面两个作用：

1. 字节码解释器通过改变程序计数器来依次读取指令，从而实现代码的流程控制，如：顺序执行、选择、循环、异常处理。
2. 在多线程的情况下，程序计数器用于记录当前线程执行的位置，从而当线程被切换回来的时候能够知道该线程上次运行到哪儿了。

需要注意的是，如果执行的是 native 方法，那么程序计数器记录的是 undefined 地址，只有执行的是 Java 代码时程序计数器记录的才是下一条指令的地址。

所以，程序计数器私有主要是为了线程切换后能恢复到正确的执行位置。

虚拟机栈和本地方法栈为什么是私有的？

- **虚拟机栈：** 每个 Java 方法在运行的同时会创建一个栈帧用于存储局部变量表、操作数栈、常量池引用等信息。从方法调用直至执行完成的过程，就对应着一个栈帧在 Java 虚拟机栈中入栈和出栈的过程。
- **本地方法栈：** 和虚拟机栈所发挥的作用非常相似，区别是：**虚拟机栈为虚拟机执行 Java 方法（也就是字节码）服务，而本地方法栈则为虚拟机使用到的 Native 方法服务。** 在 HotSpot 虚拟机中和 Java 虚拟机栈合二为一。

所以，为了保证线程中的局部变量不被别的线程访问到，虚拟机栈和本地方法栈是线程私有的。

一句话简单了解堆和方法区

堆和方法区是所有线程共享的资源，其中堆是进程中最大的一块内存，主要用于存放新创建的对象（几乎所有对象都在这里分配内存），方法区主要用于存放已被加载的类信息、常量、静态变量、即时编译器编译后的代码等数据。

并发与并行的区别

- **并发：** 两个及两个以上的作业在同一 **时间段** 内执行。
- **并行：** 两个及两个以上的作业在同一 **时刻** 执行。

最关键的点是：是否是 **同时** 执行。

同步和异步的区别

- **同步**：发出一个调用之后，在没有得到结果之前，该调用就不可以返回，一直等待。
- **异步**：调用在发出之后，不用等待返回结果，该调用直接返回。

为什么要使用多线程呢？

先从总体上来说：

- **从计算机底层来说**：线程可以比作是轻量级的进程，是程序执行的最小单位,线程间的切换和调度的成本远远小于进程。另外，多核 CPU 时代意味着多个线程可以同时运行，这减少了线程上下文切换的开销。
- **从当代互联网发展趋势来说**：现在的系统动不动就要求百万级甚至千万级的并发量，而多线程并发编程正是开发高并发系统的基础，利用好多线程机制可以大大提高系统整体的并发能力以及性能。

再深入到计算机底层来探讨：

- **单核时代**：在单核时代多线程主要是为了提高单进程利用 CPU 和 IO 系统的效率。假设只运行了一个 Java 进程的情况，当我们请求 IO 的时候，如果 Java 进程中只有一个线程，此线程被 IO 阻塞则整个进程被阻塞。CPU 和 IO 设备只有一个在运行，那么可以简单地说系统整体效率只有 50%。当使用多线程的时候，一个线程被 IO 阻塞，其他线程还可以继续使用 CPU。从而提高了 Java 进程利用系统资源的整体效率。
- **多核时代**：多核时代多线程主要是为了提高进程利用多核 CPU 的能力。举个例子：假如我们要计算一个复杂的任务，我们只用一个线程的话，不论系统有几个 CPU 核心，都只会有一个 CPU 核心被利用到。而创建多个线程，这些线程可以被映射到底层多个 CPU 上执行，在任务中的多个线程没有资源竞争的情况下，任务执行的效率会有显著性的提高，约等于（单核时执行时间/CPU 核心数）。

使用多线程可能带来什么问题？

并发编程的目的就是为了能提高程序的执行效率提高程序运行速度，但是并发编程并不总是能提高程序运行速度的，而且并发编程可能会遇到很多问题，比如：内存泄漏、死锁、线程不安全等等。

说说线程的生命周期和状态？

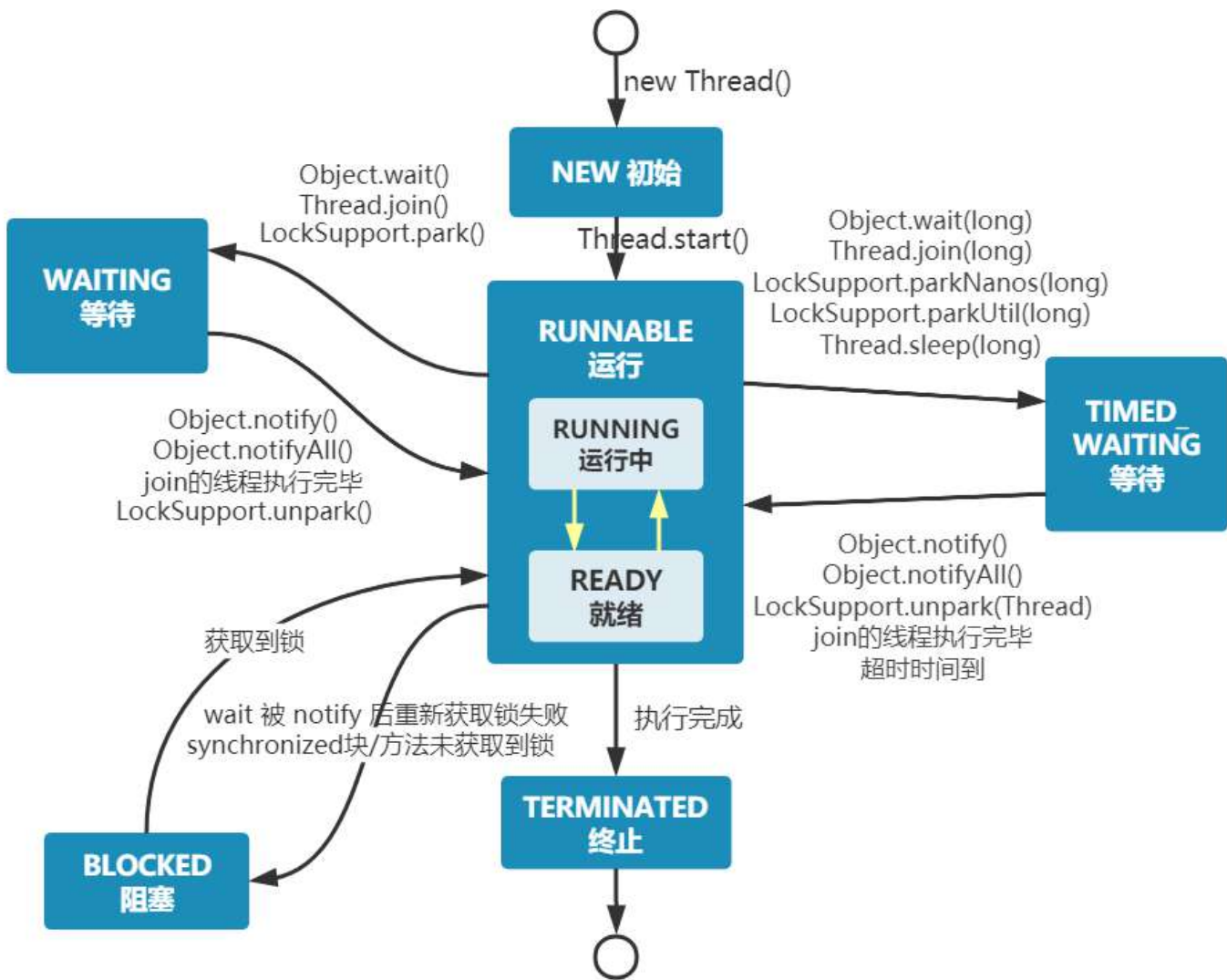
Java 线程在运行的生命周期中的指定时刻只可能处于下面 6 种不同状态的其中一个状态：

- **NEW**: 初始状态，线程被创建出来但没有被调用 `start()`。
- **RUNNABLE**: 运行状态，线程被调用了 `start()` 等待运行的状态。

- **BLOCKED**：阻塞状态，需要等待锁释放。
- **WAITING**：等待状态，表示该线程需要等待其他线程做出一些特定动作（通知或中断）。
- **TIME_WAITING**：超时等待状态，可以在指定的时间后自行返回而不是像 **WAITING** 那样一直等待。
- **TERMINATED**：终止状态，表示该线程已经运行完毕。

线程在生命周期中并不是固定处于某一个状态而是随着代码的执行在不同状态之间切换。

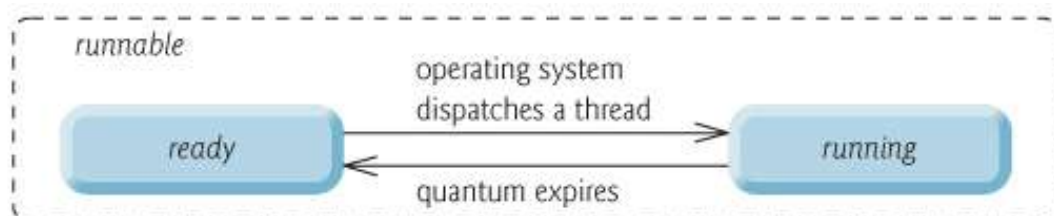
Java 线程状态变迁图(图源：[挑错 I 《Java 并发编程的艺术》中关于线程状态的三处错误](#)):



由上图可以看出：线程创建之后它将处于 **NEW（新建）** 状态，调用 `start()` 方法后开始运行，线程这时候处于 **READY（可运行）** 状态。可运行状态的线程获得了 CPU 时间片（timeslice）后就处于 **RUNNING（运行）** 状态。

在操作系统层面，线程有 **READY** 和 **RUNNING** 状态；而在 JVM 层面，只能看到 **RUNNABLE** 状态（图源：[HowToDoInJava: Java Thread Life Cycle and Thread States](#)），所以 Java 系统一般将这两个状态统称为 **RUNNABLE（运行中）** 状态。

为什么 JVM 没有区分这两种状态呢？（摘自：[Java 线程运行怎么有第六种状态？ - Dawell 的回答](#)）现在的时分（time-sharing）多任务（multi-task）操作系统架构通常都是用所谓的“时间分片（time quantum or time slice）”方式进行抢占式（preemptive）轮转调度（round-robin 式）。这个时间分片通常是很小的，一个线程一次最多只能在 CPU 上运行比如 10-20ms 的时间（此时处于 running 状态），也即大概只有 0.01 秒这一量级，时间片用后就要被切换下来放入调度队列的末尾等待再次调度。（也即回到 ready 状态）。线程切换的如此之快，区分这两种状态就没什么意义了。



- 当线程执行 `wait()` 方法之后，线程进入 **WAITING（等待）** 状态。进入等待状态的线程需要依靠其他线程的通知才能够返回到运行状态。
- **TIMED_WAITING(超时等待)** 状态相当于在等待状态的基础上增加了超时限制，比如通过 `sleep(long millis)` 方法或 `wait(long millis)` 方法可以将线程置于 **TIMED_WAITING** 状态。当超时时间结束后，线程将会返回到 **RUNNABLE** 状态。
- 当线程进入 `synchronized` 方法/块或者调用 `wait` 后（被 `notify`）重新进入 `synchronized` 方法/块，但是锁被其它线程占有，这个时候线程就会进入 **BLOCKED（阻塞）** 状态。
- 线程在执行完了 `run()` 方法之后将会进入到 **TERMINATED（终止）** 状态。

相关阅读：[线程的几种状态你真的了解么？](#)。

什么是上下文切换？

线程在执行过程中会有自己的运行条件和状态（也称上下文），比如上文所说到过的程序计数器，栈信息等。当出现如下情况的时候，线程会从占用 CPU 状态中退出。

- 主动让出 CPU，比如调用了 `sleep()`，`wait()` 等。
- 时间片用完，因为操作系统要防止一个线程或者进程长时间占用 CPU 导致其他线程或者进程饿死。
- 调用了阻塞类型的系统中断，比如请求 IO，线程被阻塞。
- 被终止或结束运行

这其中前三种都会发生线程切换，线程切换意味着需要保存当前线程的上下文，留待线程下次占用 CPU 的时候恢复现场。并加载下一个将要占用 CPU 的线程上下文。这就是所谓的 上下文切换。

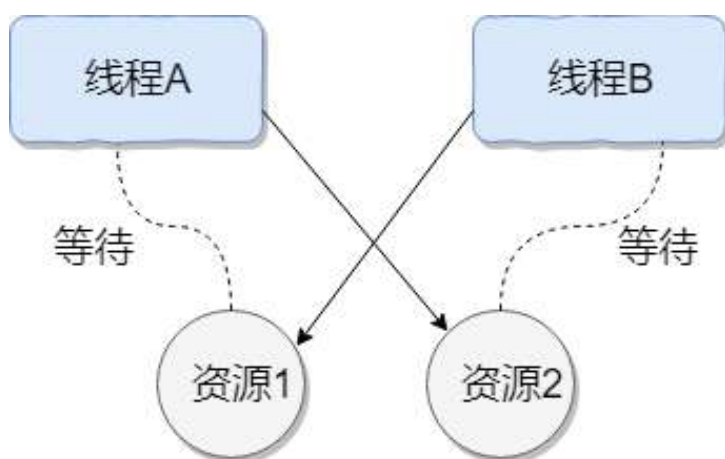
上下文切换是现代操作系统的基本功能，因其每次需要保存信息恢复信息，这将会占用 CPU，内存等系统资源进行处理，也就意味着效率会有一定损耗，如果频繁切换就会造成整体效率低下。

什么是线程死锁?如何避免死锁?

认识线程死锁

线程死锁描述的是这样一种情况：多个线程同时被阻塞，它们中的一个或者全部都在等待某个资源被释放。由于线程被无限期地阻塞，因此程序不可能正常终止。

如下图所示，线程 A 持有资源 2，线程 B 持有资源 1，他们同时都想申请对方的资源，所以这两个线程就会互相等待而进入死锁状态。



下面通过一个例子来说明线程死锁,代码模拟了上图的死锁的情况 (代码来源于《并发编程之美》):

```
public class DeadLockDemo {  
    private static Object resource1 = new Object();//资源 1  
    private static Object resource2 = new Object();//资源 2  
  
    public static void main(String[] args) {  
        new Thread(() -> {  
            synchronized (resource1) {  
                System.out.println(Thread.currentThread() + "get resource1");  
                try {  
                    Thread.sleep(1000);  
                } catch (InterruptedException e) {  

```

```

        e.printStackTrace();
    }

    System.out.println(Thread.currentThread() + "waiting get
resource2");

    synchronized (resource2) {
        System.out.println(Thread.currentThread() + "get
resource2");
    }
}

}, "线程 1").start();

new Thread(() -> {
    synchronized (resource2) {
        System.out.println(Thread.currentThread() + "get resource2");
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(Thread.currentThread() + "waiting get
resource1");

        synchronized (resource1) {
            System.out.println(Thread.currentThread() + "get
resource1");
        }
    }

}, "线程 2").start();
}
}

```

Output

```

Thread[线程 1,5,main]get resource1
Thread[线程 2,5,main]get resource2
Thread[线程 1,5,main]waiting get resource2
Thread[线程 2,5,main]waiting get resource1

```


线程 A 通过 `synchronized (resource1)` 获得 `resource1` 的监视器锁，然后通过 `Thread.sleep(1000);` 让线程 A 休眠 1s 为的是让线程 B 得到执行然后获取到 `resource2` 的监视器锁。线程 A 和线程 B 休眠结束了都开始企图请求获取对方的资源，然后这两个线程就会陷入互相等待的状态，这也就产生了死锁。

上面的例子符合产生死锁的四个必要条件：

1. 互斥条件：该资源任意一个时刻只由一个线程占用。
2. 请求与保持条件：一个线程因请求资源而阻塞时，对已获得的资源保持不放。
3. 不剥夺条件：线程已获得的资源在未使用完之前不能被其他线程强行剥夺，只有自己使用完毕后才释放资源。
4. 循环等待条件：若干线程之间形成一种头尾相接的循环等待资源关系。

如何预防和避免线程死锁？

如何预防死锁？破坏死锁的产生的必要条件即可：

1. 破坏请求与保持条件：一次性申请所有的资源。
2. 破坏不剥夺条件：占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。
3. 破坏循环等待条件：靠按序申请资源来预防。按某一顺序申请资源，释放资源则反序释放。破坏循环等待条件。

如何避免死锁？

避免死锁就是在资源分配时，借助于算法（比如银行家算法）对资源分配进行计算评估，使其进入安全状态。

安全状态 指的是系统能够按照某种线程推进顺序（`P1、P2、P3.....Pn`）来为每个线程分配所需资源，直到满足每个线程对资源的最大需求，使每个线程都可顺利完成。称 `<P1、P2、P3.....Pn>` 序列为安全序列。

我们对线程 2 的代码修改成下面这样就不会产生死锁了。

```
new Thread(() -> {
    synchronized (resource1) {
        System.out.println(Thread.currentThread() + "get resource1");
        try {
```



```

        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println(Thread.currentThread() + "waiting get
resource2");

    synchronized (resource2) {
        System.out.println(Thread.currentThread() + "get
resource2");
    }
}
}, "线程 2").start();

```

输出：

```

Thread[线程 1,5,main]get resource1
Thread[线程 1,5,main]waiting get resource2
Thread[线程 1,5,main]get resource2
Thread[线程 2,5,main]get resource1
Thread[线程 2,5,main]waiting get resource2
Thread[线程 2,5,main]get resource2

Process finished with exit code 0

```

我们分析一下上面的代码为什么避免了死锁的发生？

线程 1 首先获得到 resource1 的监视器锁,这时候线程 2 就获取不到了。然后线程 1 再去获取 resource2 的监视器锁，可以获取到。然后线程 1 释放了对 resource1、resource2 的监视器锁的占用，线程 2 获取到就可以执行了。这样就破坏了破坏循环等待条件，因此避免了死锁。

sleep() 方法和 wait() 方法对比

共同点：两者都可以暂停线程的执行。

区别：

- `sleep()` 方法没有释放锁，而 `wait()` 方法释放了锁。

- `wait()` 通常被用于线程间交互/通信, `sleep()` 通常被用于暂停执行。
- `wait()` 方法被调用后, 线程不会自动苏醒, 需要别的线程调用同一个对象上的 `notify()` 或者 `notifyAll()` 方法。 `sleep()` 方法执行完成后, 线程会自动苏醒, 或者也可以使用 `wait(long timeout)` 超时后线程会自动苏醒。
- `sleep()` 是 `Thread` 类的静态本地方法, `wait()` 则是 `Object` 类的本地方法。为什么这样设计呢?

为什么 `wait()` 方法不定义在 `Thread` 中?

`wait()` 是让获得对象锁的线程实现等待, 会自动释放当前线程占有的对象锁。每个对象 (`Object`) 都拥有对象锁, 既然要释放当前线程占有的对象锁并让其进入 `WAITING` 状态, 自然是要操作对应的对象 (`Object`) 而非当前的线程 (`Thread`) 。

类似的问题: 为什么 `sleep()` 方法定义在 `Thread` 中?

因为 `sleep()` 是让当前线程暂停执行, 不涉及到对象类, 也不需要获得对象锁。

可以直接调用 `Thread` 类的 `run` 方法吗?

这是另一个非常经典的 Java 多线程面试问题, 而且在面试中会经常被问到。很简单, 但是很多人都会答不上来!

`new` 一个 `Thread`, 线程进入了新建状态。调用 `start()` 方法, 会启动一个线程并使线程进入了就绪状态, 当分配到时间片后就可以开始运行了。 `start()` 会执行线程的相应准备工作, 然后自动执行 `run()` 方法的内容, 这是真正的多线程工作。但是, 直接执行 `run()` 方法, 会把 `run()` 方法当成一个 `main` 线程下的普通方法去执行, 并不会在某个线程中执行它, 所以这并不是多线程工作。

总结: 调用 `start()` 方法方可启动线程并使线程进入就绪状态, 直接执行 `run()` 方法的话不会以多线程的方式执行。

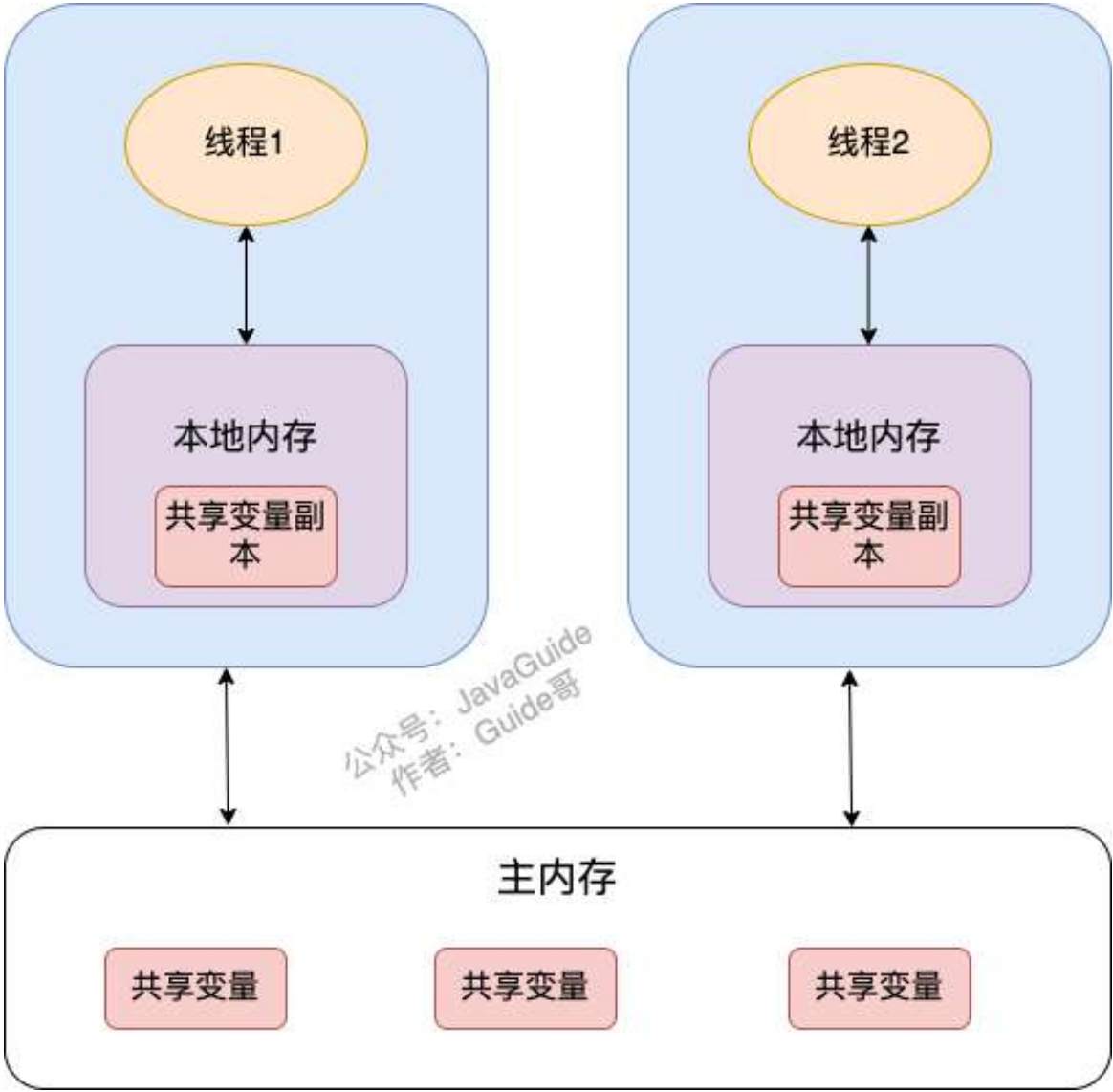
JMM(Java Memory Model)

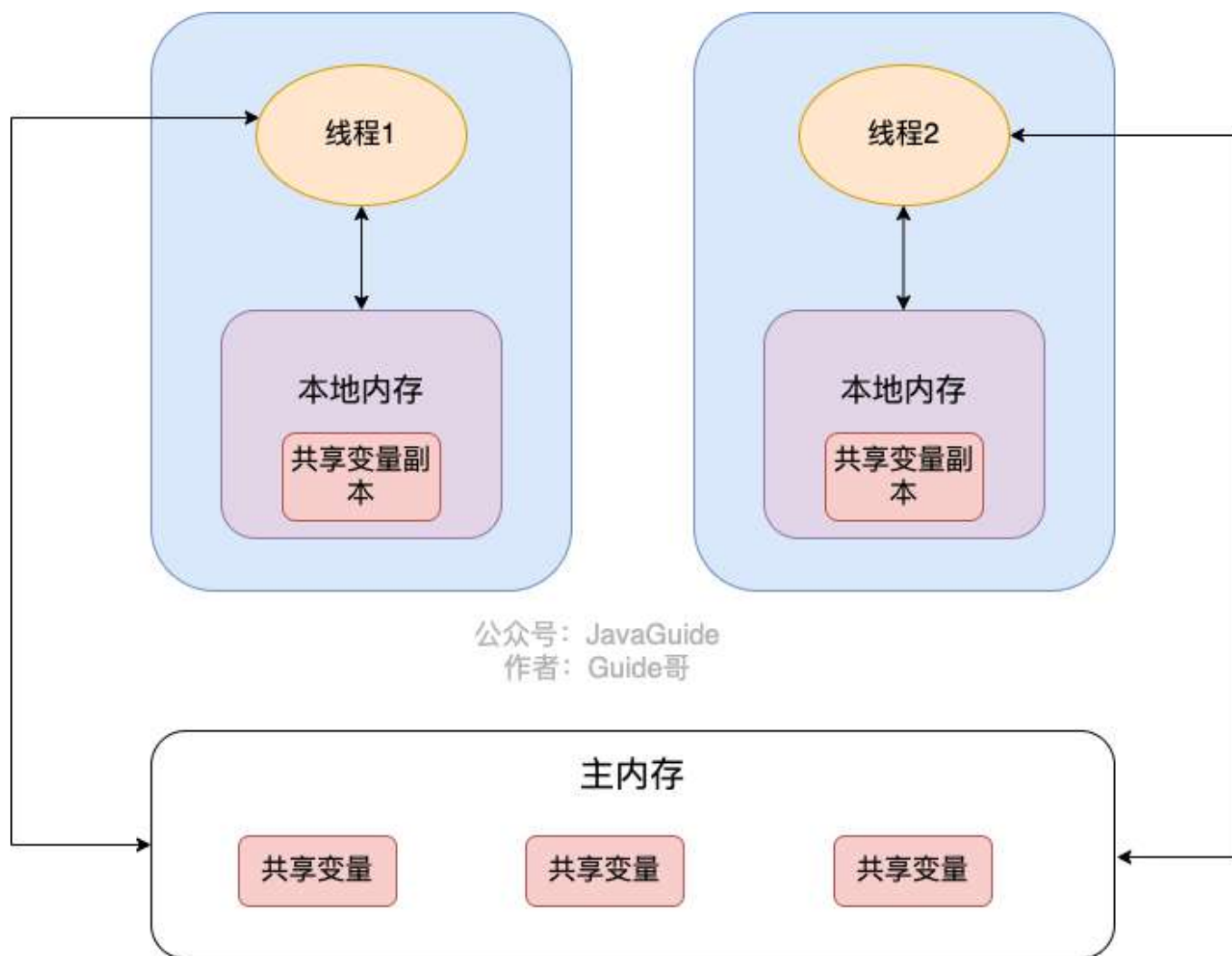
JMM (Java 内存模型) 相关的问题比较多, 也比较重要, 于是我单独抽了一篇文章来总结 JMM 相关的知识点和问题: [JMM \(Java 内存模型\) 详解](#)。

volatile 关键字

如何保证变量的可见性？

在 Java 中，`volatile` 关键字可以保证变量的可见性，如果我们将变量声明为 `volatile`，这就指示 JVM，这个变量是共享且不稳定的，每次使用它都到主存中进行读取。





`volatile` 关键字其实并非是 Java 语言特有的，在 C 语言里也有，它最原始的意义就是禁用 CPU 缓存。如果我们将一个变量使用 `volatile` 修饰，这就指示编译器，这个变量是共享且不稳定的，每次使用它都到主存中进行读取。

`volatile` 关键字能保证数据的可见性，但不能保证数据的原子性。`synchronized` 关键字两者都能保证。

如何禁止指令重排序？

在 Java 中，`volatile` 关键字除了可以保证变量的可见性，还有一个重要的作用就是防止 JVM 的指令重排序。如果我们将变量声明为 `volatile`，在对这个变量进行读写操作的时候，会通过插入特定的内存屏障的方式来禁止指令重排序。

在 Java 中，`Unsafe` 类提供了三个开箱即用的内存屏障相关的方法，屏蔽了操作系统底层的差异：

```
public native void loadFence();  
public native void storeFence();  
public native void fullFence();
```

理论上来说，你通过这三个方法也可以实现和 `volatile` 禁止重排序一样的效果，只是会麻烦一些。

下面我以一个常见的面试题为例讲解一下 `volatile` 关键字禁止指令重排序的效果。

面试中面试官经常会说：“单例模式了解吗？来给我手写一下！给我解释一下双重检验锁方式实现单例模式的原理呗！”

双重校验锁实现对象单例（线程安全）：

```
public class Singleton {  
  
    private volatile static Singleton uniqueInstance;  
  
    private Singleton() {  
    }  
  
    public static Singleton getUniqueInstance() {  
        //先判断对象是否已经实例过，没有实例化过才进入加锁代码  
        if (uniqueInstance == null) {  
            //类对象加锁  
            synchronized (Singleton.class) {  
                if (uniqueInstance == null) {  
                    uniqueInstance = new Singleton();  
                }  
            }  
        }  
        return uniqueInstance;  
    }  
}
```

`uniqueInstance` 采用 `volatile` 关键字修饰也是很有必要的，`uniqueInstance = new Singleton();` 这段代码其实是分为三步执行：

1. 为 `uniqueInstance` 分配内存空间
2. 初始化 `uniqueInstance`
3. 将 `uniqueInstance` 指向分配的内存地址

但是由于 JVM 具有指令重排的特性，执行顺序有可能变成 1->3->2。指令重排在单线程环境下不会出现问题，但是在多线程环境下会导致一个线程获得还没有初始化的实例。例如，线程 T1 执行了 1 和 3，此时 T2 调用 `getUniqueInstance()` 后发现 `uniqueInstance` 不为空，因此返回 `uniqueInstance`，但此时 `uniqueInstance` 还未被初始化。

volatile 可以保证原子性么？

`volatile` 关键字能保证变量的可见性，但不能保证对变量的操作是原子性的。

我们通过下面的代码即可证明：

```
/**
 * 微信搜 JavaGuide 回复"面试突击"即可免费领取个人原创的 Java 面试手册
 *
 * @author Guide哥
 * @date 2022/08/03 13:40
 **/

public class VolatoleAtomicityDemo {
    public volatile static int inc = 0;

    public void increase() {
        inc++;
    }

    public static void main(String[] args) throws InterruptedException {
        ExecutorService threadPool = Executors.newFixedThreadPool(5);
        VolatoleAtomicityDemo volatoleAtomicityDemo = new
VolatoleAtomicityDemo();
        for (int i = 0; i < 5; i++) {
            threadPool.execute(() -> {
                for (int j = 0; j < 500; j++) {
                    volatoleAtomicityDemo.increase();
                }
            });
        }
    }
}
```

```
// 等待1.5秒，保证上面程序执行完成
Thread.sleep(1500);
System.out.println(inc);
threadPool.shutdown();
}
}
```

正常情况下，运行上面的代码理应输出 2500。但你真正运行了上面的代码之后，你会发现每次输出结果都小于 2500。

为什么会出现这种情况呢？不是说好了，`volatile` 可以保证变量的可见性嘛！

也就是说，如果 `volatile` 能保证 `inc++` 操作的原子性的话。每个线程中对 `inc` 变量自增完之后，其他线程可以立即看到修改后的值。5 个线程分别进行了 500 次操作，那么最终 `inc` 的值应该是 $5 \times 500 = 2500$ 。

很多人会误认为自增操作 `inc++` 是原子性的，实际上，`inc++` 其实是一个复合操作，包括三步：

1. 读取 `inc` 的值。
2. 对 `inc` 加 1。
3. 将 `inc` 的值写回内存。

`volatile` 是无法保证这三个操作是具有原子性的，有可能导致下面这种情况出现：

1. 线程 1 对 `inc` 进行读取操作之后，还未对其进行修改。线程 2 又读取了 `inc` 的值并对其进行修改 (+1)，再将 `inc` 的值写回内存。
2. 线程 2 操作完毕后，线程 1 对 `inc` 的值进行修改 (+1)，再将 `inc` 的值写回内存。

这也就导致两个线程分别对 `inc` 进行了一次自增操作后，`inc` 实际上只增加了 1。

其实，如果想要保证上面的代码运行正确也非常简单，利用 `synchronized`、`Lock` 或者 `AtomicInteger` 都可以。

使用 `synchronized` 改进：


```
public synchronized void increase() {  
    inc++;  
}
```

使用 `AtomicInteger` 改进:

```
public AtomicInteger inc = new AtomicInteger();  
  
public void increase() {  
    inc.getAndIncrement();  
}
```

使用 `ReentrantLock` 改进:

```
Lock lock = new ReentrantLock();  
public void increase() {  
    lock.lock();  
    try {  
        inc++;  
    } finally {  
        lock.unlock();  
    }  
}
```



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“PDF”领取原创面试 PDF 手册
- 2、关注公众号回复“加群”进入面试交流群
- 3、关注公众号回复“八股文”获取大厂面试真题+面经

synchronized 关键字

说一说自己对于 synchronized 关键字的了解

synchronized 翻译成中文是同步的意思，主要解决的是多个线程之间访问资源的同步性，可以保证被它修饰的方法或者代码块在任意时刻只能有一个线程执行。

在 Java 早期版本中，synchronized 属于重量级锁，效率低下。因为监视器锁（monitor）是依赖于底层的操作系统的 Mutex Lock 来实现的，Java 的线程是映射到操作系统的原生线程之上的。如果要挂起或者唤醒一个线程，都需要操作系统帮忙完成，而操作系统实现线程之间的切换时需要从用户态转换到内核态，这个状态之间的转换需要相对比较长的时间，时间成本相对较高。

不过，在 Java 6 之后，Java 官方对从 JVM 层面对 synchronized 较大优化，所以现在的 synchronized 锁效率也优化得很不错了。JDK1.6 对锁的实现引入了大量的优化，如自旋锁、适应性自旋锁、锁消除、锁粗化、偏向锁、轻量级锁等技术来减少锁操作的开销。所以，你会发现目前的话，不论是各种开源框架还是 JDK 源码都大量使用了 synchronized 关键字。

如何使用 synchronized 关键字？

synchronized 关键字最主要的三种使用方式：

1. 修饰实例方法
2. 修饰静态方法
3. 修饰代码块

1、修饰实例方法（锁当前对象实例）

给当前对象实例加锁，进入同步代码前要获得 **当前对象实例的锁**。

```
synchronized void method() {  
    //业务代码  
}
```

2、修饰静态方法（锁当前类）

给当前类加锁，会作用于类的所有对象实例，进入同步代码前要获得 **当前 class 的锁**。

这是因为静态成员不属于任何一个实例对象，归整个类所有，不依赖于类的特定实例，被类的所有实例共享。

```
synchronized static void method() {  
    //业务代码  
}
```

静态 `synchronized` 方法和非静态 `synchronized` 方法之间的调用互斥么？不互斥！如果一个线程 A 调用一个实例对象的非静态 `synchronized` 方法，而线程 B 需要调用这个实例对象所属类的静态 `synchronized` 方法，是允许的，不会发生互斥现象，因为访问静态 `synchronized` 方法占用的锁是当前类的锁，而访问非静态 `synchronized` 方法占用的锁是当前实例对象锁。

3、修饰代码块（锁指定对象/类）

对括号里指定的对象/类加锁：

- `synchronized(object)` 表示进入同步代码库前要获得 **给定对象的锁**。
- `synchronized(类.class)` 表示进入同步代码前要获得 **给定 Class 的锁**

```
synchronized(this) {  
    //业务代码  
}
```

总结：

- `synchronized` 关键字加到 `static` 静态方法和 `synchronized(class)` 代码块上都是给 Class 类上锁；
- `synchronized` 关键字加到实例方法上是给对象实例上锁；
- 尽量不要使用 `synchronized(String a)` 因为 JVM 中，字符串常量池具有缓存功能。

构造方法可以使用 `synchronized` 关键字修饰么？

先说结论：构造方法不能使用 `synchronized` 关键字修饰。

构造方法本身就属于线程安全的，不存在同步的构造方法一说。

讲一下 `synchronized` 关键字的底层原理

`synchronized` 关键字底层原理属于 JVM 层面。

`synchronized` 同步语句块的情况

```
public class SynchronizedDemo {  
    public void method() {  
        synchronized (this) {  
            System.out.println("synchronized 代码块");  
        }  
    }  
}
```

通过 JDK 自带的 `javap` 命令查看 `SynchronizedDemo` 类的相关字节码信息：首先切换到类的对应目录执行 `javac SynchronizedDemo.java` 命令生成编译后的 `.class` 文件，然后执行 `javap -c -s -v -l SynchronizedDemo.class`。

```

public void method();
descriptor: ()V
flags: ACC_PUBLIC
Code:
    stack=2, locals=3, args_size=1
    0: aload_0
    1: dup
    2: astore 1
    3: monitorenter
    4: getstatic #2                // Field java/lang/System.out:Ljava/io/PrintStream;
    7: ldc #3                      // String Method 1 start
    9: invokevirtual #4            // Method java/io/PrintStream.println:(Ljava/lang/String;)V
   12: aload 1
   13: monitorexit
   14: goto 22
   17: astore_2
   18: aload_1
   19: monitorexit
   20: aload_2
   21: athrow
   22: return
Exception table:
    from    to  target type
    4       14    17    any
    17      20    17    any
LineNumberTable:
    line 5: 0
    line 6: 4
    line 7: 12
    line 8: 22
StackMapTable: number_of_entries = 2
    frame_type = 255 /* full_frame */
    offset_delta = 17
    locals = [ class test/SynchronizedDemo, class java/lang/Object ]
    stack = [ class java/lang/Throwable ]
    frame_type = 250 /* chop */
    offset_delta = 4
}
SourceFile: "SynchronizedDemo.java"

```

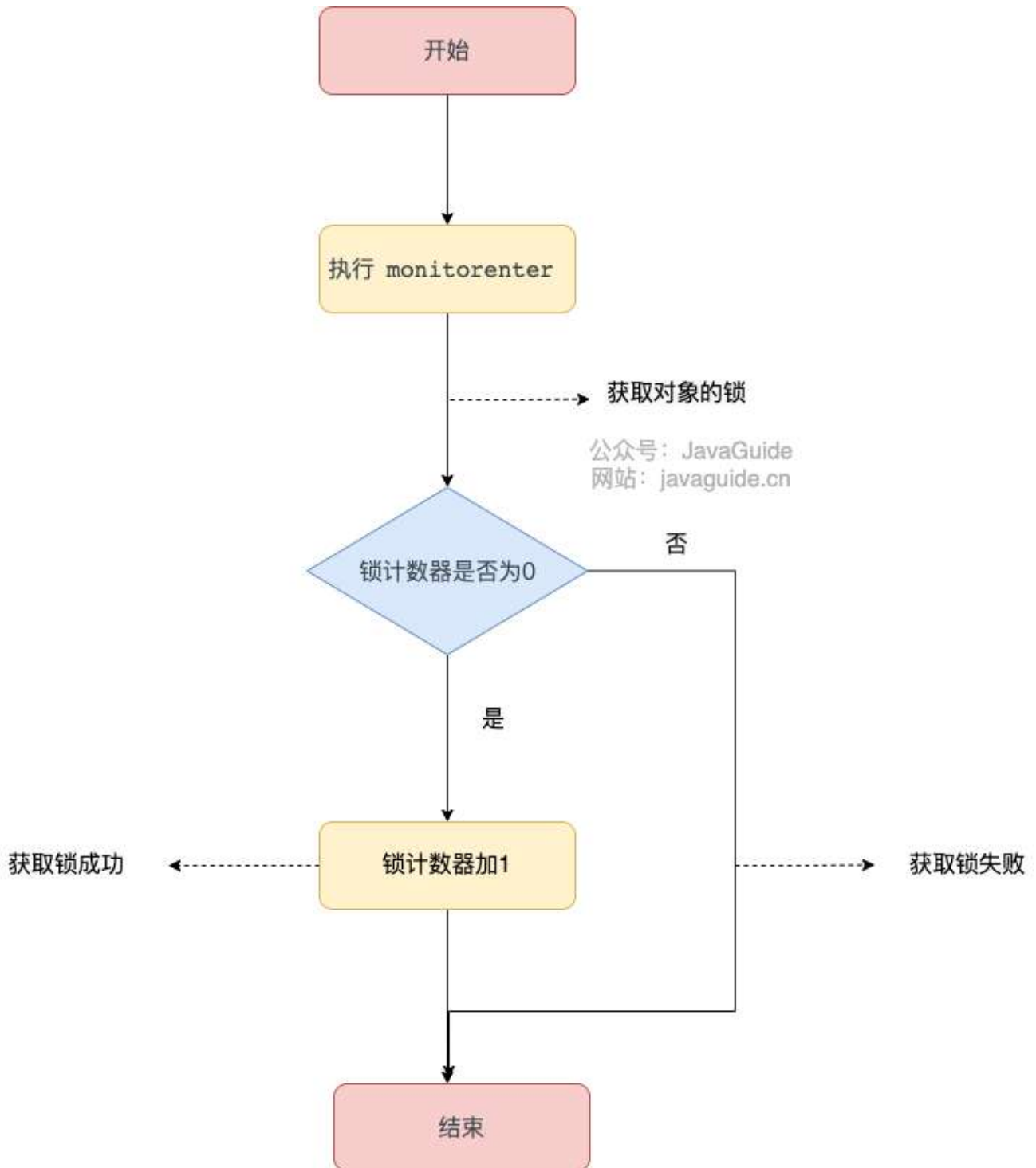
从上面我们可以看出：`synchronized` 同步语句块的实现使用的是 `monitorenter` 和 `monitorexit` 指令，其中 `monitorenter` 指令指向同步代码块的开始位置，`monitorexit` 指令则指明同步代码块的结束位置。

当执行 `monitorenter` 指令时，线程试图获取锁也就是获取 **对象监视器** `monitor` 的持有权。

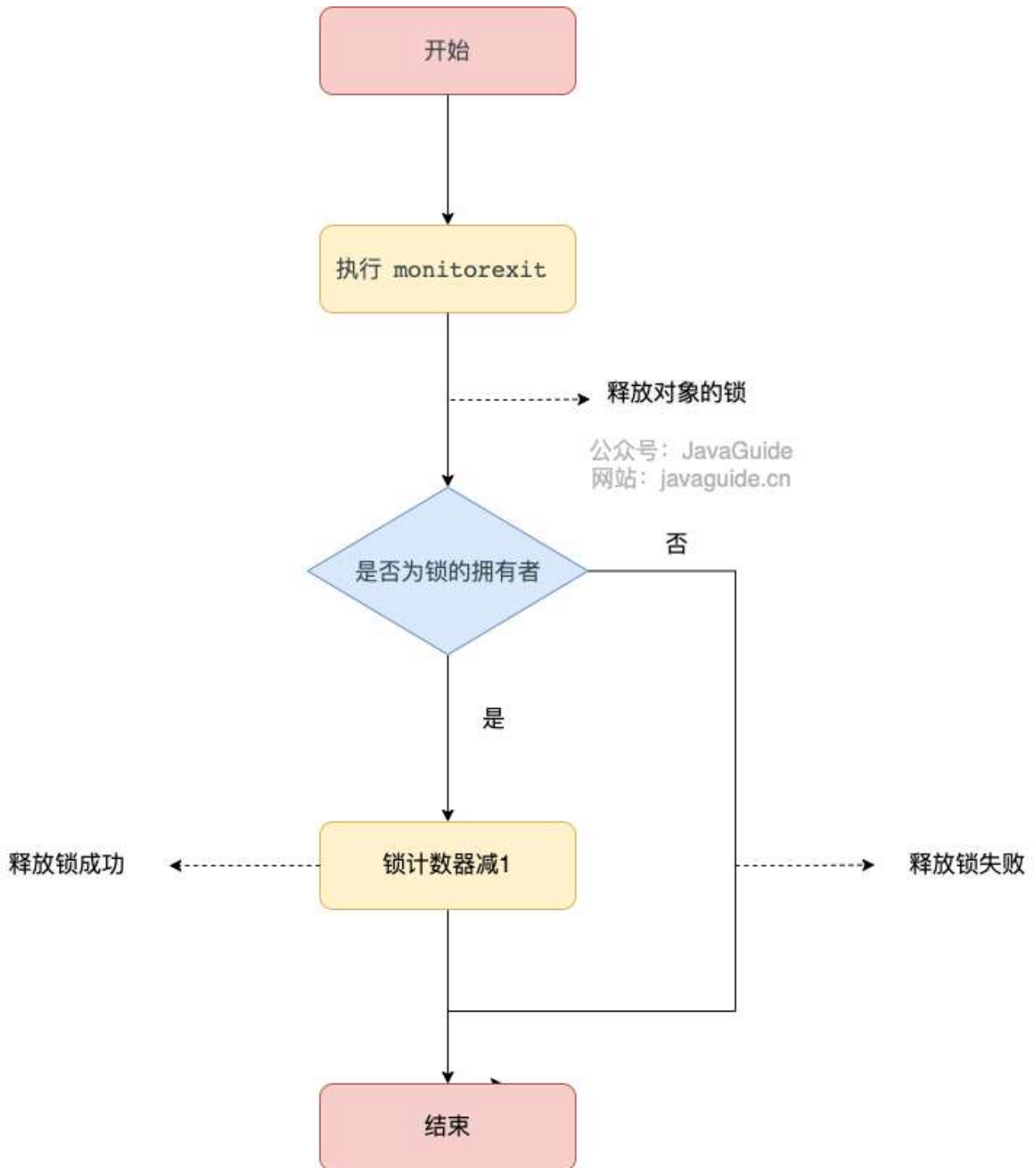
在 Java 虚拟机(HotSpot)中，Monitor 是基于 C++实现的，由 `ObjectMonitor` 实现的。每个对象中都内置了一个 `ObjectMonitor` 对象。

另外，`wait/notify` 等方法也依赖于 `monitor` 对象，这就是为什么只有在同步的块或者方法中才能调用 `wait/notify` 等方法，否则会抛出 `java.lang.IllegalMonitorStateException` 的异常的原因。

在执行 `monitorenter` 时，会尝试获取对象的锁，如果锁的计数器为 0 则表示锁可以被获取，获取后将锁计数器设为 1 也就是加 1。



对象锁的的拥有者线程才可以执行 `monitorexit` 指令来释放锁。在执行 `monitorexit` 指令后，将锁计数器设为 0，表明锁被释放，其他线程可以尝试获取锁。



如果获取对象锁失败，那当前线程就要阻塞等待，直到锁被另外一个线程释放为止。

synchronized 修饰方法的的情况


```
public class SynchronizedDemo2 {
    public synchronized void method() {
        System.out.println("synchronized 方法");
    }
}
```

```
{
public test.SynchronizedDemo2();
  descriptor: ()V
  flags: ACC_PUBLIC
  Code:
    stack=1, locals=1, args_size=1
      0: aload_0
      1: invokespecial #1          // Method java/lang/Object.<init>:()V
      4: return
  LineNumberTable:
    line 3: 0

public synchronized void method();
  descriptor: ()V
  flags: ACC_PUBLIC, ACC_SYNCHRONIZED
  Code:
    stack=2, locals=1, args_size=1
      0: getstatic      #2          // Field java/lang/System.out:Ljava/io/PrintStream;
      3: ldc            #3          // String synchronized 钀堿瑙勫
      5: invokevirtual #4          // Method java/io/PrintStream.println:(Ljava/lang/String;)V
      8: return
  LineNumberTable:
    line 5: 0
    line 6: 8
}
SourceFile: "SynchronizedDemo2.java"
```

synchronized 修饰的方法并没有 monitorenter 指令和 monitorexit 指令，取得代之的确实是 ACC_SYNCHRONIZED 标识，该标识指明了该方法是一个同步方法。JVM 通过该 ACC_SYNCHRONIZED 访问标志来辨别一个方法是否声明为同步方法，从而执行相应的同步调用。

如果是实例方法，JVM 会尝试获取实例对象的锁。如果是静态方法，JVM 会尝试获取当前 class 的锁。

总结

synchronized 同步语句块的实现使用的是 monitorenter 和 monitorexit 指令，其中 monitorenter 指令指向同步代码块的开始位置，monitorexit 指令则指明同步代码块的结束位置。

synchronized 修饰的方法并没有 monitorenter 指令和 monitorexit 指令，取得代之的确实是 ACC_SYNCHRONIZED 标识，该标识指明了该方法是一个同步方法。

不过两者的本质都是对对象监视器 monitor 的获取。

相关推荐：[Java 锁与线程的那些事 - 有赞技术团队](#)。

 进阶一下：学有余力的小伙伴可以抽时间详细研究一下对象监视器 `monitor`。

JDK1.6 之后的 `synchronized` 关键字底层做了哪些优化？

JDK1.6 对锁的实现引入了大量的优化，如偏向锁、轻量级锁、自旋锁、适应性自旋锁、锁消除、锁粗化等技术来减少锁操作的开销。

锁主要存在四种状态，依次是：无锁状态、偏向锁状态、轻量级锁状态、重量级锁状态，他们会随着竞争的激烈而逐渐升级。注意锁可以升级不可降级，这种策略是为了提高获得锁和释放锁的效率。

关于这几种优化的详细信息可以查看下面这篇文章：[Java6 及以上版本对 `synchronized` 的优化](#)

`synchronized` 和 `volatile` 的区别？

`synchronized` 关键字和 `volatile` 关键字是两个互补的存在，而不是对立的存在！

- `volatile` 关键字是线程同步的轻量级实现，所以 `volatile` 性能肯定比 `synchronized` 关键字要好。但是 `volatile` 关键字只能用于变量而 `synchronized` 关键字可以修饰方法以及代码块。
- `volatile` 关键字能保证数据的可见性，但不能保证数据的原子性。`synchronized` 关键字两者都能保证。
- `volatile` 关键字主要用于解决变量在多个线程之间的可见性，而 `synchronized` 关键字解决的是多个线程之间访问资源的同步性。

`synchronized` 和 `ReentrantLock` 的区别

两者都是可重入锁

“可重入锁”指的是自己可以再次获取自己的内部锁。比如一个线程获得了某个对象的锁，此时这个对象锁还没有释放，当其再次想要获取这个对象的锁的时候还是可以获取的，如果是不可重入锁的话，就会造成死锁。同一个线程每次获取锁，锁的计数器都自增 1，所以要等到锁的计数器下降为 0 时才能释放锁。

`synchronized` 依赖于 JVM 而 `ReentrantLock` 依赖于 API

`synchronized` 是依赖于 JVM 实现的，前面我们也讲到了虚拟机团队在 JDK1.6 为 `synchronized` 关键字进行了很多优化，但是这些优化都是在虚拟机层面实现的，并没有直接暴露给我们。`ReentrantLock` 是 JDK 层面实现的（也就是 API 层面，需要 `lock()` 和 `unlock()` 方法配合 `try/finally` 语句块来完成），所以我们可以查看它的源代码，来看它是如何实现的。

ReentrantLock 比 synchronized 增加了一些高级功能

相比 `synchronized`，`ReentrantLock` 增加了一些高级功能。主要来说主要有三点：

- **等待可中断**：`ReentrantLock` 提供了一种能够中断等待锁的线程的机制，通过 `lock.lockInterruptibly()` 来实现这个机制。也就是说正在等待的线程可以选择放弃等待，改为处理其他事情。
- **可实现公平锁**：`ReentrantLock` 可以指定是公平锁还是非公平锁。而 `synchronized` 只能是非公平锁。所谓的公平锁就是先等待的线程先获得锁。`ReentrantLock` 默认情况是非公平的，可以通过 `ReentrantLock` 类的 `ReentrantLock(boolean fair)` 构造方法来制定是否是公平的。
- **可实现选择性通知（锁可以绑定多个条件）**：`synchronized` 关键字与 `wait()` 和 `notify()` / `notifyAll()` 方法相结合可以实现等待/通知机制。`ReentrantLock` 类当然也可以实现，但是需要借助于 `Condition` 接口与 `newCondition()` 方法。

`Condition` 是 JDK1.5 之后才有的，它具有很好的灵活性，比如可以实现多路通知功能也就是在一个 `Lock` 对象中可以创建多个 `Condition` 实例（即对象监视器），线程对象可以注册在指定的 `Condition` 中，从而可以有选择性的进行线程通知，在调度线程上更加灵活。在使用 `notify()/notifyAll()` 方法进行通知时，被通知的线程是由 JVM 选择的，用 `ReentrantLock` 类结合 `Condition` 实例可以实现“选择性通知”，这个功能非常重要，而且是 `Condition` 接口默认提供的。而 `synchronized` 关键字就相当于整个 `Lock` 对象中只有一个 `Condition` 实例，所有的线程都注册在它一个身上。如果执行 `notifyAll()` 方法的话就会通知所有处于等待状态的线程这样会造成很大的效率问题，而 `Condition` 实例的 `signalAll()` 方法只会唤醒注册在该 `Condition` 实例中的所有等待线程。

如果你想使用上述功能，那么选择 `ReentrantLock` 是一个不错的选择。性能已不是选择标准

ThreadLocal

ThreadLocal 有什么用？

通常情况下，我们创建的变量是可以被任何一个线程访问并修改的。如果想实现每一个线程都有自己的专属本地变量该如何解决呢？

JDK 中自带的 `ThreadLocal` 类正是为了解决这样的问题。`ThreadLocal` 类主要解决的就是让每个线程绑定自己的值，可以将 `ThreadLocal` 类形象的比喻成存放数据的盒子，盒子中可以存储每个线程的私有数据。

如果你创建了一个 `ThreadLocal` 变量，那么访问这个变量的每个线程都会有这个变量的本地副本，这也是 `ThreadLocal` 变量名的由来。他们可以使用 `get ()` 和 `set ()` 方法来获取默认值或将其值更改为当前线程所存的副本的值，从而避免了线程安全问题。

再举个简单的例子：两个人去宝屋收集宝物，这两个共用一个袋子的话肯定会产生争执，但是给他们两个人每个人分配一个袋子的话就不会出现这样的问题。如果把这两个人比作线程的话，那么 `ThreadLocal` 就是用来避免这两个线程竞争的。

如何使用 `ThreadLocal`?

相信看了上面的解释，大家已经搞懂 `ThreadLocal` 类是个什么东西了。下面简单演示一下如何在项目中实际使用 `ThreadLocal`。

```
import java.text.SimpleDateFormat;
import java.util.Random;

public class ThreadLocalExample implements Runnable{

    // SimpleDateFormat 不是线程安全的，所以每个线程都要有自己独立的副本
    private static final ThreadLocal<SimpleDateFormat> formatter =
ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyyMMdd HHmm"));

    public static void main(String[] args) throws InterruptedException {
        ThreadLocalExample obj = new ThreadLocalExample();
        for(int i=0 ; i<10; i++){
            Thread t = new Thread(obj, "+"+i);
            Thread.sleep(new Random().nextInt(1000));
            t.start();
        }
    }

    @Override
    public void run() {
        System.out.println("Thread Name= "+Thread.currentThread().getName()+"
default Formatter = "+formatter.get().toPattern());
        try {
            Thread.sleep(new Random().nextInt(1000));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }

    //formatter pattern is changed here by thread, but it won't reflect to
    other threads

    formatter.set(new SimpleDateFormat());

    System.out.println("Thread Name= "+Thread.currentThread().getName()+"
formatter = "+formatter.get().toPattern());
    }

}

```

输出结果：

```

Thread Name= 0 default Formatter = yyyyMMdd HHmm
Thread Name= 0 formatter = yy-M-d ah:mm
Thread Name= 1 default Formatter = yyyyMMdd HHmm
Thread Name= 2 default Formatter = yyyyMMdd HHmm
Thread Name= 1 formatter = yy-M-d ah:mm
Thread Name= 3 default Formatter = yyyyMMdd HHmm
Thread Name= 2 formatter = yy-M-d ah:mm
Thread Name= 4 default Formatter = yyyyMMdd HHmm
Thread Name= 3 formatter = yy-M-d ah:mm
Thread Name= 4 formatter = yy-M-d ah:mm
Thread Name= 5 default Formatter = yyyyMMdd HHmm
Thread Name= 5 formatter = yy-M-d ah:mm
Thread Name= 6 default Formatter = yyyyMMdd HHmm
Thread Name= 6 formatter = yy-M-d ah:mm
Thread Name= 7 default Formatter = yyyyMMdd HHmm
Thread Name= 7 formatter = yy-M-d ah:mm
Thread Name= 8 default Formatter = yyyyMMdd HHmm
Thread Name= 9 default Formatter = yyyyMMdd HHmm
Thread Name= 8 formatter = yy-M-d ah:mm
Thread Name= 9 formatter = yy-M-d ah:mm

```

从输出中可以看出，虽然 Thread-0 已经改变了 formatter 的值，但 Thread-1 默认格式化值与初始值相同，其他线程也一样。

上面有一段代码用到了创建 `ThreadLocal` 变量的那段代码用到了 Java8 的知识，它等于下面这段代码，如果你写了下面这段代码的话，IDEA 会提示你转换为 Java8 的格式(IDEA 真的不错！)。因为 `ThreadLocal` 类在 Java 8 中扩展，使用一个新的方法 `withInitial()`，将 `Supplier` 功能接口作为参数。

```
private static final ThreadLocal<SimpleDateFormat> formatter = new
ThreadLocal<SimpleDateFormat>(){
    @Override
    protected SimpleDateFormat initialValue(){
        return new SimpleDateFormat("yyyyMMdd HHmm");
    }
};
```

ThreadLocal 原理了解吗？

从 `Thread` 类源代码入手。

```
public class Thread implements Runnable {
    //.....
    //与此线程有关的ThreadLocal值。由ThreadLocal类维护
    ThreadLocal.ThreadLocalMap threadLocals = null;

    //与此线程有关的InheritableThreadLocal值。由InheritableThreadLocal类维护
    ThreadLocal.ThreadLocalMap inheritableThreadLocals = null;
    //.....
}
```

从上面 `Thread` 类 源代码可以看出 `Thread` 类中有一个 `threadLocals` 和一个 `inheritableThreadLocals` 变量，它们都是 `ThreadLocalMap` 类型的变量,我们可以把 `ThreadLocalMap` 理解为 `ThreadLocal` 类实现的定制化的 `HashMap`。默认情况下这两个变量都是 `null`，只有当前线程调用 `ThreadLocal` 类的 `set` 或 `get` 方法时才创建它们，实际上调用这两个方法的时候，我们调用的是 `ThreadLocalMap` 类对应的 `get()`、`set()` 方法。

`ThreadLocal` 类的 `set()` 方法

```

public void set(T value) {
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null)
        map.set(this, value);
    else
        createMap(t, value);
}
ThreadLocalMap getMap(Thread t) {
    return t.threadLocals;
}

```

通过上面这些内容，我们足以通过猜测得出结论：最终的变量是放在了当前线程的 `ThreadLocalMap` 中，并不是存在 `ThreadLocal` 上，`ThreadLocal` 可以理解为只是 `ThreadLocalMap` 的封装，传递了变量值。`ThreadLocal` 类中可以通过 `Thread.currentThread()` 获取到当前线程对象后，直接通过 `getMap(Thread t)` 可以访问到该线程的 `ThreadLocalMap` 对象。

每个 `Thread` 中都具备一个 `ThreadLocalMap`，而 `ThreadLocalMap` 可以存储以 `ThreadLocal` 为 key，Object 对象为 value 的键值对。

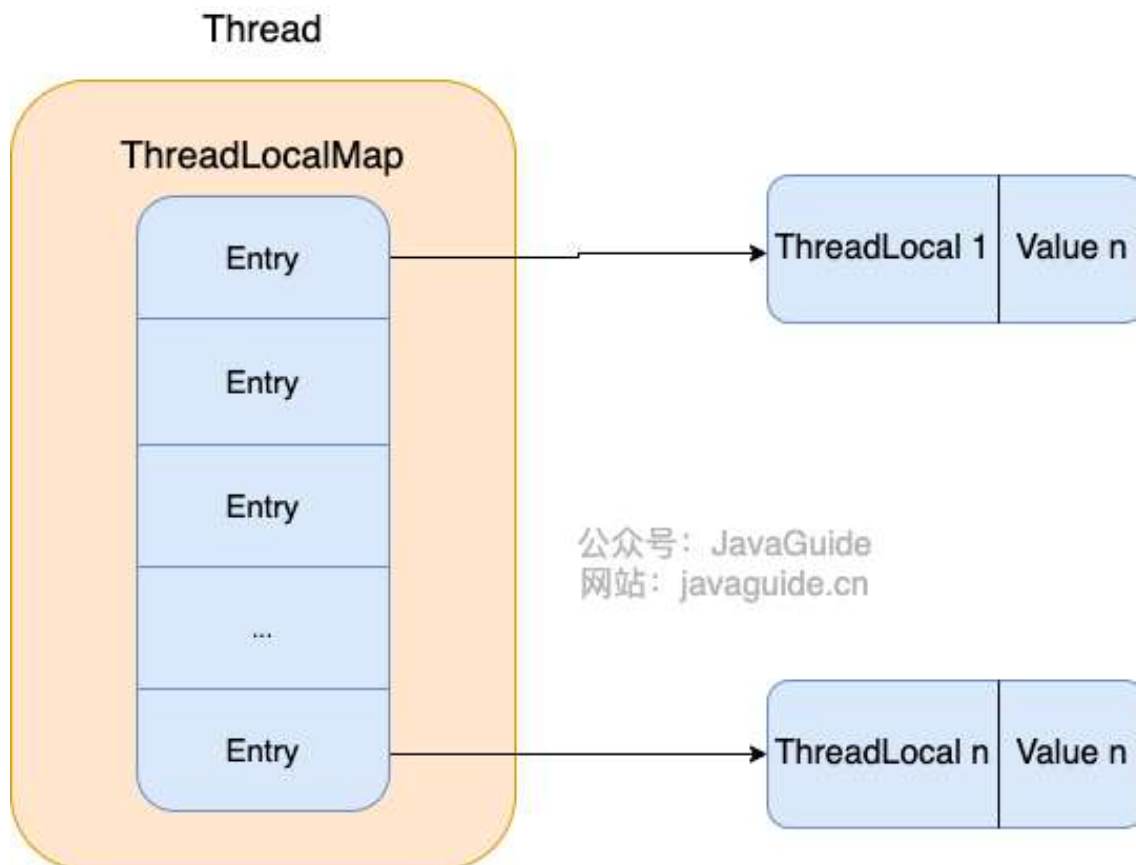
```

ThreadLocalMap(ThreadLocal<?> firstKey, Object firstValue) {
    //.....
}

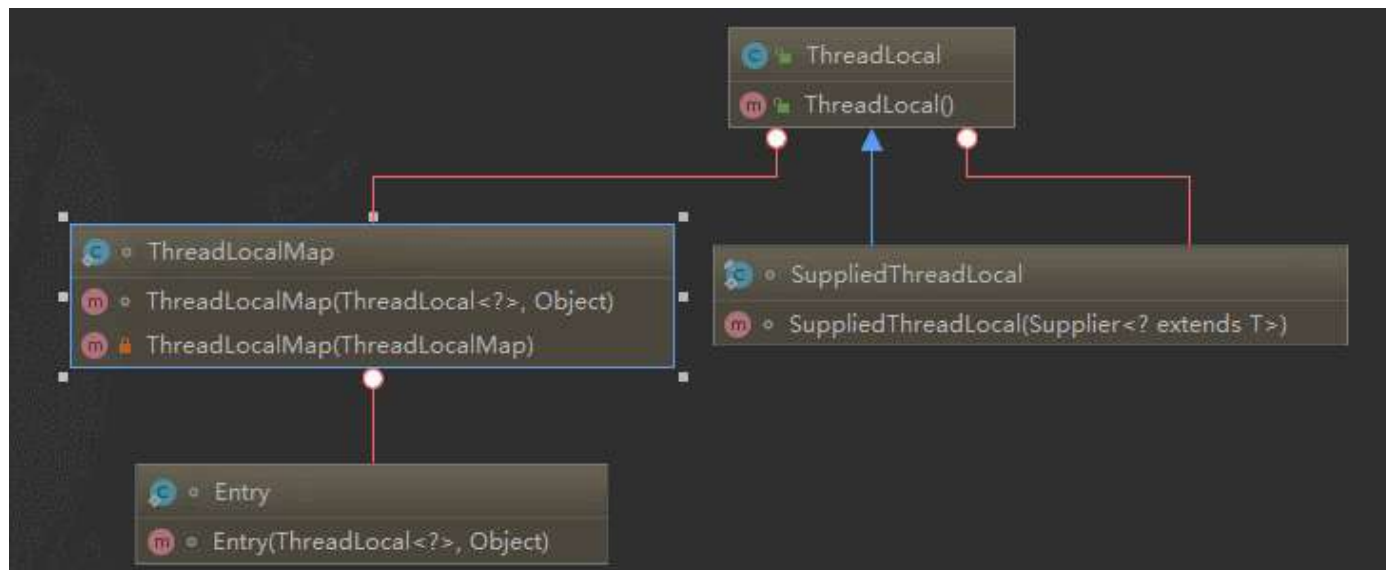
```

比如我们在同一个线程中声明了两个 `ThreadLocal` 对象的话，`Thread` 内部都是使用仅有的那个 `ThreadLocalMap` 存放数据的，`ThreadLocalMap` 的 key 就是 `ThreadLocal` 对象，value 就是 `ThreadLocal` 对象调用 `set` 方法设置的值。

`ThreadLocal` 数据结构如下图所示：



`ThreadLocalMap` 是 `ThreadLocal` 的静态内部类。



ThreadLocal 内存泄露问题是怎么导致的?

`ThreadLocalMap` 中使用的 key 为 `ThreadLocal` 的弱引用，而 value 是强引用。所以，如果 `ThreadLocal` 没有被外部强引用的情况下，在垃圾回收的时候，key 会被清理掉，而 value 不会被清理掉。

这样一来，`ThreadLocalMap` 中就会出现 key 为 null 的 Entry。假如我们不做任何措施的话，value 永远无法被 GC 回收，这个时候就可能会产生内存泄露。`ThreadLocalMap` 实现中已经考虑了这种情况，在调用 `set()`、`get()`、`remove()` 方法的时候，会清理掉 key 为 null 的记录。使用完 `ThreadLocal` 方法后 最好手动调用 `remove()` 方法

```
static class Entry extends WeakReference<ThreadLocal<?>> {
    /** The value associated with this ThreadLocal. */
    Object value;

    Entry(ThreadLocal<?> k, Object v) {
        super(k);
        value = v;
    }
}
```

弱引用介绍：

如果一个对象只具有弱引用，那就类似于**可有可无的生活用品**。弱引用与软引用的区别在于：只具有弱引用的对象拥有更短暂的生命周期。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存。不过，由于垃圾回收器是一个优先级很低的线程，因此不一定会很快发现那些只具有弱引用的对象。

弱引用可以和一个引用队列（`ReferenceQueue`）联合使用，如果弱引用所引用的对象被垃圾回收，Java 虚拟机就会把这个弱引用加入到与之关联的引用队列中。

线程池

线程池相关的知识点和面试题总结请看这篇文章：[Java 线程池详解](#)（由于内容比较多就不放在 PDF 里面了）。

AQS

AQS 相关的知识点和面试题总结请看这篇文章：[AQS 详解](#)（由于内容比较多就不放在 PDF 里面了）。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

2.4. JVM

如果你想冲击大厂的话，可以通过我根据《深入理解 Java 虚拟机：JVM 高级特性与最佳实践（第三版）》总结的下面这几篇文章来准备面试：

1. [Java 内存区域详解](#)
2. [JVM 垃圾回收详解](#)
3. [类文件结构详解](#)
4. [类加载过程详解](#)
5. [类加载器详解](#)



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“PDF”领取原创面试 PDF 手册
- 2、关注公众号回复“加群”进入面试交流群
- 3、关注公众号回复“八股文”获取大厂面试真题+面经

3. 计算机基础

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide!



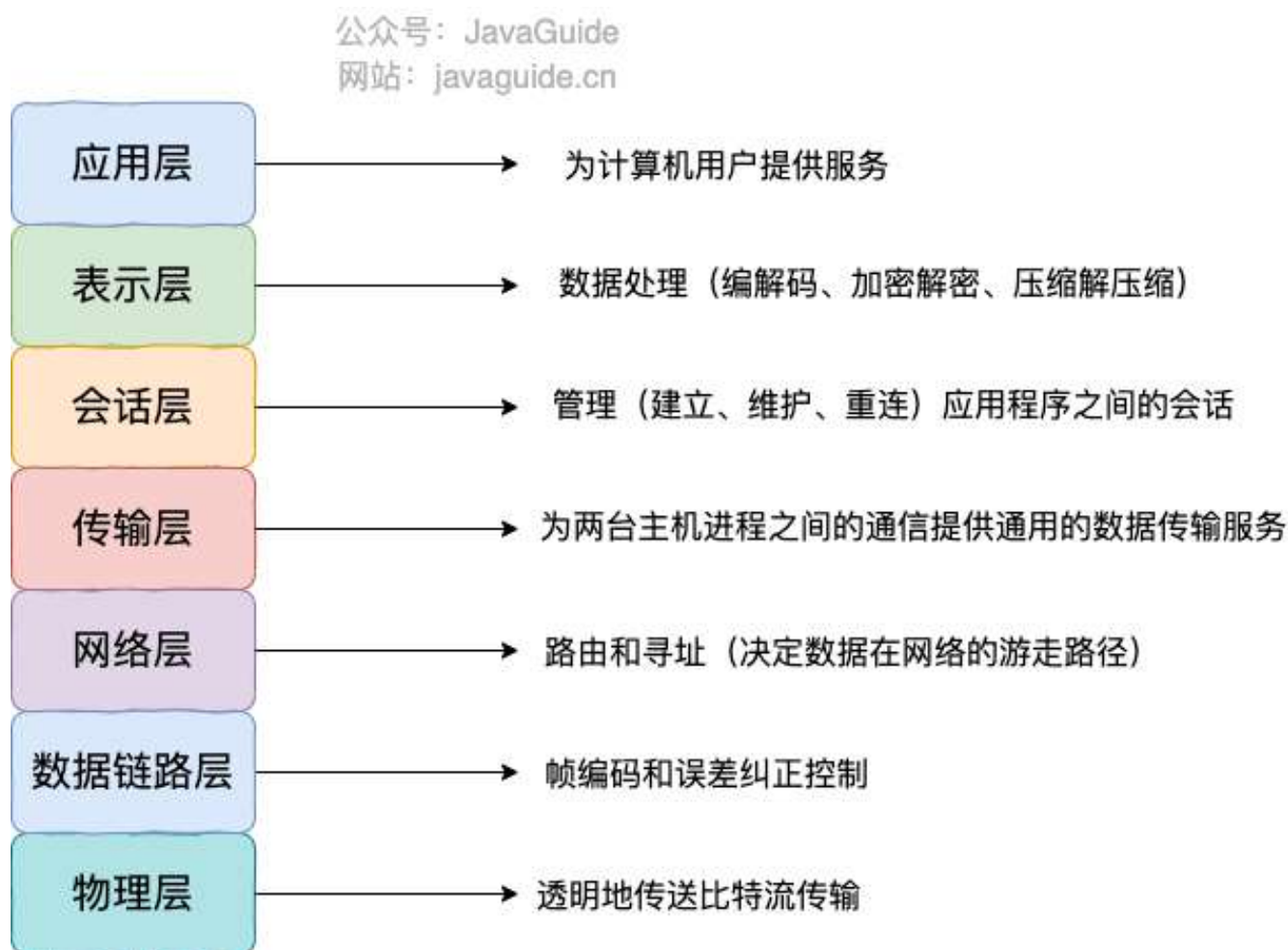
Github |

Gitee

3.1 计算机网络

OSI 七层模型是什么？每一层的作用是什么？

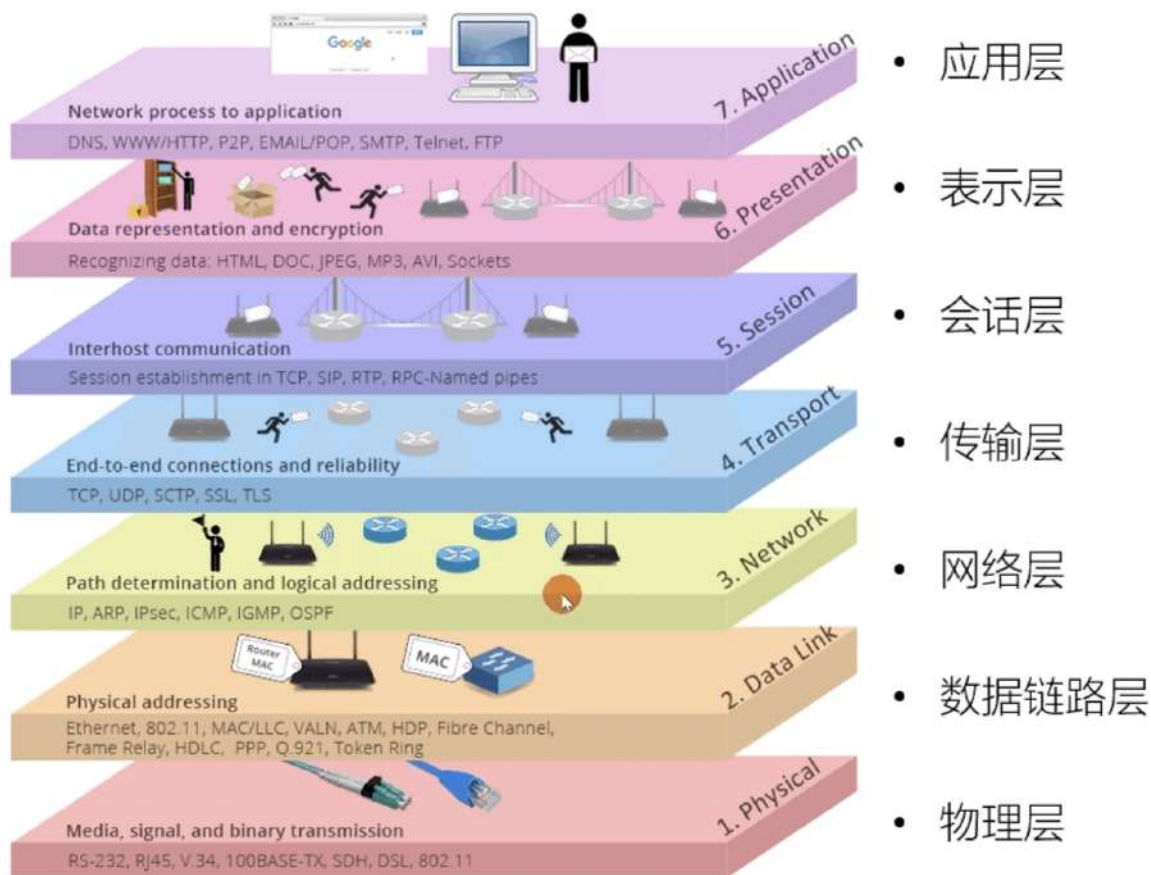
OSI 七层模型 是国际标准化组织提出一个网络分层模型，其大体结构以及每一层提供的功能如下图所示：



每一层都专注做一件事情，并且每一层都需要使用下一层提供的功能比如传输层需要使用网络层提供的路由和寻址功能，这样传输层才知道把数据传输到哪里去。

OSI 的七层体系结构概念清楚，理论也很完整，但是它比较复杂而且不实用，而且有些功能在多个层中重复出现。

上面这种图可能比较抽象，再来一个比较生动的图片。下面这个图片是我在国外的一个网站上看到的，非常赞！



既然 OSI 七层模型这么厉害，为什么干不过 TCP/IP 四层模型呢？

的确，OSI 七层模型当时一直被一些大公司甚至一些国家政府支持。这样的背景下，为什么会失败呢？我觉得主要有下面几方面原因：

1. OSI 的专家缺乏实际经验，他们在完成 OSI 标准时缺乏商业驱动力
2. OSI 的协议实现起来过分复杂，而且运行效率很低
3. OSI 制定标准的周期太长，因而使得按 OSI 标准生产的设备无法及时进入市场（20 世纪 90 年代初期，虽然整套的 OSI 国际标准都已经制定出来，但基于 TCP/IP 的互联网已经抢先在全球相当大的范围成功运行了）
4. OSI 的层次划分不太合理，有些功能在多个层次中重复出现。

OSI 七层模型虽然失败了，但是却提供了很多不错的理论基础。为了更好地去了解网络分层，OSI 七层模型还是非常有必要学习的。

最后再分享一个关于 OSI 七层模型非常不错的总结图片！

TCP/IP

第7层 应用层

各种应用程序协议，如HTTP、FTP、SMTP、POP3。

常见使用TCP协议的应用层服务

HTTP 超文本传输协议	FTP 文件传输协议	SMTP 简单邮件传输协议	TELNET TCP/IP终端仿真协议
POP3 邮局协议第3版	Finger 用户信息协议	NNTP 网络新闻传输协议	IMAP4 因特网信息访问协议第四版

UNIX网络服务

LPR UNIX远程打印协议	Rwho UNIX远程Who协议	Exec UNIX远程执行协议
Login UNIX远程登录协议	RSH UNIX远程Shell协议	

常见使用UDP协议的应用层服务

BOOTP 引导协议
DHCP 动态主机配置协议
NTP 网络时间协议
TFTP 简单文件传输协议

HP网络服务

NTF HP 网络文件传输协议	RDA HP 远程数据库访问协议	VT 虚拟终端仿真协议	RFA HP 远程文件访问协议	RPC Remote Process Comm.
--------------------	---------------------	----------------	--------------------	-----------------------------

同时使用TCP和UDP协议的应用层服务

SOCKS 安全套接字协议	FANP 流属性通知协议
SLP 服务定位协议	MSN 微软网络服务
Radius 远程用户拨号认证服务协议	DNS 域名系统

SUN网络服务

NFS 网络文件系统协议	R-STAT SUN远程状态协议	PMAP SUN端口映射协议
NIS SUN网络信息库系统协议	NSM SUN网络状态监测协议	Mount

第6层 表示层

信息的语法规则以及它们的关联，如加密解密、转换翻译、压缩解压缩。

GDP
网关发现协议

S-HTTP
安全超文本传输协议

X-Window
X-Window

CMOT
基于TCP/IP的CMIP协议

LPP
轻量级表示协议

NBSSN
NetBIOS会话服务协议

第5层 会话层

不同机器上的用户之间建立及管理会话。

安全协议

SSL
安全套接字层协议

TLS
传输层安全协议

目录访问协议

DAP
目录访问协议

LDAP
轻量级目录访问协议

第4层 传输层

接受上一层的数据，在必要的时候把数据进行分割，并将这些数据交给网络层，且保证这些数据段有效到达对端。

DSI NetBIOS	IP NetBIOS	SMB
NetBIOS	ISO-TP SSP	MSRPC

XOT
基于TCP之上的X.25协议

Van Jacobson
压缩TCP协议

ISO-DE
ISO开发环境

TALI
传输适配层接口协议

RUDP
可靠的用户数据报协议

Mobile IP
移动IP协议

TCP 传输控制协议

UDP 用户数据报协议

第3层 网络层

控制子网的运行，如逻辑编址、分组传输、路由选择。

安全协议

AH
认证头协议

ESP
安全封装有效载荷协议

路由协议

EQP 外部网关协议	NIIRP 下一跳解析协议	CCP 网关到网关协议	RSVP 资源预留协议	RIP2 路由信息协议第2版
OSPF 开放最短路径优先协议	IE-IRGP 增强内部网关路由选择协议	VRRP 虚拟路由冗余协议	PIM-DM 密集模式独立组播协议	PIM-SM 稀疏模式独立组播协议
IGRP 内部网关路由协议	RIPng for IPv6 IPv6路由信息协议	PGM 实际距离组播协议	DVMRP 距离矢量组播路由协议	MOSP 组播开放最短路径优先协议

IP/IPv6
互联网协议/互联网协议第6版

SLIP
串行线路IP协议

第2层 数据链路层

物理寻址，同时将原始比特流转变为逻辑传输线路。

隧道协议

PPTP 点对点隧道协议	L2TP 第二层隧道协议
L2F 第二层转发协议	ATMP 接入隧道管理协议

Cisco协议

CDP 思科发现协议	CGMP 思科组播管理协议
---------------	------------------

地址解析协议

ARP 地址解析协议	RARP 反向地址解析协议
---------------	------------------

第1层 物理层

机械、电子、定时接口通信信道上的原始比特流传输。

IEEE 802.2

Ethernet v.2

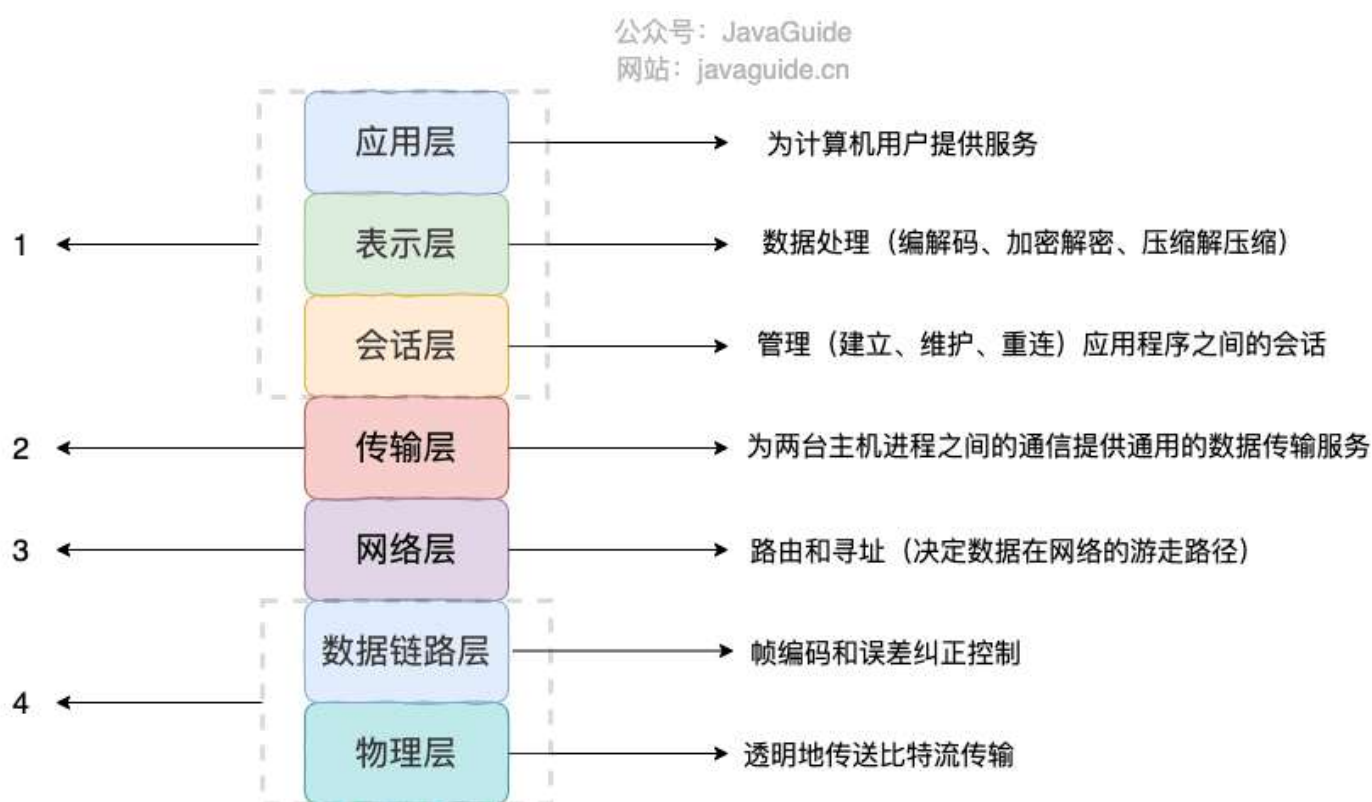
Internetwork

TCP/IP 四层模型是什么？每一层的作用是什么？

TCP/IP 四层模型 是目前被广泛采用的一种模型,我们可以将 TCP / IP 模型看作是 OSI 七层模型的精简版本，由以下 4 层组成：

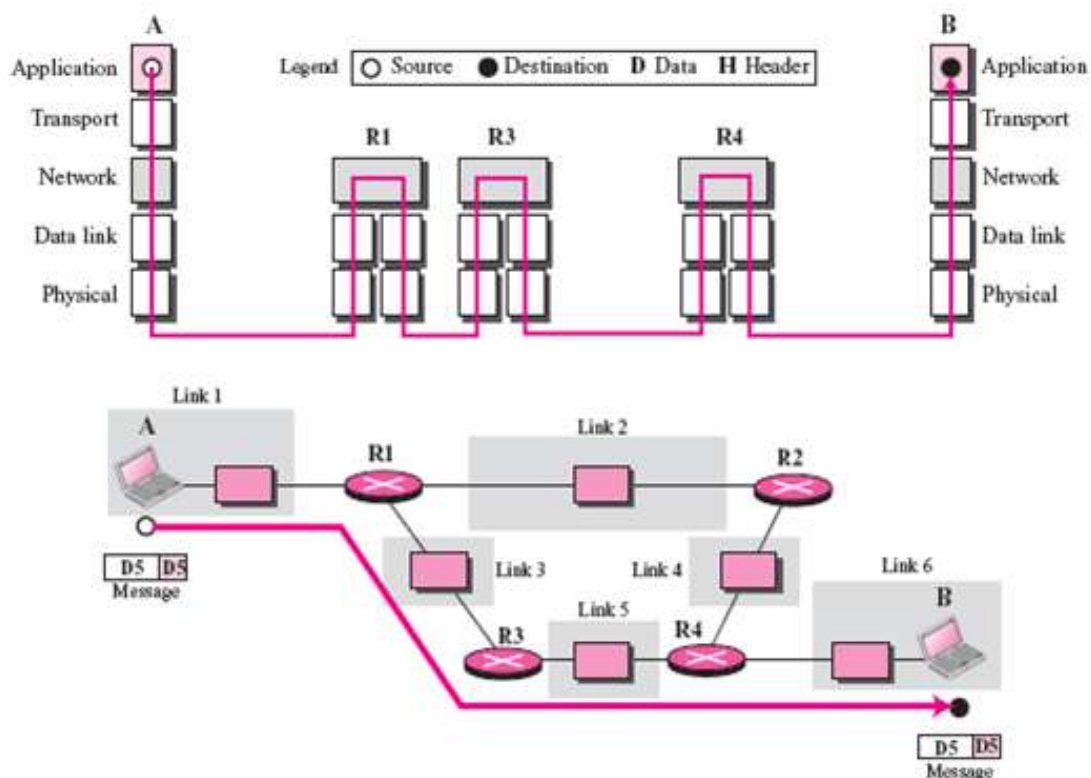
1. 应用层
2. 传输层
3. 网络层
4. 网络接口层

需要注意的是，我们并不能将 TCP/IP 四层模型 和 OSI 七层模型完全精确地匹配起来，不过可以简单将两者对应起来，如下图所示：

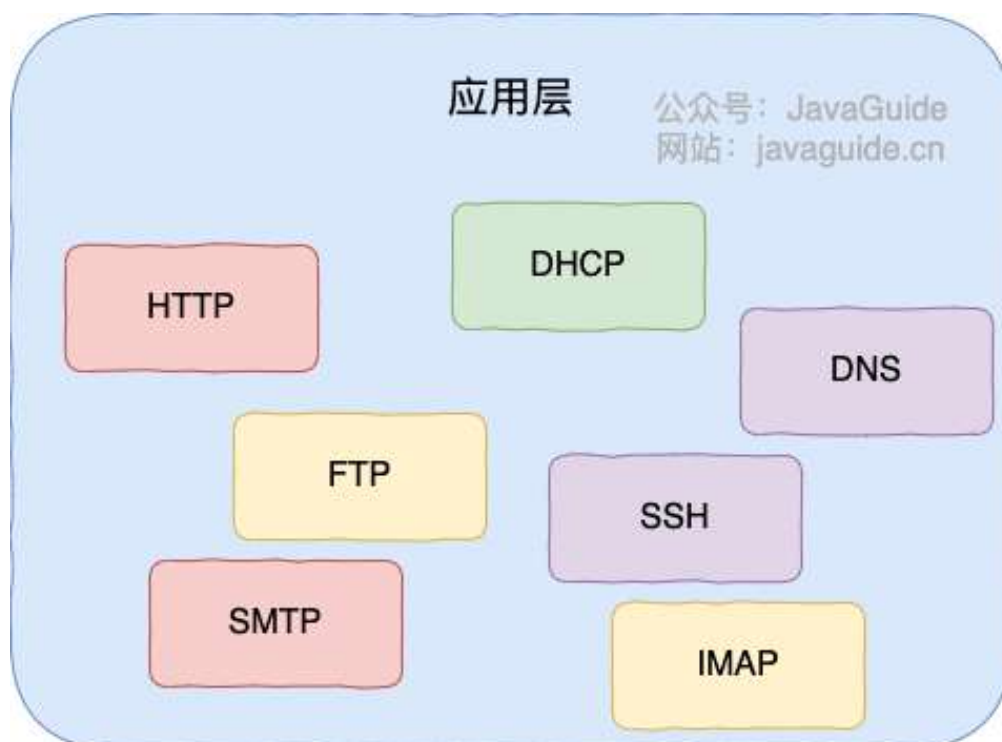


应用层（Application layer）

应用层位于传输层之上，主要提供两个终端设备上的应用程序之间信息交换的服务，它定义了信息交换的格式，消息会交给下一层传输层来传输。我们把应用层交互的数据单元称为报文。



应用层协议定义了网络通信规则，对于不同的网络应用需要不同的应用层协议。在互联网中应用层协议很多，如支持 Web 应用的 HTTP 协议，支持电子邮件的 SMTP 协议等等。



应用层常见协议总结，请看这篇文章：[应用层常见协议总结（应用层）](#)。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

传输层（Transport layer）

传输层的主要任务就是负责向两台终端设备进程之间的通信提供通用的数据传输服务。应用进程利用该服务传送应用层报文。“通用的”是指并不针对某一个特定的网络应用，而是多种应用可以使用同一个运输层服务。

运输层主要使用以下两种协议：

1. 传输控制协议 **TCP** (Transmission Control Protocol) --提供 面向连接 的，可靠的 数据传输服务。
2. 用户数据协议 **UDP** (User Datagram Protocol) --提供 无连接 的，尽最大努力的数据传输服务（不保证数据传输的可靠性）。



网络层（Network layer）

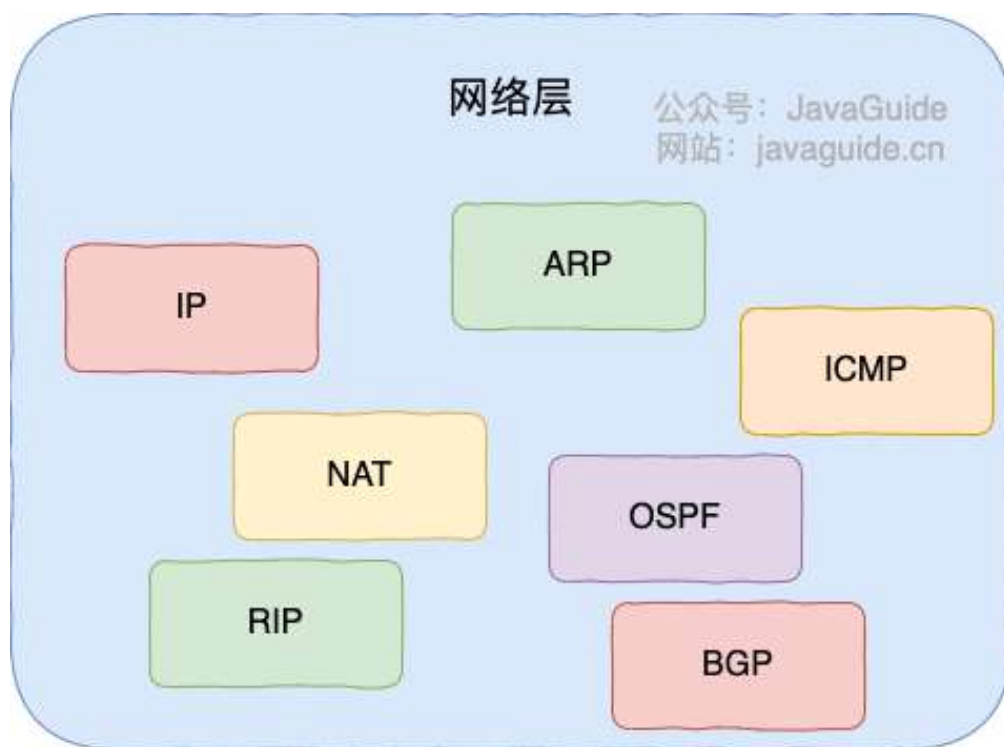
网络层负责为分组交换网上的不同主机提供通信服务。在发送数据时，网络层把运输层产生的报文段或用户数据报封装成分组和包进行传送。在 TCP/IP 体系结构中，由于网络层使用 IP 协议，因此分组也叫 IP 数据报，简称数据报。

⚠注意：不要把运输层的“用户数据报 UDP”和网络层的“IP 数据报”弄混。

网络层的还有一个任务就是选择合适的路由，使源主机运输层所传下来的分组，能通过网络层中的路由器找到目的主机。

这里强调指出，网络层中的“网络”二字已经不是我们通常谈到的具体网络，而是指计算机网络体系结构模型中第三层的名称。

互联网是由大量的异构（heterogeneous）网络通过路由器（router）相互连接起来的。互联网使用的网络层协议是无连接的网际协议（Intert Prococol）和许多路由选择协议，因此互联网的网络层也叫做 **网际层** 或 **IP 层**。



网络层常见协议：

- **IP:网际协议**：网际协议 IP 是TCP/IP协议中最重要的协议之一，也是网络层最重要的协议之一，IP协议的作用包括寻址规约、定义数据包的格式等等，是网络层信息传输的主力协议。目前IP协议主要分为两种，一种是过去的IPv4，另一种是较新的IPv6，目前这两种协议都在使用，但后者已经被提议来取代前者。
- **ARP 协议**：ARP协议，全称地址解析协议（Address Resolution Protocol），它解决的是网络

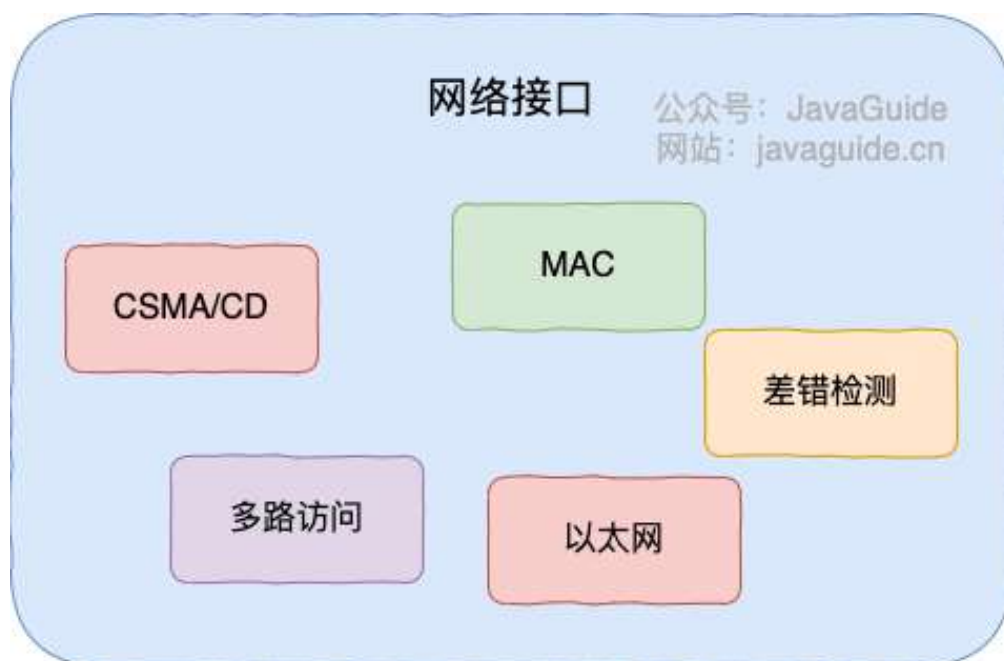
层地址和链路层地址之间的转换问题。因为一个IP数据报在物理上传输的过程中，总是需要知道下一跳（物理上的下一个目的地）该去往何处，但IP地址属于逻辑地址，而MAC地址才是物理地址，ARP协议解决了IP地址转MAC地址的一些问题。

- **NAT:网络地址转换协议**：NAT协议（Network Address Translation）的应用场景如同它的名称——网络地址转换，应用于内部网到外部网的地址转换过程中。具体地说，在一个小的子网（局域网，LAN）内，各主机使用的是同一个LAN下的IP地址，但在该LAN以外，在广域网（WAN）中，需要一个统一的IP地址来标识该LAN在整个Internet上的位置。
-

网络接口层（Network interface layer）

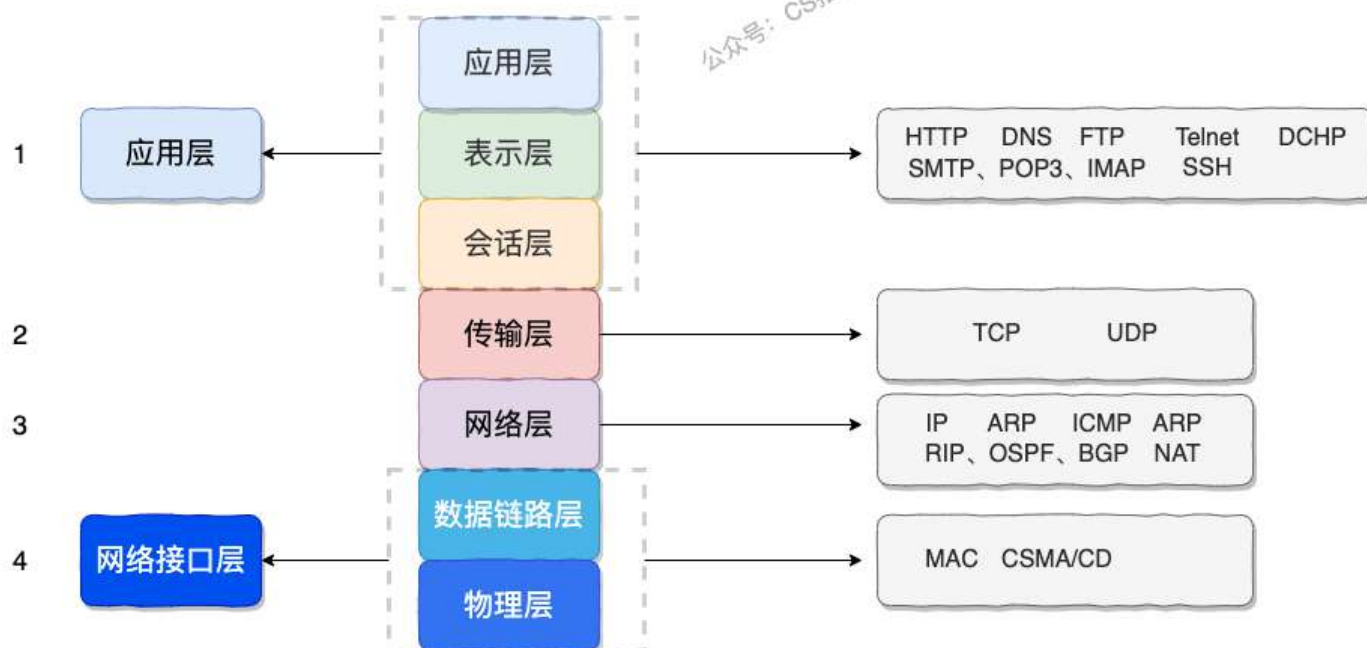
我们可以把网络接口层看作是数据链路层和物理层的合体。

1. 数据链路层(data link layer)通常简称为链路层（两台主机之间的数据传输，总是在一段一段的链路上传送的）。数据链路层的作用是将网络层交下来的 IP 数据报组装成帧，在两个相邻节点间的链路上传送帧。每一帧包括数据和必要的控制信息（如同步信息，地址信息，差错控制等）。
2. 物理层的作用是实现相邻计算机节点之间比特流的透明传送，尽可能屏蔽掉具体传输介质和物理设备的差异



总结

简单总结一下每一层包含的协议和核心技术:



应用层协议：

- HTTP 协议（超文本传输协议，网页浏览常用的协议）
- DHCP 协议（动态主机配置）
- DNS 系统原理（域名系统）
- FTP 协议（文件传输协议）
- Telnet协议（远程登陆协议）
- 电子邮件协议等（SMTP、POP3、IMAP）
-

传输层协议：

- TCP 协议
 - 报文段结构
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- UDP 协议
 - 报文段结构
 - RDT（可靠数据传输协议）

网络层协议：

- IP 协议（TCP/IP 协议的基础，分为 IPv4 和 IPv6）

- ARP 协议（地址解析协议，用于解析 IP 地址和 MAC 地址之间的映射）
- ICMP 协议（控制报文协议，用于发送控制消息）
- NAT 协议（网络地址转换协议）
- RIP 协议、OSPF 协议、BGP 协议（路由选择协议）
-

网络接口层：

- 差错检测技术
- 多路访问协议（信道复用技术）
- CSMA/CD 协议
- MAC 协议
- 以太网技术
-

为什么网络要分层？

说到分层，我们先从我们平时使用框架开发一个后台程序来说，我们往往会按照每一层做不同的事情的原则将系统分为三层（复杂的系统分层会更多）：

1. Repository（数据库操作）
2. Service（业务操作）
3. Controller（前后端数据交互）

复杂的系统需要分层，因为每一层都需要专注于一类事情。网络分层的原因也是一样，每一层只专注于做一类事情。

好了，再来说回：“为什么网络要分层？”。我觉得主要有 3 方面的原因：

1. **各层之间相互独立**：各层之间相互独立，各层之间不需要关心其他层是如何实现的，只需要知道自己如何调用下层提供好的功能就可以了（可以简单理解为接口调用）。这个和我们对开发时系统进行分层是一个道理。
2. **提高了整体灵活性**：每一层都可以使用最适合的技术来实现，你只需要保证你提供的功能以及暴露的接口的规则没有改变就行了。这个和我们平时开发系统的时候要求的高内聚、低耦合的原则也是可以对应上的。
3. **大问题化小**：分层可以将复杂的网络问题分解为许多比较小的、界线比较清晰简单的小问题来处理 and 解决。这样使得复杂的计算机网络系统变得易于设计，实现和标准化。这个和我们平时开发的时候，一般会将系统功能分解，然后将复杂的问题分解为容易理解的更小的问题是相对应的，这些较小的问题具有更好的边界（目标和接口）定义。

我想到了计算机世界非常非常有名的一句话，这里分享一下：

计算机科学领域的任何问题都可以通过增加一个间接的中间层来解决，计算机整个体系从上到下都是按照严格的层次结构设计的。

TCP 与 UDP 的区别（重要）

1. **是否面向连接**：UDP 在传送数据之前不需要先建立连接。而 TCP 提供面向连接的服务，在传送数据之前必须先建立连接，数据传送结束后要释放连接。
2. **是否是可靠传输**：远地主机在收到 UDP 报文后，不需要给出任何确认，并且不保证数据不丢失，不保证是否顺序到达。TCP 提供可靠的传输服务，TCP 在传递数据之前，会有三次握手来建立连接，而且在数据传递时，有确认、窗口、重传、拥塞控制机制。通过 TCP 连接传输的数据，无差错、不丢失、不重复、并且按序到达。
3. **是否有状态**：这个和上面的“是否可靠传输”相对应。TCP 传输是有状态的，这个有状态说的是 TCP 会去记录自己发送消息的状态比如消息是否发送了、是否被接收了等等。为此，TCP 需要维持复杂的连接状态表。而 UDP 是无状态服务，简单来说就是不管发出去之后的事情了（这很渣男！）。
4. **传输效率**：由于使用 TCP 进行传输的时候多了连接、确认、重传等机制，所以 TCP 的传输效率要比 UDP 低很多。
5. **传输形式**：TCP 是面向字节流的，UDP 是面向报文的。
6. **首部开销**：TCP 首部开销（20 ~ 60 字节）比 UDP 首部开销（8 字节）要大。
7. **是否提供广播或多播服务**：TCP 只支持点对点通信，UDP 支持一对一、一对多、多对一、多对多；
8.

我把上面总结的内容通过表格形式展示出来了！确定不点个赞嘛？

	TCP	UDP
是否面向连接	是	否
是否可靠	是	否
是否有状态	是	否
传输效率	较慢	较快
传输形式	字节流	数据报文段
首部开销	20 ~ 60 bytes	8 bytes
是否提供广播或多播服务	否	是

什么时候选择 TCP,什么时候选 UDP?

- **UDP** 一般用于即时通信，比如：语音、视频、直播等等。这些场景对传输数据的准确性要求不是特别高，比如你看视频即使少个一两帧，实际给人的感觉区别也不大。
- **TCP** 用于对传输准确性要求特别高的场景，比如文件传输、发送和接收邮件、远程登录等等。

使用 TCP 的协议有哪些?使用 UDP 的协议有哪些?

运行于 TCP 协议之上的协议：

1. **HTTP 协议**：超文本传输协议（HTTP，HyperText Transfer Protocol)主要是为 Web 浏览器与 Web 服务器之间的通信而设计的。当我们使用浏览器浏览网页的时候，我们网页就是通过 HTTP 请求进行加载的。
2. **HTTPS 协议**：更安全的超文本传输协议(HTTPS,Hypertext Transfer Protocol Secure)，身披 SSL 外衣的 HTTP 协议
3. **FTP 协议**：文件传输协议 FTP（File Transfer Protocol），提供文件传输服务，**基于 TCP** 实现可靠的传输。使用 FTP 传输文件的好处是可以屏蔽操作系统和文件存储方式。
4. **SMTP 协议**：简单邮件传输协议（SMTP，Simple Mail Transfer Protocol）的缩写，**基于 TCP** 协议，用来发送电子邮件。注意 ⚠️：接受邮件的协议不是 SMTP 而是 POP3 协议。
5. **POP3/IMAP 协议**：POP3 和 IMAP 两者都是负责邮件接收的协议。
6. **Telnet 协议**：远程登陆协议，通过一个终端登陆到其他服务器。被一种称为 SSH 的非常安全的协议所取代。
7. **SSH 协议**：SSH（Secure Shell）是目前较可靠，专为远程登录会话和其他网络服务提供安全性的协议。利用 SSH 协议可以有效防止远程管理过程中的信息泄露问题。SSH 建立在可靠的传

输协议 TCP 之上。

8.

运行于 UDP 协议之上的协议：

1. **DHCP** 协议：动态主机配置协议，动态配置 IP 地址
2. **DNS**：域名系统（DNS, Domain Name System）将人类可读的域名 (例如, www.baidu.com) 转换为机器可读的 IP 地址 (例如, 220.181.38.148)。我们可以将其理解为专为互联网设计的电话簿。实际上 DNS 同时支持 UDP 和 TCP 协议。

TCP 三次握手和四次挥手（非常重要）

相关面试题：

- 为什么要三次握手？
- 第 2 次握手传回了 ACK，为什么还要传回 SYN？
- 为什么要四次挥手？
- 为什么不能把服务器发送的 ACK 和 FIN 合并起来，变成三次挥手？
- 如果第二次挥手时服务器的 ACK 没有送达客户端，会怎样？
- 为什么第四次挥手客户端需要等待 $2 \times \text{MSL}$ （报文段最长寿命）时间后才进入 CLOSED 状态？

参考答案：[TCP 三次握手和四次挥手（传输层）](#)。

TCP 如何保证传输的可靠性？（重要）

[TCP 传输可靠性保障（传输层）](#)

从输入 URL 到页面展示到底发生了什么？（非常重要）

类似的问题：打开一个网页，整个过程会使用哪些协议？

图解（图片来源：《图解 HTTP》）：

过程	使用的协议
1. 浏览器查找域名的IP地址 (DNS查找过程: 浏览器缓存、路由器缓存、DNS 缓存)	DNS: 获取域名对应IP
2. 浏览器向web服务器发送一个HTTP请求 (cookies会随着请求发送给服务器)	
3. 服务器处理请求 (请求 处理请求 & 它的参数、cookies、生成一个HTML 响应)	• TCP: 与服务器建立TCP连接 • IP: 建立TCP协议时, 需要发送数据, 发送数据在网络层使用IP协议 • OPSF: IP数据包在路由器之间, 路由选择使用OPSF协议 • ARP: 路由器在与服务器通信时, 需要将ip地址转换为MAC地址, 需要使用ARP协议 • HTTP: 在TCP建立完成后, 使用HTTP协议访问网页
4. 服务器发回一个HTML响应	
5. 浏览器开始显示HTML	

上图有一个错误, 请注意, 是 OSPF 不是 OPSF。OSPF (Open Shortest Path First, ospf) 开放最短路径优先协议, 是由 Internet 工程任务组开发的路由选择协议

总体来说分为以下几个过程:

1. DNS 解析
2. TCP 连接
3. 发送 HTTP 请求
4. 服务器处理请求并返回 HTTP 报文
5. 浏览器解析渲染页面
6. 连接结束

具体可以参考下面这两篇文章:

- [从输入URL到页面加载发生了什么?](#)
- [浏览器从输入网址到页面展示的过程](#)

HTTP 状态码有哪些？

HTTP 状态码用于描述 HTTP 请求的结果，比如2xx 就代表请求被成功处理。

	类别	原因短语
1XX	Informational（信息性状态码）	接收的请求正在处理
2XX	Success（成功状态码）	请求正常处理完毕
3XX	Redirection（重定向状态码）	需要进行附加操作以完成请求
4XX	Client Error（客户端错误状态码）	服务器无法处理请求
5XX	Server Error（服务器错误状态码）	服务器处理请求出错

关于 HTTP 状态码更详细的总结，可以看我写的这篇文章：[HTTP 常见状态码总结（应用层）](#)。

HTTP 和 HTTPS 有什么区别？（重要）

- 端口号：HTTP 默认是 80，HTTPS 默认是 443。
- URL 前缀：HTTP 的 URL 前缀是 `http://`，HTTPS 的 URL 前缀是 `https://`。
- 安全性和资源消耗：HTTP 协议运行在 TCP 之上，所有传输的内容都是明文，客户端和服务端都无法验证对方的身份。HTTPS 是运行在 SSL/TLS 之上的 HTTP 协议，SSL/TLS 运行在 TCP 之上。所有传输的内容都经过加密，加密采用对称加密，但对称加密的密钥用服务器方的证书进行了非对称加密。所以说，HTTP 安全性没有 HTTPS 高，但是 HTTPS 比 HTTP 耗费更多服务器资源。

关于 HTTP 和 HTTPS 更详细的对比总结，可以看我写的这篇文章：[HTTP vs HTTPS（应用层）](#)。

HTTP 1.0 和 HTTP 1.1 有什么区别？

- 连接方式：HTTP 1.0 为短连接，HTTP 1.1 支持长连接。
- 状态响应码：HTTP/1.1中新加入了大量的状态码，光是错误响应状态码就新增了24种。比如说，`100 (Continue)` ——在请求大资源前的预热请求，`206 (Partial Content)` ——范围请求的标识码，`409 (Conflict)` ——请求与当前资源的规定冲突，`410 (Gone)` ——资源已被永久转移，而且没有任何已知的转发地址。
- 缓存处理：在 HTTP1.0 中主要使用 header 里的 `If-Modified-Since`,`Expires` 来做为缓存判断的标准，HTTP1.1 则引入了更多的缓存控制策略例如 `Entity tag`, `If-Unmodified-Since`, `If-Match`, `If-None-Match` 等更多可供选择的缓存头来控制缓存策略。
- 带宽优化及网络连接的使用：HTTP1.0 中，存在一些浪费带宽的现象，例如客户端只是需要某个对象的一部分，而服务器却将整个对象送过来了，并且不支持断点续传功能，HTTP1.1 则在请

求头引入了 range 头域，它允许只请求资源的某个部分，即返回码是 206 (Partial Content)，这样就方便了开发者自由的选择以便于充分利用带宽和连接。

- **Host头处理**：HTTP/1.1在请求头中加入了 Host 字段。

关于 HTTP 1.0 和 HTTP 1.1 更详细的对比总结，可以看我写的这篇文章：[HTTP 1.0 vs HTTP 1.1 \(应用层\)](#)。

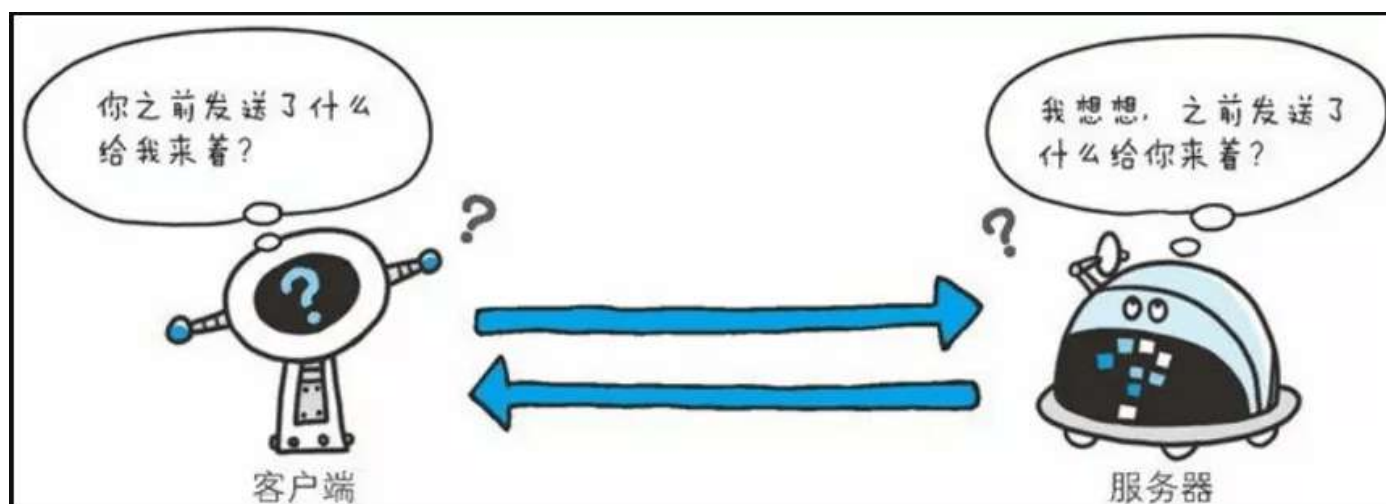
HTTP 是不保存状态的协议, 如何保存用户状态?

HTTP 是一种不保存状态，即无状态 (stateless) 协议。也就是说 HTTP 协议自身不对请求和响应之间的通信状态进行保存。那么我们保存用户状态呢？Session 机制的存在就是为了解决这个问题，Session 的主要作用就是通过服务端记录用户的状态。典型的场景是购物车，当你要添加商品到购物车的时候，系统不知道是哪个用户操作的，因为 HTTP 协议是无状态的。服务端给特定的用户创建特定的 Session 之后就可以标识这个用户并且跟踪这个用户了（一般情况下，服务器会在一定时间内保存这个 Session，过了时间限制，就会销毁这个 Session）。

在服务端保存 Session 的方法很多，最常用的就是内存和数据库(比如是使用内存数据库 redis 保存)。既然 Session 存放在服务器端，那么我们如何实现 Session 跟踪呢？大部分情况下，我们都是通过在 Cookie 中附加一个 Session ID 来方式来跟踪。

Cookie 被禁用怎么办？

最常用的就是利用 URL 重写把 Session ID 直接附加在 URL 路径的后面。



URI 和 URL 的区别是什么？

- URI(Uniform Resource Identifier) 是统一资源标志符，可以唯一标识一个资源。
- URL(Uniform Resource Locator) 是统一资源定位符，可以提供该资源的路径。它是一种具体的 URI，即 URL 可以用来标识一个资源，而且还指明了如何 locate 这个资源。

URI 的作用像身份证号一样，URL 的作用更像家庭住址一样。URL 是一种具体的 URI，它不仅唯一标识资源，而且还提供了定位该资源的信息。



JavaGuide

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

扫码关注

3.2 操作系统

操作系统基础

面试官顶着蓬松的假发向我走来，只见他一手拿着厚重的 Thinkpad，一手提着他那淡黄的长裙。

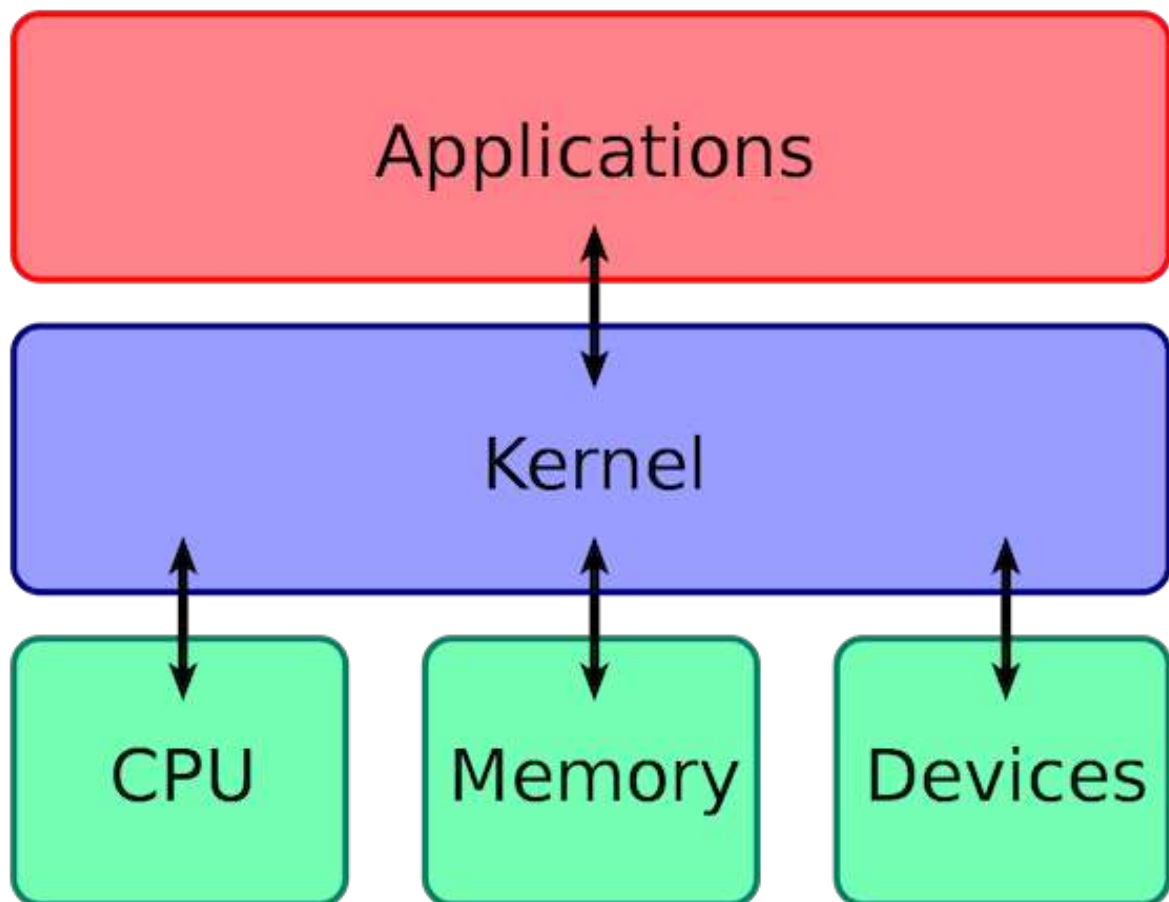
什么是操作系统？

 **面试官：** 先来个简单问题吧！什么是操作系统？

 **我：** 我通过以下四点向您介绍一下什么是操作系统吧！

1. 操作系统（Operating System，简称 OS）是管理计算机硬件与软件资源的程序，是计算机的基石。
2. 操作系统本质上是一个运行在计算机上的软件程序，用于管理计算机硬件和软件资源。举例：运行在你电脑上的所有应用程序都通过操作系统来调用系统内存以及磁盘等等硬件。

3. 操作系统存在屏蔽了硬件层的复杂性。操作系统就像是硬件使用的负责人，统筹着各种相关事项。
4. 操作系统的内核（Kernel）是操作系统的核心部分，它负责系统的内存管理，硬件设备的管理，文件系统的管理以及应用程序的管理。内核是连接应用程序和硬件的桥梁，决定着系统的性能和稳定性。



系统调用

面试官：什么是系统调用呢？能不能详细介绍一下。

我：介绍系统调用之前，我们先来了解一下用户态和系统态。

根据进程访问资源的特点，我们可以把进程在系统上的运行分为两个级别：

1. 用户态(user mode)：用户态运行的进程可以直接读取用户程序的数据。
2. 系统态(kernel mode):可以简单的理解系统态运行的进程或程序几乎可以访问计算机的任何资源，不受限制。

说了用户态和系统态之后，那么什么是系统调用呢？

我们运行的程序基本都是运行在用户态，如果我们调用操作系统提供的系统态级别的子功能咋办呢？那就需要系统调用了！

也就是说在我们运行的用户程序中，凡是与系统态级别的资源有关的操作（如文件管理、进程控制、内存管理等），都必须通过系统调用方式向操作系统提出服务请求，并由操作系统代为完成。

这些系统调用按功能大致可分为如下几类：

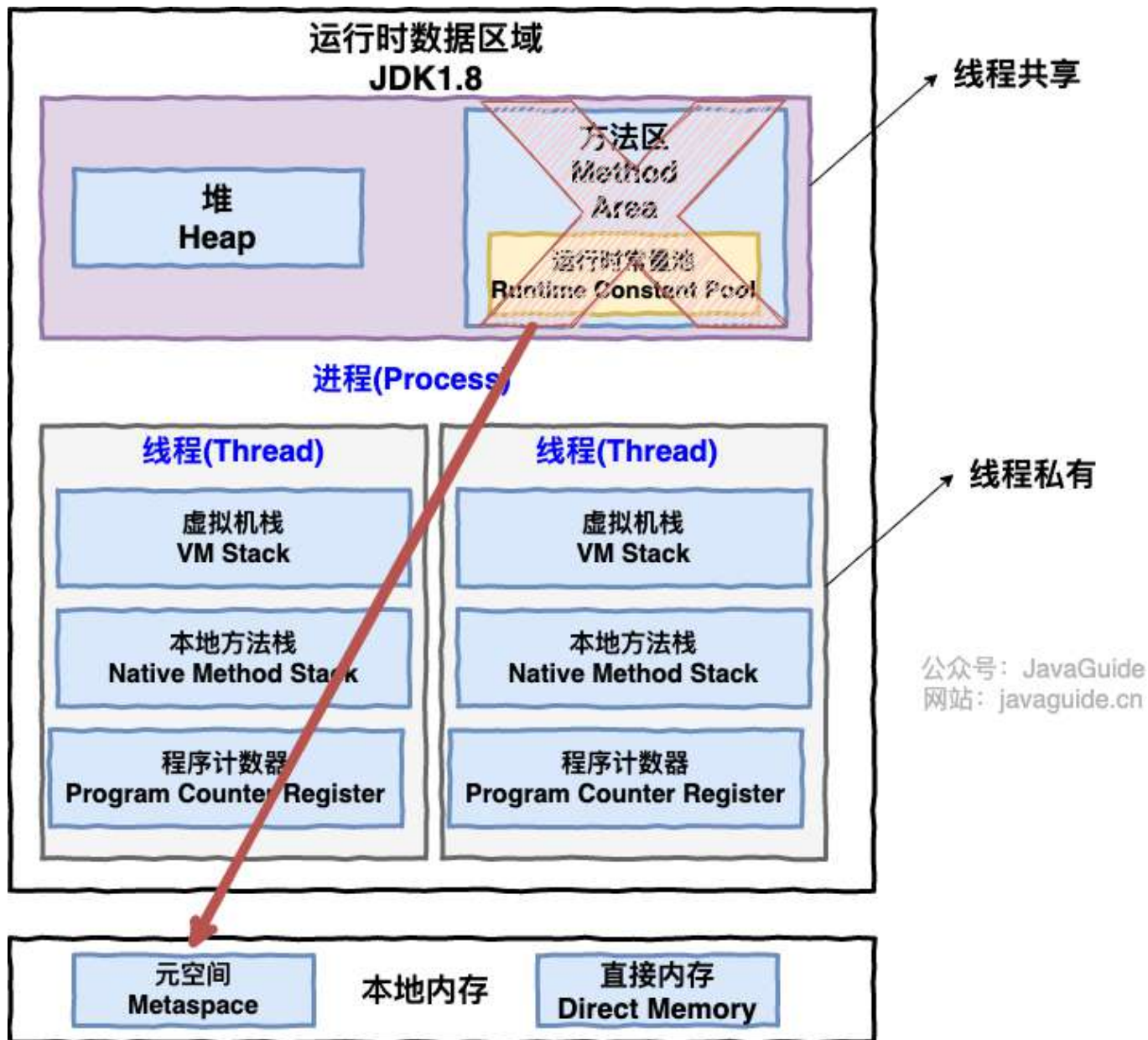
- 设备管理。完成设备的请求或释放，以及设备启动等功能。
- 文件管理。完成文件的读、写、创建及删除等功能。
- 进程控制。完成进程的创建、撤销、阻塞及唤醒等功能。
- 进程通信。完成进程之间的消息传递或信号传递等功能。
- 内存管理。完成内存的分配、回收以及获取作业占用内存区大小及地址等功能。

进程和线程

进程和线程的区别

 面试官: 好的！我明白了！那你再说一下：**进程和线程的区别**。

 我： 好的！ 下图是 Java 内存区域，我们从 JVM 的角度来说一下线程和进程之间的关系吧！



从上图可以看出：一个进程中可以有多个线程，多个线程共享进程的堆和方法区 (JDK1.8 之后的元空间)资源，但是每个线程有自己的程序计数器、虚拟机栈 和 本地方法栈。

总结： 线程是进程划分成的更小的运行单位,一个进程在其执行的过程中可以产生多个线程。线程和进程最大的不同在于基本上各进程是独立的，而各线程则不一定，因为同一进程中的线程极有可能会相互影响。线程执行开销小，但不利于资源的管理和保护；而进程正相反。

进程有哪几种状态？

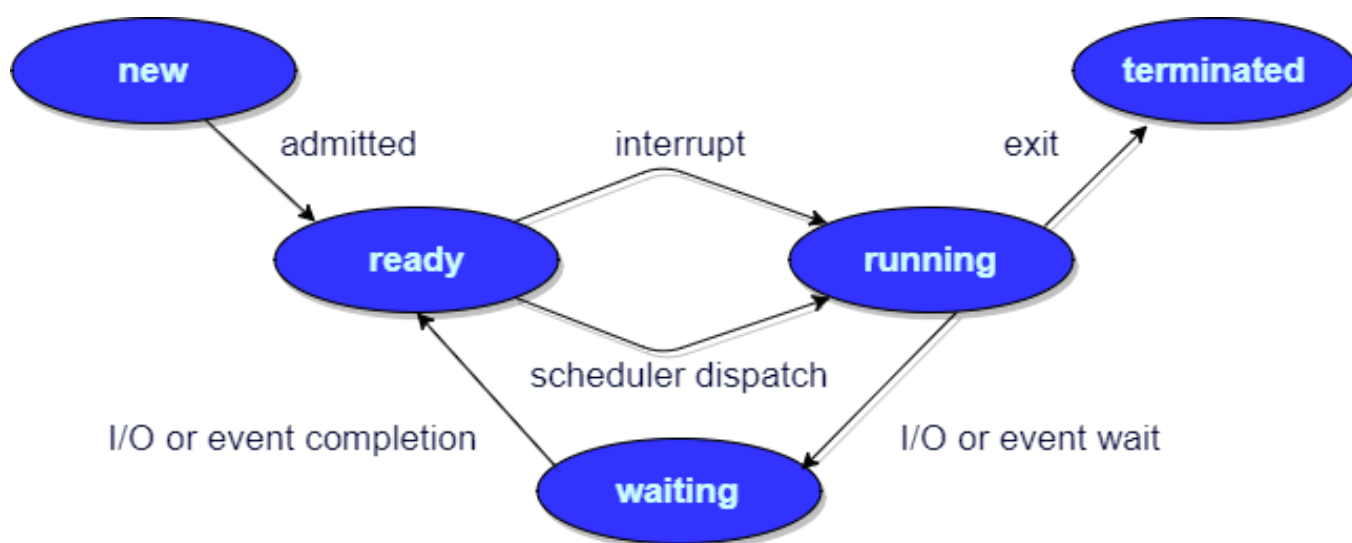
面试官： 那你再说说进程有哪几种状态？

我： 我们一般把进程大致分为 5 种状态，这一点和线程很像！

- **创建状态(new)：** 进程正在被创建，尚未到就绪状态。

- **就绪状态(ready)**：进程已处于准备运行状态，即进程获得了除了处理器之外的一切所需资源，一旦得到处理器资源(处理器分配的时间片)即可运行。
- **运行状态(running)**：进程正在处理器上运行(单核 CPU 下任意时刻只有一个进程处于运行状态)。
- **阻塞状态(waiting)**：又称为等待状态，进程正在等待某一事件而暂停运行如等待某资源为可用或等待 IO 操作完成。即使处理器空闲，该进程也不能运行。
- **结束状态(terminated)**：进程正在从系统中消失。可能是进程正常结束或其他原因中断退出运行。

订正：下图中 running 状态被 interrupt 向 ready 状态转换的箭头方向反了。



进程间的通信方式

面试官：进程间的通信常见的有哪几种方式呢？

我：大概有 7 种常见的进程间的通信方式。

下面这部分总结参考了：《[进程间通信 IPC \(InterProcess Communication\)](#)》这篇文章，推荐阅读，总结的非常不错。

1. **管道/匿名管道(Pipes)**：用于具有亲缘关系的父子进程间或者兄弟进程之间的通信。
2. **有名管道(Named Pipes)**：匿名管道由于没有名字，只能用于亲缘关系的进程间通信。为了克服这个缺点，提出了有名管道。有名管道严格遵循先进先出(first in first out)。有名管道以磁盘文件的方式存在，可以实现本机任意两个进程通信。
3. **信号(Signal)**：信号是一种比较复杂的通信方式，用于通知接收进程某个事件已经发生；
4. **消息队列(Message Queuing)**：消息队列是消息的链表,具有特定的格式,存放在内存中并由消

息队列标识符标识。管道和消息队列的通信数据都是先进先出的原则。与管道（无名管道：只存在于内存中的文件；命名管道：存在于实际的磁盘介质或者文件系统）不同的是消息队列存放在内核中，只有在内核重启(即，操作系统重启)或者显式地删除一个消息队列时，该消息队列才会被真正的删除。消息队列可以实现消息的随机查询,消息不一定要以先进先出的次序读取,也可以按消息的类型读取.比 FIFO 更有优势。消息队列克服了信号承载信息量少，管道只能承载无格式字节流以及缓冲区大小受限等缺点。

5. **信号量(Semaphores)**：信号量是一个计数器，用于多进程对共享数据的访问，信号量的意图在于进程间同步。这种通信方式主要用于解决与同步相关的问题并避免竞争条件。
6. **共享内存(Shared memory)**：使得多个进程可以访问同一块内存空间，不同进程可以及时看到对方进程中对共享内存中数据的更新。这种方式需要依靠某种同步操作，如互斥锁和信号量等。可以说这是最有用的进程间通信方式。
7. **套接字(Sockets)**：此方法主要用于在客户端和服务端之间通过网络进行通信。套接字是支持 TCP/IP 的网络通信的基本操作单元，可以看做是不同主机之间的进程进行双向通信的端点，简单的说就是通信的两方的一种约定，用套接字中的相关函数来完成通信过程。

线程间的同步的方式

 面试官：那线程间的同步的方式有哪些呢？

 我：线程同步是两个或多个共享关键资源的线程的并发执行。应该同步线程以避免关键的资源使用冲突。操作系统一般有下面三种线程同步的方式：

1. **互斥量(Mutex)**：采用互斥对象机制，只有拥有互斥对象的线程才有访问公共资源的权限。因为互斥对象只有一个，所以可以保证公共资源不会被多个线程同时访问。比如 Java 中的 synchronized 关键词和各种 Lock 都是这种机制。
2. **信号量(Semaphore)**：它允许同一时刻多个线程访问同一资源，但是需要控制同一时刻访问此资源的最大线程数量。
3. **事件(Event)** :Wait/Notify：通过通知操作的方式来保持多线程同步，还可以方便的实现多线程优先级的比较操作。

进程的调度算法

 面试官：你知道操作系统中进程的调度算法有哪些吗？

 我：嗯嗯！这个我们大学的时候学过，是一个很重要的知识点！

为了确定首先执行哪个进程以及最后执行哪个进程以实现最大 CPU 利用率，计算机科学家已经定义了一些算法，它们是：

- **先到先服务(FCFS)调度算法**：从就绪队列中选择一个最先进入该队列的进程为之分配资源，使它立即执行并一直执行到完成或发生某事件而被阻塞放弃占用 CPU 时再重新调度。

- **短作业优先(SJF)的调度算法**：从就绪队列中选出一个估计运行时间最短的进程为之分配资源，使它立即执行并一直执行到完成或发生某事件而被阻塞放弃占用 CPU 时再重新调度。
- **时间片轮转调度算法**：时间片轮转调度是一种最古老，最简单，最公平且使用最广的算法，又称 RR(Round robin)调度。每个进程被分配一个时间段，称作它的时间片，即该进程允许运行的时间。
- **多级反馈队列调度算法**：前面介绍的几种进程调度的算法都有一定的局限性。如**短进程优先的调度算法**，仅照顾了短进程而忽略了长进程。多级反馈队列调度算法既能使高优先级的作业得到响应又能使短作业（进程）迅速完成。，因而它是目前被公认的一种较好的进程调度算法，UNIX 操作系统采取的便是这种调度算法。
- **优先级调度**：为每个流程分配优先级，首先执行具有最高优先级的进程，依此类推。具有相同优先级的进程以 FCFS 方式执行。可以根据内存要求，时间要求或任何其他资源要求来确定优先级。

什么是死锁

面试官：你知道什么是死锁吗？

我：死锁描述的是这样一种情况：多个进程/线程同时被阻塞，它们中的一个或者全部都在等待某个资源被释放。由于进程/线程被无限期地阻塞，因此程序不可能正常终止。

死锁的四个条件

面试官：产生死锁的四个必要条件是什么？

我：如果系统中以下四个条件同时成立，那么就能引起死锁：

- **互斥**：资源必须处于非共享模式，即一次只有一个进程可以使用。如果另一进程申请该资源，那么必须等待直到该资源被释放为止。
- **占有并等待**：一个进程至少应该占有一个资源，并等待另一资源，而该资源被其他进程所占有。
- **非抢占**：资源不能被抢占。只能在持有资源的进程完成任务后，该资源才会被释放。
- **循环等待**：有一组等待进程 $\{P_0, P_1, \dots, P_n\}$ ， P_0 等待的资源被 P_1 占有， P_1 等待的资源被 P_2 占有，……， P_{n-1} 等待的资源被 P_n 占有， P_n 等待的资源被 P_0 占有。

注意，只有四个条件同时成立时，死锁才会出现。

解决死锁的方法

解决死锁的方法可以从多个角度去分析，一般的情况下，有**预防**，**避免**，**检测**和**解除**四种。

- **预防** 是采用某种策略，**限制并发进程对资源的请求**，从而使得死锁的必要条件在系统执行的任何时间上都不满足。

- **避免**则是系统在分配资源时，根据资源的使用情况**提前做出预测**，从而**避免死锁的发生**
- **检测**是指系统设有**专门的机构**，当死锁发生时，该机构能够检测死锁的发生，并精确地确定与死锁有关的进程和资源。
- **解除**是与检测相配套的一种措施，用于**将进程从死锁状态下解脱出来**。

死锁的预防

死锁四大必要条件上面都已经列出来了，很显然，只要破坏四个必要条件中的任何一个就能够预防死锁的发生。

破坏第一个条件 **互斥条件**：使得资源是可以同时访问的，这是种简单的方法，磁盘就可以用这种方法管理，但是我们要知道，有很多资源 **往往是不能同时访问的**，所以这种做法在大多数的场合是行不通的。

破坏第三个条件 **非抢占**：也就是说可以采用 **剥夺式调度算法**，但剥夺式调度方法目前一般仅适用于**主存资源** 和 **处理器资源** 的分配，并不适用于所有的资源，会导致 **资源利用率下降**。

所以一般比较实用的 **预防死锁的方法**，是通过考虑破坏第二个条件和第四个条件。

1、静态分配策略

静态分配策略可以破坏死锁产生的第二个条件（占有并等待）。所谓静态分配策略，就是指一个进程必须在执行前就申请到它所需要的全部资源，并且知道它所要的资源都得到满足之后才开始执行。进程要么占有所有的资源然后开始执行，要么不占有资源，不会出现占有有一些资源等待一些资源的情况。

静态分配策略逻辑简单，实现也很容易，但这种策略 **严重地降低了资源利用率**，因为在每个进程所占有的资源中，有些资源是在比较靠后的执行时间里采用的，甚至有些资源是在额外的情况下才是用的，这样就可能造成了一个进程占有了一些 **几乎不用的资源而使其他需要该资源的进程产生等待** 的情况。

2、层次分配策略

层次分配策略破坏了产生死锁的第四个条件(循环等待)。在层次分配策略下，所有的资源被分成了多个层次，一个进程得到某一次的一个资源后，它只能再申请较高一层的资源；当一个进程要释放某层的一个资源时，必须先释放所占用的较高层的资源，按这种策略，是不可能出现循环等待链的，因为那样的话，就出现了已经申请了较高层的资源，反而去申请了较低层的资源，不符合层次分配策略，证明略。

死锁的避免

上面提到的 **破坏** 死锁产生的四个必要条件之一就可以成功 **预防系统发生死锁**，但是会导致 **低效的进程运行** 和 **资源使用率**。而死锁的避免相反，它的角度是允许系统中**同时存在四个必要条件**，只要掌握并发进程中与每个进程有关的资源动态申请情况，做出 **明智和合理的选择**，仍然可以避免死锁，因为四大条件仅仅是产生死锁的必要条件。

我们将系统的状态分为 **安全状态** 和 **不安全状态**，每当在未申请者分配资源前先测试系统状态，若把系统资源分配给申请者会产生死锁，则拒绝分配，否则接受申请，并为它分配资源。

如果操作系统能够保证所有的进程在有限的时间内得到需要的全部资源，则称系统处于安全状态，否则说系统是不安全的。很显然，系统处于安全状态则不会发生死锁，系统若处于不安全状态则可能发生死锁。

那么如何保证系统保持在安全状态呢？通过算法，其中最具有代表性的 **避免死锁算法** 就是 Dijkstra 的银行家算法，银行家算法用一句话表达就是：当一个进程申请使用资源的时候，**银行家算法** 通过先 **试探** 分配给该进程资源，然后通过 **安全性算法** 判断分配后系统是否处于安全状态，若不安全则试探分配作废，让该进程继续等待，若能够进入到安全的状态，则就 **真的分配资源给该进程**。

银行家算法详情可见：[《一句话+一张图说清楚——银行家算法》](#)。

操作系统教程树中讲述的银行家算法也比较清晰，可以一看。

死锁的避免(银行家算法)改善解决了 **资源使用率低的问题**，但是它要不断地检测每个进程对各类资源的占用和申请情况，以及做 **安全性检查**，需要花费较多的时间。

死锁的检测

对资源的分配加以限制可以 **预防和避免** 死锁的发生，但是都不利于各进程对系统资源的**充分共享**。解决死锁问题的另一条途径是 **死锁检测和解除** (这里突然联想到了乐观锁和悲观锁，感觉死锁的检测和解除就像是 **乐观锁**，分配资源时不去提前管会不会发生死锁了，等到真的死锁出现了再来解决嘛，而 **死锁的预防和避免** 更像是悲观锁，总是觉得死锁会出现，所以在分配资源的时候就很谨慎)。

这种方法对资源的分配不加以任何限制，也不采取死锁避免措施，但系统 **定时地运行一个“死锁检测”** 的程序，判断系统内是否出现死锁，如果检测到系统发生了死锁，再采取措施去解除它。

进程-资源分配图

操作系统中的每一时刻的**系统状态**都可以用**进程-资源分配图**来表示，进程-资源分配图是描述进程和资源申请及分配关系的一种有向图，可用于**检测系统是否处于死锁状态**。

用一个方框表示每一个资源类，方框中的黑点表示该资源类中的各个资源，每个键进程用一个圆圈表示，用 有向边 来表示进程申请资源和资源被分配的情况。

图中 2-21 是进程-资源分配图的一个例子，其中共有三个资源类，每个进程的资源占有和申请情况已清楚地表示在图中。在这个例子中，由于存在 占有和等待资源的环路，导致一组进程永远处于等待资源的状态，发生了 死锁。

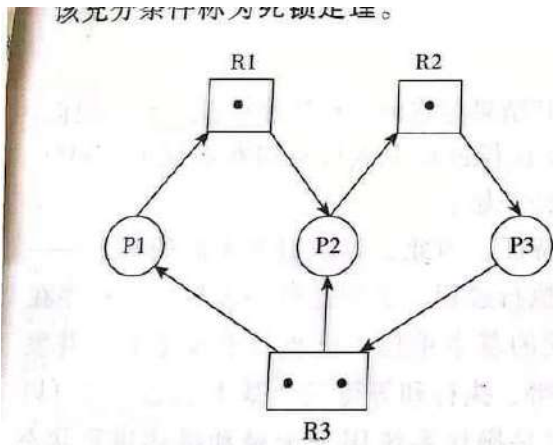


图 2-21 有环路有死锁进程资源分配图

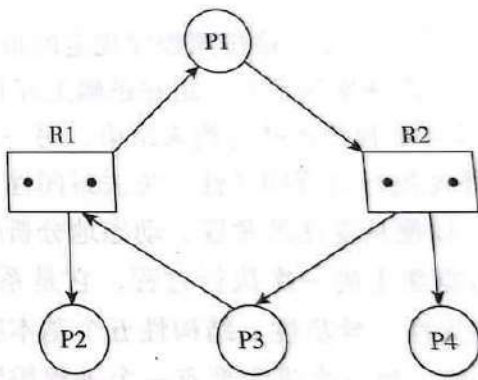


图 2-22 有环路无死锁进程资源分配图

进程-资源分配图中存在环路并不一定是发生了死锁。因为循环等待资源仅仅是死锁发生的必要条件，而不是充分条件。图 2-22 便是一个有环路而无死锁的例子。虽然进程 P1 和进程 P3 分别占用了资源 R1 和资源 R2，并且因为等待另一个资源 R2 和另一个资源 R1 形成了环路，但进程 P2 和进程 P4 分别占有了资源 R1 和资源 R2，它们申请的资源得到了满足，在有限的时间内会归还资源，于是进程 P1 或 P3 都能获得另一个所需的资源，环路自动解除，系统也就不存在死锁状态了。

死锁检测步骤

知道了死锁检测的原理，我们可以利用下列步骤编写一个 死锁检测 程序，检测系统是否产生了死锁。

- 1. 如果进程-资源分配图中无环路，则此时系统没有发生死锁
- 2. 如果进程-资源分配图中有环路，且每个资源类仅有一个资源，则系统中已经发生了死锁。
- 3. 如果进程-资源分配图中有环路，且涉及到的资源类有多个资源，此时系统未必会发生死锁。如果能在进程-资源分配图中找出一个 既不阻塞又非独立的进程，该进程能够在有限的时间内归还占有的资源，也就是把边给消除掉了，重复此过程，直到能在有限的时间内 消除所有的边，则不会发生死锁，否则会发生死锁。(消除边的过程类似于 拓扑排序)

死锁的解除

当死锁检测程序检测到存在死锁发生时，应设法让其解除，让系统从死锁状态中恢复过来，常用的解除死锁的方法有以下四种：

1. **立即结束所有进程的执行，重新启动操作系统**：这种方法简单，但以前所在的工作全部作废，损失很大。
2. **撤销涉及死锁的所有进程，解除死锁后继续运行**：这种方法能彻底打破死锁的循环等待条件，但将付出很大代价，例如有些进程可能已经计算了很长时间，由于被撤销而使产生的部分结果也被消除了，再重新执行时还要再次进行计算。
3. **逐个撤销涉及死锁的进程，回收其资源直至死锁解除**。
4. **抢占资源**：从涉及死锁的一个或几个进程中抢占资源，把夺得的资源再分配给涉及死锁的进程直至死锁解除。

操作系统内存管理基础

内存管理介绍

 **面试官**：操作系统的内存管理主要是做什么？

 **我**：操作系统的内存管理主要负责内存的分配与回收（malloc 函数：申请内存，free 函数：释放内存），另外地址转换也就是将逻辑地址转换成相应的物理地址等功能也是操作系统内存管理做的事情。

常见的几种内存管理机制

 **面试官**：操作系统的内存管理机制了解吗？内存管理有哪几种方式？

 **我**：这个在学习操作系统的时候有了解过。

简单分为**连续分配管理方式**和**非连续分配管理方式**这两种。连续分配管理方式是指为一个用户程序分配一个连续的内存空间，常见的如**块式管理**。同样地，非连续分配管理方式允许一个程序使用的内存分布在离散或者说不相邻的内存中，常见的如**页式管理**和**段式管理**。

1. **块式管理**：远古时代的计算机操作系统的内存管理方式。将内存分为几个固定大小的块，每个块中只包含一个进程。如果程序运行需要内存的话，操作系统就分配给它一块，如果程序运行只需要很小的空间的话，分配的这块内存很大一部分几乎被浪费了。这些在每个块中未被利用的空间，我们称之为碎片。
2. **页式管理**：把主存分为大小相等且固定的一页一页的形式，页较小，相比于块式管理的划分粒度更小，提高了内存利用率，减少了碎片。页式管理通过页表对应逻辑地址和物理地址。
3. **段式管理**：页式管理虽然提高了内存利用率，但是页式管理其中的页并无任何实际意义。段式

管理把主存分为一段段的，段是有实际意义的，每个段定义了一组逻辑信息，例如，有主程序段 MAIN、子程序段 X、数据段 D 及栈段 S 等。段式管理通过段表对应逻辑地址和物理地址。

简单来说：页是物理单位，段是逻辑单位。分页可以有效提高内存利用率，分段可以更好满足用户需求。

 **面试官**： 回答的还不错！不过漏掉了一个很重要的 **段页式管理机制**。段页式管理机制结合了段式管理和页式管理的优点。简单来说段页式管理机制就是把主存先分成若干段，每个段又分成若干页，也就是说 **段页式管理机制** 中段与段之间以及段的内部的都是离散的。

 **我**： 谢谢面试官！刚刚把这个给忘记了～

快表和多级页表

 **面试官**： 页表管理机制中有两个很重要的概念：快表和多级页表，这两个东西分别解决了页表管理中很重要的两个问题。你给我简单介绍一下吧！

 **我**： 在分页内存管理中，很重要的两点是：

1. 虚拟地址到物理地址的转换要快。
2. 解决虚拟地址空间大，页表也会很大的问题。

快表

为了提高虚拟地址到物理地址的转换速度，操作系统在 **页表方案** 基础之上引入了 **快表** 来加速虚拟地址到物理地址的转换。我们可以把快表理解为一种特殊的高速缓冲存储器（Cache），其中的内容是页表的一部分或者全部内容。作为页表的 Cache，它的作用与页表相似，但是提高了访问速率。由于采用页表做地址转换，读写内存数据时 CPU 要访问两次主存。有了快表，有时只要访问一次高速缓冲存储器，一次主存，这样可加速查找并提高指令执行速度。

使用快表之后的地址转换流程是这样的：

1. 根据虚拟地址中的页号查快表；
2. 如果该页在快表中，直接从快表中读取相应的物理地址；
3. 如果该页不在快表中，就访问内存中的页表，再从页表中得到物理地址，同时将页表中的该映射表项添加到快表中；
4. 当快表填满后，又要登记新页时，就按照一定的淘汰策略淘汰掉快表中的一个页。

看完了之后你会发现快表和我们平时经常在我们开发的系统使用的缓存（比如 Redis）很像，的确是这样的，操作系统中的很多思想、很多经典的算法，你都可以是我们日常开发使用的各种工具或者框架中找到它们的影子。

多级页表

引入多级页表的主要目的是为了避免把全部页表一直放在内存中占用过多空间，特别是那些根本就不需要的页表就不需要保留在内存中。

多级页表属于时间换空间的典型场景。

总结

为了提高内存的空间性能，提出了多级页表的概念；但是提到空间性能是以浪费时间性能为基础的，因此为了补充损失的时间性能，提出了快表（即 TLB）的概念。不论是快表还是多级页表实际上都利用到了程序的局部性原理，局部性原理在后面的虚拟内存这部分会介绍到。

分页机制和分段机制的共同点和区别

 面试官： 分页机制和分段机制有哪些共同点和区别呢？

 我：

1. 共同点：

- 分页机制和分段机制都是为了提高内存利用率，减少内存碎片。
- 页和段都是离散存储的，所以两者都是离散分配内存的方式。但是，每个页和段中的内存是连续的。

2. 区别：

- 页的大小是固定的，由操作系统决定；而段的大小不固定，取决于我们当前运行的程序。
- 分页仅仅是为了满足操作系统内存管理的需求，而段是逻辑信息的单位，在程序中可以体现为代码段，数据段，能够更好满足用户的需要。

逻辑(虚拟)地址和物理地址

 面试官： 你刚刚还提到了**逻辑地址**和**物理地址**这两个概念，我不太清楚，你能为我解释一下不？

 我： em...好的嘛！我们编程一般只有可能和逻辑地址打交道，比如在 C 语言中，指针里面存储的数值就可以理解成为内存里的一个地址，这个地址也就是我们说的逻辑地址，逻辑地址由操作系统决定。物理地址指的是真实物理内存中地址，更具体一点来说就是内存地址寄存器中的地址。物理地址是内存单元真正的地址。

CPU 寻址了解吗?为什么需要虚拟地址空间?

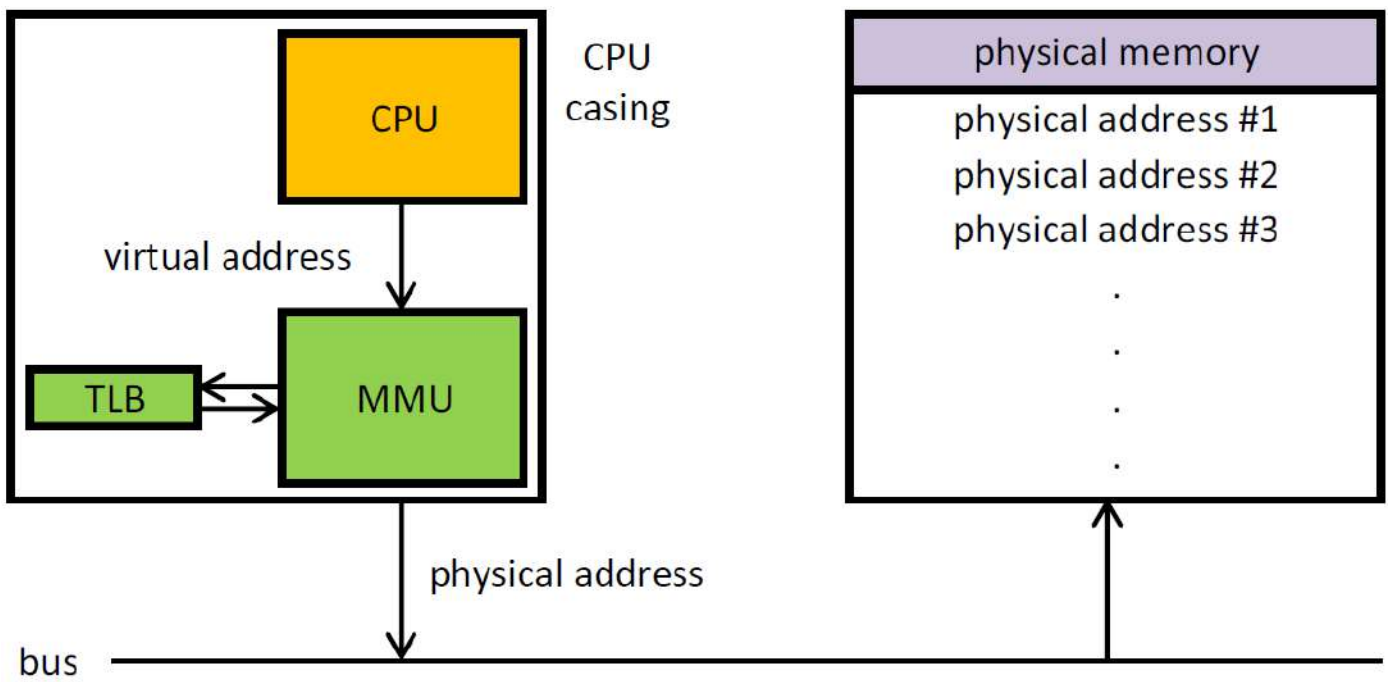
面试官：CPU 寻址了解吗?为什么需要虚拟地址空间?

我：这部分我真不清楚!

于是面试完之后我默默去查阅了相关文档! 留下了没有技术的泪水。。。

这部分内容参考了 Microsoft 官网的介绍, 地址: <https://docs.microsoft.com/zh-cn/windows-hardware/drivers/gettingstarted/virtual-address-spaces?redirectedfrom=MSDN>

现代处理器使用的是一种称为 **虚拟寻址(Virtual Addressing)** 的寻址方式。使用虚拟寻址, CPU 需要将虚拟地址翻译成物理地址, 这样才能访问到真实的物理内存。实际上完成虚拟地址转换为物理地址转换的硬件是 CPU 中含有一个被称为 **内存管理单元 (Memory Management Unit, MMU)** 的硬件。如下图所示:



CPU: Central Processing Unit
MMU: Memory Management Unit
TLB: Translation lookaside buffer

为什么要有虚拟地址空间呢?

先从没有虚拟地址空间的时候说起吧! 没有虚拟地址空间的时候, 程序直接访问和操作的都是物理内存。但是这样有什么问题呢?

1. 用户程序可以访问任意内存，寻址内存的每个字节，这样就很容易（有意或者无意）破坏操作系统，造成操作系统崩溃。
2. 想要同时运行多个程序特别困难，比如你想同时运行一个微信和一个 QQ 音乐都不行。为什么呢？举个简单的例子：微信在运行的时候给内存地址 1xxx 赋值后，QQ 音乐也同样给内存地址 1xxx 赋值，那么 QQ 音乐对内存的赋值就会覆盖微信之前所赋的值，这就造成了微信这个程序就会崩溃。

总结来说：如果直接把物理地址暴露出来的话会带来严重问题，比如可能对操作系统造成伤害以及给同时运行多个程序造成困难。

通过虚拟地址访问内存有以下优势：

- 程序可以使用一系列相邻的虚拟地址来访问物理内存中不相邻的大内存缓冲区。
- 程序可以使用一系列虚拟地址来访问大于可用物理内存的内存缓冲区。当物理内存的供应量变小时，内存管理器会将物理内存页（通常大小为 4 KB）保存到磁盘文件。数据或代码页会根据需要在物理内存与磁盘之间移动。
- 不同进程使用的虚拟地址彼此隔离。一个进程中的代码无法更改正在由另一进程或操作系统使用的物理内存。

虚拟内存

什么是虚拟内存(Virtual Memory)?

 面试官：再问你一个常识性的问题！什么是虚拟内存(Virtual Memory)?

 我：这个在我们平时使用电脑特别是 Windows 系统的时候太常见了。很多时候我们使用了很多占内存的软件，这些软件占用的内存可能已经远远超出了我们电脑本身具有的物理内存。为什么可以这样呢？正是因为 **虚拟内存** 的存在，通过 **虚拟内存** 可以让程序可以拥有超过系统物理内存大小的可用内存空间。另外，**虚拟内存** 为每个进程提供了一个一致的、私有的地址空间，它让每个进程产生了一种自己在独享主存的错觉（每个进程拥有一片连续完整的内存空间）。这样会更加有效地管理内存并减少出错。

虚拟内存是计算机系统内存管理的一种技术，我们可以手动设置自己电脑的虚拟内存。不要单纯认为虚拟内存只是“使用硬盘空间来扩展内存”的技术。**虚拟内存的重要意义是它定义了一个连续的虚拟地址空间，并且把内存扩展到硬盘空间。**推荐阅读：[《虚拟内存的那点事儿》](#)

维基百科中有几句话是这样介绍虚拟内存的。

虚拟内存 使得应用程序认为它拥有连续的可用的内存（一个连续完整的地址空间），而实际上，它通常是被分隔成多个物理内存碎片，还有部分暂时存储在外部磁盘存储器上，在需要进行数据交换。与没有使用虚拟内存技术的系统相比，使用这种技术的系统使得大型程序的编写变得更容易，对真正的物理内存（例如 RAM）的使用也更有效率。目前，大多数操作系统都使用了虚拟内存，如 Windows 家族的“虚拟内存”；Linux 的“交换空间”等。From:<https://zh.wikipedia.org/wiki/虚拟内存>

局部性原理

 **面试官**：要想更好地理解虚拟内存技术，必须要知道计算机中著名的**局部性原理**。另外，局部性原理既适用于程序结构，也适用于数据结构，是非常重要的一个概念。

 **我**：局部性原理是虚拟内存技术的基础，正是因为程序运行具有局部性原理，才可以只装入部分程序到内存就开始运行。

以下内容摘自《计算机操作系统教程》第 4 章存储器管理。

早在 1968 年的时候，就有人指出我们的程序在执行的时候往往呈现局部性规律，也就是说在某个较短的时间段内，程序执行局限于某一小部分，程序访问的存储空间也局限于某个区域。

局部性原理表现在以下两个方面：

1. **时间局部性**：如果程序中的某条指令一旦执行，不久以后该指令可能再次执行；如果某数据被访问过，不久以后该数据可能再次被访问。产生时间局部性的典型原因，是由于在程序中存在着大量的循环操作。
2. **空间局部性**：一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也将被访问，即程序在一段时间内所访问的地址，可能集中在一定的范围之内，这是因为指令通常是顺序存放、顺序执行的，数据也一般是以向量、数组、表等形式簇聚存储的。

时间局部性是通过将近来使用的指令和数据保存到高速缓存存储器中，并使用高速缓存的层次结构实现。空间局部性通常是使用较大的高速缓存，并将预取机制集成到高速缓存控制逻辑中实现。虚拟内存技术实际上就是建立了“内存—外存”的两级存储器的结构，利用局部性原理实现高速缓存。

虚拟存储器

勘误：虚拟存储器又叫做虚拟内存，都是 **Virtual Memory** 的翻译，属于同一个概念。

 面试官：都说子虚拟内存了。你再讲讲**虚拟存储器**把！

 我：

这部分内容来自：[王道考研操作系统知识点整理](#)。

基于局部性原理，在程序装入时，可以将程序的一部分装入内存，而将其他部分留在外存，就可以启动程序执行。由于外存往往比内存大很多，所以我们运行的软件的内存大小实际上是可以比计算机系统实际的内存大小大的。在程序执行过程中，当所访问的信息不在内存时，由操作系统将所需要的部分调入内存，然后继续执行程序。另一方面，操作系统将内存中暂时不使用的内容换到外存上，从而腾出空间存放将要调入内存的信息。这样，计算机好像为用户提供了一个比实际内存大得多的存储器——**虚拟存储器**。

实际上，我觉得虚拟内存同样是一种时间换空间的策略，你用 CPU 的计算时间，页的调入调出花费的时间，换来了一个虚拟的更大的空间来支持程序的运行。不得不感叹，程序世界几乎不是时间换空间就是空间换时间。

虚拟内存的技术实现

 面试官：虚拟内存技术的实现呢？

 我：虚拟内存的实现需要建立在离散分配的内存管理方式的基础上。虚拟内存的实现有以下三种方式：

1. **请求分页存储管理**：建立在分页管理之上，为了支持虚拟存储器功能而增加了请求调页功能和页面置换功能。请求分页是目前最常用的一种实现虚拟存储器的方法。请求分页存储管理系统中，在作业开始运行之前，仅装入当前要执行的部分段即可运行。假如在作业运行的过程中发现要访问的页面不在内存，则由处理器通知操作系统按照对应的页面置换算法将相应的页面调入到主存，同时操作系统也可以将暂时不用的页面置换到外存中。
2. **请求分段存储管理**：建立在分段存储管理之上，增加了请求调段功能、分段置换功能。请求分段存储管理方式就如同请求分页存储管理方式一样，在作业开始运行之前，仅装入当前要执行的部分段即可运行；在执行过程中，可使用请求调入中断动态装入要访问但又不在内存的程序段；当内存空间已满，而又需要装入新的段时，根据置换功能适当调出某个段，以便腾出空间而装入新的段。

3. 请求段页式存储管理

这里多说一下？很多人容易搞混请求分页与分页存储管理，两者有何不同呢？

请求分页存储管理建立在分页管理之上。他们的根本区别是是否将程序全部所需的全部地址空间都装入主存，这也是请求分页存储管理可以提供虚拟内存的原因，我们在上面已经分析过了。

它们之间的根本区别在于是否将一作业的全部地址空间同时装入主存。请求分页存储管理不要求将作业全部地址空间同时装入主存。基于这一点，请求分页存储管理可以提供虚存，而分页存储管理却不能提供虚存。

不管是上面那种实现方式，我们一般都需要：

1. 一定容量的内存和外存：在载入程序的时候，只需要将程序的一部分装入内存，而将其他部分留在外存，然后程序就可以执行了；
2. 缺页中断：如果需执行的指令或访问的数据尚未在内存（称为缺页或缺段），则由处理器通知操作系统将相应的页面或段调入到内存，然后继续执行程序；
3. 虚拟地址空间：逻辑地址到物理地址的变换。

页面置换算法

 **面试官：** 虚拟内存管理很重要的一个概念就是页面置换算法。那你说一下 页面置换算法的作用？常见的页面置换算法有哪些？

 **我：**

这个题目经常作为笔试题出现，网上已经给出了很不错的回答，我这里只是总结整理了一下。

地址映射过程中，若在页面中发现所要访问的页面不在内存中，则发生缺页中断。

缺页中断 就是要访问的页不在主存，需要操作系统将其调入主存后再进行访问。在这个时候，被内存映射的文件实际上成了一个分页交换文件。

当发生缺页中断时，如果当前内存中并没有空闲的页面，操作系统就必须在内存选择一个页面将其移出内存，以便为即将调入的页面让出空间。用来选择淘汰哪一页的规则叫做页面置换算法，我们可以把页面置换算法看成是淘汰页面的规则。

- **OPT 页面置换算法（最佳页面置换算法）**：最佳(Optimal, OPT)置换算法所选择的被淘汰页面

将是以后永不使用的，或者是在最长时间内不再被访问的页面,这样可以保证获得最低的缺页率。但由于人们目前无法预知进程在内存下的若干页面中哪个是未来最长时间内不再被访问的，因而该算法无法实现。一般作为衡量其他置换算法的方法。

- **FIFO (First In First Out) 页面置换算法 (先进先出页面置换算法)**：总是淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面进行淘汰。
- **LRU (Least Recently Used) 页面置换算法 (最近最久未使用页面置换算法)**：LRU 算法赋予每个页面一个访问字段，用来记录一个页面自上次被访问以来所经历的时间 T，当须淘汰一个页面时，选择现有页面中其 T 值最大的，即最近最久未使用的页面予以淘汰。
- **LFU (Least Frequently Used) 页面置换算法 (最少使用页面置换算法)**：该置换算法选择在之前时期使用最少的页面作为淘汰页。

3.3 数据结构

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

数据结构这部分的基础知识已经总结完成。

由于篇幅问题，这里直接放 JavaGuide 在线网站网站上的文章链接，小伙伴可以根据个人需求自行学习：

- [线性数据结构 :数组、链表、栈、队列](#)
- [图](#)
- [堆](#)
- [树](#)
- [红黑树](#)
- [布隆过滤器](#)

JavaGuide

面试指南 优质专栏 开源项目 技术书籍 技术文章 网站相关

面试准备

Java

计算机基础

网络

操作系统

数据结构

线性数据结构:数组、链表、栈、队列

图

堆

树

红黑树

布隆过滤器

算法

数据库

开发工具

常用框架

系统设计

分布式

高性能

HA高可用

Java 面试指南 / 线性数据结构:数组、链表、栈、队列

线性数据结构:数组、链表、栈、队列

Guide 计算机基础 数据结构 2022年3月3日 约 3403 字

开头还是求点赞，求转发！原创优质公众号，希望大家能让更多人看到我们的文章。
图片都是我们手绘的，可以说非常用心了！

1. 数组

数组 (Array) 是一种很常见的数据结构。它由相同类型的元素 (element) 组成，并且是使用一块连续的内存来存储。

我们直接可以利用元素的索引 (index) 可以计算出该元素对应的存储地址。

数组的特点是：提供随机访问 并且容量有限。

```
1 假如数组的长度为 n。  
2 访问: O(1) //访问特定位置的元素  
3 插入: O(n) //最坏的情况发生在插入发生在数组的首部并需要移动所有元素时  
4 删除: O(n) //最坏的情况发生在删除数组的开头并需要移动第一元素后面所有的元素时
```

数组下标从0开始

1	2	3	4	5	6
---	---	---	---	---	---

index=0 作者: SnailClimb index=5
公众号&Github: JavaGuide

2. 链表

此页内容

1. 数组
2. 链表
2.1. 链表简介
2.2. 链表分类
2.3. 应用场景
2.4. 数组 vs 链表
3. 栈
3.1. 栈简介
3.2. 栈的常见应用: 括号匹配应用题
3.3. 栈的实现
4. 队列
4.1. 队列简介
4.2. 队列分类
4.3. 常见应用场景

CSDN @JavaGuide



JavaGuide

扫码关注

准备 Java 面试，首选 JavaGuide!

1、关注公众号回复“PDF”领取原创面试 PDF 手册

2、关注公众号回复“加群”进入面试交流群

3、关注公众号回复“八股文”获取大厂面试真题+面经

3.3 算法

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

算法这部分目前已经总结了部分基础的常见的算法面试题。

由于篇幅问题，这里直接放网站上的文章链接，小伙伴可以根据个人需求自行学习：

- [几道常见的字符串算法题](#)
- [几道常见的链表算法题](#)
- [剑指offer部分编程题](#)
- [十大经典排序算法总结](#)



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

4. 数据库

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！



[Github](#) |

[Gitee](#)

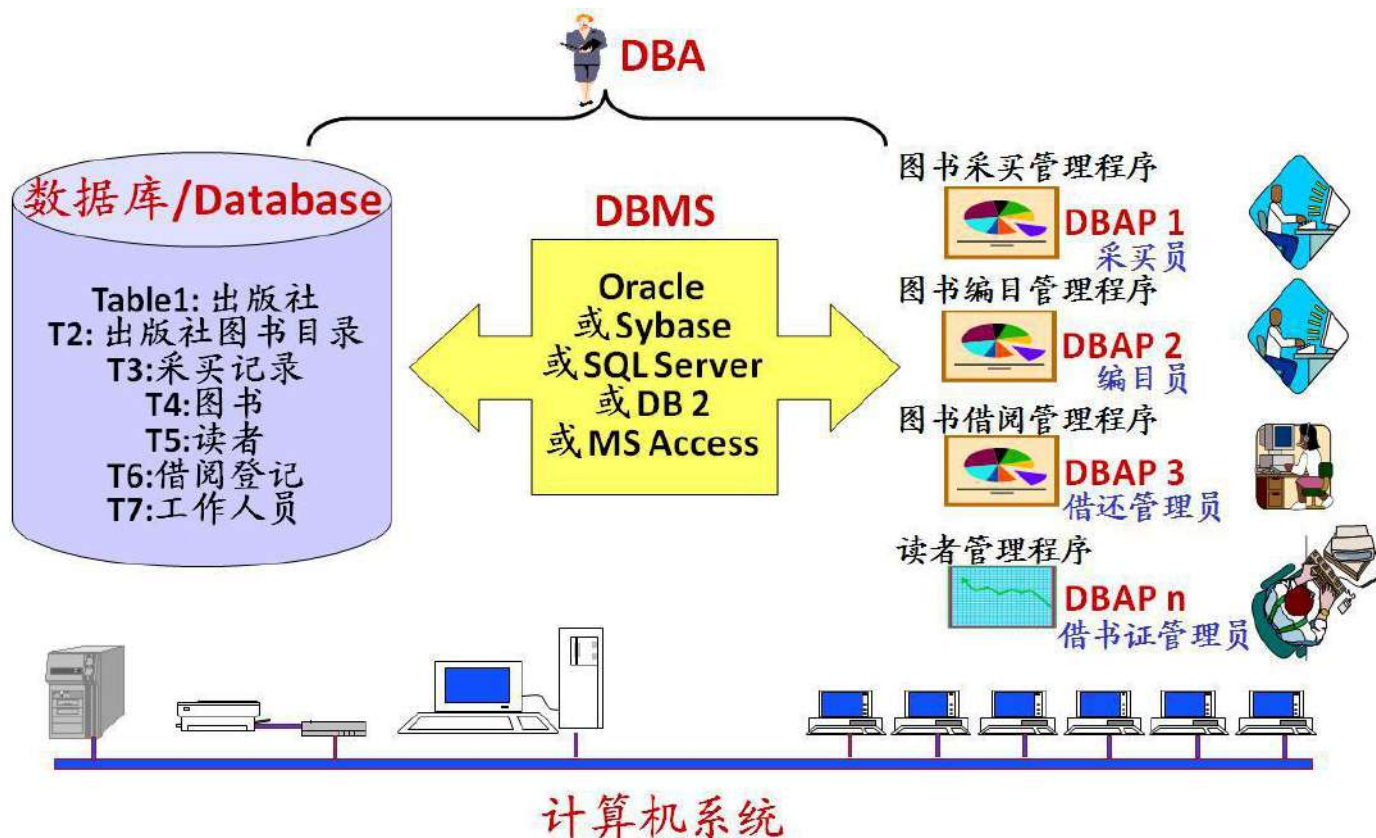
4.1 数据库基础

数据库知识基础，这部分内容一定要理解记忆。虽然这部分内容只是理论知识，但是非常重要，这是后面学习 MySQL 数据库的基础。PS: 这部分内容由于涉及太多概念性内容，所以参考了维基百科和百度百科相应的介绍。

什么是数据库, 数据库管理系统, 数据库系统, 数据库管理员?

- **数据库**：数据库(DataBase 简称 DB)就是信息的集合或者说数据库是由数据库管理系统管理的数据的集合。
- **数据库管理系统**：数据库管理系统(Database Management System 简称 DBMS)是一种操纵和管理数据库的大型软件，通常用于建立、使用和维护数据库。
- **数据库系统**：数据库系统(Data Base System, 简称 DBS)通常由软件、数据库和数据管理员(DBA)组成。
- **数据库管理员**：数据库管理员(Database Administrator, 简称 DBA)负责全面管理和控制数据库系统。

数据库系统基本构成如下图所示：



什么是元组, 码, 候选码, 主码, 外码, 主属性, 非主属性?

- **元组**：元组 (tuple) 是关系数据库中的基本概念，关系是一张表，表中的每行（即数据库中的每条记录）就是一个元组，每列就是一个属性。在二维表里，元组也称为行。
- **码**：码就是能唯一标识实体的属性，对应表中的列。
- **候选码**：若关系中的某一属性或属性组的值能唯一的标识一个元组，而其任何、子集都不能再标识，则称该属性组为候选码。例如：在学生实体中，“学号”是能唯一的区分学生实体的，同时又假设“姓名”、“班级”的属性组合足以区分学生实体，那么{学号}和{姓名，班级}都是候选码。
- **主码**：主码也叫主键。主码是从候选码中选出来的。一个实体集中只能有一个主码，但可以有多个候选码。
- **外码**：外码也叫外键。如果一个关系中的一个属性是另外一个关系中的主码则这个属性为外码。
- **主属性**：候选码中出现过的属性称为主属性。比如关系 工人（工号，身份证号，姓名，性别，部门）。显然工号和身份证号都能够唯一标示这个关系，所以都是候选码。工号、身份证号这两个属性就是主属性。如果主码是一个属性组，那么属性组中的属性都是主属性。
- **非主属性**：不包含在任何一个候选码中的属性称为非主属性。比如在关系——学生（学号，姓名，年龄，性别，班级）中，主码是“学号”，那么其他的“姓名”、“年龄”、“性别”、“班级”就都可以称为非主属性。

主键和外键有什么区别？

- **主键(主码)：**主键用于唯一标识一个元组，不能有重复，不允许为空。一个表只能有一个主键。
- **外键(外码)：**外键用来和其他表建立联系用，外键是另一表的主键，外键是可以有重复的，可以是空值。一个表可以有多个外键。

为什么不推荐使用外键与级联？

对于外键和级联，阿里巴巴开发手册这样说到：

【强制】不得使用外键与级联，一切外键概念必须在应用层解决。

说明：以学生和成绩的关系为例，学生表中的 `student_id` 是主键，那么成绩表中的 `student_id` 则为外键。如果更新学生表中的 `student_id`，同时触发成绩表中的 `student_id` 更新，即为级联更新。外键与级联更新适用于单机低并发，不适合分布式、高并发集群；级联更新是强阻塞，存在数据库更新风暴的风险；外键影响数据库的插入速度

为什么不要用外键呢？大部分人可能会这样回答：

1. **增加了复杂性：** a. 每次做DELETE 或者UPDATE都必须考虑外键约束，会导致开发的时候很痛苦，测试数据极为不方便； b. 外键的主从关系是定的，假如那天需求有变化，数据库中的这个字段根本不需要和其他表有关联的话就会增加很多麻烦。
2. **增加了额外工作：** 数据库需要增加维护外键的工作，比如当我们做一些涉及外键字段的增，删，更新操作之后，需要触发相关操作去检查，保证数据的一致性和正确性，这样会不得不消耗资源；（个人觉得这个不是不用外键的原因，因为即使你不使用外键，你在应用层面也还是要保证的。所以，我觉得这个影响可以忽略不计。）
3. **对分库分表不友好：** 因为分库分表下外键是无法生效的。
4.

我个人觉得上面这种回答不是特别的全面，只是说了外键存在的一个常见的问题。实际上，我们知道外键也是有很多好处的，比如：

1. 保证了数据库数据的一致性和完整性；
2. 级联操作方便，减轻了程序代码量；
3.

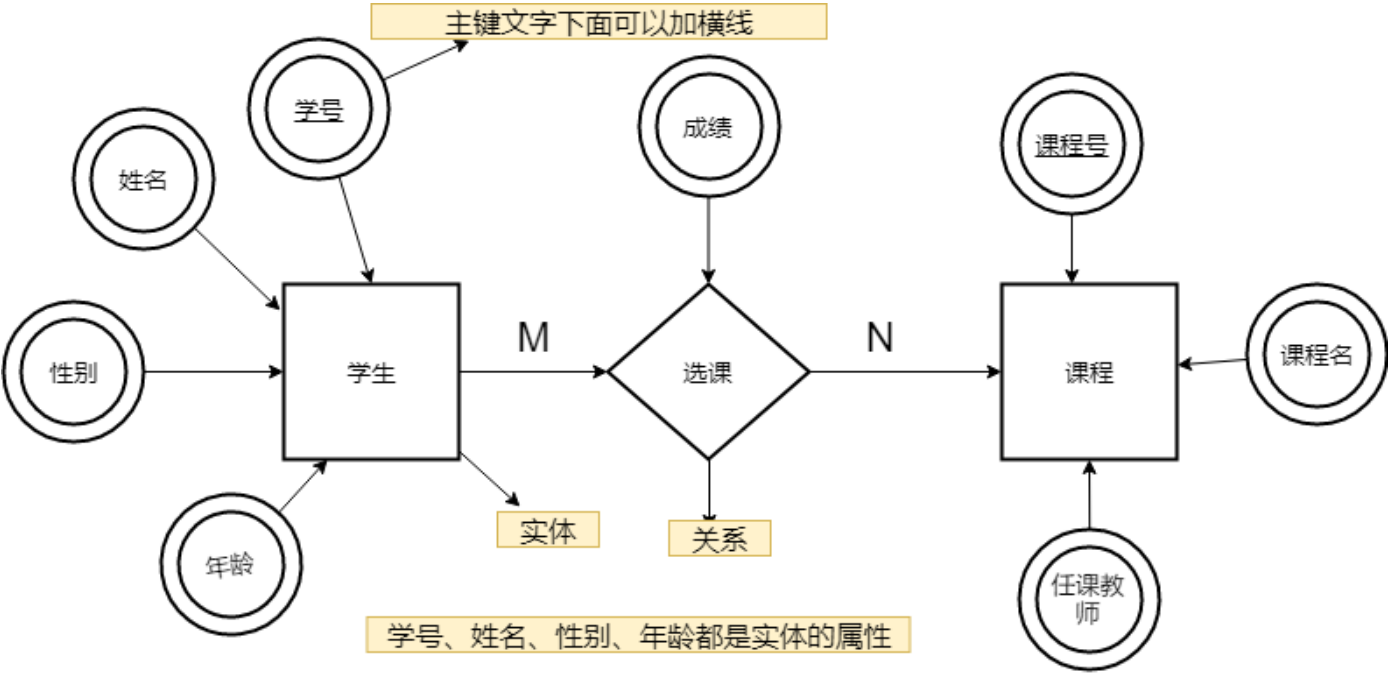
所以说，不要一股脑的就抛弃了外键这个概念，既然它存在就有它存在的道理，如果系统不涉及分库分表，并发量不是很高的情况还是可以考虑使用外键的。

什么是 ER 图？

我们做一个项目的时候一定要试着画 ER 图来捋清数据库设计，这个也是面试官问你项目的时候经常会被问到的。

E-R 图 也称实体-联系图(Entity Relationship Diagram)，提供了表示实体类型、属性和联系的方法，用来描述现实世界的概念模型。它是描述现实世界关系概念模型的有效方法。是表示概念关系模型的一种方式。

下图是一个学生选课的 ER 图，每个学生可以选若干门课程，同一门课程也可以被若干人选择，所以它们之间的关系是多对多 (M: N)。另外，还有其他两种关系是：1 对 1 (1:1)、1 对多 (1: N)。



我们试着将上面的 ER 图转换成数据库实际的关系模型(实际设计中，我们通常会将任课教师也作为一个实体来处理)：

Student	Score	Course
PK <u>student_id</u>	PK <u>score_id</u>	PK <u>course_id</u>
student_name	student_id(外键)	course_name
student_sex	course_id(外键)	course_teacher
student_age	score_num(分数)	

数据库范式了解吗？

1NF(第一范式)

属性（对应于表中的字段）不能再被分割，也就是这个字段只能是一个值，不能再分为多个其他的字段了。**1NF 是所有关系型数据库的最基本要求**，也就是说关系型数据库中创建的表一定满足第一范式。

2NF(第二范式)

2NF 在 1NF 的基础之上，消除了非主属性对于码的部分函数依赖。如下图所示，展示了第一范式到第二范式的过渡。第二范式在第一范式的基础上增加了一个列，这个列称为主键，非主属性都依赖于主键。



一些重要的概念：

- **函数依赖 (functional dependency)**：若在一张表中，在属性（或属性组）X 的值确定的情况下，必定能确定属性 Y 的值，那么就可以说 Y 函数依赖于 X，写作 $X \rightarrow Y$ 。
- **部分函数依赖 (partial functional dependency)**：如果 $X \rightarrow Y$ ，并且存在 X 的一个真子集 X_0 ，使得 $X_0 \rightarrow Y$ ，则称 Y 对 X 部分函数依赖。比如学生基本信息表 R 中（学号，身份证号，姓名）当然学号属性取值是唯一的，在 R 关系中，（学号，身份证号） $\rightarrow$ （姓名），（学号） $\rightarrow$ （姓名），（身份证号） $\rightarrow$ （姓名）；所以姓名部分函数依赖与（学号，身份证号）；
- **完全函数依赖(Full functional dependency)**：在一个关系中，若某个非主属性数据项依赖于全部关键字称之为完全函数依赖。比如学生基本信息表 R（学号，班级，姓名）假设不同的班级学号有相同的，班级内学号不能相同，在 R 关系中，（学号，班级） $\rightarrow$ （姓名），但是（学号） $\rightarrow$ （姓名）不成立，（班级） $\rightarrow$ （姓名）不成立，所以姓名完全函数依赖与（学号，班级）；
- **传递函数依赖**：在关系模式 R(U)中，设 X, Y, Z 是 U 的不同的属性子集，如果 X 确定 Y、Y 确定 Z，且有 X 不包含 Y，Y 不确定 X， $(X \cup Y) \cap Z = \text{空集合}$ ，则称 Z 传递函数依赖(transitive

functional dependency) 于 X。传递函数依赖会导致数据冗余和异常。传递函数依赖的 Y 和 Z 子集往往同属于某一个事物，因此可将其合并放到一个表中。比如在关系 R(学号, 姓名, 系名, 系主任)中, 学号 $\rightarrow$ 系名, 系名 $\rightarrow$ 系主任, 所以存在非主属性系主任对于学号的传递函数依赖。。

3NF(第三范式)

3NF 在 2NF 的基础之上, 消除了非主属性对于码的传递函数依赖。符合 3NF 要求的数据库设计, 基本上解决了数据冗余过大, 插入异常, 修改异常, 删除异常的问题。比如在关系 R(学号, 姓名, 系名, 系主任)中, 学号 $\rightarrow$ 系名, 系名 $\rightarrow$ 系主任, 所以存在非主属性系主任对于学号的传递函数依赖, 所以该表的设计, 不符合 3NF 的要求。

总结

- 1NF: 属性不可再分。
- 2NF: 1NF 的基础之上, 消除了非主属性对于码的部分函数依赖。
- 3NF: 3NF 在 2NF 的基础之上, 消除了非主属性对于码的传递函数依赖。

什么是存储过程?

我们可以把存储过程看成是一些 SQL 语句的集合, 中间加了点逻辑控制语句。存储过程在业务比较复杂的时候是非常实用的, 比如很多时候我们完成一个操作可能需要写一大串 SQL 语句, 这时候我们就可以写有一个存储过程, 这样也方便了我们下一次的调用。存储过程一旦调试完成通过后就能稳定运行, 另外, 使用存储过程比单纯 SQL 语句执行要快, 因为存储过程是预编译过的。

存储过程在互联网公司应用不多, 因为存储过程难以调试和扩展, 而且没有移植性, 还会消耗数据库资源。

阿里巴巴 Java 开发手册里要求禁止使用存储过程。

7. 【强制】禁止使用存储过程, 存储过程难以调试和扩展, 更没有移植性。

drop、delete 与 truncate 区别?

用法不同

- **drop**(丢弃数据): `drop table 表名` , 直接将表都删除掉, 在删除表的时候使用。
- **truncate** (清空数据): `truncate table 表名` , 只删除表中的数据, 再插入数据的时候自增长 id 又从 1 开始, 在清空表中数据的时候使用。
- **delete** (删除数据) : `delete from 表名 where 列名=值` , 删除某一行的数据, 如果不加 `where` 子句和 `truncate table 表名` 作用类似。

`truncate` 和不带 `where` 子句的 `delete`、以及 `drop` 都会删除表内的数据, 但是 **`truncate`** 和 **`delete`** 只删除数据不删除表的结构(定义), 执行 **`drop`** 语句, 此表的结构也会删除, 也就是执行 **`drop`** 之后对应的表不复存在。

属于不同的数据库语言

`truncate` 和 `drop` 属于 DDL(数据定义语言)语句, 操作立即生效, 原数据不放到 `rollback segment` 中, 不能回滚, 操作不触发 `trigger`。而 `delete` 语句是 DML (数据库操作语言)语句, 这个操作会放到 `rollback segment` 中, 事务提交之后才生效。

DML 语句和 DDL 语句区别:

- DML 是数据库操作语言 (Data Manipulation Language) 的缩写, 是指对数据库中表记录的操作, 主要包括表记录的插入 (`insert`)、更新 (`update`)、删除 (`delete`) 和查询 (`select`) , 是开发人员日常使用最频繁的操作。
- DDL (Data Definition Language) 是数据定义语言的缩写, 简单来说, 就是对数据库内部的对象进行创建、删除、修改的操作语言。它和 DML 语言的最大区别是 DML 只是对表内部数据的操作, 而不涉及到表的定义、结构的修改, 更不会涉及到其他对象。DDL 语句更多的被数据库管理员 (DBA) 所使用, 一般的开发人员很少使用。

由于 `select` 不会对表进行破坏, 所以有的地方也会把 `select` 单独区分开叫做数据库查询语言 DQL (Data Query Language)

执行速度不同

一般来说: `drop` > `truncate` > `delete` (这个我没有设计测试过)。

`delete` 命令执行的时候会产生数据库的 `binlog` 日志, 而日志记录是需要消耗时间的, 但是也有个好处方便数据回滚恢复。

`truncate` 命令执行的时候不会产生数据库日志，因此比 `delete` 要快。除此之外，还会把表的自增值重置和索引恢复到初始大小等。

`drop` 命令会把表占用的空间全部释放掉。

Tips：你应该更多地关注在使用场景上，而不是执行效率。

数据库设计通常分为哪几步？

1. **需求分析**：分析用户的需求，包括数据、功能和性能需求。
2. **概念结构设计**：主要采用 E-R 模型进行设计，包括画 E-R 图。
3. **逻辑结构设计**：通过将 E-R 图转换成表，实现从 E-R 模型到关系模型的转换。
4. **物理结构设计**：主要是为所设计的数据库选择合适的存储结构和存取路径。
5. **数据库实施**：包括编程、测试和试运行
6. **数据库的运行和维护**：系统的运行与数据库的日常维护。

4.2 MySQL

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

JavaGuide

面试指南 优质专栏 开源项目 技术书籍 技术文章 网站相关

搜索

面试准备

Java

基础

Java基础常见知识&面试题总结(上)

Java基础常见知识&面试题总结(中)

Java基础知识&面试题总结(下)

重要知识点

容器

并发编程

JVM

新特性

计算机基础

数据库

开发工具

常用框架

系统设计

分布式

高性能

高可用

Java 面试指南 / JavaGuide (Java学习&&面试指南)

JavaGuide (Java学习&&面试指南)

Guide 2021年12月8日 约 416 字

1. 面试专版：准备面试的小伙伴可以考虑面试专版：《Java 面试指北》(质量很高，专为面试打造，配合 JavaGuide 食用)

2. 转载须知：以下所有文章如非文首说明为转载皆为我（Guide 哥）的原创，转载在文首注明出处，如发现恶意抄袭/搬运，会动用法律武器维护自己的权益。让我们一起维护一个良好的技术创作环境！

必看专栏

《Java 面试指北》：与 JavaGuide 开源版的内容互补！

《手写 RPC 框架》：从零开始基于 Netty+Kyro+Zookeeper 实现一个简易的 RPC 框架。

项目相关

项目介绍

贡献指南

常见问题

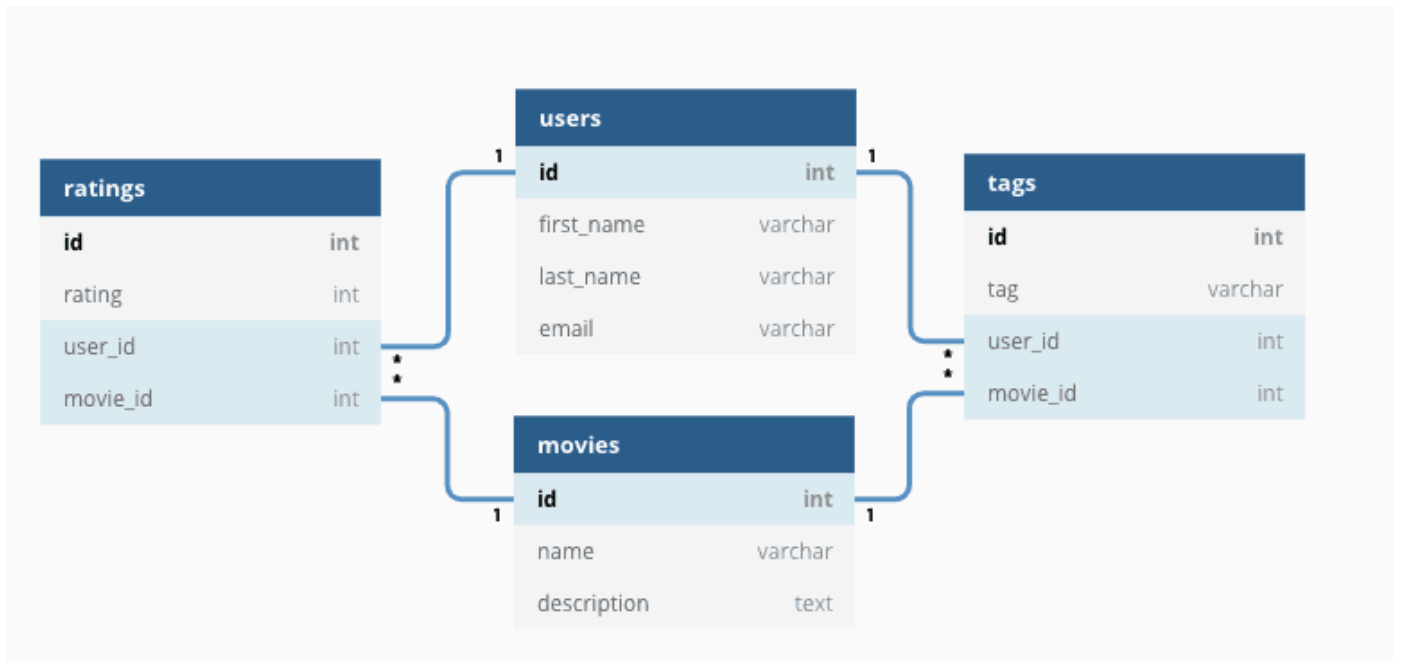
项目代办

MySQL 基础

关系型数据库介绍

顾名思义，关系型数据库就是一种建立在关系模型的基础上的数据库。关系模型表明了数据库中所存储的数据之间的联系（一对一、一对多、多对多）。

关系型数据库中，我们的数据都被存放在了各种表中（比如用户表），表中的每一行就存放着一条数据（比如一个用户的信息）。



大部分关系型数据库都使用 SQL 来操作数据库中的数据。并且，大部分关系型数据库都支持事务的四大特性(ACID)。

有哪些常见的关系型数据库呢？

MySQL、PostgreSQL、Oracle、SQL Server、SQLite（微信本地的聊天记录的存储就是用的 SQLite）……。

MySQL 介绍



MySQL 是一种关系型数据库，主要用于持久化存储我们的系统中的一些数据比如用户信息。

由于 MySQL 是开源免费并且比较成熟的数据库，因此，MySQL 被大量使用在各种系统中。任何人都可以在 GPL(General Public License) 的许可下下载并根据个性化的需要对其进行修改。MySQL 的默认端口号是**3306**。



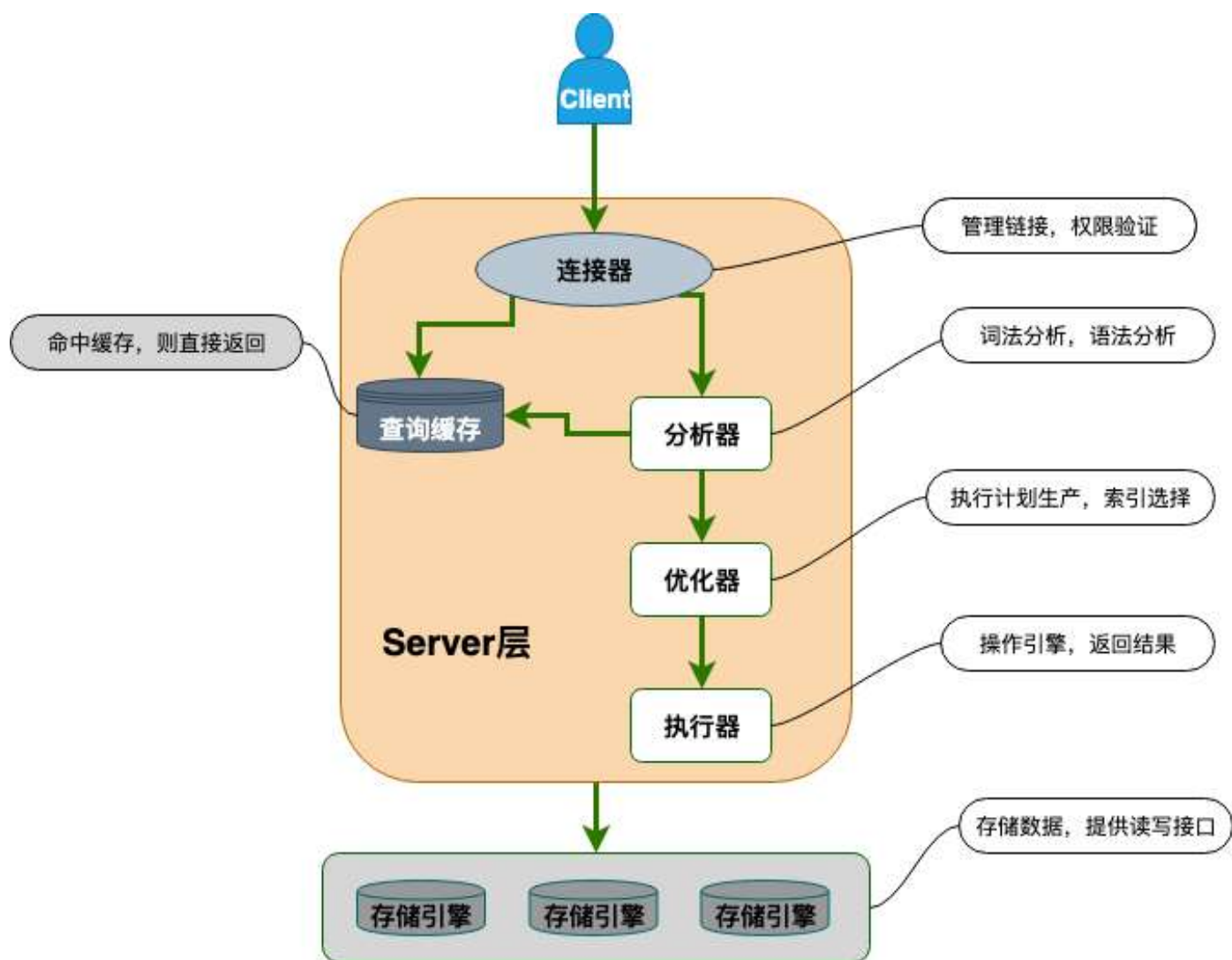
扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

MySQL 基础架构

下图是 MySQL 的一个简要架构图，从下图你可以很清晰的看到客户端的一条 SQL 语句在 MySQL 内部是如何执行的。



从上图可以看出，MySQL 主要由下面几部分构成：

- **连接器**：身份认证和权限相关(登录 MySQL 的时候)。
- **查询缓存**：执行查询语句的时候，会先查询缓存（MySQL 8.0 版本后移除，因为这个功能不太实用）。
- **分析器**：没有命中缓存的话，SQL 语句就会经过分析器，分析器说白了就是要先看你的 SQL 语句要干嘛，再检查你的 SQL 语句语法是否正确。
- **优化器**：按照 MySQL 认为最优的方案去执行。
- **执行器**：执行语句，然后从存储引擎返回数据。执行语句之前会先判断是否有权限，如果没有权限的话，就会报错。
- **插件式存储引擎**：主要负责数据的存储和读取，采用的是插件式架构，支持 InnoDB、MyISAM、Memory 等多种存储引擎。

MySQL 存储引擎

MySQL 核心在于存储引擎，想要深入学习 MySQL，必定要深入研究 MySQL 存储引擎。

MySQL 支持哪些存储引擎？默认使用哪个？

MySQL 支持多种存储引擎，你可以通过 `show engines` 命令来查看 MySQL 支持的所有存储引擎。

```
mysql> show engines;
```

Engine	Support	Comment	Transactions	XA	Savepoints
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO

9 rows in set (0.00 sec)

从上图我们可以查看出，MySQL 当前默认的存储引擎是 InnoDB。并且，所有的存储引擎中只有 InnoDB 是事务性存储引擎，也就是说只有 InnoDB 支持事务。

我这里使用的 MySQL 版本是 8.x，不同的 MySQL 版本之间可能会有差别。

MySQL 5.5.5 之前，MyISAM 是 MySQL 的默认存储引擎。5.5.5 版本之后，InnoDB 是 MySQL 的默认存储引擎。

你可以通过 `select version()` 命令查看你的 MySQL 版本。

```
mysql> select version();

+-----+
| version() |
+-----+
| 8.0.27    |
+-----+

1 row in set (0.00 sec)
```

你也可以通过 `show variables like '%storage_engine%'` 命令直接查看 MySQL 当前默认的存储引擎。

```
mysql> show variables like '%storage_engine%';
```

Variable_name	Value
default_storage_engine	InnoDB
default_tmp_storage_engine	InnoDB
disabled_storage_engines	
internal_tmp_mem_storage_engine	TempTable

```
4 rows in set (0.00 sec)
```

如果你只想查看数据库中某个表使用的存储引擎的话，可以使用 `show table status from db_name where name='table_name'` 命令。

```
mysql> show table status from eladmin where name='ant_server';
```

Name	Engine	Version	Row_format	Rows	Avg_row_length	Data_length	Max_data_length	Index_length	Data_free	Auto_increment	Create_time	Update_time	Check_time	Collation	Checksum	Create_options	Comment
ant_server	InnoDB	10	Compact	0	0	10384	0	10384	0	NULL	2022-01-29 03:05:13	NULL	NULL	utf8_general_ci	NULL	row_format=COMPACT	服务器管理

```
1 row in set (0.01 sec)
```

如果你想要深入了解每个存储引擎以及它们之间的区别，推荐你去阅读以下 MySQL 官方文档对应的介绍(面试不会问这么细，了解即可)：

- InnoDB 存储引擎详细介绍：<https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>。
- 其他存储引擎详细介绍：<https://dev.mysql.com/doc/refman/8.0/en/storage-engines.html>。

MySQL 8.0 Reference Manual / Alternative Storage Engines

version 8.0

Chapter 16 Alternative Storage Engines

Table of Contents

- 16.1 Setting the Storage Engine
- 16.2 The MyISAM Storage Engine
- 16.3 The MEMORY Storage Engine
- 16.4 The CSV Storage Engine
- 16.5 The ARCHIVE Storage Engine
- 16.6 The BLACKHOLE Storage Engine
- 16.7 The MERGE Storage Engine
- 16.8 The FEDERATED Storage Engine
- 16.9 The EXAMPLE Storage Engine
- 16.10 Other Storage Engines
- 16.11 Overview of MySQL Storage Engine Architecture

Storage engines are MySQL components that handle the SQL operations for different table types. `InnoDB` is the default and most general-purpose storage engine, and Oracle recommends using it for tables except for specialized use cases. (The `CREATE TABLE` statement in MySQL 8.0 creates `InnoDB` tables by default.)

MySQL Server uses a pluggable storage engine architecture that enables storage engines to be loaded into and unloaded from a running MySQL server.

To determine which storage engines your server supports, use the `SHOW ENGINES` statement. The value in the `support` column indicates whether an engine can be used. A value of `YES`, `NO`, or `DEFAULT` indicates that an engine is available, not available, or available and currently set as the default storage engine.

```
mysql> SHOW ENGINES
+-----+-----+-----+-----+-----+-----+
| Engine | Support | Comment | Transactions | XA | Savepoints |
+-----+-----+-----+-----+-----+-----+
| 1. row |
+-----+-----+-----+-----+-----+-----+
| Engine: PERFORMANCE_SCHEMA |
| Support: YES |
| Comment: Performance Schema |
| Transactions: NO |
| XA: NO |
| Savepoints: NO |
+-----+-----+-----+-----+-----+-----+
| 2. row |
+-----+-----+-----+-----+-----+-----+
| Engine: InnoDB |
| Support: DEFAULT |
| Comment: Supports transactions, row-level locking, and foreign keys |
| Transactions: YES |
| XA: YES |
| Savepoints: YES |
+-----+-----+-----+-----+-----+-----+
| 3. row |
+-----+-----+-----+-----+-----+-----+
| Engine: MRG_MYISAM |
| Support: YES |
| Comment: Collection of identical MyISAM tables |
| Transactions: NO |
| XA: NO |
| Savepoints: NO |
+-----+-----+-----+-----+-----+-----+
| 4. row |
+-----+-----+-----+-----+-----+-----+
| Engine: BLACKHOLE |
| Support: YES |
+-----+-----+-----+-----+-----+-----+
```

MySQL 存储引擎架构了解吗？

MySQL 存储引擎采用的是插件式架构，支持多种存储引擎，我们甚至可以为不同的数据库表设置不同的存储引擎以适应不同场景的需要。**存储引擎是基于表的，而不是数据库。**

并且，你还可以根据 MySQL 定义的存储引擎实现标准接口来编写一个属于自己的存储引擎。这些非官方提供的存储引擎可以称为第三方存储引擎，区别于官方存储引擎。像目前最常用的 InnoDB 其实刚开始就是一个第三方存储引擎，后面由于过于优秀，其被 Oracle 直接收购了。

MySQL 官方文档也有介绍到如何编写一个自定义存储引擎，地址：<https://dev.mysql.com/doc/internals/en/custom-engine.html>。

MyISAM 和 InnoDB 的区别是什么？



MySQL 5.5 之前，MyISAM 引擎是 MySQL 的默认存储引擎，可谓是风光一时。

虽然，MyISAM 的性能还行，各种特性也还不错（比如全文索引、压缩、空间函数等）。但是，MyISAM 不支持事务和行级锁，而且最大的缺陷就是崩溃后无法安全恢复。

MySQL 5.5.5 之前，MyISAM 是 MySQL 的默认存储引擎。5.5.5 版本之后，InnoDB 是 MySQL 的默认存储引擎。

言归正传！咱们下面还是来简单对比一下两者：

1.是否支持行级锁

MyISAM 只有表级锁(table-level locking)，而 InnoDB 支持行级锁(row-level locking)和表级锁,默认为行级锁。

也就说，MyISAM 一锁就是锁住了整张表，这在并发写的情况下是多么滴憋憋啊！这也是为什么 InnoDB 在并发写的时候，性能更牛皮了！

2.是否支持事务

MyISAM 不提供事务支持。

InnoDB 提供事务支持，实现了 SQL 标准定义了四个隔离级别，具有提交(commit)和回滚(rollback)事务的能力。并且，InnoDB 默认使用的 REPEATABLE-READ（可重读）隔离级别是可以解决幻读问题发生的（基于 MVCC 和 Next-Key Lock）。

关于 MySQL 事务的详细介绍，可以看看我写的这篇文章：[MySQL 事务隔离级别详解](#)。

3.是否支持外键

MyISAM 不支持，而 InnoDB 支持。

外键对于维护数据一致性非常有帮助，但是对性能有一定的损耗。因此，通常情况下，我们是不建议在实际生产项目中使用外键的，在业务代码中进行约束即可！

阿里的《Java 开发手册》也是明确规定禁止使用外键的。



6. **【强制】** 不得使用外键与级联，一切外键概念必须在应用层解决。
说明：（概念解释）学生表中的 student_id 是主键，那么成绩表中的 student_id 则为外键。如果更新学生表中的 student_id，同时触发成绩表中的 student_id 更新，即为级联更新。外键与级联更新适用于单机低并发，不适合分布式、高并发集群；级联更新是强阻塞，存在数据库更新风暴的风险；外键影响数据库的插入速度。
7. **【强制】** 禁止使用存储过程，存储过程难以调试和扩展，更没有移植性。
8. **【强制】** 数据订正（特别是删除或修改记录操作）时，要先 select，避免出现误删除的情况，确认无误才能执行更新语句。
9. **【强制】** 对于数据库中表记录的查询和变更，只要涉及多个表，都需要在列名前加表的别名（或表名）进行限定。

不过，在代码中进行约束的话，对程序员的能力要求更高，具体是否要采用外键还是要根据你的项目实际情况而定。

总结：一般我们也是不建议在数据库层面使用外键的，应用层面可以解决。不过，这样会对数据的一致性造成威胁。具体要不要使用外键还是要根据你的项目来决定。

4.是否支持数据库异常崩溃后的安全恢复

MyISAM 不支持，而 InnoDB 支持。

使用 InnoDB 的数据库在异常崩溃后，数据库重新启动的时候会保证数据库恢复到崩溃前的状态。这个恢复的过程依赖于 redo log 。

5.是否支持 MVCC

MyISAM 不支持，而 InnoDB 支持。

讲真，这个对比有点废话，毕竟 MyISAM 连行级锁都不支持。MVCC 可以看作是行级锁的一个升级，可以有效减少加锁操作，提高性能。

6.索引实现不一样。

虽然 MyISAM 引擎和 InnoDB 引擎都是使用 B+Tree 作为索引结构，但是两者的实现方式不太一样。

InnoDB 引擎中，其数据文件本身就是索引文件。相比 MyISAM，索引文件和数据文件是分离的，其表数据文件本身就是按 B+Tree 组织的一个索引结构，树的叶节点 data 域保存了完整的数据记录。

详细区别，推荐你看看我写的这篇文章：[MySQL 索引详解](#)。

MyISAM 和 InnoDB 如何选择？

大多数时候我们使用的都是 InnoDB 存储引擎，在某些读密集的情况下，使用 MyISAM 也是合适的。不过，前提是你的项目不介意 MyISAM 不支持事务、崩溃恢复等缺点（可是~我们一般都会介意啊！）。

《MySQL 高性能》上面有一句话这样写到：

不要轻易相信“MyISAM 比 InnoDB 快”之类的经验之谈，这个结论往往不是绝对的。在很多我们已知场景中，InnoDB 的速度都可以让 MyISAM 望尘莫及，尤其是用到了聚簇索引，或者需要访问的数据都可以放入内存的应用。

一般情况下我们选择 InnoDB 都是没有问题的，但是某些情况下你并不在乎可扩展能力和并发能力，也不需要事务支持，也不在乎崩溃后的安全恢复问题的话，选择 MyISAM 也是一个不错的选择。但是一般情况下，我们都是需要考虑到这些问题的。

因此，对于咱们日常开发的业务系统来说，你几乎找不到什么理由再使用 MyISAM 作为自己的 MySQL 数据库的存储引擎。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“PDF”领取原创面试 PDF 手册
- 2、关注公众号回复“加群”进入面试交流群
- 3、关注公众号回复“八股文”获取大厂面试真题+面经

MySQL 查询缓存

执行查询语句的时候，会先查询缓存。不过，MySQL 8.0 版本后移除，因为这个功能不太实用

my.cnf 加入以下配置，重启 MySQL 开启查询缓存

```
query_cache_type=1
query_cache_size=600000
```

MySQL 执行以下命令也可以开启查询缓存

```
set global query_cache_type=1;
set global query_cache_size=600000;
```

如上，开启查询缓存后在同样的查询条件以及数据情况下，会直接在缓存中返回结果。这里的查询条件包括查询本身、当前要查询的数据库、客户端协议版本号等一些可能影响结果的信息。（查询缓存不命中的情况：（1））因此任何两个查询在任何字符上的不同都会导致缓存不命中。此外，（查询缓存不命中的情况：（2））如果查询中包含任何用户自定义函数、存储函数、用户变量、临时表、MySQL 库中的系统表，其查询结果也不会被缓存。

（查询缓存不命中的情况：（3））缓存建立之后，MySQL 的查询缓存系统会跟踪查询中涉及的每张表，如果这些表（数据或结构）发生变化，那么和这张表相关的所有缓存数据都将失效。

缓存虽然能够提升数据库的查询性能，但是缓存同时也带来了额外的开销，每次查询后都要做一次缓存操作，失效后还要销毁。因此，开启查询缓存要谨慎，尤其对于写密集的应用来说更是如此。如果开启，要注意合理控制缓存空间大小，一般来说其大小设置为几十 MB 比较合适。此外，还可以通过 `sql_cache` 和 `sql_no_cache` 来控制某个查询语句是否需要缓存：

```
select sql_no_cache count(*) from usr;
```

MySQL 事务

何谓事务？

我们设想一个场景，这个场景中我们需要插入多条相关联的数据到数据库，不幸的是，这个过程可能会遇到下面这些问题：

- 数据库中途突然因为某些原因挂掉了。
- 客户端突然因为网络原因连接不上数据库了。
- 并发访问数据库时，多个线程同时写入数据库，覆盖了彼此的更改。
-

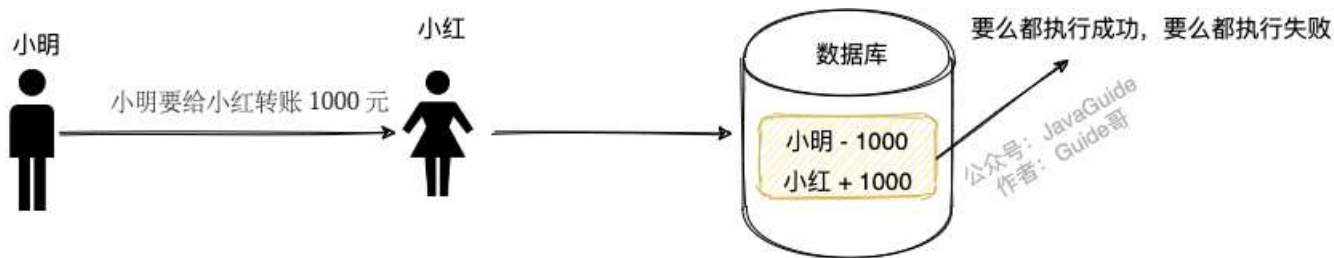
上面的任何一个问题都可能会导致数据的不一致性。为了保证数据的一致性，系统必须能够处理这些问题。事务就是我们抽象出来简化这些问题的首选机制。事务的概念起源于数据库，目前，已经成为一个比较广泛的概念。

何为事务？ 一言蔽之，事务是逻辑上的一组操作，要么都执行，要么都不执行。

事务最经典也经常被拿出来例子就是转账了。假如小明要给小红转账 1000 元，这个转账会涉及到两个关键操作，这两个操作必须都成功或者都失败。

1. 将小明的余额减少 1000 元
2. 将小红的余额增加 1000 元。

事务会把这两个操作就可以看成逻辑上的一个整体，这个整体包含的操作要么都成功，要么都要失败。这样就不会出现小明余额减少而小红的余额却并没有增加的情况。



何谓数据库事务？

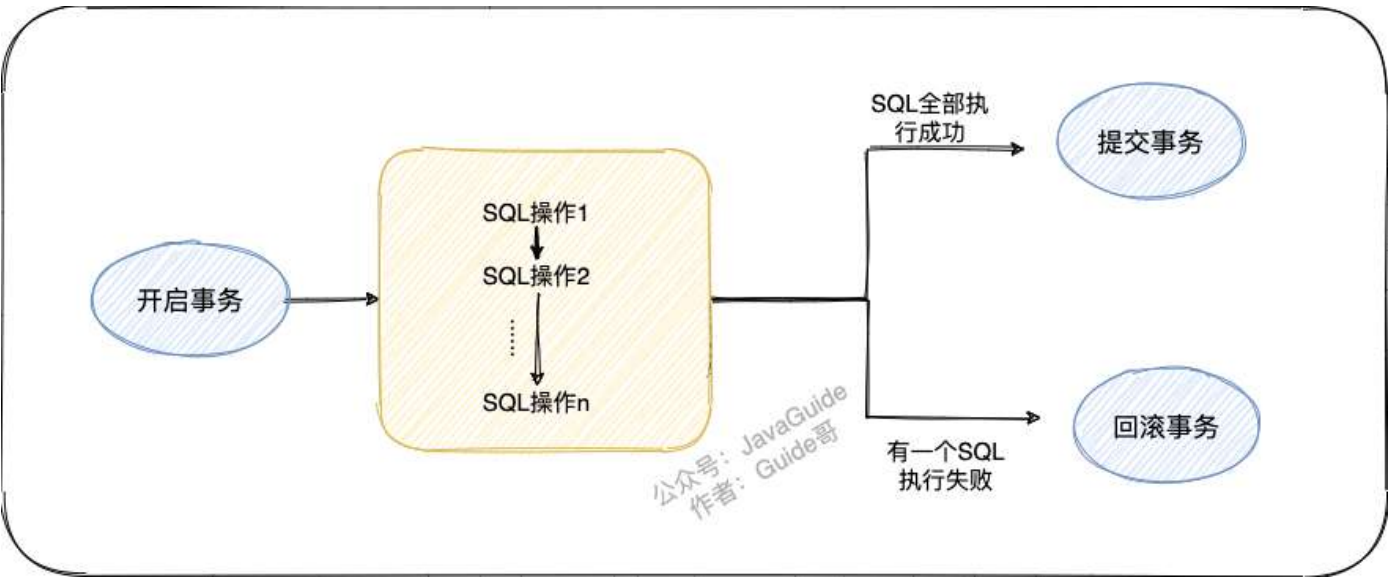
大多数情况下，我们在谈论事务的时候，如果没有特指**分布式事务**，往往指的就是**数据库事务**。

数据库事务在我们日常开发中接触的最多了。如果你的项目属于单体架构的话，你接触到的往往就是数据库事务了。

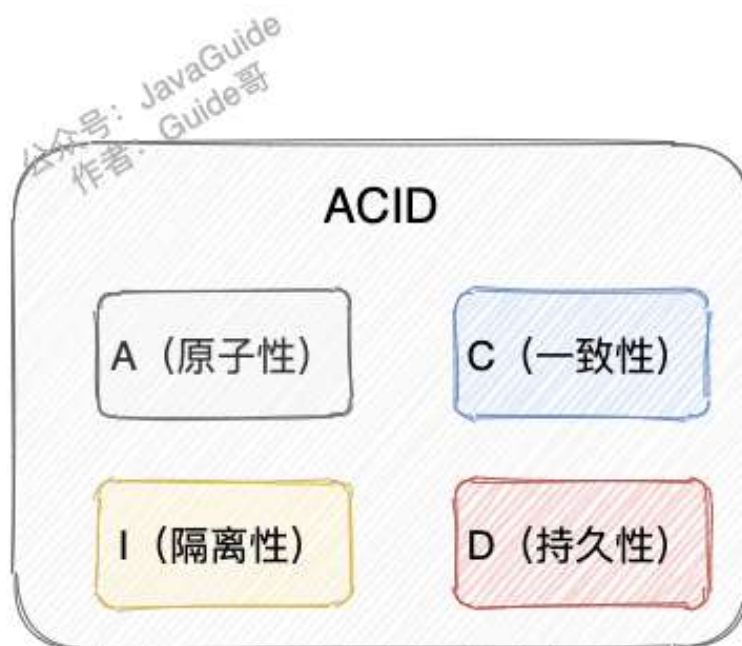
那数据库事务有什么作用呢？

简单来说，数据库事务可以保证多个对数据库的操作（也就是 SQL 语句）构成一个逻辑上的整体。构成这个逻辑上的整体的这些数据库操作遵循：**要么全部执行成功,要么全部不执行**。

```
# 开启一个事务
START TRANSACTION;
# 多条 SQL 语句
SQL1,SQL2...
### 提交事务
COMMIT;
```

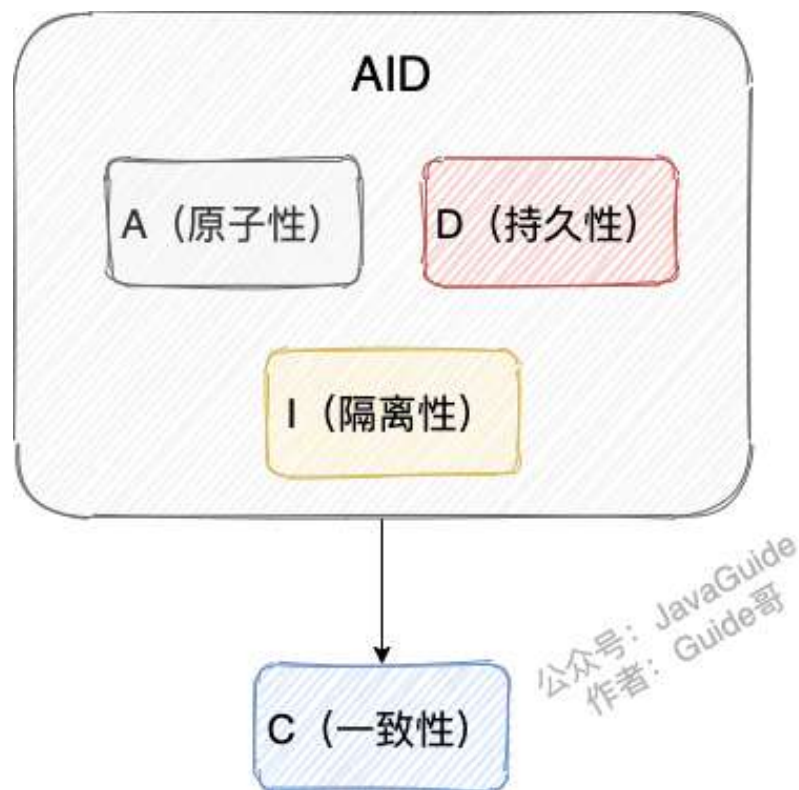


另外，关系型数据库（例如：`MySQL`、`SQL Server`、`Oracle` 等）事务都有 **ACID** 特性：



1. **原子性** (Atomicity) : 事务是最小的执行单位, 不允许分割。事务的原子性确保动作要么全部完成, 要么完全不起作用;
2. **一致性** (Consistency) : 执行事务前后, 数据保持一致, 例如转账业务中, 无论事务是否成功, 转账者和收款人的总额应该是不变的;
3. **隔离性** (Isolation) : 并发访问数据库时, 一个用户的事务不被其他事务所干扰, 各并发事务之间数据库是独立的;
4. **持久性** (Durability) : 一个事务被提交之后。它对数据库中数据的改变是持久的, 即使数据库发生故障也不应该对其有任何影响。

🌈 这里要额外补充一点: 只有保证了事务的持久性、原子性、隔离性之后, 一致性才能得到保障。也就是说 **A、I、D** 是手段, **C** 是目的! 想必大家也和我一样, 被 ACID 这个概念被误导了很久! 我也是看周志明老师的公开课《[周志明的软件架构课](#)》才搞清楚的 (多看好书!!!)。



另外，DDIA 也就是《[Designing Data-Intensive Application（数据密集型应用系统设计）](#)》的作者在他的这本书中如是说：

Atomicity, isolation, and durability are properties of the database, whereas consistency (in the ACID sense) is a property of the application. The application may rely on the database's atomicity and isolation properties in order to achieve consistency, but it's not up to the database alone.

翻译过来的意思是：原子性，隔离性和持久性是数据库的属性，而一致性（在 ACID 意义上）是应用程序的属性。应用可能依赖数据库的原子性和隔离属性来实现一致性，但这并不仅取决于数据库。因此，字母 C 不属于 ACID。

《Designing Data-Intensive Application（数据密集型应用系统设计）》这本书强推一波，值得读很多遍！豆瓣有接近 90% 的人看了这本书之后给了五星好评。另外，中文翻译版本已经在 Github 开源，地址：<https://github.com/Vonng/ddia>。

数据密集型应用系统设计



更新图书信息或封面

作者: Martin Kleppmann
出版社: 中国电力出版社
原名: Designing Data-Intensive Applications
译者: 赵军平 / 李三平 / 吕云松 / 耿煜
出版年: 2018-9-1
页数: 519
定价: 128
丛书: O'Reilly系列
ISBN: 9787519821968

豆瓣评分

9.7  750人评价

5星  88.4%
4星  9.9%
3星  1.6%
2星  0.1%
1星  0.0%

并发事务带来了哪些问题？

在典型的应用程序中，多个事务并发运行，经常会操作相同的数据来完成各自的任务（多个用户对同一数据进行操作）。并发虽然是必须的，但可能会导致以下的问题。

- **脏读（Dirty read）**：当一个事务正在访问数据并且对数据进行了修改，而这种修改还没有提交到数据库中，这时另外一个事务也访问了这个数据，然后使用了这个数据。因为这个数据是还没有提交的数据，那么另外一个事务读到的这个数据是“脏数据”，依据“脏数据”所做的操作可能是不正确的。
- **丢失修改（Lost to modify）**：指在一个事务读取一个数据时，另外一个事务也访问了该数据，那么在第一个事务中修改了这个数据后，第二个事务也修改了这个数据。这样第一个事务内的修改结果就被丢失，因此称为丢失修改。例如：事务 1 读取某表中的数据 $A=20$ ，事务 2 也读取 $A=20$ ，事务 1 修改 $A=A-1$ ，事务 2 也修改 $A=A-1$ ，最终结果 $A=19$ ，事务 1 的修改被丢失。
- **不可重复读（Unrepeatable read）**：指在一个事务内多次读同一数据。在这个事务还没有结束时，另一个事务也访问该数据。那么，在第一个事务中的两次读数据之间，由于第二个事务的修改导致第一个事务两次读取的数据可能不太一样。这就发生了在一个事务内两次读到的数据是不一样的情况，因此称为不可重复读。
- **幻读（Phantom read）**：幻读与不可重复读类似。它发生在一个事务（T1）读取了几行数据，接着另一个并发事务（T2）插入了一些数据时。在随后的查询中，第一个事务（T1）就会发现多了一些原本不存在的记录，就好像发生了幻觉一样，所以称为幻读。

不可重复读和幻读有什么区别呢？

- 不可重复读的重点是内容修改或者记录减少比如多次读取一条记录发现其中某些记录的值被修改；
- 幻读的重点在于记录新增比如多次执行同一条查询语句（DQL）时，发现查到的记录增加了。

幻读其实可以看作是不可重复读的一种特殊情况，单独把区分幻读的原因主要是解决幻读和不可重复读的方案不一样。

举个例子：执行 `delete` 和 `update` 操作的时候，可以直接对记录加锁，保证事务安全。而执行 `insert` 操作的时候，由于记录锁（Record Lock）只能锁住已经存在的记录，为了避免插入新记录，需要依赖间隙锁（Gap Lock）。也就是说执行 `insert` 操作的时候需要依赖 Next-Key Lock（Record Lock+Gap Lock）进行加锁来保证不出现幻读。

SQL 标准定义了哪些事务隔离级别？

SQL 标准定义了四个隔离级别：

- **READ-UNCOMMITTED(读取未提交)**：最低的隔离级别，允许读取尚未提交的数据变更，可能会导致脏读、幻读或不可重复读。
- **READ-COMMITTED(读取已提交)**：允许读取并发事务已经提交的数据，可以阻止脏读，但是幻读或不可重复读仍有可能发生。
- **REPEATABLE-READ(可重复读)**：对同一字段的多次读取结果都是一致的，除非数据是被本身事务自己所修改，可以阻止脏读和不可重复读，但幻读仍有可能发生。
- **SERIALIZABLE(可串行化)**：最高的隔离级别，完全服从 ACID 的隔离级别。所有的事务依次逐个执行，这样事务之间就完全不可能产生干扰，也就是说，该级别可以防止脏读、不可重复读以及幻读。

隔离级别	脏读	不可重复读	幻读
READ-UNCOMMITTED	√	√	√
READ-COMMITTED	×	√	√
REPEATABLE-READ	×	×	√
SERIALIZABLE	×	×	×

MySQL 的隔离级别是基于锁实现的吗？

MySQL 的隔离级别基于锁和 MVCC 机制共同实现的。

SERIALIZABLE 隔离级别，是通过锁来实现的。除了 SERIALIZABLE 隔离级别，其他的隔离级别都是基于 MVCC 实现。

不过，SERIALIZABLE 之外的其他隔离级别可能也需要用到锁机制，就比如 REPEATABLE-READ 在当前读情况下需要使用加锁读来保证不会出现幻读。

MySQL 的默认隔离级别是什么？

MySQL InnoDB 存储引擎的默认支持的隔离级别是 **REPEATABLE-READ**（可重读）。我们可以通过 `SELECT @@tx_isolation;` 命令来查看，MySQL 8.0 该命令改为 `SELECT @@transaction_isolation;`

```
mysql> SELECT @@tx_isolation;

+-----+
| @@tx_isolation |
+-----+
| REPEATABLE-READ |
+-----+
```

关于 MySQL 事务隔离级别的详细介绍，可以看看我写的这篇文章：[MySQL 事务隔离级别详解](#)。



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

MySQL 锁

表级锁和行级锁了解吗？有什么区别？

MyISAM 仅仅支持表级锁(table-level locking)，一锁就锁整张表，这在并发写的情况下性能非常差。

InnoDB 不光支持表级锁(table-level locking)，还支持行级锁(row-level locking)，默认为行级锁。行级锁的粒度更小，仅对相关的记录上锁即可（对一行或者多行记录加锁），所以对于并发写入操作来说，InnoDB 的性能更高。

表级锁和行级锁对比：

- **表级锁：**MySQL 中锁定粒度最大的一种锁，是针对非索引字段加的锁，对当前操作的整张表加锁，实现简单，资源消耗也比较少，加锁快，不会出现死锁。其锁定粒度最大，触发锁冲突的概率最高，并发度最低，MyISAM 和 InnoDB 引擎都支持表级锁。
- **行级锁：**MySQL 中锁定粒度最小的一种锁，是针对索引字段加的锁，只针对当前操作的行记录进行加锁。行级锁能大大减少数据库操作的冲突。其加锁粒度最小，并发度高，但加锁的开销也最大，加锁慢，会出现死锁。

行级锁的使用有什么注意事项？

InnoDB 的行锁是针对索引字段加的锁，表级锁是针对非索引字段加的锁。当我们执行 `UPDATE`、`DELETE` 语句时，如果 `WHERE` 条件中字段没有命中唯一索引或者索引失效的话，就会导致扫描全表对表中的所有行记录进行加锁。这个在我们日常工作开发中经常会遇到，一定要多多注意！！

不过，很多时候即使用了索引也有可能会走全表扫描，这是因为 MySQL 优化器的原因。

共享锁和排他锁呢？

不论是表级锁还是行级锁，都存在共享锁（Share Lock，S 锁）和排他锁（Exclusive Lock，X 锁）这两类：

- **共享锁（S 锁）：**又称读锁，事务在读取记录的时候获取共享锁，允许多个事务同时获取（锁兼容）。
- **排他锁（X 锁）：**又称写锁/独占锁，事务在修改记录的时候获取排他锁，不允许多个事务同时获取。如果一个记录已经被加了排他锁，那其他事务不能再对这条事务加任何类型的锁（锁不兼容）。

排他锁与任何的锁都不兼容，共享锁仅和共享锁兼容。

	S 锁	X 锁
S 锁	不冲突	冲突
X 锁	冲突	冲突

由于 MVCC 的存在，对于一般的 `SELECT` 语句，InnoDB 不会加任何锁。不过，你可以通过以下语句显式加共享锁或排他锁。

```
# 共享锁
SELECT ... LOCK IN SHARE MODE;
# 排他锁
SELECT ... FOR UPDATE;
```

意向锁有什么作用？

如果需要用到表锁的话，如何判断表中的记录没有行锁呢？一行一行遍历肯定是不行，性能太差。我们需要用到一个叫做意向锁的东东来快速判断是否可以对某个表使用表锁。

意向锁是表级锁，共有两种：

- **意向共享锁（Intention Shared Lock, IS 锁）**：事务有意向对表中的某些加共享锁（S 锁），加共享锁前必须先取得该表的 IS 锁。
- **意向排他锁（Intention Exclusive Lock, IX 锁）**：事务有意向对表中的某些记录加排他锁（X 锁），加排他锁之前必须先取得该表的 IX 锁。

意向锁是有数据引擎自己维护的，用户无法手动操作意向锁，在为数据行加共享 / 排他锁之前，InnoDB 会先获取该数据行所在在数据表的对应意向锁。

意向锁之间是互相兼容的。

	IS 锁	IX 锁
IS 锁	兼容	兼容
IX 锁	兼容	兼容

意向锁和共享锁和排它锁互斥（这里指的是表级别的共享锁和排他锁，意向锁不会与行级的共享锁和排他锁互斥）。

	IS 锁	IX 锁
S 锁	兼容	互斥
X 锁	互斥	互斥

《MySQL 技术内幕 InnoDB 存储引擎》这本书对应的描述应该是笔误了。

MySQL 技术内幕 InnoDB 存储引擎 第2版.pdf

封面

书籍

图书序

前言

目录

> 第1章 MySQL 体系结构和存储引擎

> 第2章 InnoDB 存储引擎

> 第3章 文件

> 第4章 表

> 第5章 索引与算法

> 第6章 锁

> 6.1 什么锁

> 6.2 lock 与 latch

> 6.3 InnoDB 存储引擎中的锁

> 6.3.1 锁的类型

> 6.3.2 一致性锁定

> 6.3.3 一致性锁定义

> 6.3.4 自增长与锁

> 6.3.5 外键和锁

> 6.4 锁的算法

> 6.5 锁问题

> 6.6 阻塞

> 6.7 死锁

> 6.8 锁升级

> 6.9 小结

> 第7章 事务

> 第8章 备份与恢复

> 第9章 性能调优

> 第10章 InnoDB 存储引擎源代码的编译和测试

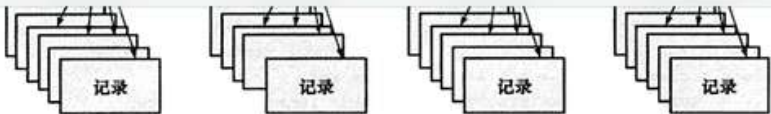


图 6-3 层次结构

其中任何一个部分导致等待，那么该操作需要等待粗粒度锁的完成。举例来说，在对记录 r 加 X 锁之前，已经有事务对表 l 进行了 S 表锁，那么表 l 上已存在 S 锁，之后事务需要对记录 r 在表 l 上加上 IX，由于不兼容，所以该事务需要等待表锁操作的完成。

InnoDB 存储引擎支持意向锁设计比较简练，其意向锁即为表级别的锁。设计目的主要是为了在一个事务中揭示下一行将被请求的锁类型。其支持两种意向锁：

- 1) 意向共享锁（IS Lock），事务想要获得一张表中某几行的共享锁
- 2) 意向排他锁（IX Lock），事务想要获得一张表中某几行的排他锁

由于 InnoDB 存储引擎支持的是行级别的锁，因此意向锁其实不会阻塞除全表扫以外的任何请求。故表级意向锁与行级锁的兼容性如表 6-4 所示。

错误描述，意向锁并不会与行级别的共享锁和排他锁互斥

表 6-4 InnoDB 存储引擎中锁的兼容性

	IS	IX	S	X
IS	兼容	兼容	兼容	不兼容
IX	兼容	兼容	不兼容	不兼容
S	兼容	不兼容	兼容	不兼容
X	不兼容	不兼容	不兼容	不兼容

InnoDB 有哪几类行锁？

MySQL InnoDB 支持三种行锁定方式：

- 记录锁（Record Lock）：也被称为记录锁，属于单个行记录上的锁。
- 间隙锁（Gap Lock）：锁定一个范围，不包括记录本身。
- 临键锁（Next-key Lock）：Record Lock+Gap Lock，锁定一个范围，包含记录本身。记录锁只能锁住已经存在的记录，为了避免插入新记录，需要依赖间隙锁。

InnoDB 的默认隔离级别 RR（可重读）是可以解决幻读问题发生的，主要有下面两种情况：

- **快照读**（一致性非锁定读）：由 MVCC 机制来保证不出现幻读。
- **当前读**（一致性锁定读）：使用 Next-Key Lock 进行加锁来保证不出现幻读。

当前读和快照读有什么区别？

快照读（一致性非锁定读）就是单纯的 `SELECT` 语句，但不包括下面这两类 `SELECT` 语句：

```
SELECT ... FOR UPDATE
SELECT ... LOCK IN SHARE MODE
```

快照即记录的历史版本，每行记录可能存在多个历史版本（多版本技术）。

快照读的情况下，如果读取的记录正在执行 UPDATE/DELETE 操作，读取操作不会因此去等待记录上 X 锁的释放，而是会去读取行的一个快照。

只有在事务隔离级别 RC(读取已提交) 和 RR（可重读）下，InnoDB 才会使用一致性非锁定读：

- 在 RC 级别下，对于快照数据，一致性非锁定读总是读取被锁定行的最新一份快照数据。
- 在 RR 级别下，对于快照数据，一致性非锁定读总是读取本事务开始时的行数据版本。

快照读比较适合对于数据一致性要求不是特别高且追求极致性能的业务场景。

当前读（一致性锁定读）就是给行记录加 X 锁或 S 锁。

当前读的一些常见 SQL 语句类型如下：

```
# 对读的记录加一个x锁
SELECT...FOR UPDATE
# 对读的记录加一个s锁
SELECT...LOCK IN SHARE MODE
# 对修改的记录加一个x锁
INSERT...
UPDATE...
DELETE...
```

参考

- 《高性能 MySQL》第 7 章 MySQL 高级特性
- 《MySQL 技术内幕 InnoDB 存储引擎》第 6 章 锁
- Relational Database: <https://www.omnisci.com/technical-glossary/relational-database>
- 技术分享 | 隔离级别：正确理解幻读: <https://opensource.actionsky.com/20210818-mysql/>
- MySQL Server Logs - MySQL 5.7 Reference Manual: <https://dev.mysql.com/doc/refman/5.7/en/server-logs.html>
- Redo Log - MySQL 5.7 Reference Manual: <https://dev.mysql.com/doc/refman/5.7/en/innodb-redo-log.html>
- Locking Reads - MySQL 5.7 Reference Manual: <https://dev.mysql.com/doc/refman/5.7/en/innodb-locking-reads.html>
- 深入理解数据库行锁与表锁 <https://zhuanlan.zhihu.com/p/52678870>
- 详解 MySQL InnoDB 中意向锁的作用: <https://juejin.cn/post/6844903666332368909>
- 在数据库中不可重复读和幻读到底应该怎么分? : <https://www.zhihu.com/question/392569386>



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

4.3 Redis

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide!

Redis 基础

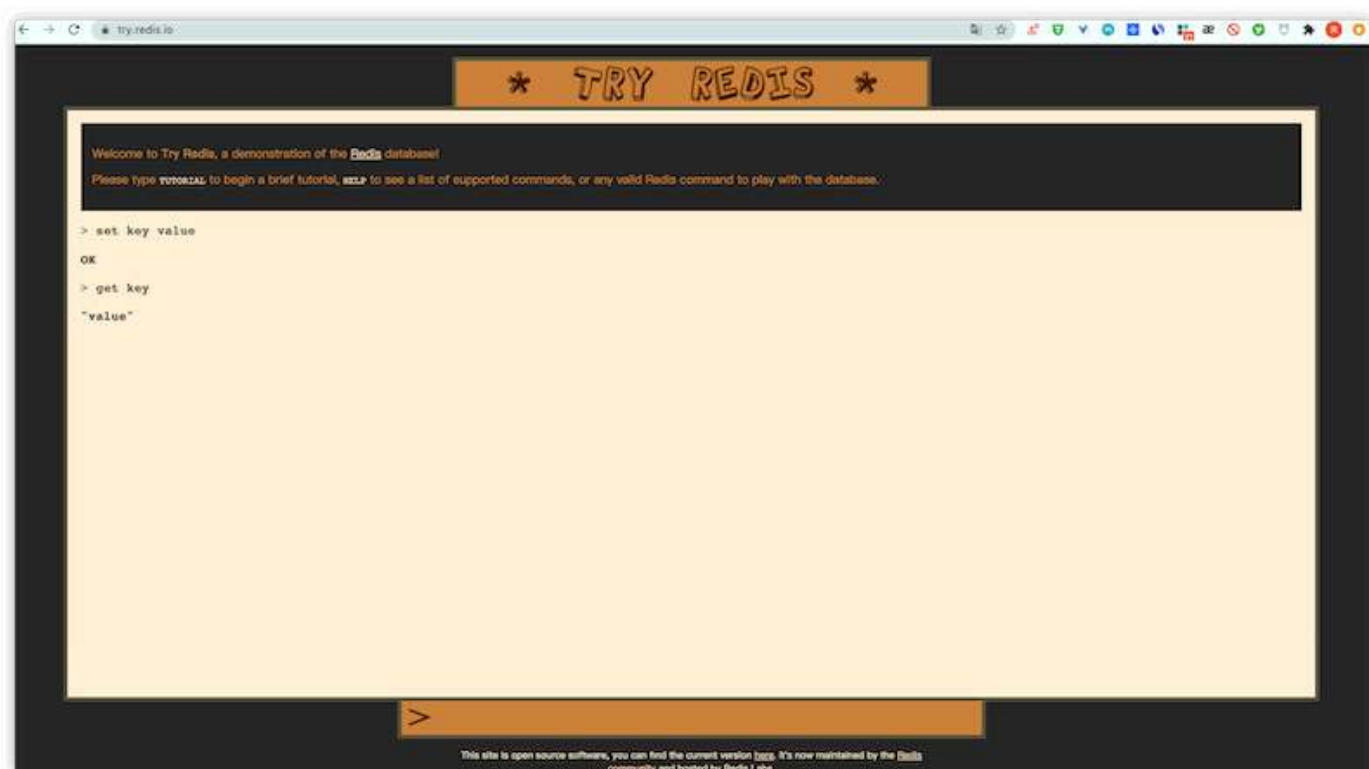
简单介绍一下 Redis!

简单来说 **Redis** 就是一个使用 **C** 语言开发的数据库，不过与传统数据库不同的是 **Redis** 的数据是存在内存中的，也就是它是内存数据库，所以读写速度非常快，因此 Redis 被广泛应用于缓存方向。

另外，Redis 除了做缓存之外，也经常用来做分布式锁，甚至是消息队列。

Redis 提供了多种数据类型来支持不同的业务场景。Redis 还支持事务、持久化、Lua 脚本、多种集群方案。

你可以自己本机安装 Redis 或者通过 Redis 官网提供的[在线 Redis 环境](#)来实际体验 Redis。



分布式缓存常见的技术选型方案有哪些？

分布式缓存的话，使用的比较多的主要是 **Memcached** 和 **Redis**。不过，现在基本没有看过还有项目使用 **Memcached** 来做缓存，都是直接用 **Redis**。

Memcached 是分布式缓存最开始兴起的那会，比较常用的。后来，随着 Redis 的发展，大家慢慢都转而使用更加强大的 Redis 了。

分布式缓存主要解决的是单机缓存的容量受服务器限制并且无法保存通用信息的问题。因为，本地缓存只在当前服务里有效，比如如果你部署了两个相同的服务，他们两者之间的缓存数据是无法共同的。

说一下 Redis 和 Memcached 的区别和共同点

现在公司一般都是用 Redis 来实现缓存，而且 Redis 自身也越来越强大了！不过，了解 Redis 和 Memcached 的区别和共同点，有助于我们在做相应的技术选型的时候，能够做到有理有据！

共同点：

1. 都是基于内存的数据库，一般都用来当做缓存使用。
2. 都有过期策略。
3. 两者的性能都非常高。

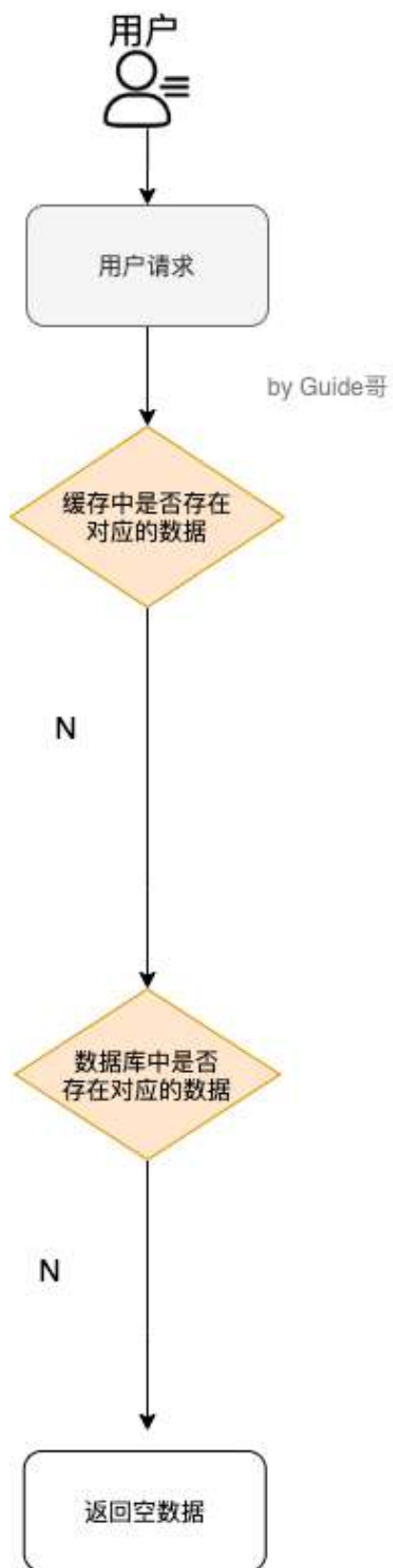
区别：

1. **Redis 支持更丰富的数据类型（支持更复杂的应用场景）**。Redis 不仅仅支持简单的 k/v 类型的数据，同时还提供 list, set, zset, hash 等数据结构的存储。Memcached 只支持最简单的 k/v 数据类型。
2. **Redis 支持数据的持久化**，可以将内存中的数据保持在磁盘中，重启的时候可以再次加载进行使用,而 Memcached 把数据全部存在内存之中。
3. **Redis 有灾难恢复机制**。因为可以把缓存中的数据持久化到磁盘上。
4. **Redis 在服务器内存使用完之后**，可以将不用的数据放到磁盘上。但是，Memcached 在服务器内存使用完之后，就会直接报异常。
5. **Memcached 没有原生的集群模式**，需要依靠客户端来实现往集群中分片写入数据；但是 Redis 目前是原生支持 cluster 模式的。
6. **Memcached 是多线程，非阻塞 IO 复用的网络模型**；Redis 使用单线程的多路 IO 复用模型。（Redis 6.0 引入了多线程 IO）
7. **Redis 支持发布订阅模型、Lua 脚本、事务等功能**，而 Memcached 不支持。并且，Redis 支持更多的编程语言。
8. **Memcached 过期数据的删除策略只用了惰性删除**，而 Redis 同时使用了惰性删除与定期删除。

相信看了上面的对比之后，我们已经没有什么理由可以选择使用 Memcached 来作为自己项目的分布式缓存了。

缓存数据的处理流程是怎样的？

作为暖男一号，我给大家画了一个草图。



简单来说就是：

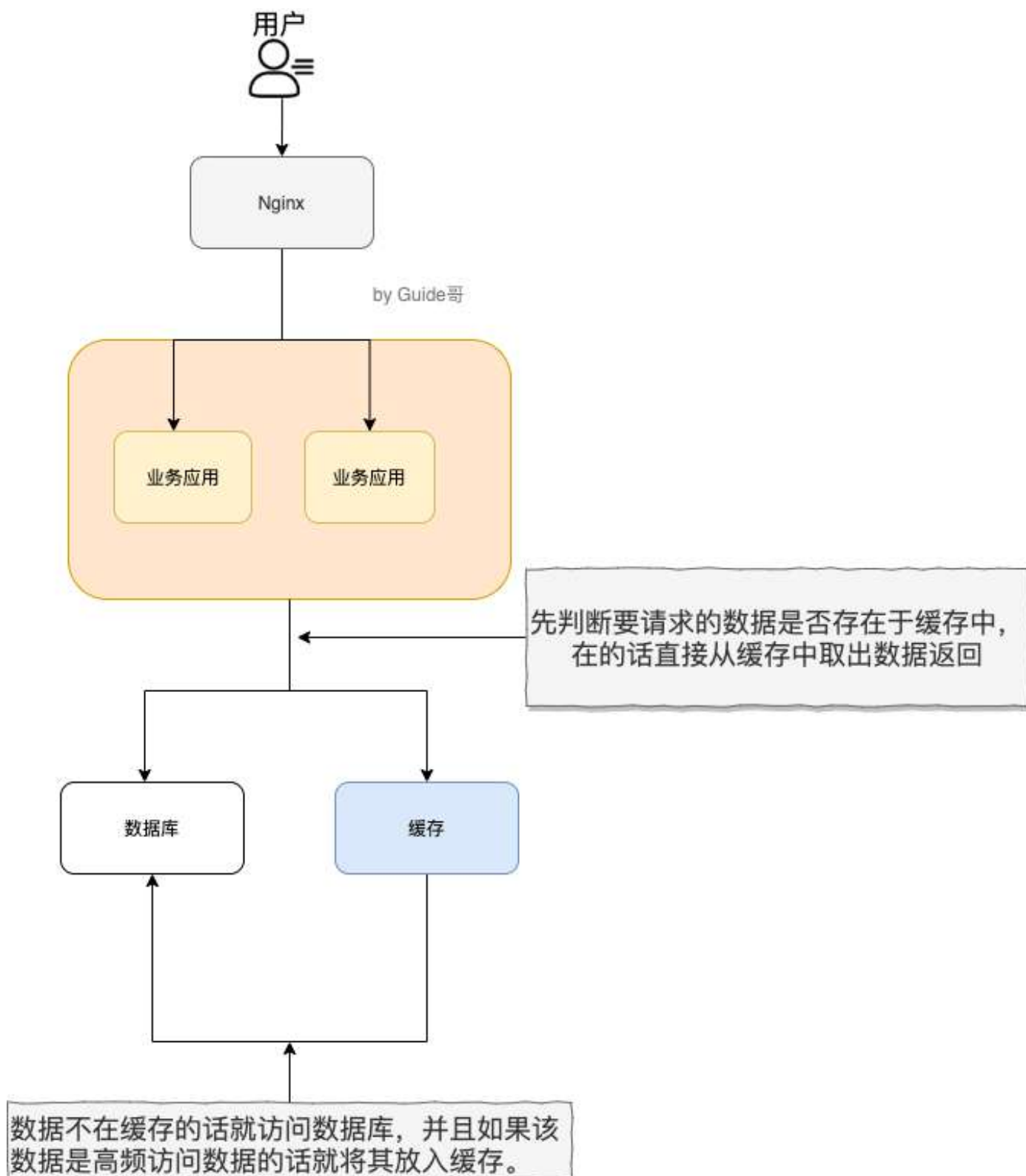
1. 如果用户请求的数据在缓存中就直接返回。

2. 缓存中不存在的话就看数据库中是否存在。
3. 数据库中存在的的话就更新缓存中的数据。
4. 数据库中不存在的话就返回空数据。

为什么要用 Redis/为什么要用缓存?

简单，来说使用缓存主要是为了提升用户体验以及应对更多的用户。

下面我们主要从“高性能”和“高并发”这两点来看待这个问题。



高性能：

对照上面 👉 我画的图。我们设想这样的场景：

假如用户第一次访问数据库中的某些数据的话，这个过程是比较慢，毕竟是从硬盘中读取的。但是，如果说，用户访问的数据属于高频数据并且不会经常改变的话，那么我们就可以很放心地将该用户访问的数据存在缓存中。

这样有什么好处呢？那就是保证用户下一次再访问这些数据的时候就可以直接从缓存中获取了。操作缓存就是直接操作内存，所以速度相当快。

不过，要保持数据库和缓存中的数据的一致性。如果数据库中的对应数据改变的之后，同步改变缓存中相应的数据即可！

高并发：

一般像 MySQL 这类的数据库的 QPS 大概都在 1w 左右（4 核 8g），但是使用 Redis 缓存之后很容易达到 10w+，甚至最高能达到 30w+（就单机 redis 的情况，redis 集群的话会更高）。

QPS（Query Per Second）：服务器每秒可以执行的查询次数；

由此可见，直接操作缓存能够承受的数据库请求数量是远远大于直接访问数据库的，所以我们可以考虑把数据库中的部分数据转移到缓存中去，这样用户的一部分请求会直接到缓存这里而不用经过数据库。进而，我们也就提高了系统整体的并发。

Redis 除了做缓存，还能做什么？

- **分布式锁**：通过 Redis 来做分布式锁是一种比较常见的方式。通常情况下，我们都是基于 Redisson 来实现分布式锁。相关阅读：[《分布式锁中的王者方案 - Redisson》](#)。
- **限流**：一般是通过 Redis + Lua 脚本的方式来实现限流。相关阅读：[《我司用了 6 年的 Redis 分布式限流器，可以说是非常厉害了！》](#)。
- **消息队列**：Redis 自带的 list 数据结构可以作为一个简单的队列使用。Redis 5.0 中增加的 Stream 类型的数据结构更加适合用来做消息队列。它比较类似于 Kafka，有主题和消费组的概念，支持消息持久化以及 ACK 机制。
- **复杂业务场景**：通过 Redis 以及 Redis 扩展（比如 Redisson）提供的数据结构，我们可以很方便地完成很多复杂的业务场景比如通过 bitmap 统计活跃用户、通过 sorted set 维护排行榜。
-

Redis 可以做消息队列么？

Redis 5.0 新增加的一个数据结构 `Stream` 可以用来做消息队列，`Stream` 支持：

- 发布 / 订阅模式
- 按照消费者组进行消费
- 消息持久化（RDB 和 AOF）

不过，和专业的消息队列相比，还是有很多欠缺的地方比如消息丢失和堆积问题不好解决。因此，我们通常建议是不使用 Redis 来做消息队列的，你完全可以选择市面上比较成熟的一些消息队列比如 RocketMQ、Kafka。

相关文章推荐：[Redis 消息队列的三种方案（List、Streams、Pub/Sub）](#)。

Redis 数据结构

Redis 常用的数据结构有哪些？

- **5 种基础数据结构**：String（字符串）、List（列表）、Set（集合）、Hash（散列）、Zset（有序集合）。
- **3 种特殊数据结构**：HyperLogLogs（基数统计）、Bitmap（位存储）、Geospatial (地理位置)。

关于 5 种基础数据结构的详细介绍请看这篇文章：[Redis 5 种基本数据结构详解](#)。

关于 3 种特殊数据结构的详细介绍请看这篇文章：[Redis 3 种特殊数据结构详解](#)。

String 的应用场景有哪些？

- 常规数据（比如 session、token、序列化后的对象）的缓存；
- 计数比如用户单位时间的请求数（简单限流可以用到）、页面单位时间的访问数；
- 分布式锁(利用 `SETNX key value` 命令可以实现一个最简易的分布式锁)；
-

关于 String 的详细介绍请看这篇文章：[Redis 5 种基本数据结构详解](#)。

String 还是 Hash 存储对象数据更好呢？

- String 存储的是序列化后的对象数据，存放的是整个对象。Hash 是对对象的每个字段单独存储，可以获取部分字段的信息，也可以修改或者添加部分字段，节省网络流量。如果对象中某些字段需要经常变动或者经常需要单独查询对象中的个别字段信息，Hash 就非常适合。
- String 存储相对来说更加节省内存，缓存相同数量的对象数据，String 消耗的内存约是 Hash 的一半。并且，存储具有多层嵌套的对象时也方便很多。如果系统对性能和资源消耗非常敏感的话，String 就非常适合。

在绝大部分情况，我们建议使用 String 来存储对象数据即可！

那根据你的介绍，购物车信息用 String 还是 Hash 存储更好呢？

购物车信息建议使用 Hash 存储：

- 用户 id 为 key
- 商品 id 为 field，商品数量为 value

由于购物车中的商品频繁修改和变动，这个时候 Hash 就非常适合了！

使用 Redis 实现一个排行榜怎么做？

Redis 中有一个叫做 `sorted set` 的数据结构经常被用在各种排行榜的场景，比如直播间送礼物的排行榜、朋友圈的微信步数排行榜、王者荣耀中的段位排行榜、话题热度排行榜等等。

相关的一些 Redis 命令：`ZRANGE`（从小到大排序）、`ZREVRANGE`（从大到小排序）、`ZREVRANK`（指定元素排名）。

排行榜

1

11071

0

2

9907

1

3

9793

0

4

9577

0

5

8582

0

6

8524

0

《[Java 面试指北](#)》的「技术面试题篇」就有一篇文章详细介绍如何使用 Sorted Set 来设计制作一个排行榜。

《Java面试指...

+

SnailClimb

目录 暂存箱

介绍

面试准备篇

技术面试题篇

系统设计

如何准备系统设计面试?

如何设计一个秒杀系统?

如何自己实现一个 RPC ...

如何设计一个排行榜?

如何设计微博 Feed 流/...

如何设计一个短链系统?

如何设计一个站内消息...

如何解决大文件上传问...

如何统计网站UV?

Java

数据库

常见框架

分布式

高并发

服务器

Devops

技术面试题自测篇

面经篇

练级攻略篇

工作篇

如何设计一个排行榜?

sorted set 基本可以满足大部分排行榜的场景。

如果我们要查看包含所有用户的排行榜怎么办? 通过 ZRANGE (从小到大排序) / ZREVRANGE (从大到小排序)

Bash

复制代码

```
1 # -1 代表的是全部的用户数据,
2 127.0.0.1:6379> ZREVRANGE cus_order_set 0 -1
3 1) "user6"
4 2) "user3"
5 3) "user1"
6 4) "user4"
7 5) "user2"
8 6) "user5"
```

如果我们要查看只包含前 3 名的排行榜怎么办? 限定范围区间即可。

Bash

复制代码

```
1 # 0 为 start 2 为 stop
2 127.0.0.1:6379> ZREVRANGE cus_order_set 0 2
3 1) "user6"
4 2) "user3"
5 3) "user1"
```

如果我们需要查询某个用户的分数怎么办呢? 通过 ZSCORE 命令即可。

Bash

复制代码

```
1 127.0.0.1:6379> ZSCORE cus_order_set "user1"
2 "112"
```

如果我们需要查询某个用户的排名怎么办呢? 通过 ZREVRANK 命令即可。

使用 Set 实现抽奖系统需要用到什么命令?

- SPOP key count : 随机移除并获取指定集合中一个或多个元素, 适合不允许重复中奖的场景。
- SRANDMEMBER key count : 随机获取指定集合中指定数量的元素, 适合允许重复中奖的场景。

使用 Bitmap 统计活跃用户怎么做？

使用日期（精确到天）作为 key，然后用户 ID 为 offset，如果当日活跃过就设置为 1。

初始化数据：

```
> SETBIT desk1 20210308 1 1
(integer) 0
> SETBIT desk1 20210308 2 1
(integer) 0
> SETBIT desk1 20210309 1 1
(integer) 0
```

统计 20210308~20210309 总活跃用户数：

```
> BITOP and desk1 20210308 20210309
(integer) 1
> BITCOUNT desk1
(integer) 1
```

统计 20210308~20210309 在线活跃用户数：

```
> BITOP or desk2 20210308 20210309
(integer) 1
> BITCOUNT desk2
(integer) 2
```

使用 HyperLogLog 统计页面 UV 怎么做？

1、将访问指定页面的每个用户 ID 添加到 HyperLogLog 中。

```
PFADD PAGE_1:UV USER1 USER2 ..... USERn
```

2、统计指定页面的 UV。

```
PFCOUNT PAGE_1:UV
```

Redis 线程模型

对于读写命令来说，Redis 一直是单线程模型。不过，在 Redis 4.0 版本之后引入了多线程来执行一些大键值对的异步删除操作，Redis 6.0 版本之后引入了多线程来处理网络请求（提高网络 IO 读写性能）。

Redis 单线程模型了解吗？

Redis 基于 **Reactor** 模式来设计开发了自己的一套高效的事件处理模型（Netty 的线程模型也基于 Reactor 模式，Reactor 模式不愧是高性能 IO 的基石），这套事件处理模型对应的是 Redis 中的文件事件处理器（file event handler）。由于文件事件处理器（file event handler）是单线程方式运行的，所以我们一般都说 Redis 是单线程模型。

既然是单线程，那怎么监听大量的客户端连接呢？

Redis 通过 **IO 多路复用程序** 来监听来自客户端的大量连接（或者说是监听多个 socket），它会将感兴趣的事件及类型（读、写）注册到内核中并监听每个事件是否发生。

这样的好处非常明显：**IO 多路复用技术**的使用让 Redis 不需要额外创建多余的线程来监听客户端的大量连接，降低了资源的消耗（和 NIO 中的 `Selector` 组件很像）。

另外，Redis 服务器是一个事件驱动程序，服务器需要处理两类事件：

- **文件事件(file event)**：用于处理 Redis 服务器和客户端之间的网络 IO。
- **时间事件(time event)**：Redis 服务器中的一些操作（比如 `serverCron` 函数）需要在给定的时间点执行，而时间事件就是处理这类定时操作的。

时间事件不需要多花时间了解，我们接触最多的还是 **文件事件**（客户端进行读取写入等操作，涉及一系列网络通信）。

《Redis 设计与实现》有一段话是如是介绍文件事件的，我觉得写得挺不错。

Redis 基于 Reactor 模式开发了自己的网络事件处理器：这个处理器被称为文件事件处理器（file event handler）。文件事件处理器使用 I/O 多路复用（multiplexing）程序来同时监听多个套接字，并根据套接字目前执行的任务来为套接字关联不同的事件处理器。

当被监听的套接字准备好执行连接应答（accept）、读取（read）、写入（write）、关闭（close）等操作时，与操作相对应的文件事件就会产生，这时文件事件处理器就会调用套接字之前关联好的事件处理器来处理这些事件。

虽然文件事件处理器以单线程方式运行，但通过使用 I/O 多路复用程序来监听多个套接字，文件事件处理器既实现了高性能的网络通信模型，又可以很好地与 Redis 服务器中其他同样以单线程方式运行的模块进行对接，这保持了 Redis 内部单线程设计的简单性。

可以看出，文件事件处理器（file event handler）主要是包含 4 个部分：

- 多个 socket（客户端连接）
- IO 多路复用程序（支持多个客户端连接的关键）
- 文件事件分派器（将 socket 关联到相应的事件处理器）
- 事件处理器（连接应答处理器、命令请求处理器、命令回复处理器）



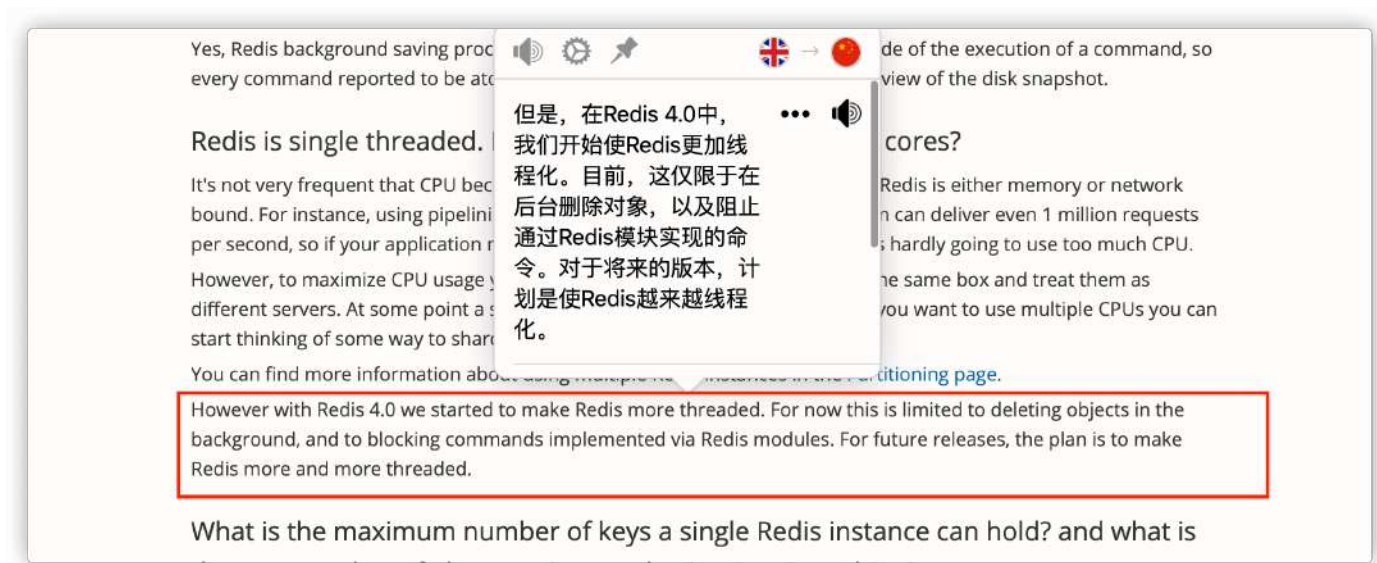
相关阅读：[Redis 事件机制详解](#)。

Redis6.0 之前为什么不使用多线程？

虽然说 Redis 是单线程模型，但是，实际上，Redis 在 4.0 之后的版本中就已经加入了对多线程的支持。

不过，Redis 4.0 增加的多线程主要是针对一些大键值对的删除操作的命令，使用这些命令就会使用主线程之外的其他线程来“异步处理”。

为此，Redis 4.0 之后新增了 UNLINK（可以看作是 DEL 的异步版本）、FLUSHALL ASYNC（清空数据库）、FLUSHDB ASYNC（清空数据库）等异步命令。



大体上来说，Redis 6.0 之前主要还是单线程处理。

那 Redis6.0 之前为什么不使用多线程？我觉得主要原因有 3 点：

- 单线程编程容易并且更容易维护；
- Redis 的性能瓶颈不在 CPU，主要在内存和网络；
- 多线程就会存在死锁、线程上下文切换等问题，甚至会影响性能。

Redis6.0 之后为何引入了多线程？

Redis6.0 引入多线程主要是为了提高网络 IO 读写性能，因为这个算是 Redis 中的一个性能瓶颈（Redis 的瓶颈主要受限于内存和网络）。

虽然，Redis6.0 引入了多线程，但是 Redis 的多线程只是在网络数据的读写这类耗时操作上使用了，执行命令仍然是单线程顺序执行。因此，你也不需要担心线程安全问题。

Redis6.0 的多线程默认是禁用的，只使用主线程。如需开启需要修改 redis 配置文件 redis.conf：

```
io-threads-do-reads yes
```

开启多线程后，还需要设置线程数，否则是不生效的。同样需要修改 redis 配置文件 `redis.conf`：

```
io-threads 4 #官网建议4核的机器建议设置为2或3个线程，8核的建议设置为6个线程
```

推荐阅读：

- [Redis 6.0 新特性-多线程连环 13 问！](#)
- [为什么 Redis 选择单线程模型](#)
- [Redis 多线程网络模型全面揭秘](#)

Redis 内存管理

Redis 给缓存数据设置过期时间有啥用？

一般情况下，我们设置保存的缓存数据的时候都会设置一个过期时间。为什么呢？

因为内存是有限的，如果缓存中的所有数据都是一直保存的话，分分钟直接 Out of memory。

Redis 自带了给缓存数据设置过期时间的功能，比如：

```
127.0.0.1:6379> exp key 60 # 数据在 60s 后过期
(integer) 1
127.0.0.1:6379> setex key 60 value # 数据在 60s 后过期 (setex:[set] + [ex]pire)
OK
127.0.0.1:6379> ttl key # 查看数据还有多久过期
(integer) 56
```

注意：Redis 中除了字符串类型有自己独有设置过期时间的命令 `setex` 外，其他方法都需要依靠 `expire` 命令来设置过期时间。另外，`persist` 命令可以移除一个键的过期时间。

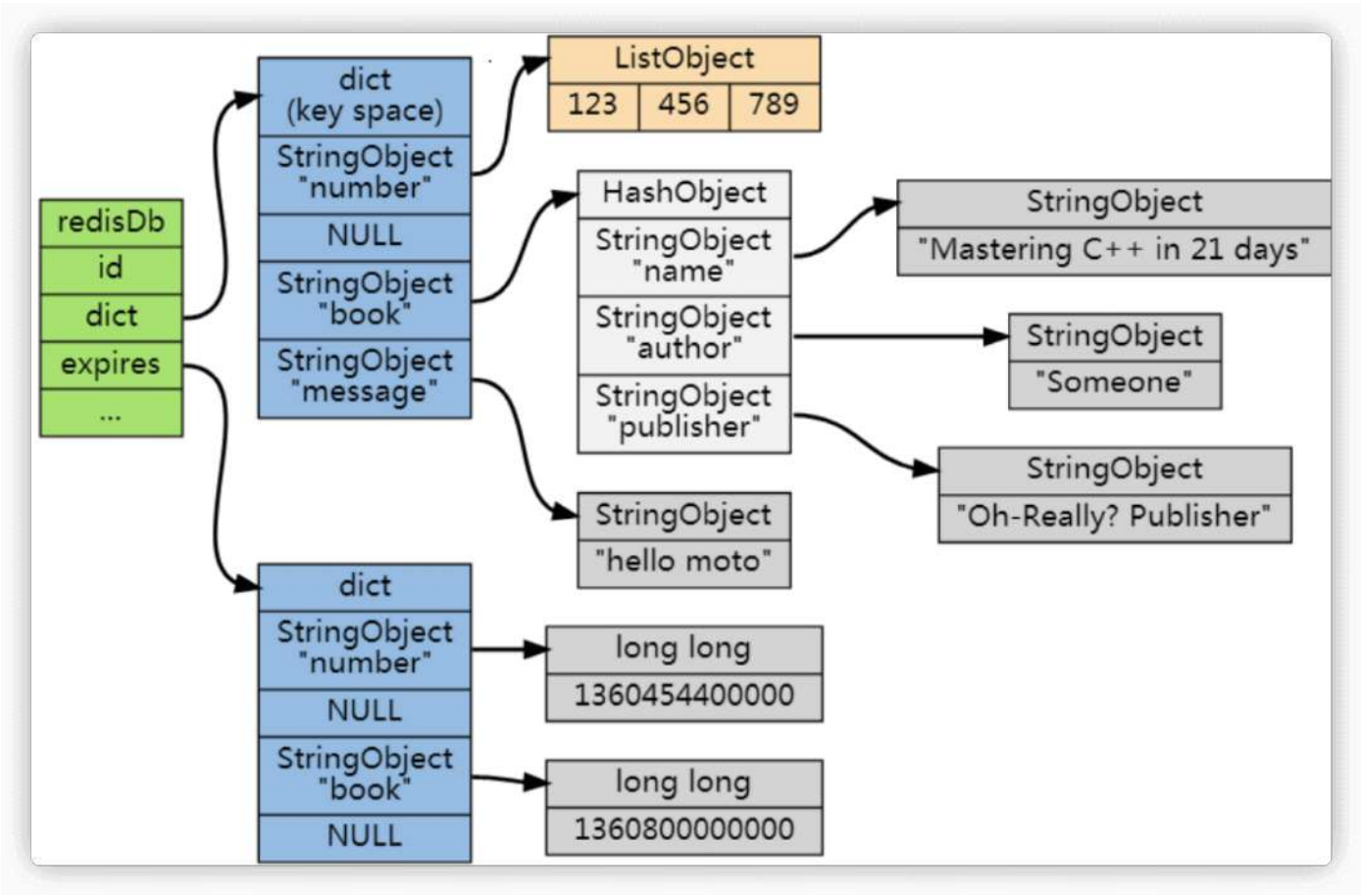
过期时间除了有助于缓解内存的消耗，还有什么其他用么？

很多时候，我们的业务场景就是需要某个数据只在某一时间段内存在，比如我们的短信验证码可能只在 1 分钟内有效，用户登录的 token 可能只在 1 天内有效。

如果使用传统的数据库来处理的话，一般都是自己判断过期，这样更麻烦并且性能要差很多。

Redis 是如何判断数据是否过期的呢？

Redis 通过一个叫做过期字典（可以看作是 hash 表）来保存数据过期的时间。过期字典的键指向 Redis 数据库中的某个 key(键)，过期字典的值是一个 long long 类型的整数，这个整数保存了 key 所指向的数据库键的过期时间（毫秒精度的 UNIX 时间戳）。



过期字典是存储在 `redisDb` 这个结构里的：

```
typedef struct redisDb {  
    ...  
  
    dict *dict;        //数据库键空间,保存着数据库中所有键值对  
    dict *expires      // 过期字典,保存着键的过期时间  
  
    ...  
} redisDb;
```

过期的数据的删除策略了解么？

如果假设你设置了一批 key 只能存活 1 分钟，那么 1 分钟后，Redis 是怎么对这批 key 进行删除的呢？

常用的过期数据的删除策略就两个（重要！自己造缓存轮子的时候需要格外考虑的东西）：

1. **惰性删除**：只会在取出 key 的时候才对数据进行过期检查。这样对 CPU 最友好，但是可能会造成太多过期 key 没有被删除。
2. **定期删除**：每隔一段时间抽取一批 key 执行删除过期 key 操作。并且，Redis 底层会通过限制删除操作执行的时长和频率来减少删除操作对 CPU 时间的影响。

定期删除对内存更加友好，惰性删除对 CPU 更加友好。两者各有千秋，所以 Redis 采用的是 **定期删除+惰性/懒汉式删除**。

但是，仅仅通过给 key 设置过期时间还是有问题的。因为还是可能存在定期删除和惰性删除漏掉了很多过期 key 的情况。这样就导致大量过期 key 堆积在内存里，然后就 Out of memory 了。

怎么解决这个问题呢？答案就是：**Redis 内存淘汰机制**。

Redis 内存淘汰机制了解么？

相关问题：MySQL 里有 2000w 数据，Redis 中只存 20w 的数据，如何保证 Redis 中的数据都是热点数据？

Redis 提供 6 种数据淘汰策略：

1. **volatile-lru (least recently used)**：从已设置过期时间的数据集（server.db[i].expires）中挑选最近最少使用的数据淘汰
2. **volatile-ttl**：从已设置过期时间的数据集（server.db[i].expires）中挑选将要过期的数据淘汰

3. **volatile-random**: 从已设置过期时间的数据集 (`server.db[i].expires`) 中任意选择数据淘汰
4. **allkeys-lru (least recently used)**: 当内存不足以容纳新写入数据时, 在键空间中, 移除最近最少使用的 key (这个是最常用的)
5. **allkeys-random**: 从数据集 (`server.db[i].dict`) 中任意选择数据淘汰
6. **no-eviction**: 禁止驱逐数据, 也就是说当内存不足以容纳新写入数据时, 新写入操作会报错。这个应该没人使用吧!

4.0 版本后增加以下两种:

7. **volatile-lfu (least frequently used)**: 从已设置过期时间的数据集 (`server.db[i].expires`) 中挑选最不经常使用的数据淘汰
8. **allkeys-lfu (least frequently used)**: 当内存不足以容纳新写入数据时, 在键空间中, 移除最不经常使用的 key

Redis 持久化机制

怎么保证 Redis 挂掉之后再重启数据可以进行恢复?

很多时候我们需要持久化数据也就是将内存中的数据写入到硬盘里面, 大部分原因是为了之后重用数据 (比如重启机器、机器故障之后恢复数据), 或者是为了防止系统故障而将数据备份到一个远程位置。

Redis 不同于 Memcached 的很重要一点就是, Redis 支持持久化, 而且支持两种不同的持久化操作。**Redis** 的一种持久化方式叫快照 (**snapshotting, RDB**), 另一种方式是只追加文件 (**append-only file, AOF**)。这两种方法各有千秋, 下面我会详细这两种持久化方法是什么, 怎么用, 如何选择适合自己的持久化方法。

什么是 RDB 持久化?

Redis 可以通过创建快照来获得存储在内存里面的数据在某个时间点上的副本。Redis 创建快照之后, 可以对快照进行备份, 可以将快照复制到其他服务器从而创建具有相同数据的服务器副本 (Redis 主从结构, 主要用来提高 Redis 性能), 还可以将快照留在原地以便重启服务器的时候使用。

快照持久化是 Redis 默认采用的持久化方式, 在 `redis.conf` 配置文件中默认有此下配置:


```
save 900 1          #在900秒(15分钟)之后, 如果至少有1个key发生变化, Redis就会自动触发
bgsave命令创建快照。

save 300 10         #在300秒(5分钟)之后, 如果至少有10个key发生变化, Redis就会自动触发
bgsave命令创建快照。

save 60 10000       #在60秒(1分钟)之后, 如果至少有10000个key发生变化, Redis就会自动触发
bgsave命令创建快照。
```

RDB 创建快照时会阻塞主线程吗?

Redis 提供了两个命令来生成 RDB 快照文件:

- `save` : 主线程执行, 会阻塞主线程;
- `bgsave` : 子线程执行, 不会阻塞主线程, 默认选项。

什么是 AOF 持久化?

与快照持久化相比, AOF 持久化的实时性更好, 因此已成为主流的持久化方案。默认情况下 Redis 没有开启 AOF (append only file) 方式的持久化, 可以通过 `appendonly` 参数开启:

```
appendonly yes
```

开启 AOF 持久化后每执行一条会更改 Redis 中的数据命令, Redis 就会将该命令写入到内存缓存 `server.aof_buf` 中, 然后再根据 `appendfsync` 配置来决定何时将其同步到硬盘中的 AOF 文件。

AOF 文件的保存位置和 RDB 文件的位置相同, 都是通过 `dir` 参数设置的, 默认的文件名是 `appendonly.aof`。

在 Redis 的配置文件中存在三种不同的 AOF 持久化方式, 它们分别是:

```
appendfsync always  #每次有数据修改发生时都会写入AOF文件, 这样会严重降低Redis的速度
appendfsync everysec #每秒钟同步一次, 显式地将多个写命令同步到硬盘
appendfsync no       #让操作系统决定何时进行同步
```

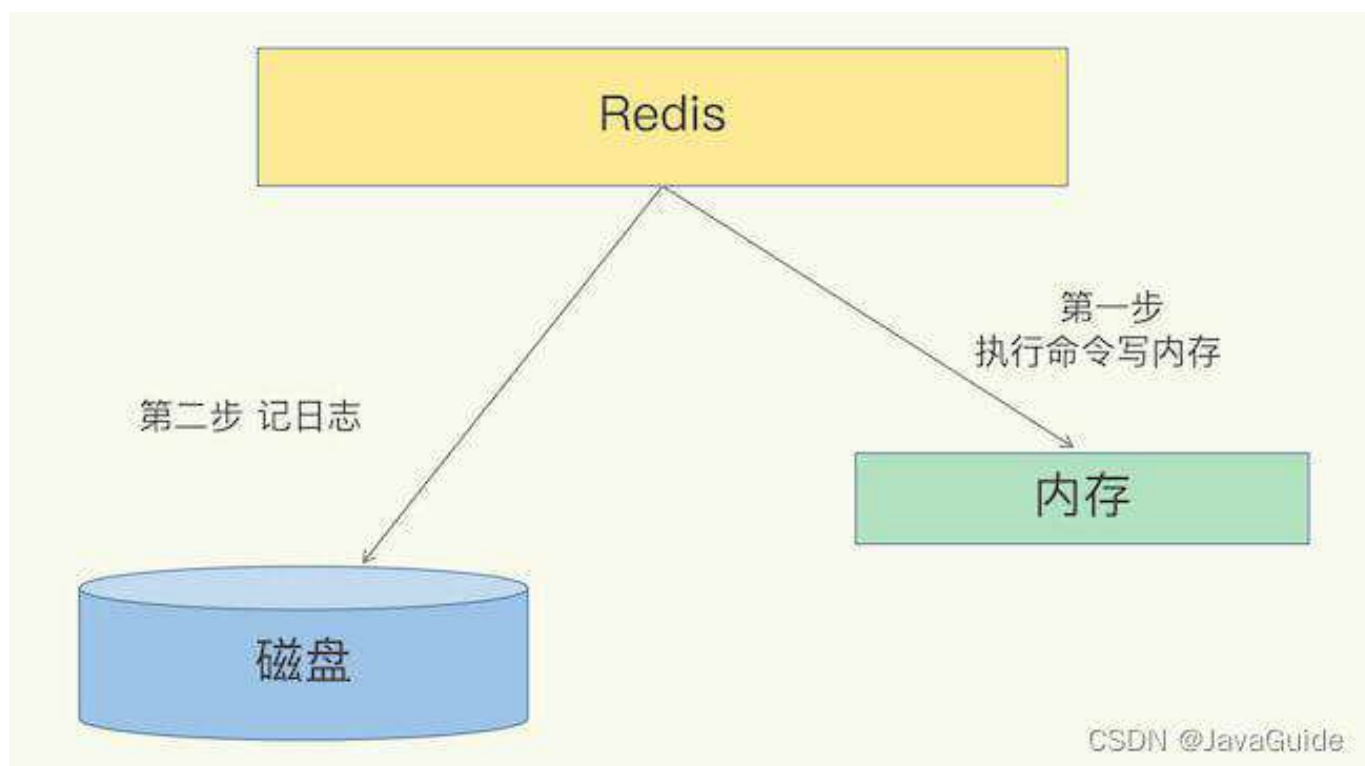

为了兼顾数据和写入性能，用户可以考虑 `appendfsync everysec` 选项，让 Redis 每秒同步一次 AOF 文件，Redis 性能几乎没受到任何影响。而且这样即使出现系统崩溃，用户最多只会丢失一秒之内产生的数据。当硬盘忙于执行写入操作的时候，Redis 还会优雅的放慢自己的速度以便适应硬盘的最大写入速度。

相关 issue：

- [Redis 的 AOF 方式 #783](#)
- [Redis AOF 重写描述不准确 #1439](#)

AOF 日志是如何实现的？

关系型数据库（如 MySQL）通常都是执行命令之前记录日志（方便故障恢复），而 Redis AOF 持久化机制是在执行完命令之后再记录日志。



为什么是在执行完命令之后记录日志呢？

- 避免额外的检查开销，AOF 记录日志不会对命令进行语法检查；
- 在命令执行完之后再记录，不会阻塞当前的命令执行。

这样也带来了风险（我在前面介绍 AOF 持久化的时候也提到过）：

- 如果刚执行完命令 Redis 就宕机会导致对应的修改丢失；
- 可能会阻塞后续其他命令的执行（AOF 记录日志是在 Redis 主线程中进行的）。

AOF 重写了解吗？

AOF 重写可以产生一个新的 AOF 文件，这个新的 AOF 文件和原有的 AOF 文件所保存的数据库状态一样，但体积更小。

AOF 重写是一个有歧义的名字，该功能是通过读取数据库中的键值对来实现的，程序无须对现有 AOF 文件进行任何读入、分析或者写入操作。

在执行 `BGREWRITEAOF` 命令时，Redis 服务器会维护一个 AOF 重写缓冲区，该缓冲区会在子进程创建新 AOF 文件期间，记录服务器执行的所有写命令。当子进程完成创建新 AOF 文件的工作之后，服务器会将重写缓冲区中的所有内容追加到新 AOF 文件的末尾，使得新的 AOF 文件保存的数据库状态与现有的数据库状态一致。最后，服务器用新的 AOF 文件替换旧的 AOF 文件，以此来完成 AOF 文件重写操作。

Redis 4.0 对于持久化机制做了什么优化？

Redis 4.0 开始支持 RDB 和 AOF 的混合持久化（默认关闭，可以通过配置项 `aof-use-rdb-preamble` 开启）。

如果把混合持久化打开，AOF 重写的时候就直接把 RDB 的内容写到 AOF 文件开头。这样做的好处是可以结合 RDB 和 AOF 的优点，快速加载同时避免丢失过多的数据。当然缺点也是有的，AOF 里面的 RDB 部分是压缩格式不再是 AOF 格式，可读性较差。

官方文档地址：<https://redis.io/topics/persistence>

Redis Persistence

Redis provides a different range of persistence options:

- **RDB** (Redis Database): The RDB persistence performs point-in-time snapshots of your dataset at specified intervals.
- **AOF** (Append Only File): The AOF persistence logs every write operation received by the server, that will be played again at server startup, reconstructing the original dataset. Commands are logged using the same format as the Redis protocol itself, in an append-only fashion. Redis is able to rewrite the log in the background when it gets too big.
- **No persistence**: If you wish, you can disable persistence completely, if you want your data to just exist as long as the server is running.
- **RDB + AOF**: It is possible to combine both AOF and RDB in the same instance. Notice that, in this case, when Redis restarts the AOF file will be used to reconstruct the original dataset since it is guaranteed to be the most complete.

Redis 事务

如何使用 Redis 事务？

Redis 可以通过 `MULTI`，`EXEC`，`DISCARD` 和 `WATCH` 等命令来实现事务(transaction)功能。

```
> MULTI
OK
> SET USER "Guide哥"
QUEUED
> GET USER
QUEUED
> EXEC
1) OK
2) "Guide哥"
```

使用 `MULTI` 命令后可以输入多个命令。Redis 不会立即执行这些命令，而是将它们放到队列，当调用了 `EXEC` 命令将执行所有命令。

这个过程是这样的：

1. 开始事务（`MULTI`）。
2. 命令入队(批量操作 Redis 的命令，先进先出（FIFO）的顺序执行)。
3. 执行事务(`EXEC`)。

你也可以通过 `DISCARD` 命令取消一个事务，它会清空事务队列中保存的所有命令。

```
> MULTI
OK
> SET USER "Guide哥"
QUEUED
> GET USER
QUEUED
> DISCARD
OK
```

`WATCH` 命令用于监听指定的键，当调用 `EXEC` 命令执行事务时，如果一个被 `WATCH` 命令监视的键被修改的话，整个事务都不会执行，直接返回失败。

```
> WATCH USER
OK
> MULTI
> SET USER "Guide哥"
OK
> GET USER
Guide哥
> EXEC
ERR EXEC without MULTI
```

Redis 官网相关介绍 <https://redis.io/topics/transactions> 如下：

Transactions

`MULTI`, `EXEC`, `DISCARD` and `WATCH` are the foundation of transactions in Redis. They allow the execution of a group of commands in a single step, with two important guarantees:

- All the commands in a transaction are serialized and executed sequentially. It can never happen that a request issued by another client is served **in the middle** of the execution of a Redis transaction. This guarantees that the commands are executed as a single isolated operation.
- Either all of the commands or none are processed, so a Redis transaction is also atomic. The `EXEC` command triggers the execution of all the commands in the transaction, so if a client loses the connection to the server in the context of a transaction before calling the `EXEC` command none of the operations are performed, instead if the `EXEC` command is called, all the operations are performed. When using the `append-only file` Redis makes sure to use a single `write(2)` syscall to write the transaction on disk. However if the Redis server crashes or is killed by the system administrator in some hard way it is possible that only a partial number of operations are registered. Redis will detect this condition at restart, and will exit with an error. Using the `redis-check-aof` tool it is possible to fix the append only file that will remove the partial transaction so that the server can start again.

Starting with version 2.2, Redis allows for an extra guarantee to the above two, in the form of optimistic locking in a way very similar to a check-and-set (CAS) operation. This is documented [later](#) on this page.

Redis 支持原子性吗？

Redis 的事务和我们平时理解的关系型数据库的事务不同。我们知道事务具有四大特性：**1. 原子性**，**2. 隔离性**，**3. 持久性**，**4. 一致性**。

- 1. 原子性 (Atomicity)：** 事务是最小的执行单位，不允许分割。事务的原子性确保动作要么全部完成，要么完全不起作用；
- 2. 隔离性 (Isolation)：** 并发访问数据库时，一个用户的事务不被其他事务所干扰，各并发事务之间数据库是独立的；
- 3. 持久性 (Durability)：** 一个事务被提交之后。它对数据库中数据的改变是持久的，即使数据库

发生故障也不应该对其有任何影响。

4. **一致性 (Consistency)**：执行事务前后，数据保持一致，多个事务对同一个数据读取的结果是相同的；

Redis 事务在运行错误的情况下，除了执行过程中出现错误的命令外，其他命令都能正常执行。并且，Redis 是不支持回滚 (roll back) 操作的。因此，Redis 事务其实是不满足原子性的（而且不满足持久性）。

Redis 官网也解释了自己为啥不支持回滚。简单来说就是 Redis 开发者们觉得没必要支持回滚，这样更简单便捷并且性能更好。Redis 开发者觉得即使命令执行错误也应该在开发过程中就被发现而不是生产过程中。

Why Redis does not support roll backs?

If you have a relational databases background, the fact that Redis commands can fail during a transaction, but still Redis will execute the rest of the transaction instead of rolling back, may look odd to you.

However there are good opinions for this behavior:

- Redis commands can fail only if called with a wrong syntax (and the problem is not detectable during the command queueing), or against keys holding the wrong data type: this means that in practical terms a failing command is the result of a programming errors, and a kind of error that is very likely to be detected during development, and not in production.
- Redis is internally simplified and faster because it does not need the ability to roll back.

An argument against Redis point of view is that bugs happen, however it should be noted that in general the roll back does not save you from programming errors. For instance if a query increments a key by 2 instead of 1, or increments the wrong key, there is no way for a rollback mechanism to help. Given that no one can save the programmer from his or her errors, and that the kind of errors required for a Redis command to fail are unlikely to enter in production, we selected the simpler and faster approach of not supporting roll backs on errors.

你可以将 Redis 中的事务就理解为：**Redis 事务提供了一种将多个命令请求打包的功能。然后，再按顺序执行打包的所有命令，并且不会被中途打断。**

除了不满足原子性之外，事务中的每条命令都会与 Redis 服务器进行网络交互，这是比较浪费资源的行为。明明一次批量执行多个命令就可以了，这种操作实在是看不懂。

因此，Redis 事务是不建议在日常开发中使用的。

相关 issue：

- [issue452: 关于 Redis 事务不满足原子性的问题](#)。
- [Issue491:关于 redis 没有事务回滚？](#)

如何解决 Redis 事务的缺陷？

Redis 从 2.6 版本开始支持执行 Lua 脚本，它的功能和事务非常类似。我们可以利用 Lua 脚本来批量执行多条 Redis 命令，这些 Redis 命令会被提交到 Redis 服务器一次性执行完成，大幅减小了网络开销。

一段 Lua 脚本可以视作一条命令执行，一段 Lua 脚本执行过程中不会有其他脚本或 Redis 命令同时执行，保证了操作不会被其他指令插入或打扰。

如果 Lua 脚本运行时出错并中途结束，出错之后的命令是不会被执行的。并且，出错之前执行的命令是无法被撤销的。因此，严格来说，通过 Lua 脚本来批量执行 Redis 命令也是不满足原子性的。

另外，Redis 7.0 新增了 [Redis functions](#) 特性，你可以将 Redis functions 看作是比 Lua 更强大的脚本。

Redis 性能优化

Redis bigkey

什么是 bigkey？

简单来说，如果一个 key 对应的 value 所占用的内存比较大，那这个 key 就可以看作是 bigkey。具体多大才算大呢？有一个不是特别精确的参考标准：string 类型的 value 超过 10 kb，复合类型的 value 包含的元素超过 5000 个（对于复合类型的 value 来说，不一定包含的元素越多，占用的内存就越多）。

bigkey 有什么危害？

除了会消耗更多的内存空间，bigkey 对性能也会有比较大的影响。

因此，我们应该尽量避免写入 bigkey！

如何发现 bigkey？

1、使用 Redis 自带的 `--bigkeys` 参数来查找。

```
# redis-cli -p 6379 --bigkeys

# Scanning the entire keyspace to find biggest keys as well as
# average sizes per key type. You can use -i 0.1 to sleep 0.1 sec
# per 100 SCAN commands (not usually needed).
```



```
[00.00%] Biggest string found so far '"ballcat:oauth:refresh_auth:f6cdb384-9a9d-4f2f-af01-dc3f28057c20"' with 4437 bytes
[00.00%] Biggest list found so far '"my-list"' with 17 items

----- summary -----

Sampled 5 keys in the keyspace!
Total key length in bytes is 264 (avg len 52.80)

Biggest list found '"my-list"' has 17 items
Biggest string found '"ballcat:oauth:refresh_auth:f6cdb384-9a9d-4f2f-af01-dc3f28057c20"' has 4437 bytes

1 lists with 17 items (20.00% of keys, avg size 17.00)
0 hashes with 0 fields (00.00% of keys, avg size 0.00)
4 strings with 4831 bytes (80.00% of keys, avg size 1207.75)
0 streams with 0 entries (00.00% of keys, avg size 0.00)
0 sets with 0 members (00.00% of keys, avg size 0.00)
0 zsets with 0 members (00.00% of keys, avg size 0.00)
```

从这个命令的运行结果，我们可以看出：这个命令会扫描(Scan) Redis 中的所有 key，会对 Redis 的性能有一点影响。并且，这种方式只能找出每种数据结构 top 1 bigkey（占用内存最大的 string 数据类型，包含元素最多的复合数据类型）。

2、分析 RDB 文件

通过分析 RDB 文件来找出 big key。这种方案的前提是你的 Redis 采用的是 RDB 持久化。

网上有现成的代码/工具可以直接拿来使用：

- [redis-rdb-tools](#)：Python 语言写的用来分析 Redis 的 RDB 快照文件用的工具
- [rdb_bigkeys](#)：Go 语言写的用来分析 Redis 的 RDB 快照文件用的工具，性能更好。

大量 key 集中过期问题

我在上面提到过：对于过期 key，Redis 采用的是 **定期删除+惰性/懒汉式删除** 策略。

定期删除执行过程中，如果突然遇到大量过期 key 的话，客户端请求必须等待定期清理过期 key 任务线程执行完成，因为这个这个定期任务线程是在 Redis 主线程中执行的。这就导致客户端请求没办法被及时处理，响应速度会比较慢。

如何解决呢？下面是两种常见的方法：

1. 给 key 设置随机过期时间。
2. 开启 lazy-free（惰性删除/延迟释放）。lazy-free 特性是 Redis 4.0 开始引入的，指的是让 Redis 采用异步方式延迟释放 key 使用的内存，将该操作交给单独的子线程处理，避免阻塞主线程。

个人建议不管是否开启 lazy-free，我们都尽量给 key 设置随机过期时间。

Redis 生产问题

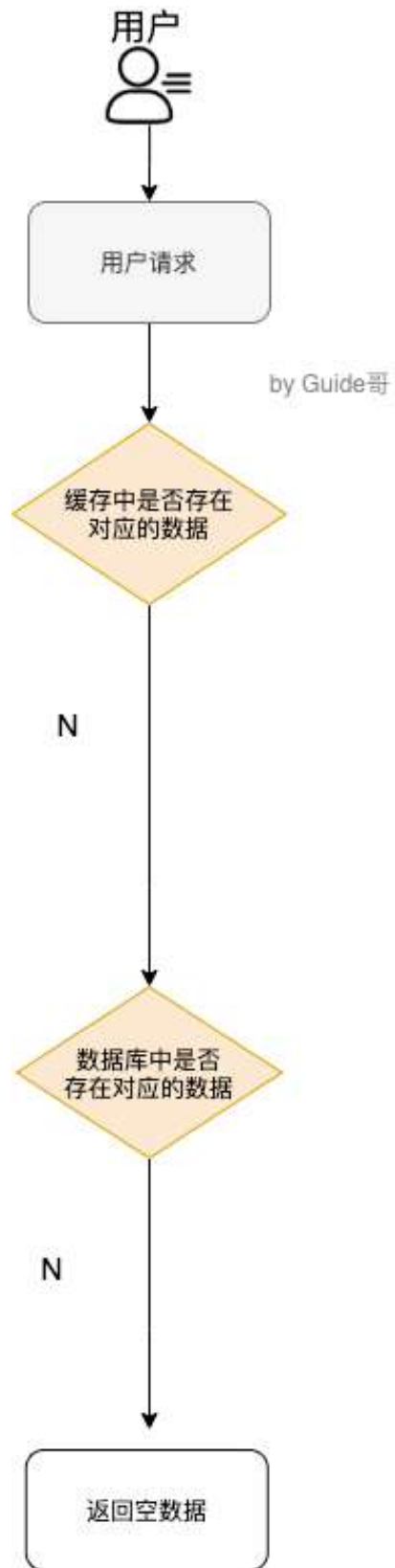
缓存穿透

什么是缓存穿透？

缓存穿透说简单点就是大量请求的 key 根本不存在于缓存中，导致请求直接到了数据库上，根本没有经过缓存这一层。举个例子：某个黑客故意制造我们缓存中不存在的 key 发起大量请求，导致大量请求落到数据库。

缓存穿透情况的处理流程是怎样的？

如下图所示，用户的请求最终都要跑到数据库中查询一遍。



有哪些解决办法？

最基本的就是首先做好参数校验，一些不合法的参数请求直接抛出异常信息返回给客户端。比如查询的数据库 id 不能小于 0、传入的邮箱格式不对的时候直接返回错误消息给客户端等等。

1) 缓存无效 key

如果缓存和数据库都查不到某个 key 的数据就写一个到 Redis 中去并设置过期时间，具体命令如下：`SET key value EX 10086`。这种方式可以解决请求的 key 变化不频繁的情况，如果黑客恶意攻击，每次构建不同的请求 key，会导致 Redis 中缓存大量无效的 key。很明显，这种方案并不能从根本上解决此问题。如果非要用这种方式来解决穿透问题的话，尽量将无效的 key 的过期时间设置短一点比如 1 分钟。

另外，这里多说一嘴，一般情况下我们是这样设计 key 的：`表名:列名:主键名:主键值`。

如果用 Java 代码展示的话，差不多是下面这样的：

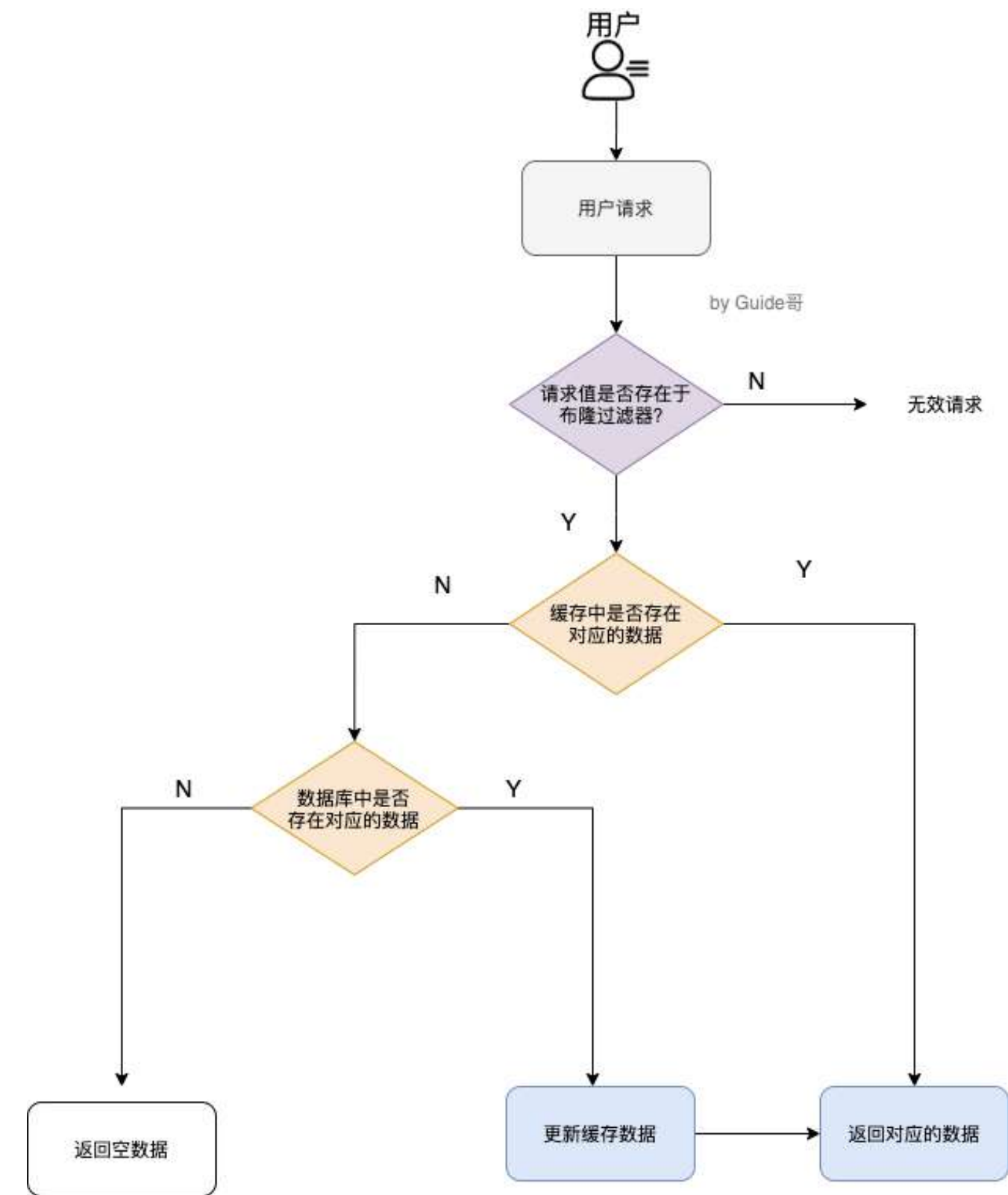
```
public Object getObjectInclNullById(Integer id) {
    // 从缓存中获取数据
    Object cacheValue = cache.get(id);
    // 缓存为空
    if (cacheValue == null) {
        // 从数据库中获取
        Object storageValue = storage.get(key);
        // 缓存空对象
        cache.set(key, storageValue);
        // 如果存储数据为空，需要设置一个过期时间(300秒)
        if (storageValue == null) {
            // 必须设置过期时间，否则有被攻击的风险
            cache.expire(key, 60 * 5);
        }
        return storageValue;
    }
    return cacheValue;
}
```

2) 布隆过滤器

布隆过滤器是一个非常神奇的数据结构，通过它我们可以非常方便地判断一个给定数据是否存在于海量数据中。我们需要的就是判断 key 是否合法，有没有感觉布隆过滤器就是我们想要找的那个“人”。

具体是这样做的：把所有可能存在的请求的值都存放在布隆过滤器中，当用户请求过来，先判断用户发来的请求的值是否存在于布隆过滤器中。不存在的话，直接返回请求参数错误信息给客户端，存在的话才会走下面的流程。

加入布隆过滤器之后的缓存处理流程图如下。



但是，需要注意的是布隆过滤器可能会存在误判的情况。总结来说就是：布隆过滤器说某个元素存在，小概率会误判。布隆过滤器说某个元素不在，那么这个元素一定不在。

为什么会出现误判的情况呢？我们还要从布隆过滤器的原理来说！

我们先来看一下，**当一个元素加入布隆过滤器中的时候，会进行哪些操作：**

1. 使用布隆过滤器中的哈希函数对元素值进行计算，得到哈希值（有几个哈希函数得到几个哈希值）。
2. 根据得到的哈希值，在位数组中把对应下标的值置为 1。

我们再来看一下，**当我们需要判断一个元素是否存在于布隆过滤器的时候，会进行哪些操作：**

1. 对给定元素再次进行相同的哈希计算；
2. 得到值之后判断位数组中的每个元素是否都为 1，如果值都为 1，那么说明这个值在布隆过滤器中，如果存在一个值不为 1，说明该元素不在布隆过滤器中。

然后，一定会出现这样一种情况：**不同的字符串可能哈希出来的位置相同。**（可以适当增加位数组大小或者调整我们的哈希函数来降低概率）

更多关于布隆过滤器的内容可以看我的这篇原创：[《不了解布隆过滤器？一文给你整的明明白白！》](#)，强烈推荐，个人感觉网上应该找不到总结的这么明明白白的文章了。

缓存雪崩

什么是缓存雪崩？

我发现缓存雪崩这名字起的有点意思，哈哈。

实际上，缓存雪崩描述的就是这样一个简单的场景：**缓存在同一时间大面积的失效，后面的请求都直接落到了数据库上，造成数据库短时间内承受大量请求。**这就好比雪崩一样，摧枯拉朽之势，数据库的压力可想而知，可能直接就被这么多请求弄宕机了。

举个例子：系统的缓存模块出了问题比如宕机导致不可用。造成系统的所有访问，都要走数据库。

还有一种缓存雪崩的场景是：**有一些被大量访问数据（热点缓存）在某一时刻大面积失效，导致对应的请求直接落到了数据库上。**这样的情况，有下面几种解决办法：

举个例子：秒杀开始 12 个小时之前，我们统一存放了一批商品到 Redis 中，设置的缓存过期时间也是 12 个小时，那么秒杀开始的时候，这些秒杀的商品的访问直接就失效了。导致的情况就是，相应的请求直接就落到了数据库上，就像雪崩一样可怕。

有哪些解决办法？

针对 **Redis** 服务不可用的情况：

1. 采用 Redis 集群，避免单机出现问题整个缓存服务都没办法使用。
2. 限流，避免同时处理大量的请求。

针对热点缓存失效的情况：

1. 设置不同的失效时间比如随机设置缓存的失效时间。
2. 缓存永不失效。

如何保证缓存和数据库数据的一致性？

细说的话可以扯很多，但是我觉得其实没太大必要（小声 BB：很多解决方案我也没太弄明白）。我个人觉得引入缓存之后，如果为了短时间的不一致性问题，选择让系统设计变得更加复杂的话，完全没必要。

下面单独对 **Cache Aside Pattern**（旁路缓存模式）来聊聊。

Cache Aside Pattern 中遇到写请求是这样的：更新 DB，然后直接删除 cache 。

如果更新数据库成功，而删除缓存这一步失败的情况的话，简单说两个解决方案：

1. **缓存失效时间变短（不推荐，治标不治本）**：我们让缓存数据的过期时间变短，这样的话缓存就会从数据库中加载数据。另外，这种解决办法对于先操作缓存后操作数据库的场景不适用。
2. **增加 cache 更新重试机制（常用）**：如果 cache 服务当前不可用导致缓存删除失败的话，我们就隔一段时间进行重试，重试次数可以自己定。如果多次重试还是失败的话，我们可以把当前更新失败的 key 存入队列中，等缓存服务可用之后，再将缓存中对应的 key 删除即可。

相关文章推荐：[缓存和数据库一致性问题，看这篇就够了 - 水滴与银弹](#)

5. 常用框架

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！



[Github](#) |

[Gitee](#)

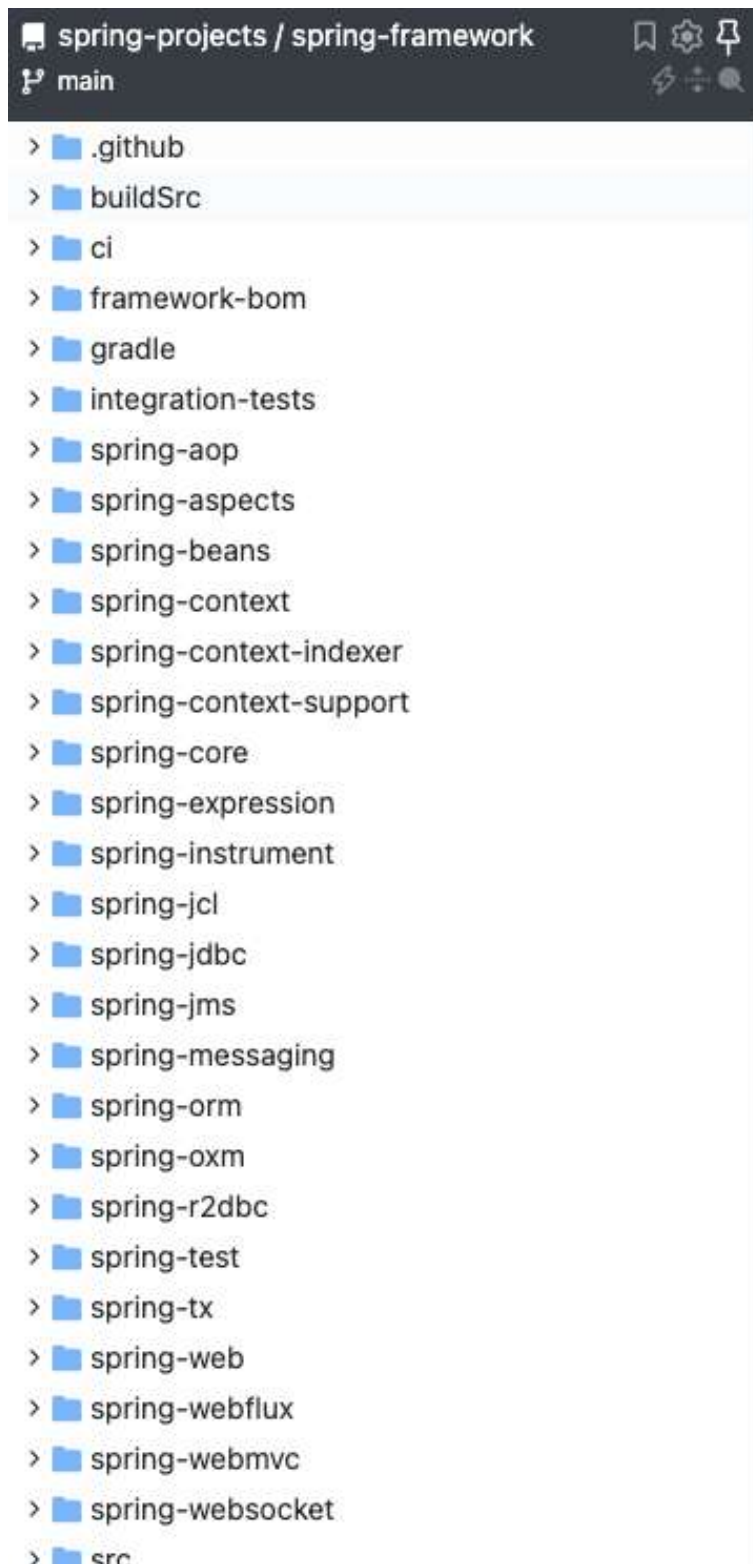
5.1 Spring

Spring 基础

什么是 Spring 框架？

Spring 是一款开源的轻量级 Java 开发框架，旨在提高开发人员的开发效率以及系统的可维护性。

我们一般说 Spring 框架指的都是 Spring Framework，它是很多模块的集合，使用这些模块可以很方便地协助我们进行开发，比如说 Spring 支持 IoC（Inverse of Control:控制反转）和 AOP(Aspect-Oriented Programming:面向切面编程)、可以很方便地对数据库进行访问、可以很方便地集成第三方组件（电子邮件，任务，调度，缓存等等）、对单元测试支持比较好、支持 RESTful Java 应用程序的开发。



Spring 最核心的思想就是不重新造轮子，开箱即用，提高开发效率。

Spring 翻译过来就是春天的意思，可见其目标和使命就是为 Java 程序员带来春天啊！感动！

🙄 多提一嘴：语言的流行通常需要一个杀手级的应用，**Spring** 就是 **Java** 生态的一个杀手级的应用框架。

Spring 提供的核心功能主要是 IoC 和 AOP。学习 Spring，一定要把 IoC 和 AOP 的核心思想搞懂！

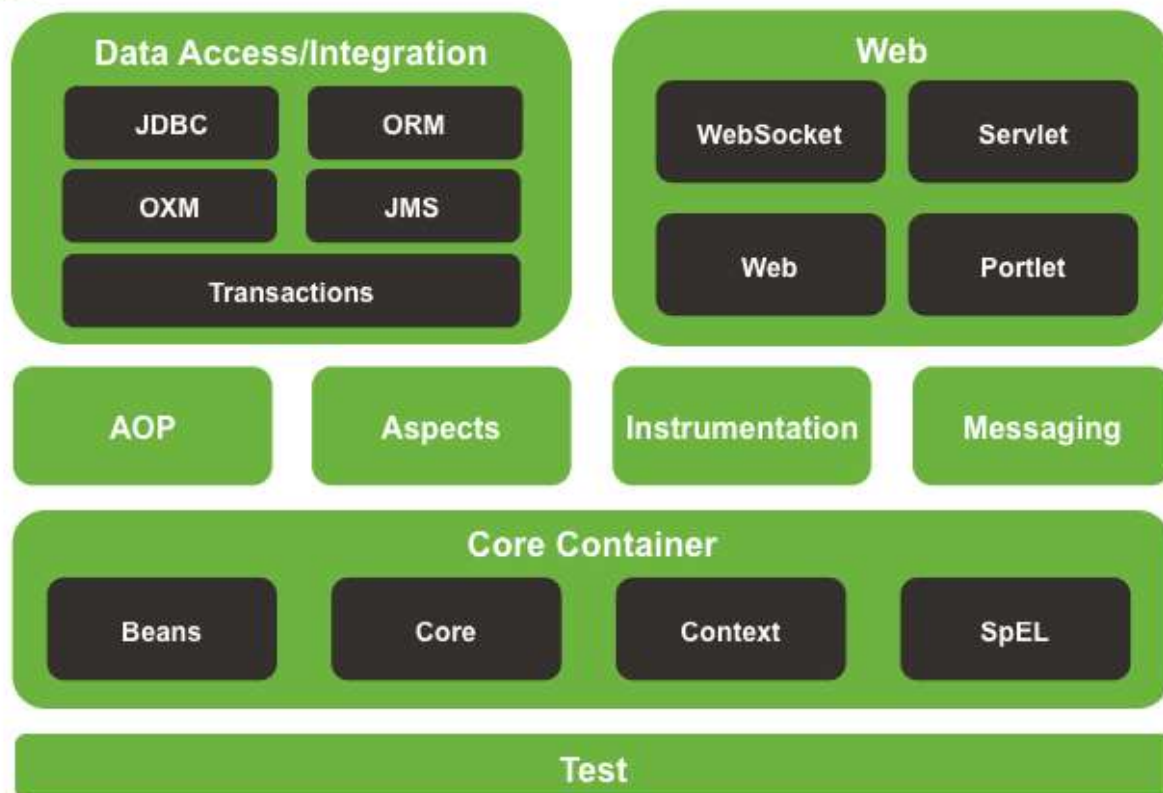
- Spring 官网： <https://spring.io/>
- Github 地址： <https://github.com/spring-projects/spring-framework>

Spring 包含的模块有哪些？

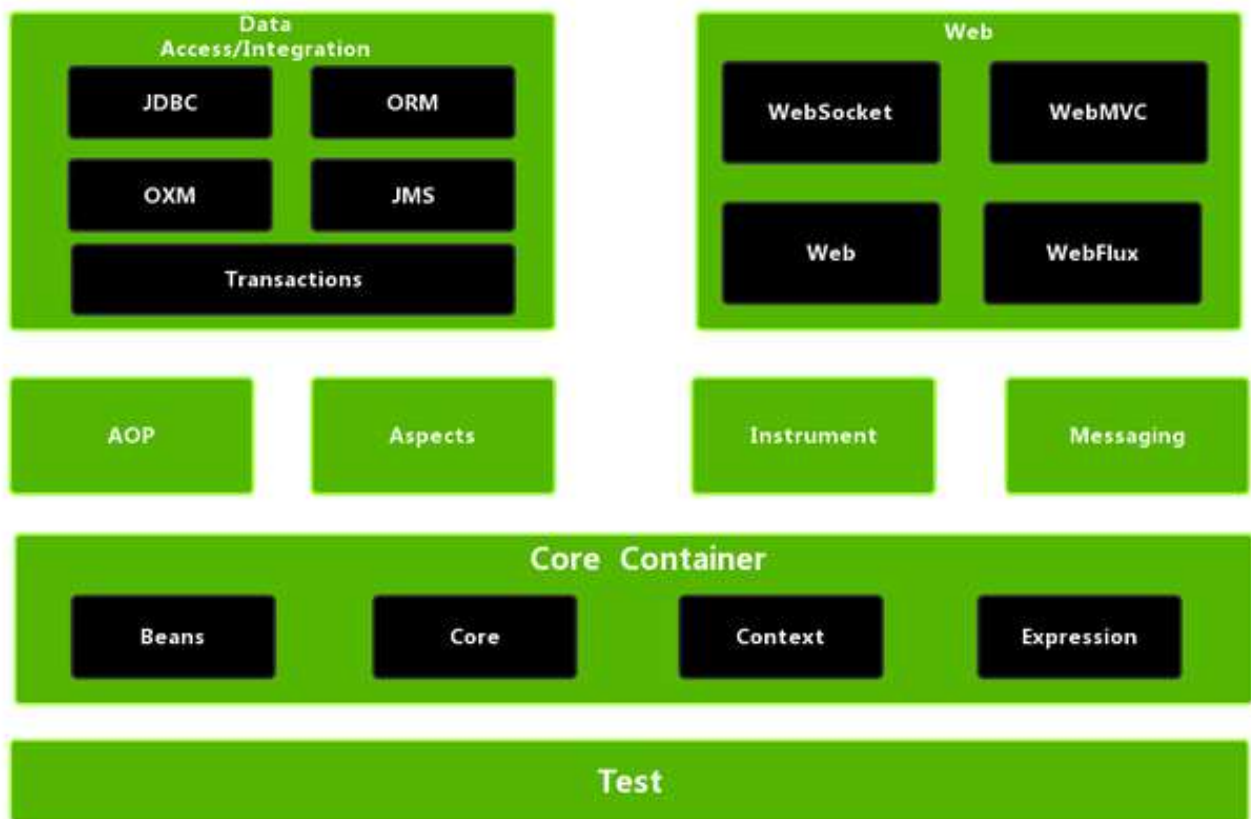
Spring4.x 版本：



Spring Framework Runtime

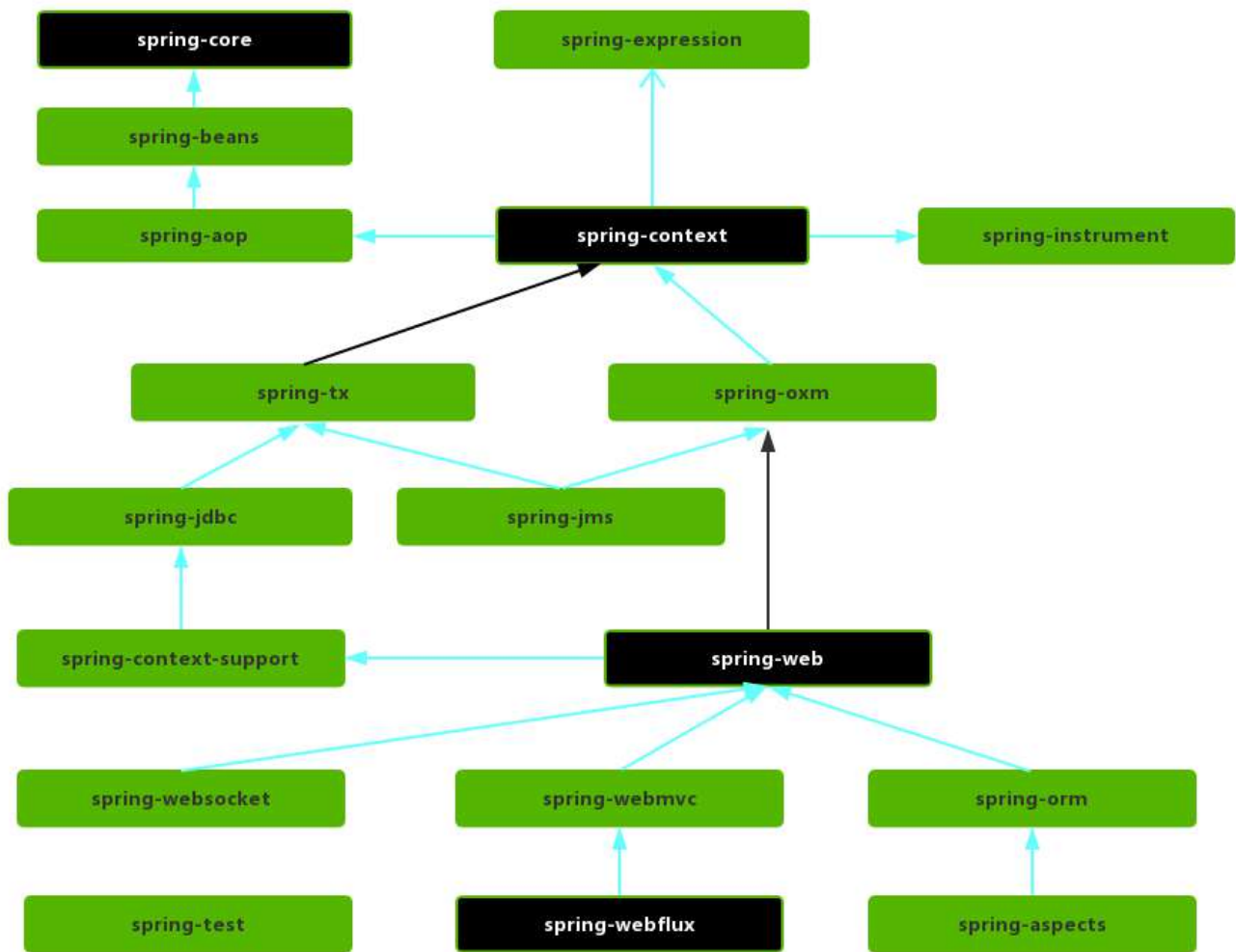


Spring5.x 版本：



Spring5.x 版本中 Web 模块的 Portlet 组件已经被废弃掉，同时增加了用于异步响应式处理的 WebFlux 组件。

Spring 各个模块的依赖关系如下：



Core Container

Spring 框架的核心模块，也可以说是基础模块，主要提供 IoC 依赖注入功能的支持。Spring 其他所有的功能基本都需要依赖于该模块，我们从上面那张 Spring 各个模块的依赖关系图就可以看出来。

- **spring-core**：Spring 框架基本的核心工具类。
- **spring-beans**：提供对 bean 的创建、配置和管理等功能的支持。
- **spring-context**：提供对国际化、事件传播、资源加载等功能的支持。
- **spring-expression**：提供对表达式语言（Spring Expression Language）SpEL 的支持，只依赖于 core 模块，不依赖于其他模块，可以单独使用。

AOP

- **spring-aspects**：该模块为与 AspectJ 的集成提供支持。
- **spring-aop**：提供了面向切面的编程实现。
- **spring-instrument**：提供了为 JVM 添加代理（agent）的功能。具体来讲，它为 Tomcat 提供了一个织入代理，能够为 Tomcat 传递类文件，就像这些文件是被类加载器加载的一样。没有理解也没关系，这个模块的使用场景非常有限。

Data Access/Integration

- **spring-jdbc**：提供了对数据库访问的抽象 JDBC。不同的数据库都有自己独立的 API 用于操作数据库，而 Java 程序只需要和 JDBC API 交互，这样就屏蔽了数据库的影响。
- **spring-tx**：提供对事务的支持。
- **spring-orm**：提供对 Hibernate、JPA、iBatis 等 ORM 框架的支持。
- **spring-oxm**：提供一个抽象层支撑 OXM(Object-to-XML-Mapping)，例如：JAXB、Castor、XMLBeans、JiBX 和 XStream 等。
- **spring-jms**：消息服务。自 Spring Framework 4.1 以后，它还提供了对 spring-messaging 模块的继承。

Spring Web

- **spring-web**：对 Web 功能的实现提供一些最基础的支持。
- **spring-webmvc**：提供对 Spring MVC 的实现。
- **spring-websocket**：提供了对 WebSocket 的支持，WebSocket 可以让客户端和服务端进行双向通信。
- **spring-webflux**：提供对 WebFlux 的支持。WebFlux 是 Spring Framework 5.0 中引入的新的响应式框架。与 Spring MVC 不同，它不需要 Servlet API，是完全异步。

Messaging

spring-messaging 是从 Spring4.0 开始新加入的一个模块，主要职责是为 Spring 框架集成一些基础的报文传送应用。

Spring Test

Spring 团队提倡测试驱动开发（TDD）。有了控制反转 (IoC) 的帮助，单元测试和集成测试变得更简单。

Spring 的测试模块对 JUnit（单元测试框架）、TestNG（类似 JUnit）、Mockito（主要用来 Mock 对象）、PowerMock（解决 Mockito 的问题比如无法模拟 final, static, private 方法）等等常用的测试框架支持的都比较好。

Spring, Spring MVC, Spring Boot 之间什么关系？

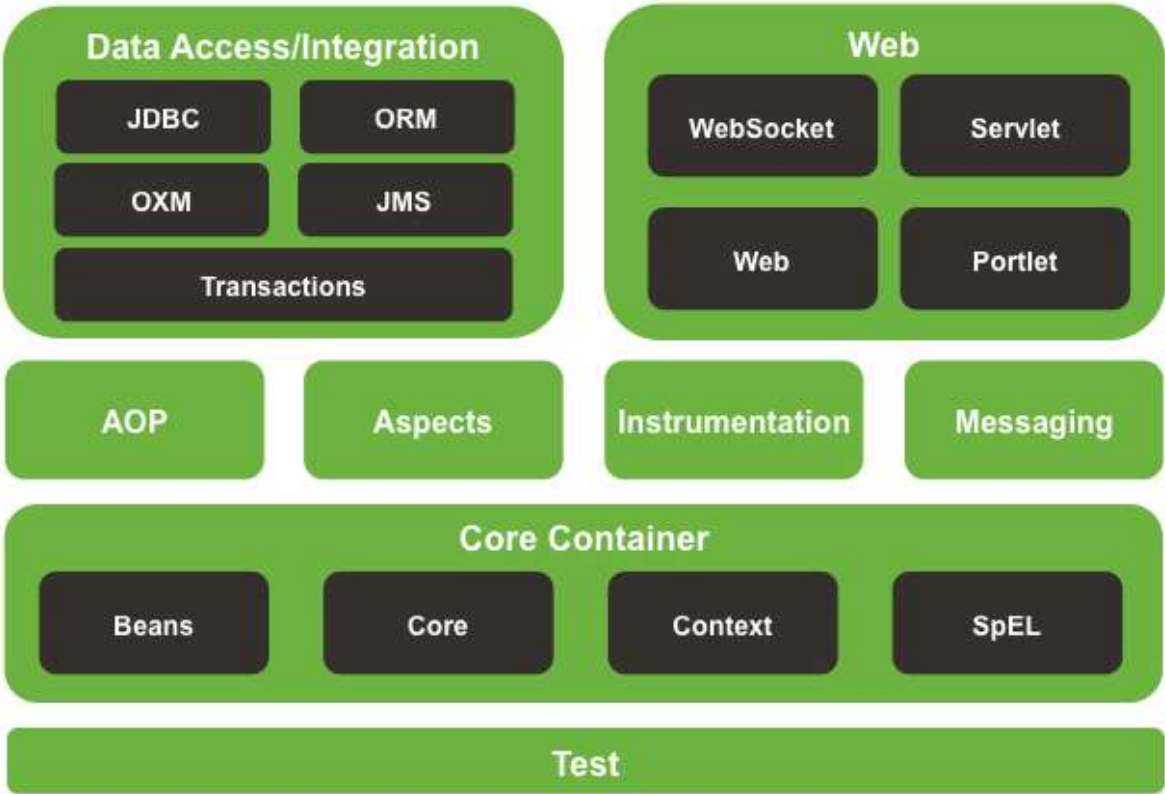
很多人对 Spring, Spring MVC, Spring Boot 这三者傻傻分不清楚！这里简单介绍一下这三者，其实很简单，没有什么高深的东西。

Spring 包含了多个功能模块（上面刚刚提高过），其中最重要的是 Spring-Core（主要提供 IoC 依赖注入功能的支持）模块，Spring 中的其他模块（比如 Spring MVC）的功能实现基本都需要依赖于该模块。

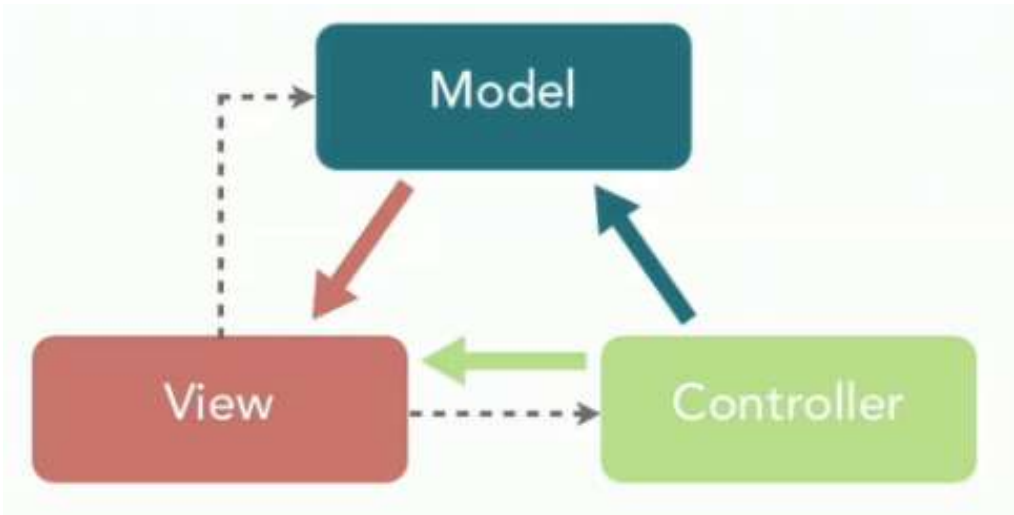
下图对应的是 Spring4.x 版本。目前最新的 5.x 版本中 Web 模块的 Portlet 组件已经被废弃掉，同时增加了用于异步响应式处理的 WebFlux 组件。



Spring Framework Runtime



Spring MVC 是 Spring 中的一个很重要的模块，主要赋予 Spring 快速构建 MVC 架构的 Web 程序的能力。MVC 是模型(Model)、视图(View)、控制器(Controller)的简写，其核心思想是通过将业务逻辑、数据、显示分离来组织代码。



使用 Spring 进行开发各种配置过于麻烦比如开启某些 Spring 特性时，需要用 XML 或 Java 进行显式配置。于是，Spring Boot 诞生了！

Spring 旨在简化 J2EE 企业应用程序开发。Spring Boot 旨在简化 Spring 开发（减少配置文件，开箱即用！）。

Spring Boot 只是简化了配置，如果你需要构建 MVC 架构的 Web 程序，你还是需要使用 Spring MVC 作为 MVC 框架，只是说 Spring Boot 帮你简化了 Spring MVC 的很多配置，真正做到开箱即用！

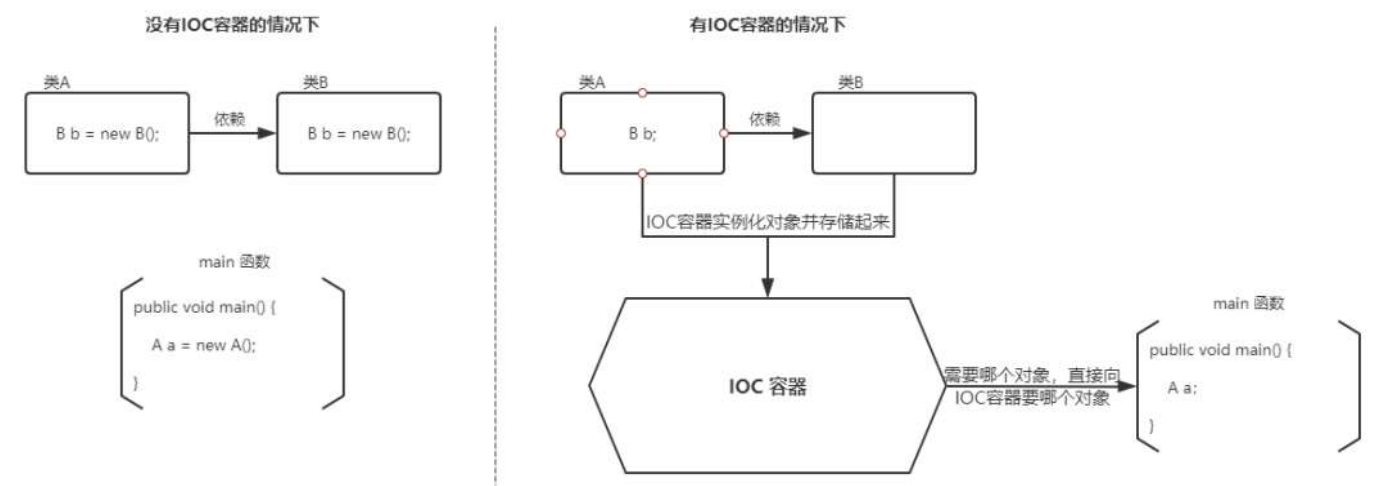
Spring IoC

谈谈自己对于 Spring IoC 的了解

IoC（Inverse of Control:控制反转） 是一种设计思想，而不是一个具体的技术实现。IoC 的思想就是将原本在程序中手动创建对象的控制权，交由 Spring 框架来管理。不过，IoC 并非 Spring 特有，在其他语言中也有应用。

为什么叫控制反转？

- **控制**：指的是对象创建（实例化、管理）的权力
- **反转**：控制权交给外部环境（Spring 框架、IoC 容器）



将对象之间的相互依赖关系交给 IoC 容器来管理，并由 IoC 容器完成对象的注入。这样可以很大程度上简化应用的开发，把应用从复杂的依赖关系中解放出来。IoC 容器就像是一个工厂一样，当我们需要创建一个对象的时候，只需要配置好配置文件/注解即可，完全不用考虑对象是如何被创建出来的。

在实际项目中一个 Service 类可能依赖了很多其他的类，假如我们需要实例化这个 Service，你可能要每次都要搞清这个 Service 所有底层类的构造函数，这可能会把人逼疯。如果利用 IoC 的话，你只需要配置好，然后在需要的地方引用就行了，这大大增加了项目的可维护性且降低了开发难度。

在 Spring 中，IoC 容器是 Spring 用来实现 IoC 的载体，IoC 容器实际上就是个 Map (key, value)，Map 中存放的是各种对象。

Spring 时代我们一般通过 XML 文件来配置 Bean，后来开发人员觉得 XML 文件来配置不太好，于是 SpringBoot 注解配置就慢慢开始流行起来。

相关阅读：

- [IoC 源码阅读](#)
- [面试被问了几百遍的 IoC 和 AOP，还在傻傻搞不清楚？](#)

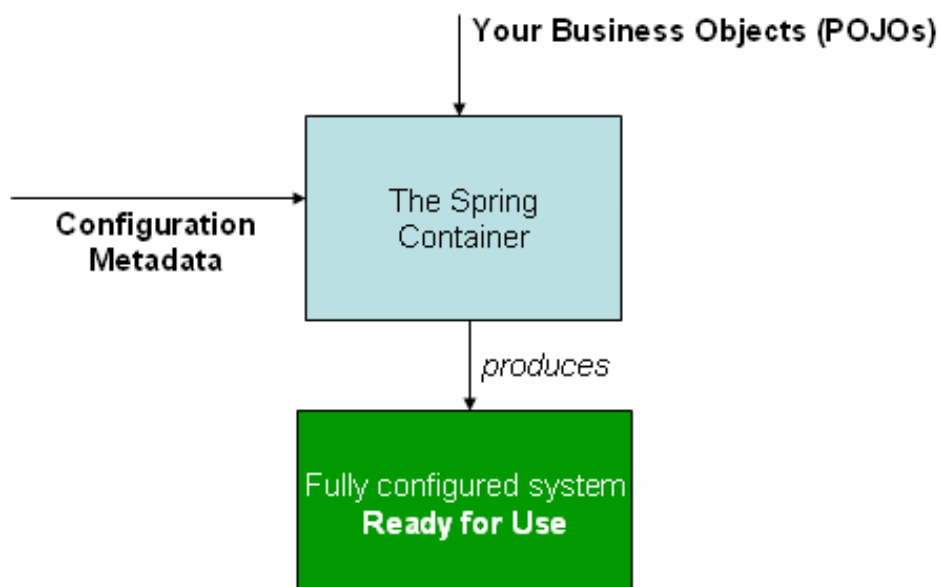
什么是 Spring Bean?

简单来说，Bean 代指的就是那些被 IoC 容器所管理的对象。

我们需要告诉 IoC 容器帮助我们管理哪些对象，这个是通过配置元数据来定义的。配置元数据可以是 XML 文件、注解或者 Java 配置类。

```
<!-- Constructor-arg with 'value' attribute -->
<bean id="..." class="...">
    <constructor-arg value="..." />
</bean>
```

下图简单地展示了 IoC 容器如何使用配置元数据来管理对象。



`org.springframework.beans` 和 `org.springframework.context` 这两个包是 IoC 实现的基础，如果想要研究 IoC 相关的源码的话，可以去看看

将一个类声明为 Bean 的注解有哪些？

- `@Component`：通用的注解，可标注任意类为 `Spring` 组件。如果一个 Bean 不知道属于哪个层，可以使用 `@Component` 注解标注。
- `@Repository`：对应持久层即 Dao 层，主要用于数据库相关操作。
- `@Service`：对应服务层，主要涉及一些复杂的逻辑，需要用到 Dao 层。
- `@Controller`：对应 Spring MVC 控制层，主要用户接受用户请求并调用 Service 层返回数据给前端页面。

@Component 和 @Bean 的区别是什么？

- `@Component` 注解作用于类，而 `@Bean` 注解作用于方法。
- `@Component` 通常是通过类路径扫描来自动侦测以及自动装配到 Spring 容器中（我们可以使用 `@ComponentScan` 注解定义要扫描的路径从中找出标识了需要装配的类自动装配到 Spring 的 bean 容器中）。`@Bean` 注解通常是在标有该注解的方法中定义产生这个 bean，`@Bean` 告诉了 Spring 这是某个类的实例，当我需要用它的时候还给我。
- `@Bean` 注解比 `@Component` 注解的自定义性更强，而且很多地方我们只能通过 `@Bean` 注解来注册 bean。比如当我们引用第三方库中的类需要装配到 `Spring` 容器时，则只能通过 `@Bean` 来实现。

@Bean 注解使用示例：

```

@Configuration
public class AppConfig {

    @Bean

    public TransferService transferService() {

        return new TransferServiceImpl();

    }

}

```

上面的代码相当于下面的 xml 配置

```

<beans>

    <bean id="transferService" class="com.acme.TransferServiceImpl"/>

</beans>

```

下面这个例子是通过 `@Component` 无法实现的。

```

@Bean
public OneService getService(status) {

    case (status) {

        when 1:

            return new serviceImpl1();

        when 2:

            return new serviceImpl2();

        when 3:

            return new serviceImpl3();

    }

}

```

注入 Bean 的注解有哪些？

Spring 内置的 `@Autowired` 以及 JDK 内置的 `@Resource` 和 `@Inject` 都可以用于注入 Bean。

Annotaion	Package	Source
@Autowired	org.springframework.bean.factory	Spring 2.5+
@Resource	javax.annotation	Java JSR-250
@Inject	javax.inject	Java JSR-330

@Autowired 和 @Resource 使用的比较多一些。

@Autowired 和 @Resource 的区别是什么？

Autowired 属于 Spring 内置的注解，默认的注入方式为 byType （根据类型进行匹配），也就是说会优先根据接口类型去匹配并注入 Bean （接口的实现类）。

这会有什么问题呢？ 当一个接口存在多个实现类的话， byType 这种方式就无法正确注入对象了，因为这个时候 Spring 会同时找到多个满足条件的选择，默认情况下它自己不知道选择哪一个。

这种情况下，注入方式会变为 byName （根据名称进行匹配），这个名称通常就是类名（首字母小写）。就比如说下面代码中的 smsService 就是我这里所说的名称，这样应该比较好理解了吧。

```
// smsService 就是我们上面所说的名称
@Autowired
private SmsService smsService;
```

举个例子， SmsService 接口有两个实现类： SmsServiceImpl1 和 SmsServiceImpl2 ，且它们都已经被 Spring 容器所管理。

```

// 报错, byName 和 byType 都无法匹配到 bean
@Autowired
private SmsService smsService;
// 正确注入 SmsServiceImpl 对象对应的 bean
@Autowired
private SmsService smsServiceImpl;
// 正确注入 SmsServiceImpl 对象对应的 bean
// smsServiceImpl 就是我们上面所说的名称
@Autowired
@Qualifier(value = "smsServiceImpl")
private SmsService smsService;

```

我们还是建议通过 `@Qualifier` 注解来显示指定名称而不是依赖变量的名称。

`@Resource` 属于 JDK 提供的注解，默认注入方式为 `byName`。如果无法通过名称匹配到对应的 Bean 的话，注入方式会变为 `byType`。

`@Resource` 有两个比较重要且日常开发常用的属性：`name`（名称）、`type`（类型）。

```

public @interface Resource {
    String name() default "";
    Class<?> type() default Object.class;
}

```

如果仅指定 `name` 属性则注入方式为 `byName`，如果仅指定 `type` 属性则注入方式为 `byType`，如果同时指定 `name` 和 `type` 属性（不建议这么做）则注入方式为 `byType + byName`。

```
// 报错, byName 和 byType 都无法匹配到 bean
@Resource

private SmsService smsService;

// 正确注入 SmsServiceImpl 对象对应的 bean
@Resource

private SmsService smsServiceImpl;

// 正确注入 SmsServiceImpl 对象对应的 bean (比较推荐这种方式)
@Resource(name = "smsServiceImpl")

private SmsService smsService;
```

简单总结一下：

- `@Autowired` 是 Spring 提供的注解，`@Resource` 是 JDK 提供的注解。
- `Autowired` 默认的注入方式为 `byType`（根据类型进行匹配），`@Resource` 默认注入方式为 `byName`（根据名称进行匹配）。
- 当一个接口存在多个实现类的情况下，`@Autowired` 和 `@Resource` 都需要通过名称才能正确匹配到对应的 Bean。`Autowired` 可以通过 `@Qualifier` 注解来显示指定名称，`@Resource` 可以通过 `name` 属性来显示指定名称。

Bean 的作用域有哪些？

Spring 中 Bean 的作用域通常有下面几种：

- **singleton** : IoC 容器中只有唯一的 bean 实例。Spring 中的 bean 默认都是单例的，是对单例设计模式的应用。
- **prototype** : 每次获取都会创建一个新的 bean 实例。也就是说，连续 `getBean()` 两次，得到的是不同的 Bean 实例。
- **request**（仅 Web 应用可用）: 每一次 HTTP 请求都会产生一个新的 bean（请求 bean），该 bean 仅在当前 HTTP request 内有效。
- **session**（仅 Web 应用可用）: 每一次来自新 session 的 HTTP 请求都会产生一个新的 bean（会话 bean），该 bean 仅在当前 HTTP session 内有效。
- **application/global-session**（仅 Web 应用可用）: 每个 Web 应用在启动时创建一个 Bean（应用 Bean），该 bean 仅在当前应用启动时间内有效。
- **websocket**（仅 Web 应用可用）: 每一次 WebSocket 会话产生一个新的 bean。

如何配置 bean 的作用域呢？

xml 方式：

```
<bean id="..." class="..." scope="singleton"></bean>
```

注解方式：

```
@Bean
@Scope(value = ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public Person personPrototype() {
    return new Person();
}
```

单例 Bean 的线程安全问题了解吗？

大部分时候我们并没有在项目中使用了多线程，所以很少有人会关注这个问题。单例 Bean 存在线程问题，主要是因为当多个线程操作同一个对象的时候是存在资源竞争的。

常见的有两种解决办法：

1. 在 Bean 中尽量避免定义可变的成员变量。
2. 在类中定义一个 `ThreadLocal` 成员变量，将需要的可变成员变量保存在 `ThreadLocal` 中（推荐的一种方式）。

不过，大部分 Bean 实际都是无状态（没有实例变量）的（比如 Dao、Service），这种情况下，Bean 是线程安全的。

Bean 的生命周期了解么？

下面的内容整理自：<https://yemengying.com/2016/07/14/spring-bean-life-cycle/>，除了这篇文章，再推荐一篇很不错的文章：<https://www.cnblogs.com/zrtqsk/p/3735273.html>。

- Bean 容器找到配置文件中 Spring Bean 的定义。
- Bean 容器利用 Java Reflection API 创建一个 Bean 的实例。
- 如果涉及到一些属性值 利用 `set()` 方法设置一些属性值。
- 如果 Bean 实现了 `BeanNameAware` 接口，调用 `setBeanName()` 方法，传入 Bean 的名字。
- 如果 Bean 实现了 `BeanClassLoaderAware` 接口，调用 `setBeanClassLoader()` 方法，传入 `ClassLoader` 对象的实例。

- 如果 Bean 实现了 `BeanFactoryAware` 接口，调用 `setBeanFactory()` 方法，传入 `BeanFactory` 对象的实例。
- 与上面的类似，如果实现了其他 `*.Aware` 接口，就调用相应的方法。
- 如果有和加载这个 Bean 的 Spring 容器相关的 `BeanPostProcessor` 对象，执行 `postProcessBeforeInitialization()` 方法
- 如果 Bean 实现了 `InitializingBean` 接口，执行 `afterPropertiesSet()` 方法。
- 如果 Bean 在配置文件中的定义包含 `init-method` 属性，执行指定的方法。
- 如果有和加载这个 Bean 的 Spring 容器相关的 `BeanPostProcessor` 对象，执行 `postProcessAfterInitialization()` 方法
- 当要销毁 Bean 的时候，如果 Bean 实现了 `DisposableBean` 接口，执行 `destroy()` 方法。
- 当要销毁 Bean 的时候，如果 Bean 在配置文件中的定义包含 `destroy-method` 属性，执行指定的方法。

图示：

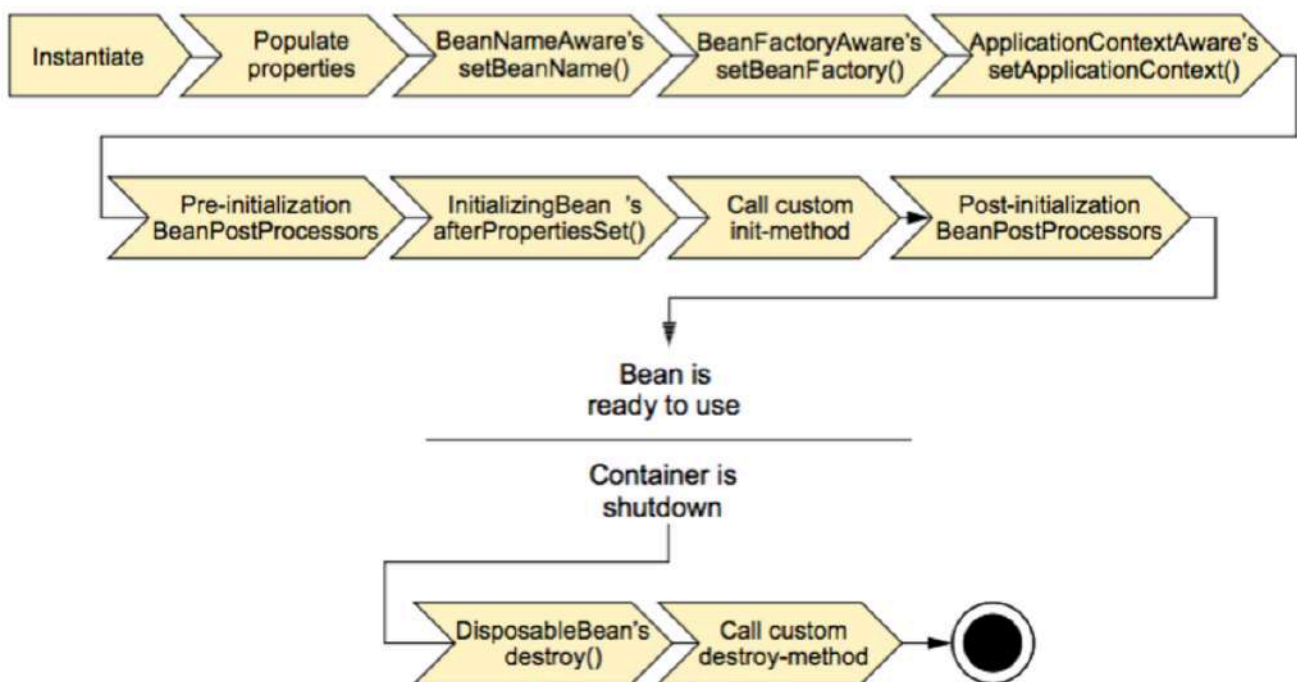
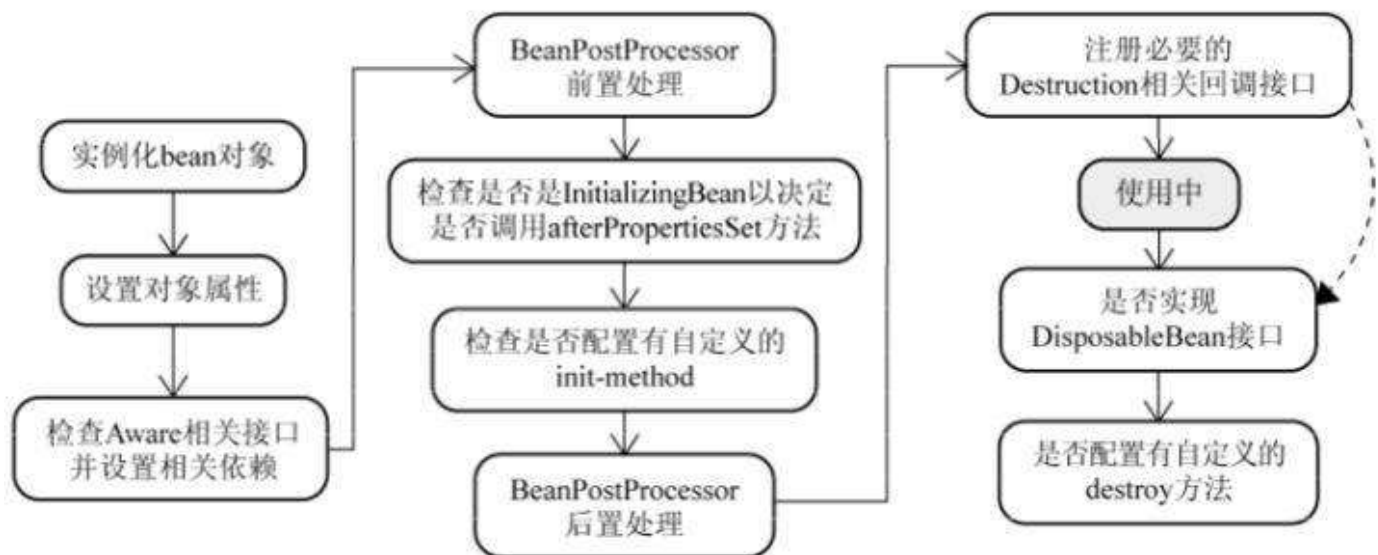


Figure 1.5 A bean goes through several steps between creation and destruction in the Spring container. Each step is an opportunity to customize how the bean is managed in Spring.

与之比较类似的中文版本：

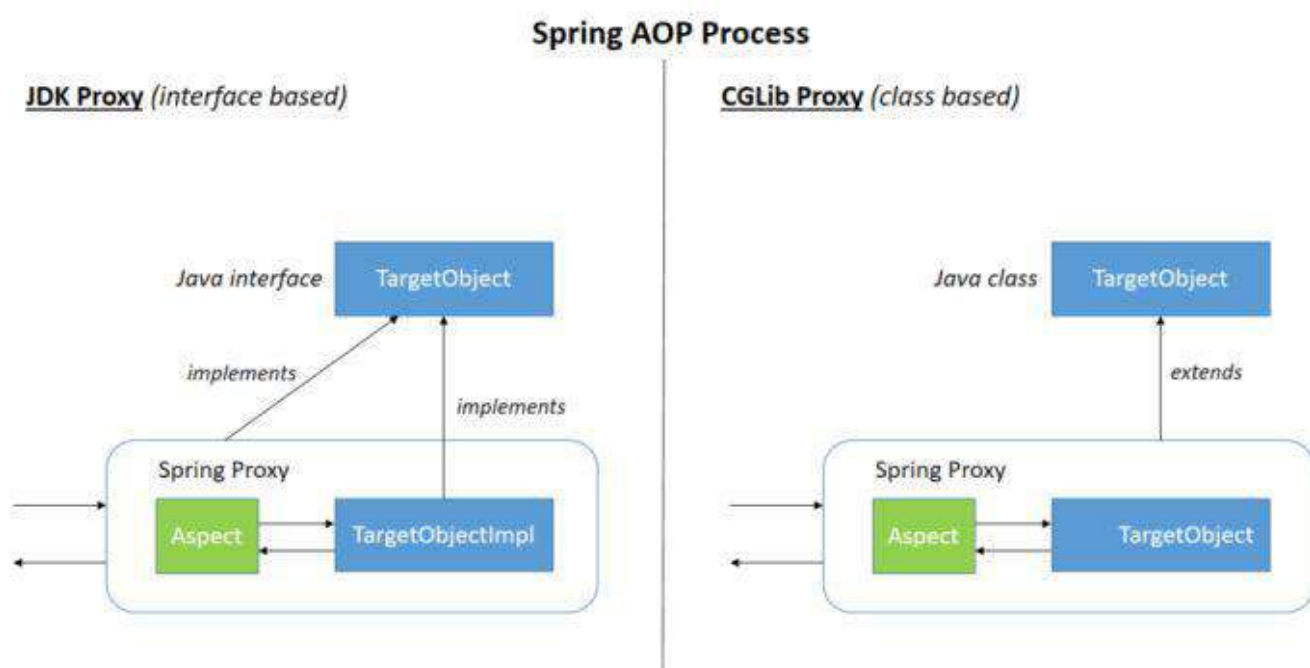


Spring AOP

谈谈自己对于 AOP 的了解

AOP(Aspect-Oriented Programming:面向切面编程)能够将那些与业务无关，却为业务模块所共同调用的逻辑或责任（例如事务处理、日志管理、权限控制等）封装起来，便于减少系统的重复代码，降低模块间的耦合度，并有利于未来的可拓展性和可维护性。

Spring AOP 就是基于动态代理的，如果要代理的对象，实现了某个接口，那么 Spring AOP 会使用 **JDK Proxy**，去创建代理对象，而对于没有实现接口的对象，就无法使用 JDK Proxy 去进行代理了，这时候 Spring AOP 会使用 **Cglib** 生成一个被代理对象的子类来作为代理，如下图所示：



当然你也可以使用 **AspectJ**！Spring AOP 已经集成了 AspectJ，AspectJ 应该算的上是 Java 生态系统最完整的 AOP 框架了。

AOP 切面编程设计到的一些专业术语：

术语	含义
目标(Target)	被通知的对象
代理(Proxy)	向目标对象应用通知之后创建的代理对象
连接点(JoinPoint)	目标对象的所属类中，定义的所有方法均为连接点
切入点(Pointcut)	被切面拦截 / 增强的连接点（切入点一定是连接点，连接点不一定是切入点）
通知(Advice)	增强的逻辑 / 代码，也即拦截到目标对象的连接点之后要做的事情
切面(Aspect)	切入点(Pointcut)+通知(Advice)
Weaving(织入)	将通知应用到目标对象，进而生成代理对象的过程动作

Spring AOP 和 AspectJ AOP 有什么区别？

Spring AOP 属于运行时增强，而 AspectJ 是编译时增强。 Spring AOP 基于代理(Proxying)，而 AspectJ 基于字节码操作(Bytecode Manipulation)。

Spring AOP 已经集成了 AspectJ，AspectJ 应该算的上是 Java 生态系统最完整的 AOP 框架了。AspectJ 相比于 Spring AOP 功能更加强大，但是 Spring AOP 相对来说更简单，

如果我们的切面比较少，那么两者性能差异不大。但是，当切面太多的话，最好选择 AspectJ，它比 Spring AOP 快很多。

AspectJ 定义的通知类型有哪些？

- **Before**（前置通知）：目标对象的方法调用之前触发
- **After**（后置通知）：目标对象的方法调用之后触发
- **AfterReturning**（返回通知）：目标对象的方法调用完成，在返回结果值之后触发
- **AfterThrowing**（异常通知）：目标对象的方法运行中抛出 / 触发异常后触发。AfterReturning 和 AfterThrowing 两者互斥。如果方法调用成功无异常，则会有返回值；如果方法抛出了异常，则不会有返回值。

- **Around**：（环绕通知）编程式控制目标对象的方法调用。环绕通知是所有通知类型中可操作范围最大的一种，因为它可以直接拿到目标对象，以及要执行的方法，所以环绕通知可以任意的在目标对象的方法调用前后搞事，甚至不调用目标对象的方法

多个切面的执行顺序如何控制？

1、通常使用 `@Order` 注解直接定义切面顺序

```
// 值越小优先级越高
@Order(3)
@Component
@Aspect

public class LoggingAspect implements Ordered {
```

2、实现 `Ordered` 接口重写 `getOrder` 方法。

```
@Component
@Aspect

public class LoggingAspect implements Ordered {

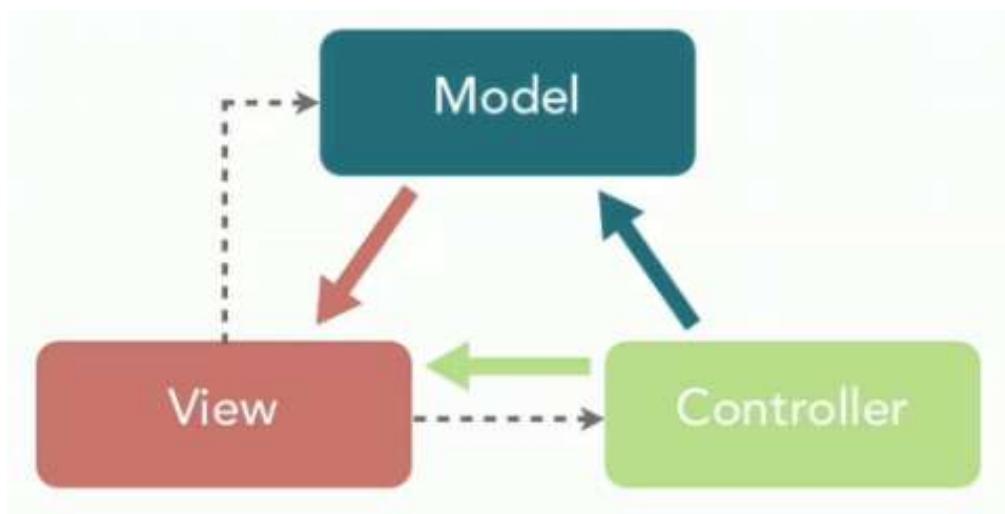
    // ....

    @Override
    public int getOrder() {
        // 返回值越小优先级越高
        return 1;
    }
}
```

Spring MVC

说说自己对于 Spring MVC 了解?

MVC 是模型(Model)、视图(View)、控制器(Controller)的简写，其核心思想是通过将业务逻辑、数据、显示分离来组织代码。



网上有很多人说 MVC 不是设计模式，只是软件设计规范，我个人更倾向于 MVC 同样是众多设计模式中的一种。[java-design-patterns](#) 项目中就有关于 MVC 的相关介绍。

iluwater / java-design-patterns

Watch

3.9k

Unstar

69.4k

Fork

21.6k

<> Code

Issues 228

Pull requests 38

Actions

Wiki

Security

Insights

master

java-design-patterns / model-view-controller /

Go to file

Add file

JackieNim fix: Fixed pages showing up in wrong language (#1752)

✓ f597fc1 on 20 May

History

..

etc

#1113 Add uml-reverse-mapper plugin

2 years ago

src

Use lombok, reformat, and optimize the code (#1560)

5 months ago

README.md

fix: Fixed pages showing up in wrong language (#1752)

1.67 KB

3 months ago

pom.xml

Set version for the next development iteration

2.43 KB

4 months ago

README.md

layout	title	folder	permalink	categories	language	tags
pattern	Model-View-Controller	model-view-controller	/patterns/model-view-controller/	Architectural	en	Decoupling

Intent

Separate the user interface into three interconnected components: the model, the view and the controller. Let the model manage the data, the view display the data and the controller mediate updating the data and redrawing the display.

Class diagram

```
classDiagram
    class GiantController {
        <<Java Class>>
        com.iluwatar
        GiantController(GiantModel, GiantView)
        getHealth() Health
        setHealth(Health) void
        getFatigue() Fatigue
        setFatigue(Fatigue) void
        getNourishment() Nourishment
        setNourishment(Nourishment) void
        updateView() void
    }
    class GiantView {
        <<Java Class>>
        com.iluwatar
        GiantView()
        displayGiant(GiantModel) void
    }
    class GiantModel {
        <<Java Class>>
        com.iluwatar
        health Health
        fatigue Fatigue
    }
    GiantController --> GiantView : -view 0..1
    GiantController --> GiantModel : -giant 0..1
```

The diagram shows three classes: GiantController, GiantView, and GiantModel. GiantController has methods for managing health, fatigue, and nourishment, and for updating the view. GiantView has a method for displaying the giant. GiantModel has attributes for health and fatigue. GiantController has a directed association to GiantView with role -view and multiplicity 0..1. GiantController has a directed association to GiantModel with role -giant and multiplicity 0..1.

https://blog.csdn.net/qq_34337272

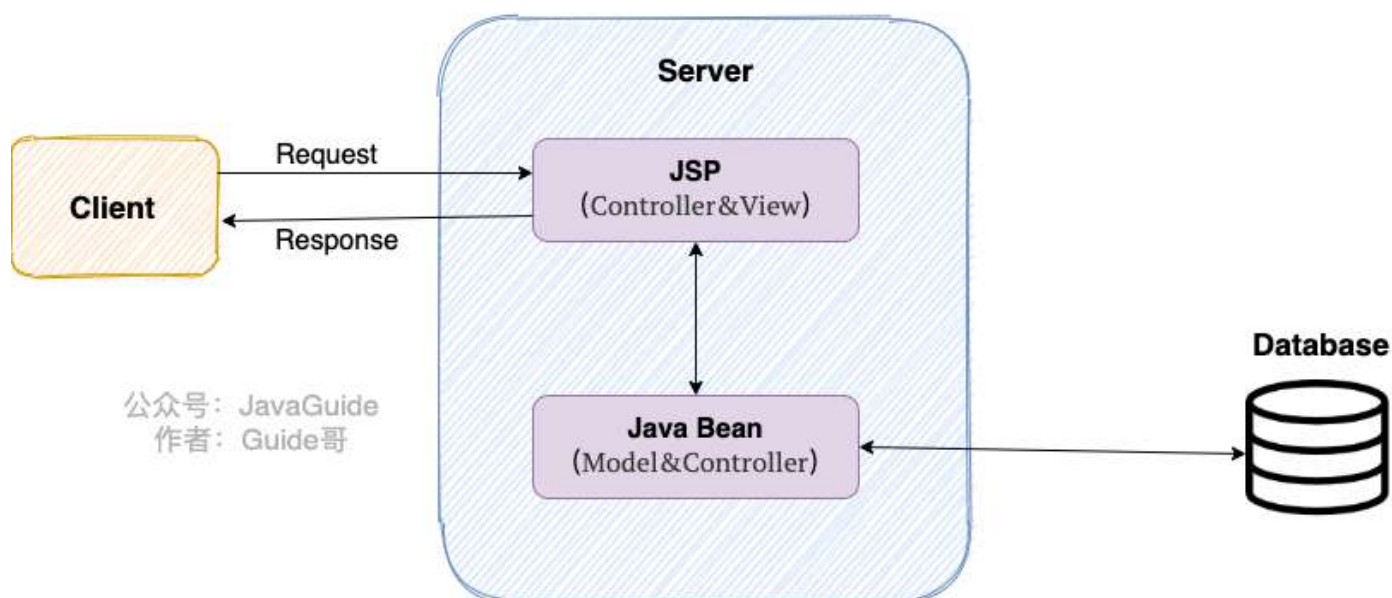
想要真正理解 Spring MVC，我们先来看看 Model 1 和 Model 2 这两个没有 Spring MVC 的时代。

Model 1 时代

很多学 Java 后端比较晚的朋友可能并没有接触过 Model 1 时代下的 JavaWeb 应用开发。在 Model1 模式下，整个 Web 应用几乎全部用 JSP 页面组成，只用少量的 JavaBean 来处理数据库连接、访问等操作。

这个模式下 JSP 即是控制层（Controller）又是表现层（View）。显而易见，这种模式存在很多问题。比如控制逻辑和表现逻辑混杂在一起，导致代码重用率极低；再比如前端和后端相互依赖，难以进行测试维护并且开发效率极低。

MVC - Model 1

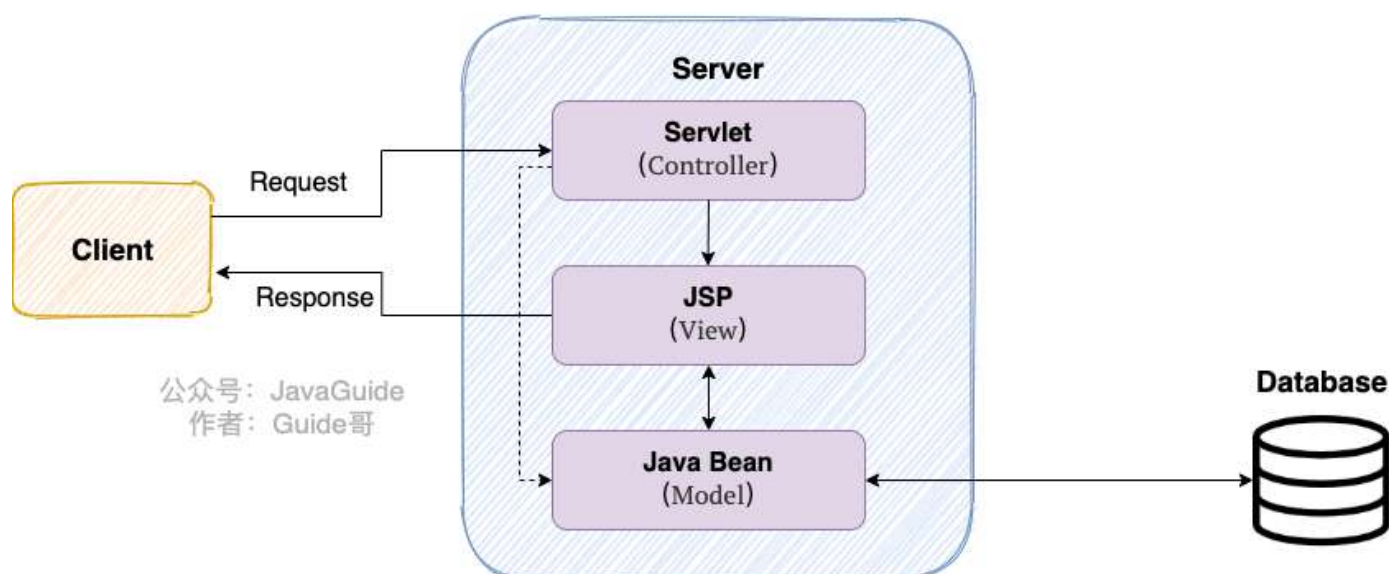


Model 2 时代

学过 Servlet 并做过相关 Demo 的朋友应该了解“Java Bean(Model)+ JSP (View) +Servlet (Controller)”这种开发模式，这就是早期的 JavaWeb MVC 开发模式。

- Model: 系统涉及的数据，也就是 dao 和 bean。
- View: 展示模型中的数据，只是用来展示。
- Controller: 处理用户请求都发送给，返回数据给 JSP 并展示给用户。

MVC - Model 2



Model2 模式下还存在很多问题，Model2 的抽象和封装程度还远远不够，使用 Model2 进行开发时不可避免地会重复造轮子，这就大大降低了程序的可维护性和复用性。

于是，很多 JavaWeb 开发相关的 MVC 框架应运而生比如 Struts2，但是 Struts2 比较笨重。

Spring MVC 时代

随着 Spring 轻量级开发框架的流行，Spring 生态圈出现了 Spring MVC 框架，Spring MVC 是当前最优秀的 MVC 框架。相比于 Struts2，Spring MVC 使用更加简单和方便，开发效率更高，并且 Spring MVC 运行速度更快。

MVC 是一种设计模式，Spring MVC 是一款很优秀的 MVC 框架。Spring MVC 可以帮助我们进行更简洁的 Web 层的开发，并且它天生与 Spring 框架集成。Spring MVC 下我们一般把后端项目分为 Service 层（处理业务）、Dao 层（数据库操作）、Entity 层（实体类）、Controller 层（控制层，返回数据给前台页面）。

Spring MVC 的核心组件有哪些？

记住了下面这些组件，也就记住了 SpringMVC 的工作原理。

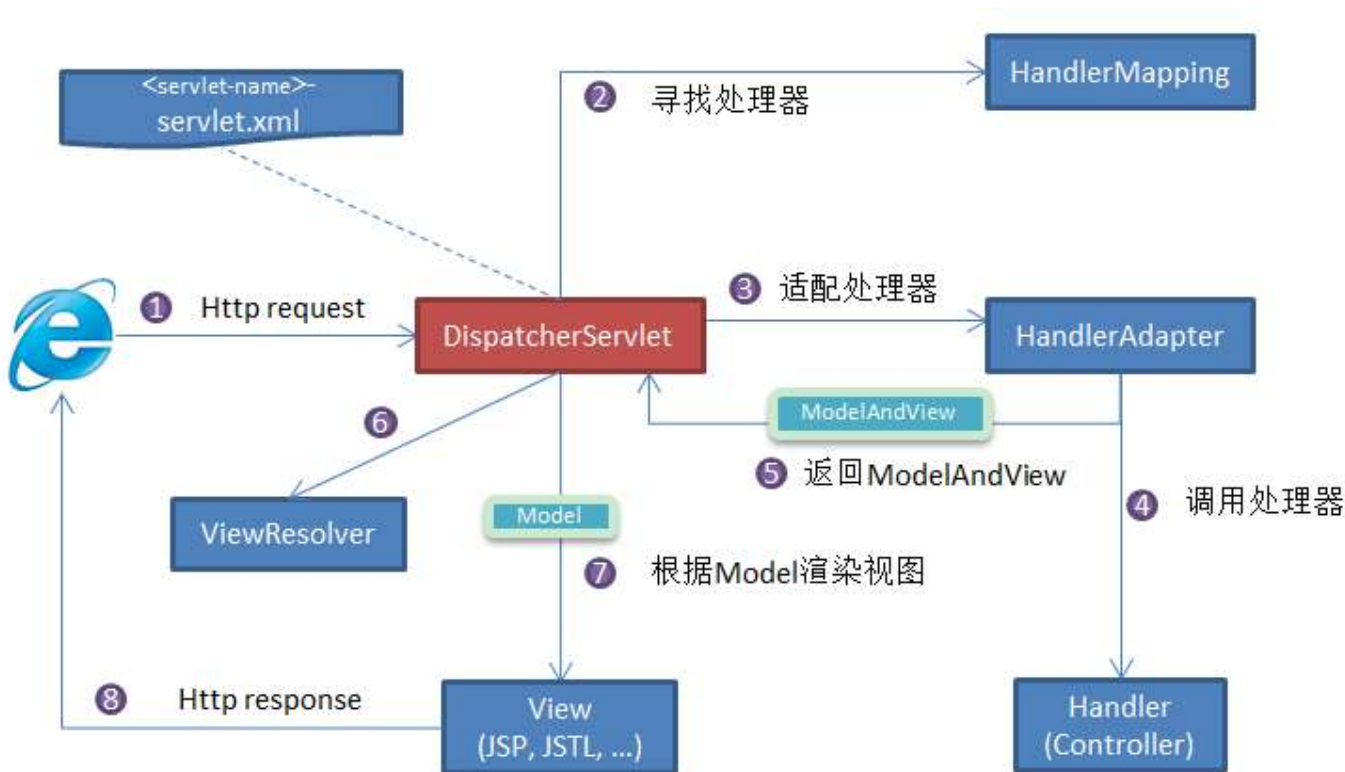
- **DispatcherServlet**：核心的中央处理器，负责接收请求、分发，并给予客户端响应。
- **HandlerMapping**：处理器映射器，根据 uri 去匹配查找能处理的 **Handler**，并会将请求涉及到的拦截器和 **Handler** 一起封装。
- **HandlerAdapter**：处理器适配器，根据 **HandlerMapping** 找到的 **Handler**，适配执行对应的 **Handler**；

- **Handler** ：请求处理器，处理实际请求的处理器。
- **ViewResolver** ：视图解析器，根据 **Handler** 返回的逻辑视图 / 视图，解析并渲染真正的视图，并传递给 **DispatcherServlet** 响应客户端

SpringMVC 工作原理了解吗？

Spring MVC 原理如下图所示：

SpringMVC 工作原理的图解我没有自己画，直接图省事在网上找了一个非常清晰直观的，原出处不明。



流程说明（重要）：

1. 客户端（浏览器）发送请求，**DispatcherServlet** 拦截请求。
2. **DispatcherServlet** 根据请求信息调用 **HandlerMapping**。**HandlerMapping** 根据 uri 去匹配查找能处理的 **Handler**（也就是我们平常说的 **Controller** 控制器），并会将请求涉及到的拦截器和 **Handler** 一起封装。
3. **DispatcherServlet** 调用 **HandlerAdapter** 适配执行 **Handler**。
4. **Handler** 完成对用户请求的处理后，会返回一个 **ModelAndView** 对象给 **DispatcherServlet**，**ModelAndView** 顾名思义，包含了数据模型以及相应的视图的信息。**Model** 是返回的数据对象，**View** 是个逻辑上的 **View**。
5. **ViewResolver** 会根据逻辑 **View** 查找实际的 **View**。

6. `DispatcherServlet` 把返回的 `Model` 传给 `View`（视图渲染）。
7. 把 `View` 返回给请求者（浏览器）

统一异常处理怎么做？

推荐使用注解的方式统一异常处理，具体会使用到 `@ControllerAdvice` + `@ExceptionHandler` 这两个注解。

```
@ControllerAdvice
@ResponseBody
public class GlobalExceptionHandler {

    @ExceptionHandler(BaseException.class)
    public ResponseEntity<?> handleAppException(BaseException ex,
HttpServletRequest request) {
        //.....
    }

    @ExceptionHandler(value = ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleResourceNotFoundException(ResourceNotFoundException ex, HttpServletRequest
request) {
        //.....
    }
}
```

这种异常处理方式下，会给所有或者指定的 `Controller` 织入异常处理的逻辑（AOP），当 `Controller` 中的方法抛出异常的时候，由被 `@ExceptionHandler` 注解修饰的方法进行处理。

`ExceptionHandlerMethodResolver` 中 `getMappedMethod` 方法决定了异常具体被哪个被 `@ExceptionHandler` 注解修饰的方法处理异常。

```
@Nullable
private Method getMappedMethod(Class<? extends Throwable> exceptionType) {
    List<Class<? extends Throwable>> matches = new ArrayList<>();
    //找到可以处理的所有异常信息。mappedMethods 中存放了异常和处理异常的方法的对应关系
```

```

        for (Class<? extends Throwable> mappedException :
this.mappedMethods.keySet()) {
            if (mappedException.isAssignableFrom(exceptionType)) {
                matches.add(mappedException);
            }
        }
        // 不为空说明有方法处理异常
        if (!matches.isEmpty()) {
            // 按照匹配程度从小到大排序
            matches.sort(new ExceptionDepthComparator(exceptionType));
            // 返回处理异常的方法
            return this.mappedMethods.get(matches.get(0));
        }
        else {
            return null;
        }
    }
}

```

从源代码看出：`getMappedMethod()` 会首先找到可以匹配处理异常的所有方法信息，然后对其进行从小到大的排序，最后取最小的那一个匹配的方法(即匹配度最高的那个)。

Spring 框架中用到了哪些设计模式？

关于下面这些设计模式的详细介绍，可以看我写的 [Spring 中的设计模式详解](#) 这篇文章。

- 工厂设计模式：Spring 使用工厂模式通过 `BeanFactory` 、 `ApplicationContext` 创建 bean 对象。
- 代理设计模式：Spring AOP 功能的实现。
- 单例设计模式：Spring 中的 Bean 默认都是单例的。
- 模板方法模式：Spring 中 `JdbcTemplate` 、 `hibernateTemplate` 等以 `Template` 结尾的对数据库操作的类，它们就使用到了模板模式。
- 包装器设计模式：我们的项目需要连接多个数据库，而且不同的客户在每次访问中根据需要会去访问不同的数据库。这种模式让我们可以根据客户的需求能够动态切换不同的数据源。
- 观察者模式：Spring 事件驱动模型就是观察者模式很经典的一个应用。
- 适配器模式：Spring AOP 的增强或通知(Advice)使用到了适配器模式、spring MVC 中也是用到了适配器模式适配 `Controller` 。
-

Spring 事务

关于 Spring 事务的详细介绍，可以看我写的 [Spring 事务详解](#) 这篇文章。

Spring 管理事务的方式有几种？

- **编程式事务**：在代码中硬编码(不推荐使用)：通过 `TransactionTemplate` 或者 `TransactionManager` 手动管理事务，实际应用中很少使用，但是对于你理解 Spring 事务管理原理有帮助。
- **声明式事务**：在 XML 配置文件中配置或者直接基于注解（推荐使用）：实际是通过 AOP 实现（基于 `@Transactional` 的全注解方式使用最多）

Spring 事务中哪几种事务传播行为？

事务传播行为是为了解决业务层方法之间互相调用的事务问题。

当事务方法被另一个事务方法调用时，必须指定事务应该如何传播。例如：方法可能继续在现有事务中运行，也可能开启一个新事务，并在自己的事务中运行。

正确的事务传播行为可能的值如下：

1. `TransactionDefinition.PROPROPAGATION_REQUIRED`

使用的最多的一个事务传播行为，我们平时经常使用的 `@Transactional` 注解默认使用就是这个事务传播行为。如果当前存在事务，则加入该事务；如果当前没有事务，则创建一个新的事务。

2. `TransactionDefinition.PROPROPAGATION_REQUIRES_NEW`

创建一个新的事务，如果当前存在事务，则把当前事务挂起。也就是说不管外部方法是否开启事务，`Propagation.REQUIRES_NEW` 修饰的内部方法会新开启自己的事务，且开启的事务相互独立，互不干扰。

3. `TransactionDefinition.PROPROPAGATION_NESTED`

如果当前存在事务，则创建一个事务作为当前事务的嵌套事务来运行；如果当前没有事务，则该取值等价于 `TransactionDefinition.PROPROPAGATION_REQUIRED`。

4. `TransactionDefinition.PROPROPAGATION_MANDATORY`

如果当前存在事务，则加入该事务；如果当前没有事务，则抛出异常。（mandatory：强制性）

这个使用的很少。

若是错误的配置以下 3 种事务传播行为，事务将不会发生回滚：

- **TransactionDefinition.PROPAGATION_SUPPORTS** : 如果当前存在事务，则加入该事务；如果当前没有事务，则以非事务的方式继续运行。
- **TransactionDefinition.PROPAGATION_NOT_SUPPORTED** : 以非事务方式运行，如果当前存在事务，则把当前事务挂起。
- **TransactionDefinition.PROPAGATION_NEVER** : 以非事务方式运行，如果当前存在事务，则抛出异常。

Spring 事务中的隔离级别有哪几种？

和事务传播行为这块一样，为了方便使用，Spring 也相应地定义了一个枚举类： `Isolation`

```
public enum Isolation {

    DEFAULT(TransactionDefinition.ISOLATION_DEFAULT),

    READ_UNCOMMITTED(TransactionDefinition.ISOLATION_READ_UNCOMMITTED),

    READ_COMMITTED(TransactionDefinition.ISOLATION_READ_COMMITTED),

    REPEATABLE_READ(TransactionDefinition.ISOLATION_REPEATABLE_READ),

    SERIALIZABLE(TransactionDefinition.ISOLATION_SERIALIZABLE);

    private final int value;

    Isolation(int value) {
        this.value = value;
    }

    public int value() {
        return this.value;
    }

}
```


下面我依次对每一种事务隔离级别进行介绍：

- `TransactionDefinition.ISOLATION_DEFAULT` :使用后端数据库默认的隔离级别，MySQL 默认采用的 `REPEATABLE_READ` 隔离级别 Oracle 默认采用的 `READ_COMMITTED` 隔离级别。
- `TransactionDefinition.ISOLATION_READ_UNCOMMITTED` :最低的隔离级别，使用这个隔离级别很少，因为它允许读取尚未提交的数据变更，可能会导致脏读、幻读或不可重复读
- `TransactionDefinition.ISOLATION_READ_COMMITTED` : 允许读取并发事务已经提交的数据，可以阻止脏读，但是幻读或不可重复读仍有可能发生
- `TransactionDefinition.ISOLATION_REPEATABLE_READ` : 对同一字段的多次读取结果都是一致的，除非数据是被本身事务自己所修改，可以阻止脏读和不可重复读，但幻读仍有可能发生。
- `TransactionDefinition.ISOLATION_SERIALIZABLE` : 最高的隔离级别，完全服从 ACID 的隔离级别。所有的事务依次逐个执行，这样事务之间就完全不可能产生干扰，也就是说，该级别可以防止脏读、不可重复读以及幻读。但是这将严重影响程序的性能。通常情况下也不会用到该级别。

@Transactional(rollbackFor = Exception.class)注解了解吗？

`Exception` 分为运行时异常 `RuntimeException` 和非运行时异常。事务管理对于企业应用来说是至关重要的，即使出现异常情况，它也可以保证数据的一致性。

当 `@Transactional` 注解作用于类上时，该类的所有 `public` 方法将都具有该类型的事务属性，同时，我们也可以在方法级别使用该标注来覆盖类级别的定义。如果类或者方法加了这个注解，那么这个类里面的方法抛出异常，就会回滚，数据库里面的数据也会回滚。

在 `@Transactional` 注解中如果不配置 `rollbackFor` 属性,那么事务只会在遇到 `RuntimeException` 的时候才会回滚，加上 `rollbackFor=Exception.class` ,可以让事务在遇到非运行时异常时也回滚。

Spring Data JPA

JPA 重要的是实战，这里仅对小部分知识点进行总结。

如何使用 JPA 在数据库中非持久化一个字段？

假如我们有下面一个类：

```
@Entity(name="USER")
public class User {

    @Id
```



```

    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "ID")
    private Long id;

    @Column(name="USER_NAME")
    private String userName;

    @Column(name="PASSWORD")
    private String password;

    private String secret;

}

```

如果我们想让 `secret` 这个字段不被持久化，也就是不被数据库存储怎么办？我们可以采用下面几种方法：

```

static String transient1; // not persistent because of static
final String transient2 = "Satish"; // not persistent because of final
transient String transient3; // not persistent because of transient
@Transient
String transient4; // not persistent because of @Transient

```

一般使用后面两种方式比较多，我个人使用注解的方式比较多。

JPA 的审计功能是做什么的？有什么用？

审计功能主要是帮助我们记录数据库操作的具体行为比如某条记录是谁创建的、什么时间创建的、最后修改人是谁、最后修改时间是什么时候。

```

@Data
@AllArgsConstructor
@NoArgsConstructor
MappedSuperclass
@EntityListeners(value = AuditingEntityListener.class)
public abstract class AbstractAuditBase {

```

```

    @CreatedDate
    @Column(updatable = false)
    @JsonIgnore
    private Instant createdAt;

    @LastModifiedDate
    @JsonIgnore
    private Instant updatedAt;

    @CreatedBy
    @Column(updatable = false)
    @JsonIgnore
    private String createdBy;

    @LastModifiedBy
    @JsonIgnore
    private String updatedBy;
}

```

- `@CreatedDate` : 表示该字段为创建时间字段，在这个实体被 insert 的时候，会设置值
- `@CreatedBy` : 表示该字段为创建人，在这个实体被 insert 的时候，会设置值
- `@LastModifiedDate` 、 `@LastModifiedBy` 同理。

实体之间的关联关系注解有哪些？

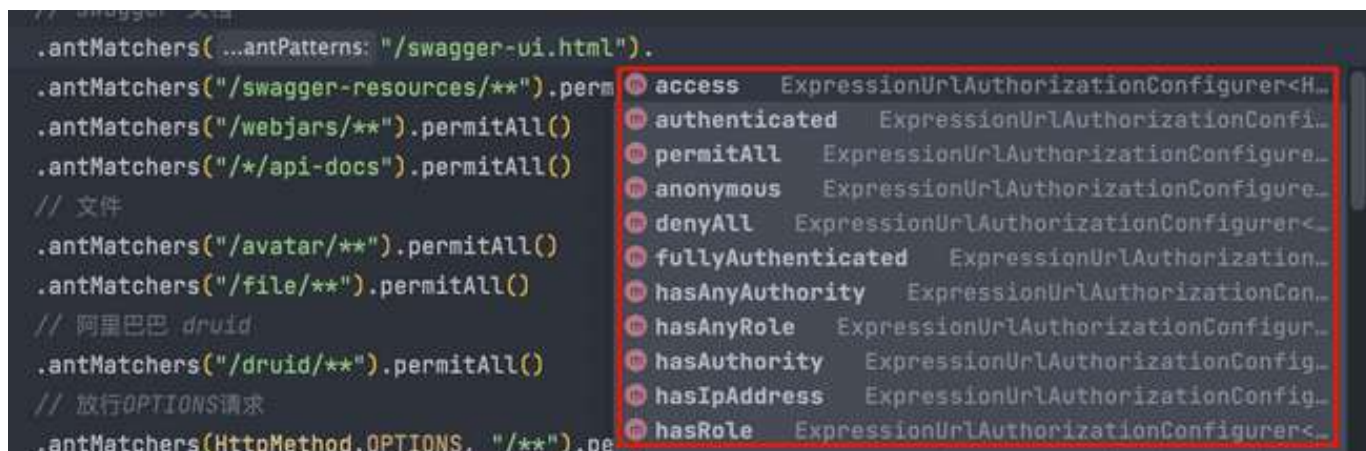
- `@OneToOne` : 一对一。
- `@ManyToMany` : 多对多。
- `@OneToMany` : 一对多。
- `@ManyToOne` : 多对一。

利用 `@ManyToOne` 和 `@OneToMany` 也可以表达多对多的关联关系。

Spring Security

Spring Security 重要的是实战，这里仅对小部分知识点进行总结。

有哪些控制请求访问权限的方法？



- `permitAll()` : 无条件允许任何形式访问，不管你登录还是没有登录。
- `anonymous()` : 允许匿名访问，也就是没有登录才可以访问。
- `denyAll()` : 无条件决绝任何形式的访问。
- `authenticated()` : 只允许已认证的用户访问。
- `fullyAuthenticated()` : 只允许已经登录或者通过 remember-me 登录的用户访问。
- `hasRole(String)` : 只允许指定的角色访问。
- `hasAnyRole(String)` : 指定一个或者多个角色，满足其一的用户即可访问。
- `hasAuthority(String)` : 只允许具有指定权限的用户访问
- `hasAnyAuthority(String)` : 指定一个或者多个权限，满足其一的用户即可访问。
- `hasIpAddress(String)` : 只允许指定 ip 的用户访问。

hasRole 和 hasAuthority 有区别吗？

可以看看松哥的这篇文章：[Spring Security 中的 hasRole 和 hasAuthority 有区别吗？](#)，介绍的比较详细。

如何对密码进行加密？

如果我们需要保存密码这类敏感数据到数据库的话，需要先加密再保存。

Spring Security 提供了多种加密算法的实现，开箱即用，非常方便。这些加密算法实现类的父类是 `PasswordEncoder`，如果你想要自己实现一个加密算法的话，也需要继承 `PasswordEncoder`。

`PasswordEncoder` 接口一共也就 3 个必须实现的方法。

```

public interface PasswordEncoder {
    // 加密也就是对原始密码进行编码
    String encode(CharSequence var1);
    // 比对原始密码和数据库中保存的密码
    boolean matches(CharSequence var1, String var2);
    // 判断加密密码是否需要再次进行加密，默认返回 false
    default boolean upgradeEncoding(String encodedPassword) {
        return false;
    }
}

```



官方推荐使用基于 bcrypt 强哈希函数的加密算法实现类。

如何优雅更换系统使用的加密算法？

如果我们在开发过程中，突然发现现有的加密算法无法满足我们的需求，需要更换成另外一个加密算法，这个时候应该怎么办呢？

推荐的做法是通过 `DelegatingPasswordEncoder` 兼容多种不同的密码加密方案，以适应不同的业务需求。

从名字也能看出来，`DelegatingPasswordEncoder` 其实就是一个代理类，并非是一种全新的加密算法，它做的事情就是代理上面提到的加密算法实现类。在 Spring Security 5.0 之后，默认就是基于 `DelegatingPasswordEncoder` 进行密码加密的。

5.2 SpringBoot（付费）

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

Spring Boot 相关的面试题为我的[知识星球](#)（[点击链接即可查看详细介绍以及加入方法](#)）专属内容，已经整理到了《[Java 面试指北](#)》（[点击链接即可查看详细介绍以及获取方法](#)）中。

《[Java 面试指北](#)》的部分内容展示如下，你可以将其看作是 **JavaGuide** 的补充完善，两者可以配合使用。

介绍	02-24 18:03
▼ 面试准备篇	+
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java 优质面试视频汇总	02-11 12:28
Java 优质开源面试资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14

《Java 面试指北》只是星球内部众多资料中的一个, 星球还有很多其他优质资料比如[专属专栏](#)、Java 编程视频、PDF 资料。

星球专属的 7 个小册：

- 1、《Java面试指北》(配合 JavaGuide 食用)：🔗 输入密码 · 语雀 (密码：)
- 2、《Java 必读源码系列》(目前已经整理了 Dubbo 2.6.x、Netty 4.x、SpringBoot2.1 的源码)：🔗 输入密码 · 语雀 (密码：)
- 3、《从零开始写一个RPC框架》：🔗 输入密码 · 语雀 (密码：) RPC 框架 Github地址：🔗 GitHub – Snailclimb/guide-rpc-framework: A custom ...。
- 4、《分布式、高并发、Devops知识扫盲》：已经并入《Java面试指北》中。
- 5、《Kafka常见面试题/知识点总结》：🔗 输入密码 · 语雀 (密码：)
- 6、《程序员副业赚钱之路》🔗 输入密码 · 语雀 (密码：)
- 7、《Guide的读书笔记与文章精选集》(经典书籍精读笔记分享) 🔗 输入密码 · 语雀 (密码：)

#球友必看#

球友必看



Guide哥：勿传播，星球专属。晚上尽量抽时间更新🤗

最近几年，市面上有越来越多的“技术大佬”开始办培训班/训练营，动辄成千上万的学费，却并没有什么干货，单纯的就是割韭菜。

为了帮助更多同学准备 Java 面试以及学习 Java，我创建了一个纯粹的[知识星球](#)。虽然收费只有培训班/训练营的百分之一，但是[知识星球](#)里的内容质量更高，提供的服务也更全面。

欢迎准备 Java 面试以及学习 Java 的同学加入我的[知识星球](#)，干货非常多，学习氛围非常好！收费虽然是白菜价，但星球里的内容或许比你参加上万的培训班质量还要高。

JavaGuide&Java面试交流圈

星主：Guide哥



1.5万
成员数量

9800+
内容数量

1024
运营天数

最优质的 Java 面试交流星球，多次被知识星球官方推荐。门票即将上调到 199 元，IOS 用户请在公众号“JavaGuide”回复“知识星球”加入/续费。

...

知识星球

微信扫码加入星球 ▶



下面是星球提供的部分服务（点击下方图片即可获取知识星球的详细介绍）：

我的知识星球能为你提供什么？

努力做一个最优质的 Java 面试交流星球！加入到我的星球之后，你将获得：

1. 6 个高质量的专栏永久阅读，内容涵盖面试，源码解析，项目实战等内容！价值远超门票！
2. 多本原创 PDF 版本面试手册。
3. 免费的简历修改服务（已经累计帮助 4000+ 位球友修改简历）。
4. 一对一免费提问交流（专属建议，走心回答）。
5. 专属求职指南和建议，帮助你逆袭大厂！
6. 海量 Java 优质面试资源分享！价值远超门票！
7. 读书交流，学习交流，让我们共同努力创建一个纯粹的学习交流社区。
8. 不定期福利：节日抽奖、送书送课、球友线下聚会等等。
9.

其中的任何一项服务单独拎出来价值都远超星球门票了。

我有自己的原则，不割韭菜，用心做内容，真心希望帮助到你！

如果你感兴趣的话，不妨花 3 分钟左右看看星球的详细介绍：[JavaGuide 知识星球详细介绍](#)（文末有优惠券）。



5.3 MyBatis

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

#{} 和 \${} 的区别是什么？

注：这道题是面试官面试我同事的。

答：

- `${}` 是 Properties 文件中的变量占位符，它可以用于标签属性值和 sql 内部，属于静态文本替换，比如 `${driver}` 会被静态替换为 `com.mysql.jdbc.Driver`。
- `#{}` 是 sql 的参数占位符，MyBatis 会将 sql 中的 `#{}` 替换为 `?` 号，在 sql 执行前会使用 `PreparedStatement` 的参数设置方法，按序给 sql 的 `?` 号占位符设置参数值，比如 `ps.setInt(0, parameterValue)`，`#{}item.name` 的取值方式为使用反射从参数对象中获取 item 对象的 name 属性值，相当于 `param.getItem().getName()`。

xml 映射文件中，除了常见的 select、insert、update、delete 标签之外，还有哪些标签？

注：这道题是京东面试官面试我时间的。

答：还有很多其他的标签，`<resultMap>`、`<parameterMap>`、`<sql>`、`<include>`、`<selectKey>`，加上动态 sql 的 9 个标签，`trim``where``set``foreach``if``choose``when``otherwise``bind` 等，其中 `<sql>` 为 sql 片段标签，通过 `<include>` 标签引入 sql 片段，`<selectKey>` 为不支持自增的主键生成策略标签。

Dao 接口的工作原理是什么？Dao 接口里的方法，参数不同时，方法能重载吗？

注：这道题也是京东面试官面试我被问的。

答：最佳实践中，通常一个 xml 映射文件，都会写一个 Dao 接口与之对应。Dao 接口就是人们常说的 `Mapper` 接口，接口的全限名，就是映射文件中的 namespace 的值，接口的方法名，就是映射文件中 `MappedStatement` 的 id 值，接口方法内的参数，就是传递给 sql 的参数。`Mapper` 接口是没有实现类的，当调用接口方法时，接口全限名+方法名拼接字符串作为 key 值，可唯一定位一个 `MappedStatement`，举例：`com.mybatis3.mappers.StudentDao.findStudentById`，可以唯一找到 namespace 为 `com.mybatis3.mappers.StudentDao` 下面 `id = findStudentById` 的 `MappedStatement`。在 MyBatis 中，每一个 `<select>`、`<insert>`、`<update>`、`<delete>` 标签，都会被解析为一个 `MappedStatement` 对象。

Dao 接口里的方法，是不能重载的，因为是全限名+方法名的保存和寻找策略。

Dao 接口里的方法可以重载，但是 Mybatis 的 xml 里面的 ID 不允许重复。

Mybatis 版本 3.3.0，亲测如下：

```
/**
 * Mapper接口里面方法重载
 */
public interface StuMapper {

    List<Student> getAllStu();

    List<Student> getAllStu(@Param("id") Integer id);
}
```

然后在 `StuMapper.xml` 中利用 Mybatis 的动态 sql 就可以实现。

```
<select id="getAllStu" resultType="com.pojo.Student">
    select * from student
    <where>
        <if test="id != null">
            id = #{id}
        </if>
    </where>
</select>
```

能正常运行，并能得到相应的结果，这样就实现了在 Dao 接口中写重载方法。

Mybatis 的 Dao 接口可以有多个重载方法，但是多个接口对应的映射必须只有一个，否则启动会报错。

相关 issue：[更正：Dao 接口里的方法可以重载，但是 Mybatis 的 xml 里面的 ID 不允许重复！](#)。

Dao 接口的工作原理是 JDK 动态代理，MyBatis 运行时会使用 JDK 动态代理为 Dao 接口生成代理 proxy 对象，代理对象 proxy 会拦截接口方法，转而执行 `MappedStatement` 所代表的 sql，然后将 sql 执行结果返回。

补充：

Dao 接口方法可以重载，但是需要满足以下条件：

1. 仅有一个无参方法和一个有参方法
2. 多个有参方法时，参数数量必须一致。且使用相同的 `@Param`，或者使用 `param1` 这种

测试如下：

PersonDao.java

```
Person queryById();

Person queryById(@Param("id") Long id);

Person queryById(@Param("id") Long id, @Param("name") String name);
```

PersonMapper.xml

```
<select id="queryById" resultMap="PersonMap">
    select
        id, name, age, address
    from person
    <where>
        <if test="id != null">
            id = #{id}
        </if>
        <if test="name != null and name != ''">
            name = #{name}
        </if>
    </where>
    limit 1
</select>
```

`org.apache.ibatis.scripting.xmltags. DynamicContext. ContextAccessor#getProperty` 方法用于获取 `<if>` 标签中的条件值

```

public Object getProperty(Map context, Object target, Object name) {
    Map map = (Map) target;

    Object result = map.get(name);
    if (map.containsKey(name) || result != null) {
        return result;
    }

    Object parameterObject = map.get(PARAMETER_OBJECT_KEY);
    if (parameterObject instanceof Map) {
        return ((Map)parameterObject).get(name);
    }

    return null;
}

```

parameterObject 为 map，存放的是 Dao 接口中参数相关信息。

((Map)parameterObject).get(name) 方法如下

```

public V get(Object key) {
    if (!super.containsKey(key)) {
        throw new BindingException("Parameter '" + key + "' not found. Available parameters are " + keySet());
    }
    return super.get(key);
}

```

1. queryById() 方法执行时，parameterObject 为 null，getProperty 方法返回 null 值，<if> 标签获取的所有条件值都为 null，所有条件不成立，动态 sql 可以正常执行。
2. queryById(1L) 方法执行时，parameterObject 为 map，包含了 id 和 param1 两个 key 值。当获取 <if> 标签中 name 的属性值时，进入 ((Map)parameterObject).get(name) 方法中，map 中 key 不包含 name，所以抛出异常。
3. queryById(1L,"1") 方法执行时，parameterObject 中包含 id，param1，name，param2 四个 key 值，id 和 name 属性都可以获取到，动态 sql 正常执行。

MyBatis 是如何进行分页的？分页插件的原理是什么？

注：我出的。

答：(1) MyBatis 使用 RowBounds 对象进行分页，它是针对 ResultSet 结果集执行的内存分页，而非物理分页；(2) 可以在 sql 内直接书写带有物理分页的参数来完成物理分页功能，(3) 也可以使用分页插件来完成物理分页。

分页插件的基本原理是使用 MyBatis 提供的插件接口，实现自定义插件，在插件的拦截方法内拦截待执行的 sql，然后重写 sql，根据 dialect 方言，添加对应的物理分页语句和物理分页参数。

举例： `select _ from student` ，拦截 sql 后重写为： `select t._ from (select \* from student) t limit 0, 10`

简述 MyBatis 的插件运行原理，以及如何编写一个插件。

注：我出的。

答：MyBatis 仅可以编写针对 `ParameterHandler` 、 `ResultSetHandler` 、 `StatementHandler` 、 `Executor` 这 4 种接口的插件，MyBatis 使用 JDK 的动态代理，为需要拦截的接口生成代理对象以实现接口方法拦截功能，每当执行这 4 种接口对象的方法时，就会进入拦截方法，具体就是 `InvocationHandler` 的 `invoke()` 方法，当然，只会拦截那些你指定需要拦截的方法。

实现 MyBatis 的 `Interceptor` 接口并复写 `intercept()` 方法，然后在给插件编写注解，指定要拦截哪一个接口的哪些方法即可，记住，别忘了在配置文件中配置你编写的插件。

MyBatis 执行批量插入，能返回数据库主键列表吗？

注：我出的。

答：能，JDBC 都能，MyBatis 当然也能。

MyBatis 动态 sql 是做什么的？都有哪些动态 sql？能简述一下动态 sql 的执行原理不？

注：我出的。

答：MyBatis 动态 sql 可以让我们在 xml 映射文件内，以标签的形式编写动态 sql，完成逻辑判断和动态拼接 sql 的功能。其执行原理为，使用 OGNL 从 sql 参数对象中计算表达式的值，根据表达式的值动态拼接 sql，以此来完成动态 sql 的功能。

MyBatis 提供了 9 种动态 sql 标签：

- `<if></if>`
- `<where></where>`(trim,set)
- `<choose></choose>` (when, otherwise)
- `<foreach></foreach>`
- `<bind/>`

关于 MyBatis 动态 SQL 的详细介绍，请看这篇文章：[Mybatis 系列全解（八）：Mybatis 的 9 大动态 SQL 标签你知道几个？](#)。

关于这些动态 SQL 的具体使用方法，请看这篇文章：[Mybatis【13】-- Mybatis 动态 sql 标签怎么用？](#)

MyBatis 是如何将 sql 执行结果封装为目标对象并返回的？都有哪些映射形式？

注：我出的。

答：第一种是使用 `<resultMap>` 标签，逐一定义列名和对象属性名之间的映射关系。第二种是使用 sql 列的别名功能，将列别名书写为对象属性名，比如 `T_NAME AS NAME`，对象属性名一般是 name，小写，但是列名不区分大小写，MyBatis 会忽略列名大小写，智能找到与之对应对象属性名，你甚至可以写成 `T_NAME AS NaMe`，MyBatis 一样可以正常工作。

有了列名与属性名的映射关系后，MyBatis 通过反射创建对象，同时使用反射给对象的属性逐一赋值并返回，那些找不到映射关系的属性，是无法完成赋值的。

MyBatis 能执行一对一、一对多的关联查询吗？都有哪些实现方式，以及它们之间的区别。

注：我出的。

答：能，MyBatis 不仅可以执行一对一、一对多的关联查询，还可以执行多对一，多对多的关联查询，多对一查询，其实就是一对一查询，只需要把 `selectOne()` 修改为 `selectList()` 即可；多对多查询，其实就是一对多查询，只需要把 `selectOne()` 修改为 `selectList()` 即可。

关联对象查询，有两种实现方式，一种是单独发送一个 sql 去查询关联对象，赋给主对象，然后返回主对象。另一种是使用嵌套查询，嵌套查询的含义为使用 join 查询，一部分列是 A 对象的属性值，另外一部分列是关联对象 B 的属性值，好处是只发一个 sql 查询，就可以把主对象和其关联对象查出来。

那么问题来了，join 查询出来 100 条记录，如何确定主对象是 5 个，而不是 100 个？其去重复的原理是 `<resultMap>` 标签内的 `<id>` 子标签，指定了唯一确定一条记录的 id 列，MyBatis 根据 `<id>` 列值来完成 100 条记录的去重复功能，`<id>` 可以有多个，代表了联合主键的语意。

同样主对象的关联对象，也是根据这个原理去重复的，尽管一般情况下，只有主对象会有重复记录，关联对象一般不会重复。

举例：下面 join 查询出来 6 条记录，一、二列是 Teacher 对象列，第三列为 Student 对象列，MyBatis 去重复处理后，结果为 1 个老师 6 个学生，而不是 6 个老师 6 个学生。

t_id	t_name	s_id
1	teacher	38
1	teacher	39
1	teacher	40
1	teacher	41
1	teacher	42
1	teacher	43

MyBatis 是否支持延迟加载？如果支持，它的实现原理是什么？

注：我出的。

答：MyBatis 仅支持 association 关联对象和 collection 关联集合对象的延迟加载，association 指的就是一对一，collection 指的就是一对多查询。在 MyBatis 配置文件中，可以配置是否启用延迟加载 `lazyLoadingEnabled=true/false`。

它的原理是，使用 `CGLIB` 创建目标对象的代理对象，当调用目标方法时，进入拦截器方法，比如调用 `a.getB().getName()`，拦截器 `invoke()` 方法发现 `a.getB()` 是 `null` 值，那么就会单独发送事先保存好的查询关联 B 对象的 sql，把 B 查询上来，然后调用 `a.setB(b)`，于是 a 的对象 b 属性就有值了，接着完成 `a.getB().getName()` 方法的调用。这就是延迟加载的基本原理。

当然了，不光是 MyBatis，几乎所有的包括 Hibernate，支持延迟加载的原理都是一样的。

MyBatis 的 xml 映射文件中，不同的 xml 映射文件，id 是否可以重复？

注：我出的。

答：不同的 xml 映射文件，如果配置了 namespace，那么 id 可以重复；如果没有配置 namespace，那么 id 不能重复；毕竟 namespace 不是必须的，只是最佳实践而已。

原因就是 namespace+id 是作为 `Map<String, MappedStatement>` 的 key 使用的，如果没有 namespace，就剩下 id，那么，id 重复会导致数据互相覆盖。有了 namespace，自然 id 就可以重复，namespace 不同，namespace+id 自然也就不同。

MyBatis 中如何执行批处理？

注：我出的。

答：使用 `BatchExecutor` 完成批处理。

MyBatis 都有哪些 Executor 执行器？它们之间的区别是什么？

注：我出的

答：MyBatis 有三种基本的 `Executor` 执行器：

- **SimpleExecutor**：每执行一次 update 或 select，就开启一个 Statement 对象，用完立刻关闭 Statement 对象。
- **ReuseExecutor**：执行 update 或 select，以 sql 作为 key 查找 Statement 对象，存在就使用，不存在就创建，用完后，不关闭 Statement 对象，而是放置于 `Map<String, Statement>` 内，供下一次使用。简言之，就是重复使用 Statement 对象。

BatchExecutor 执行 update（没有 select，JDBC 批处理不支持 select），将所有 sql 都添加到批处理中（`addBatch()`），等待统一执行（`executeBatch()`），它缓存了多个 Statement 对象，每个 Statement 对象都是 `addBatch()` 完毕后，等待逐一执行 `executeBatch()` 批处理。与

JDBC 批处理相同。

作用范围：`Executor` 的这些特点，都严格限制在 `SqlSession` 生命周期范围内。

MyBatis 中如何指定使用哪一种 `Executor` 执行器？

注：我出的

答：在 MyBatis 配置文件中，可以指定默认的 `ExecutorType` 执行器类型，也可以手动给 `DefaultSqlSessionFactory` 的创建 `SqlSession` 的方法传递 `ExecutorType` 类型参数。

MyBatis 是否可以映射 Enum 枚举类？

注：我出的

答：MyBatis 可以映射枚举类，不单可以映射枚举类，MyBatis 可以映射任何对象到表的一列上。映射方式为自定义一个 `TypeHandler`，实现 `TypeHandler` 的 `setParameter()` 和 `getResult()` 接口方法。`TypeHandler` 有两个作用：

- 一是完成从 `javaType` 至 `jdbcType` 的转换；
- 二是完成 `jdbcType` 至 `javaType` 的转换，体现为 `setParameter()` 和 `getResult()` 两个方法，分别代表设置 sql 问号占位符参数和获取列查询结果。

MyBatis 映射文件中，如果 A 标签通过 `include` 引用了 B 标签的内容，请问，B 标签能否定义在 A 标签的后面，还是说必须定义在 A 标签的前面？

注：我出的

答：虽然 MyBatis 解析 xml 映射文件是按照顺序解析的，但是，被引用的 B 标签依然可以定义在任何地方，MyBatis 都可以正确识别。

原理是，MyBatis 解析 A 标签，发现 A 标签引用了 B 标签，但是 B 标签尚未解析到，尚不存在，此时，MyBatis 会将 A 标签标记为未解析状态，然后继续解析余下的标签，包含 B 标签，待所有标签解析完毕，MyBatis 会重新解析那些被标记为未解析的标签，此时再解析 A 标签时，B 标签已经存在，A 标签也就可以正常解析完成了。

简述 MyBatis 的 xml 映射文件和 MyBatis 内部数据结构之间的映射关系？

注：我出的

答：MyBatis 将所有 xml 配置信息都封装到 All-In-One 重量级对象 Configuration 内部。在 xml 映射文件中，`<parameterMap>` 标签会被解析为 `ParameterMap` 对象，其每个子元素会被解析为 `ParameterMapping` 对象。`<resultMap>` 标签会被解析为 `ResultMap` 对象，其每个子元素会被解析为 `ResultMapping` 对象。每一个 `<select>`、`<insert>`、`<update>`、`<delete>` 标签均会被解析为 `MappedStatement` 对象，标签内的 sql 会被解析为 `BoundSql` 对象。

为什么说 MyBatis 是半自动 ORM 映射工具？它与全自动的区别在哪里？

注：我出的

答：Hibernate 属于全自动 ORM 映射工具，使用 Hibernate 查询关联对象或者关联集合对象时，可以根据对象关系模型直接获取，所以它是全自动的。而 MyBatis 在查询关联对象或关联集合对象时，需要手动编写 sql 来完成，所以，称之为半自动 ORM 映射工具。

面试题看似都很简单，但是想要能正确回答上来，必定是研究过源码且深入的人，而不是仅会使用的人或者用的很熟的人，以上所有面试题及其答案所涉及的内容，在我的 MyBatis 系列博客中都有详细讲解和原理分析。

5.4 Netty

Netty 相关的面试题为我的[知识星球](#)（[点击链接即可查看详细介绍以及加入方法](#)）专属内容，已经整理到了《[Java 面试指北](#)》（[点击链接即可查看详细介绍以及获取方法](#)）中。

《[Java 面试指北](#)》的部分内容展示如下，你可以将其看作是 [JavaGuide](#) 的补充完善，两者可以配合使用。

介绍	02-24 18:03
▼ 面试准备篇	+
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java 优质面试视频汇总	02-11 12:28
Java 优质开源面试资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14

《Java 面试指北》只是星球内部众多资料中的一个, 星球还有很多其他优质资料比如[专属专栏](#)、Java 编程视频、PDF 资料。

星球专属的 7 个小册：

- 1、《Java面试指北》(配合 JavaGuide 食用)：🔗 输入密码 · 语雀 (密码：)
- 2、《Java 必读源码系列》(目前已经整理了 Dubbo 2.6.x、Netty 4.x、SpringBoot2.1 的源码)：🔗 输入密码 · 语雀 (密码：)
- 3、《从零开始写一个RPC框架》：🔗 输入密码 · 语雀 (密码：) RPC 框架 Github地址：🔗 GitHub – Snailclimb/guide-rpc-framework: A custom ...
- 4、《分布式、高并发、Devops知识扫盲》：已经并入《Java面试指北》中。
- 5、《Kafka常见面试题/知识点总结》：🔗 输入密码 · 语雀 (密码：)
- 6、《程序员副业赚钱之路》🔗 输入密码 · 语雀 (密码：)
- 7、《Guide的读书笔记与文章精选集》(经典书籍精读笔记分享) 🔗 输入密码 · 语雀 (密码：)

#球友必看#

球友必看



等85人赞过

Guide哥：勿传播，星球专属。晚上尽量抽时间更新🤗

最近几年，市面上有越来越多的“技术大佬”开始办培训班/训练营，动辄成千上万的学费，却并没有什么干货，单纯的就是割韭菜。

为了帮助更多同学准备 Java 面试以及学习 Java，我创建了一个纯粹的[知识星球](#)。虽然收费只有培训班/训练营的百分之一，但是[知识星球](#)里的内容质量更高，提供的服务也更全面。

欢迎准备 Java 面试以及学习 Java 的同学加入我的[知识星球](#)，干货非常多，学习氛围非常好！收费虽然是白菜价，但星球里的内容或许比你参加上万的培训班质量还要高。

JavaGuide&Java面试交流圈

星主：Guide哥



1.5万
成员数量

9800+
内容数量

1024
运营天数

最优质的 Java 面试交流星球，多次被知识星球官方推荐。门票即将上调到 199 元，IOS 用户请在公众号“JavaGuide”回复“知识星球”加入/续费。

...

知识星球

微信扫码加入星球 ▶



下面是星球提供的部分服务（点击下方图片即可获取知识星球的详细介绍）：

我的知识星球能为你提供什么？

努力做一个最优质的 Java 面试交流星球！加入到我的星球之后，你将获得：

1. 6 个高质量的专栏永久阅读，内容涵盖面试，源码解析，项目实战等内容！价值远超门票！
2. 多本原创 PDF 版本面试手册。
3. 免费的简历修改服务（已经累计帮助 4000+ 位球友修改简历）。
4. 一对一免费提问交流（专属建议，走心回答）。
5. 专属求职指南和建议，帮助你逆袭大厂！
6. 海量 Java 优质面试资源分享！价值远超门票！
7. 读书交流，学习交流，让我们共同努力创建一个纯粹的学习交流社区。
8. 不定期福利：节日抽奖、送书送课、球友线下聚会等等。
9.

其中的任何一项服务单独拎出来价值都远超星球门票了。

我有自己的原则，不割韭菜，用心做内容，真心希望帮助到你！

如果你感兴趣的话，不妨花 3 分钟左右看看星球的详细介绍：[JavaGuide 知识星球详细介绍](#)（文末有优惠券）。



6. 系统设计

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！



[Github](#) |

[Gitee](#)

6.1 设计模式

设计模式 相关的面试题已经整理到了 PDF 手册中，你可以在我的公众号“**JavaGuide**”后台回复“**PDF**”获取。



JavaGuide

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

扫码关注

《设计模式》PDF 电子书内容概览：

前言

软件设计原则有哪些？

什么是设计模式？

设计模式的分类了解吗？

你知道哪些设计模式？

✓ 工厂模式

说一说简单工厂模式

工厂方法模式了解吗？

抽象工厂模式了解吗？

✓ 单例模式

什么是单例模式？单例模式的特点是什么？

> 单例模式的常见写法有哪些？

✓ 适配器模式

适配器模式了解吗？

适配器模式的优缺点

✓ 代理模式 (proxy pattern)

什么是代理模式？

静态代理和动态代理的区别

✓ 观察者模式

说一说观察者模式

观察者模式的优缺点

你的项目是怎么用的观察者模式？

面试资料

✓ 装饰器模式

什么是装饰器模式？

讲讲装饰器模式的应用场景

> 责任链模式

> 策略模式

策略模式有什么好处？

Spring 使用了哪些设计模式？

JDK 使用了哪些设计模式？

参考资料

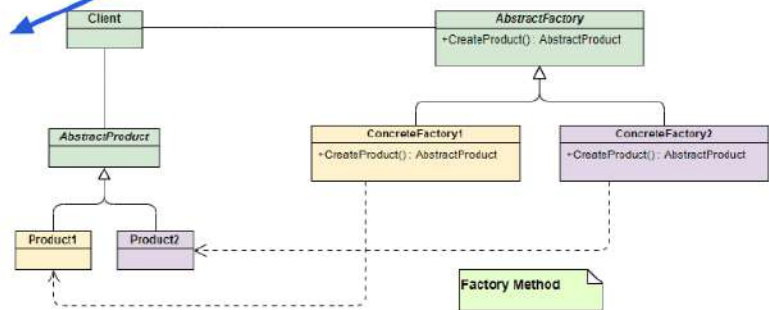
面试资料

工厂方法模式了解吗？

和简单工厂模式中工厂负责生产所有产品相比，工厂方法模式将生成具体产品的任务分发给具体的产品工厂。

UML 类图如下：

内容全面，质量高！



也就是定义一个抽象工厂，其定义了产品的生产接口，但不负责具体的产品，将生产任务交给不同的派生类工厂。这样不用通过指定类型来创建对象了。

抽象工厂模式了解吗？

简单工厂模式和工厂方法模式不管工厂怎么拆分抽象，都只是针对一类产品，如果要生成另一种产品，就比较难办了！

抽象工厂模式通过在 AbstractFactory 中增加创建产品的接口，并在具体子工厂中实现新加产品的创建，当然前提是子工厂支持生产该产品。否则继承的这个接口可以什么也不干。

UML 类图如下：

6.2 认证授权

认证 (Authentication) 和授权 (Authorization)的区别是什么？

这是一个绝大多数人都会混淆的问题。首先先从读音上来认识这两个名词，很多人都会把它俩的读音搞混，所以我建议你先先去查一查这两个单词到底该怎么读，他们的具体含义是什么。

说简单点就是：

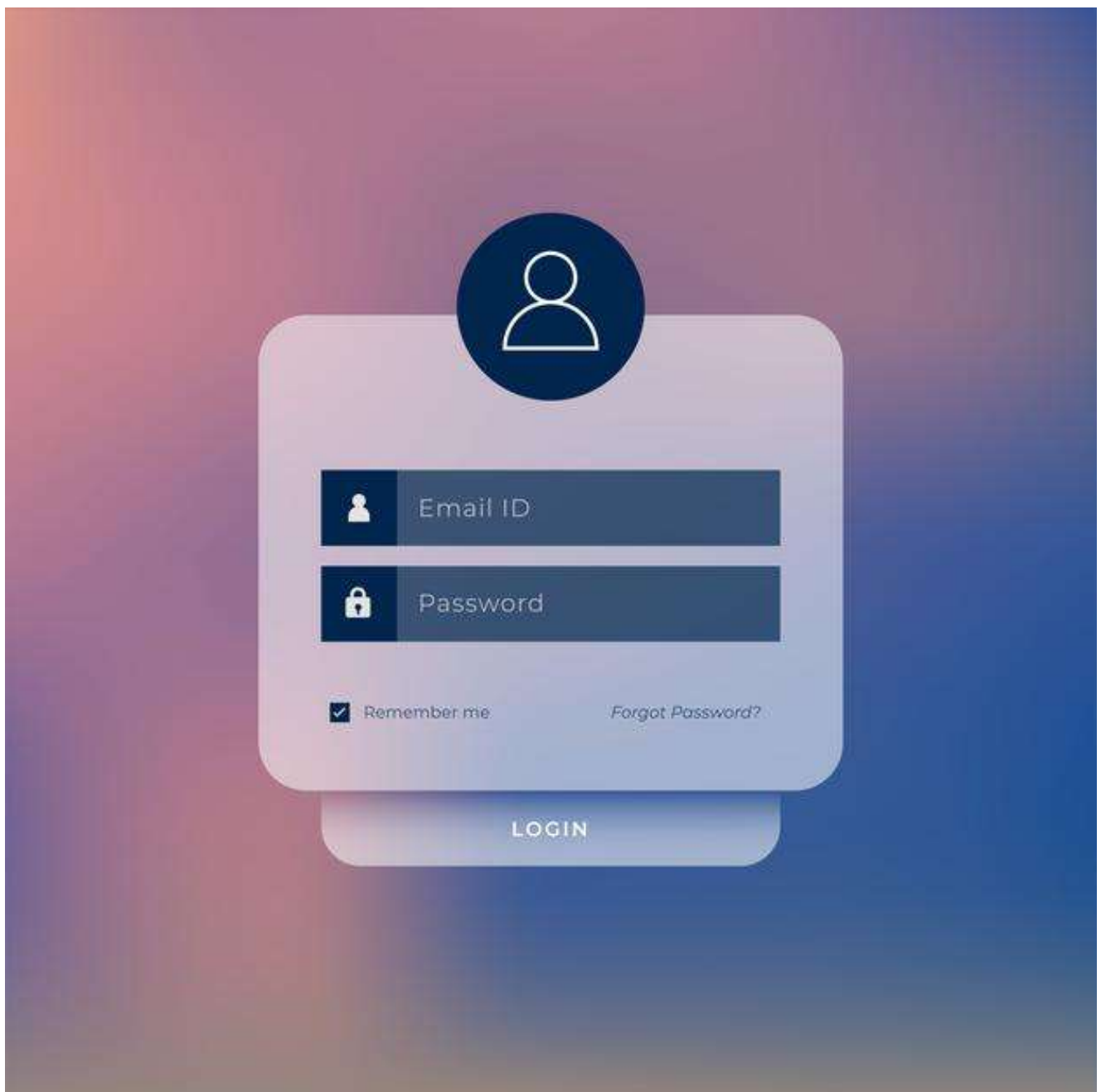
- 认证 (Authentication)：你是谁。

- **授权 (Authorization):** 你有权限干什么。

稍微正式点（啰嗦点）的说法就是：

- **Authentication (认证)** 是验证您的身份的凭据（例如用户名/用户 ID 和密码），通过这个凭据，系统得以知道你就是你，也就是说系统存在你这个用户。所以，Authentication 被称为身份/用户验证。
- **Authorization (授权)** 发生在 **Authentication (认证)** 之后。授权嘛，光看意思大家应该就明白，它主要掌管我们访问系统的权限。比如有些特定资源只能具有特定权限的人才能访问比如 admin，有些对系统资源操作比如删除、添加、更新只能特定人才具有。

认证：



授权：



这两个一般在我们的系统中被结合在一起使用，目的就是为了保护我们系统的安全性。

RBAC 模型了解吗？

系统权限控制最常采用的访问控制模型就是 **RBAC 模型**。

什么是 RBAC 呢？

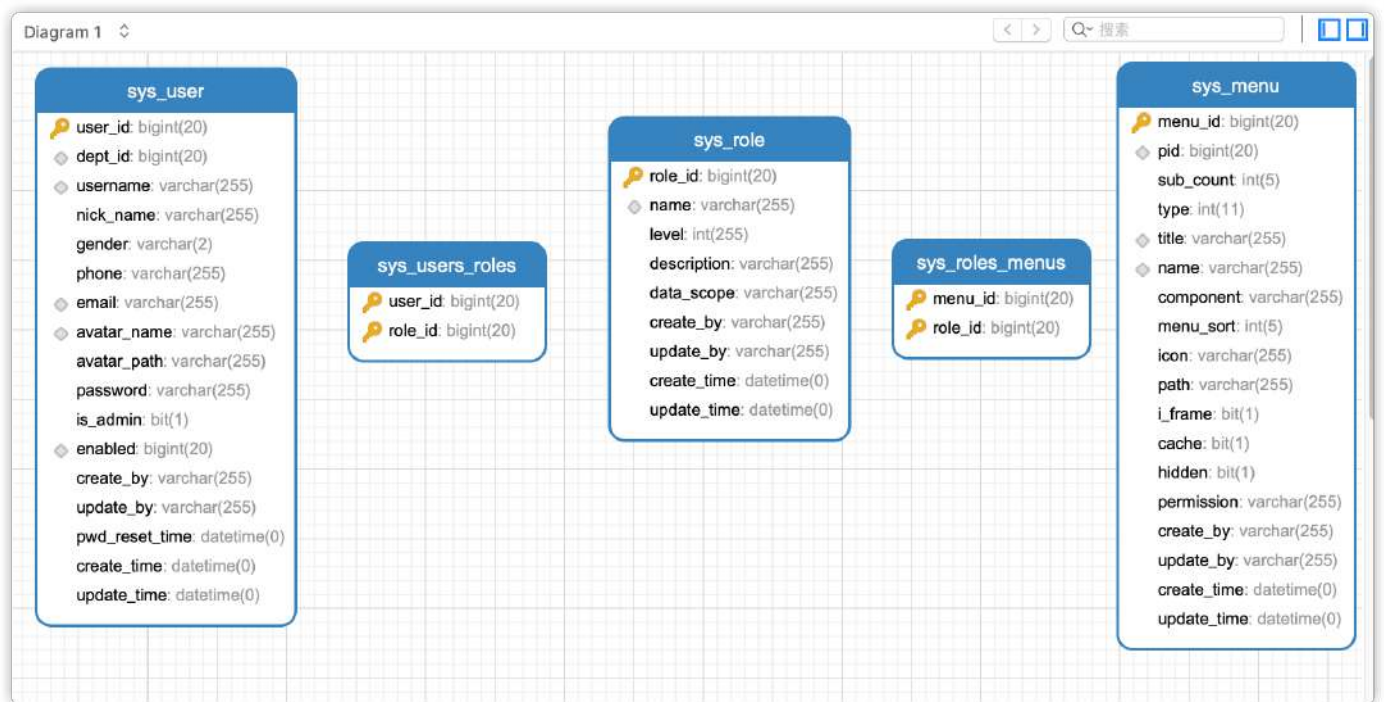
RBAC 即基于角色的权限访问控制（Role-Based Access Control）。这是一种通过角色关联权限，角色同时又关联用户的授权的方式。

简单地说：一个用户可以拥有若干角色，每一个角色又可以被分配若干权限，这样就构成“用户-角色-权限”的授权模型。在这种模型中，用户与角色、角色与权限之间构成了多对多的关系，如下图



在 **RBAC** 中，权限与角色相关联，用户通过成为适当角色的成员而得到这些角色的权限。这就极大地简化了权限的管理。

本系统的权限设计相关的表如下（一共 5 张表，2 张用户建立表之间的联系）：



通过这个权限模型，我们可以创建不同的角色并为不同的角色分配不同的权限范围（菜单）。

输入名称或者描述搜索

开始日期： 结束日期： 搜索 重置

+ 新增 - 修改 删除 + 导出

角色列表

☐	名称	数据权限	角色级别	描述	创建日期	操作
☐	超级管理员	全部	1	-	2018-11-23 11:04:37	删除 重置
☐	普通用户	自定义	2	-	2018-11-23 13:09:06	删除 重置

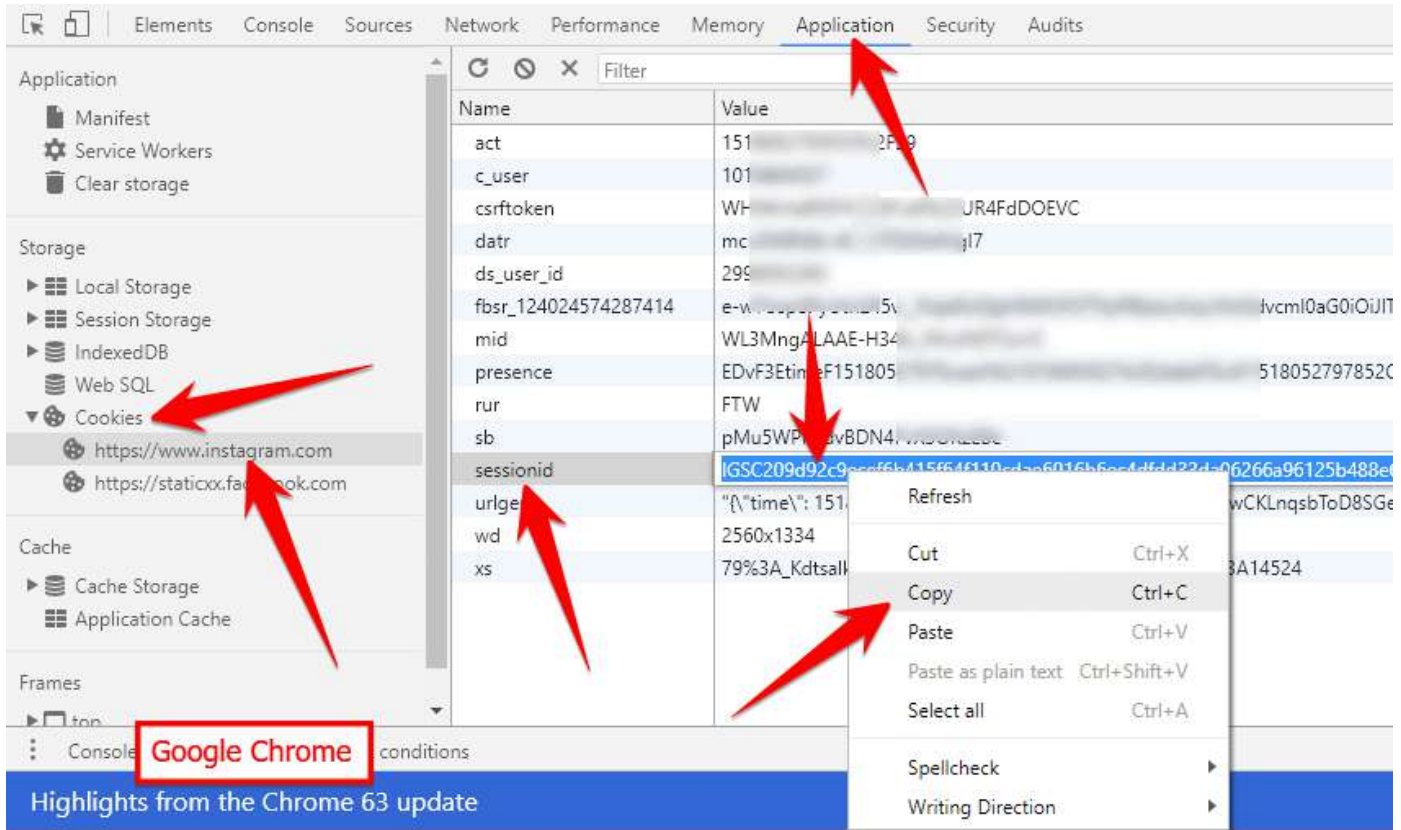
共 2 条 < 1 > 10条/页

菜单分配 保存

- ☐ 系统管理
 - ☐ 用户管理
 - ☐ 角色管理
 - ☐ 菜单管理
 - ☐ 任务调度
 - ☐ 部门管理
 - ☐ 岗位管理
 - ☐ 字典管理
- ☐ 系统监控
 - ☐ 组件管理
- ☐ 多级菜单
- ☐ 系统工具
- ☐ 运维管理

通常来说，如果系统对于权限控制要求比较严格的话，一般都会选择使用 RBAC 模型来做权限控制。

什么是 Cookie ? Cookie 的作用是什么？



Cookie 和 Session 都是用来跟踪浏览器用户身份的会话方式，但是两者的应用场景不太一样。

维基百科是这样定义 Cookie 的：

Cookies 是某些网站为了辨别用户身份而储存在用户本地终端上的数据（通常经过加密）。

简单来说：Cookie 存放在客户端，一般用来保存用户信息。

下面是 Cookie 的一些应用案例：

1. 我们在 Cookie 中保存已经登录过的用户信息，下次访问网站的时候页面可以自动帮你登录的一些基本信息给填了。除此之外，Cookie 还能保存用户首选项，主题和其他设置信息。
2. 使用 Cookie 保存 SessionId 或者 Token，向后端发送请求的时候带上 Cookie，这样后端就能取到 Session 或者 Token 了。这样就能记录用户当前的状态了，因为 HTTP 协议是无状态的。
3. Cookie 还可以用来记录和分析用户行为。举个简单的例子你在网上购物的时候，因为 HTTP 协议是没有状态的，如果服务器想要获取你在某个页面的停留状态或者看了哪些商品，一种常用的实现方式就是将这些信息存放在 Cookie
4.

如何在项目中使用 Cookie 呢?

我这里以 Spring Boot 项目为例。

1) 设置 Cookie 返回给客户端

```
@GetMapping("/change-username")
public String setCookie(HttpServletResponse response) {
    // 创建一个 cookie
    Cookie cookie = new Cookie("username", "Jovan");
    // 设置 cookie 过期时间
    cookie.setMaxAge(7 * 24 * 60 * 60); // expires in 7 days
    // 添加到 response 中
    response.addCookie(cookie);

    return "Username is changed!";
}
```

2) 使用 Spring 框架提供的 @CookieValue 注解获取特定的 cookie 的值

```
@GetMapping("/")
public String readCookie(@CookieValue(value = "username", defaultValue = "Atta")
String username) {
    return "Hey! My username is " + username;
}
```

3) 读取所有的 Cookie 值

```
@GetMapping("/all-cookies")
public String readAllCookies(HttpServletRequest request) {

    Cookie[] cookies = request.getCookies();
    if (cookies != null) {
        return Arrays.stream(cookies)
            .map(c -> c.getName() + "=" +
c.getValue()).collect(Collectors.joining(", "));
    }

    return "No cookies";
}
```

更多关于如何在 Spring Boot 中使用 `Cookie` 的内容可以查看这篇文章：[How to use cookies in Spring Boot](#)。

Cookie 和 Session 有什么区别？

`Session` 的主要作用就是通过服务端记录用户的状态。典型的场景是购物车，当你要添加商品到购物车的时候，系统不知道是哪个用户操作的，因为 HTTP 协议是无状态的。服务端给特定的用户创建特定的 `Session` 之后就可以标识这个用户并且跟踪这个用户了。

`Cookie` 数据保存在客户端(浏览器端)，`Session` 数据保存在服务器端。相对来说 `Session` 安全性更高。如果使用 `Cookie` 的一些敏感信息不要写入 `Cookie` 中，最好能将 `Cookie` 信息加密然后使用到的时候再去服务器端解密。

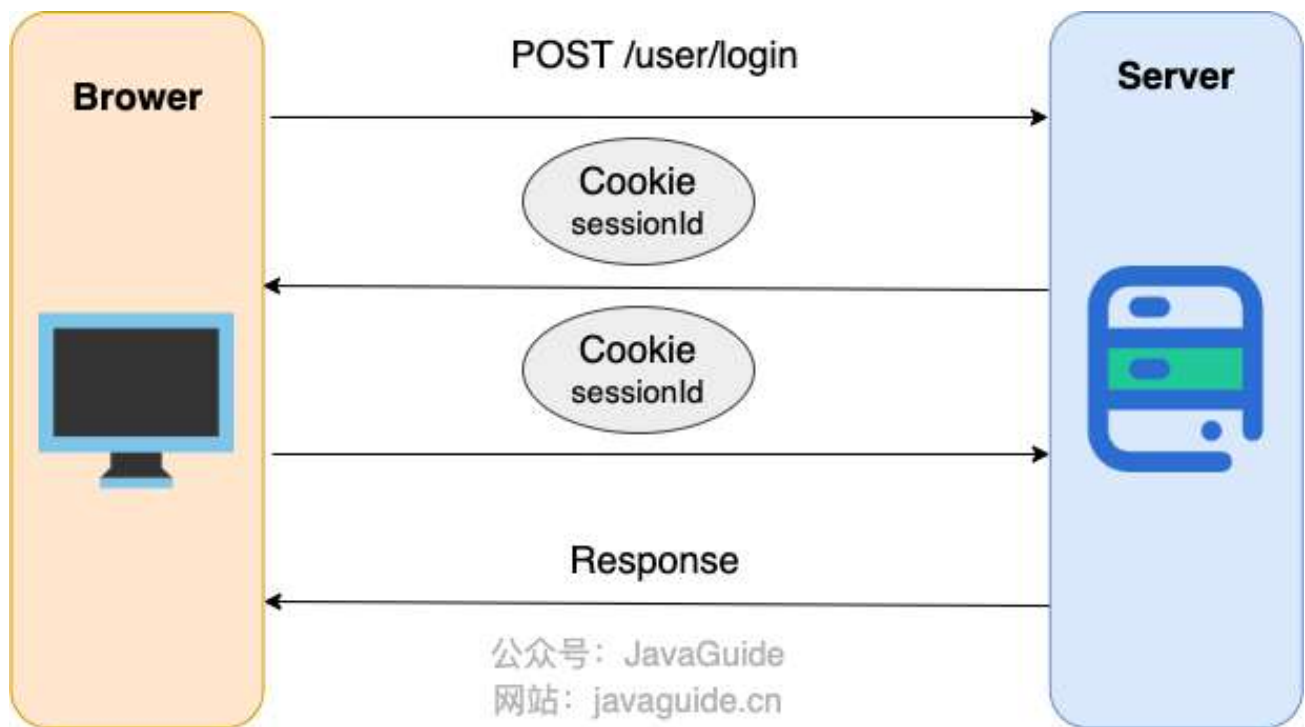
那么，如何使用 `Session` 进行身份验证？

如何使用 Session-Cookie 方案进行身份验证？

很多时候我们都是通过 `SessionID` 来实现特定的用户，`SessionID` 一般会选择存放在 Redis 中。举个例子：

1. 用户成功登陆系统，然后返回给客户端具有 `SessionID` 的 `Cookie`。
2. 当用户向后端发起请求的时候会带 `SessionID` 带上，这样后端就知道你的身份状态了。

关于这种认证方式更详细的过程如下：



1. 用户向服务器发送用户名、密码、验证码用于登陆系统。
2. 服务器验证通过后，服务器为用户创建一个 `Session`，并将 `Session` 信息存储起来。
3. 服务器向用户返回一个 `SessionID`，写入用户的 `Cookie`。
4. 当用户保持登录状态时，`Cookie` 将与每个后续请求一起被发送出去。
5. 服务器可以将存储在 `Cookie` 上的 `SessionID` 与存储在内存中或者数据库中的 `Session` 信息进行比较，以验证用户的身份，返回给用户客户端响应信息的时候会附带用户当前的状态。

使用 `Session` 的时候需要注意下面几个点：

- 依赖 `Session` 的关键业务一定要确保客户端开启了 `Cookie`。
- 注意 `Session` 的过期时间。

另外，Spring Session 提供了一种跨多个应用程序或实例管理用户会话信息的机制。如果想详细了解可以查看下面几篇很不错的文章：

- [Getting Started with Spring Session](#)
- [Guide to Spring Session](#)
- [Sticky Sessions with Spring Session & Redis](#)

多服务器节点下 Session-Cookie 方案如何做？

Session-Cookie 方案在单体环境是一个非常好的身份认证方案。但是，当服务器水平拓展成多节点时，Session-Cookie 方案就要面临挑战了。

举个例子：假如我们部署了两份相同的服务 A，B，用户第一次登陆的时候，Nginx 通过负载均衡机制将用户请求转发到 A 服务器，此时用户的 Session 信息保存在 A 服务器。结果，用户第二次访问的时候 Nginx 将请求路由到 B 服务器，由于 B 服务器没有保存用户的 Session 信息，导致用户需要重新进行登陆。

我们应该如何避免上面这种情况的出现呢？

有几个方案供大家参考：

1. 某个用户的所有请求都通过特性的哈希策略分配给同一个服务器处理。这样的话，每个服务器都保存了一部分用户的 Session 信息。服务器宕机，其保存的所有 Session 信息就完全丢失了。
2. 每一个服务器保存的 Session 信息都是互相同步的，也就是说每一个服务器都保存了全量的 Session 信息。每当一个服务器的 Session 信息发生变化，我们就将其同步到其他服务器。这种方案成本太大，并且，节点越多时，同步成本也越高。
3. 单独使用一个所有服务器都能访问到的数据节点（比如缓存）来存放 Session 信息。为了保证高可用，数据节点尽量要避免是单点。

如果没有 Cookie 的话 Session 还能用吗？

这是一道经典的面试题！

一般是通过 Cookie 来保存 SessionID，假如你使用了 Cookie 保存 SessionID 的方案的话，如果客户端禁用了 Cookie，那么 Session 就无法正常工作。

但是，并不是没有 Cookie 之后就不能用 Session 了，比如你可以将 SessionID 放在请求的 url 里面 `https://javaguide.cn/?Session_id=xxx`。这种方案的话可行，但是安全性和用户体验感降低。当然，为了你也可以对 SessionID 进行一次加密之后再传入后端。

为什么 Cookie 无法防止 CSRF 攻击，而 Token 可以？

CSRF(Cross Site Request Forgery) 一般被翻译为 跨站请求伪造。那么什么是 跨站请求伪造 呢？说简单用你的身份去发送一些对你不友好的请求。举个简单的例子：

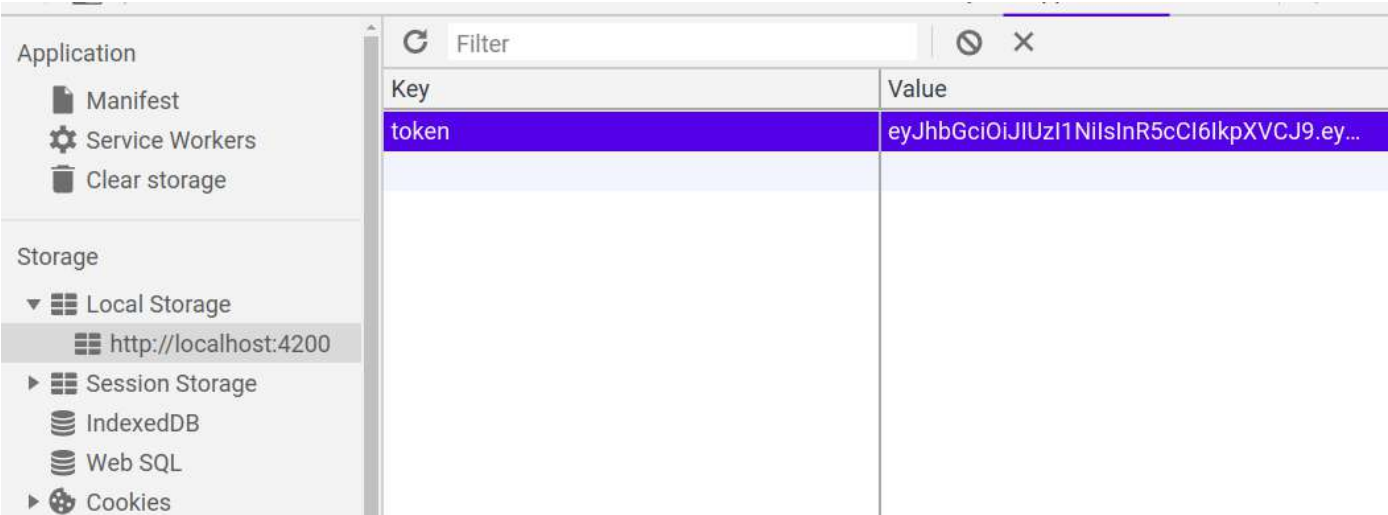
小壮登录了某网上银行，他来到了网上银行的帖子区，看到一个帖子下面有一个链接写着“科学理财，年盈利率过万”，小壮好奇的点开了这个链接，结果发现自己的账户少了 10000 元。这是这么回事呢？原来黑客在链接中藏了一个请求，这个请求直接利用小壮的身份给银行发送了一个转账请求，也就是通过你的 Cookie 向银行发出请求。

```
<a src=http://www.mybank.com/Transfer?bankId=11&money=10000>科学理财，年盈利率过万</>
```

上面也提到过，进行 Session 认证的时候，我们一般使用 Cookie 来存储 SessionId ,当我们登陆后后端生成一个 SessionId 放在 Cookie 中返回给客户端，服务端通过 Redis 或者其他存储工具记录保存着这个 SessionId ，客户端登录以后每次请求都会带上这个 SessionId ，服务端通过这个 SessionId 来标示你这个人。如果别人通过 Cookie 拿到了 SessionId 后就可以代替你的身份访问系统了。

Session 认证中 Cookie 中的 SessionId 是由浏览器发送到服务端的，借助这个特性，攻击者就可以通过让用户误点攻击链接，达到攻击效果。

但是，我们使用 Token 的话就不会存在这个问题，在我们登录成功获得 Token 之后，一般会选择存放在 localStorage （浏览器本地存储）中。然后我们在前端通过某些方式会给每个发到后端的请求加上这个 Token ,这样就不会出现 CSRF 漏洞的问题。因为，即使有个你点击了非法链接发送了请求到服务端，这个非法请求是不会携带 Token 的，所以这个请求将是非法的。



需要注意的是：不论是 Cookie 还是 Token 都无法避免 跨站脚本攻击（Cross Site Scripting）XSS 。

跨站脚本攻击（Cross Site Scripting）缩写为 CSS 但这会与层叠样式表（Cascading Style Sheets, CSS）的缩写混淆。因此，有人将跨站脚本攻击缩写为 XSS。

XSS 中攻击者会用各种方式将恶意代码注入到其他用户的页面中。就可以通过脚本盗用信息比如 Cookie 。

推荐阅读：[如何防止 CSRF 攻击？—美团技术团队](#)

什么是 JWT?JWT 由哪些部分组成？

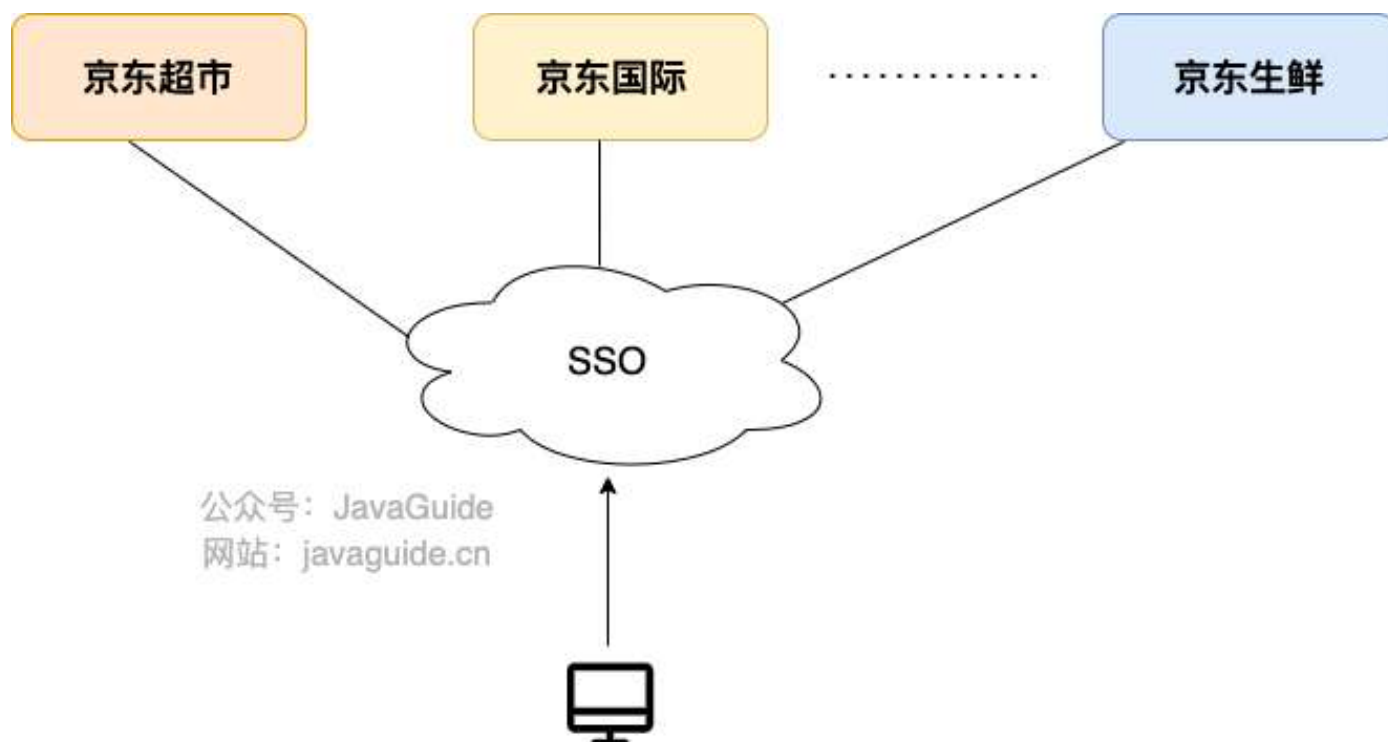
[JWT 基础概念详解](#)

如何基于 JWT 进行身份验证？ 如何防止 JWT 被篡改？

[JWT 基础概念详解](#)

什么是 SSO？

SSO(Single Sign On)即单点登录说的是用户登陆多个子系统的其中一个就有权访问与其相关的其他系统。举个例子我们在登陆了京东金融之后，我们同时也成功登陆京东的京东超市、京东国际、京东生鲜等子系统。



SSO 有什么好处?

- **用户角度** :用户能够做到一次登录多次使用, 无需记录多套用户名和密码, 省心。
- **系统管理员角度** : 管理员只需维护好一个统一的账号中心就可以了, 方便。
- **新系统开发角度**: 新系统开发时只需直接对接统一的账号中心即可, 简化开发流程, 省时。

如何设计实现一个 SSO 系统?

[SSO 单点登录详解](#)

什么是 OAuth 2.0?

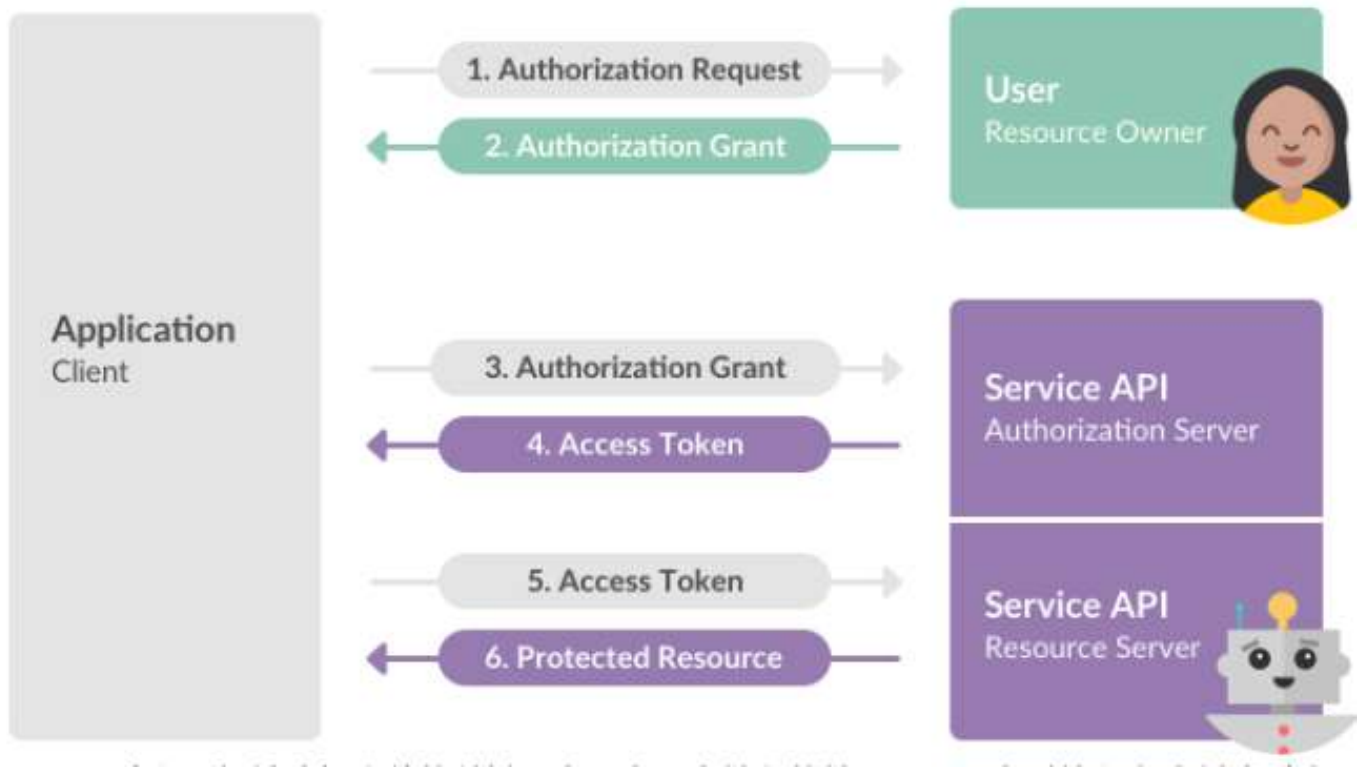
OAuth 是一个行业的标准授权协议, 主要用来授权第三方应用获取有限的权限。而 OAuth 2.0 是对 OAuth 1.0 的完全重新设计, OAuth 2.0 更快, 更容易实现, OAuth 1.0 已经被废弃。详情请见: [rfc6749](#)。

实际上它就是一种授权机制, 它的最终目的是为第三方应用颁发一个有时效性的令牌 Token, 使得第三方应用能够通过该令牌获取相关的资源。

OAuth 2.0 比较常用的场景就是第三方登录, 当你的网站接入了第三方登录的时候一般就是使用的 OAuth 2.0 协议。

另外, 现在 OAuth 2.0 也常见于支付场景(微信支付、支付宝支付)和开发平台(微信开放平台、阿里开放平台等等)。

下图是 [Slack OAuth 2.0 第三方登录](#)的示意图:



推荐阅读：

- OAuth 2.0 的一个简单解释
- 10 分钟理解什么是 OAuth 2.0 协议
- OAuth 2.0 的四种方式
- GitHub OAuth 第三方登录示例教程

参考

- 不要用 JWT 替代 session 管理（上）：全面了解 Token,JWT,OAuth,SAML,SSO: <https://zhuanglan.zhihu.com/p/38942172>
- Introduction to JSON Web Tokens: <https://jwt.io/introduction>
- JSON Web Token Claims: <https://auth0.com/docs/secure/tokens/json-web-tokens/json-web-token-claims>

6.3 定时任务

为什么需要定时任务？

我们来看一下几个非常常见的业务场景：

1. 某系统凌晨要进行数据备份。
2. 某电商平台，用户下单半个小时未支付的情况下需要自动取消订单。
3. 某媒体聚合平台，每 10 分钟动态抓取某某网站的数据为自己所用。
4. 某博客平台，支持定时发送文章。
5. 某基金平台，每晚定时计算用户当日收益情况并推送给用户最新的数据。
6.

这些场景往往都要求我们在某个特定的时间去做某个事情。

单机定时任务技术选型有哪些？

Timer

`java.util.Timer` 是 JDK 1.3 开始就已经支持的一种定时任务的实现方式。

`Timer` 内部使用一个叫做 `TaskQueue` 的类存放定时任务，它是一个基于最小堆实现的优先级队列。`TaskQueue` 会按照任务距离下一次执行时间的大小将任务排序，保证在堆顶的任务最先执行。这样在需要执行任务时，每次只需要取出堆顶的任务运行即可！

`Timer` 使用起来比较简单，通过下面的方式我们就能创建一个 1s 之后执行的定时任务。

```
// 示例代码：
TimerTask task = new TimerTask() {
    public void run() {
        System.out.println("当前时间： " + new Date() + "n" +
            "线程名称： " + Thread.currentThread().getName());
    }
};
System.out.println("当前时间： " + new Date() + "n" +
    "线程名称： " + Thread.currentThread().getName());
Timer timer = new Timer("Timer");
long delay = 1000L;
timer.schedule(task, delay);
```

```
//输出:
```

```
当前时间: Fri May 28 15:18:47 CST 2021\n线程名称: main
```

```
当前时间: Fri May 28 15:18:48 CST 2021\n线程名称: Timer
```

不过其缺陷较多, 比如一个 `Timer` 一个线程, 这就导致 `Timer` 的任务的执行只能串行执行, 一个任务执行时间过长的话会影响其他任务 (性能非常差), 再比如发生异常时任务直接停止 (`Timer` 只捕获了 `InterruptedException`) 。

`Timer` 类上的有一段注释是这样写的:

```
* This class does not offer real-time guarantees: it schedules
* tasks using the <tt>Object.wait(long)</tt> method.
* Java 5.0 introduced the {@code java.util.concurrent} package and
* one of the concurrency utilities therein is the {@link
* java.util.concurrent.ScheduledThreadPoolExecutor
* ScheduledThreadPoolExecutor} which is a thread pool for repeatedly
* executing tasks at a given rate or delay. It is effectively a more
* versatile replacement for the {@code Timer}/{@code TimerTask}
* combination, as it allows multiple service threads, accepts various
* time units, and doesn't require subclassing {@code TimerTask} (just
* implement {@code Runnable}). Configuring {@code
* ScheduledThreadPoolExecutor} with one thread makes it equivalent to
* {@code Timer}.
```

大概的意思就是: `ScheduledThreadPoolExecutor` 支持多线程执行定时任务并且功能更强大, 是 `Timer` 的替代品。

ScheduledExecutorService

`ScheduledExecutorService` 是一个接口, 有多个实现类, 比较常用的是 `ScheduledThreadPoolExecutor`

。



`ScheduledThreadPoolExecutor` 本身就是一个线程池，支持任务并发执行。并且，其内部使用 `DelayQueue` 作为任务队列。

// 示例代码：

```
TimerTask repeatedTask = new TimerTask() {
    @SneakyThrows
    public void run() {
        System.out.println("当前时间: " + new Date() + "n" +
            "线程名称: " + Thread.currentThread().getName());
    }
};

System.out.println("当前时间: " + new Date() + "n" +
    "线程名称: " + Thread.currentThread().getName());

ScheduledExecutorService executor = Executors.newScheduledThreadPool(3);
long delay = 1000L;
long period = 1000L;
executor.scheduleAtFixedRate(repeatedTask, delay, period,
    TimeUnit.MILLISECONDS);
```

```
Thread.sleep(delay + period * 5);
executor.shutdown();
//输出:
当前时间: Fri May 28 15:40:46 CST 2021n线程名称: main
当前时间: Fri May 28 15:40:47 CST 2021n线程名称: pool-1-thread-1
当前时间: Fri May 28 15:40:48 CST 2021n线程名称: pool-1-thread-1
当前时间: Fri May 28 15:40:49 CST 2021n线程名称: pool-1-thread-2
当前时间: Fri May 28 15:40:50 CST 2021n线程名称: pool-1-thread-2
当前时间: Fri May 28 15:40:51 CST 2021n线程名称: pool-1-thread-2
当前时间: Fri May 28 15:40:52 CST 2021n线程名称: pool-1-thread-2
```

不论是使用 `Timer` 还是 `ScheduledExecutorService` 都无法使用 Cron 表达式指定任务执行的具体时间。

Spring Task



我们直接通过 Spring 提供的 `@Scheduled` 注解即可定义定时任务，非常方便！

```
/**
 * cron: 使用Cron表达式。 每分钟的1, 2秒运行
 */
@Scheduled(cron = "1-2 * * * * ? ")
public void reportCurrentTimeWithCronExpression() {
    log.info("Cron Expression: The time is now {}", dateFormat.format(new
    Date()));
}
```

我在大学那会做的一个 SSM 的企业级项目，就是用的 Spring Task 来做的定时任务。

并且，Spring Task 还支持 **Cron 表达式** 的。Cron 表达式主要用于定时作业(定时任务)系统定义执行时间或执行频率的表达式，非常厉害，你可以通过 Cron 表达式进行设置定时任务每天或者每个月什么时候执行等等操作。咱们要学习定时任务的话，Cron 表达式是一定要重点关注的。推荐一个在线 Cron 表达式生成器：<http://cron.qqe2.com/>。

但是，Spring 自带的定时调度只支持单机，并且提供的功能比较单一。之前写过一篇文章：[《5 分钟搞懂如何在 Spring Boot 中 Schedule Tasks》](#)，不了解的小伙伴可以参考一下。

Spring Task 底层是基于 JDK 的 `ScheduledThreadPoolExecutor` 线程池来实现的。

优缺点总结：

- 优点：简单，轻量，支持 Cron 表达式
- 缺点：功能单一

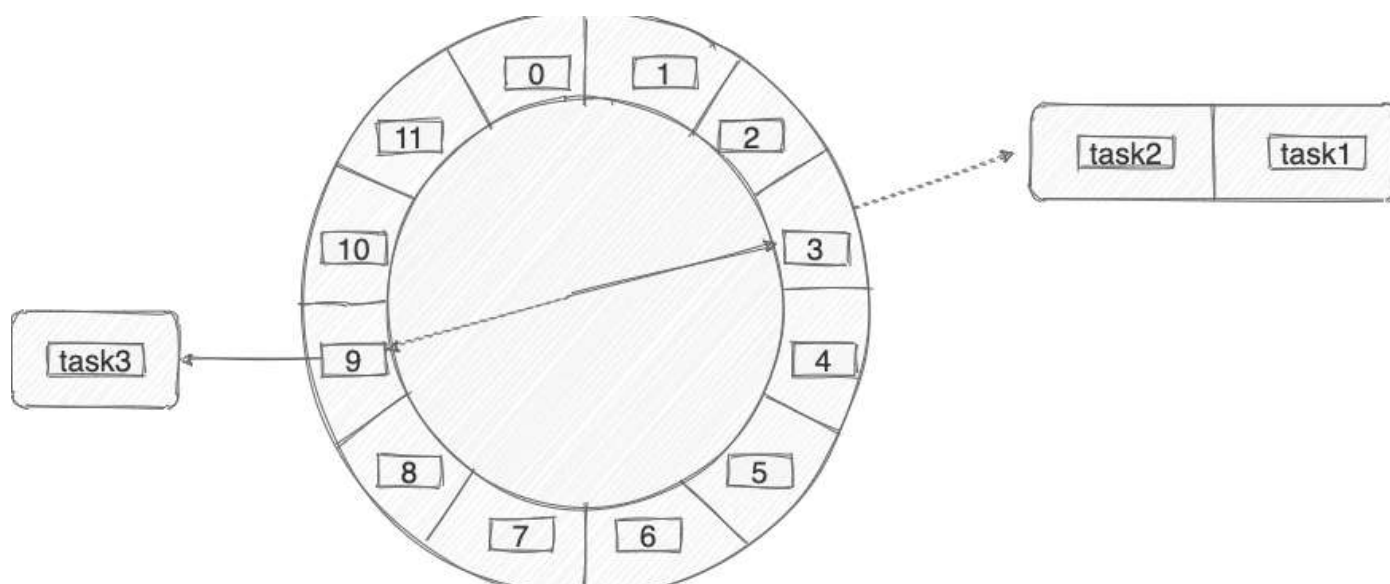
时间轮

Kafka、Dubbo、ZooKeeper、Netty、Caffeine、Akka 中都有对时间轮的实现。

时间轮简单来说就是一个环形的队列（底层一般基于数组实现），队列中的每一个元素（时间格）都可以存放一个定时任务列表。

时间轮中的每个时间格代表了时间轮的基本时间跨度或者说时间精度，加入时间一秒走一个时间格的话，那么这个时间轮的最高精度就是 1 秒（也就是说 3 s 和 3.9s 会在同一个时间格中）。

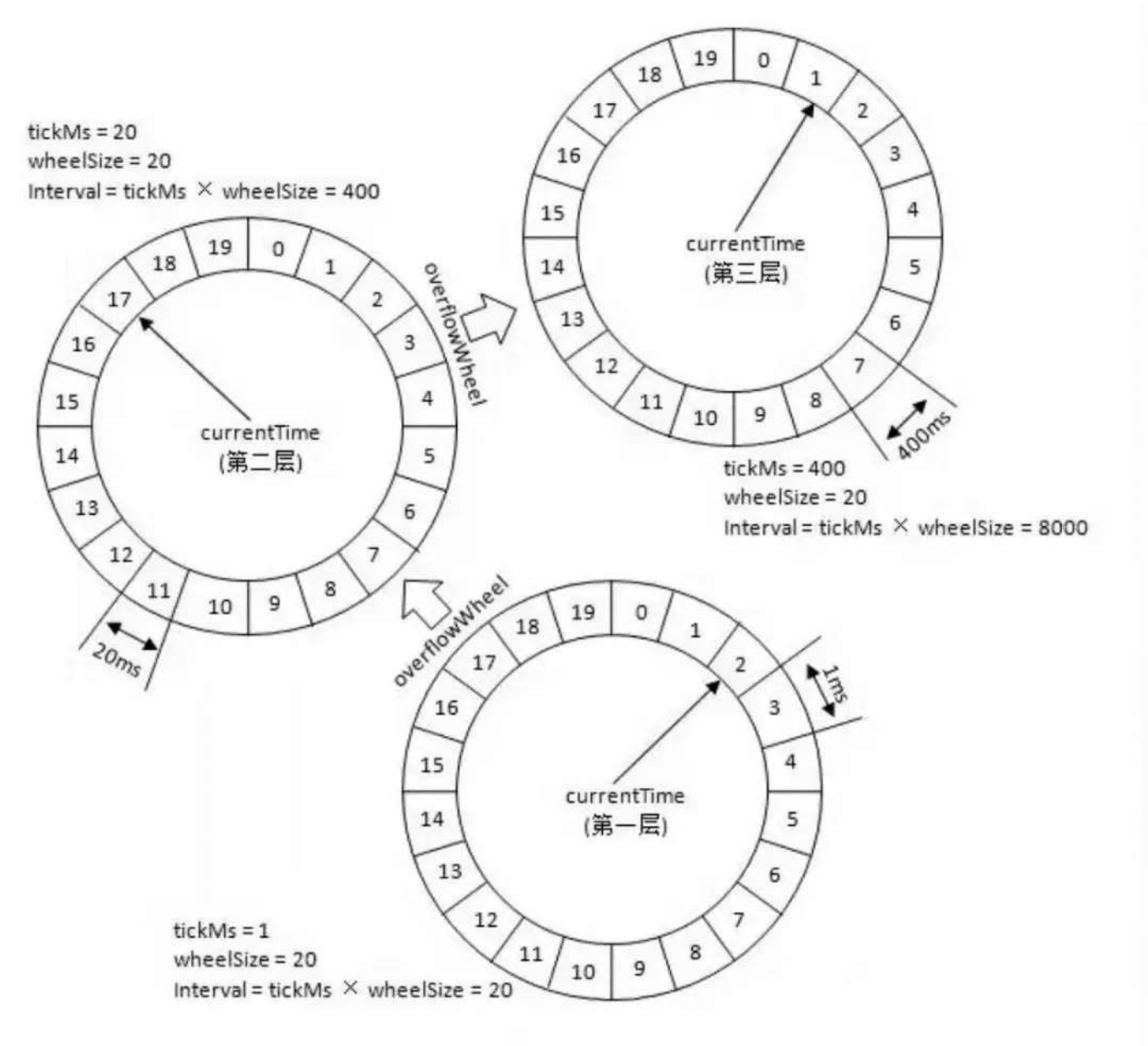
下图是一个有 12 个时间格的时间轮，转完一圈需要 12 s。当我们需要新建一个 3s 后执行的定时任务，只需要将定时任务放在下标为 3 的时间格中即可。当我们需要新建一个 9s 后执行的定时任务，只需要将定时任务放在下标为 9 的时间格中即可。



那当我们需要创建一个 13s 后执行的定时任务怎么办呢？这个时候可以引入一叫做 **圈数/轮数** 的概念，也就是说这个任务还是放在下标为 3 的时间格中， 不过它的圈数为 2 。

除了增加圈数这种方法之外，还有一种 **多层次时间轮**（类似手表）， Kafka 采用的就是这种方案。

针对下图的时间轮，我来举一个例子便于大家理解。



上图的时间轮，第 1 层的时间精度为 1，第 2 层的时间精度为 20，第 3 层的时间精度为 400。假如我们需要添加一个 350s 后执行的任务 A 的话（当前时间是 0s），这个任务会被放在第 2 层（因为第二层的时间跨度为 $20 \times 20 = 400 > 350$ ）的第 $350 / 20 = 17$ 个时间格子。

当第一层转了 17 圈之后，时间过去了 340s，第 2 层的指针此时来到第 17 个时间格子。此时，第 2 层第 17 个格子的任务会被移动到第 1 层。

任务 A 当前是 10s 之后执行，因此它会被移动到第 1 层的第 10 个时间格子。

这里在层与层之间的移动也叫做时间轮的升降级。参考手表来理解就好！



时间轮比较适合任务数量比较多的定时任务场景，它的任务写入和执行的时间复杂度都是 $O(1)$ 。

分布式定时任务技术选型有哪些？

上面提到的一些定时任务的解决方案都是在单机下执行的，适用于比较简单的定时任务场景比如每天凌晨备份一次数据。

如果我们需要一些高级特性比如支持任务在分布式场景下的分片和高可用的话，我们就需要用到分布式任务调度框架了。

通常情况下，一个定时任务的执行往往涉及到下面这些角色：

- **任务**：首先肯定是要执行的任务，这个任务就是具体的业务逻辑比如定时发送文章。
- **调度器**：其次是调度中心，调度中心主要负责任务管理，会分配任务给执行器。
- **执行器**：最后就是执行器，执行器接收调度器分派的任务并执行。

Quartz



一个很火的开源任务调度框架，完全由 Java 写成。Quartz 可以说是 Java 定时任务领域的老大哥或者说参考标准，其他的任务调度框架基本都是基于 Quartz 开发的，比如当当网的 elastic-job 就是基于 quartz 二次开发之后的分布式调度解决方案。

使用 Quartz 可以很方便地与 Spring 集成，并且支持动态添加任务和集群。但是，Quartz 使用起来也比较麻烦，API 繁琐。

并且，Quartz 并没有内置 UI 管理控制台，不过你可以使用 quartzui 这个开源项目来解决这个问题。

另外，Quartz 虽然也支持分布式任务。但是，它是在数据库层面，通过数据库的锁机制做的，有非常多的弊端比如系统侵入性严重、节点负载不均衡。有点伪分布式的味道。

优缺点总结：

- 优点：可以与 Spring 集成，并且支持动态添加任务和集群。
- 缺点：分布式支持不友好，没有内置 UI 管理控制台、使用麻烦（相比于其他同类型框架来说）

Elastic-Job



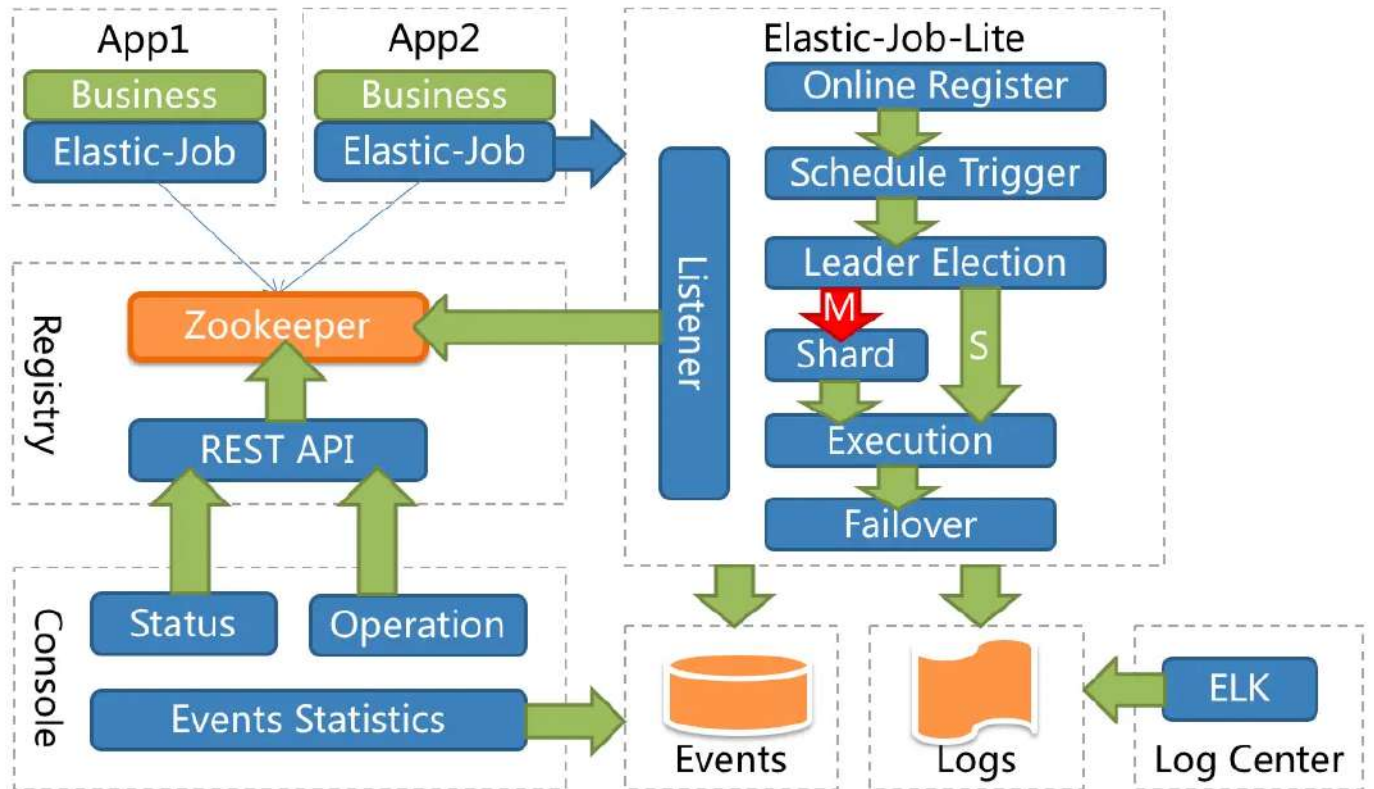
Elastic-Job 是当当网开源的一个基于 Quartz 和 ZooKeeper 的分布式调度解决方案，由两个相互独立的子项目 Elastic-Job-Lite 和 Elastic-Job-Cloud 组成，一般我们只要使用 Elastic-Job-Lite 就好。

ElasticJob 支持任务在分布式场景下的分片和高可用、任务可视化管理等功能。

功能列表

- 弹性调度
 - 支持任务在分布式场景下的分片和高可用
 - 能够水平扩展任务的吞吐量和执行效率
 - 任务处理能力随资源配备弹性伸缩
- 资源分配
 - 在适合的时间将适合的资源分配给任务并使其生效
 - 相同任务聚合至相同的执行器统一处理
 - 动态调配追加资源至新分配的任务
- 作业治理
 - 失效转移
 - 错过作业重新执行
 - 自诊断修复
- 作业依赖(TODO)
 - 基于有向无环图（DAG）的作业间依赖
 - 基于有向无环图（DAG）的作业分片间依赖
- 作业开放生态
 - 可扩展的作业类型统一接口
 - 丰富的作业类型库，如数据流、脚本、HTTP、文件、大数据等
 - 易于对接业务作业，能够与 Spring 依赖注入无缝整合
- 可视化管控端
 - 作业管控端
 - 作业执行历史数据追踪
 - 注册中心管理

ElasticJob-Lite 的架构设计如下图所示：



从上图可以看出，Elastic-Job 没有调度中心这一概念，而是使用 ZooKeeper 作为注册中心，注册中心负责协调分配任务到不同的节点上。

Elastic-Job 中的定时调度都是由执行器自行触发，这种设计也被称为去中心化设计（调度和处理都是执行器单独完成）。

```
@Component
@ElasticJobConf(name = "dayJob", cron = "0/10 * * * * ?", shardingTotalCount =
2,
    shardingItemParameters = "0=AAAA,1=BBBB", description = "简单任务",
    failover = true)
public class TestJob implements SimpleJob {
    @Override
    public void execute(ShardingContext shardingContext) {
        log.info("TestJob任务名: [{ }], 片数: [{ }], param= [{ }]",
            shardingContext.getJobName(), shardingContext.getShardingTotalCount(),
            shardingContext.getShardingParameter());
    }
}
```

相关地址：

- Github 地址：<https://github.com/apache/shardingsphere-elasticjob>。
- 官方网站：https://shardingsphere.apache.org/elasticjob/index_zh.html。

优缺点总结：

- 优点：可以与 Spring 集成、支持分布式、支持集群、性能不错
- 缺点：依赖了额外的中间件比如 Zookeeper（复杂度增加，可靠性降低、维护成本变高）

XXL-JOB



XXL-JOB 于 2015 年开源，是一款优秀的轻量级分布式任务调度框架，支持任务可视化管理、弹性扩容缩容、任务失败重试和告警、任务分片等功能，

1.3 特性

- 1、简单：支持通过Web页面对任务进行CRUD操作，操作简单，一分钟上手；
- 2、动态：支持动态修改任务状态、启动/停止任务，以及终止运行中任务，即时生效；
- 3、调度中心HA（中心式）：调度采用中心式设计，“调度中心”自研调度组件并支持集群部署，可保证调度中心HA；
- 4、执行器HA（分布式）：任务分布式执行，任务“执行器”支持集群部署，可保证任务执行HA；
- 5、注册中心：执行器会周期性自动注册任务，调度中心将会自动发现注册的任务并触发执行。同时，也支持手动录入执行器地址；
- 6、弹性扩容缩容：一旦有新执行器机器上线或者下线，下次调度时将会重新分配任务；
- 7、触发策略：提供丰富的任务触发策略，包括：Cron触发、固定间隔触发、固定延时触发、API（事件）触发、人工触发、父子任务触发；
- 8、调度过期策略：调度中心错过调度时间的补偿处理策略，包括：忽略、立即补偿触发一次等；
- 9、阻塞处理策略：调度过于密集执行器来不及处理时的处理策略，策略包括：单机串行（默认）、丢弃后续调度、覆盖之前调度；
- 10、任务超时控制：支持自定义任务超时时间，任务运行超时将会主动中断任务；
- 11、任务失败重试：支持自定义任务失败重试次数，当任务失败时将会按照预设的失败重试次数主动进行重试；其中分片任务支持分片粒度的失败重试；
- 12、任务失败告警：默认提供邮件方式失败告警，同时预留扩展接口，可方便的扩展短信、钉钉等告警方式；
- 13、路由策略：执行器集群部署时提供丰富的路由策略，包括：第一个、最后一个、轮询、随机、一致性HASH、最不经使用、最近最久未使用、故障转移、忙碌转移等；
- 14、分片广播任务：执行器集群部署时，任务路由策略选择“分片广播”情况下，一次任务调度将会广播触发集群中所有执行器执行一次任务，可根据分片参数开发分片任务；
- 15、动态分片：分片广播任务以执行器为维度进行分片，支持动态扩容执行器集群从而动态增加分片数量，协同进行业务处理；在进行大数据量业务操作时可显著提升任务处理能力和速度。
- 16、故障转移：任务路由策略选择“故障转移”情况下，如果执行器集群中某一台机器故障，将会自动Failover切换到一台正常的执行器发送调度请求。
- 17、任务进度监控：支持实时监控任务进度；
- 18、Rolling实时日志：支持在线查看调度结果，并且支持以Rolling方式实时查看执行器输出的完整的执行日志；
- 19、GLUE：提供Web IDE，支持在线开发任务逻辑代码，动态发布，实时编译生效，省略部署上线的过程。支持30个版本的历史版本回溯。
- 20、脚本任务：支持以GLUE模式开发和运行脚本任务，包括Shell、Python、NodeJS、PHP、PowerShell等类型脚本；
- 21、命令行任务：原生提供通用命令行任务Handler（Bean任务，“CommandJobHandler”）；业务方只需要提供命令即可；
- 22、任务依赖：支持配置子任务依赖，当父任务执行结束且执行成功后将会主动触发一次子任务的执行，多个子任务用逗号分隔；
- 23、一致性：“调度中心”通过DB锁保证集群分布式调度的一致性，一次任务调度只会触发一次执行；
- 24、自定义任务参数：支持在线配置调度任务入参，即时生效；
- 25、调度线程池：调度系统多线程触发调度运行，确保调度精确执行，不被堵塞；
- 26、数据加密：调度中心和执行器之间的通讯进行数据加密，提升调度信息安全；
- 27、邮件报警：任务失败时支持邮件报警，支持配置多邮件地址群发报警邮件；
- 28、推送maven中央仓库：将会把最新稳定版推送到maven中央仓库，方便用户接入和使用；
- 29、运行报表：支持实时查看运行数据，如任务数量、调度次数、执行器数量等；以及调度报表，如调度日期分布图，调度成功分布图等；
- 30、全异步：任务调度流程全异步化设计实现，如异步调度、异步运行、异步回调等，有效对密集调度进行流量削峰，理论上支持任意时长任务的运行；
- 31、跨语言：调度中心与执行器提供语言无关的 RESTful API 服务，第三方任意语言可据此对接调度中心或者实现执行器。除此之外，还提供了“多任务模式”和“httpJobHandler”等其他跨语言方案；
- 32、国际化：调度中心支持国际化设置，提供中文、英文两种可选语言，默认为中文；
- 33、容器化：提供官方docker镜像，并实时更新推送dockerhub，进一步实现产品开箱即用；
- 34、线程池隔离：调度线程池进行隔离拆分，慢任务自动降级进入“Slow”线程池，避免耗尽调度线程，提高系统稳定性；
- 35、用户管理：支持在线管理系统用户，存在管理员、普通用户两种角色；
- 36、权限控制：执行器维度进行权限控制，管理员拥有全量权限，普通用户需要分配执行器权限后才允许相关操作；

根据 XXL-JOB 官网介绍，其解决了很多 Quartz 的不足。

5.4 调度模块剖析

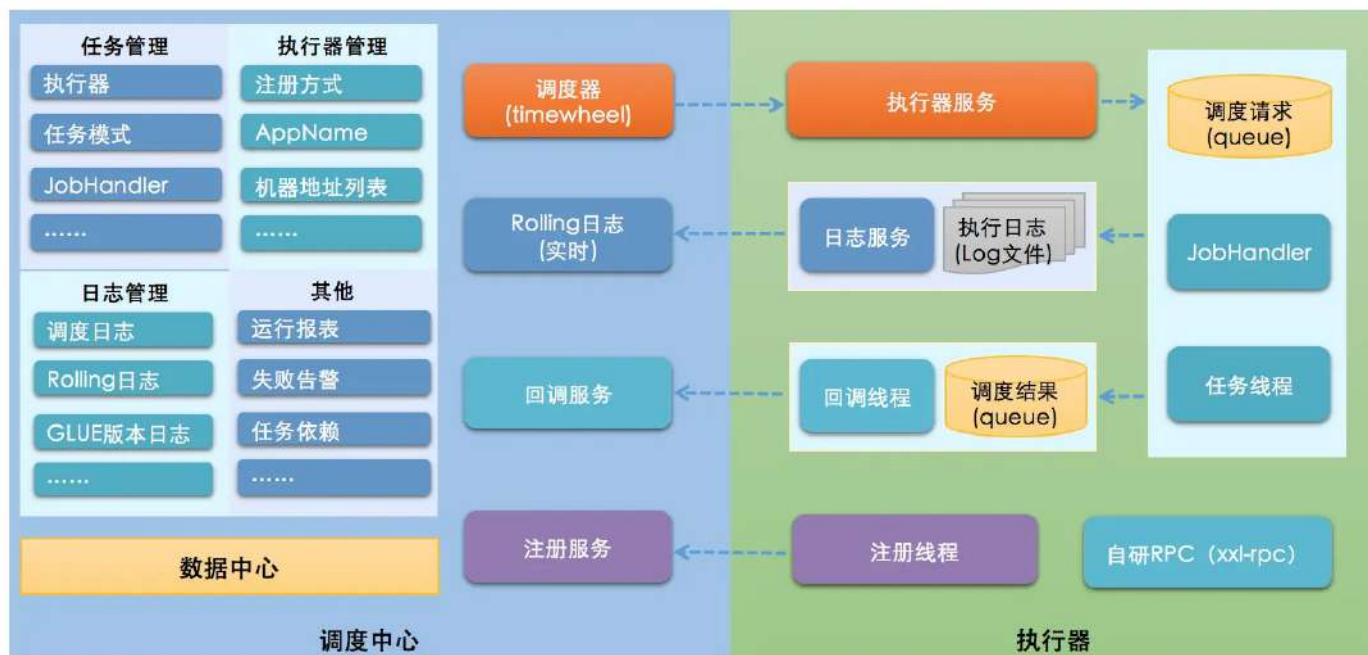
5.4.1 quartz的不足

Quartz作为开源作业调度中的佼佼者，是作业调度的首选。但是集群环境中Quartz采用API的方式对任务进行管理，从而可以避免上述问题，但是同样存在以下问题：

- 问题一：调用API的方式操作任务，不人性化；
- 问题二：需要持久化业务Quartz.JobBean到底层数据表中，系统侵入性相当严重。
- 问题三：调度逻辑和Quartz.JobBean耦合在同一个项目中，这将导致一个问题，在调度任务数量逐渐增多，同时调度任务逻辑逐渐加重的情况下，此时调度系统的性能将大大受限；
- 问题四：quartz底层以“抢占式”获取DB锁并由抢占成功节点负责运行任务，会导致节点负载悬殊非常大；而XXL-JOB通过执行器实现“协同分配式”运行任务，充分发挥集群优势，负载各节点均衡。

XXL-JOB弥补了quartz的上述不足之处。

XXL-JOB 的架构设计如下图所示：



XXL-JOB架构图 v2.1.0

从上图可以看出，XXL-JOB 由 **调度中心** 和 **执行器** 两大部分组成。调度中心主要负责任务管理、执行器管理以及日志管理。执行器主要是接收调度信号并处理。另外，调度中心进行任务调度时，是通过自研 RPC 来实现的。

不同于 Elastic-Job 的去中心化设计，XXL-JOB 的这种设计也被称为中心化设计（调度中心调度多个执行器执行任务）。

和 Quartz 类似，XXL-JOB 也是基于数据库锁调度任务，存在性能瓶颈。不过，一般在任务量不是特别大的情况下，没有什么影响的，可以满足绝大部分公司的要求。

不要被 XXL-JOB 的架构图给吓着了，实际上，我们要用 XXL-JOB 的话，只需要重写 `IJobHandler` 自定义任务执行逻辑就可以了，非常易用！

```

@JobHandler(value="myApiJobHandler")
@Component
public class MyApiJobHandler extends IJobHandler {

    @Override
    public ReturnT<String> execute(String param) throws Exception {
        //.....
        return ReturnT.SUCCESS;
    }
}

```

还可以直接基于注解定义任务。

```

@XxlJob("myAnnotationJobHandler")
public ReturnT<String> myAnnotationJobHandler(String param) throws Exception {
    //.....
    return ReturnT.SUCCESS;
}

```

XXL

☰

注销

任务管理 任务调度中心

执行器

示例执行器

↓

JobHandler

搜索

+新增任务

调度列表

每页

10

↓

条记录

JobKey	描述	运行模式	Cron	负责人	状态	操作
1_35	测试任务04	GLUE模式(Python)	1 11 * * ? *	张三	ⓄNORMAL	执行 暂停 日志 编辑 GLUE 删除
1_34	测试任务03	GLUE模式(Shell)	1 11 * * ? *	张三	ⓄNORMAL	执行 暂停 日志 编辑 GLUE 删除
1_33	测试任务02	GLUE模式(Java)	1 11 * * ? *	张三	ⓄNORMAL	执行 暂停 日志 编辑 GLUE 删除
1_32	测试任务01	BEAN模式: demoJobHandler	1 11 * * ? *	张三	ⓄNORMAL	执行 暂停 日志 编辑 删除

第 1 页 (总共 1 页, 4 条记录)

上页

1

下页

Powered by XXL-JOB 1.7

Copyright © 2015-2017 [github](#) [oschina](#)

相关地址：

- Github 地址：<https://github.com/xuxueli/xxl-job/>。
- 官方介绍：<https://www.xuxueli.com/xxl-job/>。

优缺点总结：

- 优点：开箱即用（学习成本比较低）、与 Spring 集成、支持分布式、支持集群、内置了 UI 管理控制台。
- 缺点：不支持动态添加任务（如果一定想要动态创建任务也是支持的，参见：[xxl-job issue277](#)）。

PowerJob



非常值得关注的一个分布式任务调度框架，分布式任务调度领域的新星。目前，已经有很多公司接入比如 OPPO、京东、中通、思科。

这个框架的诞生也挺有意思的，PowerJob 的作者当时在阿里巴巴实习过，阿里巴巴那会使用的是内部自研的 SchedulerX（阿里云付费产品）。实习期满之后，PowerJob 的作者离开了阿里巴巴。想着说自研一个 SchedulerX，防止哪天 SchedulerX 满足不了需求，于是 PowerJob 就诞生了。

更多关于 PowerJob 的故事，小伙伴们可以去看看 PowerJob 作者的视频 [《我和我的任务调度中间件》](#)。简单点概括就是：“游戏没啥意思了，我要扛起了新一代分布式任务调度与计算框架的大旗！”。

由于 SchedulerX 属于人民币产品，我这里就不过多介绍。PowerJob 官方也对比过其和 Quartz、XXL-JOB 以及 SchedulerX。

	Quartz	xxl-job	SchedulerX 2.0	PowerJob
定时类型	CRON	CRON	CRON、固定频率、固定延迟、OpenAPI	CRON、固定频率、固定延迟、OpenAPI
任务类型	内置Java	内置Java、GLUE Java、Shell、Python等脚本	内置Java、外置Java（FatJar）、Shell、Python等脚本	内置Java、外置Java（容器）、Shell、Python等脚本
分布式计算	无	静态分片	MapReduce动态分片	MapReduce动态分片
在线任务治理	不支持	支持	支持	支持
日志白屏化	不支持	支持	不支持	支持
调度方式及性能	基于数据库锁，有性能瓶颈	基于数据库锁，有性能瓶颈	不详	无锁化设计，性能强劲无上限
报警监控	无	邮件	短信	WebHook、邮件、钉钉与自定义扩展
系统依赖	JDBC支持的关系型数据库（MySQL、Oracle...）	MySQL	人民币	任意Spring Data Jpa支持的关系型数据库（MySQL、Oracle...）
DAG工作流	不支持	不支持	支持	支持

总结

这篇文章中，我主要介绍了：

- **定时任务的相关概念**：为什么需要定时任务、定时任务中的核心角色、分布式定时任务。
- **定时任务的技术选型**：XXL-JOB 2015 年推出，已经经过了很多年的考验。XXL-JOB 轻量级，并且使用起来非常简单。虽然存在性能瓶颈，但是，在绝大多数情况下，对于企业的基本需求来说是没有影响的。PowerJob 属于分布式任务调度领域里的新星，其稳定性还有待继续考察。ElasticJob 由于在架构设计上是基于 Zookeeper，而 XXL-JOB 是基于数据库，性能方面的话，ElasticJob 略胜一筹。

这篇文章并没有介绍到实际使用，但是，并不代表实际使用不重要。我在写这篇文章之前，已经动手写过相应的 Demo。像 Quartz，我在大学那会就用过。不过，当时用的是 Spring。为了更好地体验，我自己又在 Spring Boot 上实际体验了一下。如果你并没有实际使用某个框架，就直接说它并不好用的话，是站不住脚的。

最后，这篇文章要感谢芳芳的帮助，写这篇文章的时候向芳芳询问过一些问题。推荐一篇芳芳写的偏实战类型的硬核文章：《[Spring Job? Quartz? XXL-Job? 年轻人才做选择，芳芳全莽~](#)》。-----

7. 分布式

[JavaGuide](#)：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！



[Github](#) |
[Gitee](#)

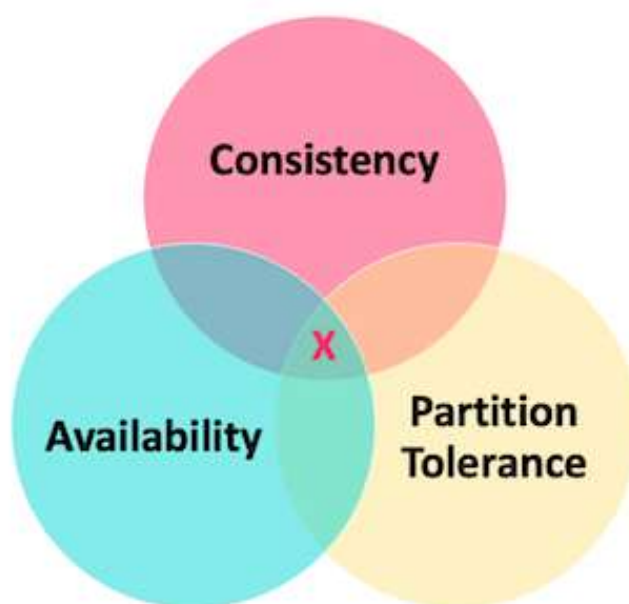
7.1 理论&算法&协议

什么是 CAP理论？

[CAP 理论/定理](#)起源于 2000年，由加州大学伯克利分校的Eric Brewer教授在分布式计算原理研讨会（PODC）上提出，因此 CAP定理又被称作 **布鲁尔定理（Brewer's theorem）**

2年后，麻省理工学院的Seth Gilbert和Nancy Lynch 发表了布鲁尔猜想的证明，CAP理论正式成为分布式领域的定理。

CAP 也就是 **Consistency**（一致性）、**Availability**（可用性）、**Partition Tolerance**（分区容错性）这三个单词首字母组合。



CAP 理论的提出者布鲁尔在提出 CAP 猜想的时候，并没有详细定义 **Consistency**、**Availability**、**Partition Tolerance** 三个单词的明确定义。

因此，对于 CAP 的民间解读有很多，一般比较被大家推荐的是下面 📌 这种版本的解读。

在理论计算机科学中，CAP 定理（CAP theorem）指出对于一个分布式系统来说，当设计读写操作时，只能同时满足以下三点中的两个：

- **一致性（Consistency）**：所有节点访问同一份最新的数据副本
- **可用性（Availability）**：非故障的节点在合理的时间内返回合理的响应（不是错误或者超时的响应）。
- **分区容错性（Partition tolerance）**：分布式系统出现网络分区的时候，仍然能够对外提供服务。

什么是网络分区？

分布式系统中，多个节点之前的网络本来是连通的，但是因为某些故障（比如部分节点网络出了问题）某些节点之间不连通了，整个网络就分成了几块区域，这就叫网络分区。



大部分人解释这一定律时，常常简单的表述为：“一致性、可用性、分区容忍性三者你只能同时达到其中两个，不可能同时达到”。实际上这是一个非常具有误导性质的说法，而且在 CAP 理论诞生 12 年之后，CAP 之父也在 2012 年重写了之前的论文。

当发生网络分区的时候，如果我们要继续服务，那么强一致性和可用性只能 2 选 1。也就是说当网络分区之后 P 是前提，决定了 P 之后才有 C 和 A 的选择。也就是说分区容错性（Partition tolerance）我们是必须要实现的。

简而言之就是：CAP 理论中分区容错性 P 是一定要满足的，在此基础上，只能满足可用性 A 或者一致性 C。

因此，分布式系统理论上不可能选择 **CA** 架构，只能选择 **CP** 或者 **AP** 架构。比如 ZooKeeper、HBase 就是 CP 架构，Cassandra、Eureka 就是 AP 架构，Nacos 不仅支持 CP 架构也支持 AP 架构。

为啥不可能选择 **CA** 架构呢？举个例子：若系统出现“分区”，系统中的某个节点在进行写操作。为了保证 C，必须要禁止其他节点的读写操作，这就和 A 发生冲突了。如果为了保证 A，其他节点的读写操作正常的话，那就和 C 发生冲突了。

选择 **CP** 还是 **AP** 的关键在于当前的业务场景，没有定论，比如对于需要确保强一致性的场景如银行一般会选择保证 **CP**。

另外，需要补充说明的一点是：如果网络分区正常的话（系统在绝大部分时候所处的状态），也就不需要保证 P 的时候，C 和 A 能够同时保证。

什么是 Base 理论？

BASE 理论起源于 2008 年，由 eBay 的架构师 Dan Pritchett 在 ACM 上发表。

BASE 是 **Basically Available**（基本可用）、**Soft-state**（软状态）和 **Eventually Consistent**（最终一致性）三个短语的缩写。BASE 理论是对 CAP 中一致性 C 和可用性 A 权衡的结果，其来源于对大规模互联网系统分布式实践的总结，是基于 CAP 定理逐步演化而来的，它大大降低了我们对系统的要求。

即使无法做到强一致性，但每个应用都可以根据自身业务特点，采用适当的方式来使系统达到最终一致性。

也就是牺牲数据的一致性来满足系统的高可用性，系统中一部分数据不可用或者不一致时，仍需要保持系统整体“主要可用”。

BASE 理论本质上是对 **CAP** 的延伸和补充，更具体地说，是对 **CAP** 中 **AP** 方案的一个补充。

为什么这样说呢？

CAP 理论这节我们也说过了：

如果系统没有发生“分区”的话，节点间的网络连接通信正常的话，也就不存在 P 了。这个时候，我们就可以同时保证 C 和 A 了。因此，如果系统发生“分区”，我们要考虑选择 **CP** 还是 **AP**。如果系统没有发生“分区”的话，我们要思考如何保证 **CA**。

因此，AP 方案只是在系统发生分区的时候放弃一致性，而不是永远放弃一致性。在分区故障恢复后，系统应该达到最终一致性。这一点其实就是 BASE 理论延伸的地方。

聊聊你对 Paxos 算法的了解？

Paxos 算法是兰伯特在 1990 年提出了一种分布式系统共识算法。

兰伯特当时提出的 Paxos 算法主要包含 2 个部分：

- **Basic Paxos 算法**：描述的是多节点之间如何就某个值(提案 Value)达成共识。
- **Multi-Paxos 思想**：描述的是执行多个 Basic Paxos 实例，就一系列值达成共识。Multi-Paxos 说白了就是执行多次 Basic Paxos，核心还是 Basic Paxos。

由于 Paxos 算法在国际上被公认的非常难以理解和实现，因此不断有人尝试简化这一算法。到了 2013 年才诞生了一个比 Paxos 算法更易理解和实现的共识算法—[Raft 算法](#)。更具体点来说，Raft 是 Multi-Paxos 的一个变种，其简化了 Multi-Paxos 的思想，变得更容易被理解以及工程实现。

关于 Paxos 算法的详细介绍，请看[Paxos 算法](#)这篇文章。

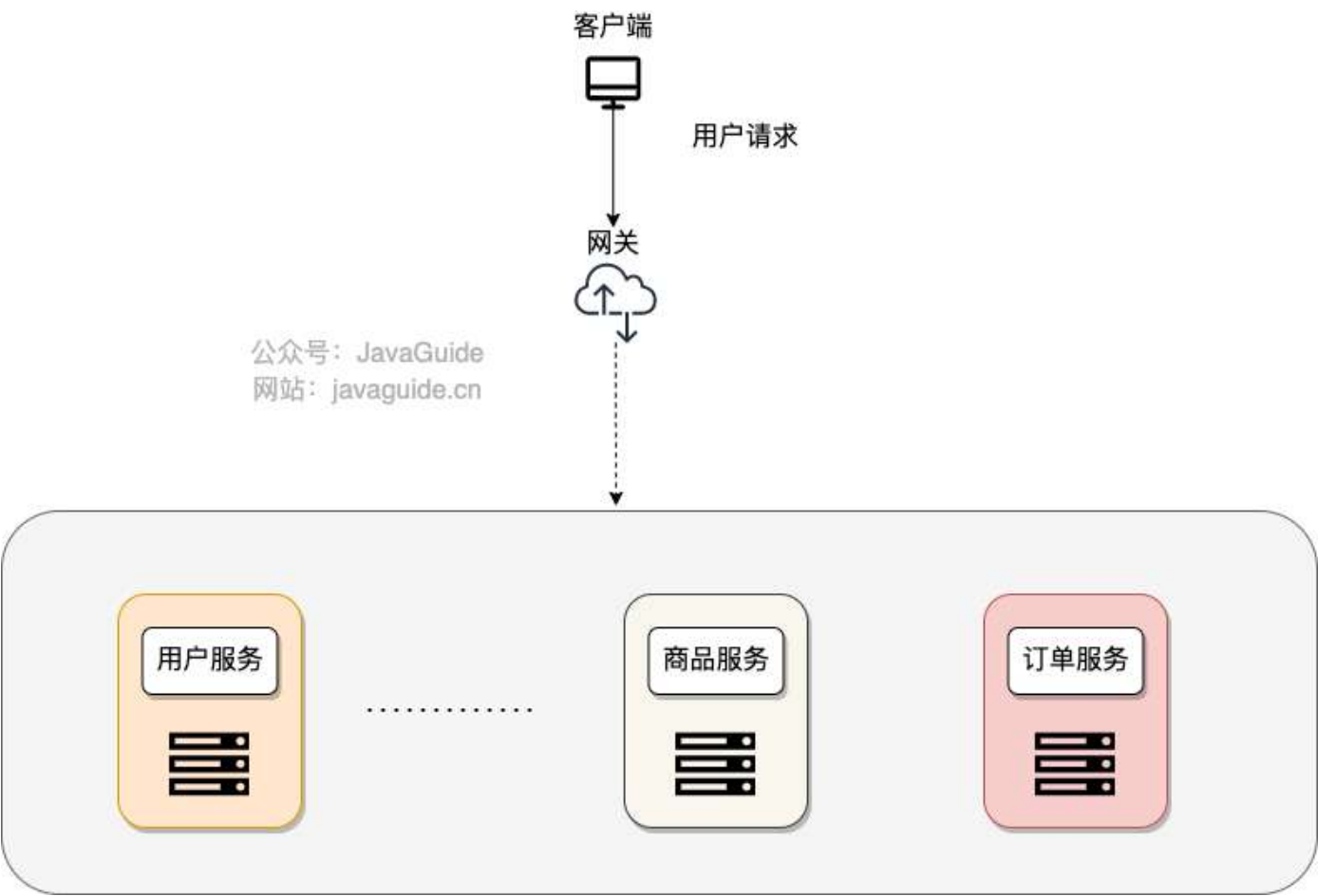
聊聊你对 Raft 算法的了解？

[Raft 算法](#)

7.2 网关

何为网关？为什么要网关？

微服务背景下，一个系统被拆分为多个服务，但是像安全认证，流量控制，日志，监控等功能是每个服务都需要的，没有网关的话，我们就需要在每个服务中单独实现，这使得我们做了很多重复的事情并且没有一个全局的视图来统一管理这些功能。



一般情况下，网关可以为我们提供请求转发、安全认证（身份/权限认证）、流量控制、负载均衡、降级熔断、日志、监控等功能。

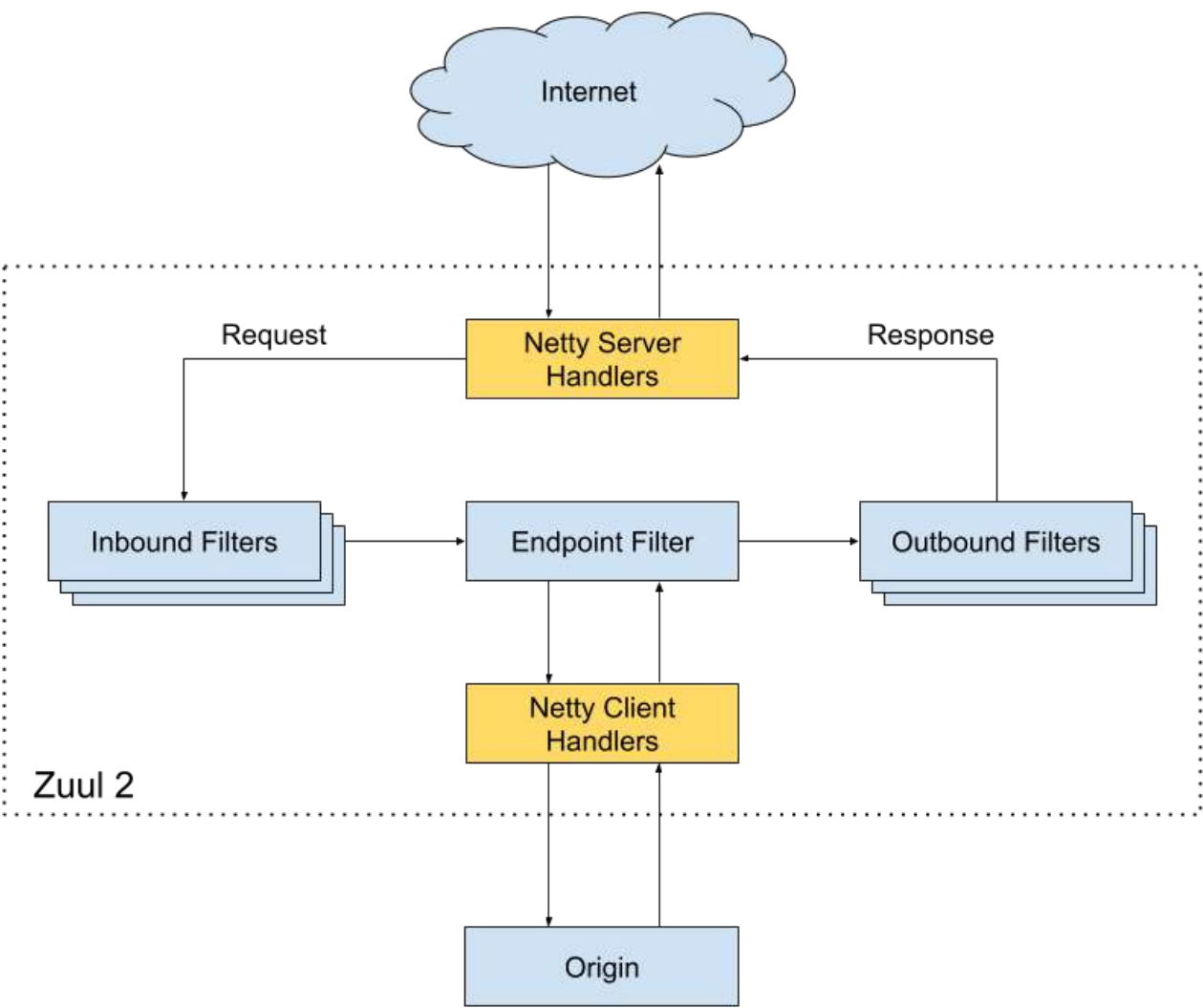
上面介绍了这么多功能，实际上，网关主要做了一件事情：**请求过滤**。

有哪些常见的网关系统？

Netflix Zuul

Zuul 是 Netflix 开发的一款提供动态路由、监控、弹性、安全的网关服务。

Zuul 主要通过过滤器（类似于 AOP）来过滤请求，从而实现网关必备的各种功能。



我们可以自定义过滤器来处理请求，并且，Zuul 生态本身就有很多现成的过滤器供我们使用。就比如限流可以直接用国外朋友写的 [spring-cloud-zuul-ratelimit](#) (这里只是举例说明，一般是配合 hystrix 来做限流)：

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>
<dependency>
  <groupId>com.marcosbarbero.cloud</groupId>
  <artifactId>spring-cloud-zuul-ratelimit</artifactId>
  <version>2.2.0.RELEASE</version>
</dependency>
```

Zuul 1.x 基于同步 IO，性能较差。Zuul 2.x 基于 Netty 实现了异步 IO，性能得到了大幅改进。

- Github 地址：<https://github.com/Netflix/zuul>
- 官方 Wiki：<https://github.com/Netflix/zuul/wiki>

Spring Cloud Gateway

SpringCloud Gateway 属于 Spring Cloud 生态系统中的网关，其诞生的目标是为了替代老牌网关 **Zuul**。准确点来说，应该是 Zuul 1.x。SpringCloud Gateway 起步要比 Zuul 2.x 更早。

为了提升网关的性能，SpringCloud Gateway 基于 Spring WebFlux。Spring WebFlux 使用 Reactor 库来实现响应式编程模型，底层基于 Netty 实现异步 IO。

Spring Cloud Gateway 的目标，不仅提供统一的路由方式，并且基于 Filter 链的方式提供了网关基本的功能，例如：安全，监控/指标，和限流。

Spring Cloud Gateway 和 Zuul 2.x 的差别不大，也是通过过滤器来处理请求。不过，目前更加推荐使用 Spring Cloud Gateway 而非 Zuul，Spring Cloud 生态对其支持更加友好。

- Github 地址：<https://github.com/spring-cloud/spring-cloud-gateway>
- 官网：<https://spring.io/projects/spring-cloud-gateway>

Kong

Kong 是一款基于 [OpenResty](#) 的高性能、云原生、可扩展的网关系统。

OpenResty 是一个基于 Nginx 与 Lua 的高性能 Web 平台，其内部集成了大量精良的 Lua 库、第三方模块以及大多数的依赖项。用于方便地搭建能够处理超高并发、扩展性极高的动态 Web 应用、Web 服务和动态网关。

Kong 提供了插件机制来扩展其功能。比如、在服务上启用 Zipkin 插件

```
$ curl -X POST http://kong:8001/services/{service}/plugins \
  --data "name=zipkin" \
  --data "config.http_endpoint=http://your.zipkin.collector:9411/api/v2/spans" \
  --data "config.sample_ratio=0.001"
```

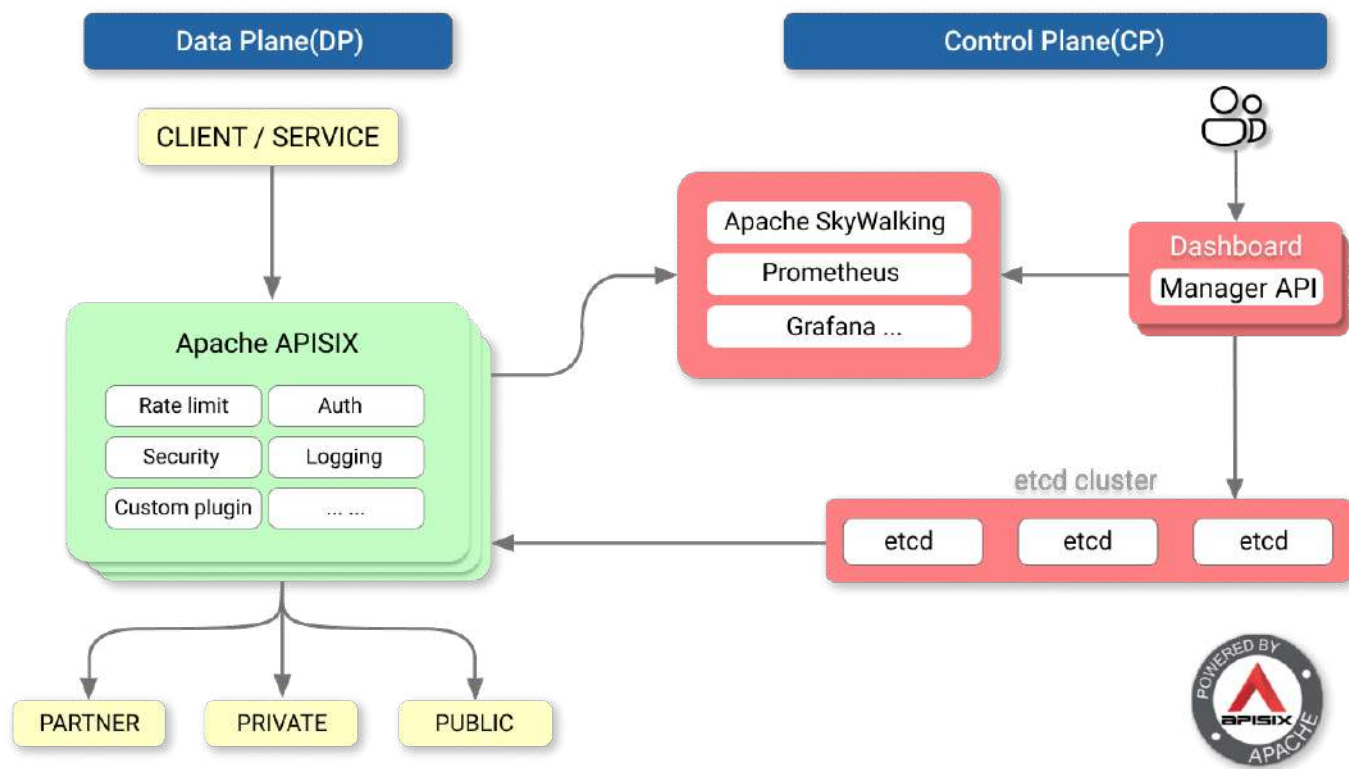
- Github 地址： <https://github.com/Kong/kong>
- 官网地址： <https://konghq.com/kong>

APISIX

APISIX 是一款基于 Nginx 和 etcd 的高性能、云原生、可扩展的网关系统。

*etcd*是使用 Go 语言开发的一个开源的、高可用的分布式 key-value 存储系统，使用 Raft 协议做分布式共识。

与传统 API 网关相比，APISIX 具有动态路由和插件热加载，特别适合微服务系统下的 API 管理。并且，APISIX 与 SkyWalking（分布式链路追踪系统）、Zipkin（分布式链路追踪系统）、Prometheus（监控系统）等 DevOps 生态工具对接都十分方便。



作为 NGINX 和 Kong 的替代项目，APISIX 目前已经是 Apache 顶级开源项目，并且是最快毕业的国产开源项目。国内目前已经有很多知名企业（比如金山、有赞、爱奇艺、腾讯、贝壳）使用 APISIX 处理核心的业务流量。

根据官网介绍：“APISIX 已经生产可用，功能、性能、架构全面优于 Kong”。

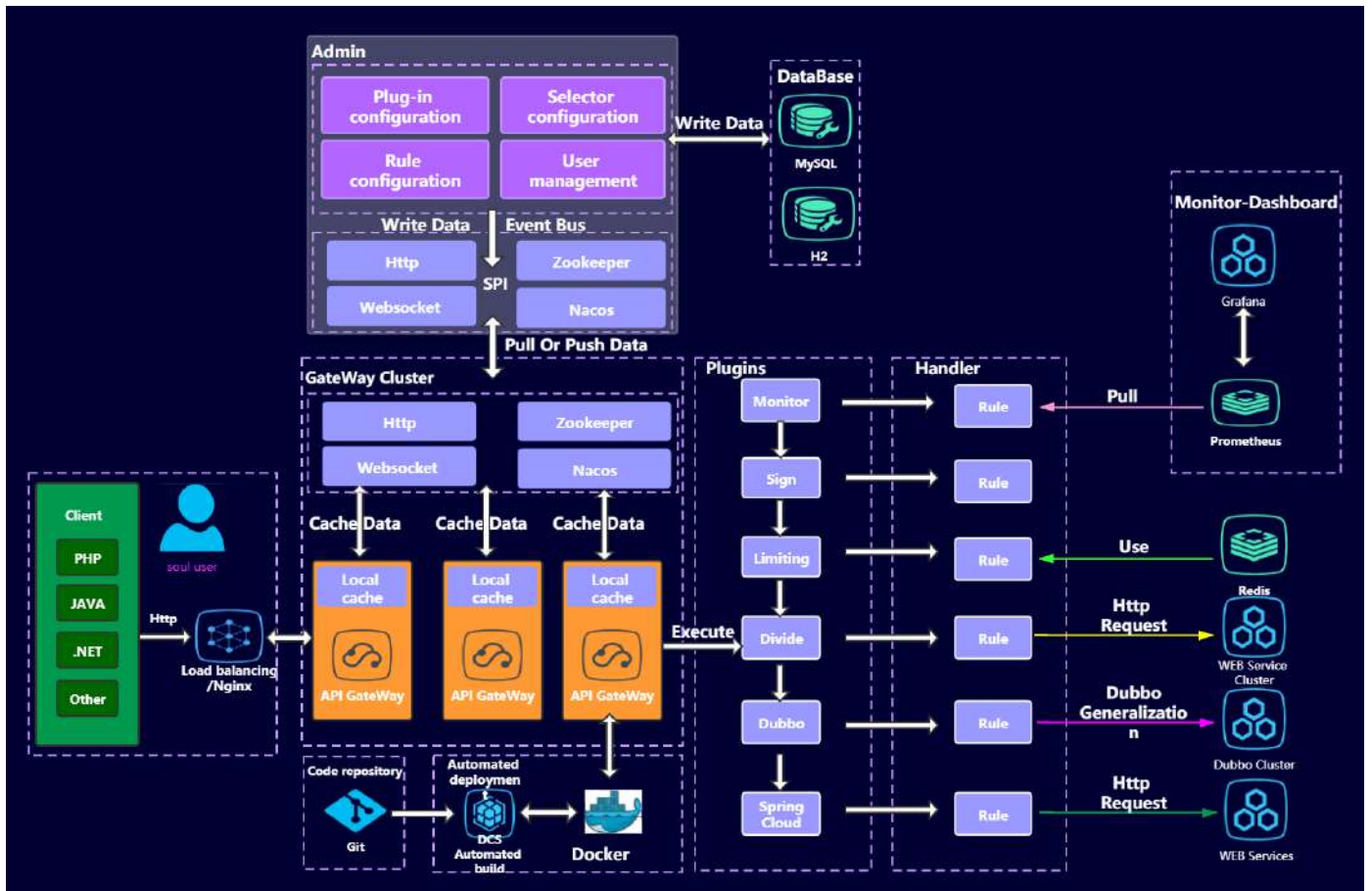
- Github 地址：<https://github.com/apache/apisix>
- 官网地址：<https://apisix.apache.org/zh/>

相关阅读：

- [有了 NGINX 和 Kong，为什么还需要 Apache APISIX](#)
- [APISIX 技术博客](#)
- [APISIX 用户案例](#)

Shenyu

Shenyu 是一款基于 WebFlux 的可扩展、高性能、响应式网关，Apache 顶级开源项目。



ShenYu 通过插件扩展功能，插件是 ShenYu 的灵魂，并且插件也是可扩展和热插拔的。不同的插件实现不同的功能。ShenYu 自带了诸如限流、熔断、转发、重写、重定向、和路由监控等插件。

- Github 地址：<https://github.com/apache/incubator-shenyu>
- 官网地址：<https://shenyu.apache.org/>

7.3 分布式 id

分布式 ID 介绍

什么是 ID?

日常开发中，我们需要对系统中的各种数据使用 ID 唯一表示，比如用户 ID 对应且仅对应一个人，商品 ID 对应且仅对应一件商品，订单 ID 对应且仅对应一个订单。

我们现实生活中也有各种 ID，比如身份证 ID 对应且仅对应一个人、地址 ID 对应且仅对应

简单来说，ID 就是数据的唯一标识。

什么是分布式 ID?

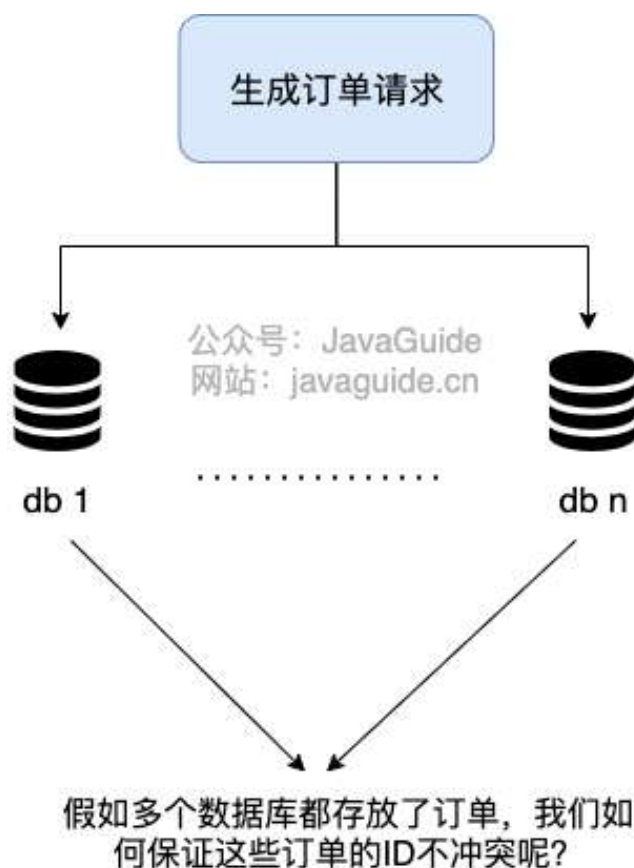
分布式 ID 是分布式系统下的 ID。分布式 ID 不存在与现实生活中，属于计算机系统中的一个概念。

我简单举一个分库分表的例子。

我司的一个项目，使用的是单机 MySQL。但是，没想到的是，项目上线一个月之后，随着使用人数越来越多，整个系统的数据量将越来越大。单机 MySQL 已经没办法支撑了，需要进行分库分表（推荐 Sharding-JDBC）。

在分库之后，数据遍布在不同服务器上的数据库，数据库的自增主键已经没办法满足生成的主键唯一了。我们如何为不同的数据节点生成全局唯一主键呢？

这个时候就需要生成分布式 ID 了。



分布式 ID 需要满足哪些要求?

分布式 ID 需要满足哪些要求？



分布式 ID 作为分布式系统中必不可少的一环，很多地方都要用到分布式 ID。

一个最基本的分布式 ID 需要满足下面这些要求：

- **全局唯一**：ID 的全局唯一性肯定是首先要满足的！
- **高性能**：分布式 ID 的生成速度要快，对本地资源消耗要小。
- **高可用**：生成分布式 ID 的服务要保证可用性无限接近于 100%。
- **方便易用**：拿来即用，使用方便，快速接入！

除了这些之外，一个比较好的分布式 ID 还应保证：

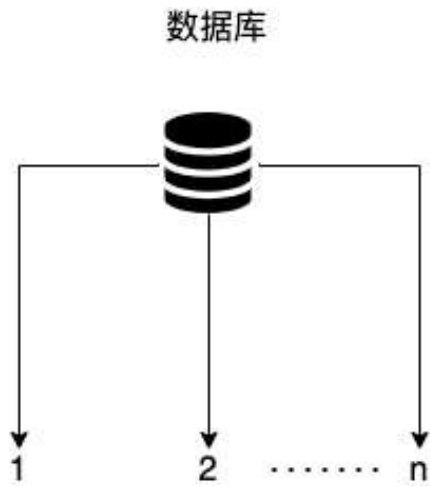
- **安全**：ID 中不包含敏感信息。
- **有序递增**：如果要把 ID 存放在数据库的话，ID 的有序性可以提升数据库写入速度。并且，很多时候，我们还很有可能会直接通过 ID 来进行排序。
- **有具体的业务含义**：生成的 ID 如果能有具体的业务含义，可以让定位问题以及开发更透明化（通过 ID 就能确定是哪个业务）。
- **独立部署**：也就是分布式系统单独有一个发号器服务，专门用来生成分布式 ID。这样就生成 ID 的服务可以和业务相关的服务解耦。不过，这样同样带来了网络调用消耗增加的问题。总的来说，如果需要用到分布式 ID 的场景比较多的话，独立部署的发号器服务还是很有必要的。

分布式 ID 常见解决方案有哪些？

数据库

数据库主键自增

这种方式就比较简单直白了，就是通过关系型数据库的自增主键产生来唯一的 ID。



以 MySQL 举例，我们通过下面的方式即可。

1. 创建一个数据库表。

```
CREATE TABLE `sequence_id` (  
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,  
  `stub` char(10) NOT NULL DEFAULT '',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `stub` (`stub`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

`stub` 字段无意义，只是为了占位，便于我们插入或者修改数据。并且，给 `stub` 字段创建了唯一索引，保证其唯一性。

2. 通过 `replace into` 来插入数据。

```
BEGIN;  
REPLACE INTO sequence_id (stub) VALUES ('stub');  
SELECT LAST_INSERT_ID();  
COMMIT;
```

插入数据这里，我们没有使用 `insert into` 而是使用 `replace into` 来插入数据，具体步骤是这样的：

1) 第一步： 尝试把数据插入到表中。

2) 第二步： 如果主键或唯一索引字段出现重复数据错误而插入失败时，先从表中删除含有重复关键字值的冲突行，然后再次尝试把数据插入到表中。

这种方式的优缺点也比较明显：

- **优点：** 实现起来比较简单、ID 有序递增、存储消耗空间小
- **缺点：** 支持的并发量不大、存在数据库单点问题（可以使用数据库集群解决，不过增加了复杂度）、ID 没有具体业务含义、安全问题（比如根据订单 ID 的递增规律就能推算出每天的订单量，商业机密啊！）、每次获取 ID 都要访问一次数据库（增加了对数据库的压力，获取速度也慢）

数据库号段模式

数据库主键自增这种模式，每次获取 ID 都要访问一次数据库，ID 需求比较大的时候，肯定是不行的。

如果我们可以批量获取，然后存在在内存里面，需要用到的时候，直接从内存里面拿就舒服了！这也就是我们说的 **基于数据库的号段模式来生成分布式 ID**。

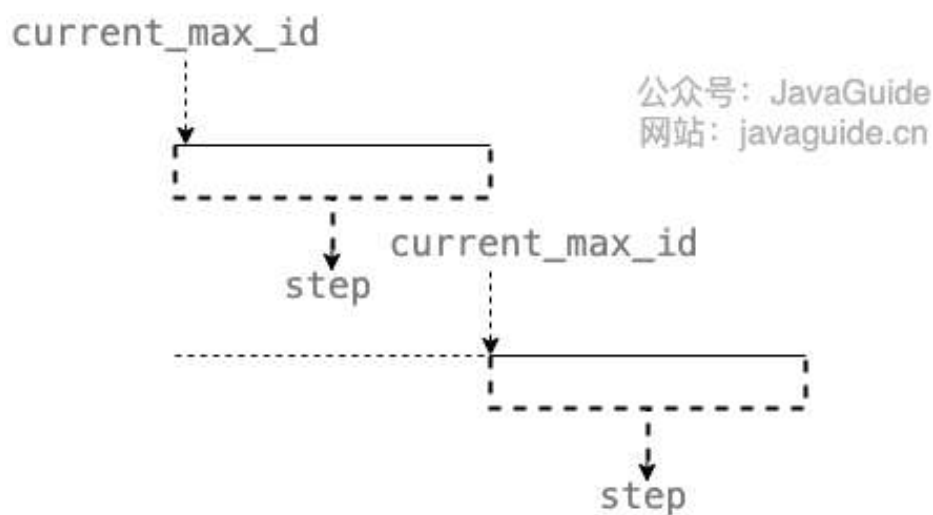
数据库的号段模式也是目前比较主流的一种分布式 ID 生成方式。像滴滴开源的 [Tinyid](#) 就是基于这种方式来做的。不过，TinyId 使用了双号段缓存、增加多 db 支持等方式来进一步优化。

以 MySQL 举例，我们通过下面的方式即可。

1. 创建一个数据库表。

```
CREATE TABLE `sequence_id_generator` (
  `id` int(10) NOT NULL,
  `current_max_id` bigint(20) NOT NULL COMMENT '当前最大id',
  `step` int(10) NOT NULL COMMENT '号段的长度',
  `version` int(20) NOT NULL COMMENT '版本号',
  `biz_type` int(20) NOT NULL COMMENT '业务类型',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

`current_max_id` 字段和 `step` 字段主要用于获取批量 ID，获取的批量 id 为：`current_max_id ~ current_max_id+step`。



`version` 字段主要用于解决并发问题（乐观锁），`biz_type` 主要用于表示业务类型。

2.先插入一行数据。

```
INSERT INTO `sequence_id_generator` (`id`, `current_max_id`, `step`, `version`,
`biz_type`)
VALUES
(1, 0, 100, 0, 101);
```

3.通过 **SELECT** 获取指定业务下的批量唯一 ID

```
SELECT `current_max_id`, `step`, `version` FROM `sequence_id_generator` where  
`biz_type` = 101
```

结果：

```
id  current_max_id  step  version  biz_type  
1  0  100  0  101
```

4.不够用的话，更新之后重新 SELECT 即可。

```
UPDATE sequence_id_generator SET current_max_id = 0+100, version=version+1 WHERE  
version = 0 AND `biz_type` = 101  
SELECT `current_max_id`, `step`, `version` FROM `sequence_id_generator` where  
`biz_type` = 101
```

结果：

```
id  current_max_id  step  version  biz_type  
1  100  100  1  101
```

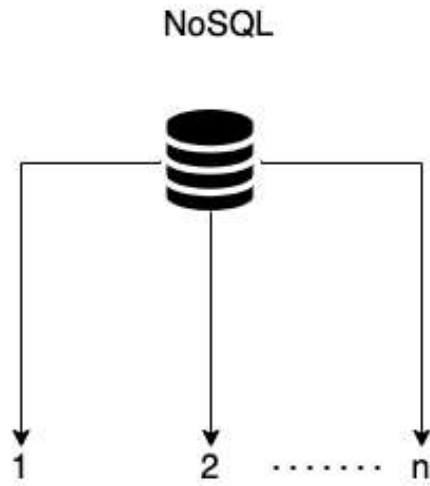
相比于数据库主键自增的方式，数据库的号段模式对于数据库的访问次数更少，数据库压力更小。

另外，为了避免单点问题，你可以从使用主从模式来提高可用性。

数据库号段模式的优缺点：

- 优点：ID 有序递增、存储消耗空间小
- 缺点：存在数据库单点问题（可以使用数据库集群解决，不过增加了复杂度）、ID 没有具体业务含义、安全问题（比如根据订单 ID 的递增规律就能推算出每天的订单量，商业机密啊！）

NoSQL



一般情况下，NoSQL 方案使用 Redis 多一些。我们通过 Redis 的 `incr` 命令即可实现对 id 原子顺序递增。

```
127.0.0.1:6379> set sequence_id_biz_type 1
OK
127.0.0.1:6379> incr sequence_id_biz_type
(integer) 2
127.0.0.1:6379> get sequence_id_biz_type
"2"
```

为了提高可用性和并发，我们可以使用 Redis Cluster。Redis Cluster 是 Redis 官方提供的 Redis 集群解决方案（3.0+版本）。

除了 Redis Cluster 之外，你也可以使用开源的 Redis 集群方案 [Codis](#)（大规模集群比如上百个节点的时候比较推荐）。

除了高可用和并发之外，我们知道 Redis 基于内存，我们需要持久化数据，避免重启机器或者机器故障后数据丢失。Redis 支持两种不同的持久化方式：**快照（snapshotting, RDB）**、**只追加文件（append-only file, AOF）**。并且，Redis 4.0 开始支持 **RDB 和 AOF 的混合持久化**（默认关闭，可以通过配置项 `aof-use-rdb-preamble` 开启）。

关于 Redis 持久化，我这里就不过多介绍。不了解这部分内容的小伙伴，可以看看 [JavaGuide 对于 Redis 知识点的总结](#)。

Redis 方案的优缺点：

- 优点：性能不错并且生成的 ID 是有序递增的
- 缺点：和数据库主键自增方案的缺点类似

除了 Redis 之外，MongoDB ObjectId 经常也会被拿来当做分布式 ID 的解决方案。

BSON ObjectId Specification

A BSON ObjectId is a 12-byte value consisting of a 4-byte timestamp (seconds since epoch), a 3-byte machine id, a 2-byte process id, and a 3-byte counter. Note that the timestamp and counter fields must be stored big endian unlike the rest of BSON. This is because they are compared byte-by-byte and we want to ensure a mostly increasing order. The format:

0	1	2	3	4	5	6	7	8	9	10	11
time				machine			pid	inc			

MongoDB ObjectId 一共需要 12 个字节存储：

- 0~3：时间戳
- 3~6：代表机器 ID
- 7~8：机器进程 ID
- 9~11：自增值

MongoDB 方案的优缺点：

- 优点：性能不错并且生成的 ID 是有序递增的
- 缺点：需要解决重复 ID 问题（当机器时间不对的情况下，可能导致会产生重复 ID）、有安全性问题（ID 生成有规律性）

算法

UUID

UUID 是 Universally Unique Identifier（通用唯一标识符）的缩写。UUID 包含 32 个 16 进制数字（8-4-4-4-12）。

JDK 就提供了现成的生成 UUID 的方法，一行代码就行了。

```
//输出示例：cb4a9ede-fa5e-4585-b9bb-d60bce986eaa  
UUID.randomUUID()
```

[RFC 4122](#) 中关于 UUID 的示例是这样的：

Version: 4 (can be 1, 2, 3, 4, or 5)

0a87341d-e931-4b53-B2d2-31e97b184f54

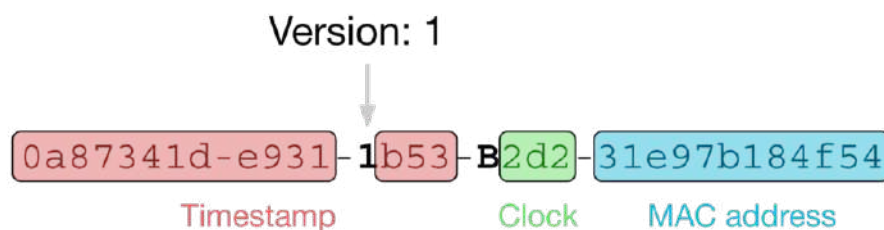
Variant: RFC compliant (10xx binary = 8, 9, A, or B)

我们这里重点关注一下这个 Version(版本), 不同的版本对应的 UUID 的生成规则是不同的。

5 种不同的 Version(版本)值分别对应的含义 (参考[维基百科对于 UUID 的介绍](#)) :

- 版本 1 : UUID 是根据时间和节点 ID (通常是 MAC 地址) 生成;
- 版本 2 : UUID 是根据标识符 (通常是组或用户 ID)、时间和节点 ID 生成;
- 版本 3、版本 5 : 版本 5 - 确定性 UUID 通过散列 (hashing) 名字空间 (namespace) 标识符和名称生成;
- 版本 4 : UUID 使用[随机性](#)或[伪随机性](#)生成。

下面是 Version 1 版本下生成的 UUID 的示例:



JDK 中通过 `UUID` 的 `randomUUID()` 方法生成的 UUID 的版本默认为 4。

```
UUID uuid = UUID.randomUUID();  
int version = uuid.version();// 4
```

另外, Variant(变体)也有 4 种不同的值, 这种值分别对应不同的含义。这里就不介绍了, 貌似平时也不怎么需要关注。

需要用的时候, 去看看[维基百科对于 UUID 的 Variant\(变体\) 相关的介绍](#)即可。

从上面的介绍中可以看出，UUID 可以保证唯一性，因为其生成规则包括 MAC 地址、时间戳、名字空间（Namespace）、随机或伪随机数、时序等元素，计算机基于这些规则生成的 UUID 是肯定不会重复的。

虽然，UUID 可以做到全局唯一性，但是，我们一般很少会使用它。

比如使用 UUID 作为 MySQL 数据库主键的时候就非常不合适：

- 数据库主键要尽量越短越好，而 UUID 的消耗的存储空间比较大（32 个字符串，128 位）。
- UUID 是无顺序的，InnoDB 引擎下，数据库主键的无序性会严重影响数据库性能。

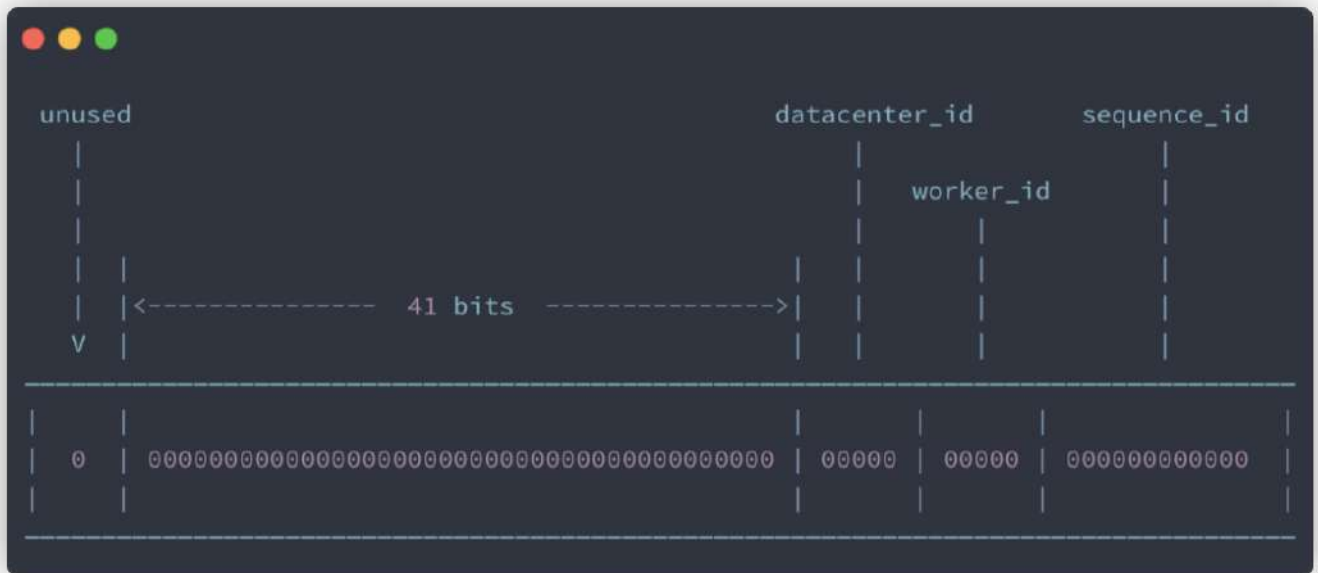
最后，我们再简单分析一下 **UUID 的优缺点**（面试的时候可能会被问到的哦！）：

- **优点：** 生成速度比较快、简单易用
- **缺点：** 存储消耗空间大（32 个字符串，128 位）、不安全（基于 MAC 地址生成 UUID 的算法会造成 MAC 地址泄露）、无序（非自增）、没有具体业务含义、需要解决重复 ID 问题（当机器时间不对的情况下，可能导致会产生重复 ID）

Snowflake(雪花算法)

Snowflake 是 Twitter 开源的分布式 ID 生成算法。Snowflake 由 64 bit 的二进制数字组成，这 64bit 的二进制被分成了几部分，每一部分存储的数据都有特定的含义：

- **第 0 位：** 符号位（标识正负），始终为 0，没有用，不用管。
- **第 1~41 位：** 一共 41 位，用来表示时间戳，单位是毫秒，可以支撑 2^{41} 毫秒（约 69 年）
- **第 42~52 位：** 一共 10 位，一般来说，前 5 位表示机房 ID，后 5 位表示机器 ID（实际项目中可以根据实际情况调整）。这样就可以区分不同集群/机房的节点。
- **第 53~64 位：** 一共 12 位，用来表示序列号。序列号为自增值，代表单台机器每毫秒能够产生的最大 ID 数($2^{12} = 4096$)，也就是说单台机器每毫秒最多可以生成 4096 个唯一 ID。



如果你想要使用 Snowflake 算法的话，一般不需要你自己再造轮子。有很多基于 Snowflake 算法的开源实现比如美团的 Leaf、百度的 UidGenerator，并且这些开源实现对原有的 Snowflake 算法进行了优化。

另外，在实际项目中，我们一般也会对 Snowflake 算法进行改造，最常见的就是在 Snowflake 算法生成的 ID 中加入业务类型信息。

我们再来看看 Snowflake 算法的优缺点：

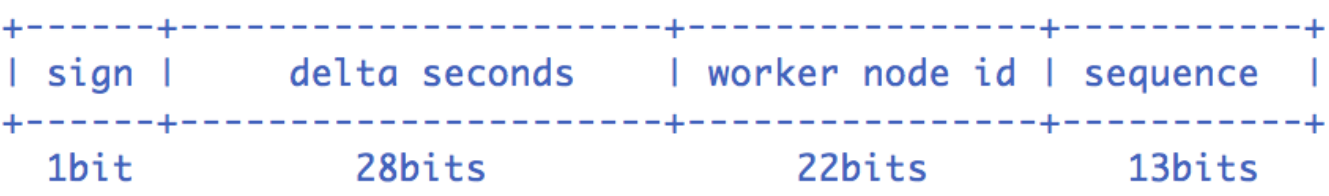
- **优点：** 生成速度比较快、生成的 ID 有序递增、比较灵活（可以对 Snowflake 算法进行简单的改造比如加入业务 ID）
- **缺点：** 需要解决重复 ID 问题（依赖时间，当机器时间不对的情况下，可能导致会产生重复 ID）。

开源框架

UidGenerator(百度)

UidGenerator 是百度开源的一款基于 Snowflake(雪花算法)的唯一 ID 生成器。

不过，UidGenerator 对 Snowflake(雪花算法)进行了改进，生成的唯一 ID 组成如下。



可以看出，和原始 Snowflake(雪花算法)生成的唯一 ID 的组成不太一样。并且，上面这些参数我们都可以自定义。

UidGenerator 官方文档中的介绍如下：

UidGenerator

In English

UidGenerator是Java实现的, 基于Snowflake算法的唯一ID生成器。UidGenerator以组件形式工作在应用项目中, 支持自定义workerId位数和初始化策略, 从而适用于docker等虚拟化环境下实例自动重启、漂移等场景。在实现上, UidGenerator通过借用未来时间来解决sequence天然存在的并发限制; 采用RingBuffer来缓存已生成的UID, 并行化UID的生产和消费, 同时对CacheLine补齐, 避免了由RingBuffer带来的硬件级「伪共享」问题。最终单机QPS可达600万。

依赖版本: [Java8](#)及以上版本, [MySQL](#)(内置WorkerID分配器, 启动阶段通过DB进行分配; 如自定义实现, 则DB非必选依赖)

Snowflake算法

+-----+-----+-----+-----+

| sign | delta seconds | worker node id | sequence |

+-----+-----+-----+-----+

1bit 28bits 22bits 13bits

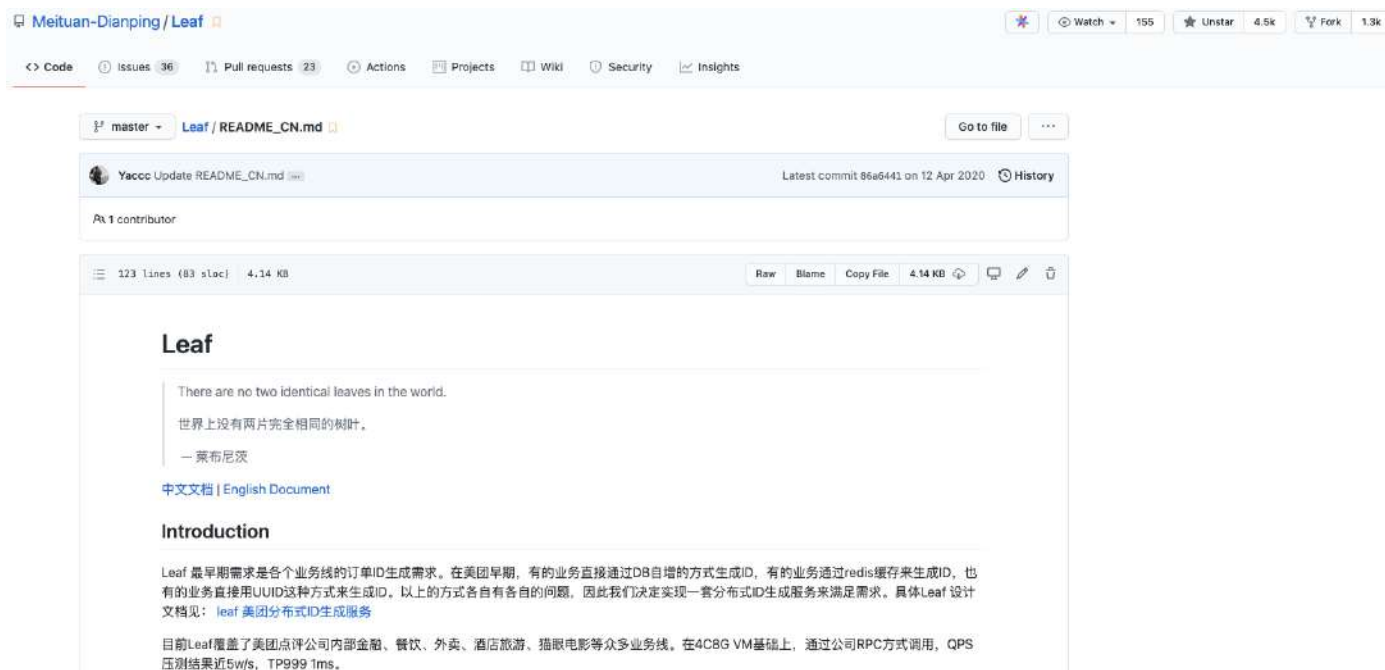
Snowflake算法描述: 指定机器 & 同一时刻 & 某一并发序列, 是唯一的。据此可生成一个64 bits的唯一ID (long)。默认采用上图字节分配方式:

- sign(1bit)
固定1bit符号标识, 即生成的UID为正数。
- delta seconds (28 bits)
当前时间, 相对于时间基点"2016-05-20"的增量值, 单位: 秒, 最多可支持约8.7年
- worker id (22 bits)
机器id, 最多可支持约420w次机器启动。内置实现为在启动时由数据库分配, 默认分配策略为用后即弃, 后续可提供复用策略。
- sequence (13 bits)
每秒下的并发序列, 13 bits可支持每秒8192个并发。

自 18 年后, UidGenerator 就基本没有再维护了, 我这里也过多介绍。想要进一步了解的朋友, 可以看看 [UidGenerator 的官方介绍](#)。

Leaf(美团)

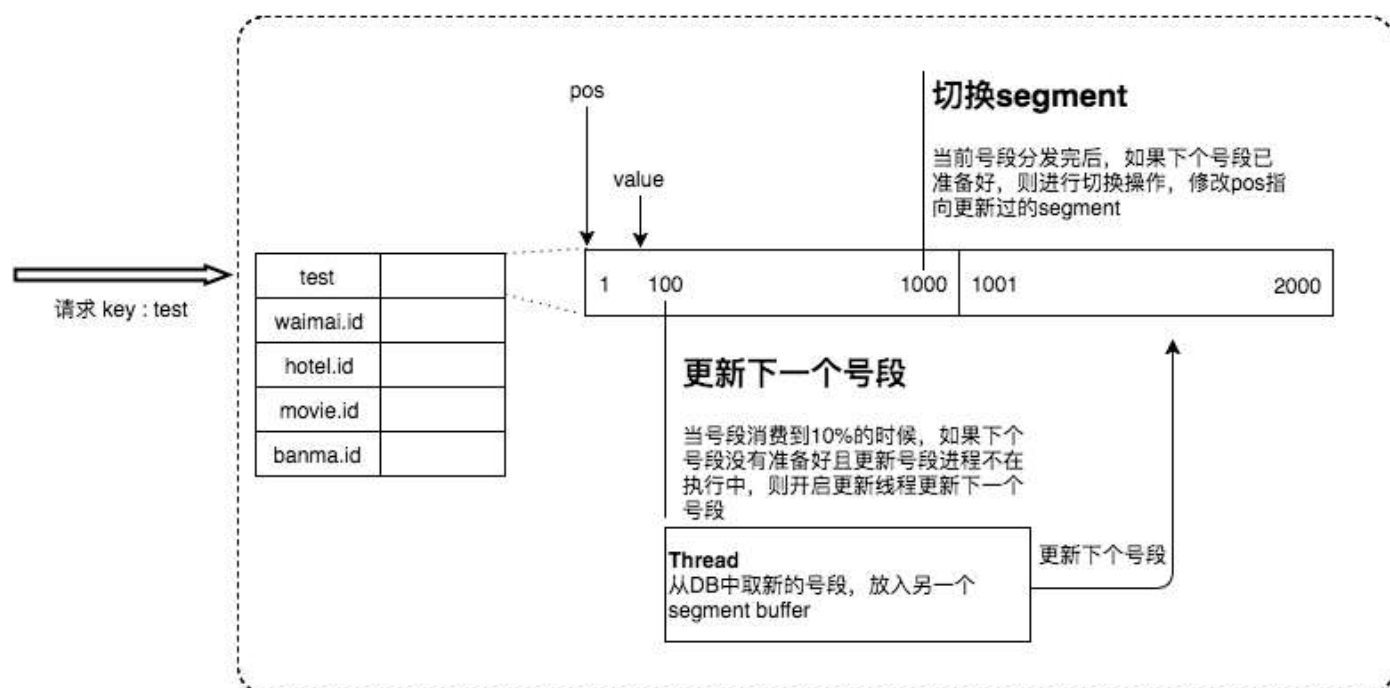
Leaf 是美团开源的一个分布式 ID 解决方案。这个项目的名字 Leaf（树叶）起源于德国哲学家、数学家莱布尼茨的一句话：“There are no two identical leaves in the world”（世界上没有两片相同的树叶）。这名字起得真心挺不错的，有点文艺青年那味了！



Leaf 提供了 **号段模式** 和 **Snowflake(雪花算法)** 这两种模式来生成分布式 ID。并且，它支持双号段，还解决了雪花 ID 系统时钟回拨问题。不过，时钟问题的解决需要弱依赖于 Zookeeper。

Leaf 的诞生主要是为了解决美团各个业务线生成分布式 ID 的方法多种多样以及不可靠的问题。

Leaf 对原有的号段模式进行改进，比如它这里增加了双号段避免获取 DB 在获取号段的时候阻塞请求获取 ID 的线程。简单来说，就是我一个号段还没用完之前，我自己就主动提前去获取下一个号段（图片来自于美团官方文章：《[Leaf——美团点评分布式 ID 生成系统](#)》）。



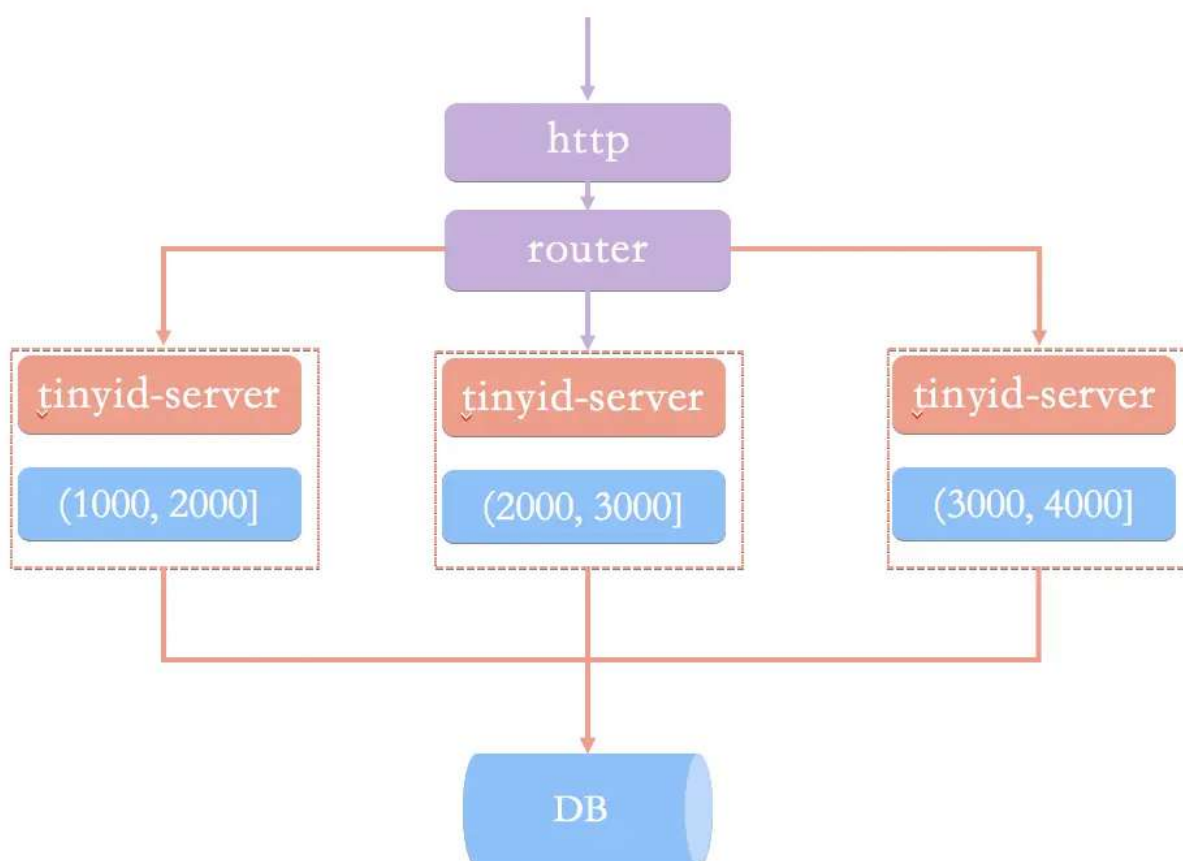
根据项目 README 介绍，在 4C8G VM 基础上，通过公司 RPC 方式调用，QPS 压测结果近 5w/s，TP999 1ms。

Tinyid(滴滴)

Tinyid 是滴滴开源的一款基于数据库号段模式的唯一 ID 生成器。

数据库号段模式的原理我们在上面已经介绍过了。Tinyid 有哪些亮点呢？

为了搞清楚这个问题，我们先来看看基于数据库号段模式的简单架构方案。（图片来自于 Tinyid 的官方 wiki: [《Tinyid 原理介绍》](#)）



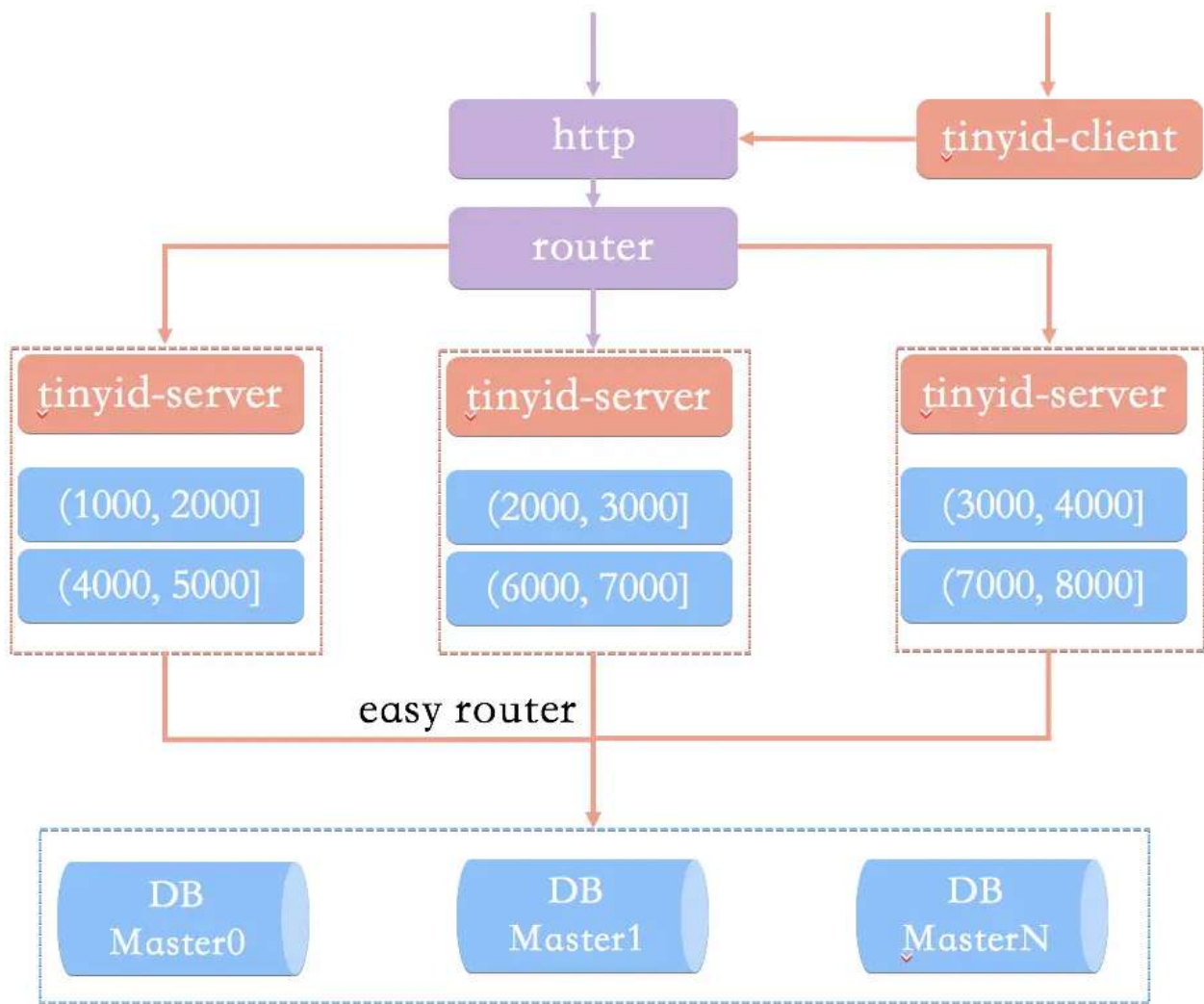
在这种架构模式下，我们通过 HTTP 请求向发号器服务申请唯一 ID。负载均衡 router 会把我们的请求送往其中的一台 tinyid-server。

这种方案有什么问题呢？在我看来（Tinyid 官方 wiki 也有介绍到），主要由下面这 2 个问题：

- 获取新号段的情况下，程序获取唯一 ID 的速度比较慢。
- 需要保证 DB 高可用，这个是比较麻烦且耗费资源的。

除此之外，HTTP 调用也存在网络开销。

Tinyid 的原理比较简单，其架构如下图所示：



相比于基于数据库号段模式的简单架构方案，Tinyid 方案主要做了下面这些优化：

- **双号段缓存**：为了避免在获取新号段的情况下，程序获取唯一 ID 的速度比较慢。Tinyid 中的号段在用到一定程度的时候，就会去异步加载下一个号段，保证内存中始终有可用号段。
- **增加多 db 支持**：支持多个 DB，并且，每个 DB 都能生成唯一 ID，提高了可用性。
- **增加 tinyid-client**：纯本地操作，无 HTTP 请求消耗，性能和可用性都有很大提升。

Tinyid 的优缺点这里就不分析了，结合数据库号段模式的优缺点和 Tinyid 的原理就能知道。

总结

通过这篇文章，我基本上已经把最常见的分布式 ID 生成方案都总结了一波。除了上面介绍的方式之外，像 ZooKeeper 这类中间件也可以帮助我们生成唯一 ID。没有银弹，一定要结合实际项目来选择最适合自己的方案。

7.4 分布式锁

网上有很多分布式锁相关的文章，写了一个相对简洁易懂的版本，针对面试和工作完全够用了。

什么是分布式锁？

对于单机多线程，在 Java 中，我们通常使用 `ReentrantLock` 这类 JDK 自带的 **本地锁** 来控制本地多个线程对本地共享资源的访问。对于分布式系统，我们通常使用 **分布式锁** 来控制多个服务对共享资源的访问。

一个最基本的分布式锁需要满足：

- **互斥**：任意一个时刻，锁只能被一个线程持有；
- **高可用**：锁服务是高可用的。并且，即使客户端的释放锁的代码逻辑出现问题，锁最终一定还是会被释放，不会影响其他线程对共享资源的访问。

通常情况下，我们一般会选择基于 Redis 或者 ZooKeeper 实现分布式锁，Redis 用的要更多一点，我这里也以 Redis 为例介绍分布式锁的实现。

基于 Redis 实现分布式锁

如何基于 Redis 实现一个最简易的分布式锁？

不论是实现锁还是分布式锁，核心都在于“互斥”。

在 Redis 中，`SETNX` 命令是可以帮助我们实现互斥。`SETNX` 即 **SET if Not eXists** (对应 Java 中的 `setIfAbsent` 方法)，如果 key 不存在的话，才会设置 key 的值。如果 key 已经存在，`SETNX` 啥也不做。

```
> SETNX lockKey uniqueValue
(integer) 1
> SETNX lockKey uniqueValue
(integer) 0
```

释放锁的话，直接通过 `DEL` 命令删除对应的 key 即可。

```
> DEL lockKey  
(integer) 1
```

为了误删到其他的锁，这里我们建议使用 Lua 脚本通过 key 对应的 value（唯一值）来判断。

选用 Lua 脚本是为了保证解锁操作的原子性。因为 Redis 在执行 Lua 脚本时，可以以原子性的方式执行，从而保证了锁释放操作的原子性。

```
// 释放锁时，先比较锁对应的 value 值是否相等，避免锁的误释放  
if redis.call("get",KEYS[1]) == ARGV[1] then  
    return redis.call("del",KEYS[1])  
else  
    return 0  
end
```

这是一种最简易的 Redis 分布式锁实现，实现方式比较简单，性能也很高效。不过，这种方式实现分布式锁存在一些问题。就比如应用程序遇到一些问题比如释放锁的逻辑突然挂掉，可能会导致锁无法被释放，进而造成共享资源无法再被其他线程/进程访问。

为什么要给锁设置一个过期时间？

为了避免锁无法被释放，我们可以想到的一个解决办法就是：给这个 key（也就是锁）设置一个过期时间。

```
127.0.0.1:6379> SET lockKey uniqueValue EX 3 NX  
OK
```

- **lockKey**：加锁的锁名；
- **uniqueValue**：能够唯一标示锁的随机字符串；
- **NX**：只有当 lockKey 对应的 key 值不存在的时候才能 SET 成功；
- **EX**：过期时间设置（秒为单位）EX 3 标示这个锁有一个 3 秒的自动过期时间。与 EX 对应的是 PX（毫秒为单位），这两个都是过期时间设置。

一定要保证设置指定 key 的值和过期时间是一个原子操作！！！ 不然的话，依然可能会出现锁无法被释放的问题。

这样确实可以解决问题，不过，这种解决办法同样存在漏洞：如果操作共享资源的时间大于过期时间，就会出现锁提前过期的问题，进而导致分布式锁直接失效。如果锁的超时时间设置过长，又会影响到性能。

你或许在想：如果操作共享资源的操作还未完成，锁过期时间能够自己续期就好了！

对于 Java 开发的小伙伴来说，已经有了现成的解决方案：**Redisson**。其他语言的解决方案，可以在 Redis 官方文档中找到，地址：<https://redis.io/topics/distlock>。

Distributed locks with Redis

Distributed locks are a very useful primitive in many environments where different processes must operate with shared resources in a mutually exclusive way.

There are a number of libraries and blog posts describing how to implement a DLM (Distributed Lock Manager) with Redis, but every library uses a different approach, and many use a simple approach with lower guarantees compared to what can be achieved with slightly more complex designs.

This page is an attempt to provide a more canonical algorithm to implement distributed locks with Redis. We propose an algorithm, called **Redlock**, which implements a DLM which we believe to be safer than the vanilla single instance approach. We hope that the community will analyze it, provide feedback, and use it as a starting point for the implementations or more complex or alternative designs.

Implementations

Before describing the algorithm, here are a few links to implementations already available that can be used for reference.

- [Redlock-rb](#) (Ruby implementation). There is also a [fork of Redlock-rb](#) that adds a gem for easy distribution and perhaps more.
- [Redlock-py](#) (Python implementation).
- [Pottery](#) (Python implementation).
- [AioRedlock](#) (Asyncio Python implementation).
- [Redlock-php](#) (PHP implementation).
- [PHPRedisMutex](#) (further PHP implementation)
- [cheprasov/php-redis-lock](#) (PHP library for locks)
- [rtckit/react-redlock](#) (Async PHP implementation)
- [Redsync](#) (Go implementation).
- [Redisson](#) (Java implementation).
- [Redis::DistLock](#) (Perl implementation).
- [Redlock-cpp](#) (C++ implementation).
- [Redlock-cs](#) (C#/.NET implementation).
- [RedLock.net](#) (C#/.NET implementation). Includes async and lock extension support.
- [ScarletLock](#) (C# .NET implementation with configurable datastore)
- [Redlock4Net](#) (C# .NET implementation)
- [node-redlock](#) (NodeJS implementation). Includes support for lock extension.

Safety and Liveness guarantees

We are going to model our design with just three properties that, from our point of view, are the minimum guarantees needed to use distributed locks in an effective way.

1. Safety property: Mutual exclusion. At any given moment, only one client can hold a lock.
2. Liveness property A: Deadlock free. Eventually it is always possible to acquire a lock, even if the client that locked a resource crashes or gets partitioned.
3. Liveness property B: Fault tolerance. As long as the majority of Redis nodes are up, clients are able to acquire and release locks.

1998 2001 2004 2007 2010 2013 2016 2019 2022

Redisson 是一个开源的 Java 语言 Redis 客户端，提供了很多开箱即用的功能，不仅仅包括多种分布式锁的实现。

Redisson 中的分布式锁自带自动续期机制，它提供了一个专门用来监控锁的 **Watch Dog**（看门狗），如果操作共享资源的还未完成的话，Watch Dog 会不断地延长锁的过期时间，进而保证锁不会因为超时而被释放。

我这里以 Redisson 的分布式可重入锁 `RLock` 为例来说明如何使用 Redisson 实现分布式锁：

```
// 1. 获取指定的分布式锁对象
RLock lock = redisson.getLock("lock");
// 2. 拿锁，具有 Watch Dog 自动续期机制
lock.lock();
// 3. 执行业务
...
// 4. 释放锁
lock.unlock();
```

可以看出，代码非常简洁直观。

如果使用 Redis 来实现分布式锁的话，还是比较推荐直接基于 Redisson 来做的。

Redis 如何解决集群情况下分布式锁的可靠性？

为了避免单点故障，生产环境下的 Redis 服务通常是集群化部署的。

Redis 集群下，上面介绍到的分布式锁的实现会存在一些问题。由于 Redis 集群数据同步到各个节点时是异步的，如果在 Redis 主节点获取到锁后，在没有同步到其他节点时，Redis 主节点宕机了，此时新的 Redis 主节点依然可以获取锁，所以多个应用服务就可以同时获取到锁。

针对这个问题，Redis 之父 antirez 设计了 [Redlock 算法](#) 来解决。

Redlock 算法的思想是让客户端向 Redis 集群中的多个独立的 Redis 实例依次请求申请加锁，如果客户端能够和半数以上的实例成功地完成加锁操作，那么我们就认为，客户端成功地获得分布式锁，否则加锁失败。

即使部分 Redis 节点出现问题，只要保证 Redis 集群中有半数以上的 Redis 节点可用，分布式锁服务就是正常的。

Redlock 是直接操作 Redis 节点的，并不是通过 Redis 集群操作的，这样才可以避免 Redis 集群主从切换导致的锁丢失问题。

Redlock 实现比较复杂，性能比较差，发生时钟变迁的情况下还存在安全性隐患。《数据密集型应用系统设计》一书的作者 Martin Kleppmann 曾经专门发文怼过 Redlock，他认为这是一个很差的分布式锁实现。感兴趣的朋友可以看看[Redis 锁从面试连环炮聊到神仙打架](#)这篇文章，有详细介绍到 antirez 和 Martin Kleppmann 关于 Redlock 的激烈辩论。

实际项目中不建议使用 Redlock 算法，成本和收益不成正比。

如果不是非要实现绝对可靠的分布式锁的话，其实单机版 Redis 就完全够了，实现简单，性能也非常高。如果你必须要实现一个绝对可靠的分布式锁的话，可以基于 Zookeeper 来做，只是性能会差一些。

-----## 7.5 RPC

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

RPC 这部分目前已经总结了 RPC 基础常见面试题和 Dubbo 常见面试题。

由于篇幅问题，这里直接放 JavaGuide 在线网站网站上的文章链接，小伙伴可以根据个人需求自行学习：

- [RPC 基础常见面试题总结](#)
- [Dubbo 常见面试题总结](#)



JavaGuide

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

扫码关注

-----## 7.6 分布式事务(付费)

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

分布式事务 相关的面试题为我的[知识星球](#)（[点击链接即可查看详细介绍以及加入方法](#)）专属内容，已经整理到了《[Java 面试指北](#)》（[点击链接即可查看详细介绍以及获取方法](#)）中。

《[Java 面试指北](#)》的部分内容展示如下，你可以将其看作是 **JavaGuide** 的补充完善，两者可以配合使用。

介绍	02-24 18:03
▼ 面试准备篇	+ :
如何准备 Java 面试?	02-11 11:50
程序员简历到底该怎么写? 有哪些注意的点?	02-11 12:30
为什么要学习源码? 源码这块面试会怎么问呢? 如何阅读源码?	02-11 12:07
为什么要准备算法面试? 怎么高效刷 Leetcode?	02-24 08:36
没有项目经验怎么办? 跟着视频做的项目会被面试官嫌弃不?	02-11 12:08
接触不到高并发场景咋办? 如何获得高并发的经验?	02-11 12:14
什么项目算是有亮点的或者是面试官认为有价值的?	02-11 12:19
去外包对自己的简历有影响么?	02-11 12:26
如果面试官问“你有什么问题问我吗?”, 该如何回答?	02-11 12:27
Java 优质面试视频汇总	02-11 12:28
Java 优质开源面试资源汇总	昨天 19:14
▼ 技术面试题篇	02-23 19:06
▸ 系统设计	02-13 20:25
▸ Java	02-13 20:27
▸ 数据库	02-13 20:27
▸ 常见框架	02-13 20:27
▸ 分布式	02-13 20:28
▸ 高并发	02-21 18:24
▸ 服务器	02-18 23:52
▸ Devops	02-11 16:48
▸ 技术面试题自测	02-13 20:33
▼ 面经篇	今天 10:22
双非本科、0实习、0比赛项目经历。3个月上岸百度	02-23 21:05
2021 华为 字节 腾讯 京东 网易 滴滴面经分享 (6个offer)	02-23 21:04
从考研失败到收获到自己满意的Offer	02-23 18:49
2年经验, 2021 阿里、头条、美团, 滴滴, 京东面经	02-23 18:48
2021 虾皮, 网易云, 京东, 阿里校招面经! 附参考答案	02-23 18:48
4年经验, 面试Bigo挂在了第三轮	02-23 19:01
五面阿里, 终获 offer!	02-23 21:06
▼ 练级攻略篇	02-28 18:56
如何提高编程能力?	02-24 13:14

《Java 面试指北》只是星球内部众多资料中的一个, 星球还有很多其他优质资料比如[专属专栏](#)、Java 编程视频、PDF 资料。

星球专属的 7 个小册：

- 1、《Java面试指北》(配合 JavaGuide 食用)：🔗 输入密码 · 语雀 (密码：)
- 2、《Java 必读源码系列》(目前已经整理了 Dubbo 2.6.x、Netty 4.x、SpringBoot2.1 的源码)：🔗 输入密码 · 语雀 (密码：)
- 3、《从零开始写一个RPC框架》：🔗 输入密码 · 语雀 (密码：) RPC 框架 Github地址：🔗 GitHub – Snailclimb/guide-rpc-framework: A custom ...。
- 4、《分布式、高并发、Devops知识扫盲》：已经并入《Java面试指北》中。
- 5、《Kafka常见面试题/知识点总结》：🔗 输入密码 · 语雀 (密码：)
- 6、《程序员副业赚钱之路》🔗 输入密码 · 语雀 (密码：)
- 7、《Guide的读书笔记与文章精选集》(经典书籍精读笔记分享) 🔗 输入密码 · 语雀 (密码：)

#球友必看#

球友必看



Guide哥：勿传播，星球专属。晚上尽量抽时间更新🤗

最近几年，市面上有越来越多的“技术大佬”开始办培训班/训练营，动辄成千上万的学费，却并没有什么干货，单纯的就是割韭菜。

为了帮助更多同学准备 Java 面试以及学习 Java，我创建了一个纯粹的[知识星球](#)。虽然收费只有培训班/训练营的百分之一，但是[知识星球](#)里的内容质量更高，提供的服务也更全面。

欢迎准备 Java 面试以及学习 Java 的同学加入我的[知识星球](#)，干货非常多，学习氛围非常好！收费虽然是白菜价，但星球里的内容或许比你参加上万的培训班质量还要高。

JavaGuide&Java面试交流圈

星主：Guide哥



1.5万
成员数量

9800+
内容数量

1024
运营天数

最优质的 Java 面试交流星球，多次被知识星球官方推荐。门票即将上调到 199 元，IOS 用户请在公众号“JavaGuide”回复“知识星球”加入/续费。

...

知识星球

微信扫码加入星球 ▶



下面是星球提供的部分服务（点击下方图片即可获取知识星球的详细介绍）：

我的知识星球能为你提供什么？

努力做一个最优质的 Java 面试交流星球！加入到我的星球之后，你将获得：

1. 6 个高质量的专栏永久阅读，内容涵盖面试，源码解析，项目实战等内容！价值远超门票！
2. 多本原创 PDF 版本面试手册。
3. 免费的简历修改服务（已经累计帮助 4000+ 位球友修改简历）。
4. 一对一免费提问交流（专属建议，走心回答）。
5. 专属求职指南和建议，帮助你逆袭大厂！
6. 海量 Java 优质面试资源分享！价值远超门票！
7. 读书交流，学习交流，让我们共同努力创建一个纯粹的学习交流社区。
8. 不定期福利：节日抽奖、送书送课、球友线下聚会等等。
9.

其中的任何一项服务单独拎出来价值都远超星球门票了。

我有自己的原则，不割韭菜，用心做内容，真心希望帮助到你！

如果你感兴趣的话，不妨花 3 分钟左右看看星球的详细介绍：[JavaGuide 知识星球详细介绍](#)（文末有优惠券）。



7.7 分布式协调(ZooKeeper)

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！

由于篇幅问题，这里直接放 JavaGuide 在线网站网站上的文章链接，小伙伴可以根据个人需求自行学习：

- [ZooKeeper 相关概念总结\(入门\)](#)

- ZooKeeper 相关概念总结(进阶)
- ZooKeeper 实战



扫码关注

准备 Java 面试，首选 JavaGuide!

- 1、关注公众号回复“PDF”领取原创面试 PDF 手册
- 2、关注公众号回复“加群”进入面试交流群
- 3、关注公众号回复“八股文”获取大厂面试真题+面经

8. 高性能

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！



8.1 数据库读写分离和分库分表

相信很多小伙伴们对于这两个概念已经比较熟悉了，这篇文章全程都是大白话的形式，希望能够给你带来不一样的感受。

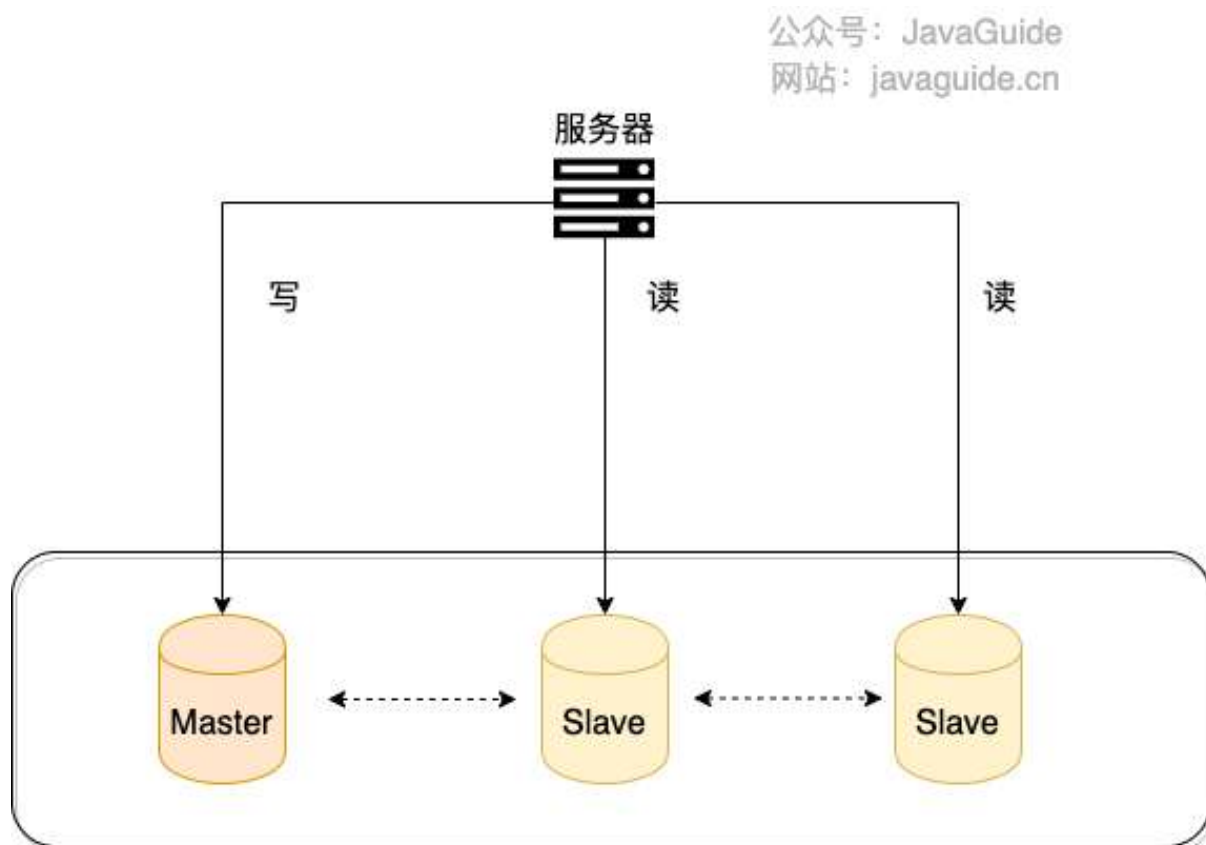
如果你之前不太了解这两个概念，那我建议你搞懂之后，可以把自己对于读写分离以及分库分表的理解讲给你的同事/朋友听听。

读写分离

何为读写分离？

见名思意，根据读写分离的名字，我们就可以知道：**读写分离主要是为了将对数据库的读写操作分散到不同的数据库节点上。**这样的话，就能够小幅提升写性能，大幅提升读性能。

我简单画了一张图来帮助不太清楚读写分离的小伙伴理解。



一般情况下，我们都会选择一主多从，也就是一台主数据库负责写，其他的从数据库负责读。主库和从库之间会进行数据同步，以保证从库中数据的准确性。这样的架构实现起来比较简单，并且也符合系统的写少读多的特点。

读写分离会带来什么问题？如何解决？

读写分离对于提升数据库的并发非常有效，但是，同时也会引来一个问题：主库和从库的数据存在延迟，比如你写完主库之后，主库的数据同步到从库是需要时间的，这个时间差就导致了主库和从库的数据不一致性问题。这也就是我们经常说的 **主从同步延迟**。

主从同步延迟问题的解决，没有特别好的一种方案（可能是我太菜了，欢迎评论区补充）。你可以根据自己的业务场景，参考一下下面几种解决办法。

1.强制将读请求路由到主库处理。

既然你从库的数据过期了，那我就直接从主库读取嘛！这种方案虽然会增加主库的压力，但是，实现起来比较简单，也是我了解到的使用最多的一种方式。

比如 `Sharding-JDBC` 就是采用的这种方案。通过使用 `Sharding-JDBC` 的 `HintManager` 分片键值管理器，我们可以强制使用主库。

```
HintManager hintManager = HintManager.getInstance();  
hintManager.setMasterRouteOnly();  
// 继续JDBC操作
```

对于这种方案，你可以将那些必须获取最新数据的读请求都交给主库处理。

2.延迟读取。

还有一些朋友肯定会想既然主从同步存在延迟，那我就在延迟之后读取啊，比如主从同步延迟 0.5s，那我就 1s 之后再读取数据。这样多方便啊！方便是方便，但是也很扯淡。

不过，如果你是这样设计业务流程就会好很多：对于一些对数据比较敏感的场景，你可以在完成写请求之后，避免立即进行请求操作。比如你支付成功之后，跳转到一个支付成功的页面，当你点击返回之后才返回自己的账户。

另外，[《MySQL 实战 45 讲》](#) 这个专栏中的 [《读写分离有哪些坑？》](#) 这篇文章还介绍了很多其他比较实际的解决办法，感兴趣的小伙伴可以自行研究一下。

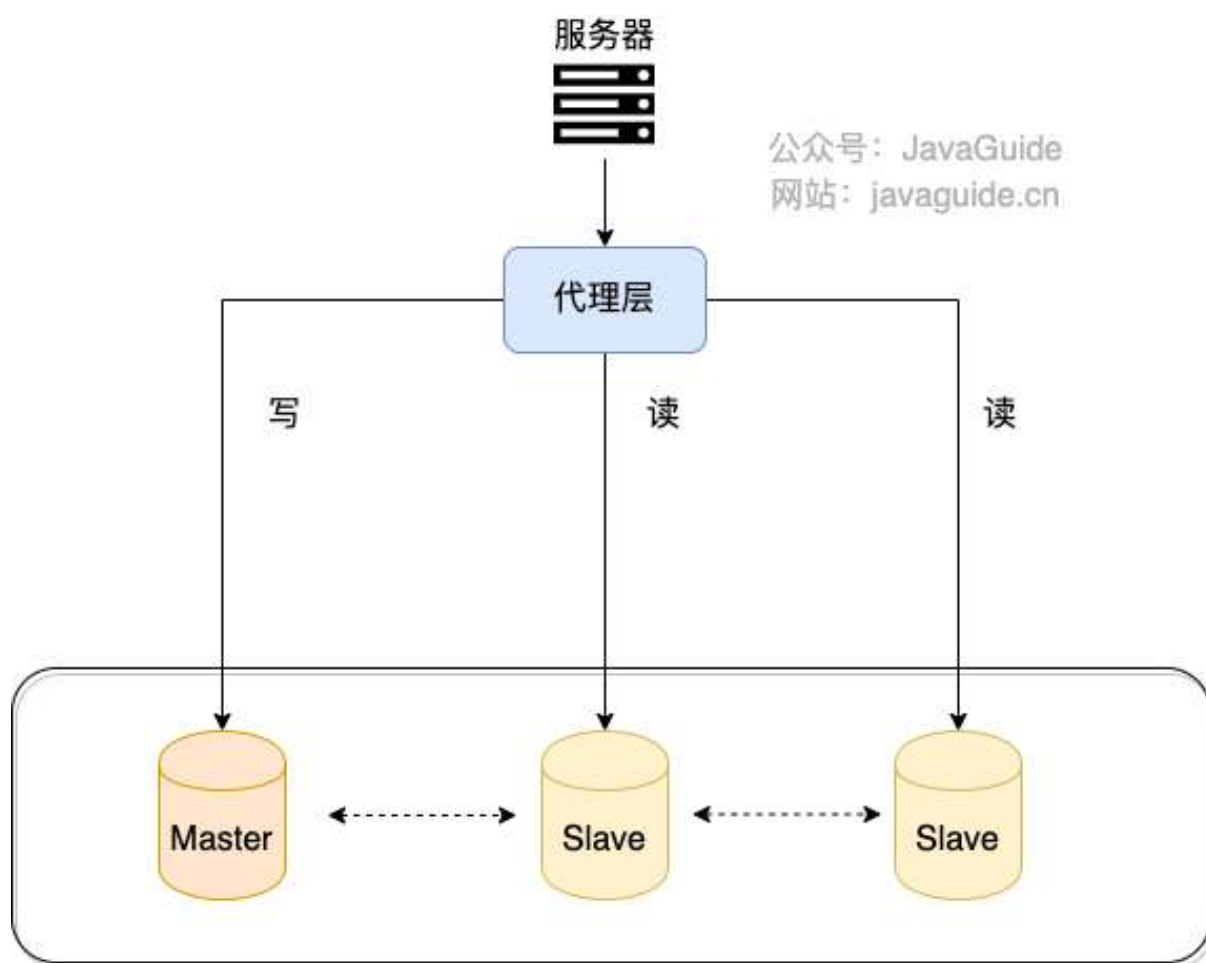
如何实现读写分离？

不论是使用哪一种读写分离具体的实现方案，想要实现读写分离一般包含如下几步：

1. 部署多台数据库，选择其中的一台作为主数据库，其他的一台或者多台作为从数据库。
2. 保证主数据库和从数据库之间的数据是实时同步的，这个过程也就是我们常说的主从复制。
3. 系统将写请求交给主数据库处理，读请求交给从数据库处理。

落实到项目本身的话，常用的方式有两种：

1.代理方式



我们可以在应用和数据中间加了一个代理层。应用程序所有的数据请求都交给代理层处理，代理层负责分离读写请求，将它们路由到对应的数据库中。

提供类似功能的中间件有 **MySQL Router**（官方）、**Atlas**（基于 MySQL Proxy）、**Maxscale**、**MyCat**。

2.组件方式

在这种方式中，我们可以通过引入第三方组件来帮助我们读写请求。

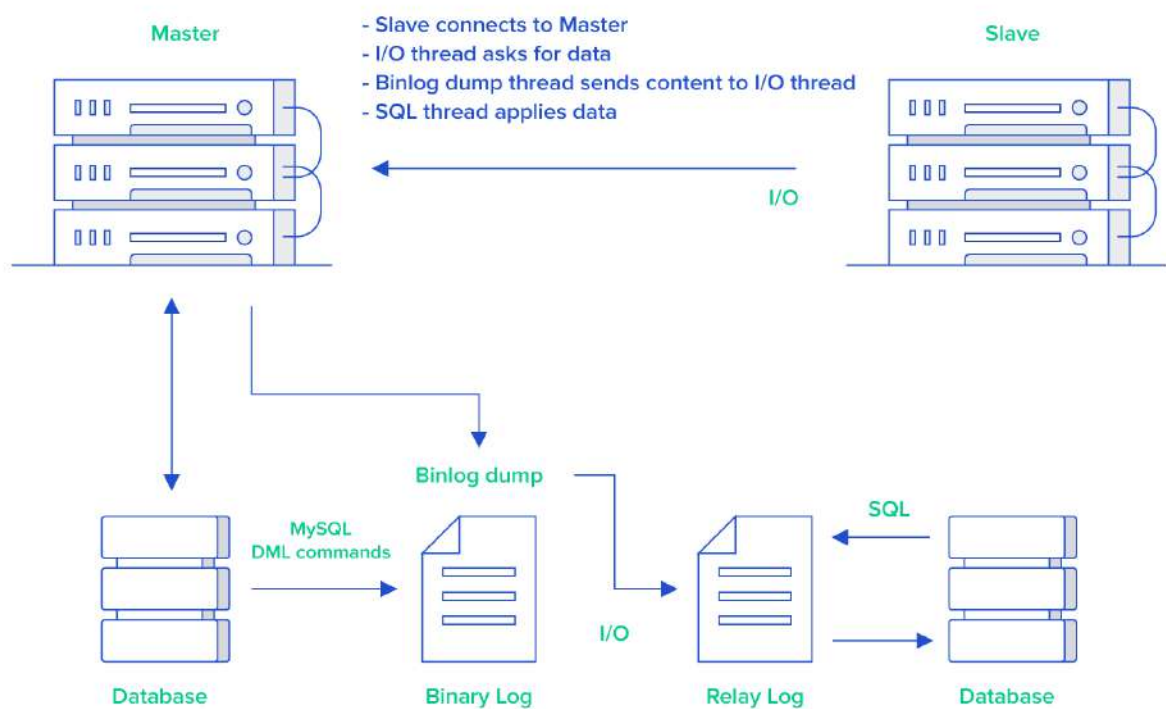
这也是我比较推荐的一种方式。这种方式目前在各种互联网公司中用的最多的，相关的实际的案例也非常多。如果你要采用这种方式的话，推荐使用 `sharding-jdbc`，直接引入 jar 包即可使用，非常方便。同时，也节省了很多运维的成本。

你可以在 `shardingsphere` 官方找到 [sharding-jdbc 关于读写分离的操作](#)。

主从复制原理了解么？

MySQL binlog(binary log 即二进制日志文件) 主要记录了 MySQL 数据库中数据的所有变化(数据库执行的所有 DDL 和 DML 语句)。因此，我们根据主库的 MySQL binlog 日志就能够将主库的数据同步到从库中。

更具体和详细的过程是这个样子的（图片来自于：《MySQL Master-Slave Replication on the Same Machine》）：



1. 主库将数据库中数据的变化写入到 binlog
2. 从库连接主库

3. 从库会创建一个 I/O 线程向主库请求更新的 binlog
4. 主库会创建一个 binlog dump 线程来发送 binlog，从库中的 I/O 线程负责接收
5. 从库的 I/O 线程将接收的 binlog 写入到 relay log 中。
6. 从库的 SQL 线程读取 relay log 同步数据本地（也就是再执行一遍 SQL）。

怎么样？看了我对主从复制这个过程的讲解，你应该搞明白了吧！

你一般看到 binlog 就要想到主从复制。当然，除了主从复制之外，binlog 还能帮助我们实现数据恢复。

 拓展一下：

不知道大家有没有使用过阿里开源的一个叫做 canal 的工具。这个工具可以帮助我们实现 MySQL 和其他数据源比如 Elasticsearch 或者另外一台 MySQL 数据库之间的数据同步。很显然，这个工具的底层原理肯定也是依赖 binlog。canal 的原理就是模拟 MySQL 主从复制的过程，解析 binlog 将数据同步到其他的数据源。

另外，像咱们常用的分布式缓存组件 Redis 也是通过主从复制实现的读写分离。

 简单总结一下：

MySQL 主从复制是依赖于 binlog。另外，常见的一些同步 MySQL 数据到其他数据源的工具（比如 canal）的底层一般也是依赖 binlog。

分库分表

读写分离主要应对的是数据库读并发，没有解决数据库存储问题。试想一下：**如果 MySQL 一张表的数据量过大怎么办？**

换言之，我们该如何解决 **MySQL 的存储压力**呢？

答案之一就是 **分库分表**。

何为分库？

分库 就是将数据库中的数据分散到不同的数据库上。

下面这些操作都涉及到了分库：

- 你将数据库中的用户表和用户订单表分别放在两个不同的数据库。
- 由于用户表数据量太大，你对用户表进行了水平切分，然后将切分后的 2 张用户表分别放在两

个不同的数据库。

何为分表？

分表 就是对单表的数据进行拆分，可以是垂直拆分，也可以是水平拆分。

何为垂直拆分？

简单来说，垂直拆分是对数据表列的拆分，把一张列比较多的表拆分为多张表。

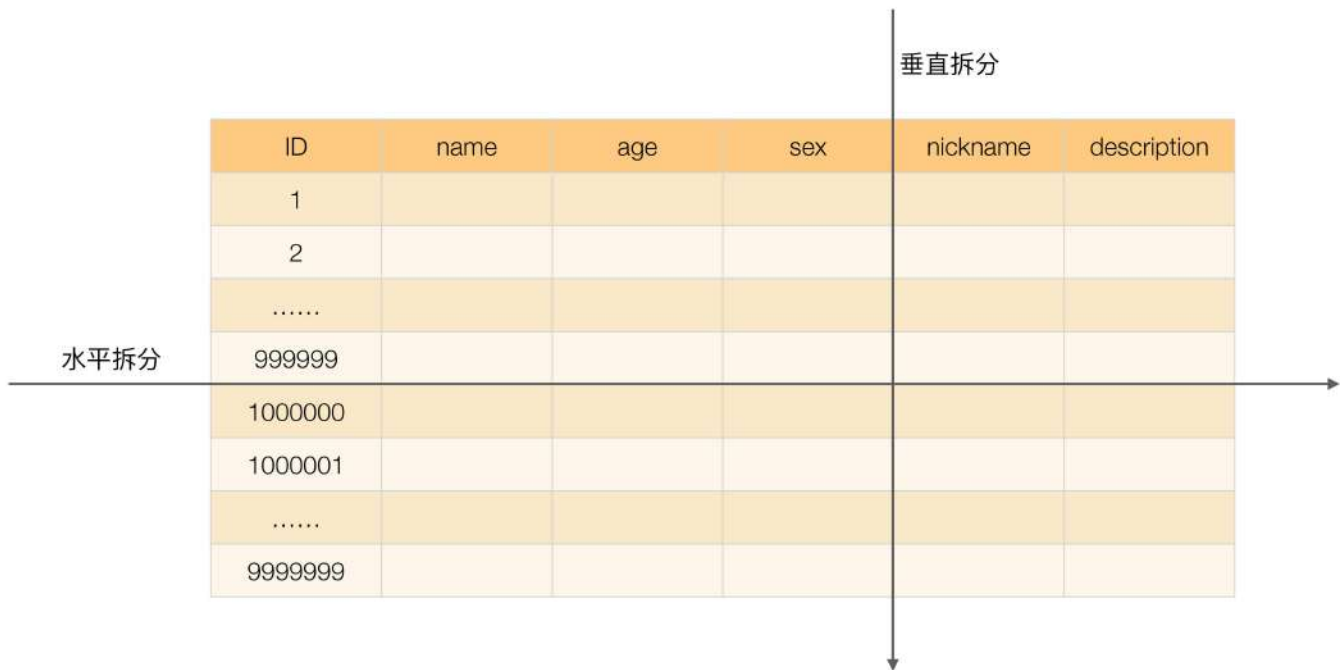
举个例子：我们可以将用户信息表中的一些列单独抽出来作为一个表。

何为水平拆分？

简单来说，水平拆分是对数据表行的拆分，把一张行比较多的表拆分为多张表。

举个例子：我们可以将用户信息表拆分成多个用户信息表，这样就可以避免单一表数据量过大对性能造成影响。

《从零开始学架构》 中的有一张图片对于垂直拆分和水平拆分的描述还挺直观的。



什么情况下需要分库分表？

遇到下面几种场景可以考虑分库分表：

- 单表的数据达到千万级别以上，数据库读写速度比较缓慢（分表）。
- 数据库中的数据占用的空间越来越大，备份时间越来越长（分库）。
- 应用的并发量太大（分库）。

分库分表会带来什么问题呢？

记住，你在公司做的任何技术决策，不光是要考虑这个技术能不能满足我们的要求，是否适合当前业务场景，还要重点考虑其带来的成本。

引入分库分表之后，会给系统带来什么挑战呢？

- **join 操作**： 同一个数据库中的表分布在了不同的数据库中，导致无法使用 join 操作。这样就导致我们需要手动进行数据的封装，比如你在一个数据库中查询到一个数据之后，再根据这个数据去另外一个数据库中找对应的数据。
- **事务问题**： 同一个数据库中的表分布在了不同的数据库中，如果单个操作涉及到多个数据库，那么数据库自带的事务就无法满足我们的要求了。
- **分布式 id**： 分库之后，数据遍布在不同服务器上的数据库，数据库的自增主键已经没办法满足生成的主键唯一了。我们如何为不同的数据节点生成全局唯一主键呢？这个时候，我们就需要为我们的系统引入分布式 id 了。
-

另外，引入分库分表之后，一般需要 DBA 的参与，同时还需要更多的数据库服务器，这些都属于成本。

分库分表有没有什么比较推荐的方案？

ShardingSphere 项目（包括 Sharding-JDBC、Sharding-Proxy 和 Sharding-Sidecar）是当当捐入 Apache 的，目前主要由京东数科的一些巨佬维护。

Apache ShardingSphere核心功能

数据分片

- 读写分离
- 数据拆分 **UP**

分布式事务 **NEW**

- 强一致性事务
- 柔性事务

数据库治理

- 配置动态管理
- 拓扑图展示
- 高可用管理
- 数据脱敏安全 **NEW**
- 权限控制 **NEW**

ShardingSphere 绝对可以说是当前分库分表的首选！ShardingSphere 的功能完善，除了支持读写分离和分库分表，还提供分布式事务、数据库治理等功能。

另外，ShardingSphere 的生态体系完善，社区活跃，文档完善，更新和发布比较频繁。

芳芳之前写了一篇分库分表的实战文章，各位朋友可以看看：[《芋道 Spring Boot 分库分表入门》](#)。

分库分表后，数据怎么迁移呢？

分库分表之后，我们如何将老库（单库单表）的数据迁移到新库（分库分表后的数据库系统）呢？

比较简单同时也是非常常用的方案就是**停机迁移**，写个脚本老库的数据写到新库中。比如你在凌晨 2 点，系统使用的人数非常少的时候，挂一个公告说系统要维护升级预计 1 小时。然后，你写一个脚本将老库的数据都同步到新库中。

如果你不想停机迁移数据的话，也可以考虑**双写方案**。双写方案是针对那种不能停机迁移的场景，实现起来要稍微麻烦一些。具体原理是这样的：

- 我们对老库的更新操作（增删改），同时也要写入新库（双写）。如果操作的数据不存在于新库

的话，需要插入到新库中。这样就能保证，咱们新库里的数据是最新的。

- 在迁移过程，双写只会让被更新操作过的老库中的数据同步到新库，我们还需要自己写脚本将老库中的数据和新库的数据做比对。如果新库中没有，那咱们就把数据插入到新库。如果新库有，旧库没有，就把新库对应的数据删除（冗余数据清理）。
- 重复上一步的操作，直到老库和新库的数据一致为止。

想要在项目中实施双写还是比较麻烦的，很容易会出现问题。我们可以借助上面提到的数据库同步工具 Canal 做增量数据迁移（还是依赖 binlog，开发和维护成本较低）。

总结

- 读写分离主要是为了将对数据库的读写操作分散到不同的数据库节点上。这样的话，就能够小幅提升写性能，大幅提升读性能。
- 读写分离基于主从复制，MySQL 主从复制是依赖于 binlog 。
- **分库** 就是将数据库中的数据分散到不同的数据库上。**分表** 就是对单表的数据进行拆分，可以是垂直拆分，也可以是水平拆分。
- 引入分库分表之后，需要系统解决事务、分布式 id、无法 join 操作问题。
- ShardingSphere 绝对可以说是当前分库分表的首选！ShardingSphere 的功能完善，除了支持读写分离和分库分表，还提供分布式事务、数据库治理等功能。另外，ShardingSphere 的生态体系完善，社区活跃，文档完善，更新和发布比较频繁。

8.2 CDN(内容分发网络)

什么是 CDN ？

CDN 全称是 Content Delivery Network/Content Distribution Network，翻译过的意思是 **内容分发网络**。

我们可以将内容分发网络拆开来：

- **内容**：指的是静态资源比如图片、视频、文档、JS、CSS、HTML。
- **分发网络**：指的是将这些静态资源分发到位于多个不同的地理位置机房中的服务器上，这样，就可以实现静态资源的就近访问比如北京的用户直接访问北京机房的数据。

所以，简单来说，**CDN 就是将静态资源分发到多个不同的地方以实现就近访问，进而加快静态资源的访问速度，减轻服务器以及带宽的负担。**

类似于京东建立的庞大的仓储运输体系，京东物流在全国拥有非常多的仓库，仓储网络几乎覆盖全国所有区县。这样的话，用户下单的第一时间，商品就从距离用户最近的仓库，直接发往对应的配送站，再由京东小哥送到你家。



你可以将 CDN 看作是服务上一层的特殊缓存服务，分布在全国各地，主要用来处理静态资源的请求。



我们经常拿全站加速和内容分发网络做对比，不要把两者搞混了！全站加速（不同云服务商叫法不同，腾讯云叫 ECDN、阿里云叫 DCDN）既可以加速静态资源又可以加速动态资源，内容分发网络（CDN）主要针对的是 **静态资源**。

对比项	CDN	全站加速
加速资源	加速静态资源。	同时加速静态和动态资源。
加速方式	支持静态加速。 将服务器上的图片、视频等静态资源缓存在CDN边缘节点，供用户从最近的节点获取静态资源。	支持纯动态加速和动静混合加速： • 纯动态加速 针对POST请求等不能在边缘缓存的业务，基于智能选路技术，从众多回源线路中择优选择一条线路进行传输。 • 动静混合加速 智能识别动态和静态资源，静态资源缓存在边缘节点，供用户就近访问；动态资源基于智能选路技术，从众多回源线路中择优选择一条线路进行传输。
协议支持	支持HTTP、HTTPS。	支持HTTP、HTTPS、TCP、UDP、Websocket。
源站适配	不能区分动态和静态资源。 您需要自行分离动态和静态资源，静态资源使用CDN加速，动态资源无法被CDN加速。	智能区分动态和静态资源。 您无需分离动态和静态资源，全站加速支持智能区分动态和静态资源并分别加速。
边缘计算	支持在边缘节点使用JavaScript构建边缘程序，例如A/B Test、预热等。	支持在边缘节点和回源侧构建全链路的边缘计算，场景更加丰富，例如A/B Test、回源URI改写、封禁拦截等。

绝大部分公司都会在项目开发中交使用 CDN 服务，但很少会有自建 CDN 服务的公司。基于成本、稳定性和易用性考虑，建议直接选择专业的云厂商（比如阿里云、腾讯云、华为云、青云）或者 CDN 厂商（比如网宿、蓝汛）提供的开箱即用的 CDN 服务。

很多朋友可能要问了：既然是就近访问，为什么不直接将服务部署在多个不同的地方呢？

- 成本太高，需要部署多份相同的服务。
- 静态资源通常占用空间比较大且经常会被访问到，如果直接使用服务器或者缓存来处理静态资源请求的话，对系统资源消耗非常大，可能会影响到系统其他服务的正常运行。

同一个服务在在多个不同的地方部署多份（比如同城灾备、异地灾备、同城多活、异地多活）是为了实现系统的高可用而不是就近访问。

CDN 工作原理是什么？

搞懂下面 3 个问题也就搞懂了 CDN 的工作原理：

1. 静态资源是如何被缓存到 CDN 节点中的？
2. 如何找到最合适的 CDN 节点？
3. 如何防止静态资源被盗用？

静态资源是如何被缓存到 CDN 节点中的？

你可以通过预热的方式将源站的资源同步到 CDN 的节点中。这样的话，用户首次请求资源可以直接从 CDN 节点中取，无需回源。这样可以降低源站压力，提升用户体验。

如果不预热的话，你访问的资源可能不再 CDN 节点中，这个时候 CDN 节点将请求源站获取资源，这个过程是大家经常说的 **回源**。

命中率 和 **回源率** 是衡量 CDN 服务质量两个重要指标。命中率越高越好，回源率越低越好。

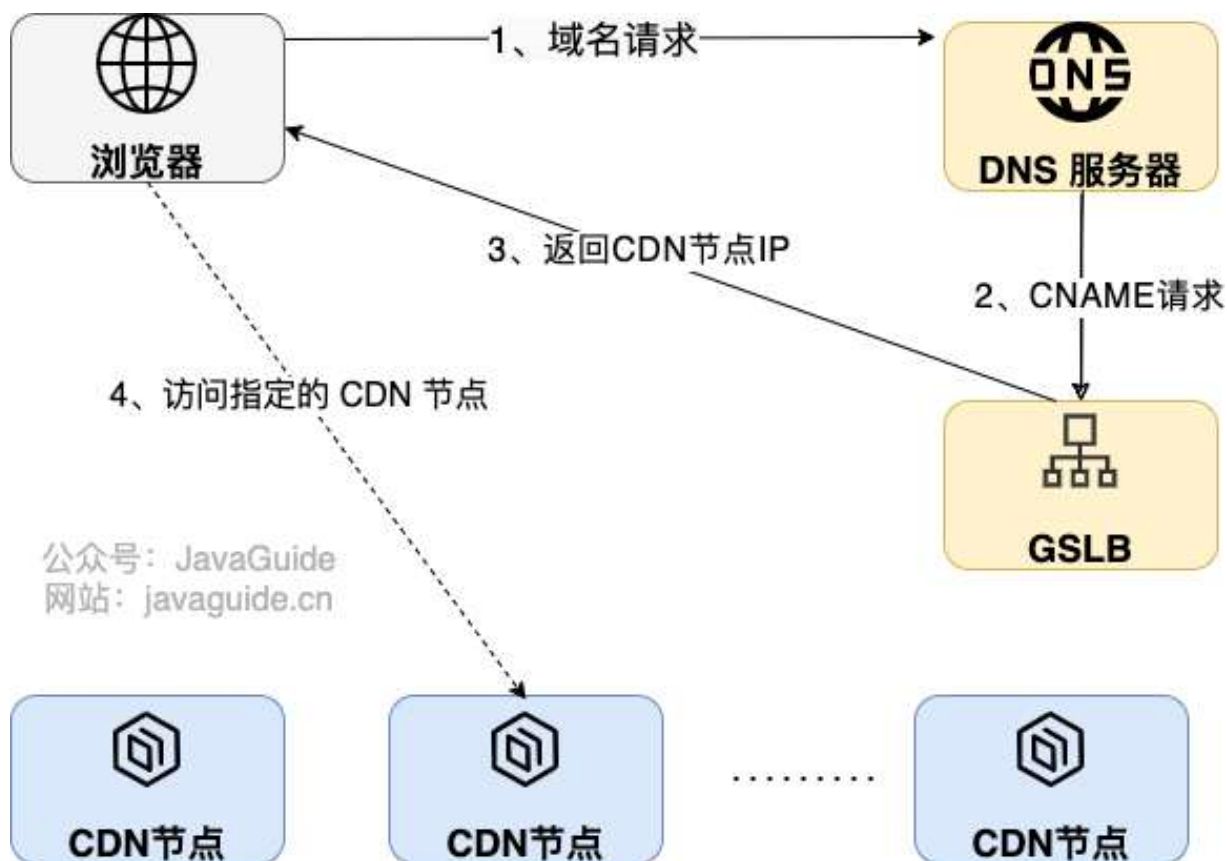
如果资源有更新的话，你也可以对其 **刷新**，删除 CDN 节点上缓存的资源，当用户访问对应的资源时直接回源获取最新的资源，并重新缓存。

如何找到最合适的 CDN 节点？

GSLB（Global Server Load Balance，全局负载均衡）是 CDN 的大脑，负责多个 CDN 节点之间相互协作，最常用的是基于 DNS 的 GSLB。

CDN 会通过 GSLB 找到最合适的 CDN 节点，更具体点来说是下面这样的：

1. 浏览器向 DNS 服务器发送域名请求；
2. DNS 服务器向根据 CNAME(Canonical Name) 别名记录向 GSLB 发送请求；
3. GSLB 返回性能最好（通常距离请求地址最近）的 CDN 节点（边缘服务器，真正缓存内容的地方）的地址给浏览器；
4. 浏览器直接访问指定的 CDN 节点。



为了方便理解，上图其实做了一点简化。GSLB 内部可以看作是 CDN 专用 DNS 服务器和负载均衡系统组合。CDN 专用 DNS 服务器会返回负载均衡系统 IP 地址给浏览器，浏览器使用 IP 地址请求负载均衡系统进而找到对应的 CDN 节点。

GSLB 是如何选择出最合适的 CDN 节点呢？ GSLB 会根据请求的 IP 地址、CDN 节点状态（比如负载情况、性能、响应时间、带宽）等指标来综合判断具体返回哪一个 CDN 节点的地址。

如何防止资源被盗刷？

如果我们的资源被其他用户或者网站非法盗刷的话，将会是一笔不小的开支。

解决这个问题最常用最简单的办法设置 **Referer 防盗链**，具体来说就是根据 HTTP 请求的头信息里面的 Referer 字段对请求进行限制。我们可以通过 Referer 字段获取到当前请求页面的来源页面的网站地址，这样我们就能确定请求是否来自合法的网站。

CDN 服务提供商几乎都提供了这种比较基础的防盗链机制。

开启防盗链配置



- 不需要输入网址符http://，以换行符相隔，一行输入一个，不可重复；
- 当未勾选“空referer”且输入内容不为空时，表示当前未开启referer防盗链功能
- 黑名单和白名单为互斥选项，您只能选择其中一种方式配置您的防盗链。

防盗链类型



白名单



黑名单

请输入域名，如www.test.com；或者IP，如203.123.123.123；支持
前端通配符，如*.test.com

还可以输入400个

空referer选项



拒绝空referer访问 ⓘ

确认

取消

不过，如果站点的防盗链配置允许 Referer 为空的话，通过隐藏 Referer，可以直接绕开防盗链。

通常情况下，我们会配合其他机制来确保静态资源被盗用，一种常用的机制是 **时间戳防盗链**。相比之下，**时间戳防盗链** 的安全性更强一些。时间戳防盗链加密的 URL 具有时效性，过期之后就无法再被允许访问。

时间戳防盗链的 URL 通常会有两个参数一个是签名字符串，一个是过期时间。签名字符串一般是通过对用户设定的加密字符串、请求路径、过期时间通过 MD5 哈希算法取哈希的方式获得。

时间戳防盗链 URL 示例：

```
http://cdn.wangsu.com/4/123.mp3?  
wsSecret=79aead3bd7b5db4adeffb93a010298b5&wsTime=1601026312
```


- wsSecret ： 签名字符串。
- wsTime ： 过期时间。

wangsu/4/123.mp31601026312

加密字符串

请求路径

过期时间

↓ MD5

16位大寫:	D7B5DB4ADEFFB93A
16位小寫:	d7b5db4adeffb93a
32位大寫:	79AEAD3BD7B5DB4ADEFFB93A010298B5
32位小寫:	79aead3bd7b5db4adeffb93a010298b5

时间戳防盗链的实现也比较简单，并且可靠性较高，推荐使用。并且，绝大部分 CDN 服务提供商都提供了开箱即用的时间戳防盗链机制。

CDN

概览

域名管理

内容优化

刷新预取

统计分析

日志下载

访问控制

配置项	描述	当前配置	操作
Referer 防盗链	配置 Request Header 中 referer 黑白名单，从而限制访问来源。	未开启	修改配置
时间戳防盗链	设置密钥，配合签名过期时间来控制资源内容的访问时限。 帮助文档	未开启	修改配置
回源鉴权	配置回源验证策略，对 CDN 接收到的访问请求进行鉴权，适用于对防盗链有很高实时性要求的场景。 帮助文档	未开启	修改配置
IP 黑白名单	允许或者禁止某些 IP 或 IP 段的访问，帮助您解决恶意 IP 盗刷、攻击等问题。	未开启	修改配置

除了 Referer 防盗链和时间戳防盗链之外，你还可以 IP 黑白名单配置、IP 访问限频配置等机制来防盗刷。

总结

- CDN 就是将静态资源分发到多个不同的地方以实现就近访问，进而加快静态资源的访问速度，减轻服务器以及带宽的负担。
- 基于成本、稳定性和易用性考虑，建议直接选择专业的云厂商（比如阿里云、腾讯云、华为云、青云）或者 CDN 厂商（比如网宿、蓝汛）提供的开箱即用的 CDN 服务。
- GSLB（Global Server Load Balance，全局负载均衡）是 CDN 的大脑，负责多个 CDN 节点之间相互协作，最常用的是基于 DNS 的 GSLB。CDN 会通过 GSLB 找到最合适的 CDN 节点。
- 为了防止静态资源被盗用，我们可以利用 **Referer 防盗链 + 时间戳防盗链**。

参考

- 时间戳防盗链 - 七牛云 CDN: <https://developer.qiniu.com/fusion/kb/1670/timestamp-hotlinking-prevention>
- CDN 是个啥玩意? 一文说个明白: <https://mp.weixin.qq.com/s/Pp0C8ALUXsmYCUkM5QnkQw>
- 《透视 HTTP 协议》- 37 | CDN: 加速我们的网络服务: <http://gk.link/a/11yOG----->

8.3 消息队列

JavaGuide: 「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试, 首选 JavaGuide!

由于篇幅问题, 这里直接放 JavaGuide 在线网站网站上的文章链接, 小伙伴可以根据个人需求自行学习:

- 消息队列常见问题总结
- **RabbitMQ**: [RabbitMQ 基础知识总结](#)、[RabbitMQ 常见面试题](#)
- **RocketMQ**: [RocketMQ 基础知识总结](#)、[RocketMQ 常见面试题总结](#)
- **Kafka**: [Kafka 常见问题总结](#)



扫码关注

准备 Java 面试, 首选 JavaGuide!

- 1、关注公众号回复“**PDF**”领取原创面试 PDF 手册
- 2、关注公众号回复“**加群**”进入面试交流群
- 3、关注公众号回复“**八股文**”获取大厂面试真题+面经

9. 高可用

JavaGuide：「Java学习+面试指南」一份涵盖大部分 Java 程序员所需要掌握的核心知识。准备 Java 面试，首选 JavaGuide！



[Github](#) |

[Gitee](#)

9.1 高可用系统设计指南

什么是高可用？可用性的判断标准是啥？

高可用描述的是一个系统在大部分时间都是可用的，可以为我们提供服务的。高可用代表系统即使在发生硬件故障或者系统升级的时候，服务仍然是可用的。

一般情况下，我们使用多少个 9 来评判一个系统的可用性，比如 99.9999% 就是代表该系统在所有的运行时间中只有 0.0001% 的时间是不可用的，这样的系统就是非常非常高可用的了！当然，也会有系统如果可用性不太好的话，可能连 9 都上不了。

除此之外，系统的可用性还可以用某功能的失败次数与总的请求次数之比来衡量，比如对网站请求 1000 次，其中有 10 次请求失败，那么可用性就是 99%。

哪些情况会导致系统不可用？

1. 黑客攻击；
2. 硬件故障，比如服务器坏掉。
3. 并发量/用户请求量激增导致整个服务宕掉或者部分服务不可用。
4. 代码中的坏味道导致内存泄漏或者其他问题导致程序挂掉。
5. 网站架构某个重要的角色比如 Nginx 或者数据库突然不可用。
6. 自然灾害或者人为破坏。
7.

有哪些提高系统可用性的方法？

注重代码质量，测试严格把关

我觉得这个是最最重要的，代码质量有问题比如比较常见的内存泄漏、循环依赖都是对系统可用性极大的损害。大家都喜欢谈限流、降级、熔断，但是我觉得从代码质量这个源头把关是首先要做好的一件很重要的事情。如何提高代码质量？比较实际可用的就是 CodeReview，不要在乎每天多花的那 1 个小时左右的时间，作用可大着呢！

另外，安利几个对提高代码质量有实际效果的神器：

- [Sonarqube](#)；
- Alibaba 开源的 Java 诊断工具 [Arthas](#)；
- [阿里巴巴 Java 代码规范](#)（Alibaba Java Code Guidelines）；
- IDEA 自带的代码分析等工具。

使用集群，减少单点故障

先拿常用的 Redis 举个例子！我们如何保证我们的 Redis 缓存高可用呢？答案就是使用集群，避免单点故障。当我们使用一个 Redis 实例作为缓存的时候，这个 Redis 实例挂了之后，整个缓存服务可能就挂了。使用了集群之后，即使一台 Redis 实例挂了，不到一秒就会有另外一台 Redis 实例顶上。

限流

流量控制（flow control），其原理是监控应用流量的 QPS 或并发线程数等指标，当达到指定的阈值时对流量进行控制，以避免被瞬时的流量高峰冲垮，从而保障应用的高可用性。——来自 [alibaba-Sentinel](#) 的 wiki。

超时和重试机制设置

一旦用户请求超过某个时间的得不到响应，就抛出异常。这个是非常重要的，很多线上系统故障都是因为没有进行超时设置或者超时设置的方式不对导致的。我们在读取第三方服务的时候，尤其适合设置超时和重试机制。一般我们使用一些 RPC 框架的时候，这些框架都自带的超时重试的配置。如果不进行超时设置可能会导致请求响应速度慢，甚至导致请求堆积进而让系统无法再处理请求。重试的次数一般设为 3 次，再多次的重试没有好处，反而会加重服务器压力（部分场景使用失败重试机制会不太适合）。

熔断机制

超时和重试机制设置之外，熔断机制也是很重要的。熔断机制说的是系统自动收集所依赖服务的资源使用情况和性能指标，当所依赖的服务恶化或者调用失败次数达到某个阈值的时候就迅速失败，让当前系统立即切换依赖其他备用服务。比较常用的流量控制和熔断降级框架是 Netflix 的 Hystrix 和 alibaba 的 Sentinel。

异步调用

异步调用的话我们不需要关心最后的结果，这样我们就可以用户请求完成之后就立即返回结果，具体处理我们可以后续再做，秒杀场景用这个还是蛮多的。但是，使用异步之后我们可能需要 **适当修改业务流程进行配合**，比如用户在提交订单之后，不能立即返回用户订单提交成功，需要在消息队列的订单消费者进程真正处理完该订单之后，甚至出库后，再通过电子邮件或短信通知用户订单成功。除了可以在程序中实现异步之外，我们常常还使用消息队列，消息队列可以通过异步处理提高系统性能（削峰、减少响应所需时间）并且可以降低系统耦合性。

使用缓存

如果我们的系统属于并发量比较高的话，如果我们单纯使用数据库的话，当大量请求直接落到数据库可能数据库就会直接挂掉。使用缓存缓存热点数据，因为缓存存储在内存中，所以速度相当快！

其他

- 核心应用和服务优先使用更好的硬件
- 监控系统资源使用情况增加报警设置。
- 注意备份，必要时候回滚。
- 灰度发布：将服务器集群分成若干部分，每天只发布一部分机器，观察运行稳定没有故障，第二天继续发布一部分机器，持续几天才把整个集群全部发布完毕，期间如果发现问题，只需要回滚已发布的一部分服务器即可
- 定期检查/更换硬件：如果不是购买的云服务的话，定期还是需要对硬件进行一波检查的，对于一些需要更换或者升级的硬件，要及时更换或者升级。
------

9.2 冗余

冗余设计是保证系统和数据高可用的最常用手段。

对于服务来说，冗余的思想就是相同的服务部署多份，如果正在使用的服务突然挂掉的话，系统可以很快切换到备份服务上，大大减少系统的不可用时间，提高系统的可用性。

对于数据来说，冗余的思想就是相同的数据备份多份，这样就可以很简单地提高数据的安全性。

实际上，日常生活中就有非常多的冗余思想的应用。

拿我自己来说，我对于重要文件的保存方法就是冗余思想的应用。我日常所使用的重要文件都会同步一份在 Github 以及个人云盘上，这样就可以保证即使电脑硬盘损坏，我也可以通过 Github 或者个人云盘找回自己的重要文件。

高可用集群（High Availability Cluster，简称 HA Cluster）、同城灾备、异地灾备、同城多活和异地多活是冗余思想在高可用系统设计中最典型的应用。

- **高可用集群**：同一份服务部署两份或者多份，当正在使用的服务突然挂掉的话，可以切换到另外一台服务，从而保证服务的高可用。
- **同城灾备**：一整个集群可以部署在同一个机房，而同城灾备中相同服务部署在同一个城市的不同机房中。并且，备用服务不处理请求。这样可以避免机房出现意外情况比如停电、火灾。
- **异地灾备**：类似于同城灾备，不同的是，相同服务部署在异地（通常距离较远，甚至是在不同的城市或者国家）的不同机房中
- **同城多活**：类似于同城灾备，但备用服务可以处理请求，这样可以充分利用系统资源，提高系统的并发。
- **异地多活**：将服务部署在异地的不同机房中，并且，它们可以同时对外提供服务。

高可用集群单纯是服务的冗余，并没有强调地域。同城灾备、异地灾备、同城多活和异地多活实现了地域上的冗余。

同城和异地的主要区别在于机房之间的距离。异地通常距离较远，甚至是在不同的城市或者国家。

和传统的灾备设计相比，同城多活和异地多活最明显的改变在于“多活”，即所有站点都是同时在对外提供服务的。异地多活是为了应对突发状况比如火灾、地震等自然或者人为灾害。

光做好冗余还不够，必须要配合上 **故障转移** 才可以！所谓故障转移，简单来说就是实现不可用服务快速且自动地切换到可用服务，整个过程不需要人为干涉。

举个例子：哨兵模式的 Redis 集群中，如果 Sentinel（哨兵）检测到 master 节点出现故障的话，它就会帮助我们实现故障转移，自动将某一台 slave 升级为 master，确保整个 Redis 系统的可用性。整个过程完全自动，不需要人工介入。我在《[Java 面试指北](#)》的「技术面试题篇」中的数据库部分详细介绍了 Redis 集群相关的知识点&面试题，感兴趣的小伙伴可以看看。

再举个例子：Nginx 可以结合 Keepalived 来实现高可用。如果 Nginx 主服务器宕机的话，Keepalived 可以自动进行故障转移，备用 Nginx 主服务器升级为主服务。并且，这个切换对外是透明的，因为使用的虚拟 IP，虚拟 IP 不会改变。我在《[Java 面试指北](#)》的「技术面试题篇」中的「服务器」部分详细介绍了 Nginx 相关的知识点&面试题，感兴趣的小伙伴可以看看。

异地多活架构实施起来非常难，需要考虑的因素非常多。本人不才，实际项目中并没有实践过异地多活架构，我对其了解还停留在书本知识。

如果你想要深入学习异地多活相关的知识，我这里推荐几篇我觉得还不错的文章：

- [搞懂异地多活，看这篇就够了- 水滴与银弹 - 2021](#)
- [四步构建异地多活](#)
- [《从零开始学架构》— 28 | 业务高可用的保障：异地多活架构](#)

不过，这些文章大多也都是在介绍概念知识。目前，网上还缺少真正介绍具体要如何去实践落地异地多活架构的资料。-----

9.3 限流

何为限流？为什么要限流？

针对软件系统来说，限流就是对请求的速率进行限制，避免瞬时的大量请求击垮软件系统。毕竟，软件系统的处理能力是有限的。如果说超过了其处理能力的范围，软件系统可能直接就挂掉了。

限流可能会导致用户的请求无法被正确处理，不过，这往往也是权衡了软件系统的稳定性之后得到的最优解。

现实生活中，处处都有限流的实际应用，就比如排队买票是为了避免大量用户涌入购票而导致售票员无法处理。



常见限流算法

简单介绍 4 种非常好理解并且容易实现的限流算法！

图片来源于 InfoQ 的一篇文章 [《分布式服务限流实战，已经为你排好坑了》](#)。

固定窗口计数器算法

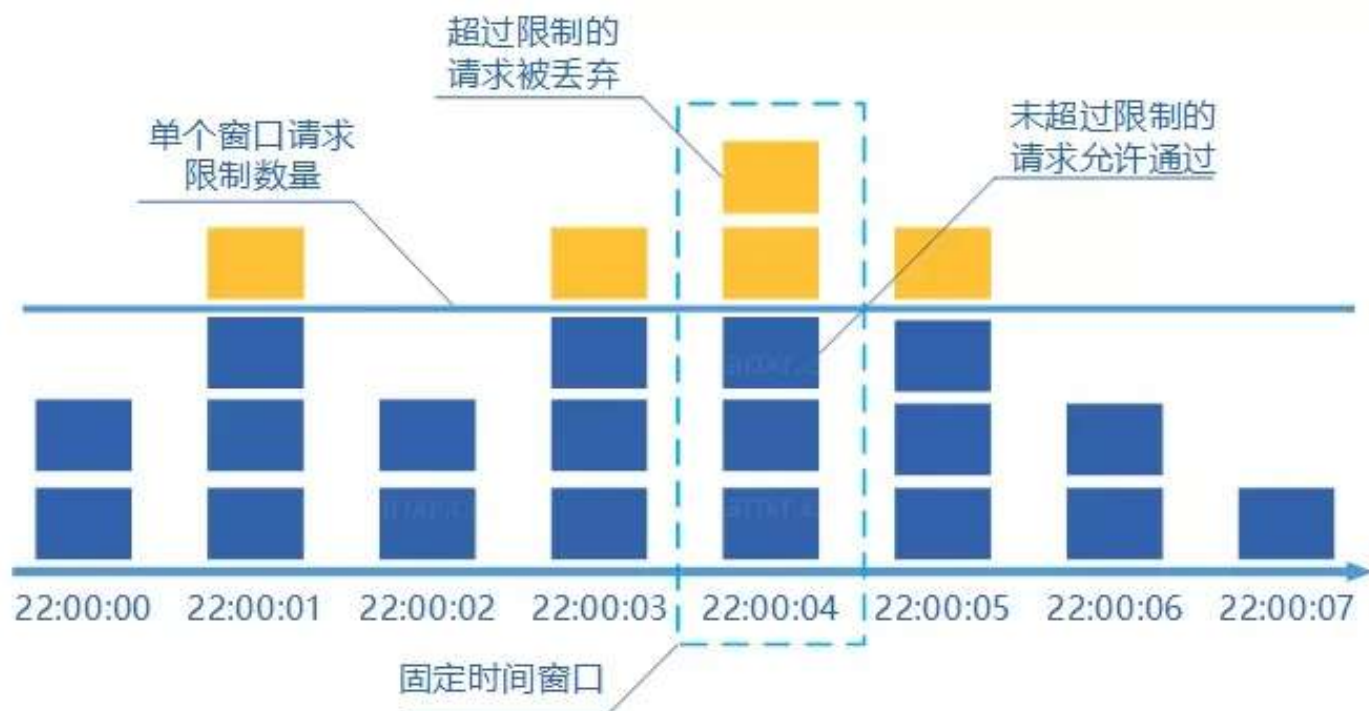
固定窗口其实就是时间窗口。**固定窗口计数器算法** 规定了我们单位时间处理的请求数量。

假如我们规定系统中某个接口 1 分钟只能访问 33 次的话，使用固定窗口计数器算法的实现思路如下：

- 给定一个变量 `counter` 来记录当前接口处理的请求数量，初始值为 0（代表接口当前 1 分钟内还未处理请求）。
- 1 分钟之内每处理一个请求之后就将 `counter+1`，当 `counter=33` 之后（也就是说在这 1 分钟内接口已经被访问 33 次的话），后续的请求就会被全部拒绝。
- 等到 1 分钟结束后，将 `counter` 重置 0，重新开始计数。

这种限流算法无法保证限流速率，因而无法保证突然激增的流量。

就比如说我们限制某个接口 1 分钟只能访问 1000 次，该接口的 QPS 为 500，前 55s 这个接口 1 个请求没有接收，后 1s 突然接收了 1000 个请求。然后，在当前场景下，这 1000 个请求在 1s 内是没办法被处理的，系统直接就被瞬时的大量请求给击垮了。



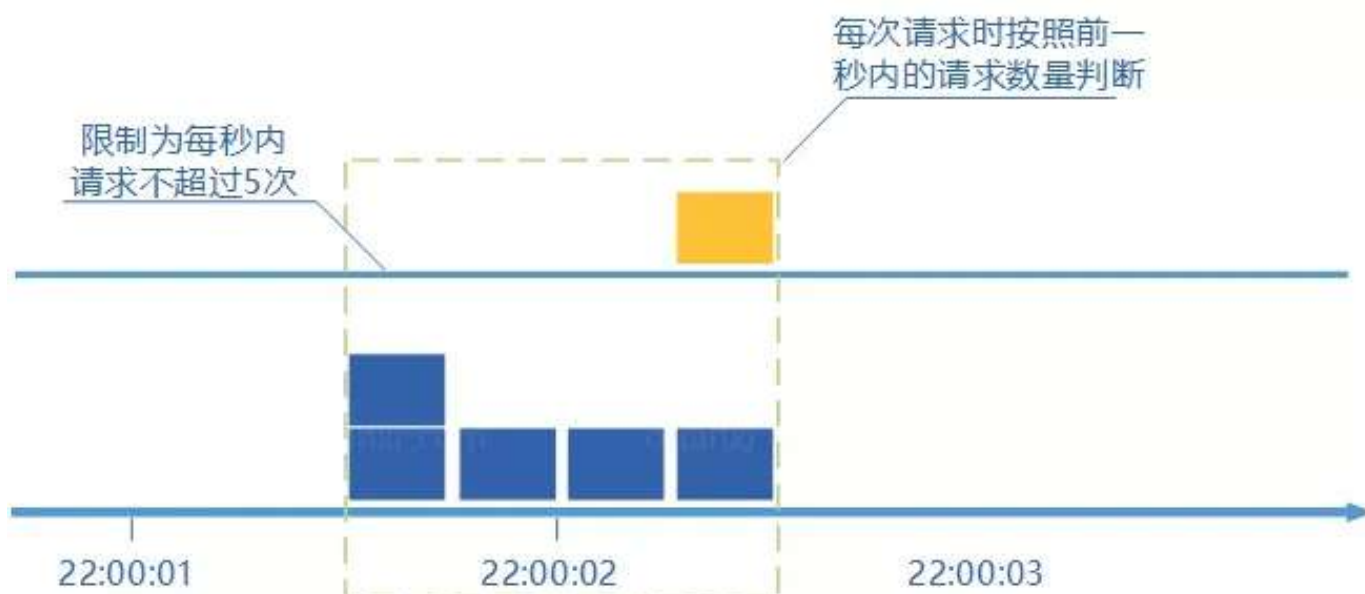
滑动窗口计数器算法

滑动窗口计数器算法 算的上是固定窗口计数器算法的升级版。

滑动窗口计数器算法相比于固定窗口计数器算法的优化在于：它把时间以一定比例分片。

例如我们的接口限流每分钟处理 60 个请求，我们可以把 1 分钟分为 60 个窗口。每隔 1 秒移动一次，每个窗口一秒只能处理 不大于 $60(\text{请求数})/60(\text{窗口数})$ 的请求，如果当前窗口的请求计数总和超过了限制的数量的话就不再处理其他请求。

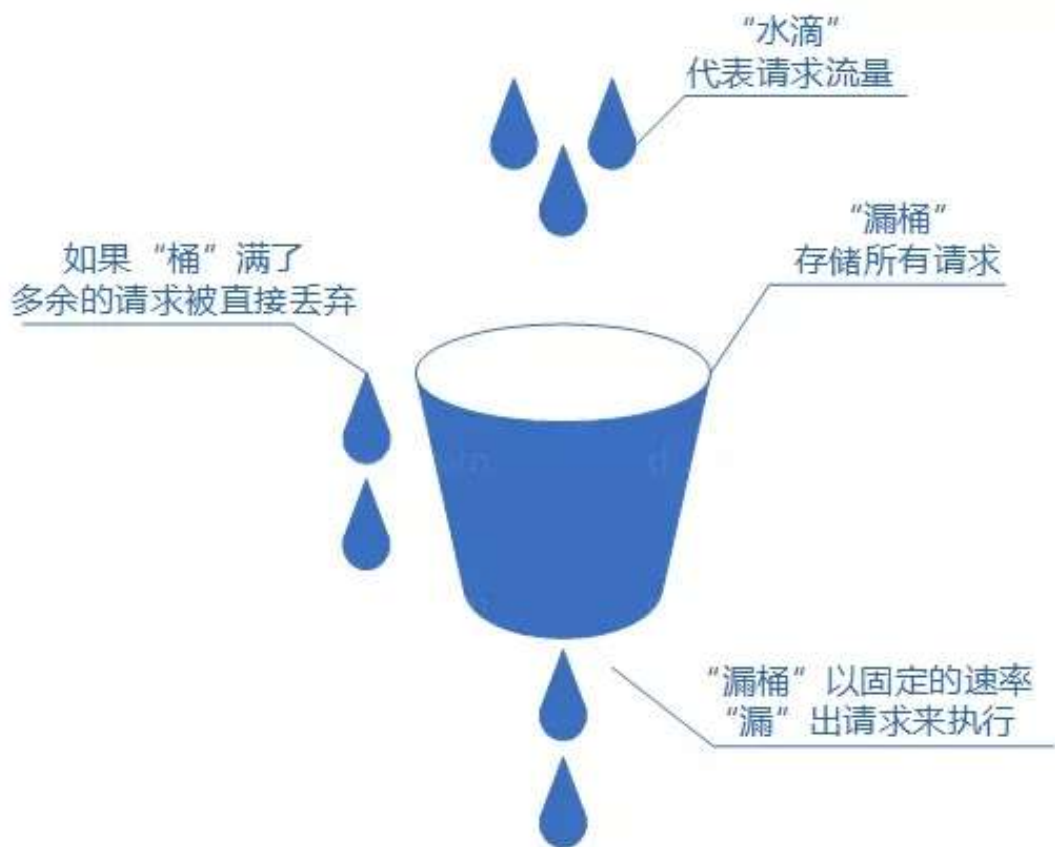
很显然，当滑动窗口的格子划分的越多，滑动窗口的滚动就越平滑，限流的统计就会越精确。



漏桶算法

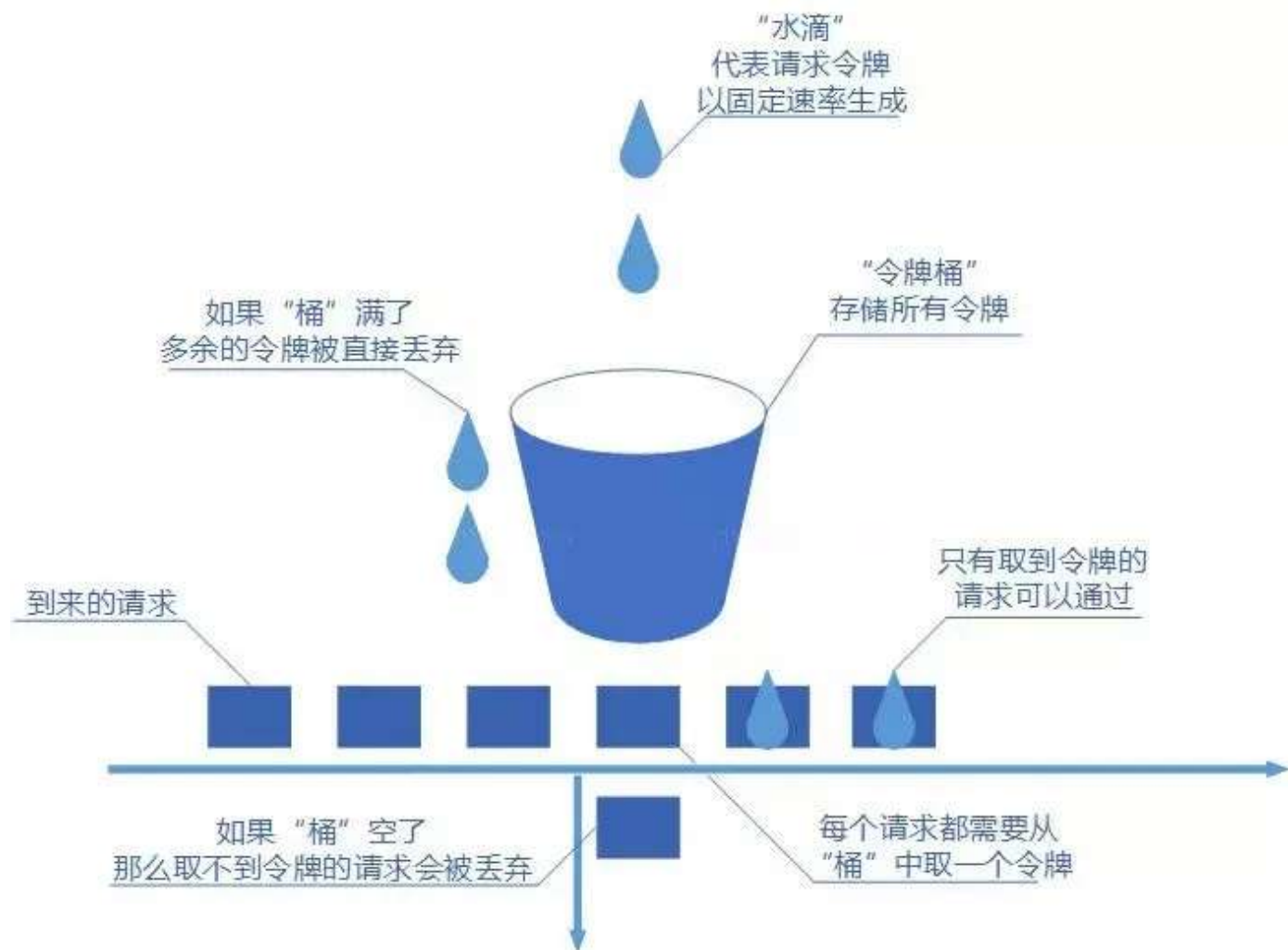
我们可以把发请求的动作比作成注水到桶中，我们处理请求的过程可以比喻为漏桶漏水。我们往桶中以任意速率流入水，以一定速率流出水。当水超过桶容量则丢弃，因为桶容量是不变的，保证了整体的速率。

如果想要实现这个算法的话也很简单，准备一个队列用来保存请求，然后我们定期从队列中拿请求来执行就好了（和消息队列削峰/限流的思想是一样的）。



令牌桶算法

令牌桶算法也比较简单。和漏桶算法算法一样，我们的主角还是桶（这限流算法和桶过不去啊）。不过现在桶里装的是令牌了，请求在被处理之前需要拿到一个令牌，请求处理完毕之后将这个令牌丢弃（删除）。我们根据限流大小，按照一定的速率往桶里添加令牌。如果桶装满了，就不能继续往里面继续添加令牌了。



单机限流

单机限流可以直接使用 Google Guava 自带的限流工具类 `RateLimiter` 。 `RateLimiter` 基于令牌桶算法，可以应对突发流量。

Guava 地址： <https://github.com/google/guava>

除了最基本的令牌桶算法(平滑突发限流)实现之外，Guava 的 `RateLimiter` 还提供了 **平滑预热限流** 的算法实现。

平滑突发限流就是按照指定的速率放令牌到桶里，而平滑预热限流会有一段预热时间，预热时间之内，速率会逐渐提升到配置的速率。

我们下面通过两个简单的小例子来详细了解吧！

我们直接在项目中引入 Guava 相关的依赖即可使用。

```
<dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>31.0.1-jre</version>
</dependency>
```

下面是一个简单的 Guava 平滑突发限流的 Demo。

```
import com.google.common.util.concurrent.RateLimiter;

/**
 * 微信搜 JavaGuide 回复"面试突击"即可免费领取个人原创的 Java 面试手册
 *
 * @author Guide哥
 * @date 2021/10/08 19:12
 */
public class RateLimiterDemo {

    public static void main(String[] args) {
        // 1s 放 5 个令牌到桶里也就是 0.2s 放 1个令牌到桶里
        RateLimiter rateLimiter = RateLimiter.create(5);
        for (int i = 0; i < 10; i++) {
            double sleepingTime = rateLimiter.acquire(1);
            System.out.printf("get 1 tokens: %ss%n", sleepingTime);
        }
    }
}
```

输出：

```
get 1 tokens: 0.0s
get 1 tokens: 0.188413s
get 1 tokens: 0.197811s
get 1 tokens: 0.198316s
get 1 tokens: 0.19864s
get 1 tokens: 0.199363s
get 1 tokens: 0.193997s
get 1 tokens: 0.199623s
get 1 tokens: 0.199357s
get 1 tokens: 0.195676s
```

下面是一个简单的 Guava 平滑预热限流的 Demo。

```
import com.google.common.util.concurrent.RateLimiter;
import java.util.concurrent.TimeUnit;

/**
 * 微信搜 JavaGuide 回复"面试突击"即可免费领取个人原创的 Java 面试手册
 *
 * @author Guide哥
 * @date 2021/10/08 19:12
 */
public class RateLimiterDemo {

    public static void main(String[] args) {
        // 1s 放 5 个令牌到桶里也就是 0.2s 放 1个令牌到桶里
        // 预热时间为3s,也就说刚开始的 3s 内发牌速率会逐渐提升到 0.2s 放 1 个令牌到桶里
        RateLimiter rateLimiter = RateLimiter.create(5, 3, TimeUnit.SECONDS);
        for (int i = 0; i < 20; i++) {
            double sleepingTime = rateLimiter.acquire(1);
            System.out.printf("get 1 tokens: %sds%n", sleepingTime);
        }
    }
}
```

输出：

```
get 1 tokens: 0.0s
get 1 tokens: 0.561919s
get 1 tokens: 0.516931s
get 1 tokens: 0.463798s
get 1 tokens: 0.41286s
get 1 tokens: 0.356172s
get 1 tokens: 0.300489s
get 1 tokens: 0.252545s
get 1 tokens: 0.203996s
get 1 tokens: 0.198359s
```

另外，**Bucket4j** 是一个非常不错的基于令牌/漏桶算法的限流库。

Bucket4j 地址: <https://github.com/vladimir-bukhtoyarov/bucket4j>

相对于，Guava 的限流工具类来说，Bucket4j 提供的限流功能更加全面。不仅支持单机限流和分布式限流，还可以集成监控，搭配 Prometheus 和 Grafana 使用。

不过，毕竟 Guava 也只是一个功能全面的工具类库，其提供的开箱即用的限流功能在很多单机场景下还是比较实用的。

Spring Cloud Gateway 中自带的单机限流的早期版本就是基于 Bucket4j 实现的。后来，替换成了 **Resilience4j**。

Resilience4j 是一个轻量级的容错组件，其灵感来自于 Hystrix。自 [Netflix 宣布不再积极开发 Hystrix](#) 之后，Spring 官方和 Netflix 都更推荐使用 Resilience4j 来做限流熔断。

Resilience4j 地址: <https://github.com/resilience4j/resilience4j>

一般情况下，为了保证系统的高可用，项目的限流和熔断都是要一起做的。

Resilience4j 不仅提供限流，还提供了熔断、负载保护、自动重试等保障系统高可用开箱即用的功能。并且，Resilience4j 的生态也更好，很多网关都使用 Resilience4j 来做限流熔断的。

因此，在绝大部分场景下 Resilience4j 或许会是更好的选择。如果是一些比较简单的限流场景的话，Guava 或者 Bucket4j 也是不错的选择。

分布式限流

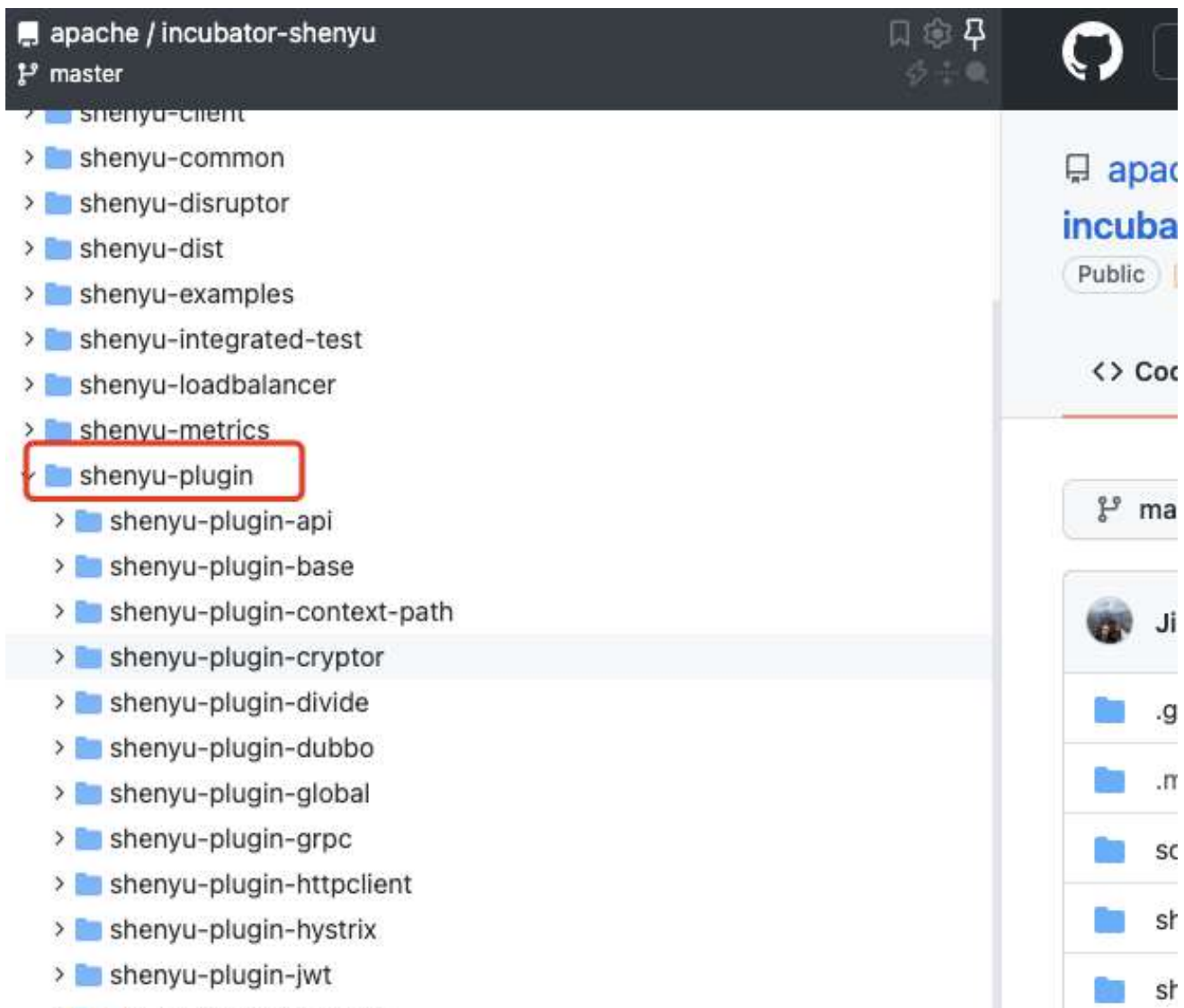
分布式限流常见的方案：

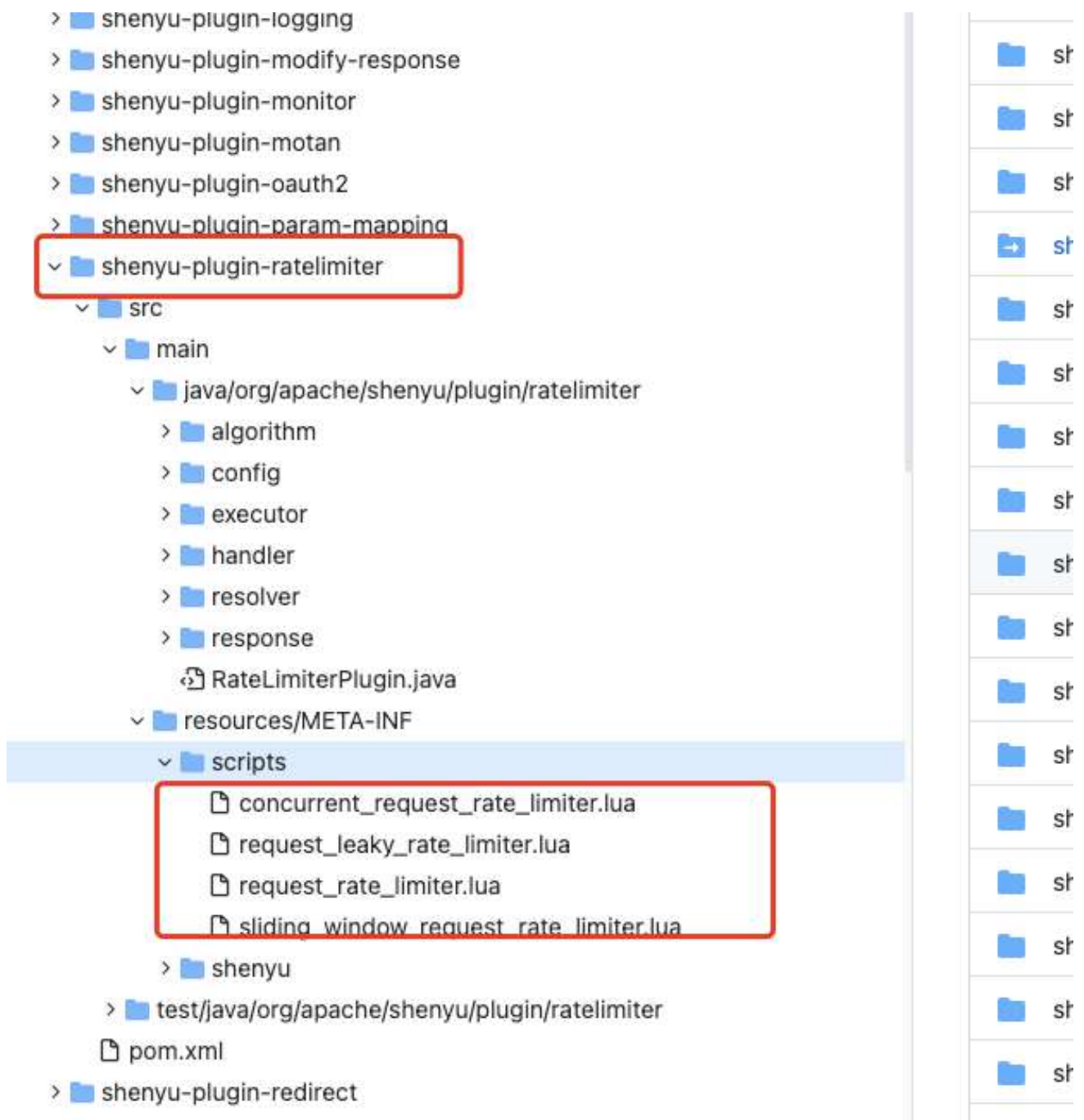
- **借助中间件架限流**：可以借助 Sentinel 或者使用 Redis 来自己实现对应的限流逻辑。
- **网关层限流**：比较常用的一种方案，直接在网关层把限流给安排上了。不过，通常网关层限流通常也需要借助到中间件/框架。就比如 Spring Cloud Gateway 的分布式限流实现 `RedisRateLimiter` 就是基于 Redis+Lua 来实现的，再比如 Spring Cloud Gateway 还可以整合 Sentinel 来做限流。

如果你要基于 Redis 来手动实现限流逻辑的话，建议配合 Lua 脚本来做。

网上也有很多现成的脚本供你参考，就比如 Apache 网关项目 ShenYu 的 RateLimiter 限流插件就基于 Redis + Lua 实现了令牌桶算法/并发令牌桶算法、漏桶算法、滑动窗口算法。

ShenYu 地址: <https://github.com/apache/incubator-shenyu>





相关阅读

- 服务治理之轻量级熔断框架 Resilience4j : <https://xie.infoq.cn/article/14786e571c1a4143ad1ef8f19>
- 超详细的 Guava RateLimiter 限流原理解析: <https://cloud.tencent.com/developer/article/1408819>
- 实战 Spring Cloud Gateway 之限流篇👍: <https://www.aneasystone.com/archives/2020/08/spring-cloud-gateway-current-limiting.html>-----

9.4 超时和重试

由于网络问题、系统或者服务内部的 Bug、服务器宕机、操作系统崩溃等问题的不确定性，我们的系统或者服务永远不可能保证时刻都是可用的状态。

为了最大限度的减小系统或者服务出现故障之后带来的影响，我们需要用到的 **超时 (Timeout)** 和 **重试 (Retry)** 机制。

想要把超时和重试机制讲清楚其实很简单，因为它俩本身就不是什么高深的概念。

虽然超时和重试机制的思想很简单，但是它俩是真的非常实用。你平时接触到的绝大部分涉及到远程调用的系统或者服务都会应用超时和重试机制。尤其是对于微服务系统来说，正确设置超时和重试非常重要。单体服务通常只涉及数据库、缓存、第三方 API、中间件等的网络调用，而微服务系统内部各个服务之间还存在着网络调用。

超时机制

什么是超时机制？

超时机制说的是当一个请求超过指定的时间（比如 1s）还没有被处理的话，这个请求就会直接被取消并抛出指定的异常或者错误（比如 504 Gateway Timeout）。

我们平时接触到的超时可以简单分为下面 2 种：

- **连接超时 (ConnectTimeout)**：客户端与服务端建立连接的最长等待时间。
- **读取超时 (ReadTimeout)**：客户端和服务端已经建立连接，客户端等待服务端处理完请求的最长时间。实际项目中，我们关注比较多的还是读取超时。

一些连接池客户端框架中可能还会有获取连接超时和空闲连接清理超时。

如果没有设置超时的话，就可能会导致服务端连接数爆炸和大量请求堆积的问题。

这些堆积的连接和请求会消耗系统资源，影响新收到的请求的处理。严重的情况下，甚至会拖垮整个系统或者服务。

我之前在实际项目就遇到过类似的问题，整个网站无法正常处理请求，服务器负载直接快被拉满。后面发现原因是项目超时设置错误加上客户端请求处理异常，导致服务端连接数直接接近 40w+，这么多堆积的连接直接把系统干趴了。

超时时间应该如何设置？

超时到底设置多长时间是一个难题！超时值设置太高或者太低都有风险。如果设置太高的话，会降低超时机制的有效性，比如你设置超时为 10s 的话，那设置超时就没啥意义了，系统依然可能会出现大量慢请求堆积的问题。如果设置太低的话，就可能会导致在系统或者服务在某些处理请求速度变慢的情况下（比如请求突然增多），大量请求重试（超时通常会结合重试）继续加重系统或者服务的压力，进而导致整个系统或者服务被拖垮的问题。

通常情况下，我们建议读取超时设置为 **1500ms**，这是一个比较普适的值。如果你的系统或者服务对于延迟比较敏感的话，那读取超时值可以适当在 **1500ms** 的基础上进行缩短。反之，读取超时值也可以在 **1500ms** 的基础上进行加长，不过，尽量还是不要超过 **1500ms**。连接超时可以适当设置长一些，建议在 **1000ms ~ 5000ms** 之内。

没有银弹！超时值具体该设置多大，还是要根据实际项目的需求和情况慢慢调整优化得到。

更上一层，参考[美团的Java线程池参数动态配置](#)思想，我们也可以将超时弄成可配置化的参数而不是固定的，比较简单的一种办法就是将超时的值放在配置中心中。这样的话，我们就可以根据系统或者服务的状态动态调整超时值了。

重试机制

什么是重试机制？

重试机制一般配合超时机制一起使用，指的是多次发送相同的请求来避免瞬态故障和偶然性故障。

瞬态故障可以简单理解为某一瞬间系统偶然出现的故障，并不会持久。偶然性故障可以理解为哪些在某些情况下偶尔出现的故障，频率通常较低。

重试的核心思想是通过消耗服务器的资源来尽可能获得请求更大概率被成功处理。由于瞬态故障和偶然性故障是很少发生的，因此，重试对于服务器的资源消耗几乎是可以被忽略的。

重试的次数如何设置？

重试的次数不宜过多，否则依然会对系统负载造成比较大的压力。

重试的次数通常建议设为 3 次。并且，我们通常还会设置重试的间隔，比如说我们要重试 3 次的话，第 1 次请求失败后，等待 1 秒再进行重试，第 2 次请求失败后，等待 2 秒再进行重试，第 3 次请求失败后，等待 3 秒再进行重试。

重试幂等

超时和重试机制在实际项目中使用的话，需要注意保证同一个请求没有被多次执行。

什么情况下会出现一个请求被多次执行呢？客户端等待服务端完成请求完成超时但此时服务端已经执行了请求，只是由于短暂的网络波动导致响应在发送给客户端的过程中延迟了。

举个例子：用户支付购买某个课程，结果用户支付的请求由于重试的问题导致用户购买同一门课程支付了两次。对于这种情况，我们在执行用户购买课程的请求的时候需要判断一下用户是否已经购买过。这样的话，就不会因为重试的问题导致重复购买了。

参考

- 微服务之间调用超时的设置治理：<https://www.infoq.cn/article/eyrslar53l6hjm5yjgyx>
- 超时、重试和抖动回退：<https://aws.amazon.com/cn/builders-library/timeouts-retries-and-back-off-with-jitter/>